# Book I — System and Hardware Reference

BMSX

# Table of contents

**About these manuals** 1

**1 System overview** 3
1.1 System structure . . . 3
1.2 Processor and system control . . . 4
1.3 Storage and the address map . . . 4
1.4 From cartridge data to a displayed picture . . . 5
1.5 APU sound and sample-memory route . . . 7
1.6 Mapped access and DMA transfers . . . 8
1.7 Timing and input capture . . . 9
1.8 Interrupts, faults and reset . . . 9

**2 Memory and program storage** 11
2.1 Addresses and data sizes . . . 11
2.2 Installed memory and address windows . . . 11
2.3 I/O register-bank directory . . . 12
2.4 Reserved and cartridge storage in main RAM . . . 14
2.5 Firmware reservation in VRAM . . . 14
2.6 ROM contents, RAM sections, and startup . . . 15
2.7 Reset and cartridge startup . . . 16
2.8 Typed byte storage and address views . . . 22
2.9 Language execution storage and byte RAM . . . 22
2.10 Execution addresses and instruction reads . . . 24
2.11 Scalar memory accesses . . . 25
2.12 Bus-fault status and CPU exceptions . . . 26
2.13 Installed response to a memory exception . . . 28
2.14 Worked example: prepare and copy two records . . . 31
2.15 DMA buffer lifetime . . . 32
2.16 Device-specific buffer lifetimes . . . 34
2.17 Shared GPU storage and display publication . . . 36

**3 PCRTC Display Controller** 41
3.1 Purpose and data flow . . . 41
3.2 Selecting and displaying an image . . . 41
3.3 Timing and interrupt programming examples . . . 53
3.4 REGISTER REFERENCE . . . 55
3.5 SOURCE ORIGIN AND DISPLAY-WINDOW POSITION . . . 63
3.6 HORIZONTAL SOURCE SAMPLING . . . 67
3.7 Composition: constant coefficient and source alpha . . . 68

**4 Geometry Engine (GTE)** 73
4.1 Function and data flow . . . 73
4.2 From a vertex to a screen position . . . 73
4.3 Programming geometry . . . 74
4.4 A first projection in BLua . . . 75
4.5 Technical specifications . . . 75
4.6 Coordinate spaces and the supplied transform . . . 77
4.7 Constructing and packing a transform . . . 81
4.8 Composing transforms . . . 84
4.9 Perspective and geometric decisions . . . 88
4.10 Normals, light directions and color mixing . . . 90
4.11 Choosing a color calculation . . . 93
4.12 Interpolation and general arithmetic . . . 95
4.13 Choosing depth-cue coefficients . . . 98

4.14 Worked geometry sequences . . . 100
4.15 From projected vertices to a visible image . . . 103
4.16 Hardware frame sequence . . . 105
4.17 A further geometry sequence . . . 108
4.18 Diagnosing numerical results . . . 109
4.19 Command reference . . . 111
4.20 Budgeting a complete geometry work interval . . . 156
4.21 Timing and bus programming . . . 158
4.22 Register reference . . . 160
4.23 Normal-command arithmetic . . . 168
4.24 Projection stage RT(v, last) . . . 170
4.25 GTE_PLUS multiply-add . . . 171
4.26 PLUS programming example . . . 171
4.27 Repeated PLUS work . . . 172
4.28 Depth-cued color: four equal swatches . . . 174
4.29 Near-plane clipping: actual output . . . 178
4.30 Appendix: reciprocal seed ROM . . . 180

**5 GPU Drawing Engine 181**
5.1 Function and data flow . . . 181
5.2 Technical specifications . . . 181
5.3 Program a complete picture in BLua . . . 182
5.4 GP0, GPUREAD, GP1 and GPUSTAT . . . 188
5.5 GP1 control commands . . . 191
5.6 Query drawing state through GPUREAD . . . 194
5.7 Render successive frames . . . 195
5.8 Hand a GP0 stream between CPU and DMA . . . 196
5.9 Organize local memory for drawing and display . . . 197
5.10 Choose a primitive for the required image . . . 197
5.11 Place primitives across coordinate boundaries . . . 198
5.12 Predict shared-edge and joint writes . . . 199
5.13 Predict interpolated colors and texture coordinates . . . 202
5.14 Predict line samples and color ramps . . . 204
5.15 Locate texture texels and palette colors . . . 205
5.16 Keep a clipped or flipped rectangle on its texture . . . 206
5.17 Choose texture gain and dither deliberately . . . 207
5.18 Separate texture transparency, blending and destination protection . . . 210
5.19 Choose fill, drawing or a raw transfer . . . 211
5.20 Frame the GP0 command stream . . . 212
5.21 Flat untextured polygons — GP0(20H–23H,28H–2BH) . . . 214
5.22 Gouraud untextured polygons — GP0(30H–33H,38H–3BH) . . . 216
5.23 Flat textured polygons — GP0(24H–27H,2CH–2FH) . . . 217
5.24 Gouraud textured polygons — GP0(34H–37H,3CH–3FH) . . . 221
5.25 Finite flat and Gouraud lines — GP0(40H–47H,50H–57H) . . . 224
5.26 Flat and Gouraud polylines — GP0(48H–4FH,58H–5FH) . . . 225
5.27 Drawing rectangles — GP0(60H–7FH) . . . 227
5.28 Fill raw local memory — GP0(02H) . . . 229
5.29 Raw image upload — GP0(A0H–BFH) . . . 231
5.30 Read raw local memory — GP0(C0H–DFH) . . . 232
5.31 Raw local-memory copy — GP0(80H–9FH) . . . 235
5.32 Drawing mode — GP0(E1H) . . . 236
5.33 Texture window, drawing bounds, offset and masks — GP0(E2H–E6H) . . . 238
5.34 Budget command work separately from issuing words . . . 240
5.35 Count a complete issue-and-fence interval . . . 242
5.36 Move a raw image through DMA . . . 244

**6 Interrupt controller 247**
6.1 IRQ specifications . . . 247
6.2 CPU delivery and waiting . . . 248
6.3 Timing and service cost . . . 250
6.4 Acknowledge the source and the controller separately . . . 251
6.5 Program and service a GPU marker . . . 252

6.6 Register reference . . . 253
6.7 Register access and faults . . . 255

**7 Direct memory access** **257**
7.1 DMA specifications . . . 257
7.2 Follow a buffer through a transfer . . . 258
7.3 Program a transfer . . . 259
7.4 Copy words in BLua . . . 260
7.5 Register reference . . . 261
7.6 Programmed state and an admitted block . . . 263
7.7 Requests and device-port ownership . . . 264
7.8 Faults and recovery . . . 266
7.9 Completion and interrupts . . . 266
7.10 Transfer timing and sharing the bus . . . 266

**8 Audio processing unit** **271**
8.1 APU specifications . . . 271
8.2 Programming sequence . . . 272
8.3 Commands and slot ownership . . . 274
8.4 Register reference . . . 275
8.5 Voice sources and their lifetime . . . 277
8.6 PCM byte layouts and interpolation . . . 278
8.7 BADP compressed source format . . . 279
8.8 Source phase, rate and duration . . . 282
8.9 Square generator . . . 283
8.10 Gain, stereo mixing and clipping . . . 283
8.11 Per-voice Q14 filter . . . 284
8.12 Stop and fade . . . 284
8.13 Pause, resume and live programming . . . 285
8.14 Clocks and transfer timing . . . 286
8.15 Sample RAM transfer . . . 287
8.16 End events and interrupts . . . 290
8.17 Faults and reset . . . 290
8.18 Worked download: requested data and surplus prefetch . . . 291

**9 Geometry batch unit** **293**
9.1 GEO specifications . . . 293
9.2 An authored sprite and its collision hull . . . 294
9.3 Launching and observing a job . . . 298
9.4 Register reference . . . 301
9.5 Work budget and timing . . . 303
9.6 Completion, faults, and recovery . . . 303
9.7 XFORM2_BATCH: transform vertices and build bounds . . . 304
9.8 SAT2_BATCH: test convex-polygon pairs . . . 306
9.9 OVERLAP2D_PASS: translated-shape contacts . . . 309

**10 Image decompression unit** **315**
10.1 Function and resources . . . 315
10.2 Starting a transfer . . . 316
10.3 Programming IMGDEC in BLua . . . 316
10.4 Register reference . . . 317
10.5 Data transfer and completion . . . 319
10.6 Work timing and transfer budget . . . 321
10.7 Compressed word stream . . . 323
10.8 Generated GP0 output . . . 324
10.9 Worked stream: 19 input words, 26 output words . . . 326
10.10 Worked stream: overlapping history across the CLUT header . . . 327
10.11 Completion, malformed input, and partial output . . . 328
10.12 Use the output packets . . . 330

**11 Input controller** **333**
11.1 Function and resources . . . 333

11.2 Sampling and snapshot reads . . . 334
11.3 Sample input in BLua . . . 334
11.4 Raw input representations . . . 335
11.5 Register map . . . 337
11.6 Control words and reset . . . 338
11.7 VBlank ordering, interrupts and access faults . . . 339
11.8 Pad output commands . . . 339

**12 System control and timing** **341**
12.1 SYSTEM specifications . . . 341
12.2 Measure elapsed console time . . . 341
12.3 Control register reference . . . 342
12.4 Privilege and supervisor entry . . . 343
12.5 Device boundaries during supervisor service . . . 343
12.6 Supervisor fault registers . . . 344
12.7 Faults, interrupts and continuation . . . 346
12.8 Reset requests . . . 346
12.9 System time and frame timing . . . 346
12.10 Diagnostic text output . . . 347
12.11 Send a diagnostic line . . . 348

# About these manuals

The BMSX Developer Reference describes the console, its programming architecture and the interfaces available to cartridge software. The three books separate installed hardware, processor and firmware programming, and cartridge devices.

**Book I — System and Hardware Reference** introduces the machine and its memory system, drawing and display units, audio, direct memory access, interrupts and peripheral control. Start with the system overview, then consult the individual unit chapters for programming sequences and reference details.

**Book II — BLua32 and BIOS Reference** describes the processor, instruction set, BLua language extensions and resident BIOS services. It supplies the program and data representations needed to implement cartridge software.

**Book III — Cartridge and Expansion Reference** describes cartridge sockets and mapped card devices, including their memory and interrupt interfaces.

Addresses are byte addresses unless stated otherwise. Register descriptions give bit positions within the named word; binary layouts give byte offsets within the described structure. Capacities, units and synchronization conditions are specified with the interface to which they apply.

# 1 System overview

The BMSX is a cartridge-based game console. Its BLua32 processor runs the game and coordinates the units that calculate geometry, draw pictures, read the display and play sound. Main RAM holds writable game data and transfer buffers; the graphics and audio units have their own memories for pixels and samples.

These resources give a program several ways to divide its work. It can calculate screen positions itself or use GTE, construct drawing commands in RAM and send them by DMA, and leave the APU playing a sound while it prepares the next picture. The units have separate controls and completion indications. The program decides both what each unit will do and when its storage can be reused.

This chapter establishes the relationships between those operations. The following hardware chapters give their programming procedures and complete register, command, format and timing references. Book II describes BLua32 and the BIOS; Book III describes the cartridge sockets and mapped card devices.

## 1.1 System structure

The CPU connects the game's calculations to the drawing and sound paths. System ROM supplies the BIOS, and cartridge ROM supplies the game's program and read-only data. Main RAM holds writable records and buffers. The GPU writes pixels into its local memory, and PCRTC reads a selected picture from that memory for display. The APU reads its sound sources and produces stereo output.

Figure 1.1 locates these resources and their principal access paths. The detailed graphics and sound diagrams later in this chapter expand the two output paths. The lines identify functional access and control, rather than electrical bus wiring.

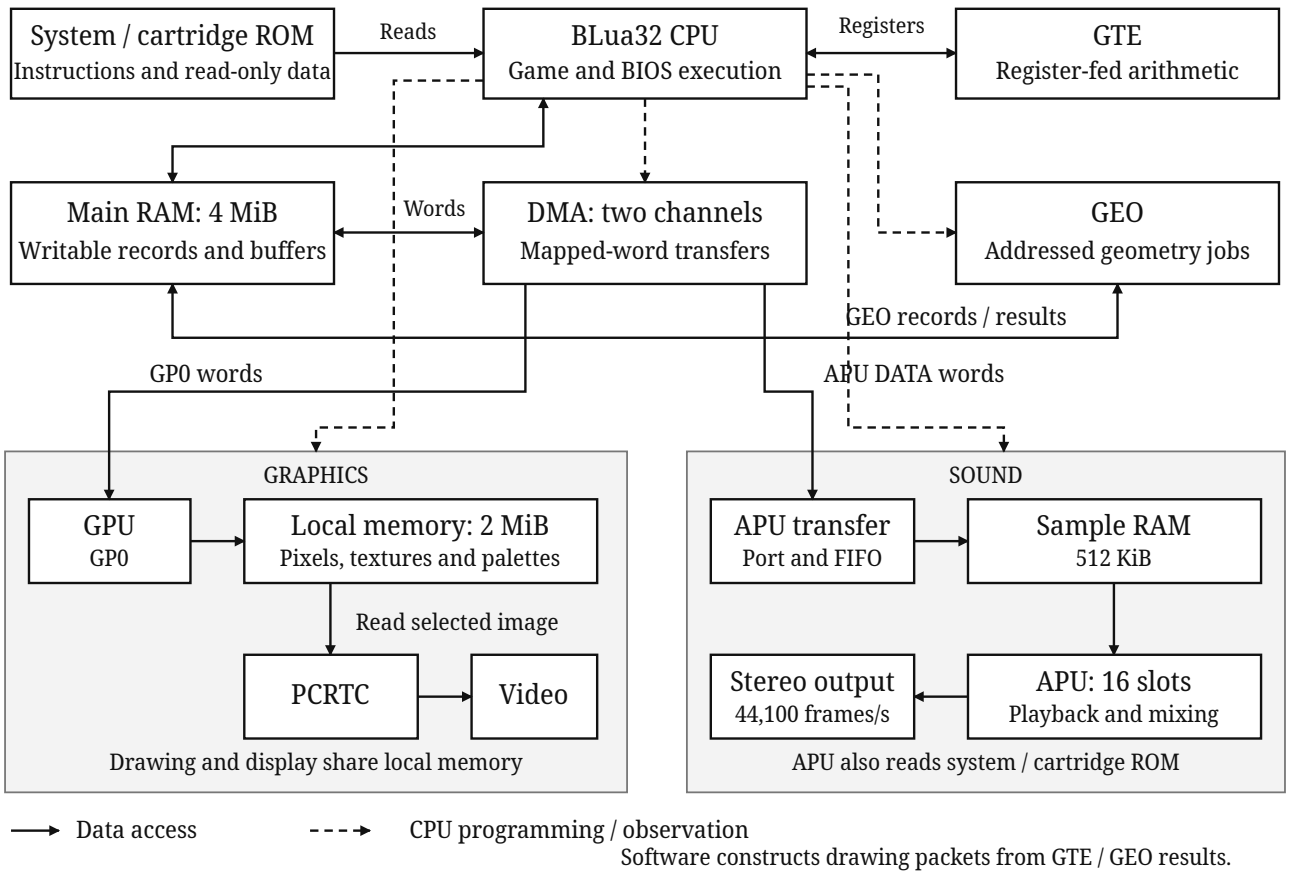



Figure 1.1: CPU control, main-memory access and the separate graphics and sound memories

GTE returns arithmetic results through registers; GEO reads and writes addressed records. Software uses the results when constructing drawing commands. Neither unit submits those commands to the GPU. Likewise, GPU local memory and APU sample RAM are accessed through their respective unit interfaces, not as additional main-RAM addresses.

| Installed resource | Specification |
|---|---|
| CPU/system timebase | 33,868,800 cycles per second |
| Main RAM | 4 MiB, including reserved areas and firmware static storage |
| GPU local memory | 2 MiB shared by drawing destinations, textures, palettes and display sources |
| APU sample RAM | 512 KiB, separate from main RAM and GPU local memory |
| System DMA | Two equivalent channels; one transfer block is serviced at a time |
| GTE | Register-fed transformation, projection and lighting arithmetic |
| GEO | Memory-addressed geometry and collision jobs |
| PCRTC | Two programmable reading circuits with output composition; 13,500,000 Hz beam-timing reference |
| APU | 16 playback slots; 44,100 stereo output frames per second |
| Input | Keyboard, pointer and four game-pad records |
| Cartridge interface | Two physical sockets sharing the cartridge address windows |

The CPU timebase expresses instruction and transfer costs. PCRTC's reference is used to calculate the programmed display timing, while the APU output rate defines the interval between stereo sample frames. These quantities describe different operations; the device chapters specify their timing relationships.

## 1.2 Processor and system control

At reset, the CPU enters the system-ROM startup program. The BIOS initializes its working storage and selects an executable cartridge. Cartridge ROM supplies the game's program and read-only data; main RAM supplies writable data and buffers. System-ROM routines remain available through the BIOS interfaces. The startup and storage requirements are described in Memory and program storage.

BLua32 uses four-byte instruction words; a WIDE prefix extends an instruction to eight bytes. Device registers are reached through memory-mapped ports. A word write can set a parameter, supply data or issue a command, depending on the port; a read can return status, retained configuration or a result. The device's programming sequence establishes which values must be prepared before its start command. The Book II, "CPU and BLua32 instruction reference" defines instruction operation and cycle costs.

Three supporting units connect program execution to the rest of the machine:

- **IRQ** combines device requests into the CPU's shared maskable interrupt line. Programs use it to receive events such as VBlank and DMA completion.
- **SYSTEM** provides console-time and frame-period readouts, reset requests and supervisor controls. Its diagnostic words retain CPU fault information for BIOS service.
- **INPUT** captures keyboard, pointer and pad states at a requested display boundary. The program interprets those states as game actions.

Their operation is introduced below after the picture, sound and transfer paths.

## 1.3 Storage and the address map

The three writable memories serve different purposes. Main RAM supplies CPU variables, geometry records and DMA buffers. GPU local memory supplies pixels, textures and palettes. APU sample RAM supplies playback samples. Moving bytes into one memory does not make them available in another: an image upload sends words through the GPU, and a sample upload sends words through the APU transfer ports.

BMSX uses 32-bit byte addresses. A register or DMA word occupies four bytes. The address map below locates the CPU-visible regions. It uses half-open intervals: the start is included and the end is excluded. An address window is the range assigned to an interface; installed storage can occupy less than that range.

| Region | Byte-address interval | Capacity or use |
|---|---|---|
| System ROM | `[0x00000000, 0x01000000)` | 16 MiB address window for the installed firmware |
| Unmapped | `[0x01000000, 0x08000000)` | No mapped storage or device |
| Main RAM | `[0x08000000, 0x10000000)` | 128 MiB window; 4 MiB installed in `[0x08000000, 0x08400000)`, including the reservations below |
| Cartridge ROM | `[0x10000000, 0x30000000)` | 512 MiB window; capacity depends on the selected card |
| Cartridge RAM | `[0x30000000, 0x30f00000)` | 15 MiB window for optional card RAM |
| Cartridge devices | `[0x30f00000, 0x31000000)` | 1 MiB window for the selected card's mapped devices |
| Unmapped | At or above `0x31000000` | No mapped storage or device through the end of the address space |

The main-RAM window beyond the installed 4 MiB does not provide additional or mirrored RAM. Both cartridge sockets use the same windows; the socket selection chooses which card an access reaches. The capacities of the two cards are not combined into a larger address range. See Book III, "Cartridge sockets and mapped card devices" for CPU, instruction-fetch and DMA selection.

### 1.3.1 Reservations before cartridge storage

The beginning of installed main RAM is reserved as follows:

| Reservation | Byte-address interval or extent | Purpose |
|---|---|---|
| I/O | `[0x08000000, 0x08040000)` | 256 KiB reservation containing the device-register aperture |
| GEO scratch | `[0x08040000, 0x080c0000)` | 512 KiB reserved area |
| Firmware static storage | Starting at `0x080c0000`, through the actual end of system BSS | Initialized DATA and zero-initialized BSS used by the BIOS |

The I/O and GEO reservations occupy 768 KiB. This leaves 3.25 MiB before subtracting firmware and cartridge static storage; it is not a free-memory figure. Reserve both complete areas even where no register is decoded. Place cartridge data and buffers outside the firmware's complete DATA/BSS extent. The Memory chapter specifies the register aperture, section layout and startup initialization.

Firmware also retains part of GPU local memory for its terminal and system texture. Preserve the complete rectangle returned by BIOS `gpu/system_vram_region`, including after cartridge entry. The query and its coordinate units are given under Firmware reservation in VRAM.

## 1.4 From cartridge data to a displayed picture

Consider a game character made from textured polygons. Its stored vertices specify its shape; they are not yet the screen positions required by a drawing command. The program transforms and projects them, prepares the texture in GPU local memory, and supplies the resulting drawing packets. After the GPU has drawn the picture, the program selects that picture for display.

These are distinct operations on different representations:

| Operation | Information supplied | Result used by the next operation |
|---|---|---|
| Calculate geometry and color | Vertices, transforms and lighting parameters | Screen coordinates and colors for software to place in drawing packets |
| Prepare image storage | Texture and palette words, optionally compressed for IMGDEC | Texture and palette data in GPU local memory |

| Operation | Information supplied | Result used by the next operation |
|---|---|---|
| Draw | GP0 packets containing screen positions, colors, texture coordinates and drawing settings | Pixels in a selected local-memory drawing region |
| Display | PCRTC source, display-window and composition settings | Video output reading the selected local-memory regions |

For sprites or other screen-space graphics, the program can supply positions directly. The geometry calculation is useful when the scene requires it, not a mandatory first stage for every drawing command.

### 1.4.1 GTE and GEO: two different arithmetic routes

A three-dimensional vertex must be expressed relative to the viewer before it can be projected onto the screen. GTE performs the matrix/vector, translation and perspective calculations for this route, as well as lighting and color arithmetic. Software writes operands and controls into its registers, issues the appropriate command, then reads the results for its drawing packets.

GEO instead operates on memory records. A program supplies record addresses, counts, strides and parameters for a batch. XFORM2 transforms two-dimensional vertices and can produce bounds; SAT2 tests convex-polygon pairs; OVERLAP2D produces translated-shape contacts. Results are written to addressed RAM for the program to use. For example, a character's transformed collision hull can be tested separately from the polygons or sprite used to draw it.

The choice is therefore between register-fed arithmetic and addressed record processing, according to the task. The GTE and GEO chapters explain the coordinate systems, numeric formats, commands and result handling. GEO's work budget is expressed in outer-record service units, not a fixed number of vertices or collision pairs per second.

### 1.4.2 Image decoding

Compressed image data needs a different path from vertex arithmetic. IMGDEC expands its compressed word stream into GPU upload headers and texture or palette words. One DMA channel can feed the compressed input while the other carries the generated stream from IMGDEC DATA to GPU GP0. The GPU then performs the local-memory writes described by those upload packets.

This route changes the representation of image data, not the drawing geometry. The program supplies the source stream, destination fields and word counts; request signals pace the DMA transfers as the decoder consumes input and produces output. The IMGDEC chapter specifies the compressed tokens, packet output, FIFO capacities and decoding costs.

### 1.4.3 Graphics storage and drawing

The GPU draws triangles, quads, lines, polylines and rectangles. Polygons may use flat or Gouraud color and optional texturing; rectangles can also be textured. Texture sources are 4-bit indexed, 8-bit indexed or direct16. Indexed textures obtain their colors from palettes in the same local memory.

GP0 receives drawing and image-transfer packets as a stream of 32-bit words, either from CPU stores or DMA. The GPU executes the packets in order and updates local memory. A texture upload followed by a textured drawing command therefore puts the image data before the draw that uses it.

The 2 MiB local memory contains 1,048,576 sixteen-bit words. Drawing addresses use periods of 1024 words horizontally and 1024 rows vertically; these are storage coordinates, not a screen-resolution specification. Drawing regions, texture regions and display regions all occupy this same memory. The program must place them so that drawing or uploading one does not overwrite data still needed by another.

The GPU chapter gives the packet formats, drawing environment, storage mappings and execution costs. Its capabilities describe supported operations; throughput depends on the actual command sequence.

### 1.4.4 Programmable PCRTC output

Drawing produces stored pixels. PCRTC determines which of those pixels become the displayed picture. Its two reading circuits each select a local-memory source and a display window; output controls combine their contributions with a background. Two circuits can read different regions of the same 2 MiB, rather than requiring a separate memory bank for each circuit.

A source selection establishes the framebuffer base, width, storage interpretation and source origin. A display window establishes placement and magnification in signal space. Moving the source origin selects another part of the stored picture; moving the display window places that selection at another output position. The PCRTC chapter illustrates this distinction and specifies the supported storage interpretations.

Display configuration is latched at VBlank entry. The program can prepare a new source selection, then observe the edge at which it becomes the displayed selection. Beam timing has separate immediate write effects. The programmed SMODE, SYNCH and SYNCV words determine beam timing; the enabled display windows and signal step determine output extent. BMSX therefore has programmable display dimensions and refresh rather than a single fixed screen mode.

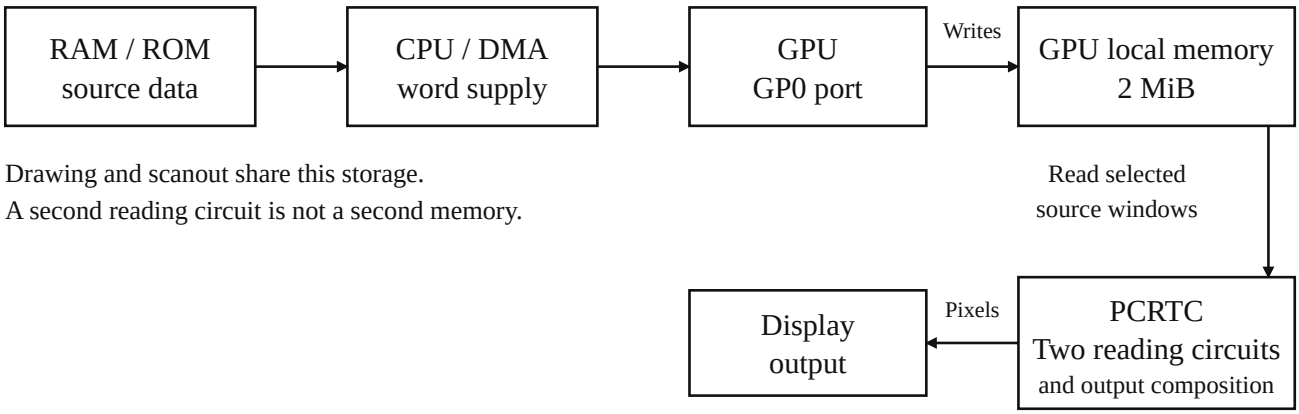



Figure 1.2: GPU command supply, shared local memory and two-circuit PCRTC reading

## 1.5 APU sound and sample-memory route

To play a stored sound, a program describes its sample data, selects a playback slot and issues PLAY. The APU advances through that source using the programmed rate and loop settings, applies gain and the optional stereo biquad filter, and combines the active voices into stereo output. A square generator provides an alternative source that needs no sample bytes.

The 16 slots support PCM8, PCM16, 4-bit BADP and square generation. The mixer produces 44,100 stereo frames per second, each containing signed sixteen-bit left and right samples after saturation. Source rate and playback step control how a voice moves through its samples; they do not change this output cadence.

A sample source can reside in system ROM, cartridge ROM or the private 512 KiB sample RAM. Main-RAM samples first pass through the APU transfer ports into sample RAM. This distinction matters for generated or modified sounds: writing a main-RAM buffer prepares the source of an upload, not a buffer that PLAY can read directly.

The upload path is main RAM, DMA, APU DATA and its transfer FIFO, then sample RAM. Downloads use the reverse sample-memory transfer direction. For an upload, DMA completion establishes that the words have reached the FIFO; the APU must finish its sample-RAM transfer before playback uses them. Sample-memory service has a budget of 1,058,400 thirty-two-bit words per second. The source bus and request waits also contribute to upload time.

During playback the APU continues to read the selected source. Preserve its bytes for the voice’s use. Cartridge-ROM playback retains the socket selected when PLAY executes. The APU chapter gives the source formats, slot controls, transfer modes and completion procedures.

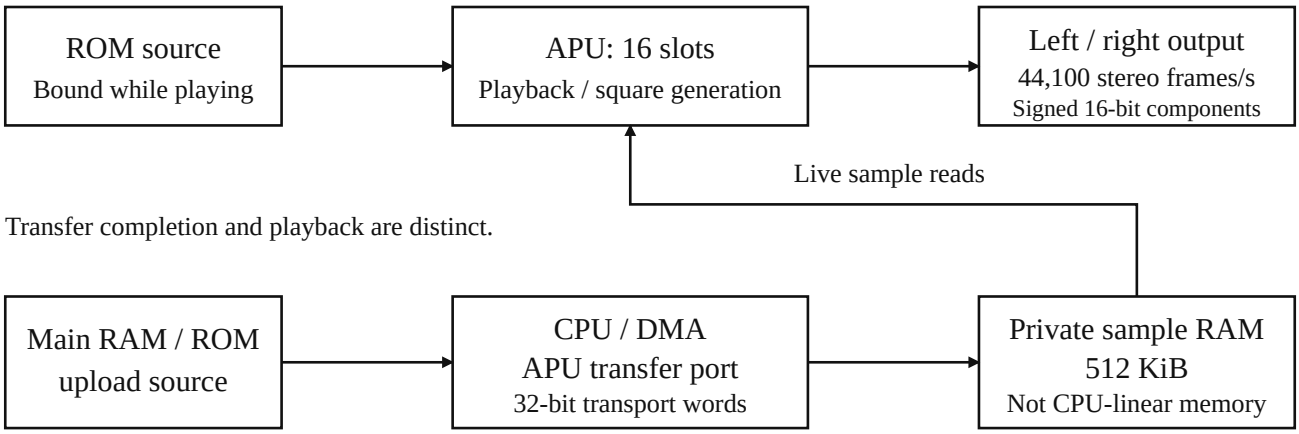



Figure 1.3: ROM and private sample-RAM playback sources, with a separate CPU or DMA upload path

## 1.6 Mapped access and DMA transfers

DMA supplies words to an operation; the receiving unit interprets those words. For example, a GPU upload consists of a GP0 header followed by image words. The DMA channel moves that sequence to the fixed GP0 address, while the GPU uses the header to determine where its local-memory writes belong. DMA does not need a CPU-linear VRAM address for this transfer.

Each of the two channels has source and destination byte addresses, a word count, address increments, request selectors and a block length of 1–16 words. For a RAM-to-GP0 transfer, increment the RAM address and keep the GP0 address fixed. Both channels can have outstanding work, but the shared transfer path services one block at a time, selecting ready channels round-robin at block boundaries.

The endpoint regions determine the transfer charge. Main RAM, system ROM and the cartridge bus have different charges; cartridge RAM and mapped card devices use the cartridge-bus charge too. Device requests and competing-channel blocks can add waiting time. The DMA timing reference supplies the route calculations, including GPU and APU transfers. CPU instruction costs are specified separately in the CPU reference.

### 1.6.1 Completion and storage reuse

Suppose DMA reads a command buffer while the CPU begins to construct the next picture in that same buffer. Words not yet transferred could then belong to the new picture while earlier words belong to the old one. Keep the source stable until DMA completes, or prepare the next stream in different storage.

Once those command words have been delivered, the buffer can be reused, but the GPU can still be drawing. If the next operation overwrites the drawing region, GPU completion matters. If that region is the current display source, the program must also arrange a new display selection before reusing it. Each hand-off answers a different question:

| Question | Relevant operation or boundary |
|---|---|
| Has the command source been consumed? | Completion of the DMA transfer reading it |
| Have the drawing commands finished? | GPU completion, observed through the appropriate GPU status |
| Has a new display source become active? | PCRTC display latch at VBlank entry |
| Have uploaded sound words reached sample RAM? | APU transfer completion after DMA delivery |
| Can sample bytes be overwritten? | Completion of every voice or transfer still using those bytes |

This division permits overlap: the CPU can prepare another command buffer while the GPU draws, and sound playback can continue during both. The requirement is to keep each producer’s writes separate from storage still being read. The Memory chapter gives buffer lifetimes, and its two-surface sequence follows preparation, display publication and reuse.

## 1.7 Timing and input capture

A game's update routine needs elapsed time and a definite set of controls. SYSTEM TIME_MS measures elapsed console cycles in milliseconds; its frame words report the programmed PCRTC period in fractional milliseconds and whole system cycles. These readouts measure intervals. VBlank interrupts provide the event used to synchronize with the display.

INPUT ARM requests one capture at the next VBlank-begin edge. The controller latches the raw key, button and axis states, increments the sample sequence and consumes the request. Those values remain available while the game processes them; another capture requires another ARM. The keyboard bitmap covers 256 HID usage bits. Pointer state and four pad records provide buttons and axes.

The INPUT chapter explains raw representations and sampling. SYSTEM explains the time readouts. Display timing determines the capture opportunity, not how long the CPU takes to process the resulting controls.

## 1.8 Interrupts, faults and reset

The IRQ controller records pending requests from the devices and enables selected sources onto the CPU's maskable interrupt line. CPU interrupt enable also controls acceptance. For example, a program can receive DMA completion without continuously polling the channel. The handler identifies and services the source, then acknowledges its pending request; some devices also retain a local event that needs its own acknowledgement. The IRQ chapter gives the source map, acknowledgement rules and handler entry sequence.

A synchronous exception follows the system exception route rather than the maskable interrupt route. BIOS fault handling records the CPU cause, exception program counter and other diagnostic words in SYSTEM. Supervisor controls and the separate supervisor-request input coordinate entry to firmware service; they are not application input mappings. The SYSTEM chapter specifies privilege, device boundaries and fault capture.

A SYSTEM reset command requests a reboot at a CPU execution boundary. The command write finishes before the reboot, so software must allow for a variable request-to-reboot interval. See Reset requests and Reset and cartridge startup for the reset and subsequent firmware sequence.

# 2 Memory and program storage

The CPU uses one byte-addressed map for system ROM, main RAM, cartridge storage and device registers. Main RAM holds writable program data and transfer buffers. ROM supplies read-only bytes. Device registers provide word ports whose reads and writes can consume results, change configuration or start work.

GPU local memory and APU sample RAM have separate access paths. Use GPU image and readback commands for local memory, and APU transfer ports for sample RAM. Before placing cartridge buffers in main RAM, reserve the I/O area, GEO scratch and the installed firmware's static storage as specified below.

## 2.1 Addresses and data sizes

BMSX uses 32-bit unsigned byte addresses. Each address identifies one byte; the address space runs from `0x00000000` through `0xffffffff`, inclusive. A 32-bit word occupies four consecutive bytes. For 16-bit and 32-bit values in main RAM, byte order is little-endian: the least significant byte is at the lowest address.

All capacities in this section are byte counts: 1 KiB is 1024 bytes and 1 MiB is 1048576 bytes. A range written `[start, end)` includes `start` but excludes `end`; its last byte is at `end - 1`. The tables use inclusive starts and exclusive ends. `0x100000000` denotes the exclusive end of the address space, not an address that a program can supply.

## 2.2 Installed memory and address windows

BMSX has **4 MiB of main RAM**, occupying `[0x08000000, 0x08400000)` before I/O mapping and reservation restrictions; its last byte is `0x083fffff`. The main-RAM address window is larger than the installed memory. Similarly, the system-ROM window is an address-space allocation, not the byte length of the installed firmware. Cartridge windows describe the cartridge bus, not how much ROM or RAM a cartridge contains.

| Address-space region | Start, inclusive | End, exclusive | Window size or status |
|---|---|---|---|
| System ROM | `0x00000000` | `0x01000000` | 16 MiB window; firmware image length is separate |
| Unmapped gap | `0x01000000` | `0x08000000` | No mapped storage |
| Main RAM | `0x08000000` | `0x10000000` | 128 MiB window; 4 MiB installed at its beginning |
| Cartridge ROM | `0x10000000` | `0x30000000` | 512 MiB window; installed media capacity is not specified by this window |
| Cartridge RAM | `0x30000000` | `0x30f00000` | 15 MiB window; does not imply 15 MiB is present |
| Cartridge MMIO | `0x30f00000` | `0x31000000` | 1 MiB window |
| Unmapped upper range | `0x31000000` | `0x100000000` | Outside the cartridge bus and other declared windows |

The cartridge bus begins at `0x10000000` and ends just before `0x31000000`; the system's main-RAM window ends at that cartridge-bus start. Within the main-RAM window, `[0x08400000, 0x10000000)` has no installed main-RAM backing. It is not an extension or mirror of the installed 4 MiB.

The I/O-slot aperture contains 16656 four-byte slots: 66624 bytes in `[0x08000000, 0x08010440)`. The reservation and allocation boundaries below remain separate from that decoded aperture.

GPU VRAM is a separate **2 MiB** capacity and is not included in the 4 MiB main-RAM total.

## 2.3 I/O register-bank directory

The following directory covers the complete I/O aperture in address order. Each word position occupies four bytes; its address is the first byte of that word. The end column is exclusive. Counts include status, snapshot and result words as well as control words: they are **addressed word positions**, not counts of independently writable registers. Access permissions, read side effects, command admission and reset values belong to the individual register descriptions linked in the last column, not to a whole directory range.

### 2.3.1 Lower register banks and reserved slots

| First word address | Exclusive end | 32-bit word positions | Unit and subbank |
|---|---|---|---|
| `0x08000000` | `0x0800000c` | 3 | IRQ — pending flags, acknowledgement and mask |
| `0x0800000c` | `0x08000024` | 6 | DMA — channel 0 |
| `0x08000024` | `0x08000064` | 16 | GEO — operands, command, control, progress and fault |
| `0x08000064` | `0x08000120` | 47 | Input — sampling, keyboard, pointer, four pads and output port |
| `0x08000120` | `0x08000174` | 21 | APU — command-staging parameters |
| `0x08000174` | `0x0800017c` | 2 | APU — CMD and STATUS |
| `0x0800017c` | `0x0800018c` | 4 | APU — retained end-event words |
| `0x0800018c` | `0x08000194` | 2 | APU — selected source and active-slot mask |
| `0x08000194` | `0x080001e8` | 21 | APU — live selected-slot parameter window |
| `0x080001e8` | `0x080001f4` | 3 | APU — sample-transfer address, data and control |
| `0x080001f4` | `0x08000200` | 3 | APU — command-queue occupancy, free space and capacity |
| `0x08000200` | `0x08000204` | 1 | APU — sample sequence |
| `0x08000204` | `0x08000208` | 1 | Reserved retained word |
| `0x08000208` | `0x08010208` | 16384 | Reserved retained-slot range |

### 2.3.2 Upper register banks

| First word address | Exclusive end | 32-bit word positions | Unit and subbank |
|---|---|---|---|
| `0x08010208` | `0x08010218` | 4 | Memory bus fault — code, address, access and acknowledgement |
| `0x08010218` | `0x08010224` | 3 | APU — fault code, detail and acknowledgement |
| `0x08010224` | `0x08010228` | 1 | GEO — fault acknowledgement |
| `0x08010228` | `0x08010230` | 2 | SYSTEM — elapsed milliseconds and frame interval |
| `0x08010230` | `0x08010238` | 2 | SYSTEM — diagnostic character and line flush |
| `0x08010238` | `0x0801023c` | 1 | SYSTEM — cycles per frame |
| `0x0801023c` | `0x08010244` | 2 | GPU — GP0/GPUREAD and GP1/GPUSTAT |

| First word address | Exclusive end | 32-bit word positions | Unit and subbank |
|---|---|---|---|
| `0x08010244` | `0x080102c4` | 32 | GTE — data words D[0]–D[31] |
| `0x080102c4` | `0x08010344` | 32 | GTE — control words C[0]–C[31] |
| `0x08010344` | `0x0801034c` | 2 | GTE — normal COMMAND and CYCLES |
| `0x0801034c` | `0x08010354` | 2 | SYSTEM — CONTROL and STATUS |
| `0x08010354` | `0x08010384` | 12 | PCRTC — composition words, PMODE through BGCOLOR |
| `0x08010384` | `0x080103ac` | 10 | GTE PLUS — operands, results, flags, command and cycles |
| `0x080103ac` | `0x080103e4` | 14 | PCRTC — beam timing, CSR events and IMR masks |
| `0x080103e4` | `0x080103fc` | 6 | DMA — channel 1 |
| `0x080103fc` | `0x08010420` | 9 | IMGDEC — configuration, status, data and progress |
| `0x08010420` | `0x08010428` | 2 | Book III, “Cartridge sockets and mapped card devices”, under “Cartridge controller” — socket selection and status |
| `0x08010428` | `0x08010440` | 6 | SYSTEM — retained supervisor fault report |

These ranges partition all 16656 word positions without gaps or overlaps. A unit need not occupy one contiguous bank: DMA channel 1 is separate from channel 0; GEO and APU have separate upper fault-acknowledgement banks; GTE PLUS separates PCRTC’s composition and timing banks; and SYSTEM has timing, diagnostic, control and fault-report banks. The reserved retained slots are part of the decoded aperture, not an additional device or a payload-transfer protocol.

### 2.3.3 Reserved retained words and decoder boundary

Of the 16656 positions, **16385 are reserved retained words** in `[0x08000204, 0x08010208)`. An aligned word read returns the stored raw word; an accepted aligned word write replaces it. These words reset to zero and have no command, FIFO, automatic address increment or transfer-completion behavior. They remain reserved: retention does not make them cartridge RAM or grant a scratch-buffer allocation. Do not apply this read/write description to other directory rows; their unit-specific permissions and side effects still apply.

I/O decoding and the I/O reservation have different limits. Addresses in `[0x08010440, 0x08040000)` reach installed main-RAM bytes rather than register slots, but remain within the reserved 256 KiB. They are not additional registers or newly available cartridge storage.

For byte, halfword and word bus transfers, the **complete transfer** must fit inside the decoded I/O aperture for its I/O width rules to apply. A transfer crossing that end uses main-RAM backing for its complete width; it is not split between a final register and RAM. These examples describe mapped-bus transfers, not permission to bypass ordinary CPU instruction alignment:

| Mapped transfer | Selected path |
|---|---|
| U32 at `0x0801043c` | Last complete I/O slot: supervisor fault DOMAIN |
| U32 at `0x0801043e` | Four main-RAM bytes spanning the decoder boundary |
| U16 at `0x0801043f` | Two main-RAM bytes spanning the decoder boundary |
| U32 at `0x08010440` | Main-RAM backing within the reservation |

Ordinary CPU U16 and U32 instructions reject the two unaligned examples before reaching the mapped bus. The scalar-access rules separate that CPU alignment check from I/O decoding. F64 instead consists of two word transfers: an aligned F64 access at `0x0801043c` reaches the last I/O slot first and RAM at `0x08010440` second, subject to the ordered fault rules.

## 2.4 Reserved and cartridge storage in main RAM

The installed main RAM occupies `0x08000000` through `0x083fffff`, a total of 4 MiB. This is a physical storage extent, not a cartridge allocation range. It includes the I/O reservation, GEO scratch storage, and the system's static storage as well as the cartridge program's static storage.

Subtracting the 256 KiB I/O reservation and 512 KiB GEO scratch reservation leaves **3.25 MiB** in `[0x080c0000, 0x08400000)` before accounting for system and cartridge static storage. The installed firmware occupies the beginning of this span. Its actual DATA/BSS footprint, rather than 3.25 MiB alone, determines where cartridge placement can begin.

The following ranges use an exclusive end: the end address is the first byte outside the range.

| Use | Start | Exclusive end | Size |
|---|---|---|---|
| Declared I/O reservation | `0x08000000` | `0x08040000` | 256 KiB |
| GEO scratch reservation | `0x08040000` | `0x080c0000` | 512 KiB |
| System static storage | `0x080c0000` | Actual end of system BSS | Specified by the system executable image |
| Cartridge static storage, compact layout below | Actual end of system BSS | Actual end of cartridge BSS | Specified by the cartridge executable image |

The implemented I/O register aperture ends at `0x08010440`, before the end of the declared I/O reservation. This smaller aperture does not make the remainder of the reservation available for cartridge variables or buffers. Likewise, the GEO scratch reservation is not cartridge static storage.

`0x080c0000` is the start of **system DATA**, not a universal cartridge RAM base. The installed firmware's BSS follows its DATA. Account for their actual occupied extent in the matching BIOS image, rather than assuming a fixed cartridge base. The image's declared BSS address and byte count identify that span; their sum is its exclusive end. These are linking information, not a request to allocate or clear memory when the image is selected.

The following compact layout places cartridge DATA at the system BSS end and cartridge BSS immediately after that DATA. This adjacency is a layout choice, **not a hardware requirement**. Another cartridge placement can use different unreserved RAM addresses; its records, code and startup initialization must all use that chosen placement without overwriting firmware storage.

| Section in the compact layout, in address order | Run address |
|---|---|
| System DATA | `0x080c0000` |
| System BSS | End of system DATA |
| Cartridge DATA | Exported actual end of system BSS |
| Cartridge BSS | End of cartridge DATA |

Each end in this table is exclusive and includes the section's alignment padding. The compiler aligns static objects to their type alignment and rounds section sizes to multiples of four bytes.

Choosing a static layout is not a hardware allocation service. Neither the installed RAM extent nor the end of cartridge BSS defines a free arena, stack reservation, or byte-addressed heap. Use the actual installed system image when calculating firmware occupancy.

## 2.5 Firmware reservation in VRAM

The 2 MiB VRAM is shared GPU storage, arranged as 1024 rows of 1024 **16-bit words**. Each row spans 2048 bytes. A drawing destination, texture source or display source selects storage within that same memory; it does not create a separate CPU byte-address allocation.

The firmware uses VRAM both for the full terminal drawing surface and for its system texture atlas, which supplies terminal glyphs. Preserve the **bounding rectangle covering both**, rather than reserve only the currently visible text or the atlas alone. This VRAM reservation is separate from the I/O, GEO scratch and static-storage reservations in main RAM.

### 2.5.1 Handoff is not release

Clearing terminal text or status, or selecting a smaller active terminal viewport, does not shrink the queried full-layout reservation. Cartridge handoff does not release it either. The BIOS monitor later uses the same terminal coordinates and uploads the system atlas again before drawing its output.

Keep the complete queried region out of cartridge allocations. The query does not install hardware write protection or grant permission to overwrite that region, and it does not establish that every coordinate outside it is free.

For the limits of later device-context restoration, see System control and timing — Device boundaries during supervisor service. Restoring device context does not restore overwritten VRAM pixels.

### 2.5.2 Query the firmware reservation

The named BIOS function `gpu/system_vram_region` takes no arguments and returns two packed words describing the installed firmware's VRAM rectangle.

The first return word contains x in its low half and y in its high half. The second contains width and height, not right and bottom coordinates. The returned rectangle is `[x, x + width) × [y, y + height)`.

Here x and width count **16-bit VRAM words** across a row; y and height count **rows**. Reserve that rectangle in the cartridge's VRAM layout. The query reports the terminal and atlas extent; uploading and synchronizing those bytes remain separate operations.

Call it through `require`; do not substitute a numerical function address or treat its name as an asset identifier.

```
local system_vram_region<const> = require('gpu/system_vram_region')
local system_origin<const>, system_size<const> = system_vram_region()
local x<const> = system_origin & 0x0000ffff
local y<const> = system_origin >> 16
local width<const> = system_size & 0x0000ffff
local height<const> = system_size >> 16
```

### 2.5.3 Current-firmware example

For the current firmware image's terminal layout and packed atlas, the rectangles are:

| Use | x range, in 16-bit words | y range, in rows |
|---|---|---|
| Full terminal, 320 × 240 | `[704, 1024)` | `[720, 960)` |
| Actual system atlas, 216 × 12 | `[768, 984)` | `[960, 972)` |
| Returned bounding rectangle, 320 × 252 | `[704, 1024)` | `[720, 972)` |

For those dimensions, the returned origin word is `0x02d002c0` and the size word is `0x00fc0140`. These numbers are a **current-firmware example**, not universal constants for cartridge code. The atlas's configured maximum is 256 × 64, but its actual packed extent need not fill that maximum; consume the query result instead.

The 320 × 252 bounding rectangle covers 80640 16-bit words, or 161280 bytes of storage footprint. Its row stride is nevertheless **1024 words**, or 2048 bytes: it is a rectangle within VRAM, not a contiguous CPU byte-address allocation. The bounding rectangle also includes space outside the exact terminal-plus-atlas union; the query does not identify internal holes for reuse.

## 2.6 ROM contents, RAM sections, and startup

ROM stores program instructions, read-only data and the initial contents of writable data. During execution, mutable static objects occupy main RAM. The following section names distinguish their storage and initialization rules.

| Section | Contents in the ROM image | Address used during execution |
|---|---|---|
| TEXT | Executable code | Instruction byte address in ROM |
| RODATA | Read-only data bytes | Read-only byte address in ROM |
| DATA | Initializer bytes | DATA run address in main RAM, after startup copies the initializers |
| BSS | No section byte payload | BSS address in main RAM, after startup clears the section |

DATA consequently has two distinct addresses:

- Its **load address** identifies the initializer bytes in the ROM image.
- Its **run address** identifies the writable storage in main RAM.

Instructions access a DATA object's RAM run address. Startup reads the initializer at its ROM load address and copies it to that run address. RODATA is used in place in ROM. BSS reserves a RAM address range, whose initial zero contents are written by startup; the ROM needs only its address and size.

### 2.6.1 When initialization occurs

A program's startup routine establishes its static storage before entering application code. The standard BLua startup sequence is:

| Order | Operation |
|---|---|
| 1 | Copy the complete DATA section from its ROM load address to its RAM run address. |
| 2 | Write zero throughout the BSS section in RAM. |
| 3 | Run static module initialization. |
| 4 | Call the program's entry function. |

The DATA copy and BSS clear use four-byte word accesses over their complete padded byte counts. System startup initializes the firmware sections; cartridge startup initializes the cartridge program's sections. System startup also removes the temporary boot-primitive bindings after module initialization.

EXEC starts the addressed function. DATA initialization and BSS clearing occur only when the instructions performing them execute. Therefore the ROM startup address must select the initialization routine when that initialization is required. Entering another function leaves the sections as they stand. Re-entering startup repeats the copy and clear, replacing the program's writable static contents with their initial values.

## 2.7 Reset and cartridge startup

### 2.7.1 From reset to the system program

At reset the CPU selects system ROM and enters the startup function specified in its ROM header. The call stack and transient execution state are cleared, there are no supplied arguments, and Status starts at `00000001H`: supervisor mode with current interrupt enable set.

The firmware startup routine initializes system DATA and BSS, runs module initializers, then enters the BIOS boot program. That program selects an executable cartridge and performs the handoff described below.

### 2.7.2 ROM header fields

The ROM header begins at the ROM base. Provide at least its first **60 bytes**, through offset `0x3b`, for CPU program selection. All offsets in the following table are byte offsets from that base. Header words are unsigned 32-bit little-endian. The CPU uses the program and vector fields; BIOS also checks the identification and startup fields when selecting a cartridge.

| Byte offset | Field | CPU program selection | BIOS cartridge selection |
|---|---|---|---|
| `0x00` | Magic | Not read. | Requires `0x58534d42`; bytes `42 4d 53 58` spell `BMSX`. |
| `0x04` | Header size | Not read. | Requires at least 60 bytes. |
| `0x08`–`0x1f` | Other prefix words | Not read. | Not read. |
| `0x20` | BLua32 image offset | Byte offset relative to the ROM base; zero means no executable image. | Requires a nonzero word. |
| `0x24` | Declared image byte count | Not read. | Not read. |
| `0x28` | Startup function | Absolute function-record address. | Requires a nonzero word; supplies the EXEC target unchanged. |
| `0x2c` | IRQ function | Absolute function-record address. | Not read. |
| `0x30` | Exception function | Absolute function-record address. | Not read. |
| `0x34`, `0x38` | Declared static-layout token words | Not read. | Not read. |

System ROM begins at `00000000H` and has execution-domain ID −1. Either cartridge socket uses ROM base `10000000H`, with domain ID 0 or 1 selecting the socket. Add the program-offset word to this base and retain the low 32 bits to obtain the BLua32 program-header address. The result must address installed ROM. The startup and vector fields are already absolute function-record addresses.

All 60 prefix bytes must be present in ROM. A zero program offset marks a ROM without a CPU program; an absent prefix likewise provides no program. System ROM must supply the reset program. CPU selection uses the fields shown in its column; identification checks belong to BIOS cartridge selection. The exception address's use follows the exception-routing rules, including the system route for synchronous cartridge faults.

### 2.7.3 BLua32 executable image

The BLua32 program header occupies **96 bytes** in ROM. Its offsets are relative to the program-header address obtained above. The CPU uses these six unsigned 32-bit little-endian fields to establish the program's constants and name banks:

| Image byte offset | Field | Unit and interpretation |
|---|---|---|
| `0x18` | Constant-table address | Absolute machine byte address in installed ROM. |
| `0x1c` | Constant count | Number of 16-byte constant records. |
| `0x20` | Global-name-table address | Absolute machine byte address in installed ROM. |
| `0x24` | Global-name count | Number of 8-byte name records. |
| `0x28` | System-global-name-table address | Absolute machine byte address in installed ROM. |
| `0x2c` | System-global-name count | Number of 8-byte name records. |

The other four-byte header slots describe the program's sections. They retain the following format; CPU program selection does not use them to locate the constant or name tables.

| Image byte offset | Descriptive field |
|---|---|
| `0x00` | Magic; declared constant `0x32334c42`. |
| `0x04` | Version; declared constant 1. |
| `0x08`, `0x0c` | Image byte count; flags. |
| `0x10`, `0x14` | Function-table address; function count. |
| `0x30`, `0x34` | String address; string byte count. |
| `0x38`, `0x3c` | RODATA address; RODATA byte count. |
| `0x40`, `0x44`, `0x48` | DATA load address; DATA byte count; DATA run address. |

| Image byte offset | Descriptive field |
|---|---|
| `0x4c`, `0x50` | BSS address; BSS byte count. |
| `0x54`, `0x58` | TEXT address; TEXT byte count. |
| `0x5c` | Reserved. |

Provide the complete 96-byte header, including its descriptive slots. The CPU reads the explicit constant/name addresses and counts. Magic, version, function-table enumeration, section order and declared section extents do not alter those reads or bound instruction fetching. DATA and BSS are initialized by the startup instructions.

Table addresses are **absolute machine byte addresses**. Tables and their text must occupy installed ROM; their placement is independent of table order and of the declared image extent. Cartridge ROM reads select the program's execution socket, while system-ROM addresses select system ROM. A zero table count leaves its address unused.

#### 2.7.3.1 Constant and name records

Constant record `i` begins at `constantTableAddress + 16i`, with zero-based `i`. The tag is a four-byte unsigned little-endian word at record offset `0x00`:

| Tag | Value | Payload consumed |
|---|---|---|
| 0 | Nil | None. |
| 1 | False | None. |
| 2 | True | None. |
| 3 | Number | Eight-byte IEEE-754 binary64, little-endian, at `0x04`–`0x0b`. |
| 4 | String | Absolute ROM text byte address at `0x04`; text byte count at `0x08`, both u32 little-endian. |

Constant indices select records in table order. Tags 0–2 encode their value without a payload; bytes `0x0c`–`0x0f` are unused. A record with another tag is invalid and prevents program activation.

Both name tables use the same layout. Name record `i` begins at `tableAddress + 8i`:

| Record byte offset | Field |
|---|---|
| `0x00` | Absolute ROM text byte address, u32 little-endian. |
| `0x04` | Text byte count, u32 little-endian. |

Text lengths count **bytes**, not codepoints. Reads consume the specified span, not a NUL-terminated sequence. A zero byte within the span is part of the text. A zero byte count produces empty text without reading the address.

Constant text and names use complete, shortest-form UTF-8 encodings of Unicode scalar values. Invalid leading or continuation bytes, truncated sequences, overlong encodings, surrogate values D800H–DFFFH and values above 10FFFFH make image activation fail. An initial `EF BB BF` encodes the ordinary character U+FEFF; it is retained rather than removed as a byte-order marker. A string beginning with U+FEFF therefore differs from the string without that character.

The two name tables establish separate ordinary-global and system-global namespaces. Within each namespace an existing name retains its value and a new name begins nil. A name-table index selects that name's slot; initial values are established by executed instructions.

#### 2.7.3.2 Function and upvalue records

A function address identifies a **32-byte function record**, not its first instruction. Record fields consumed below are unsigned 32-bit little-endian words:

| Record byte offset | Field | Interpretation |
|---|---|---|
| `0x00` | Code address | Absolute machine byte address of the initial instruction. |
| `0x04` | Declared code byte count | Not read. |
| `0x08` | Parameter count | Number of fixed parameter slots. |
| `0x0c` | Maximum stack | Number of register-value slots, not bytes or a RAM address. |
| `0x10` | Flags | Bit 0 VARARG; bit 1 STATIC. |
| `0x14` | Upvalue-table address | Absolute machine byte address. |
| `0x18` | Upvalue count | Number of four-byte descriptors. |
| `0x1c` | Reserved | Not read. |

VARARG retains arguments beyond the fixed parameters. STATIC permits direct entry through EXEC; the closure-construction rules are described in Book II, “CPU and BLua32 instruction reference”, under “Closures and captured variables”. The immediate-address form of `CLOSURE` encodes a function-record address in **16-byte units**; its register-address form supplies the full address. Function-record reads themselves do not impose a 16-byte alignment check. The declared code byte count does not bound instruction fetches.

Upvalue descriptor `i` is a four-byte unsigned little-endian word at `upvalueTableAddress + 4i`. Bit 31 set captures a parent-frame register; bit 31 clear captures a parent-closure upvalue. Bits 0–30 contain the zero-based index. Function and upvalue records use ordinary mapped-bus reads and may therefore occupy mapped memory. Constant and name-table records and their text require ROM.

Function-record words are little-endian; Book II, “CPU and BLua32 instruction reference”, under “Instruction words and execution resources”. Neither a function-record address nor a record’s code address is relative to the image header.

### 2.7.4 Optional BIOS diagnostic directory

The ROM word at byte offset `0x3c` supplies a **ROM-relative byte offset** to an optional diagnostic directory. Zero means absent. This word extends beyond the CPU’s 60-byte boot prefix: it is consumed by BIOS source reporting and selected-frame evaluation, not by instruction execution or image activation. Without the directory, executable code and ordinary boot remain usable, but those BIOS services cannot recover its source locations or named frame scopes.

All directory words are unsigned 32-bit little-endian. The header occupies **76 bytes**. Table offsets below are relative to the directory base; a count is a number of records, not a byte count.

| Header byte offset | Field |
|---|---|
| `0x00`, `0x04` | Source-range count; range-table offset |
| `0x08`, `0x0c` | File count; file-table offset |
| `0x10`, `0x14` | Line-offset count; line-offset-table offset |
| `0x18`, `0x1c` | Path-byte-table offset; path byte count |
| `0x20`, `0x24` | Scope-function count; function-table offset |
| `0x28`, `0x2c` | Scope-frame count; frame-table offset |
| `0x30`, `0x34` | Binding count; binding-table offset |
| `0x38`, `0x3c` | Interval count; interval-table offset |
| `0x40`, `0x44` | Scope-name byte count; name-byte-table offset |
| `0x48` | Static-global binding count; these occupy the first binding records |

#### 2.7.4.1 Source locations

| Record | Bytes | Four-byte fields, in increasing byte-offset order |
|---|---|---|
| Range | 12 | Absolute PC start; packed line/file; column |

| Record | Bytes | Four-byte fields, in increasing byte-offset order |
|---|---|---|
| File | 20 | Directory-relative display-path offset; path byte count; ROM-relative source offset; source byte count; first line-offset index |
| Line offset | 4 | UTF-8 byte offset from the file's source start |

Range records are sorted by PC. Lookup selects the last record whose start is not greater than the PC. The packed location holds the one-based line in bits 31:16 and the zero-based file index in bits 15:0; `FFFFFFFFH` marks no source. Column is one-based. Include a no-source record at PC zero and another at the text end so addresses outside mapped source do not inherit its final location.

Each file's line-offset sequence starts at zero and includes an end-of-source sentinel. For line *L*, BIOS reads offsets at `first + L - 1` and `first + L`; it strips terminal CR/LF bytes from the displayed line. Paths and source spans use explicit byte counts, not NUL termination. Source bytes may occupy their ordinary ROM location; they need not be duplicated inside the directory.

#### 2.7.4.2 Named frame scopes

Scope functions are sorted by their absolute function-record address. BIOS matches that address, computes instruction-word position `(PC - codeAddress) >> 2`, and selects a frame at the requested inline depth whose active interval contains that word. These are source scopes for BIOS evaluation, not additional CPU call frames or instruction-fetch bounds.

| Record | Bytes | Four-byte fields, in increasing byte-offset order |
|---|---|---|
| Scope function | 16 | Absolute function-record address; absolute code address; first frame index; frame count |
| Scope frame | 28 | Inline depth; directory-relative name offset; name byte count; first active-interval index; active-interval count; first binding index; binding count |
| Binding | 32 | Directory-relative name offset; name byte count; flags; location; first visible-interval index; visible-interval count; first live-interval index; live-interval count |
| Interval | 8 | First instruction-word position; exclusive end position |

Interval lists are sorted nonoverlapping half-open ranges in instruction-word positions relative to the function's code address. Names use UTF-8 byte spans. A frame's visible bindings overlay the initial static-global bindings; later same-named bindings shadow earlier ones even when their value is unavailable. Visibility and availability are separate: the visible interval selects the name, while the live interval determines whether its stored value is available.

Binding flags are bit 0 UPVALUE, bit 1 CONST, bit 2 ADDRESS and bit 3 TYPE. Without ADDRESS or TYPE, location identifies a register or, with UPVALUE, an upvalue slot. ADDRESS selects a raw static-storage address and remains available without a live interval. TYPE describes a type rather than a readable value. `FFFFFFFFH` is the no-location word. CONST marks a binding that evaluation must not reassign; it does not freeze a Table's fields.

This directory is distinct from private development symbols or a cartridge resource directory. Neither is required to locate this BIOS-consumed format: its address comes directly from the ROM header word.

### 2.7.5 BIOS cartridge selection

The BIOS scans cartridge slot 0 before slot 1, considering slots whose presence bits are set in the sampled cartridge-status word. For each candidate it selects that slot and checks:

- ROM magic is `0x58534d42`;
- the header-size word is at least 60;
- the executable-image-offset word is nonzero;
- the startup-function word is nonzero.

The first candidate passing all four checks supplies the startup-function address. The CPU then applies the function-record and program rules described above.

The boot screen updates at VBlank. BIOS hands over when an executable cartridge is ready and its boot-frame counter has reached at least 50. A change between the missing, invalid and ready cartridge states resets that counter; changing the selected card while its classification remains ready does not by itself reset it. At handoff BIOS clears and flushes terminal status, masks central IRQ delivery, and writes the selected startup-function address to CP0 EXEC.

### 2.7.6 EXEC and cartridge entry

A startup function address identifies a **function record**, not the address of its first instruction. EXEC reads the target record and requires its STATIC flag. An unreadable record or one without that flag causes a hard halt.

Successful EXEC clears the active call stack, selects the target executable image and execution domain, establishes its entry status, clears CPU halt, wait, blocked-write and yield state, and pushes the selected static function with no arguments. A successful `EXEC` of a cartridge image sets CPU status to IE and user mode (bits 0 and 1) before starting the selected static function—not at a later entry-function call. The BIOS's zero IRQ-mask write must not be confused with clearing the CPU interrupt-enable bit.

The standard cartridge startup routine then performs:

1. Copy the cartridge image's DATA initializer bytes from their load address to their RAM run address.
2. Zero the cartridge image's BSS.
3. Run static module initialization.
4. Call the cartridge entry function with no arguments.

EXEC enters exactly the selected static function. Choose the startup routine when section initialization is required; entering another function skips that sequence. Re-entering startup repeats the DATA copy and BSS clear. Initialize all cartridge-owned storage whose contents the program requires.

### 2.7.7 Taking ownership after the BIOS

At normal BIOS handoff, video timing is running and the display contains BIOS drawing. Cartridge code may retain that display or establish its own drawing and display configuration. Program each unit before use; its reset-value table applies to reset, while its inherited boot state follows the rules here. System timing describes the live readouts, and Book II, "CPU and BLua32 instruction reference", under "CP0 and execution control" describes entry privilege and interrupt enable.

The BIOS writes zero to the central interrupt mask before handoff. This masks delivery; it does not clear pending flags or clear CPU interrupt enable, which is set at cartridge entry. Enabling a pending source can make it deliverable without another event. Choose one dispatch owner, install the first source's handler and prepare that source before enabling its delivery. Acknowledge local source state and central pending state as required by the unit; they are separate responsibilities. See Interrupt controller — Choose one cartridge dispatch route and Acknowledge the source and the controller separately. The GPU marker example's Starting conditions show this ordering for its chosen configuration, not a universal device-reset recipe.

Execution handoff also does not release the Firmware reservation in VRAM; see Handoff is not release. Later supervisor service preserves selected device context, not overwritten VRAM pixels. Its boundaries are described in System control and timing — Device boundaries during supervisor service.

INPUT likewise carries BIOS sampling state into cartridge entry rather than starting from its reset words. Follow BIOS handoff and supervisor sampling to establish a fresh cartridge sample, including after supervisor return.

#### 2.7.7.1 Firmware work at normal boot handoff

On the normal reset-to-cartridge boot path, BIOS has supplied its final GPU command batch and reached that batch's completion marker. Firmware DMA and VBlank handlers have stopped submitting work. The following unit state is then available to the cartridge. Entry through another EXEC route must establish its own device state.

| Surface | Firmware work at cartridge entry |
|---|---|
| DMA0 and DMA1 | No outstanding firmware transfer or admitted transfer block. DMA0's final transfer has finished; this boot path never starts DMA1. |
| GP0 input and readback | No active firmware writer, unfinished firmware packet, or outstanding firmware VRAM-to-CPU readback. |
| GEO | No outstanding firmware geometry job. |
| IMGDEC | No outstanding firmware decode, input, or output job. |
| APU | No queued firmware command, active playback slot, or outstanding sample-transfer batch. This does not imply that the APU clock or output activity has stopped. |

Submitted BIOS drawing may still be pending after its final marker, although the firmware has stopped producing commands. Before changing resources used by that work, use the GPU completion protocol. Display publication remains a separate PCRTC event. EXEC leaves the device registers and submitted work in place.

## 2.8 Typed byte storage and address views

Use byte-addressed storage for the records, command words and sample bytes that hardware consumes. BLua's `bss`, `data` and `rodata` declarations provide linked storage addresses; a local typed view supplies an access layout for an existing address. Neither a local value nor an ordinary Table is a device buffer.

The complete declaration, initializer, type-size, alignment, array and struct rules are in Book II, "BLua source programs and extensions", under "Typed storage, layouts, and address views". That reference also defines `sizeof`, `offsetof`, address-of and zero-based typed indexing. The memory rules in this chapter determine what happens when the resulting address reaches RAM, ROM or an I/O port.

For example, a word view of an existing DMA register bank accesses consecutive four-byte ports. It does not reserve a DMA channel or alter the port layout:

```
local dma0<const>: *word = 0x0800000c
local remaining = dma0[2]
```

The read reaches TRANSFER_COUNT at `08000014H`. By contrast, `dma0 + 2` is the byte address `0800000EH`, not that third port. Use an indexed typed access or an explicit byte offset that preserves the device's required alignment.

A storage address still carries the region's ownership and lifetime rules. Keep cartridge objects outside the reserved main-RAM ranges, use ROM as a read-only byte source, and keep a device's input bytes unchanged until its completion condition releases them. The record-copy example below applies typed layouts to exact bytes; DMA buffer lifetime and the following unit-specific sections cover asynchronous use.

## 2.9 Language execution storage and byte RAM

Function calls use language value slots. A frame's register area follows its variable-argument area in the active thread's slot stack. The function record's maximum-stack value is a **slot count**, not a RAM address or byte count. These slots have no cartridge-addressable byte-RAM base; do not place an assumed stack at the top of main RAM.

Language values use a shared allocation budget, described under Book II, "CPU and BLua32 instruction reference", under "Language allocation and lifetime". That budget does not describe a mapped byte-RAM heap or report free RAM bytes. Mapped DATA/BSS placement and the language budget are separate constraints: placing a static byte buffer neither allocates a language Table nor yields a byte address for the CPU's value slots.

### 2.9.1 Values are not byte buffers

A language value can hold a Number, text, a Table, a function or a Thread. Copying a Table or function value preserves its identity; it does not copy its contents into main RAM. A device needs the numeric address of storage containing its prescribed bytes, not the Table that the program uses to organize them.

Use typed static storage and scalar writes to construct that byte layout. Loads return numeric scalars under the selected access format; they do not reconstruct arbitrary language values from words. Integer stores retain the specified low bits, while floating stores encode floating-point bits. The scalar memory rules give the alignment, conversion and partial-access behavior.

### 2.9.2 Function arguments, results and BIOS calls

Passing an address to a function passes its numeric value. It neither copies the pointed-to bytes nor extends their lifetime. A callee can establish a typed view of that address before accessing it, as in the example below. Ordinary calls use value slots, not a cartridge-addressable byte stack.

Use Book II, “CPU and BLua32 instruction reference”, under “Calling and returning values” for argument/result placement and closure capture, and Book II, “BLua source programs and extensions”, under “BLua source semantics” for source-level varargs and result expansion. Named static BIOS targets and installed global library values are distinct call routes; see Book II, “BLua source programs and extensions”, under “Named BIOS function imports”. Calling either kind of BIOS function is an ordinary call, not EXEC or a privilege transition. Its documented memory and device requirements still apply.

### 2.9.3 Text, codepoints and UTF-8 bytes

A String holds Unicode text, not the numeric address of encoded bytes. `#text` counts codepoints; a byte count for DMA or a hardware record counts bytes. For example, U+00E9 is one codepoint but occupies two bytes in UTF-8. U+0000 is an ordinary character rather than a String terminator.

The BIOS bridge `string.from_utf8(address, byte_count)` reads UTF-8 from byte memory. Supply a complete, well-formed encoded span; it is not a malformed-input validator, and continuation reads for an incomplete final sequence can extend past the supplied count. A zero byte within a valid span becomes U+0000. The call reads the supplied storage; it does not give a String a byte address or turn that source into a retained DMA buffer.

A static `string` field instead stores an image constant reference. Its address locates that reference, not inline UTF-8. Likewise, `string.pack` produces a String value rather than an addressed hardware buffer. Use explicit byte storage when supplying a device’s payload. Book II, “BIOS String operations” defines the text operations, conversions and their full argument contracts.

### 2.9.4 Worked values, byte storage and text

After normal BIOS entry, typed storage supplies the destination bytes; passing its address to a function does not allocate a new destination:

```
bss written: word[2]
local write_pair = function(address, first, second)
    local destination<const>: *word = address
    destination[0] = first
    destination[1] = second
    return first + second, 2
end
local sum, count = write_pair(written, 17, 25)
```

`sum` is 42, `count` is 2, and the two stored words are 17 and 25. The function’s address view and the declared array select the same storage. Neither a Table nor a String can be substituted for its byte address.

Text uses a different route:

```
rodata utf8_bytes: u8[8] = { 0x41, 0xc3, 0xa9, 0, 0xf0, 0x9f, 0x99, 0x82 }
local text = string.from_utf8(utf8_bytes, #utf8_bytes)
local count = #text
local zero = string.byte(text, 3)
local face = string.byte(text, 4)
```

Eight bytes decode to four codepoints: `0041H`, `00E9H`, `0000H`, `1F642H`. `count` is 4, `zero` is 0 and `face` is `1F642H`. Embedded zero does not terminate the String; the supplementary codepoint counts as one. Conversely, `'\xc3\xa9'` contains two codepoints rather than decoding two UTF-8 bytes. Use Book II, “BIOS String operations” for explicit text/byte conversion, and Book II, “CPU and BLua32 instruction reference”, under “Calls, closures, globals, and tables” for argument/result counts, captures and language-value identity.

## 2.10 Execution addresses and instruction reads

### 2.10.1 Function records and instruction addresses

A function address points to a **function record**, not to the first instruction. The record’s `CODE_ADDRESS` word supplies the instruction address. These two addresses are not interchangeable.

The record and the instructions also use different byte orders:

| Item | Representation |
|---|---|
| Function record’s `CODE_ADDRESS` word | 32-bit little-endian |
| Instruction word | Four bytes, big-endian |

Reading a function record therefore does not use the byte order used to decode its instructions.

### 2.10.2 Execution domains and cartridge sockets

BMSX has three execution domains:

| Domain ID | Execution image |
|---|---|
| −1 | System |
| 0 | Cartridge socket 0 |
| 1 | Cartridge socket 1 |

For cartridge execution, the active image binds instruction reads to its cartridge socket. Domain identifiers are not RAM addresses and do not allocate isolated RAM.

### 2.10.3 Instruction alignment and read faults

Before instruction decoding, the PC is checked for four-byte alignment. An unaligned PC raises an address-error-load exception at that instruction address.

An ordinary instruction occupies one four-byte word. A `WIDE`-prefixed instruction occupies two words: the prefix at PC and its body at PC + 4. Instruction reads use the memory map and the big-endian instruction representation described above.

A fault during an instruction read raises an instruction-bus-error exception. For a `WIDE` instruction, a fault while reading either word is attributed to the original instruction PC—the prefix address—not the body address.

### 2.10.4 Selected instruction base charges

The CPU/system timebase is **33,868,800 cycles per second**. The following are selected instruction base charges, not a complete timing table:

| Instruction | Base charge |
|---|---|
| Scalar `LOAD_MEM` and `STORE_MEM` | 2 cycles |
| Scalar displaced forms `LOAD_MEM_D` and `STORE_MEM_D` | 2 cycles |
| `ADD`, `SUB`, `MUL` | 1 cycle |
| `WIDE` prefix | 0 cycles |

A `WIDE` instruction uses its body's base charge; the prefix does not add another base charge. These charges do not specify the cost of a source expression, total program execution time, or sustained memory bandwidth.

Use Book II, "CPU and BLua32 instruction reference", under "Cycle accounting and execution boundaries" for admission, retries and additional instruction work, and its Book II, "CPU and BLua32 instruction reference", under "Memory instruction reference" for the complete transfer-instruction table. DMA — Transfer timing and sharing the bus supplies endpoint rates, block and tail budgets, contention and GPU transport examples; CPU instruction charges are not DMA transfer rates.

## 2.11 Scalar memory accesses

The rules in this section apply to ordinary scalar `LOAD_MEM` and `STORE_MEM` instructions with the access kinds listed below. Addresses are unsigned 32-bit byte addresses. I/O registers occupy four-byte slots.

### 2.11.1 Width and alignment

| Access kind | Kind ID | Width | Required address alignment |
|---|---|---|---|
| Word | 0 | 4 bytes | 4 bytes |
| U8 | 1 | 1 byte | 1 byte |
| U16LE | 2 | 2 bytes | 2 bytes |
| U32LE | 3 | 4 bytes | 4 bytes |
| F32LE | 4 | 4 bytes | 4 bytes |
| F64LE | 5 | 8 bytes | **4 bytes**, not 8 |

The CPU checks alignment before making the mapped access. A misaligned ordinary load enters a synchronous address-error-load exception; a misaligned ordinary store enters a synchronous address-error-store exception. For stores, this check also precedes the write-readiness check. Thus an ordinary misaligned U32 access raises an address error before it can reach any I/O width or slot-alignment check.

### 2.11.2 System-ROM data accesses

The declared system-ROM window is `0x00000000` through `0x00ffffff` (16 MiB). When the full width of a mapped scalar read fits this window, bytes beyond the installed ROM image are returned as zero. A U16 or U32 read that straddles the installed image's end is zero-filled only for its missing bytes.

Scalar writes to the system-ROM window report `BUS_FAULT_UNMAPPED` and leave ROM unchanged. They do not report `BUS_FAULT_READ_ONLY`. The CPU alignment check precedes these mapped-access rules. These are system-ROM data-access rules, not cartridge-ROM or instruction-fetch rules.

### 2.11.3 I/O widths and read side effects

The following I/O rules concern accesses whose entire byte range lies inside the I/O aperture, after the CPU alignment check has passed. They do not define accesses that cross the aperture's end.

| Access | I/O behavior |
|---|---|
| U8 read at a four-byte-aligned slot address | Reads the complete slot, then returns its low eight bits. |
| U8 read at another byte address | Reports `BUS_FAULT_UNALIGNED_IO`; the slot is not read. |
| U8 write, including at a slot-aligned address | Reports `BUS_FAULT_UNALIGNED_IO`; the slot is not written. |
| U16 read or write | Reports `BUS_FAULT_UNALIGNED_IO`; the slot is not accessed. |
| Aligned Word or U32 access | Uses the complete four-byte slot, subject to that register's access rules. |
| Aligned F32 access | Uses the U32 access path and reinterprets one word, with the F32 access flag. It is not a separate floating-point register protocol. |
| Aligned F64 access | Performs two ordered four-byte accesses as described below, with the F64 access flag on each. |

A byte read is therefore not necessarily a side-effect-free way to inspect part of a register. If reading a slot has side effects, an aligned U8 read performs the complete slot read and its side effects before selecting the low eight bits. Byte and halfword writes are not substitutes for word writes to I/O commands.

### 2.11.4 Eight-byte accesses and partial completion

An F64 access first accesses the low word at the starting address, then the high word at that address plus four. A new fault on the first word stops the access before the second word is attempted. A fault on the second word reports that second byte address, unless an older pending fault already occupies the fault latch.

These two accesses are not an atomic eight-byte transaction. If an F64 store successfully writes the low word and then faults on the high word, the low-word write is not undone. An F64 read returns zero at the mapped-access level if either word faults; the ordinary CPU load does not commit that zero to its destination register.

An aligned store also checks whether the mapped write is ready. If it is not ready, the store waits and performs no write on that pass. This is distinct from reporting a bus fault.

An unmapped scalar access reports `BUS_FAULT_UNMAPPED`. The unmapped U8, U16 and U32 read paths return zero, while the unmapped write paths perform no target write. Rejected I/O U8 and U16 reads likewise return zero. These returned zeros are not successful results of faulting ordinary CPU loads: the CPU exception check described below occurs before the destination register is assigned.

## 2.12 Bus-fault status and CPU exceptions

The bus-fault latch records the first pending fault. Its retained status and the CPU's response to a new fault are separate mechanisms: a later fault can cause a synchronous CPU exception without replacing the older recorded fault.

### 2.12.1 Register reference

These four registers use byte addresses. The access labels describe aligned Word/U32 accesses; they do not grant byte or halfword writes.

| Byte address | Register | Word access and contents | System reset / ACK value |
|---|---|---|---|
| `0x08010208` | `IO_SYS_BUS_FAULT_CODE` | Read-only. Numerical code of the first pending fault. | `0x00000000` (`BUS_FAULT_NONE`) |
| `0x0801020c` | `IO_SYS_BUS_FAULT_ADDR` | Read-only. Byte address supplied by the first pending fault. | `0x00000000` |
| `0x08010210` | `IO_SYS_BUS_FAULT_ACCESS` | Read-only. Access flags supplied by the first pending fault. | `0x00000000` |
| `0x08010214` | `IO_SYS_BUS_FAULT_ACK` | Reads as zero. A zero word write does not acknowledge; **any nonzero word write** clears the latch. | `0x00000000` |

System reset and a nonzero ACK clear all three report words. CODE, ADDR and ACCESS each retain a complete 32-bit word: CODE is an enumerated code, ADDR is a byte address and ACCESS is a flags word.

Reading the status registers does not acknowledge a fault. An aligned Word/U32 write to CODE, ADDR or ACCESS reports `BUS_FAULT_READ_ONLY` before writing the destination slot. If no fault is already pending, that rejected write becomes the pending fault, recording its destination address and write access flags. Otherwise, the older recorded fault remains.

### 2.12.2 CODE values

| Name | Word value | Meaning |
|---|---|---|
| `BUS_FAULT_NONE` | `0x00000000` | No pending fault. |
| `BUS_FAULT_UNMAPPED` | `0x00000001` | Unmapped access. |
| `BUS_FAULT_UNALIGNED_IO` | `0x00000002` | Rejected I/O width or slot alignment, as specified above. |
| `BUS_FAULT_READ_ONLY` | `0x00000003` | Rejected write to a read-only register. |

### 2.12.3 ACCESS flags

The ACCESS word combines a direction flag with an access-kind flag by bitwise OR.

| Direction flag | Bit | Mask |
|---|---|---|
| `BUS_FAULT_ACCESS_READ` | 0 | `0x00000001` |
| `BUS_FAULT_ACCESS_WRITE` | 1 | `0x00000002` |

| Access-kind flag | Bit | Mask | Read ACCESS word | Write ACCESS word |
|---|---|---|---|---|
| `BUS_FAULT_ACCESS_U8` | 8 | `0x00000100` | `0x00000101` | `0x00000102` |
| `BUS_FAULT_ACCESS_U16` | 9 | `0x00000200` | `0x00000201` | `0x00000202` |
| `BUS_FAULT_ACCESS_U32` | 10 | `0x00000400` | `0x00000401` | `0x00000402` |
| `BUS_FAULT_ACCESS_WORD` | 11 | `0x00000800` | `0x00000801` | `0x00000802` |
| `BUS_FAULT_ACCESS_F32` | 12 | `0x00001000` | `0x00001001` | `0x00001002` |
| `BUS_FAULT_ACCESS_F64` | 13 | `0x00002000` | `0x00002001` | `0x00002002` |

Word and U32 have different ACCESS flags even though both access four bytes. These flags are also distinct from the kind IDs in the scalar-access table.

### 2.12.4 Capture, subsequent faults and acknowledgment

The first fault immediately records CODE, ADDR and ACCESS. These words retain that fault until acknowledgement, including when later accesses fault.

An aligned nonzero word write to ACK clears CODE to BUS_FAULT_NONE and clears ADDR and ACCESS. ACK reads zero. Any nonzero value performs the clear; a zero write leaves the report unchanged. Read and save a required report before acknowledging it.

### 2.12.5 Synchronous CPU exception entry

A new fault during a CPU scalar load enters DataBusError before the destination register is written. A new fault during a ready CPU scalar store enters the same exception; any earlier effects of a multiword access remain.

Each new CPU bus fault enters this exception even when an older report still occupies CODE/ADDR/ACCESS. A pending report by itself leaves successful accesses unaffected. Device-master faults update the report without vectoring the CPU. Clearing the report acknowledges those diagnostics; exception return is a separate CPU operation.

The CPU cause-code field occupies bits 6:2, selected by `CPU_CAUSE_CODE_MASK = 0x0000007c`.

| Ordinary-access condition | CPU cause code | Encoded cause-field value |
|---|---|---|
| Misaligned load | 4 | `0x00000010` |
| Misaligned store | 5 | `0x00000014` |
| New bus fault during the mapped access | 7 | `0x0000001c` |

These CPU cause codes are not the values 1–3 in `IO_SYS_BUS_FAULT_CODE`. They are not IRQ bits either: IRQ and NMI are independently represented by bits 10 and 16. Acknowledging the fault record and entering a CPU exception are distinct operations.

## 2.13 Installed response to a memory exception

A synchronous memory exception enters the **system exception entry**, even when the faulting instruction belongs to a cartridge. This differs from a maskable IRQ, whose entry is selected according to the interrupted mode. Declaring a cartridge `exception` function does not install it as the handler for this memory-fault route.

The system exception entry calls the BIOS exception function, which enters the BIOS monitor. Acknowledging the Memory fault record is not exception return: it clears the retained report but does not repair an address or advance the faulting instruction. Likewise, returning from an exception does not itself acknowledge that record.

### 2.13.1 Inspect an actual unmapped load

This cartridge fragment deliberately accesses unmapped storage. Its Word view addresses `0x01000000`, the first byte of the unmapped gap after the system-ROM window. Startup initially clears `reached_after_load`; the attempted load faults before its result can be assigned there.

```
module<entry>

bss reached_after_load: word
local unmapped<const>: *word = 0x01000000

-- This unmapped load enters the BIOS monitor before the assignment completes.
*reached_after_load = *unmapped

while true do
    halt_until_irq
end
```

This load produces a data bus error, displayed as **DBE**. The CPU cause field is `0x0000001c` (code 7); the Memory fault CODE is instead `0x00000001` (unmapped). These are different reports, not two encodings to write to the same register. DBE does not establish a new CP0 bad-address value. Inspect the Memory ADDR word for this bus fault's address.

At the BIOS monitor prompt, type the following command and press Enter. Its address is hexadecimal, and the count selects three consecutive 32-bit words:

```
MEM 08010208 3
```

The displayed result is:

```
08010208: 00000001 01000000 00000801
```

| Displayed word | Meaning for this load |
|---|---|
| `00000001` | Memory CODE: unmapped access |
| `01000000` | Memory ADDR: the invalid byte address |
| `00000801` | Memory ACCESS: read direction and Word access kind |

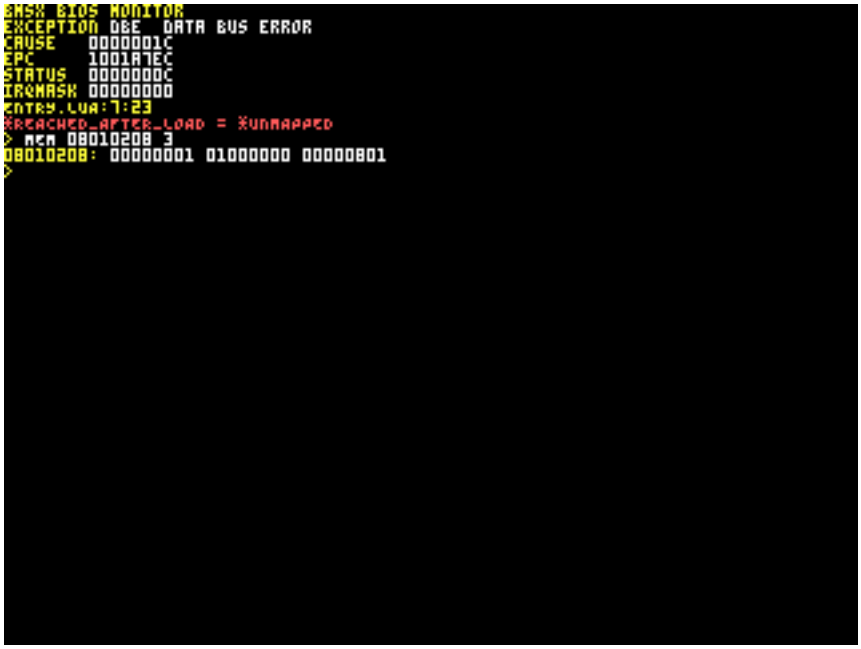



Figure 2.1: BIOS monitor after the unmapped Word load and the three-word MEM command

**Unmapped-load output.** The BIOS reports DBE, the faulting source statement and its EPC; the `MEM` line shows the three Memory report words. EPC is the instruction's address in this program, not a constant to copy into another cartridge.

### 2.13.2 Clearing a report is not repairing the program

The monitor's `FAULT` command displays its saved fault report when one is retained. `FAULT CLEAR` discards that saved BIOS report; it does not write `IO_SYS_BUS_FAULT_ACK`, clear the Memory CODE/ADDR/ACCESS words, or resume the cartridge.

`CONT` returns using the saved EPC without advancing it. In the program above, the invalid address is unchanged, so `CONT` retries the load and enters the monitor again with another DBE. The first Memory fault remains latched. Clearing either report does not turn an unmapped address into usable storage.

CP0 reads and writes require non-user mode; attempting them from ordinary cartridge user mode causes the coprocessor-unusable exception. Do not attempt to repair this example by reading or replacing CP0 fields in ordinary cartridge code. The example demonstrates inspection and an unchanged-instruction retry, not a cartridge recovery handler or a successful recovery.

### 2.13.3 Repair a word load and retry

The preceding constant-address probe remains faulty after its report is cleared. This example instead makes the address a mutable local pointer, so you can repair the operand of the stopped read. Include the BIOS diagnostic directory for source and frame-scope lookup. Frame evaluation needs the correct source scope and an available, mutable live binding; it cannot recover arbitrary optimized-out locals.

```
module<entry>

struct memory_recovery_result
    first: word
    second: word
    fault_before_ack: word
    fault_after_ack: word
    completed: word
end

data source_address: word = 0x01000000
data safe_words: word[2] = { 0x12345678, 0x9abcdef0 }
```

```
bss result: memory_recovery_result

local source: *word = *source_address
local fault_code<const>: *word = 0x08010208
local fault_ack<const>: *word = 0x08010214

result.first = *source
source = source + sizeof(word)
result.second = *source
result.fault_before_ack = *fault_code
*fault_ack = 1
result.fault_after_ack = *fault_code
result.completed = 1
print('RECOVERED')
print(string.format('WORDS %08X %08X', result.first, result.second))
print(string.format('FAULT %08X -> %08X', result.fault_before_ack, result.fault_after_ack))

while true do
    halt_until_irq
end
```

`source` first receives the invalid address stored in `source_address`. The first read faults before assigning `result.first`. `safe_words`, by contrast, names the address of two DATA words in main RAM. Repair the existing `source` binding to that address: changing the `source_address` word alone would leave the stopped read's already loaded pointer unchanged. Do not change EPC; the same read must be retried with the repaired operand.

At the monitor, inspect `FRAMES`. Each physical frame row gives its index, function-record address and PC. Following `DEPTH` rows list the installed source-scope names active at that frame's suspension PC, including inlined functions. Choose the physical index from the preceding frame row and the depth from the matching source-scope row. Missing diagnostic information is reported instead of supplying a guessed scope. A binding must also be live and mutable at that PC to be assigned.

In the supplied sample, the source frame is physical index `00000001`, depth `00000000`, and its PC matches the fault EPC. Both selectors are displayed as eight hexadecimal digits without a prefix; bare command selectors are decimal, so use `0x` when copying them. Other programs or builds need not use this pair. For the supplied sample, the sequence is:

```
FRAMES
LUA --FRAME 0x00000001 0x00000000 source = safe_words
CONT
```

The single `LUA` assignment changes the selected frame's live pointer; it does not redeclare a typed local or change the `safe_words` address binding. It returns to the prompt without resuming execution or acknowledging Memory's report. A success banner is not required.




Figure 2.2: Monitor after inspecting frames and assigning the repaired pointer, before continuing.

**Repair before retry.** The monitor after `FRAMES` and `source = safe_words`, still at the prompt before `CONT`; this is not a completed-recovery screen.

`CONT` retries the saved instruction. The first read now succeeds; the program advances `source` by `sizeof(word)` (four bytes) and reads the second word. The old Memory CODE can remain `1` throughout both successful reads: a pending report does not make a valid address invalid. Only afterward does the program save that old code, write `1` to ACK, and read the cleared code.

The successful run produces these diagnostic lines, not newly drawn text in the monitor image:

```
RECOVERED
WORDS 12345678 9ABCDEF0
FAULT 00000001 -> 00000000
```

`REBOOT` is a restart request, not this repair or a saved-EPC continuation. With the cartridge unchanged, its bad initial pointer is encountered again. See System control and timing — Reset requests.

For a programming example of privileged CP0 reads inside an ordinary cartridge IRQ callback, see Book II, “CPU and BLua32 instruction reference”, under “Read CP0 in a cartridge IRQ”. The callback returns normally, allowing the standard IRQ entry to perform RFE and resume the interrupted cartridge code.

## 2.14 Worked example: prepare and copy two records

This sequence prepares bytes without submitting a device job. Start after normal cartridge DATA/BSS initialization. The declared objects supply the source and destination addresses.

```
module<entry>
struct memory_sample
    tag: u8
    value: word
    samples: u16[2]
end
rodata inputs: memory_sample[2] = {
    { tag = 0x11, value = 0x12345678, samples = { 0x1122, 0x3344 } },
    { tag = 0x22, value = 0x9abcdef0, samples = { 0x5566, 0x7788 } },
}
data counter: word = 0x12345678
```

```
bss output: memory_sample[2]
local source<const>: *memory_sample = inputs
local destination<const>: *memory_sample = output

for index = 0, #inputs - 1 do
    destination[index].tag = source[index].tag
    destination[index].value = source[index].value
    destination[index].samples[0] = source[index].samples[0]
    destination[index].samples[1] = source[index].samples[1]
end
*counter = *counter + #inputs
```

### 2.14.1 Record indexing is not byte arithmetic

| Part of `memory_sample` | Byte offset | Bytes |
|---|---|---|
| `tag` | 0 | 1 |
| Padding | 1 | 3 |
| `value` | 4 | 4 |
| `samples` | 8 | 4 |

Record alignment is 4 and record size is 12 bytes. Each two-record array occupies 24 bytes. Typed indices start at zero: `&source[1]` is 12 bytes after `inputs`, `&source[0].value` is 4 bytes after it, and `&source[1].samples[1]` is 22 bytes after it. Advancing a byte view by one would instead select the next byte, here padding, not the next record.

### 2.14.2 Resulting bytes

BSS supplies initial zero output bytes. The copy writes scalar fields rather than assigning whole records, so their three padding bytes remain zero.

| Output byte offset | Word | Contents |
|---|---|---|
| 0 | `00000011H` | First tag and three zero padding bytes |
| 4 | `12345678H` | First value |
| 8 | `33441122H` | First record’s two samples |
| 12 | `00000022H` | Second tag and three zero padding bytes |
| 16 | `9ABCDEF0H` | Second value |
| 20 | `77885566H` | Second record’s two samples |

The final counter is `1234567AH`: `#inputs` counts **two records**, not bytes. No interrupt, rendering or buffer-release event follows this CPU copy. A hardware-facing use must still satisfy the receiving unit’s address, layout, count, completion and lifetime rules.

## 2.15 DMA buffer lifetime

DMA uses byte addresses but counts transfers in **32-bit words**. Starting a transfer does not copy its source into a private payload buffer. Source storage, destination storage and the channel’s programmed words therefore have different lifetimes. The rules below describe aligned, backed main-RAM copies and the stated DMA-to-GPU example, not every hardware buffer.

### 2.15.1 Attribute a bus fault before START

The shared record contains CODE (`0x08010208`), ADDR (`0x0801020c`) and ACCESS (`0x08010210`); ACCESS identifies direction and access kind, not a channel or bus master. Read them with aligned Word/U32 MMIO accesses from ordinary cartridge code, not privileged CP0 accesses; reads do not acknowledge them.

To associate the report with one transfer, first arrange exclusive reporting for that interval: other DMA work, CPU accesses, interrupt handlers and BIOS service must neither introduce a bus fault nor acknowledge one. The report has no channel or bus-master identifier.

**Before START**, read CODE and preserve any older nonzero report's CODE/ADDR/ACCESS in owned storage. Write aligned Word/U32 `1` to ACK (`0x08010214`), then read CODE to establish a zero baseline. Start the prepared operation and wait for its real completion predicate. Read CODE again; if nonzero, preserve CODE/ADDR/ACCESS before another ACK. Under the stated assumption, this is the first reported fault since clearing and belongs to the assessed operation.

If the old report must remain latched, preserve it in place and treat later transfer faults as unattributed: the report cannot record another fault until cleared. Also check the destination unit's status, since a device can refuse an unready port write without raising a bus fault.

### 2.15.2 Prepare, start, wait, check, reuse

1. **Prepare the storage.** Fill the source and reserve sufficient destination bytes at known program-storage addresses. Use nonoverlapping ranges for an ordinary copy. A count of 20 transfers 20 words, or 80 bytes. Keep the source stable through its last DMA read and wait for the final destination write before consuming the completed buffer.
2. **Program an idle channel completely.** Set its read address, write address, count, increment choices, request selectors and block length before START. An increment advances its address by four bytes per word; a fixed address advances by zero. The programmed block-length field specifies 1 through 16 words, and the last block uses only the remaining count. Both selected requests must be asserted before a block is admitted. `FORCE` is always asserted; `DISABLED` is never asserted. For a device port, choose its real request selectors rather than use FORCE to send words while the port is not ready.
3. **Start once and preserve the live inputs.** An accepted START with a nonzero count sets BUSY; START while BUSY is ignored, not queued as a second transfer. A zero count finishes immediately, before testing requests. Admission retains one block's addresses, remaining count, control and length, but not its source words. At block service, each source word is read and immediately written to the destination before proceeding to the next word. Bytes changed after START can therefore affect later reads; START is not a source-buffer snapshot.

   Leave the channel's programmed address/count/control words alone through completion. Writes to those words are not automatically rejected while BUSY: an already admitted block uses its captured configuration, and its later address/count writeback can overwrite changes made meanwhile. Reprogramming a busy channel is not a way to revise that admitted block safely.
4. **Wait for completion, then check the outcome.** Finish follows the final mapped transfer accesses, replaces channel status with DONE (`2`), and clears BUSY by that replacement. It also raises the channel's DMA interrupt. Wait for channel status or service its completion IRQ rather than assuming a transfer delay.

   Check the fault report using the procedure above. DMA advances after a faulting access: an unmapped source can supply zero, and an unmapped or read-only destination can reject the write. An unready I/O write can also be discarded without retry. Use the correct device requests and verify the destination's applicable completion/status condition.
5. **Reuse according to the completed operation.** Once DMA has finished its source reads, that transfer no longer needs the source bytes; once its final RAM writes have completed, the RAM destination can be inspected as the completed transfer's output. A receiving device may still have work to do. DMA transport completion does not establish GPU-job completion or permission to reuse a live display surface.

### 2.15.3 Overlap is not a buffered move

For backed RAM, reads observe the current four little-endian bytes and writes immediately update their destination bytes. There is no block-sized source snapshot or special overlap-copy behavior. For example, start with RAM words `[1, 2, 3, 4, 5]` and copy a four-word block from *base* to *base* + *4*, with both addresses incrementing. The access order produces `[1, 1, 1, 1, 1]`: each write changes the next source word before it is read.

### 2.15.4 DMA programming example

The DMA chapter's BLua sequence prepares 20 source words and performs two nonoverlapping RAM copies using declared storage addresses. It programs both channels before either START and uses 16-word and four-word blocks. The two completed destination copies retain the words to inspect.

The following GP0 sequence sends a five-word ROM stream, waits for DMA DONE, then separately waits for a GPU marker and GPU idle. These are distinct completion conditions; the order of software reads does not establish a universal ordering of DMA and device completion.

## 2.16 Device-specific buffer lifetimes

### 2.16.1 GEO addressed-buffer lifetime

GEO captures a job's configuration, not its source data. Prepare the buffers before launch, keep them stable during work, and check the terminal state before consuming output or reusing storage. Follow the GEO launch procedure for operation-specific programming and observation.

1. **Prepare the job.** Wait for a nonbusy unit, then populate the descriptors and source records and reserve separate output buffers. XFORM2 vertex and AABB destinations must occupy writable RAM outside reserved storage. Program SRC0, SRC1, SRC2, DST0, DST1, COUNT, PARAM0, PARAM1 and all three strides before writing CMD.
2. **Know what launch captures.** An accepted, supported nonzero command captures those register words, without reading source bytes. With nonzero `COUNT`, GEO becomes `BUSY` and schedules service. `OVERLAP2D_PASS` writes its summary at launch, before the count test; even a zero-count launch is not necessarily free of buffer writes. Never submit `CMD` while `BUSY`: it neither waits nor queues. It discards the running job, rejects the new command, resets `PROCESSED` to zero, publishes `REJECTED` with `REJECT_BUSY`, and raises the ERROR interrupt.
3. **Preserve live inputs.** `XFORM2_BATCH` reads each record during processing. A valid empty record with `flags = 0` and `vertexCount = 0` succeeds without reading a matrix or vertices. For a nonempty record that passes its checks, GEO reads the selected matrix during that record and each local vertex immediately before its corresponding output x/y writes. `SAT2_BATCH` reads pair, descriptor, and vertex words during processing; `OVERLAP2D_PASS` likewise reads candidate, instance, shape, and vertex data during processing. Changing these bytes after `CMD` can change pending work. Leave inputs and outputs unmodified until work stops; separate outputs do not establish arbitrary in-place or aliasing safety.
4. **Check the actual terminal state.** `PROCESSED` advances after successful records. Successful completion latches `DONE` without a fault and raises the DONE interrupt. An execution fault stops the job, retains the completed-record count, latches `DONE|ERROR` and `FAULT`, and raises ERROR. `DONE` alone does not prove whole-batch success: inspect `ERROR`, `FAULT`, and `PROCESSED`.
5. **Classify output before reuse.** Faults do not roll back issued writes. Completed-record counts do not describe every retained partial write or launch-time summary. `ERROR` or `REJECTED` suppresses later commands until local fault acknowledgement. Any nonzero `FAULT_ACK` clears `ERROR`, `REJECTED`, and `FAULT`, preserving `DONE`, `BUSY`, and `PROCESSED`; it does not resume discarded work. This local acknowledgement is distinct from interrupt acknowledgement. Use the completion/fault procedure before consuming results or reusing buffers.

### 2.16.2 GTE PLUS operand-register lifetime

GTE PLUS uses an operand/result register bank, not a main-RAM source-address and length interface. Its five writable operand words are `ADD_XY`, `ADD_Z`, `MUL_XY`, `MUL_Z`, and `SCALAR`. This differs from an addressed RAM payload whose contents may be read as work progresses.

An accepted, supported `VMAD3` command computes pending XY, Z, and flag results from the current operand words. Later writes to those five operand registers replace their register contents, but do not change that pending result. They can prepare operands for subsequent work; they do not submit another command. Until completion, result reads still see the previously published values. The pending XY, Z, and flag results publish together at completion, when `BUSY` clears.

Submit `COMMAND` through the CPU. CPU command submission is interlocked while a command remains pending, rather than starting overlapping work. DMA cannot issue PLUS `COMMAND`. Follow GTE timed VMAD3 for the command sequence and completion procedure; do not substitute a guessed delay for completion observation.

### 2.16.3 IMGDEC input and output have separate lifetimes

IMGDEC accepts supplied stream words through DATA, not a cartridge source-buffer pointer. An admitted CPU DATA write copies that word into its input storage; the decoder does not retain the original CPU-visible storage location. Texture and CLUT destination words become output command headers, not addresses that IMGDEC itself dereferences.

DMA programming and block admission capture addresses, not the source contents. Keep the source words unchanged until their last DMA read, including repeated reads from a fixed source address. For a whole buffer with no other reader, input DMA completion ends that source transfer's need for the buffer: reuse need not wait for decoder completion or output drainage. This does not certify successful decoding; misprogrammed input writes can be counted without entering the FIFO.

Output is another word stream. An output DMA count must include the **three-word texture header** and, when CLUT data is present, its **three-word CLUT header**, as well as both payloads. Input words received count submissions, not consumption; decoded words count payload, not headers. Neither progress count is completion. Require normal IMGDEC DONE, not merely an IRQ: faults also raise that interrupt. When output DMA captures into RAM, also require that DMA transfer's completion before treating the destination as completed output; the final source DATA read precedes its destination write. Neither completion establishes downstream GPU execution or visible display completion.

Do not poll DATA for freshness. An empty read returns the retained DATA latch, which input writes also change. A CPU read while DMA owns the DATA read port likewise returns that latch without popping output. OUTPUT_REQUEST describes a block threshold, not permission for an arbitrary number of reads. Extra DATA reads after successful termination are not harmless polling: they can raise the completion interrupt again.

A format fault stops processing but can leave queued partial output and words already delivered. CONTROL has no ABORT operation. A new START or IMGDEC reset discards queued internal output; it does not cancel a separately admitted DMA transfer or retract prior output. Restarting IMGDEC is therefore not an all-or-nothing recovery of an arbitrary partial output stream.

Use the IMGDEC chapter and its BLua programming sequence for stream layout and transfer programming.

### 2.16.4 APU sample storage and lifetime

Sampled voices read installed ROM or the APU's 512 KiB private sample RAM, not CPU main RAM. Main RAM can hold upload staging or a download destination, but it is not a direct voice source. The private source address `0x40000000` names APU storage; it is not a CPU pointer. The transfer address instead names an offset within that private RAM.

For a supported nonzero command, an accepted CMD write captures the staging parameters into the command queue, not the sample bytes. Later staging writes do not change that queued entry. PLAY binds its source when the command executes; cartridge selection is taken at that binding, not when CMD is queued. The voice retains a source view and reads backing bytes during playback rather than taking a complete payload snapshot. Preserve those bytes for every remaining playback, loop, seek, or queued use. Do not expect live-byte changes to be immediately reread. Selected-slot source-window writes are individually live: each can rebind an active voice using its current full description, not an atomic staged replacement. A failed rebind can stop that voice and fault without releasing other source references.

Keep upload completion separate from playback lifetime. A DMA upload first delivers words to the transfer FIFO; private-RAM writes can remain pending after DMA DONE. After the intended DMA upload completes, with no competing transfer writer, read APU STATUS and wait for TRANSFER_BUSY, bit 10 (`0x400`), to clear before starting a voice over the uploaded bytes. Keep the operation's DMA and fault checks: a clear bit alone does not prove correct data. CPU DATA writes in MANUAL_WRITE commit immediately. Do not change the transfer address while an upload commit is pending. A download may continue prefetching after its DMA completes; select transfer STOP to cancel that work and clear its FIFO before reprogramming. This does not undo committed writes or stop voices.

A paused voice retains its source, and a looping voice continues reading it. STOP enters the command queue. A fading STOP keeps the source in use until natural source end or fade completion, whichever comes first. An executed no-fade STOP clears the resident voice without SLOT_ENDED.

Release sample bytes only after every voice and queued PLAY using them has finished. SLOT_ENDED identifies one ended slot; another slot can still read the same source. Upload DMA completion releases the upload buffer, while playback has its separate source lifetime.

Fault acknowledgment clears the fault report, not the voices or command and transfer queues. Reset is destructive: it clears playback, queues, and private sample RAM rather than completing or preserving their contents. See the APU chapter for source binding and playback, and its complete raw-port download for transfer completion and surplus-prefetch control.

## 2.17 Shared GPU storage and display publication

Treat RAM staging, GP0 command words, shared GPU local memory, and the PCRTC display selection as different lifetimes. Uploading a picture does not allocate a texture, and selecting a framebuffer does not copy its pixels. Textures, palettes, drawing destinations, copy sources, and display sources can address the same storage. The cartridge must choose regions and order their uses; neither GP0 port readiness nor a VBlank event locks or releases those regions.

| Data or role | What is retained or accessed | Relevant boundary |
|---|---|---|
| RAM staging source | DMA reads RAM words as the transfer progresses; GP0 receives copied words. | Completion of the RAM reads ends that source use, not later GPU or display uses. |
| Upload payload | The upload retains payload words consumed by image assembly. | The upload uses those words, not a later reread of the original RAM source. |
| Texture, CLUT, copy source | Commands read shared VRAM when their ordered memory operation executes. | Queueing a command does not preserve a private copy of these pixels. |
| Drawing destination | Writes update shared VRAM; masking and blending can read its current contents. | Destination addresses do not establish exclusive ownership. |
| PCRTC source | Retained display configuration selects addresses in shared VRAM. | Configuration publication is not a selected-surface pixel snapshot. |
| Readback result | READY readback holds copied result pixels. | GP0 reads drain that result; the final read returns the readback port to IDLE. |

### 2.17.1 Command order is not resource allocation

A queued drawing command retains its packet and drawing-context words, not texture or CLUT bytes. A VRAM-to-VRAM copy likewise reads its source from shared memory when executed. Preserve the contents required by earlier uses in the ordered command stream. A later, correctly ordered upload can change the contents for later uses; queue-time capture does not give earlier commands their own texture image. This ordering is not a promise that arbitrary source/destination aliasing or texture feedback is safe.

DMA completion, a GP0 interrupt marker, GPU IDLE, and readback readiness answer different questions. DMA DONE follows the channel's mapped accesses; it does not itself establish successful GPU delivery, completed drawing, or display publication. The GP0 interrupt marker raises its retained request when reached in command order, but is not a display fence. GPU IDLE requires empty input, no incomplete command or upload, no pending command timing, and an idle readback port. It does not certify a particular frame's pixels or display publication.

Readback becomes active at its ordered command boundary and blocks further GP0 parsing until drained. READY means its pixels have been copied into readback storage, not that a DMA destination in RAM has already been filled. Draining GP0 and finishing any downstream DMA RAM writes are separate steps. See GPU and DMA for their command and transfer interfaces.

### 2.17.2 A display switch selects storage; it does not save it

PCRTC has programmed and retained presentation configuration, not two private pixel banks. Ordinary display configuration writes stage new words. At VBlank begin, presentation configuration is latched and the frame's command boundary is selected before the system VBlank interrupt is raised. The separate PCRTC VSINT interrupt is not this publication acknowledgement. Beam-timing writes can restart live timing immediately; do not treat them as ordinary deferred display-source switches.

Before composing the selected source, commands through the published boundary are processed. Normally that boundary is the executed-command count; an active readback instead supplies its fence boundary. This is not an exact historical pixel version: later commands already applied to shared memory are not undone. Neither the boundary nor VBlank supplies an all-VRAM release, an atomic framebuffer snapshot, or a physical pixel-fetch deadline.

With a fixed format, pitch, geometry, and source offsets, switching to a disjoint region changes the selected addresses. It does not prove that every other reader has stopped using the old region. If the old region is later selected again, it supplies its then-current contents, not pixels saved by the earlier switch. See PCRTC for source-address selection.

A fresh-after-write observation needs more than an already-pending VBlank flag. Under a sole cartridge IRQ/counter owner, no earlier in-flight callback, and no other configuration, mask, or counter writer: set IRQ MASK to 0 before the final display

write; after that write, write ACK = 4 and capture the VBlank callback counter while still masked; set MASK to 4 and wait for the counter to change. The accepted event then follows the final write. This does not make multiple register writes atomic or free any resource. Reset and context operations are additional configuration-publication boundaries, not ordinary VBlank switches.

### 2.17.3 Prepare, publish and reuse two surfaces

This programming sequence prepares two fixed surfaces, selects A, selects B, rebuilds A while B is selected, and selects A again. It uses ordinary fresh BIOS cartridge entry with CPU interrupts enabled. It is the sole GP0 producer and owns the VBlank handler, IRQ mask and callback counter; there is no earlier in-flight callback, supervisor/context takeover, or competing writer. Only circuit 1 is enabled, and no other texture, CLUT, copy or readback operation uses either region. These are the conditions of this example, not an allocation or general surface-release service.

The callable BIOS function `gpu/system_vram_region` reports the installed system reservation. For the reservation example above it returns origin `02D002C0H`, or `(704,720)`, and size `00FC0140H`, or `(320,252)`: the bounding region `[704,1024) × [720,972)`. Both chosen surfaces exclude that reservation and each other. These are the current installed BIOS values, not immutable hardware dimensions or proof that every other address is free.

| Surface | GX16 source region | DISPFB1.low | Word base |
|---|---|---|---|
| A | `[0,256) × [0,192)` | `000FA000H` | `00000H` |
| B | `[0,256) × [256,448)` | `000FA040H` | `40000H` |

`DISPFB1.high` stays zero. Both low words select GX16, PSM 31, and FBW 16: a row stride of 1024 sixteen-bit words, or 2048 bytes. Each surface contains 49,152 source words, or 98,304 bytes of pixels, across those rows; it is not one contiguous CPU allocation. Changing FBP from `00H` to `40H` changes the word base from zero to `40000H`, equivalent to `(0,256)` in this layout.

The program resets PCRTC with CSR `00000200H`; that is not a VRAM clear. It holds PMODE at zero during preparation. `DISPLAY1.low = 01800000H` supplies zero signal origin and magnification 4 horizontally, 1 vertically; `DISPLAY1.high = 000BF3FFH` supplies a 1024 × 192 signal extent. With the reset horizontal signal step of 4 and progressive mode, the source footprint is exactly 256 × 192 at zero phase. PMODE `0000FF21H` then enables only circuit 1. Pitch, format, source offsets and geometry remain unchanged throughout the switches.

#### 2.17.3.1 Prepare the storage and display route

The following fragments form one ordered sequence under the ownership assumptions above. A final write to a display register is not a drawing fence. First define the VBlank callback and disable delivery during preparation:

```
bss vblank_sequence: word
function irq(flags)
    if (flags & 4) ~= 0 then
        *vblank_sequence = *vblank_sequence + 1
    end
    mem[0x08000004] = flags & 4
end
mem[0x08000008] = 0
mem[0x080103d4] = 0x00000200  -- PCRTC reset; not a VRAM clear
mem[0x08010354] = 0           -- PMODE: no read circuit
mem[0x0801035c] = 0x000fa000  -- DISPFB1 low: surface A
mem[0x08010360] = 0           -- DISPFB1 high: zero source offsets
mem[0x08010364] = 0x01800000  -- DISPLAY1 low
mem[0x08010368] = 0x000bf3ff  -- DISPLAY1 high
mem[0x08010240] = 0x09000000  -- Close the VRAM Y-address extension gate
```

Send the following **four three-word GP0 fill commands**, in row order, to the fixed port `0801023CH`. Each cell is one raw word:

| Fill | Command/color | Position | Extent |
|---|---|---|---|
| Red A | `020000FFH` | `00000000H` | `00C00100H` |
| White marker on A | `02FFFFFFH` | `00400010H` | `00200020H` |
| Blue B | `02FF0000H` | `01000000H` | `00C00100H` |
| White marker on B | `02FFFFFFH` | `014000D0H` | `00200020H` |

For example, the first command is exactly:

```
mem[0x0801023c] = 0x020000ff
mem[0x0801023c] = 0x00000000
mem[0x0801023c] = 0x00c00100
```

After all four fills, issue a one-pixel readback from each surface origin: GP0 words `C0000000H`, position (`00000000H` for A, `01000000H` for B), `00010001H`. For each request, wait until GPUSTAT at `08010240H` has `08000000H` set, then read one word from GPUREAD at `0801023CH`. The words are `0000001FH` and `00007C00H`. Drain each result before the next request; then wait separately for GPUSTAT IDLE, bit `04000000H`.

#### 2.17.3.2 Publish A, then B

Enable circuit 1, discard stale VBlank pending **after** the final display write, and wait for a newly serviced callback:

```
mem[0x08010354] = 0x0000ff21
mem[0x08000004] = 4
local before = *vblank_sequence
mem[0x08000008] = 4
while *vblank_sequence == before do
    halt_until_irq
end
```

A is now selected at the new publication edge. To publish B, use the same mask/write/ACK/sample/enable/wait order:

```
mem[0x08000008] = 0
mem[0x0801035c] = 0x000fa040
mem[0x08000004] = 4
local before = *vblank_sequence
mem[0x08000008] = 4
while *vblank_sequence == before do
    halt_until_irq
end
```

With B selected, rebuild only A using GP0 triples `0200FF00H, 00000000H, 00C00100H` and `02FFFFFFH, 00500070H, 00200020H`. Read back both origins again and drain the results before waiting for GPU IDLE. Repeat the publication sequence with `DISPFB1.low = 000FA000H` to select A's new contents.

#### 2.17.3.3 What each phase establishes

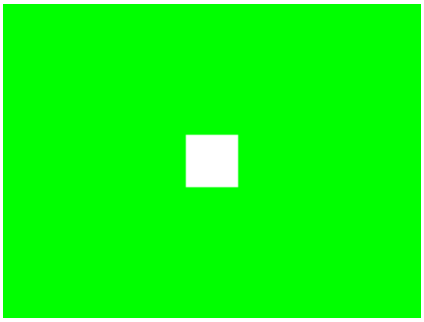
GP0 readback follows the fills in command order. A one-pixel request consumes one whole GPUREAD word: the pixel is in the low half and the unused high half is zero. Waiting for IDLE after draining the readbacks is separate from waiting for VBlank publication. There is no DMA transfer in this sequence.

Masking delivery before the display write and acknowledging afterward prevents an old pending VBlank from satisfying the callback-counter wait. Under the stated sole-owner conditions, the changed counter observes a fresh post-write VBlank. It does not measure exact pixel-fetch timing or a fixed latency.

| Origin sampled | Initial preparation | After rebuilding A |
|---|---|---|
| A `(0,0)` | `0000001FH` (red) | `000003E0H` (green) |
| B `(0,256)` | `00007C00H` (blue) | `00007C00H` (unchanged blue) |

The framebuffer words select A/B/A: `000FA000H`, `000FA040H`, `000FA000H`. The final selection reads A's **current green contents**, not a saved copy of its earlier red image. It does not release other users of that storage or grant permission to overwrite the BIOS reservation.

#### 2.17.3.4 Ordinary playback pictures

| A prepared and selected | B selected | A rebuilt and selected again |
|---|---|---|
|  |  |  |
| Red background; white marker `[16,48) × [64,96)`. | Blue background; white marker `[208,240) × [64,96)`. | Green background; white marker `[112,144) × [80,112)`. |

The resulting 256 × 192 pictures are shown above. Each contains 1024 white marker pixels and 48,128 background pixels. They demonstrate this controlled selection and reuse, not physical pixel-fetch timing, arbitrary alias safety, or ownership of other regions.

# 3 PCRTC Display Controller

## 3.1 Purpose and data flow

The PCRTC turns images in GPU local memory into the displayed picture. The GPU writes the stored pixels; PCRTC selects which pixels to read and how they appear in the output. Two reading circuits can select different portions of the same memory, each with its own source origin, display window and magnification. Their colors are combined over a programmable background.

Use one circuit to display a completed framebuffer. Use both when two images need independent source selections or window positions in the same output. The source is still GPU local memory: changing a reading circuit's settings selects stored pixels rather than drawing or moving them.

Each circuit has two groups of controls. **DISPFB** describes its stored image and source origin. **DISPLAY** describes its output window and magnification. **PMODE** enables the circuits and controls their composition. Separate timing registers establish the beam sequence and its display-event interrupts.

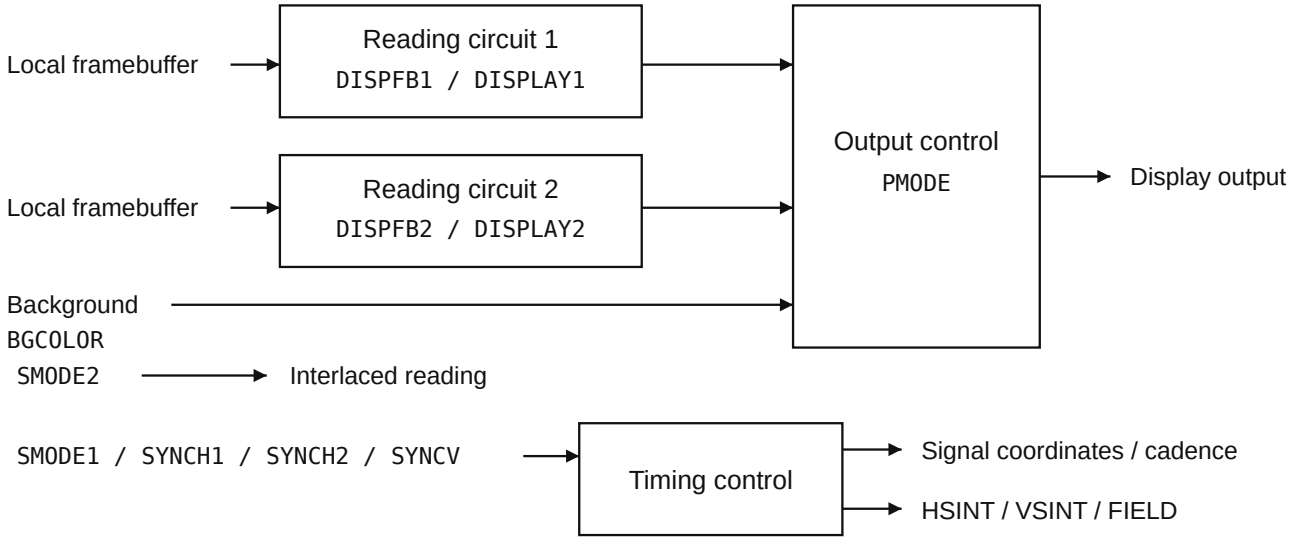



Figure 3.1: PCRTC display data path and controls

## 3.2 Selecting and displaying an image

### 3.2.1 Framebuffer reading and display windows

A source origin answers **which stored pixels to read**. A display window answers **where and over what extent a circuit contributes to the output**. Magnification connects the source pixels to the window's signal coordinates; the horizontal signal step then determines which signal positions become output pixels. Keep these quantities separate when choosing a picture size.

#### 3.2.1.1 Choose the stored source

Consider a 256 × 192-pixel view of a GX16 image. Its framebuffer base is local-memory word address zero, its row width is 1024 pixels, and the desired source origin is `(32,16)`. The selected source rectangle is therefore `[32,288) × [16,208)`: its last source pixel is `(287,207)`.

The framebuffer row width describes the stored image, not the displayed window. In this GX16 example it advances 1024 sixteen-bit words between successive source rows, even though only 256 pixels from each row will be shown. GX16 uses one sixteen-bit word per pixel. Other storage selections have the packing and address layouts specified under Section 3.2.5 and Section 3.2.6.

DISPFB supplies the base field FBP, the width field FBW, the storage selection, and source X/Y. Its base and width units are:

```
Framebuffer base B = FBP × 4096 local-memory words
Framebuffer width W = FBW × 64 pixels
Source origin      = (source X, source Y)
```

A local-memory word is **16 bits, or two bytes**; a PCRTC register word is **32 bits, or four bytes**. One FBP unit therefore covers 8192 bytes. For the chosen source, use FBP=0, FBW=16 and storage selector 31 (GX16).

#### 3.2.1.2 Relate source pixels to the output window

Keep the reset noninterlaced timing and choose signal origin `(680,37)`, X magnification 4 and Y magnification 1. X magnification 4 means that one source pixel occupies four horizontal **signal units**. It does not by itself mean four output pixels. The horizontal signal step is also 4, meaning that output pixels are selected four signal units apart. These equal steps give one source pixel per output pixel. Y magnification 1 in noninterlaced output similarly selects the next source row for the next output row.

To show 256 columns, the horizontal signal span must be `256 × 4 = 1024`. The window runs from signal X 680 up to, but excluding, 1704. Dividing these aligned boundaries by the signal step gives 170 and 426; their difference is 256 output pixels. The vertical window has 192 rows starting at signal Y 37.

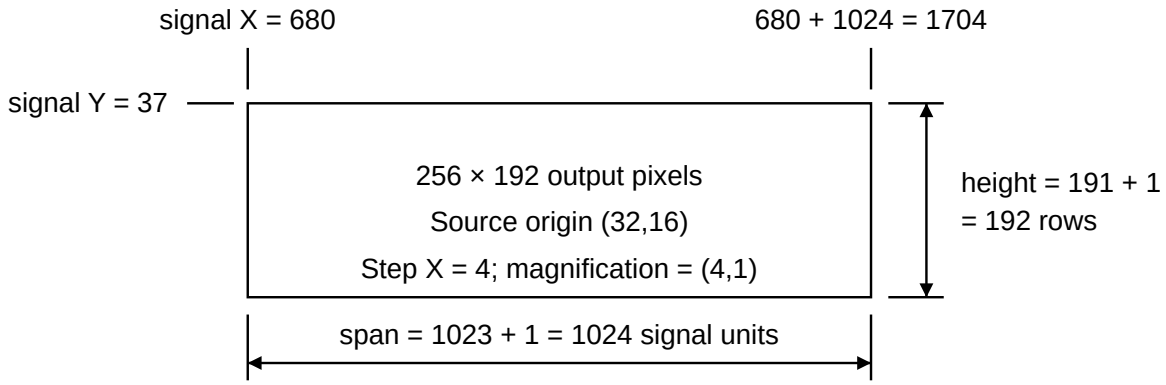



Figure 3.2: The worked window: source origin (32,16), signal origin (680,37), a 1024-signal-unit horizontal span, and 256 × 192 output pixels with signal step 4.

With only this circuit enabled, its window defines the cropped output, so output `(0,0)` reads source `(32,16)` and output `(255,191)` reads source `(287,207)`. With both circuits enabled, the output origin uses their minimum signal X and Y and the bounds cover both windows. Window position then places the circuits relative to one another. Moving a lone enabled window also moves the crop; Section 3.5 uses a second circuit as a fixed canvas to demonstrate a visible placement change.

#### 3.2.1.3 Encode the source and window

Both magnification fields and both extent fields use encoded-plus-one values:

```
X magnification = encoded X magnification + 1
Y magnification = encoded Y magnification + 1
Horizontal span = horizontal extent + 1 signal units
Vertical height = vertical extent + 1
```

Thus this example stores encoded X magnification 3, encoded Y magnification 0, horizontal extent 1023 and vertical extent 191. The quantities just derived produce these circuit-1 words:

| Word | Field values | Encoded word |
|---|---|---|
| DISPFB1_LOW | FBP=0; FBW=16; storage=31 (GX16) | `000FA000H` |
| DISPFB1_HIGH | Source X=32; source Y=16 | `00008020H` |
| DISPLAY1_LOW | Signal X=680; signal Y=37; X magnification encoded 3; Y encoded 0 | `018252A8H` |
| DISPLAY1_HIGH | Horizontal extent=1023; vertical extent=191 | `000BF3FFH` |

For other windows, round each horizontal boundary upward to a signal-step boundary before converting it to output pixels. `SMODE1_LOW[24:21]` sets this step. Output width comes from the converted window boundaries, not from dividing its span by X magnification. The complete sampling equation and a case with unequal steps are in Section 3.6.

Pixel output requires running **latched** timing, a nonzero latched signal step, and at least one latched circuit enable. Configuration writes first change the programmed words; the display latch selects them at VBlank entry, as explained after the following register sequence.

### 3.2.2 Set a 256 × 192 window on circuit 1

**Starting condition:** PCRTC reset configuration and reset timing. Both reading circuits are disabled. Circuit 1's framebuffer base is word address 0, its width is 1024, and its format selection is GX16. Its reset signal position is (680, 37), with X magnification 4 and Y magnification 1.

1. Set the source origin to (32, 16) in `DISPFB1_HIGH`.
2. Set the display extents for a 256 × 192 output window. The horizontal span is `256 × 4 = 1024` signal units, so write horizontal extent 1023 and vertical extent 191.
3. Enable circuit 1 last with the PMODE word shown below. Leave circuit 2 disabled.

The table gives raw address/value word writes, in order.

| Order | Address | Register | Write value | Setting |
|---|---|---|---|---|
| 1 | `08010360H` | `DISPFB1_HIGH` | `00008020H` | Source X = 32; source Y = 16 |
| 2 | `08010368H` | `DISPLAY1_HIGH` | `000BF3FFH` | Horizontal extent = 1023; vertical extent = 191 |
| 3 | `08010354H` | `PMODE_LOW` | `0000FF21H` | EN1 = 1; EN2 = 0; MMOD = 1; AMOD = 0; SLBG = 0; ALP = 255 |

All three writes change their reset values. None restarts the beam. Other framebuffer, signal-position, magnification, and timing words remain at their reset values.

After the display latch takes these settings, the output is the 256 × 192 view derived above, with output origin `(0,0)` and source origin `(32,16)`.

The PMODE word selects circuit-1 RGB directly. Output alpha is the original GX16 sample alpha, 0 or 128 from bit 15; it is not forced to 255 by ALP = 255.

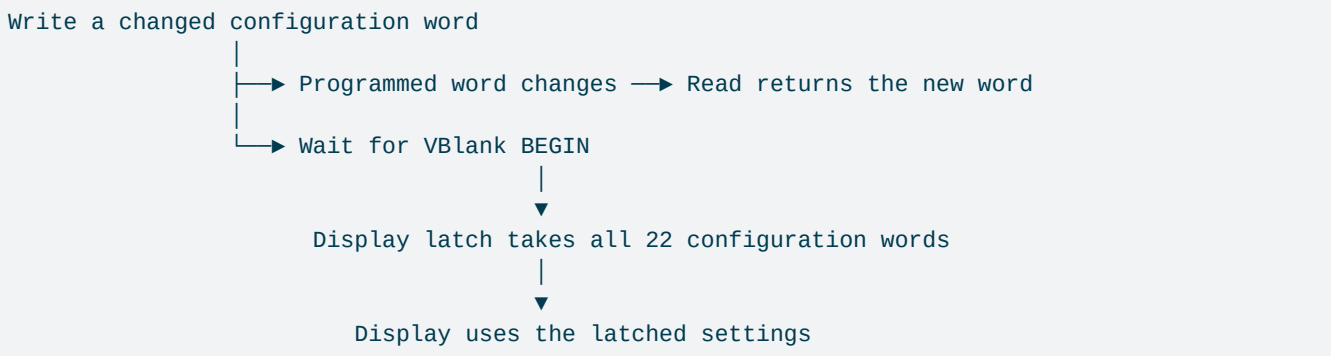
Read back the three programmed words to check the settings. The display uses them at VBlank entry when the latch takes the new configuration, as shown in Section 3.2.3.

For a complete two-surface cartridge and its observed output, see Prepare, publish and reuse two surfaces.

### 3.2.3 Programmed words and displayed settings

Finishing an image and selecting it for display are separate operations. Finish GPU drawing before publishing the completed image's source and window settings. GPU completion establishes that the drawing work has finished; it does not select the PCRTC framebuffer. Likewise, a display-selection change does not copy the pixels into private PCRTC storage. Preserve the source image while it is still in use; Memory — A display switch selects storage; it does not save it explains the storage lifetime when changing surfaces.

Configuration registers retain complete written words. Readback returns the programmed value immediately, but the new words are not yet the displayed selection. Display uses a separate configuration latch:




The display latch takes all 22 configuration words together on **VBlank entry**, at the end of the active-display interval. This precedes the system VBlank interrupt request and operates independently of the PCRTC event masks.

Complete the configuration before its intended VBlank entry. An edge between separate store instructions takes the values written up to that point, including an intermediate configuration. To establish that new settings have been displayed, observe a fresh VBlank entry after their final write, not merely an already-pending request. The display-switch sequence gives this observation with unchanged timing. Use Section 3.3.3 when replacing the global timing configuration itself.

Writing an unchanged configuration word has no effect. A changed write to either `SMODE1` word or any `SYNCH1`, `SYNCH2`, or `SYNCV` word also recalculates timing and **restarts the beam immediately**. The display latch does not defer that restart. Neither `SMODE2` word restarts the beam. Live beam timing is distinct from latched display timing: stopping the beam does not itself blank an already-latched display. PCRTC reset is the exception to waiting for VBlank entry: it installs the reset display settings immediately.

#### 3.2.3.1 Firmware use of the two circuits

The standard BIOS convention assigns cartridge display to circuit 1. During supervisor service, the cartridge's displayed circuit-1 framebuffer and window become circuit 2's underlay, while firmware programs circuit 1 for its terminal. The underlay continues to read the cartridge's stored image. Preserve the firmware VRAM reservation.

Both circuits remain directly programmable. A program using circuit 2 must allow for supervisor entry displaying only the cartridge's circuit-1 view as its underlay. The complete twelve-word composition banks, including circuit 2, are retained separately for programmed and displayed state and restored on leave. Timing words, CSR, IMR and the physical beam are global: supervisor changes to them persist. See Device boundaries during supervisor service for entry, leave and shared-memory lifetime.

### 3.2.4 Interlaced reading

Set `SMODE2_LOW.INT` to select interlaced traversal. Interlaced output advances by two rows; non-interlaced output advances by one. `FFMD` selects frame mode only while `INT` is set.

For interlaced, non-frame operation:

```
Source row stride = 2
Source row phase  = (display signal Y XOR FIELD) AND 1
```

In other modes, the source row stride is one and its phase is zero. Read `CSR_LOW.FIELD` for the current field value. VSync updates this value as described in Section 3.2.9.

#### 3.2.4.1 Output rows and source rows

The following mapping applies independently to either enabled reading circuit. Use its latched display configuration (Section 3.2.3), and the current field value `F`. Define:

```
S = circuit's display signal Y
D = S - minimum enabled display signal Y
H = encoded vertical extent + 1
M = encoded Y magnification + 1
B = framebuffer source Y
n = output Y - D
```

Output Y counts from the top of the combined output, not from signal Y zero. The circuit's window contains local rows `0 <= n < H`. Non-interlaced traversal selects every such row. Interlaced traversal selects only rows whose **output Y** parity equals `F`:

```
First selected local row = (D XOR F) AND 1
Following local rows     = first selected local row + 2, + 4, ...
```

Stop before `n` reaches or exceeds `H`, the window's exclusive bottom. If the first selected local row is outside that window, the circuit supplies no row for that field. For a selected row, calculate the framebuffer coordinate below. `div` means nonnegative integer division, discarding the remainder.

| Traversal | Framebuffer source Y |
|---|---|
| Non-interlaced (`INT=0`) | `B + (n div M)` |
| Interlaced, non-frame (`INT=1`, `FFMD=0`) | `B + 2 * (n div (2*M)) + ((S XOR F) AND 1)` |
| Interlaced, frame (`INT=1`, `FFMD=1`) | `B + (n div (2*M))` |

Divide before applying the source row stride; add the source row phase last. Output-row selection uses the cropped top `D`, but the non-frame source phase uses the original signal origin `S`. They are not interchangeable. Pass the resulting source coordinate to the storage-address calculation in Section 3.2.6; it is not a byte address. These equations select this circuit's samples, not the final result of the two-circuit composition in Section 3.2.7.

#### 3.2.4.2 Worked row selection

Let this circuit start at signal Y 7, with source Y 20, Y magnification 2 and height 8. A second enabled circuit starts at signal Y 5. Thus `D=2`, and this circuit occupies output rows 2 through 9. All numbers below are decimal; each source row corresponds to the output row in the same position.

| Traversal | FIELD | Selected output rows | Framebuffer source rows |
|---|---|---|---|
| Non-interlaced | — | 2, 3, 4, 5, 6, 7, 8, 9 | 20, 20, 21, 21, 22, 22, 23, 23 |
| Interlaced, non-frame | 0 | 2, 4, 6, 8 | 21, 21, 23, 23 |
| Interlaced, non-frame | 1 | 3, 5, 7, 9 | 20, 20, 22, 22 |
| Interlaced, frame | 0 | 2, 4, 6, 8 | 20, 20, 21, 21 |
| Interlaced, frame | 1 | 3, 5, 7, 9 | 20, 20, 21, 21 |

Here the odd signal origin selects source phase one when `FIELD=0`, even though the output window starts on an even row. `INT` alone does not make FIELD alternate; the VSync rule in Section 3.2.9 still applies. FIELD is live state, not one of the 22 latched configuration words; a later read reports its value at that read.

### 3.2.5 Local-memory samples

Local memory holds `00200000H` bytes, or `00100000H` sixteen-bit words. Within each word, the low byte comes first. The storage selector determines both pixel packing and the address layout in Section 3.2.6.

In the table, `L` is the word at the mapped pixel address and `H` is the following word. RGB and alpha samples are eight-bit values.

| Selector | Format | Circuits | RGB packing | Sample alpha |
|---|---|---|---|---|
| 0 | CT32 | 1, 2 | R = L[7:0]; G = L[15:8]; B = H[7:0] | H[15:8] |
| 1 | CT24 | 1, 2 | Same RGB placement as CT32; H[15:8] is ignored | 128 |
| 2 | CT16 | 1, 2 | R = L[4:0]; G = L[9:5]; B = L[14:10]; expand each component | 128 if L[15] = 1; otherwise 0 |
| 10 | CT16S | 1, 2 | Same packing as CT16; different address layout | 128 if L[15] = 1; otherwise 0 |
| 18 | GPU24 | 1 only | Three packed bytes per pixel; see below | 128 |
| 31 | GX16 | 1, 2 | Same packing as CT16; linear address layout | 128 if L[15] = 1; otherwise 0 |

Expand each five-bit RGB component `c` as follows:

```
Eight-bit component = (c << 3) | (c >> 2)
```

GPU24 obtains two words through the CT16 address layout. For framebuffer coordinate `(x, y)`, fetch `L` at `((3 × x) >> 1, y)` and `H` at `(((3 × x) >> 1) + 1, y)`:

| Pixel X | R | G | B |
|---|---|---|---|
| Even | L[7:0] | L[15:8] | H[7:0] |
| Odd | L[15:8] | H[7:0] | H[15:8] |

Every unlisted selector, and selector 18 on circuit 2, supplies **zero RGBA samples**. It neither disables the circuit nor raises a format fault. The circuit still performs its selected PMODE operation inside its window. For example, copying zero RGB blacks that area, whereas blending with zero source alpha preserves the prior RGB. Alpha selection still applies as specified in Section 3.2.7.

### 3.2.6 Local-memory address layout

The following formulas map framebuffer sample coordinates `(x, y)`, including the programmed source origin, to local-memory word addresses. They do not produce CPU byte addresses. All arithmetic is integer arithmetic; `&`, `|`, `<<`, and `>>` denote bitwise AND, OR, left shift, and unsigned right shift. The coordinates are nonnegative.

```
B = FBP << 12
P = FBW
W = P × 64
M = 000FFFFFH
```

Every word fetch wraps with mask `M`, including CT32/CT24's second word. The base and pixel offset are added before this wrap. All nine FBP bits remain representable; a base beyond one installed-memory period aliases through the same mask.

#### 3.2.6.1 CT32 and CT24

These formats use 64 × 32-pixel pages:

```
page   = (y >> 5) × P + (x >> 6)
px     = x & 63
py     = y & 31
bx     = px >> 3
by     = py >> 3

block  = (bx & 1) | ((by & 1) << 1)
       | ((bx & 2) << 1) | ((by & 2) << 2) | ((bx & 4) << 2)
column = (px & 1) | ((py & 1) << 1)
       | ((px & 6) << 1) | ((py & 6) << 3)

A = (B + (page << 12) + (block << 7) + (column << 1)) & M
```

Fetch `L` at `A` and `H` at `(A + 1) & M`.

#### 3.2.6.2 CT16 and CT16S

These formats use 64 × 64-pixel pages and share their column calculation:

```
page   = (y >> 6) × P + (x >> 6)
px     = x & 63
py     = y & 63
bx     = px >> 4
by     = py >> 3

column = ((px & 1) << 1) | ((px & 2) << 2)
       | ((px & 4) << 2) | ((px & 8) >> 3)
       | ((py & 1) << 2) | ((py & 2) << 4) | ((py & 4) << 4)
```

Select the block calculation for the programmed format:

```
CT16:
block = ((bx & 1) << 1) | (by & 1) | ((bx & 2) << 2)
      | ((by & 2) << 1) | ((by & 4) << 2)

CT16S:
block = (by & 1) | ((bx & 1) << 1) | (by & 4)
      | ((by & 2) << 2) | ((bx & 2) << 3)

A = (B + (page << 12) + (block << 7) + column) & M
```

#### 3.2.6.3 GX16 and GPU24

GX16 uses linear word addressing:

```
A = (B + y × W + x) & M
```

GPU24 uses the CT16 calculation twice, at the two word coordinates in Section 3.2.5.

**Zero-width setting:** FBW = 0 is not a disable setting. It supplies a zero row stride for GX16 and zero pages per row for the page-based formats. The remaining coordinate terms and address wrap still operate; no width fault is raised.

### 3.2.7 Output composition

Composition begins with the background, applies circuit 2, then applies circuit 1. Background R, G, and B come from `BGCOLOR_LOW[7:0]`, `[15:8]`, and `[23:16]`; background alpha is zero. A disabled circuit performs no operation.

**Window limits:** each table below applies only inside that enabled circuit's display window and selected field rows. Outside its window, the circuit leaves the prior values unchanged. The two windows need not overlap.

Circuit 2 selects the background or its own RGB using SLBG. AMOD determines whether it also supplies alpha:

| SLBG | AMOD | RGB after circuit 2 | Alpha after circuit 2 |
|---|---|---|---|
| 0 | 0 | Circuit-2 RGB | Background alpha 0 |
| 0 | 1 | Circuit-2 RGB | Circuit-2 sample alpha |
| 1 | 0 | Background RGB | Background alpha 0 |
| 1 | 1 | Background RGB | Circuit-2 sample alpha |

For circuit 1, let `C1` be one source RGB component, `A1` its original sample alpha, and `D` and `Ad` the prior RGB component and alpha at that pixel. The prior values come from the background and any circuit-2 operation at that pixel.

| MMOD | ALP | Circuit-1 RGB action | Alpha, AMOD = 0 | Alpha, AMOD = 1 |
|---|---|---|---|---|
| 0 | Ignored | Blend C1 over D with coefficient `min(2 × A1, 255)` | Select A1 | Preserve Ad |
| 1 | 0 | Preserve D | Select A1 | Preserve Ad |
| 1 | 1–254 | Blend C1 over D with coefficient ALP | Select A1 | Preserve Ad |
| 1 | 255 | Select C1 | Select A1 | Preserve Ad |

The RGB blend is computed independently for R, G, and B:

```
MMOD = 0:  a = min(2 × A1, 255)
MMOD = 1:  a = ALP

N = C1 × a + D × (255 - a) + 127
Output component = integer quotient of N / 255
```

The `+127` rounds the nonnegative weighted result to nearest. Inputs and coefficient are 0–255, so the RGB result is also 0–255 without further saturation. Source alpha is doubled and saturated only when deriving the MMOD = 0 **RGB coefficient**. ALP is used directly, not doubled. Output alpha is selected by AMOD, never blended by this equation.

In particular, a sample alpha of 128 gives coefficient 255 under MMOD = 0: circuit 1 supplies full source RGB, but AMOD = 0 still selects alpha **128**, not 255. This applies to CT24, GPU24, and sixteen-bit samples whose bit 15 is set. With AMOD = 1, a pixel can receive circuit-1 RGB while retaining background alpha zero if circuit 2 supplies no alpha there.

### 3.2.8 TECHNICAL SPECIFICATIONS

PCRTC supplies two reading circuits and one output-composition path. Local memory stores sixteen-bit words; PCRTC registers are programmed with thirty-two-bit words. The table gives the installed resources and field ranges.

| Resource or interface | Specification |
|---|---|
| Installed main RAM | 4 MiB; separate from GPU local memory and PCRTC registers |
| Shared GPU local memory | 2 MiB: 1,048,576 sixteen-bit words; low byte first |
| Reading circuits | Two, independently enabled, with separate framebuffer, source and display-window settings |

| Resource or interface | Specification |
|---|---|
| Register map | 26 thirty-two-bit slots in separate 12-word and 14-word address ranges |
| Configuration storage | 22 programmed words and a separate 22-word display latch |
| Programming access | Aligned thirty-two-bit register-word loads and stores; four-byte alignment |
| Configuration admission | No PCRTC write-ready wait, command FIFO or CYCLES register |
| Installed CPU/system-cycle frequency | 33,868,800 Hz; approximately 29.526 ns per system cycle |
| Beam-timing reference | 13,500,000 Hz |
| Horizontal signal step | 0–15 signal units per output pixel; a latched zero prevents pixel output |
| Per-circuit window fields | Horizontal span 1–4096 signal units; vertical height 1–2048; magnification X=1–16, Y=1–4 |

Use the signal-step conversion in Section 3.2.1 to calculate horizontal output boundaries. Section 3.3.2 gives complete example configurations. Both reading circuits are disabled at reset.

### 3.2.9 Beam timing

The timing reference clock is **13,500,000 Hz**. The `SMODE1`, `SYNCH1`, `SYNCH2`, and `SYNCV` fields define the half-line period and vertical landmarks:

```
H = HFP + HBP + HS + HF + HB
D = RC × (divider field + 1)
U = 1 when SMODE1_HIGH[4] = 0; otherwise 2

Half-line period = (H × D) / (2 × 13,500,000 × LC) seconds

Active-display half-lines = VDP × U
VSync half-line           = (VDP + VFP + VFPE) × U
Vertical sum              = VFP + VFPE + VBP + VBPE + VDP + VS
Total half-lines          = Vertical sum × U
```

Timing runs only when PRST and SINT are both zero and the half-line numerator, denominator, and total half-lines are all nonzero. A stopped live beam generates no timing events; displayed output still uses its latched timing. VBlank entry is at the active-display boundary, VSync at its computed landmark, and field end at total half-lines.

A beam restart begins at half-line zero, with VBlank inactive. The first HSync event point follows at half-line two. Restarting does not acknowledge CSR events or change FIELD. Event timing continues from the beam epoch, not from the time an event is observed.

At reset, H = 3456, D = 8, LC = 32, and U = 1. The half-line period is:

```
T = 27648 / 864000000 seconds = 32 microseconds
```

The reset beam sequence is:

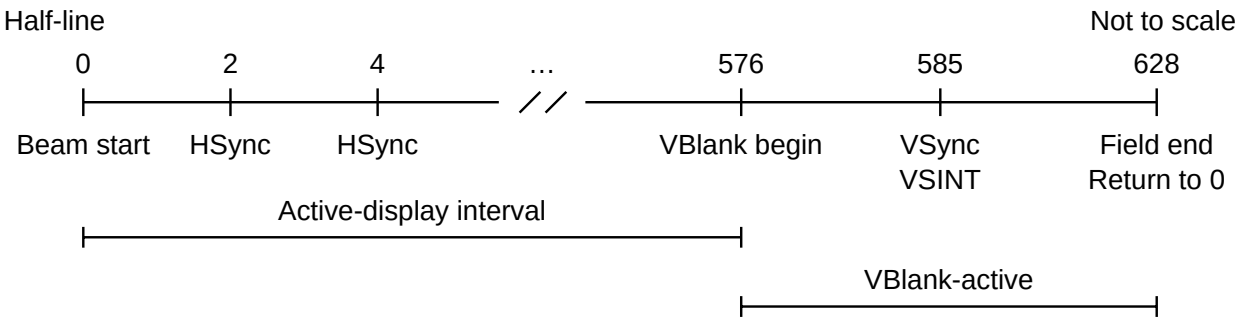



Figure 3.3: Reset beam events and active-display and VBlank intervals

The reset vertical counts are VDP = 576, VFP = 4, VFPE = 5, VBP = 33, VBPE = 5, and VS = 5. VBlank is inactive below half-line 576 and active for the remainder of the field.

For a beam restart at system cycle zero, the calculated event deadlines are:

| Landmark | Half-line | Time from restart | Rounded system-cycle deadline |
|---|---:|---:|---:|
| First HSync point | 2 | 64 µs | 2168 |
| VBlank entry; display latch | 576 | 18.432 ms | 624270 |
| VSync; VSINT | 585 | 18.720 ms | 634024 |
| Field end | 628 | 20.096 ms | 680628 |

This gives approximately 49.761146497 field repetitions per second. Fractional system cycles are retained between events; deadlines round upward without accumulating a fresh rounding error each half-line. An executing CPU instruction may finish before a crossed deadline is serviced. Allow for CPU interrupt acceptance and handler execution after these event deadlines. HSync points repeat every two half-lines, but a pending HSINT suppresses further interrupt requests until acknowledged.

At VSync, nonzero CMOD together with odd VFP toggles FIELD; otherwise FIELD becomes one. Reset VFP is even: reset FIELD is zero, and the first VSync sets it to one. VSync also raises VSINT.

### 3.2.10 Events and interrupts

Normal operation generates **HSINT and VSINT**. The remaining event positions, SIGNAL, FINISH and EDWINT, are retained pending bits and remain clear after reset during normal operation. All five positions use the acknowledgement and masking rules below. Use the GPU’s own completion interface for drawing.

```
Event raised ──► Is its CSR bit already set?
                    │                  │
                   yes                no
                    │                  ▼
            No new request       Set CSR event bit
                                       │
                              Is its IMR mask zero?
                                   │          │
                                  yes         no
                                   │          │
                             Request IRQ   Keep pending
```

A mask value of one prevents an interrupt request; it does not clear the event. Unmasking an already pending event requests the PCRTC interrupt immediately. Acknowledge an event by writing one to its CSR bit; writing zero leaves that event unchanged.

The PCRTC interrupt request is `IRQ_GX_PCRTC`, interrupt-controller mask bit `00000002H`. PCRTC acknowledgement and masking do not replace interrupt-controller acknowledgement or CPU interrupt control.

HSync event points start at half-line 2 and repeat every two half-lines. While HSINT is pending, later HSync points do not generate another interrupt request. Acknowledge HSINT to enable the first HSync point strictly after the current half-line; elapsed points are not replayed. VSINT is raised by VSync (Section 3.2.9); VBlank entry itself is the display-latch edge.

### 3.2.11 RESET and FLUSH

`CSR_LOW.FLUSH` and `CSR_LOW.RESET` are write-one actions, not retained settings. Event acknowledgements in the same write are processed first. RESET takes priority if both action bits are one.

Both actions drop queued DMA ingress and GP0 FIFO input, discard partial command and polyline input state, and clear pending command-completion timing. Both terminate readback and handle a partial image upload as described below. **Neither action is a general cancel or undo of accepted drawing.**

| Readback condition | Effect on accepted command work |
|---|---|
| No active readback; next uncompleted command is not a readback | Retain accepted raster and upload work |
| Readback accepted but not yet activated | Remove that readback and subsequent work; retain preceding commands |
| Readback pending, with its command still retained | Remove that readback and subsequent work; retain preceding commands |
| Readback submitted or data ready; other non-idle readback conditions | Clear the retained command work |

A still-retained in-progress raster or upload command is marked complete as pending completion timing is cleared; this does not undo its memory writes. Input queued after an active readback is discarded, not executed past that readback. The readback port returns to idle, loses its dimensions and cursor, and drops its DMA request. A canceled completion cannot later make that readback ready again.

A partial CPU-to-local-memory image upload is treated according to the words already consumed by image assembly and retained with the upload. Words merely admitted to the GP0 FIFO or DMA ingress FIFO do not count; the action discards those still-queued words.

| Image assembly state | Effect |
|---|---|
| No upload words | No upload action |
| Three-word header only | Discard the header |
| Header and at least one retained payload word | Retain an upload of that payload prefix |

If `n` payload words have been consumed by image assembly and retained, the prefix contains `min(width × height, 2 × n)` pixels. They are processed in row order under the accepted upload's retained mask-bit controls. With mask checking enabled, destination bit 15 = 1 suppresses that pixel's write. On a write, mask-setting mode forces bit 15; otherwise source bit 15 is retained. FLUSH or RESET therefore processes the retained prefix under these mask rules and leaves the missing remainder untouched. Here the count is in input command words; each supplies up to two local-memory pixels.

FLUSH leaves the PCRTC and GPU draw/display settings unchanged. RESET restores the PCRTC configuration, CSR, and IMR, sets FIELD to zero, restarts the beam, and installs reset display settings immediately. Both reading circuits are disabled.

RESET also clears the GP0/GP1 command words, drawing mode, texture window, drawing-area corners, drawing offset, mask-bit mode, and display start. It restores display mode, status, and horizontal/vertical display-range defaults, and disables readback DMA mode. The programmed VRAM Y-address extension is retained and carried into displayed settings. Neither action performs a local-memory clear or restores earlier contents. Accepted drawing and upload data can still change local memory.

### 3.2.12 Bus access

Use aligned 32-bit register-word loads and stores at the listed addresses. These accesses are available in either CPU mode and stores do not wait on a PCRTC ready signal. Subword access is not a byte-lane update:

| CPU data access | Address condition | PCRTC result |
|---|---|---|
| Register-word or U32 load/store | Four-byte aligned | Read/write one complete register word |
| U8 load | Four-byte aligned | Read the complete word, return bits 7:0 |
| U8 load | Word offset 1, 2, or 3 | UNALIGNED_IO bus fault; no register access |
| U8 store | Any PCRTC byte address | UNALIGNED_IO bus fault; no register access |
| U16 load/store | Even address, including four-byte aligned | UNALIGNED_IO bus fault; no register access |
| U16 load/store | Odd address | CPU address exception before register access |
| Register-word, U32, F32, or F64 load/store | Not four-byte aligned | CPU address exception before register access |

F32 transfers one word's bits. F64 transfers the low word at address A followed by the high word at A + 4; it requires four-byte, not eight-byte, alignment. If the first access faults, the second is not attempted. Both word accesses occur within one CPU instruction.

Multiword stores likewise perform successive word writes and stop at the first bus fault. Earlier writes remain effective. The two PCRTC address ranges do not become contiguous through a multiword instruction. Writes to `CSR_HIGH` and `IMR_HIGH` are accepted but ignored; they do not raise READ_ONLY faults.

**CPU transfer costs.** Each ordinary scalar register-word load or store instruction, including its displacement form, has a two-system-cycle base cost. PCRTC adds no write-ready wait. Address and value preparation, polling and surrounding control flow have their own instruction costs. Thus 22 separately issued configuration stores contribute 44 base cycles before preparation or display-edge waiting; they do not form an atomic display update. A word-sequence store instead costs two base cycles plus `n/4` rounded upward for its `n` words. Respect the two address ranges. Immediate readback confirms programmed words. Use a subsequent VBlank entry to establish that the display latch has taken them.

### 3.2.13 Faults

Access faults are **synchronous CPU exceptions**, not PCRTC events. They do not set CSR pending bits or request `IRQ_GX_PCRTC`.

| Fault | CPU action |
|---|---|
| Misalignment rejected by the CPU | Address-error-load, CAUSE `00000010H`, or address-error-store, CAUSE `00000014H`; BAD_ADDRESS records the attempted address |
| Rejected subword I/O access | UNALIGNED_IO bus fault, code 2; CPU data-bus-error, CAUSE `0000001CH` |

CPU alignment rejection occurs before the bus access and does not enter a bus-fault latch record. A faulting load does not overwrite its destination. The access rules in Section 3.2.12 apply equally to absolute and displacement-form data accesses.

The system bus retains the first fault's code, address, and access description until acknowledged. Later faults still advance the fault sequence but do not replace those latched details. These system words are separate from PCRTC:

| Address | System bus word | Operation |
|---|---|---|
| `08010208H` | BUS_FAULT_CODE | First fault code; zero means no latched fault |
| `0801020CH` | BUS_FAULT_ADDR | First fault address |
| `08010210H` | BUS_FAULT_ACCESS | First fault access description |
| `08010214H` | BUS_FAULT_ACK | A nonzero write clears code, address, and access |

The three detail words reset to zero. Fault codes are NONE = 0, UNMAPPED = 1, UNALIGNED_IO = 2, and READ_ONLY = 3. ACCESS combines READ bit 0 or WRITE bit 1 with the access kind: U8 bit 8, U16 bit 9, U32 bit 10, register-word bit 11, F32 bit 12, or F64 bit 13.

Storage selectors and framebuffer address wrap have the deterministic behavior in Section 3.2.5 and Section 3.2.6; they are not format, width, or framebuffer-address exceptions.

## 3.3 Timing and interrupt programming examples

### 3.3.1 Acknowledge stale HSINT and unmask only HSINT

**Starting condition:** no particular event-pending state is required. This sequence replaces the PCRTC event masks and acknowledges HSINT only.

| Order | Address | Register | Write value | Result |
|---|---|---|---|---|
| 1 | `080103DCH` | `IMR_LOW` | `00007F00H` | Mask all five PCRTC events |
| 2 | `080103D4H` | `CSR_LOW` | `00000004H` | Acknowledge HSINT; do not request RESET or FLUSH |
| 3 | `080103DCH` | `IMR_LOW` | `00007B00H` | Unmask HSINT; keep the other four events masked |

```
Mask all events ⟶ Acknowledge HSINT ⟶ Unmask HSINT
  00007F00H           00000004H            00007B00H
```

`7F00H = 6000H | 1F00H`. The final word clears IMR bit 10 only, giving `7B00H = 6000H | 1B00H`. CSR bit 2 is acknowledged by `00000004H`.

**Completion:** IMR reads `00007B00H`. FIELD and the other four CSR event bits are unchanged by step 2. HSINT need not remain clear: if it becomes pending between steps 2 and 3, unmasking it requests an interrupt immediately. Otherwise, a newly raised HSINT requests an interrupt while this mask remains installed.

If step 2 clears a pending HSINT while timing runs, the next HSync event point is after the current half-line. Masked events remain pending until acknowledged; a later unmask can therefore request an interrupt immediately.

### 3.3.2 Select a display geometry and its timing

Output size, source sampling and beam repetition are separate settings. Changing `DISPLAY1_HIGH` changes the window; it does not select a corresponding field rate. Use the following complete combinations when a 256–640-column GX16 picture is required. They give the required register combinations; other configurations follow the field equations in Section 3.2.

Each example enables circuit 1 only, uses source origin (0,0), and reads linear GX16 storage with a **1024-word source stride**. Prepare the source image before publishing it. Use these common composition words:

| Address | Register | Value |
|---|---|---|
| `08010354H` | PMODE_LOW | `0000FF21H` |
| `08010358H` | PMODE_HIGH | `00000000H` |
| `0801035CH` | DISPFB1_LOW | `000FA000H` |
| `08010360H` | DISPFB1_HIGH | `00000000H` |
| `0801037CH` | BGCOLOR_LOW | `00000000H` |
| `08010380H` | BGCOLOR_HIGH | `00000000H` |

`DISPFB1_LOW` selects base word zero, FBW=16 and GX16 storage. The two `DISPLAY1` words below choose the signal position, magnification and output extent. All rows use X magnification 4, Y magnification 1 and horizontal signal step 4: adjacent output columns therefore select adjacent source words.

| Output geometry | Timing group | DIS-PLAY1_LOW | DIS-PLAY1_HIGH | SYNCV_LOW | SMODE2_LOW |
|---|---|---|---|---|---|
| 256 × 192 | A | 018252A8H | 000BF3FFH | 02101404H | 00000000H |
| 256 × 212 | A | 018252A8H | 000D33FFH | 02101404H | 00000000H |
| 256 × 240 | A | 018252A8H | 000EF3FFH | 02101404H | 00000000H |
| 320 × 240 | A | 018252A8H | 000EF4FFH | 02101404H | 00000000H |
| 368 × 240 | A | 018252A8H | 000EF5BFH | 02101404H | 00000000H |
| 512 × 240 | A | 018252A8H | 000EF7FFH | 02101404H | 00000000H |
| 640 × 240 | A | 018252A8H | 000EF9FFH | 02101404H | 00000000H |
| 640 × 448, interlaced | B | 0183228CH | 001BF9FFH | 01A01801H | 00000001H |
| 640 × 480, interlaced | B | 0183228CH | 001DF9FFH | 01A01801H | 00000001H |
| 640 × 512, interlaced | A | 018482A8H | 001FF9FFH | 02101401H | 00000001H |

Write DISPLAY1_LOW at 08010364H, DISPLAY1_HIGH at 08010368H, SYNCV_LOW at 080103CCH and SMODE2_LOW at 080103B4H. Write SMODE2_HIGH (080103B8H) as zero. The remaining timing words come from the selected group:

| Address | Register or programming phase | Group A | Group B |
|---|---|---|---|
| 080103ACH | SMODE1_LOW, hold beam during setup | 40836504H | 40834504H |
| 080103ACH | SMODE1_LOW, run after setup | 40806504H | 40804504H |
| 080103B0H | SMODE1_HIGH | 00000007H | 00000007H |
| 080103BCH | SYNCH1_LOW | 1FC83030H | 1F06F040H |
| 080103C0H | SYNCH1_HIGH | 0007F5C2H | 0007F5B6H |
| 080103C4H | SYNCH2_LOW | 003484BCH | 0033A4D8H |
| 080103C8H | SYNCH2_HIGH | 00000000H | 00000000H |
| 080103D0H | SYNCV_HIGH | 00A90005H | 00C78006H |

The resulting repetition rates follow the beam equation, not the width or height labels:

| Configuration | Half-line period | Half-lines per field | Field repetitions per second | Complete two-field pairs per second |
|---|---|---|---|---|
| Group A, non-interlaced rows above | 32 µs | 628 | Approximately 49.7611465 | Not interlaced |
| Group B, interlaced rows above | 27456 / 864000000 seconds | 525 | Approximately 59.9400599 | Approximately 29.9700300 |
| Group A, 640 × 512 interlaced | 32 µs | 625 | 50 | 25 |

For an interlaced row, the stated height is the complete output rectangle. One field traverses alternate rows: 224 of 448, 240 of 480, or 256 of 512. FFMD=0 in these examples, so the source row stride is two and source phase depends on the signal Y and FIELD as described in Section 3.2.4. In particular, the two group-B examples start at signal Y=50, while the 512-line example starts at signal Y=72. Both positions are even, so FIELD selects the source row parity directly. Keep both sets of source rows valid.

### 3.3.3 Program the complete configuration before restarting the beam

This sequence replaces the global beam timing. Use it only while the cartridge owns timing configuration and no other writer is changing the display words. For an ordinary buffer switch with unchanged timing, use Section 3.2.3 instead.

1. Write the selected group's **hold-beam** SMODE1_LOW word. PRST and SINT are both set, so no live beam events can occur during the remaining setup.
2. Write SMODE1_HIGH, both SYNCH1 words, both SYNCH2 words, the selected SYNCV_LOW, its group's SYNCV_HIGH, SMODE2_LOW and zero SMODE2_HIGH.
3. Write the framebuffer, source origin, DISPLAY1 and background words. Enable circuit 1 with the complete PMODE words above; EN2 remains zero.
4. Write the group's **run** SMODE1_LOW word last. This clears PRST and SINT and restarts at half-line zero. Observe a subsequent VBlank entry before treating the configuration as displayed.

Holding the live beam is not a request to blank the previously latched picture. The new configuration becomes displayed at VBlank entry. Existing pending interrupt flags are not acknowledged by this sequence; use a fresh VBlank observation as in the display-switch sequence.

These writes configure PCRTC. GP1 display-mode and range words are separate GPU state: writing them does not substitute for this timing and window setup. Drawing bounds, offsets, texture storage and the GP1 Y-address gate likewise remain the GPU's programming responsibility. To display another region, replace the DISPFB base and source origin while preserving the selected format and source stride; a DISPLAY window is not a local-memory allocation.

## 3.4 REGISTER REFERENCE

PCRTC register words occupy four bytes. The composition registers start at `08010354H`; the timing and status registers start at `080103ACH`. Use the individual addresses below: the intervening addresses are not PCRTC registers.

`R/W` below means that the complete configuration word is retained and returned on read. In its layout, `R` marks retained bits with no named operational field. These bits are not a must-zero requirement. CSR and IMR instead use the per-bit access rules in their entries.

### 3.4.1 PMODE — Circuit enables and output control

| Address | Name | Purpose | Access | Reset |
|---|---|---|---|---|
| `08010354H` | `PMODE_LOW` | Circuit enables and output controls | R/W | `00000000H` |
| `08010358H` | `PMODE_HIGH` | Retained high word | R/W | `00000000H` |

| Word | Bits | Field |
|---|---|---|
| LOW | 31:16 | R |
| LOW | 15:8 | ALP |
| LOW | 7 | SLBG |
| LOW | 6 | AMOD |
| LOW | 5 | MMOD |
| LOW | 4:2 | R |
| LOW | 1 | EN2 |
| LOW | 0 | EN1 |

| Word | Bits | Field |
|---|---|---|
| HIGH | 31:0 | R |

| Field | Setting |
|---|---|
| EN1, EN2 | 0: disable the corresponding reading circuit; 1: enable it |
| MMOD | 0: derive the circuit-1 RGB coefficient from sample alpha; 1: use ALP |
| AMOD | 0: circuit 1 selects its sample alpha; 1: circuit 2 supplies alpha and circuit 1 preserves prior alpha |

| Field | Setting |
|---|---|
| SLBG | 0: circuit 2 supplies RGB; 1: retain background RGB at the circuit-2 stage |
| ALP | Eight-bit circuit-1 RGB coefficient, 0–255, used directly when MMOD = 1 |

These actions apply only inside the enabled circuit's window. Section 3.2.7 gives the complete operation tables, coefficient and rounding equation, and alpha selection. Program `DISPFB` and `DISPLAY` before enabling a circuit. The setting in Section 3.2.2 supplies a complete PMODE word for that example.

### 3.4.2 DISPFB1 — Circuit 1 framebuffer

| Address | Name | Purpose | Access | Reset |
|---|---|---|---|---|
| `0801035CH` | `DISPFB1_LOW` | Framebuffer base, width, and storage selection | R/W | `000FA000H` |
| `08010360H` | `DISPFB1_HIGH` | Framebuffer source origin | R/W | `00000000H` |

| Word | Bits | Field |
|---|---|---|
| LOW | 31:20 | R |
| LOW | 19:15 | Storage |
| LOW | 14:9 | Width |
| LOW | 8:0 | Base |

| Word | Bits | Field |
|---|---|---|
| HIGH | 31:22 | R |
| HIGH | 21:11 | Source Y |
| HIGH | 10:0 | Source X |

Base (`FBP`) is in groups of 4096 sixteen-bit local-memory words; width (`FBW`) is in groups of 64 framebuffer pixels. Source X and source Y specify the source origin. Packing and address calculations are in Section 3.2.5 and Section 3.2.6, including FBW = 0 and wrap.

| Storage selector | Circuit 1 format |
|---|---|
| 0 | CT32 |
| 1 | CT24 |
| 2 | CT16 |
| 10 | CT16S |
| 18 | GPU24 |
| 31 | GX16 |
| Other | Zero RGBA sample; circuit remains enabled |

### 3.4.3 DISPLAY1 — Circuit 1 signal window

| Address | Name | Purpose | Access | Reset |
|---|---|---|---|---|
| `08010364H` | `DISPLAY1_LOW` | Signal origin and magnification | R/W | `018252A8H` |
| `08010368H` | `DISPLAY1_HIGH` | Signal-window extents | R/W | `000EF4FFH` |

| Word | Bits | Field |
|---|---|---|
| LOW | 31:29 | R |
| LOW | 28:27 | Y mag |
| LOW | 26:23 | X mag |
| LOW | 22:12 | Signal Y |
| LOW | 11:0 | Signal X |

| Word | Bits | Field |
|---|---|---|
| HIGH | 31:23 | R |
| HIGH | 22:12 | Vertical extent |
| HIGH | 11:0 | Horizontal extent |

| Field | Actual setting |
|---|---|
| X magnification | Encoded value + 1; range 1–16 |
| Y magnification | Encoded value + 1; range 1–4 |
| Horizontal extent | Signal span = encoded value + 1 |
| Vertical extent | Height = encoded value + 1 |

Set signal X/Y for the window origin. Horizontal extents use signal units, not final output-pixel counts; apply the signal-step boundary conversion in Section 3.2.1.

### 3.4.4 DISPFB2 — Circuit 2 framebuffer

| Address | Name | Purpose | Access | Reset |
|---|---|---|---|---|
| `0801036CH` | `DISPFB2_LOW` | Framebuffer base, width, and storage selection | R/W | `000FA000H` |
| `08010370H` | `DISPFB2_HIGH` | Framebuffer source origin | R/W | `00000000H` |

| Word | Bits | Field |
|---|---|---|
| LOW | 31:20 | R |
| LOW | 19:15 | Storage |
| LOW | 14:9 | Width |
| LOW | 8:0 | Base |

| Word | Bits | Field |
|---|---|---|
| HIGH | 31:22 | R |
| HIGH | 21:11 | Source Y |
| HIGH | 10:0 | Source X |

Base (`FBP`) is in groups of 4096 sixteen-bit local-memory words; width (`FBW`) is in groups of 64 framebuffer pixels. Circuit 2 has its own source X/Y origin. Packing and address calculations are in Section 3.2.5 and Section 3.2.6, including FBW = 0 and wrap.

| Storage selector | Circuit 2 format |
|---|---|
| 0 | CT32 |
| 1 | CT24 |

| Storage selector | Circuit 2 format |
|---|---|
| 2 | CT16 |
| 10 | CT16S |
| 31 | GX16 |
| 18 or other | Zero RGBA sample; circuit remains enabled |

### 3.4.5 DISPLAY2 — Circuit 2 signal window

| Address | Name | Purpose | Access | Reset |
|---|---|---|---|---|
| 08010374H | DISPLAY2_LOW | Signal origin and magnification | R/W | 018252A8H |
| 08010378H | DISPLAY2_HIGH | Signal-window extents | R/W | 000EF4FFH |

| Word | Bits | Field |
|---|---|---|
| LOW | 31:29 | R |
| LOW | 28:27 | Y mag |
| LOW | 26:23 | X mag |
| LOW | 22:12 | Signal Y |
| LOW | 11:0 | Signal X |

| Word | Bits | Field |
|---|---|---|
| HIGH | 31:23 | R |
| HIGH | 22:12 | Vertical extent |
| HIGH | 11:0 | Horizontal extent |

The encoded-plus-one rules are the same as for circuit 1: X magnification is 1–16, Y magnification is 1–4, and each extent specifies one less than the corresponding span or height. Program this window independently; enabled circuit windows jointly determine the output bounds.

### 3.4.6 BGCOLOR — Background color

| Address | Name | Purpose | Access | Reset |
|---|---|---|---|---|
| 0801037CH | BGCOLOR_LOW | Background color input | R/W | 00000000H |
| 08010380H | BGCOLOR_HIGH | Retained high word | R/W | 00000000H |

| Word | Bits | Field |
|---|---|---|
| LOW | 31:24 | R |
| LOW | 23:0 | Background color |

| Word | Bits | Field |
|---|---|---|
| HIGH | 31:0 | R |

Write R in bits 7–0, G in bits 15–8, and B in bits 23–16. Background alpha is zero. PMODE controls composition with the enabled circuits as described in Section 3.2.7.

### 3.4.7 SMODE1 — Timing factors and signal step

| Address | Name | Purpose | Access | Reset |
|---|---|---|---|---|
| 080103ACH | SMODE1_LOW | Timing factors, run controls, and horizontal signal step | R/W | 40806504H |
| 080103B0H | SMODE1_HIGH | Vertical-unit selection | R/W | 00000007H |

| Word | Bits | Field |
|---|---|---|
| LOW | 31:25 | R |
| LOW | 24:21 | Step X |
| LOW | 20:18 | R |
| LOW | 17 | SINT |
| LOW | 16 | PRST |
| LOW | 15 | R |
| LOW | 14:13 | CMOD |
| LOW | 12 | R |
| LOW | 11:10 | Divider |
| LOW | 9:3 | LC |
| LOW | 2:0 | RC |

| Word | Bits | Field |
|---|---|---|
| HIGH | 31:5 | R |
| HIGH | 4 | U |
| HIGH | 3:0 | R |

RC, LC, and the divider field set the half-line period through the equations in Section 3.2.9. Step X is the number of horizontal signal units per output pixel. A latched Step X of zero prevents pixel output.

| Divider field | Multiplier used with RC |
|---|---|
| 0 | 1 |
| 1 | 2 |
| 2 | 3 |
| 3 | 4 |

| High-word bit 4 | Vertical multiplier U |
|---|---|
| 0 | 1 |
| 1 | 2 |

Timing runs only when PRST and SINT are both zero and the calculated half-line numerator, denominator, and total half-lines are all nonzero. Setting either run-control bit stops timing. CMOD participates in FIELD updates at VSync: nonzero CMOD and odd VFP select toggling; otherwise VSync sets FIELD to one.

**Timing-write caution:** changing either word restarts the beam, even when the changed bits are retained bits. A restart does not acknowledge CSR events or assign FIELD.

### 3.4.8 SMODE2 — Interlaced traversal

| Address | Name | Purpose | Access | Reset |
|---|---|---|---|---|
| 080103B4H | SMODE2_LOW | Interlace and frame-mode controls | R/W | 00000000H |
| 080103B8H | SMODE2_HIGH | Retained high word | R/W | 00000000H |

| Word | Bits | Field |
|---|---|---|
| LOW | 31:2 | R |
| LOW | 1 | FFMD |
| LOW | 0 | INT |

| Word | Bits | Field |
|---|---|---|
| HIGH | 31:0 | R |

| INT | FFMD | Traversal |
|---|---|---|
| 0 | Either | Non-interlaced; FFMD does not select frame mode |
| 1 | 0 | Interlaced, non-frame mode |
| 1 | 1 | Interlaced, frame mode |

Use the source stride and phase rules in Section 3.2.4. Writes to these words follow the display-latch rule without restarting the beam.

### 3.4.9 SYNCH1 — Horizontal timing counts

| Address | Name | Purpose | Access | Reset |
|---|---|---|---|---|
| 080103BCH | SYNCH1_LOW | HFP and HBP counts | R/W | 1FC83030H |
| 080103C0H | SYNCH1_HIGH | HS count | R/W | 0007F5C2H |

| Word | Bits | Field |
|---|---|---|
| LOW | 31:22 | R |
| LOW | 21:11 | HBP |
| LOW | 10:0 | HFP |

| Word | Bits | Field |
|---|---|---|
| HIGH | 31:27 | R |
| HIGH | 26:11 | HS |
| HIGH | 10:0 | R |

HFP, HBP, and HS contribute horizontal timing counts to H. Their time scale is set by RC, LC, and the divider, not by the output-pixel magnification fields. A changed write to either word recalculates timing and restarts the beam.

| Address | Name | Purpose | Access | Reset |
|---|---|---|---|---|

### 3.4.10 SYNCH2 — Additional horizontal timing counts

| Address | Name | Purpose | Access | Reset |
|---|---|---|---|---|
| `080103C4H` | `SYNCH2_LOW` | HF and HB counts | R/W | `003484BCH` |
| `080103C8H` | `SYNCH2_HIGH` | Retained timing high word | R/W | `00000000H` |

| Word | Bits | Field |
|---|---|---|
| LOW | 31:27 | R |
| LOW | 26:11 | HB |
| LOW | 10:0 | HF |

| Word | Bits | Field |
|---|---|---|
| HIGH | 31:0 | R |

HF and HB complete the horizontal sum H. **Changing `SYNCH2_HIGH` also restarts the beam**, although its bits have no named timing field. Writing its current value does not restart it.

### 3.4.11 SYNCV — Vertical timing and beam phases

| Address | Name | Purpose | Access | Reset |
|---|---|---|---|---|
| `080103CCH` | `SYNCV_LOW` | VFP, VFPE, and VBP counts | R/W | `02101404H` |
| `080103D0H` | `SYNCV_HIGH` | VBPE, VDP, and VS counts | R/W | `00A90005H` |

| Word | Bits | Field |
|---|---|---|
| LOW | 31:20 | VBP |
| LOW | 19:10 | VFPE |
| LOW | 9:0 | VFP |

| Word | Bits | Field |
|---|---|---|
| HIGH | 31:21 | VS |
| HIGH | 20:10 | VDP |
| HIGH | 9:0 | VBPE |

These fields count vertical units. Multiplying by U converts them to half-lines. VDP sets the active-display interval. VFP and VFPE offset VSync from its end; all six fields determine field length. A changed write to either word restarts the beam.

Section 3.2.9 gives the half-line equations, reset timing landmarks, and FIELD update rule. Section 3.2.3 specifies the VBlank-entry display latch. These counts are not output-pixel extents.

### 3.4.12 CSR — Event acknowledgement, field status, and actions

| Address | Name | Purpose | Access | Reset |
|---|---|---|---|---|
| 080103D4H | CSR_LOW | Event status, acknowledgements, FIELD, FLUSH, and RESET | Read; W1C events; write actions | 551B4000H |
| 080103D8H | CSR_HIGH | Zero high word | Read zero; writes have no action | 00000000H |

| Word | Bits | Field |
|---|---|---|
| LOW | 31:14 | — |
| LOW | 13 | FIELD |
| LOW | 12:10 | — |
| LOW | 9 | RESET |
| LOW | 8 | FLUSH |
| LOW | 7:5 | — |
| LOW | 4 | EDWINT |
| LOW | 3 | VSINT |
| LOW | 2 | HSINT |
| LOW | 1 | FINISH |
| LOW | 0 | SIGNAL |

| Word | Bits | Field |
|---|---|---|
| HIGH | 31:0 | Read zero; writes have no action |

| Bits | Field | Access and operation |
|---|---|---|
| 0 | SIGNAL | Read pending status; write one to clear |
| 1 | FINISH | Read pending status; write one to clear |
| 2 | HSINT | Read pending status; write one to clear |
| 3 | VSINT | Read pending status; write one to clear |
| 4 | EDWINT | Read pending status; write one to clear |
| 8 | FLUSH | Write one to request a flush action; does not latch |
| 9 | RESET | Write one to request reset; does not latch |
| 13 | FIELD | Read-only field status; CSR writes do not assign it |

Write zero to event bits that must remain unchanged. Do not acknowledge events by writing back a complete CSR read value: every event bit written as one is cleared. Use an explicit acknowledgement mask, as in Section 3.3.1.

Only HSINT and VSINT are normally generated. SIGNAL, FINISH, and EDWINT remain retained, W1C and maskable positions, not hardwired-zero fields; see Section 3.2.10. CSR writes do not assign the unnamed positions.

RESET takes priority over FLUSH. Both clear queued input and terminate readback; they can retain accepted drawing and commit a partial image upload. RESET restores PCRTC registers and installs reset display settings immediately; its GPU register effects are bounded as listed in Section 3.2.11. FLUSH does not reset the display. Section 3.2.11 also gives the accepted-work and upload rules.

### 3.4.13 IMR — PCRTC event masks

| Address | Name | Purpose | Access | Reset |
|---|---|---|---|---|
| 080103DCH | IMR_LOW | Five event masks and fixed read bits | R/W masks; fixed bits as below | 00007F00H |

| Address | Name | Purpose | Access | Reset |
|---|---|---|---|---|
| 080103E0H | IMR_HIGH | Zero high word | Read zero; writes have no action | 00000000H |

| Word | Bits | Field |
|---|---|---|
| LOW | 31:15 | Zero |
| LOW | 14:13 | Ones |
| LOW | 12 | EDWINT |
| LOW | 11 | VSINT |
| LOW | 10 | HSINT |
| LOW | 9 | FINISH |
| LOW | 8 | SIGNAL |
| LOW | 7:0 | Zero |

| Word | Bits | Field |
|---|---|---|
| HIGH | 31:0 | Read zero; writes have no action |

| Mask bit | Corresponding CSR event | 0 | 1 |
|---|---|---|---|
| 8 | SIGNAL, bit 0 | Unmask | Mask |
| 9 | FINISH, bit 1 | Unmask | Mask |
| 10 | HSINT, bit 2 | Unmask | Mask |
| 11 | VSINT, bit 3 | Unmask | Mask |
| 12 | EDWINT, bit 4 | Unmask | Mask |

Writes retain mask bits 8–12, force bits 13–14 to one, and clear all other bits. Thus bits 13–14 always read one; bits 31–15 and 7–0 read zero. Reset masks all five events.

A mask write does not acknowledge CSR. Check pending-event handling when unmasking: any already pending event whose mask becomes zero can request the interrupt immediately. Use the mask–acknowledge–unmask sequence in Section 3.3.1 when enabling HSINT.

## 3.5 SOURCE ORIGIN AND DISPLAY-WINDOW POSITION

A source origin selects which pixels a circuit reads. A display-window position selects where that circuit contributes to the picture. This example changes them separately while leaving the image in local memory unchanged. Circuit 2 supplies a fixed 256 × 192-pixel canvas; circuit 1 places a 128 × 96-pixel cutout over it. Both circuits read the same image.

The image contains twelve coloured 64 × 64-pixel cells, a white L, a black 16 × 16-pixel square and a red 8 × 8-pixel square. Finish drawing and readback before enabling display, then leave the stored image unchanged through stages A, B and C. Each change of source origin or window position then selects a different view of the same pixels.

### 3.5.1 Program the fixed canvas and initial cutout

Both framebuffer descriptors select GX16 storage, base zero, and a row width of 1024 sixteen-bit words, corresponding to 1024 pixels. Use normal reset timing, noninterlaced output, horizontal magnification 4, and vertical magnification 1. With a horizontal signal step of 4, one source pixel occupies one output pixel.

All table addresses and register words are hexadecimal. Keep both circuits disabled while preparing their descriptors; enable the final PMODE word last. The table describes stage A, not an ordered sequence of writes.

| Register | Address | Stage-A raw word |
|---|---|---|
| PMODE_LOW | 08010354 | 0000FF23 |

| Register | Address | Stage-A raw word |
|---|---|---|
| DISPFB1_LOW | 0801035C | 000FA000 |
| DISPFB1_HIGH | 08010360 | 00000000 |
| DISPLAY1_LOW | 08010364 | 018553A8 |
| DISPLAY1_HIGH | 08010368 | 0005F1FF |
| DISPFB2_LOW | 0801036C | 000FA000 |
| DISPFB2_HIGH | 08010370 | 00000000 |
| DISPLAY2_LOW | 08010374 | 018252A8 |
| DISPLAY2_HIGH | 08010378 | 000BF3FF |

Circuit 2 starts at signal position (680, 37), spans 1024 horizontal signal units and 192 rows, and reads source (0, 0). Its window fixes the cropped output origin at (0, 0). Keep it enabled and unchanged throughout the example. The minimum enabled signal X and Y establish that origin; moving a lone enabled circuit would move the crop with it.

`PMODE_LOW = 0000FF23` enables both circuits. Circuit 2 supplies raw RGB; circuit 1 supplies direct raw RGB and its sample alpha inside its window, without RGB blending. The programmed ALP value of 255 does not force sample alpha to 255.

### 3.5.2 Change selection, then placement

The following table gives retained register words and decoded geometry. Source origins are framebuffer pixels; signal origins use signal coordinates. The output cutout uses output pixels, with its end coordinates excluded.

| Stage | DISPFB1_HIGH | DISPLAY1_LOW | Source origin | Signal origin | Output cutout |
|---|---|---|---|---|---|
| A | 00000000 | 018553A8 | (0, 0) | (936, 85) | [64, 192) × [48, 144) |
| B | 00008020 | 018553A8 | (32, 16) | (936, 85) | [64, 192) × [48, 144) |
| C | 00008020 | 01855428 | (32, 16) | (1064, 85) | [96, 224) × [48, 144) |

From A to B, write only `DISPFB1_HIGH`. The visible source selection changes inside the stationary window. From B to C, write only `DISPLAY1_LOW`. The selected cutout moves 32 output pixels right. Neither change alters its extent or the 256 × 192-pixel canvas.

### 3.5.3 Prepare the source and publish the changes

Own GP0, both circuits, the VBlank handler and source rectangle `(0,0)…(255,191)`. Keep this storage unchanged through A, B and C, and keep other writers and supervisor entry outside the sequence. The rectangle must not overlap the firmware reservation; querying that reservation does not allocate the remaining memory.

With interrupt delivery masked, write CSR RESET (`00000200H`), then PMODE=0. Program the eight framebuffer/window words in the stage-A table. To reproduce the source picture, send these complete GP0 packets after E1–E6 establish zero offset, drawing area `(0,0)…(255,191)`, both row parities, no dither and E6=1 (force stored bit 15, do not check old bit 15):

```
rodata scene: word[] = {
    0x600000ff, 0, 0x00400040,
    0x6000ff00, 64, 0x00400040,
    0x60ff0000, 128, 0x00400040,
    0x6000ffff, 192, 0x00400040,
    0x60ffff00, 64 << 16, 0x00400040,
    0x60ff00ff, 64 | (64 << 16), 0x00400040,
    0x60ffffff, 128 | (64 << 16), 0x00400040,
    0x60202020, 192 | (64 << 16), 0x00400040,
    0x600080ff, 128 << 16, 0x00400040,
    0x60800080, 64 | (128 << 16), 0x00400040,
    0x60808000, 128 | (128 << 16), 0x00400040,
```

```
    0x60808080, 192 | (128 << 16), 0x00400040,
    0x60ffffff, 8 | (8 << 16), 4 | (40 << 16),
    0x60ffffff, 8 | (8 << 16), 40 | (4 << 16),
    0x60000000, 96 | (64 << 16), 16 | (16 << 16),
    0x600000ff, 144 | (96 << 16), 8 | (8 << 16),
}
local gp0<const>: *word = 0x0801023c
local gp1<const>: *word = 0x08010240
for index = 0, #scene - 1 do
    *gp0 = scene[index]
end
while (*gp1 & 0x04000000) == 0 do
end
```

The three marked source pixels `(8,8)`, `(96,64)` and `(144,96)` store FFFFH, 8000H and 801FH respectively. A 1×1 C0 read after READY returns that raw word in its low half and zero in its high half; it is not a read of the composed output.

Use a VBlank sequence word shared by the entry code and its interrupt handler. This fragment owns the central interrupt mask and uses ordinary cartridge interrupt entry with CPU interrupt acceptance enabled:

```
bss vblank_sequence: word
local irq_ack<const>: *word = 0x08000004
local irq_mask<const>: *word = 0x08000008
local pmode<const>: *word = 0x08010354
local source1<const>: *word = 0x08010360
local window1<const>: *word = 0x08010364

function irq(flags)
    if (flags & 4) ~= 0 then
        *vblank_sequence = *vblank_sequence + 1
    end
    *irq_ack = flags & 4
end

-- A: enable the prepared composition; observe a fresh latch edge.
*irq_mask = 0
*pmode = 0x0000ff23
*irq_ack = 4
local before<const> = *vblank_sequence
*irq_mask = 4
while *vblank_sequence == before do
    halt_until_irq
end
```

For B, repeat the masked publication sequence with `*source1 = 0x00008020` in place of the PMODE store. For C, use `*window1 = 0x01855428` instead. Finish observing each stage before programming the next. An edge discarded by the stale-request acknowledgement can add a refresh; the later observed edge has latched the new setting. Masking delivery does not stop the latch or make several stores atomic. An immediate source/window read reports the programmed word, not display acceptance.

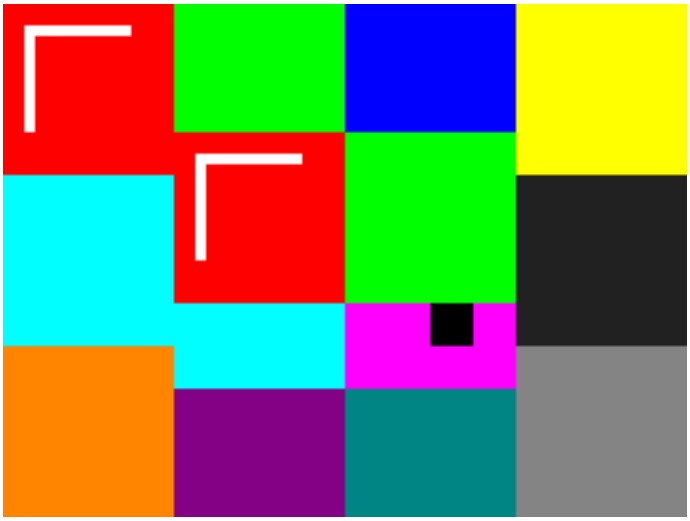


Figure 3.4: Stage A

*Figure A. Source (0, 0), cutout at (64, 48). The copied white L starts at (72, 56), and the copied black square at (160, 112).*

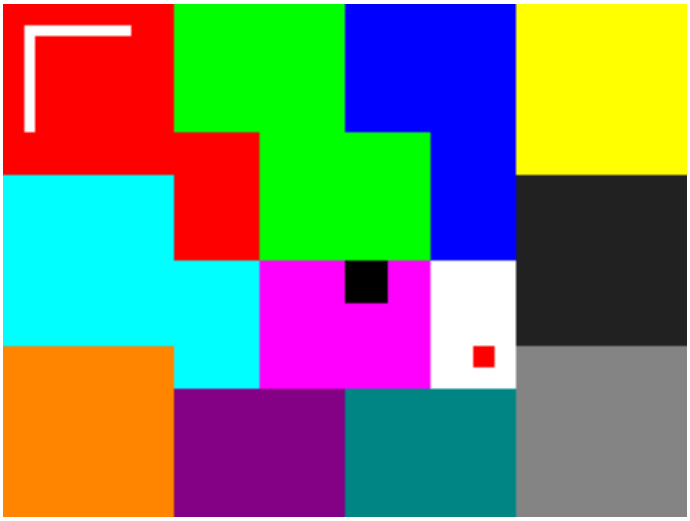


Figure 3.5: Stage B

*Figure B. Source (32, 16), cutout still at (64, 48). The copied black square starts at (128, 96), and the copied red square at (176, 128).*

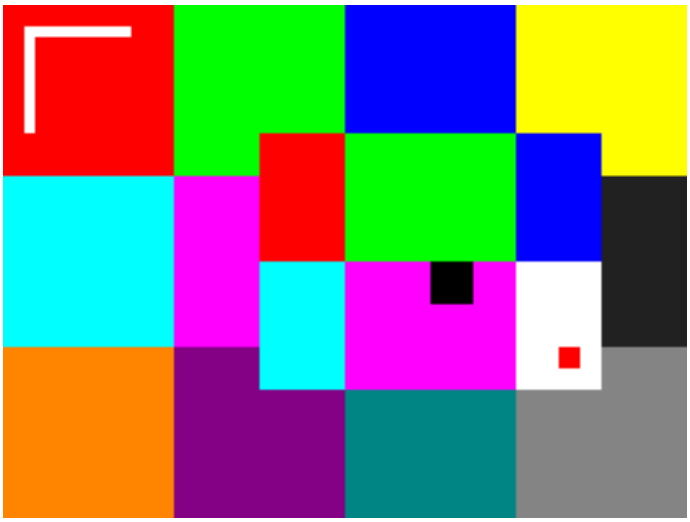


Figure 3.6: Stage C

*Figure C. The same source cutout at (96, 48). The copied black and red squares start at (160, 96) and (208, 128).*

The canvas white L stays at (8, 8) in all three pictures. The source origin (32, 16) excludes its cutout copy in B and C. These feature positions distinguish a source-origin change from a window-position change without adding a border or altering the image.

## 3.6 HORIZONTAL SOURCE SAMPLING

Let `s` be signal units per output pixel, `S` the horizontal signal origin, and `M` signal units per source pixel:

```
s = (SMODE1_LOW >> 21) & 15
S = DISPLAY_LOW & 4095
M = ((DISPLAY_LOW >> 23) & 15) + 1
```

For an enabled active horizontal window with nonzero `s`, `div` denotes the nonnegative integer quotient:

```
A = (S + s - 1) div s
sourceX(n) = framebufferX + ((A*s - S + n*s) div M)
```

`framebufferX` is the source X field of `DISPFB_HIGH`. `n` is the local output-pixel index from the circuit's left display edge, not an absolute coordinate. `A*s - S` is the first-pixel phase in signal units. Each subsequent output pixel advances by `s` signal units, whereas each source pixel occupies `M` signal units. Unequal steps need not advance source coordinates by one.

Window width is separate from source magnification. Convert the left and exclusive right signal boundaries independently upward to output-pixel boundaries, then subtract. Output width is not signal width divided by `M`.

### 3.6.1 Eight-pixel GX16 strip

Use reset timing with `SMODE1_LOW = 40806504H` (`s = 4`) and `SMODE2_LOW = 00000000H` (noninterlaced), with these display words:

| Register | Word |
|---|---|
| PMODE_LOW | 0000FF21H |
| DISPFB1_LOW | 000FA000H |
| DISPFB1_HIGH | 00008020H |
| DISPLAY1_LOW | 010252AAH |
| DISPLAY1_HIGH | 0000001FH |

Circuit 1 selects raw GX16 RGBA from framebuffer base zero, stride 1024 source pixels, and source origin `(32,16)`. The signal origin is `(682,37)`, the signal width is 32, the height is one row, and `M = 3`.

The horizontal signal interval is `[682,714)`. Its upward-converted boundaries are 171 and 179, yielding an **8 × 1** output. The first sample is at signal X 684, giving phase `684 − 682 = 2`. Upload the sixteen GX16 words `8000H` through `800FH` at source positions `(32,16)` through `(47,16)`.

In the table, `N = 2 + 4*n`; source offset is `N div 3`. Addresses are **16-bit word addresses**, not byte offsets.

| n | Signal X | N | Source offset | Source X,Y | Word address | GX16 word |
|---|---|---|---|---|---|---|
| 0 | 684 | 2 | 0 | (32,16) | 4020H | 8000H |
| 1 | 688 | 6 | 2 | (34,16) | 4022H | 8002H |
| 2 | 692 | 10 | 3 | (35,16) | 4023H | 8003H |
| 3 | 696 | 14 | 4 | (36,16) | 4024H | 8004H |
| 4 | 700 | 18 | 6 | (38,16) | 4026H | 8006H |
| 5 | 704 | 22 | 7 | (39,16) | 4027H | 8007H |
| 6 | 708 | 26 | 8 | (40,16) | 4028H | 8008H |
| 7 | 712 | 30 | 10 | (42,16) | 402AH | 800AH |

Observed RGB, left to right:

```
(0,0,0) (16,0,0) (24,0,0) (33,0,0)
(49,0,0) (57,0,0) (66,0,0) (82,0,0)
```

These expanded RGB555 values are not raw GX16 words. Every sample has alpha `80H`.



Figure 3.7: Eight PCRTC output pixels from unequal horizontal magnification

*Eight-pixel output, enlarged without smoothing. From left to right, source offsets are 0, 2, 3, 4, 6, 7, 8 and 10. The source words are consecutive; the displayed selection is not.*

### 3.6.2 Upload and select the strip

Start from reset timing with both circuits disabled and exclusive ownership of GP0, circuit 1, the VBlank handler and raw row 16, X=32…47. The following eight payload words store the sixteen consecutive GX16 samples. GP0 coordinates pack Y at bit 16; DISPFB source coordinates pack it at bit 11.

```
rodata pixels: word[8] = {
    0x80018000, 0x80038002, 0x80058004, 0x80078006,
    0x80098008, 0x800b800a, 0x800d800c, 0x800f800e,
}
local framebuffer<const>: *word = 0x0801035c
local source<const>: *word = 0x08010360
local window<const>: *word = 0x08010364
local extent<const>: *word = 0x08010368
*irq_mask = 0
*pmode = 0
*framebuffer = 0x000fa000
*source = 0x00008020
*window = 0x010252aa
*extent = 0x0000001f
*gp1 = 0x09000000
*gp0 = 0xe6000000
*gp0 = 0xa0000000
*gp0 = 32 | (16 << 16)
*gp0 = 16 | (1 << 16)
for index = 0, 7 do
    *gp0 = pixels[index]
end
while (*gp1 & 0x04000000) == 0 do
end
```

Use the ports and VBlank handler from the preceding example. Publish with PMODE=`0000FF21H`, acknowledge stale controller VBlank bit 2, sample the sequence while masked, unmask bit 2 and wait for a changed sequence. The 32-signal-unit window becomes eight output pixels; the selected source words are exactly those in the sampling table. Keep the source row unchanged while it remains displayed. A C0 request for the original 16×1 row, if used, returns eight packed words; it does not return the eight-pixel composed strip.

## 3.7 Composition: constant coefficient and source alpha

Both reading circuits cover the same 256 × 192 output. Circuit 1 contains red across the whole picture, divided into two 128-column halves: the left half has sample alpha 0, and the right half has sample alpha 128. Circuit 2 contains blue with sample alpha 128 throughout. The background is green. These sources, their windows and the background remain unchanged through all four phases; only PMODE changes.

The stored GX16 words are `0x001F` for circuit 1's left half, `0x801F` for its right half, and `0xFC00` for circuit 2. Both red words decode to RGB `(255,0,0)`; bit 15 supplies their different sample alpha values. Both reading circuits remain enabled, and ALP remains 128:

| Phase | PMODE_LOW | MMOD | AMOD | SLBG | ALP |
|---|---|---|---|---|---|
| A | `0x00008023` | 1 | 0 | 0 | 128 |
| B | `0x00008003` | 0 | 0 | 0 | 128 |
| C | `0x00008083` | 0 | 0 | 1 | 128 |
| D | `0x00008043` | 0 | 1 | 0 | 128 |

For each RGB channel, composition uses `Q = (S*k + D*(255-k) + 127) div 255`, where S is circuit 1's channel and D is the destination channel. Multiplication and addition precede the integer quotient; the denominator is 255, with 127 added for rounding.

Compare A with B to isolate MMOD. A selects constant `k = ALP = 128`, independently of sample alpha. Both halves therefore become `(128, 0, 127)`: even the left half, whose output alpha is zero, retains this purple RGB. B selects `k = min(2*A1, 255)`, where A1 is circuit 1's sample alpha. Alpha 0 leaves blue; alpha 128 saturates k to 255 and gives full red, not half-strength red.

| Phase | Left RGB | Right RGB | Left output alpha | Right output alpha |
|---|---|---|---|---|
| A | `(128, 0, 127)` | `(128, 0, 127)` | 0 | 128 |
| B | `(0, 0, 255)` | `(255, 0, 0)` | 0 | 128 |
| C | `(0, 255, 0)` | `(255, 0, 0)` | 0 | 128 |
| D | `(0, 0, 255)` | `(255, 0, 0)` | 128 | 128 |

Compare B with C for SLBG: the green background replaces circuit 2's blue RGB. Compare B with D for AMOD: RGB stays the same, while output alpha changes from circuit 1's split values to circuit 2's constant 128. The pictures illustrate RGB composition; the table gives the independently selected output alpha. Use B as the reference when examining either control separately.

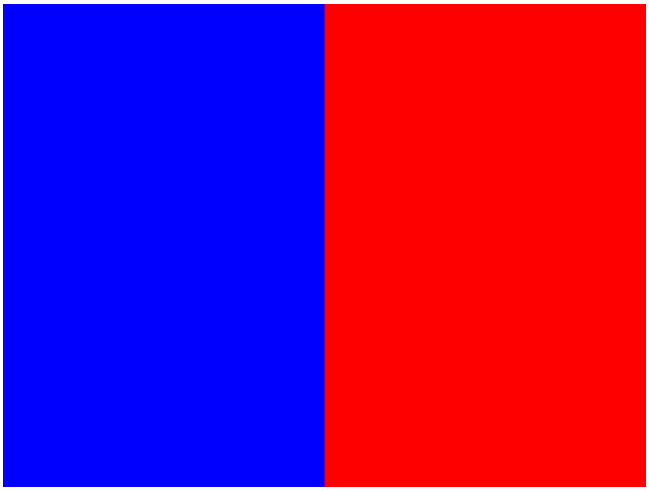
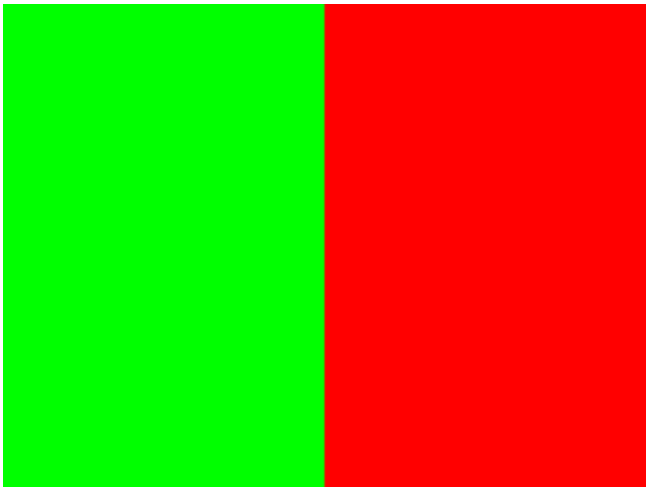
Prepare the two 256×192 sources in raw VRAM rows 0–191 and 256–447, with pitch 1024 words and both source origins (0,0). Circuit 2 uses FBP=64, base word 40000H. Keep both images unchanged while stepping through the four PMODE settings, and serialize other users of the display registers and GP0 port. Both regions lie outside the queried BIOS reservation.

After each PMODE write, acknowledge stale VBlank pending state and wait for the VBlank counter to advance, using the sequence below. This establishes that the display latch has taken the new composition setting. E5 is used while drawing the source images; C0 reads their absolute raw coordinates. Such reads return the stored words 001FH, 801FH and FC00H. To predict the displayed composite, apply the PMODE equation to the source colors and alpha values.



Figure 3.8: A: uniform purple output

*A — PMODE=8023H: constant mixing of red source 1 and blue source 2 produces uniform RGB (128,0,127), despite source 1's split alpha.*

| B — source-alpha selection | C — background selection |
|---|---|
|  |  |
| *PMODE=8003H: source 1's alpha split yields blue source 2 on the left and red source 1 on the right.* | *PMODE=8083H: SLBG substitutes green background for blue source 2; source 1's right half stays red.* |

### 3.7.1 Program the composition comparison

Use the mapped GP0/GP1 ports and VBlank handler above. Under exclusive ownership of both circuits and the two surfaces, mask delivery, request PCRTC RESET and keep PMODE zero. Program these framebuffer and window words; both source origins are zero:

| Register | Address | Word |
|---|---|---|
| DISPFB1_LOW / HIGH | `0801035C` / `08010360` | `000FA000` / `00000000` |
| DISPLAY1_LOW / HIGH | `08010364` / `08010368` | `018252A8` / `000BF3FF` |
| DISPFB2_LOW / HIGH | `0801036C` / `08010370` | `000FA040` / `00000000` |
| DISPLAY2_LOW / HIGH | `08010374` / `08010378` | `018252A8` / `000BF3FF` |
| BGCOLOR_LOW | `0801037C` | `0000FF00` |

Close the GP1 Y-address gate with `09000000H`. Establish E1=`0400H`, E2=0, drawing area `(0,0)…(255,191)` and zero drawing offset, then prepare both raw sources using complete GP0 packets:

```
-- Red, left half with bit15 clear; right half with bit15 set.
*gp0 = 0xe6000000
*gp0 = 0x600000ff
*gp0 = 0
*gp0 = 128 | (192 << 16)
*gp0 = 0xe6000001
*gp0 = 0x600000ff
*gp0 = 128
*gp0 = 128 | (192 << 16)

-- Blue with bit15 set, in the second framebuffer at raw rows 256..447.
*gp0 = 0xe3040000
*gp0 = 0xe406fcff
*gp0 = 0xe5080000
*gp0 = 0x60ff0000
*gp0 = 0
*gp0 = 256 | (192 << 16)
while (*gp1 & 0x04000000) == 0 do
end
```

The red source words are 001FH and 801FH; the blue source word is FC00H. E5 affects these drawing coordinates, whereas a C0 read takes absolute raw coordinates. No source pixels change during the four comparisons.

To select any one phase, use its PMODE word from the table in the same masked publication sequence shown for stage A. For example, phase C is:

```
*irq_mask = 0
*pmode = 0x00008083
*irq_ack = 4
local before<const> = *vblank_sequence
*irq_mask = 4
while *vblank_sequence == before do
    halt_until_irq
end
```

Observe that phase before selecting another. Immediate PMODE readback confirms the programmed word only. The output plates show RGB; compare the separate alpha results in the table when distinguishing phases B and D.

# 4 Geometry Engine (GTE)

## 4.1 Function and data flow

The Geometry Engine performs the coordinate and color calculations used to prepare three-dimensional drawing. It transforms and projects vertices, calculates lighting and depth-cued colors, tests projected winding, and forms scaled depth averages. Its integer datapaths use programmable scales, shifts and saturation so that a program can choose its coordinate range and precision.

For a projected primitive, load its vertices and the model-to-view matrix, write a projection command, then read the screen-coordinate and depth histories. Lighting commands use the supplied normals, light matrix, color matrix and material color. Software collects the screen positions and colors into GPU packets. Near-plane clipping, primitive ordering and packet submission remain program operations.

## 4.2 From a vertex to a screen position

Projection needs a vertex's position relative to the viewing origin and axes. For this example, choose +X to the right, +Y downward, and +Z forward from that origin. Use one spatial unit per coordinate count. The vertex `V=(64,32,512)` is therefore 64 units to the right, 32 units down, and 512 units in front of the viewing origin. These axes and units are choices made by the program.

### 4.2.1 Transform into the viewing coordinates

Vertices defined in model coordinates are measured from the model's own origin. A supplied matrix and translation express them in the chosen viewing coordinates. The matrix sets how the model axes contribute to the view axes; the translation places the model origin in that view. GTE applies this transform before perspective.

Keep the first example simple: its input already uses the viewing origin and axes. An **identity matrix** and zero translation therefore leave `(64,32,512)` unchanged. More elaborate model placement and viewing transforms use the same path; Coordinate spaces and the supplied transform explains how to derive them.

### 4.2.2 Depth determines the projected displacement

Perspective converts the transformed X and Y into a displacement on the drawing plane. In geometric projection, multiplying by **H/depth** gives that displacement. The projection numerator **H** therefore sets magnification: increasing H enlarges the displacement, while increasing positive depth with X and Y unchanged reduces it. GTE evaluates this relation with integer reciprocal arithmetic.

Choose H=256. At depth 512, GTE produces a projection factor of exactly one half, so the transformed X and Y are halved:

```
projection factor   = 256/512 = 1/2
screen displacement = (64,32) × 1/2 = (32,16)
```

Z is still needed even though a screen position has only two components: it supplies the depth that determines this factor. GTE also retains the depth as a result for later geometric decisions. The exact reciprocal calculation, rounding and result limits are specified under Perspective and geometric decisions and in the command reference; the values in this first example are exact and remain within those limits.

### 4.2.3 Place the projected origin on the screen

The displacement `(32,16)` is measured from the projection axis, not from the screen's coordinate zero. Choose a screen offset of `(128,96)` to place that axis, then add the displacement:

```
screen position = (128,96) + (32,16) = (160,112)
```

OFX and OFY hold this screen placement. They are applied after perspective and do not move the vertex in space or change its depth. A spatial translation, by contrast, changes the viewing coordinates before perspective; its screen effect depends on depth. The two settings serve different purposes even when they happen to move one vertex by the same amount.

The complete calculation is:

| Stage | Value | Meaning |
|---|---|---|
| Input position V | `(64,32,512)` | Position relative to the chosen viewing origin and axes |
| Identity transform, zero translation | `(64,32,512)` | The same position in view coordinates |
| Perspective with H=256 | `(32,16)` | Displacement from the projected origin at depth 512 |
| Screen offset `(128,96)` | `(160,112)` | Final screen position; depth remains 512 |

The next sections encode these same choices as register words and issue one projection command.

## 4.3 Programming geometry

The worked projection chose spatial units, a model-to-view transform, H, and a screen offset. GTE retains these settings across commands, so successive vertices can share one setup. Encode the settings first, load a vertex, and write the projection command last. In the register notation below, D[n] identifies data word n and C[n] identifies control word n.

V0, V1 and V2 are three independent input-vector slots, each holding an `(X,Y,Z)` triple. Every component is a signed 16-bit integer. X and Y share one word; Z occupies the low half of the next. A coordinate count represents the spatial unit chosen by the program, which may be a whole model unit or a fraction of one.

### 4.3.1 Encode spatial and screen quantities

The quantities in the worked projection use different scales in their register words. Store positions and translations in the same integer spatial units, multiply matrix coefficients by 4096, and select `SF=1`. The command removes twelve low bits from each accumulated matrix result. Without overflow or saturation, the resulting IR coordinates are the transformed positions rounded toward negative infinity.

| Quantity under this convention | Programmed representation | Representable range | Consumer |
|---|---|---|---|
| Input position X, Y, Z | Signed halfword; one raw count per coordinate unit | −32768–32767 coordinate units | RTPS, RTPT, or a selected MVMVA vector |
| Matrix coefficient | Signed halfword; coefficient × 4096 | −8 to `8-1/4096` | Matrix dot product; use `SF=1` to remove this factor |
| Translation X, Y, Z | Signed complete word; one raw count per coordinate unit | −2147483648–2147483647 coordinate units | Translation is multiplied by 4096 before the dot-product additions |
| H | Unsigned low halfword; unscaled projection numerator whose value sets screen magnification | 0–65535 | Projection numerator, compared with the new unsigned depth |
| OFX, OFY | Signed complete word; screen-coordinate offset × 65536 | −32768 to `32768-1/65536` screen units | Added before the final sixteen-bit screen shift |

Choose coordinate ranges with allowance for every stage: dot products wrap at 44 bits, visible MAC words wrap at 32 bits, and IR coordinates are limited to signed 16-bit results. A translation that fits its control word can still overflow or saturate a later stage.

The identity matrix has `m00=m11=m22=4096` and all other coefficients zero. With `SF=1`, it leaves an input coordinate unchanged before translation. The coefficient 1 is `0x1000`, ½ is `0x0800`, −½ is the halfword `0xf800`, and −1 is `0xf000`.

Select `LM=0` for the signed projection convention used here. It permits negative transformed X and Y. `LM=1` instead limits negative IR results to zero; projection would then use those zeroes in its screen calculation. The command's LM selection is a lower-limit choice, not a clipping enable.

## 4.4 A first projection in BLua

The following sequence programs the calculation just derived. Start from reset-zero state, retaining zero translation and zero off-diagonal matrix coefficients. Store 4096 in the three diagonal coefficients to encode the identity matrix. OFX and OFY contain the chosen screen offset multiplied by 65536; H is the unscaled numerator 256.

Use input slot V0 for `V=(64,32,512)`. Its X=64 occupies the low half of D[0] and Y=32 the high half, giving `0x00200040`; Z=512 occupies the low half of D[1]. The register windows are addressed in 32-bit words. RTPS transforms and projects this one input; `SF=1` removes the matrix coefficient scale, and `LM=0` permits signed transformed coordinates.

```
local data<const>: *word = 0x08010244
local control<const>: *word = 0x080102c4
local command<const>: *word = 0x08010344
control[0] = 0x00001000             -- C[0]: m00=4096, m01=0
control[2] = 0x00001000             -- C[2]: m11=4096, m12=0
control[4] = 0x00001000             -- C[4]: m22=4096
control[24] = 128 << 16
control[25] = 96 << 16
control[26] = 256
data[0] = 0x00200040                  -- V0: X=64 low, Y=32 high
data[1] = 512                         -- V0: Z=512
*command = 0x00080001                 -- RTPS, SF=1, LM=0
local flags<const> = control[31]
local screen<const> = data[14]        -- SXY2 = 0x007000a0: (160,112)
local depth<const> = data[19]         -- SZ3 = 512
```

D[14] contains the predicted screen position `(160,112)`, D[19] retains depth 512, and FLAG is zero. The normal command's results are already available when its store completes; do not poll normal CYCLES as a busy flag. Reading these words does not draw a pixel. To draw, place the screen word in a GPU packet; to display, select the resulting VRAM area through PCRTC. Retain FLAG and any needed results before a later command replaces them.

## 4.5 Technical specifications

GTE storage consists of registers for operands, coefficients, intermediate values and recent results. Keep vertex lists and drawing packets in the installed 4 MiB main RAM or cartridge ROM as appropriate. GPU local memory provides the separate 2 MiB drawing and texture store. The CPU transfers words between these resources and GTE's register interface.

The normal command path has 32 data words, `D[0]`–`D[31]`, and 32 control words, `C[0]`–`C[31]`. Controls retain the matrices and coefficients shared by successive commands. Data words hold vectors, arithmetic results and the screen, depth and color FIFOs. Each FIFO retains a short history of results, allowing a triple command to leave three vertices or colors available together. Its entries also accept direct register writes.

GTE_PLUS is a separate ten-word, three-lane multiply-add unit. It has its own operands, results, flags and command status. Use it for a signed vector plus a scaled vector; transfer its results explicitly when they are needed as normal GTE operands.

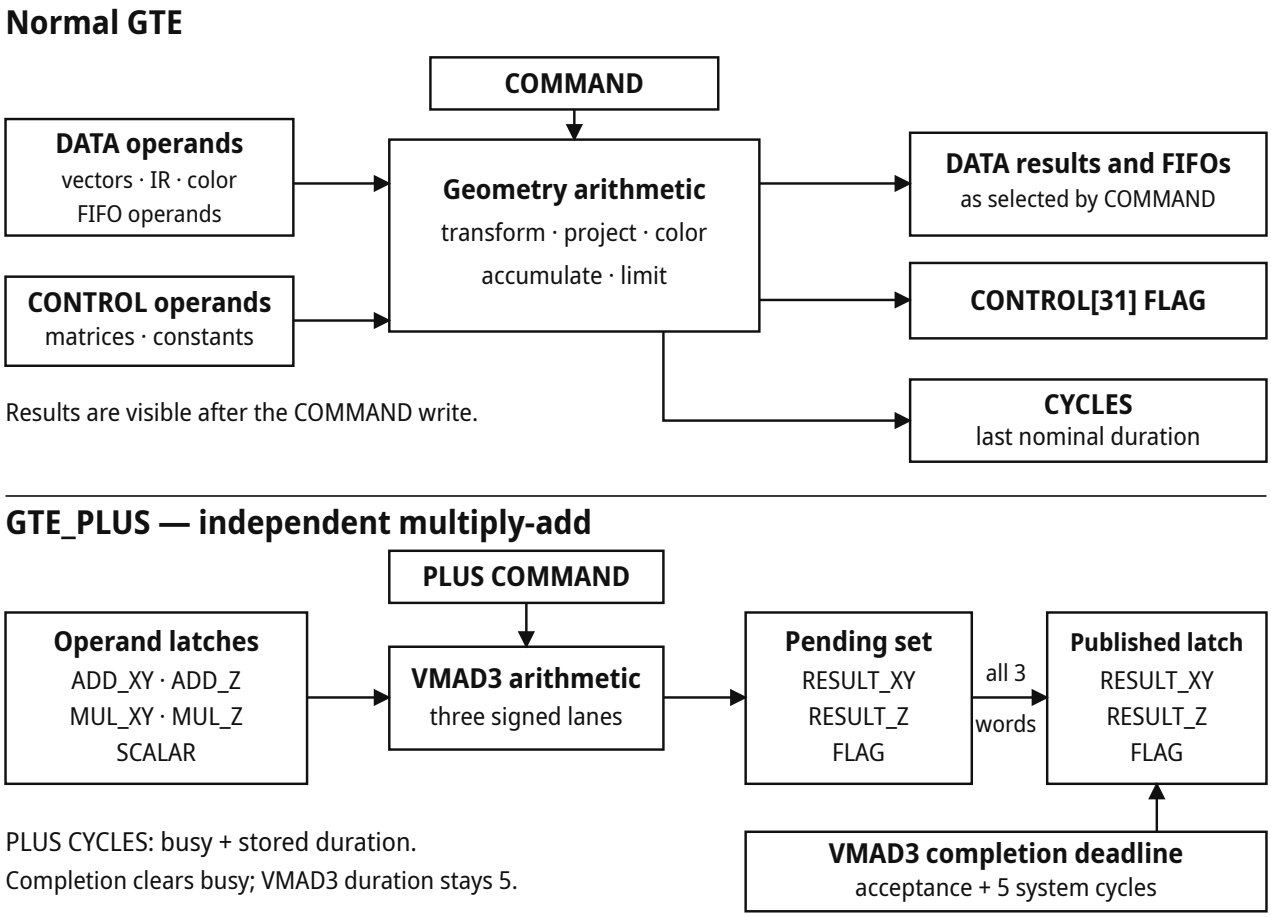



Figure 4.1: GTE register and arithmetic paths

### 4.5.1 Capacity and arithmetic

| Item | Specification |
|---|---|
| Installed CPU/system-cycle frequency | 33,868,800 Hz (33.8688 MHz) |
| Programming transfer width | One 32-bit register word at a four-byte-aligned address |
| Normal data register window | 32 words, D[0]–D[31]; 128 bytes of register address space |
| Normal control register window | 32 words, C[0]–C[31]; 128 bytes of register address space |
| Normal command/status | Separate 32-bit COMMAND and CYCLES words |
| PLUS register window | Ten 32-bit words; 40 bytes of register address space |

Some register slots provide aliases or conversion results. Others sign-extend their low halfword or ignore writes. The register reference specifies each port.

| Arithmetic resource | Capacity and representation |
|---|---|
| Input vectors | Three XYZ vectors, each with signed 16-bit components |
| Stored matrices | Three 3 × 3 matrices with signed 16-bit coefficients |
| Control vectors | Three XYZ vectors with signed 32-bit components |
| Main multiply-accumulate path | Three signed 44-bit accumulators, with shifted 32-bit visible MAC words and 16-bit clamped IR results |
| Screen history | Three packed SX/SY pairs; each component occupies a signed 16-bit halfword; projected coordinates clamp to −1024…1023 |
| Depth history | Four unsigned 16-bit depths |
| Color history | Three words, each containing 8-bit red, green, blue and CODE fields |

| Arithmetic resource | Capacity and representation |
|---|---|
| PLUS multiply-add path | Three signed 16-bit lanes, a signed 16-bit scalar, a fixed 12-bit product shift, and saturated signed 16-bit results |

Normal commands provide vector transformation and projection, matrix/vector arithmetic, lighting and color calculations, doubled oriented screen area, and depth averages. There are 22 recognized normal functions. Multiplication, accumulation, shift and saturation are integer operations. The normal path's 44-bit wrap and its IR/output clamps are distinct stages. PLUS uses the separate multiply-add rule given in its command entry.

### 4.5.2 Completion and throughput

The two command interfaces indicate completion as follows:

| Path | Result visibility | Admission of subsequent work |
|---|---|---|
| Normal COMMAND | The command's results are visible when the command write completes | No GTE busy wait or command queue; the next normal command may use those results |
| PLUS VMAD3 | One result set becomes available five system cycles after acceptance | One pending command; another direct CPU command store waits for completion |
| Invalid PLUS command | Previous result values are preserved; ERROR and INVALID publish after one system cycle | Occupies the same single pending-command position |

The PLUS arithmetic admission ceiling is one VMAD3 command per five system cycles: 6,773,760 commands per second at the installed frequency, with three lane results per command. Include operand writes, command issue and result reads when calculating the rate of a complete program sequence.

Normal CYCLES reports a nominal count from 5 through 44. Results are already available when the command write completes; the issuing instruction pays its ordinary CPU cost. Timing and bus programming and the complete work-interval example explain how to budget transfers and arithmetic together.

Normal commands may be used while PLUS VMAD3 is pending. Save results before issuing work that will replace them, and inspect FLAG before using an overflowed or saturated result in a geometric decision.

## 4.6 Coordinate spaces and the supplied transform

Before programming a transform, decide what a coordinate means. The triple `(64,32,0)` does not identify a location by itself: it needs an origin, three axis directions, and a unit. The same vertex can have different triples when measured from an object's origin, from a level origin, and from a chosen viewing position. These are different **coordinate spaces**, not different kinds of GTE register.

### 4.6.1 Choose the spaces in software

A useful division is:

| Space | What its coordinates describe | Who chooses it |
|---|---|---|
| **Model**, also called local | A vertex relative to the model's own origin and axes. An origin at a centre, corner, or hinge gives the same stored geometry a different placement rule. | The cartridge's geometry and transform code. |
| **World** | A position relative to a common origin and axes used to place objects together. Two models can have different local origins but share this reference. | The cartridge. World coordinates need not be sent to GTE. |

| Space | What its coordinates describe | Who chooses it |
|---|---|---|
| **View** | A position relative to a chosen viewing origin, expressed along chosen viewing axes. These are the spatial coordinates to which perspective is applied. | The cartridge supplies the transform that produces them. |
| **Screen** | A two-component position after perspective, screen offset, rounding, and limiting. It is no longer a three-dimensional displacement. | The cartridge chooses H and offsets; GTE fixes the arithmetic and result limits. |

An intermediate world space is useful for reasoning about a scene, but it is not a required hardware stage. The input can be model coordinates transformed directly to view coordinates. Alternatively, software can supply vertices already in view coordinates and program an identity transform. V0, V1, and V2 are three input-vector slots; they do not mean model, world, and view.

For the examples below, choose one raw coordinate count per spatial unit, view +X towards screen right, view +Y towards screen down, and view +Z farther in front of the viewing origin. Also choose to display increasing screen X to the right and increasing screen Y downward. These are **software conventions**. The projection equations add the X and Y contributions without an implicit Y sign change. If a model uses Y upward, its supplied transform must account for that choice. Define the viewing axes and handedness when constructing the transform. Use positive depth for projection, then apply the scene's clipping and visibility decisions to the projected primitive.

### 4.6.2 Read the matrix as three dot products

Projection uses matrix 0, C[0]–C[4], and translation C[5]–C[7]. The matrix contains nine signed 16-bit coefficients; the translation components are signed 32-bit words. Use `m[r,c]` for the raw coefficient in row r and column c. The three accumulations are:

```
A_X = 4096*TRX + m[0,0]*X + m[0,1]*Y + m[0,2]*Z
A_Y = 4096*TRY + m[1,0]*X + m[1,1]*Y + m[1,2]*Z
A_Z = 4096*TRZ + m[2,0]*X + m[2,1]*Y + m[2,2]*Z
```

The **first row produces X**, the second Y, and the third Z. The products in a row are not three successive transformations; they are contributions to one output component.

For this section, select `SF=1` and `LM=0`, and interpret each matrix coefficient as its raw value divided by 4096. Let this interpreted matrix be R, and let t be the three translation words in the same spatial units as the input. Ignoring rounding and limits for the moment, the equations read:

```
[ X_view ]   [ R00 R01 R02 ] [ X_model ]   [ tX ]
[ Y_view ] = [ R10 R11 R12 ] [ Y_model ] + [ tY ]
[ Z_view ]   [ R20 R21 R22 ] [ Z_model ]   [ tZ ]
```

This is the **column-vector convention**: R multiplies the input column on its left. It follows from the actual dot products, not from the order in which the packed words are stored. Using row-vector formulas from another mathematics library without transposing the coefficients would describe a different mapping.

Rows and columns give two complementary ways to understand the same matrix:

- A **row** says how input X, Y, and Z contribute to one output coordinate.
- A **column** says where one input-axis displacement goes. An input displacement `(d,0,0)` produces `d*(R00,R10,R20)`; `(0,d,0)` uses the second column, and `(0,0,d)` uses the third.

Thus the columns are the transformed input basis vectors. They need not be perpendicular or of unit length: the programmed matrix can scale, mix, reflect, or collapse axes as well as rotate them. GTE does not repair it into a rotation. The input origin maps to t. A displacement between two points has no translation term, since `(R*p2+t) - (R*p1+t) = R*(p2-p1)` in this ideal arithmetic. That is why moving the origin and changing the basis are separate parts of an affine transform. Small basis probes may themselves be rounded by the integer datapath; use a suitable displacement when checking a fractional coefficient.

GTE executes one supplied transform per projected vertex. It does not receive a model placement followed by a second, hidden camera transform. If software keeps those transformations separately, it must supply their combined matrix and translation, or do an earlier transformation explicitly. RTPT applies the same programmed transform to its three input slots; it does not interpret them in three spaces.

### 4.6.3 A view origin is subtracted, not added

To make the sign unambiguous, define the following software convention. Let `p_world = R_model*p_model + b`, where b is the model origin's world position. Let c be the chosen viewing origin in world coordinates. Let R_view convert a world-axis displacement into the chosen view axes. Then define:

```
p_view = R_view * (p_world - c)
```

The subtraction is necessary because the viewing origin itself must have view coordinates zero: substituting `p_world=c` gives zero. The view-only translation is consequently `-R_view*c`, **not generally `-c`**. The latter works only when R_view leaves that particular vector unchanged. For example, if R_view maps `(X,Y,Z)` to `(-Y,X,Z)`, a viewing origin `(120,56,256)` requires view-only translation `(56,-120,-256)`.

Substituting the model placement into the chosen view definition gives the single transform that projection needs:

```
p_view = (R_view*R_model)*p_model + R_view*(b-c)
```

This identifies the meaning of the supplied matrix and translation; it is not a claim that multiplying separately quantized transforms is exact. Construction, packing, and intermediate precision are separate programming steps. In particular, copying a camera's world position into TR would implement the wrong operation under this convention.

The first projection used an input already in view coordinates. To see how a model placement and viewing origin combine, now use a different vertex, `(64,32,0)`, with a rotated model. Choose:

```
model-axis mapping: (X,Y,Z) -> (-Y,X,Z)
model world origin b = (200,40,768)
viewing origin c    = (120,56,256)
view axes          = world axes, so R_view is identity
```

The combined matrix therefore has rows `(0,-4096,0)`, `(4096,0,0)`, and `(0,0,4096)` in raw coefficient units. Its translation is:

```
t = b-c = (80,-16,512)
TRX = 0x00000050   TRY = 0xfffffff0   TRZ = 0x00000200
```

The following probes show both the origin and the basis. Their values are exact with `SF=1`, `LM=0`; none reaches an arithmetic limit.

| Model coordinates | View coordinates | Meaning |
|---|---|---|
| `(0,0,0)` | `(80,-16,512)` | The model origin appears at the supplied translation. |
| `(64,0,0)` | `(80,48,512)` | Local +X adds 64 along view +Y: the first column is `(0,1,0)`. |
| `(0,64,0)` | `(16,-16,512)` | Local +Y subtracts 64 along view X: the second column is `(-1,0,0)`. |
| `(0,0,64)` | `(80,-16,576)` | Local +Z adds 64 along view +Z: the third column is `(0,0,1)`. |

For the actual vertex `(64,32,0)`, model placement gives world coordinates `(-32+200,64+40,0+768) = (168,104,768)`. Subtracting c gives view coordinates `(48,48,512)`. GTE obtains those same coordinates directly from the model input:

```
A_X = 4096*80  + 0*64    - 4096*32 + 0*0 = 196608
A_Y = 4096*-16 + 4096*64 + 0*32    + 0*0 = 196608
A_Z = 4096*512 + 0*64    + 0*32    + 4096*0 = 2097152

(A_X,A_Y,A_Z) ASR 12 = (48,48,512)
```

Moving only the viewing origin by +32 along world X changes this point's view X from 48 to 16. The scene moves the opposite way relative to that origin. No GTE camera state changes: software updates the supplied translation.

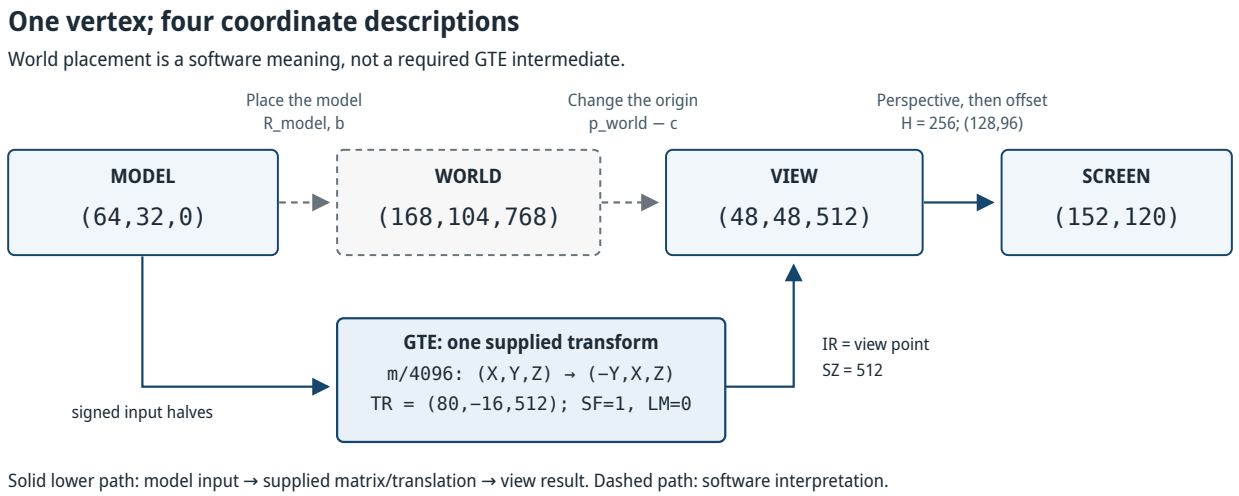



Figure 4.2: The rotated-model example: vertex (64,32,0) becomes view position (48,48,512) and screen position (152,120). World coordinates are a software interpretation; the supplied GTE transform maps model input directly to view output.

### 4.6.4 Screen placement is not spatial translation

The view-space origin and the projected screen origin answer different questions. TR moves geometry before perspective. OFX and OFY place the zero-X/zero-Y projection axis on the screen **after** perspective. They do not change transformed coordinates or depth.

With q the unit's reciprocal-table quotient, the screen stage is:

```
SX = clamp((OFX + IR1*q) ASR 16, -1024, 1023)
SY = clamp((OFY + IR2*q) ASR 16, -1024, 1023)
```

IR1 and IR2 are already limited spatial results. The quotient has 65536 counts per unit ratio; it approximates `65536*H/depth`, but the specified reciprocal calculation, not ideal division, determines the actual bits. Program an offset of `(128,96)` as `OFX=0x00800000`, `OFY=0x00600000`. With H=256 and depth 512, the quotient is exactly 32768. The example vertex `(48,48,512)` then becomes `(128+24,96+24) = (152,120)` on the screen. The choice `(128,96)` is an example projection centre, not a built-in display centre or a framebuffer address.

A second example exposes the difference between the two kinds of movement. Take transformed X=64 at two depths, with H=256 and screen X offset 128. Compare adding 32 spatial units through TRX with adding 16 screen units through OFX:

| Depth | Original SX | Add 32 to TRX | Add `16*65536` to OFX |
|---|---|---|---|
| 512, q=32768 | 160 | 176: a 16-unit screen move | 176: a 16-unit screen move |
| 1024, q=16384 | 144 | 152: an 8-unit screen move | 160: a 16-unit screen move |

The spatial shift is diminished by distance. The screen offset moves both by the same amount. These exact cases are unsaturated; at the screen limits, a further offset can leave an already limited result unchanged. Changing TRZ is different again: it changes depth and therefore magnification, not just the position of the projection axis. Use TR for placement in the chosen spatial frame, and OFX/OFY for placement of that frame's projection on the drawing plane.

### 4.6.5 Keep the coordinate convention within the datapath

`SF=0` leaves the dot-product result unshifted. It does not change the input words into a different universal numeric format. In particular, projection depth always comes from `A_Z ASR 12`, independently of SF. Switching the Q12 identity above to `SF=0` multiplies its visible X/Y/Z results by 4096 before the IR limits, but does not multiply the depth FIFO result by 4096. It is not an integer-coordinate projection shortcut.

A useful coordinate system must fit the arithmetic, not merely have convenient names. Under the convention used here:

- Input components have signed 16-bit range `-32768..32767`. Choosing sixteen raw counts per game unit improves input resolution to 1/16 unit but reduces the corresponding range to `-2048..2047 15/16` game units. Positions and translations must use the same chosen spatial unit when R is an ordinary dimensionless orientation/scale matrix. A unit conversion can be included in R, but it must be deliberate.
- Coefficients have resolution 1/4096 and interpreted range `-8..8-1/4096`. There is no automatic normalization. If software rounds an ideal coefficient to the nearest Q12 integer and that rounded integer fits the signed halfword, each coefficient's error is at most 1/8192. Its contribution to a row's spatial error grows with the input magnitude: the coefficient-only error bound is `(|X|+|Y|+|Z|)/8192` counts before the final integer shift. Repeatedly changing coordinate frames also requires an explicit intermediate-precision policy.
- SF=1 discards twelve low accumulator bits by arithmetic shift, rounding towards negative infinity. A raw coefficient 2048 (`0x0800`, meaning 1/2) maps input X=3 to 1 and X=-3 to -2 in an otherwise zero X row. It does not preserve the fractional outputs +1.5 and -1.5. Neither the extra width of TR nor a later screen offset recovers those lost bits.
- Account for the narrower result stages when choosing translation. The dot products accumulate with ordered signed 44-bit wrap, visible MAC words wrap to 32 bits, and the IR values are limited to signed 16 bits for LM=0. Projection uses the limited IR1/IR2, so a large translated X or Y can distort the projected position even when the input vertex fits. Keep large world positions in the cartridge's world representation and supply their appropriately computed view-relative transform rather than assuming GTE can retain a large unsaturated world-space intermediate.
- Projection depth comes from the retained Z accumulation shifted twelve bits, limited to `0..65535`; it is not read back from limited IR3. This depth rule does not change with SF. For positive H, ordinary non-overflow projection needs depth greater than H/2. A zero or negative spatial depth does not continue signed mathematical perspective behind the viewing origin: depth limiting and the divide-overflow path still produce results.

Finally, screen pairs are limited to `-1024..1023`, not to the cartridge's chosen display rectangle. GTE does not turn any of these bounds into geometric clipping. Software must decide which spatial geometry to process and what to do with flagged or out-of-window results. Choosing origins near the geometry, consistent units, and sufficient precision is part of supplying a meaningful transform; it is not work performed by a hidden camera or normalization stage.

## 4.7 Constructing and packing a transform

A transform is ready to program only when every output coordinate can be written as a row equation. Start with the intended mapping, derive its nine coefficients and three translation components, then encode and pack them. This separates a mathematical mistake from a halfword-packing mistake. C[n] and D[n] identify the destination registers in the following tables.

This example constructs one model-to-view transform directly. It uses the previous section's column vectors, one raw count per spatial unit, +X to the right, +Y downward, and positive Z in front of the chosen viewing origin. It does not require a world-space intermediate or a separately programmed camera.

### 4.7.1 Read the packed matrix words

Matrix coefficients are packed in row order, **two successive coefficients per word**; word boundaries do not coincide with row boundaries. The following examples show all five words for two transforms. Translation is separate.

| Control word | Identity | Mapping `(X,Y,Z)` to `(−Y,X,Z)` |
|---|---|---|
| C[0]: m00 low, m01 high | `0x00001000` | `0xf0000000` |
| C[1]: m02 low, m10 high | `0x00000000` | `0x10000000` |
| C[2]: m11 low, m12 high | `0x00001000` | `0x00000000` |
| C[3]: m20 low, m21 high | `0x00000000` | `0x00000000` |
| C[4]: m22 low | `0x00001000` | `0x00001000` |

For the second matrix, `SF=1`, zero translation, and `(X,Y,Z)=(64,32,512)` give `(−32,64,512)`. The negative coefficient is a signed halfword inside a packed word; it is not a negative floating-point value.

### 4.7.2 Define the turn, scale, and translation

Choose a software XY-plane angle convention in which a positive angle maps `(1,0,0)` to `(cos(theta),sin(theta),0)`. Its equations are:

```
X_rotated = cos(theta)*X - sin(theta)*Y
Y_rotated = sin(theta)*X + cos(theta)*Y
Z_rotated = Z
```

With Y displayed downward, this positive turn appears clockwise on the screen. That observation follows from the chosen axes; it is not an angle convention imposed by GTE. The unit has coefficient inputs, not sine, cosine, or angle inputs. Software chooses the angle convention and obtains the coefficients.

Use the exact ratios `cos(theta)=3/5` and `sin(theta)=4/5`. Their squares sum to one, so they specify a unit direction for the turned +X axis. They describe an oblique turn of approximately 53.13 degrees, not a quarter-turn or an axis swap. Before this turn, enlarge local X by 5/4 and local Y by 5/2, leaving local Z unchanged. Finally, place the transformed model origin at view position `(40,-24,512)`.

The order matters: these are scales along the **local** axes before they turn. Substitute the scaled components into the rotation equations:

```
X_view = (3/5)*((5/4)*X) - (4/5)*((5/2)*Y) + 40
       = (3/4)*X - 2*Y + 40

Y_view = (4/5)*((5/4)*X) + (3/5)*((5/2)*Y) - 24
       = X + (3/2)*Y - 24

Z_view = Z + 512
```

These equations completely specify the transform. The two scaling operations and the rotation were steps in deriving its coefficients; they are not three commands that projection must secretly execute. In interpreted coefficient units, the supplied matrix and translation are:

```
    [ 3/4   -2    0 ]         [  40 ]
R = [  1    3/2   0 ]     t = [ -24 ]
    [  0     0    1 ]         [ 512 ]
```

The last matrix column multiplies input Z. It is **not** the translation column of a two-dimensional homogeneous matrix. All three translation components have separate registers. Translation is added after the linear mapping: it is neither scaled nor turned again by these rows.

### 4.7.3 Encode coefficients, then pack successive halves

Use `SF=1`. Multiply each interpreted coefficient by 4096 to obtain its signed 16-bit stored coefficient. This example's products are all exact integers:

| Row | Interpreted coefficients | Raw signed coefficients | Halfword bit patterns |
|---|---|---|---|
| X | `(3/4,-2,0)` | `(3072,-8192,0)` | `(0x0c00,0xe000,0x0000)` |
| Y | `(1,3/2,0)` | `(4096,6144,0)` | `(0x1000,0x1800,0x0000)` |
| Z | `(0,0,1)` | `(0,0,4096)` | `(0x0000,0x0000,0x1000)` |

For a negative coefficient, encode the signed value in sixteen-bit two's complement. Here `-8192` becomes `65536-8192 = 57344 = 0xe000`. That halfword represents −2 in this coefficient convention. It does not represent the positive coefficient 14, and the packed word containing it is not a floating-point number.

Pack coefficients in row order, **two consecutive coefficients per word**. A word boundary need not be a row boundary:

| Control word | Low half, bits 15–0 | High half, bits 31–16 | Complete word to supply |
|---|---|---|---|
| C[0] | `m[0,0]=0x0c00` | `m[0,1]=0xe000` | `0xe0000c00` |
| C[1] | `m[0,2]=0x0000` | `m[1,0]=0x1000` | `0x10000000` |
| C[2] | `m[1,1]=0x1800` | `m[1,2]=0x0000` | `0x00001800` |
| C[3] | `m[2,0]=0x0000` | `m[2,1]=0x0000` | `0x00000000` |
| C[4] | `m[2,2]=0x1000` | Not another coefficient; supply zero here | `0x00001000` |

In particular, C[1] contains the **end of the X row and the start of the Y row**. Putting each new row at the low half of a new word would not program this matrix. For a low half a and high half b, the complete word consists of a's low 16 bits and b's low 16 bits shifted left 16 places. Keep each coefficient to its own half; sign extension of a negative coefficient must not fill its neighbour's bits.

The upper half of C[4] is not independent matrix storage. Its register write retains and sign-extends the low signed halfword. With this positive final coefficient, reading C[4] returns `0x00001000`.

The translations use **complete signed words** in the chosen spatial units:

| Control word | Meaning | Signed value | Complete raw word |
|---|---|---|---|
| C[5] | TRX | 40 | `0x00000028` |
| C[6] | TRY | −24 | `0xffffffe8` |
| C[7] | TRZ | 512 | `0x00000200` |

Do not multiply these values by 4096 before writing them. The row calculation already multiplies TR by 4096 before adding the coefficient products. That puts translation and products at the same accumulator scale; the selected twelve-bit shift then removes the common factor.

Complete all five matrix words and all three translation words before issuing geometry commands. Include the zero coefficients: matrix storage retains its previous contents, so an omitted zero does not erase an old coefficient. A batch of vertices using this transform can reuse the setup without reloading it for every vertex.

For an arbitrary chosen angle or scale, the desired coefficients may not be exact multiples of 1/4096. Software must choose its coefficient-rounding rule and check the resulting signed-halfword range. Do this after deriving the intended row coefficients; do not assume independently rounded trigonometric terms and intermediate products produce the same stored matrix. The example above deliberately has no coefficient-quantization error, making its packing and row calculations independently checkable.

### 4.7.4 Check the origin and basis before a complicated vertex

Use `LM=0` so signed transformed coordinates remain signed. Submitting the model origin should return the translation. Submitting a known displacement along each local axis then checks the columns without relying on the appearance of a whole model. Use 64-unit probes here so the quarter- and half-unit coefficients produce exact integer results:

| Local input | View output at SF=1, LM=0 | Displacement from the transformed origin |
|---|---|---|
| `(0,0,0)` | `(40,-24,512)` | `(0,0,0)` |
| `(64,0,0)` | `(88,40,512)` | `(48,64,0)` |
| `(0,64,0)` | `(-88,72,512)` | `(-128,96,0)` |
| `(0,0,64)` | `(40,-24,576)` | `(0,0,64)` |

The first XY displacement has length 80: a 64-unit local X probe was scaled by 5/4. The second has length 160: the same-length local Y probe was scaled by 5/2. Their dot product is `48*(-128) + 64*96 = 0`, so the turned axes remain perpendicular. These checks distinguish the intended unequal local scales from a uniform scale or a transpose of the matrix.

Notice the negative transformed X of the local Y probe. It is expected, not an invalid vertex: that axis points mostly towards negative view X. Selecting LM=1 would replace a negative IR1 with zero before screen projection, changing the mapping being tested. A small one-count probe can also hide a fractional coefficient through integer rounding; it is not a reliable substitute for the 64-unit probes in this example.

### 4.7.5 Follow a nonzero vertex through the actual rows

Now submit local vertex `(64,16,0)`. Its V0 input words are D[0]=`0x00100040` (X in the low half, Y in the high half) and D[1]=`0x00000000` (Z in the low half). Applying the programmed raw rows gives:

```
A_X = 4096*40 + 3072*64 - 8192*16 + 0*0
    = 163840 + 196608 - 131072 = 229376

A_Y = 4096*(-24) + 4096*64 + 6144*16 + 0*0
    = -98304 + 262144 + 98304 = 262144

A_Z = 4096*512 + 0*64 + 0*16 + 4096*0
    = 2097152
```

With `SF=1`, the shifted MAC values are `(56,64,512)`. They fit the IR limits, so IR1, IR2, and IR3 retain those same values with LM=0. The new depth is 512. No overflow or limiting has replaced the intended transform result.

To check screen placement as well, use H=256, OFX=`0x00800000` and OFY=`0x00600000`, corresponding to projection offset `(128,96)`. Set DQA and DQB to zero for this check; the final depth-cue calculation then gives IR0=0 without introducing a separate depth-cue setup. RTPS with SF=1 and LM=0 has normal command word `0x00080001`. At this depth its reciprocal-table quotient is exactly 32768:

```
SX = (128*65536 + 56*32768) ASR 16 = 156
SY = ( 96*65536 + 64*32768) ASR 16 = 128
```

The newest packed screen pair, D[14], is therefore `0x0080009c`: X=156 in the low signed half and Y=128 in the high signed half. FLAG is zero for this fully specified case. These are values to compare with the command result, not a claim that the command itself draws a point.

The origin and the two XY probes also produce a useful screen-space shape:

| Local point | Screen pair |
|---|---|
| Origin `(0,0,0)` | `(148,84)` |
| Local X probe `(64,0,0)` | `(172,116)` |
| Local Y probe `(0,64,0)` | `(84,132)` |

Joining those points would give a triangle whose two probe sides have lengths 40 and 80 screen units. Both lie at depth 512, so the common perspective factor is exactly 1/2; the side directions and unequal lengths expose the programmed turn and scale. Primitive construction and drawing remain software's next hand-off.

Finally, check the extent of the actual input geometry, not only these sample vertices. For the flat local box `-64<=X<=64`, `-64<=Y<=64`, `Z=0`, the row extrema give view X in `-136..216`, view Y in `-184..136`, and depth exactly 512. Under the same projection setup, its screen bounds are X=`60..236` and Y=`4..164`. These values fit this datapath without IR, depth, divide, or screen limiting. They do not define a GTE viewport or prove a larger model safe. The chosen model extent, stored coefficients, translation, and projection setup must be considered together.

## 4.8 Composing transforms

An object can be positioned relative to a parent whose own coordinates must be transformed again. A model placement can likewise be followed by a world-to-view mapping. Projection needs the resulting single transform, not a list of those relationships. Composition determines its coefficients and translation. In integer arithmetic it must also determine **where precision is discarded**.

### 4.8.1 Derive the order and the translated origin

Write the local transform as `(B,b)` and the transform applied after it as `(A,a)`. With column vectors:

```
u = B*p + b
v = A*u + a
  = A*(B*p+b) + a
  = (A*B)*p + (A*b+a)
```

Thus the combined matrix is `A*B`, and its translation is `A*b+a`. The matrix nearest the input acts first. Merely adding a and b would leave the local origin unturned and unscaled by A. Multiplying `B*A` would reverse the linear operations.

For row r and column c, the combined coefficient is:

```
C[r,c] = A[r,0]*B[0,c] + A[r,1]*B[1,c] + A[r,2]*B[2,c]
```

This is a row of A dotted with a **column** of B. Equivalently, transform each of B's three columns by A, with zero translation. The three resulting vectors are the three columns of C. Transforming b with translation a gives the combined origin. These observations let MVMVA perform the products even though GTE has no matrix/matrix command or destination-matrix register.

For view/model composition under the earlier convention, substitute `A=R_view`, `a=-R_view*c`, `B=R_model`, and `b=model_world_origin`. The ideal translation is then `R_view*(b-c)`. It is the model origin expressed relative to the viewing origin and along the viewing axes, not the camera position copied into TR.

### 4.8.2 Work from the stored coefficients

Reuse the preceding section's parent linear mapping and translation:

```
        [ 3072  -8192     0 ]
A_raw = [ 4096   6144     0 ]    a = (40,-24,512)
        [    0      0  4096 ]
```

The subscript `raw` means signed coefficient integers with 4096 counts per unit coefficient. Introduce a local transform with the following stored values:

```
        [ 4096     0      0 ]
B_raw = [    0  2458  -3277 ]    b = (5,-3,7)
        [    0  3277   2458 ]
```

Its YZ coefficients approximate a chosen software turn with cosine 3/5 and sine 4/5: nearest coefficient rounding gives 2458 and 3277. The stored fractions `2458/4096` and `3277/4096`, not the ideal 3/5 and 4/5, are now the inputs to composition. The hardware does not recognize the intended angle or correct the rounded matrix into an exact rotation.

Choose this explicit storage policy for the combined transform:

- Accumulate each raw coefficient dot product before shifting. It has two factors of 4096, so shift twelve bits to produce one Q12 coefficient.
- Accumulate `4096*a + A_raw*b` before shifting. Shift twelve bits to produce an integer translation in the chosen spatial counts.
- Use arithmetic shifts, not rounding to nearest or truncation towards zero. Retain a signed16 matrix and signed32 translation only when the resulting values fit those representations and the preceding arithmetic has not wrapped.

For these small operands no wrap or limiting occurs. The nine unshifted matrix sums are:

```
A_raw*B_raw =
[ 12582912  -20135936   26845184 ]
[ 16777216   15101952  -20133888 ]
[        0   13422592   10067968 ]
```

For example, row 1, column 2 is `4096*0 + 6144*(-3277) + 0*2458 = -20133888`. Dividing by 4096 gives −4915.5; its arithmetic shift produces **−4916**, not −4915. Shifting every complete coefficient sum gives:

```
        [ 3072  -4916   6554 ]
C_raw = [ 4096   3687  -4916 ]
        [    0   3277   2458 ]
```

The translation calculation is separate:

```
tX sum = 4096*40    + 3072*5 - 8192*(-3) + 0*7 = 203776
tY sum = 4096*(-24) + 4096*5 + 6144*(-3) + 0*7 = -96256
tZ sum = 4096*512   + 0*5    + 0*(-3)    + 4096*7 = 2125824

t = (203776,-96256,2125824) ASR 12 = (49,-24,519)
```

Before the shift the translated origin is `(49.75,-23.5,519)`. The chosen integer TR representation loses those fractions now; this decision will affect later vertices. It is not a screen-offset adjustment.

### 4.8.3 Use MVMVA as three column products and one origin product

Keep A in matrix 0 and a in control vector 0 while computing the entire result. Place B's columns, not its rows, into the input slots:

| Input slot | Signed components | Packed XY word | Packed Z word |
|---|---|---|---|
| V0 | `(4096,0,0)` | `0x00001000` | `0x00000000` |
| V1 | `(0,2458,3277)` | `0x099a0000` | `0x00000ccd` |
| V2 | `(0,-3277,2458)` | `0xf3330000` | `0x0000099a` |

These inputs are coefficient columns in Q12 units. The same vector slots can hold positions in another command; their interpretation comes from the chosen calculation. Here multiplying by another Q12 matrix and selecting SF=1 leaves Q12 coefficient outputs, not integer spatial positions.

Select `mx=0`, `cv=3`, `SF=1`, and `LM=0`. Control-vector selector 3 supplies the zero vector, excluding a from the column products. The three input selections and results are:

| Vector selector | Normal command word | MAC1–MAC3 and IR1–IR3 | Destination in C |
|---|---|---|---|
| v=0 | `0x00086012` | `(3072,4096,0)` | Column 0 |
| v=1 | `0x0008e012` | `(-4916,3687,3277)` | Column 1 |
| v=2 | `0x00096012` | `(6554,-4916,2458)` | Column 2 |

Read and retain each column and its FLAG before the next command overwrites MAC, IR, and FLAG. All three flags are zero here. LM=0 is important: a negative coefficient is a legitimate result, not an intensity to clamp at zero. Do not use cv=2 as another ordinary affine control-vector selection: its specified two-pass calculation omits the CV and X contribution from the final result.

Next put b into V0: packed words `0xfffd0005` and `0x00000007`. Select `mx=0`, `v=0`, **`cv=0`**, `SF=1`, `LM=0`, normal word `0x00080012`. This time the initial term must include a. The result is `(49,-24,519)`, with FLAG zero. Read it before reusing the arithmetic registers.

Reassemble the retained columns into **rows** before packing the destination matrix. The complete result words are:

| Word | Value | Word | Value |
|---|---|---|---|
| C[0] | `0xeccc0c00` | C[5], TRX | `0x00000031` |
| C[1] | `0x1000199a` | C[6], TRY | `0xfffffe8` |
| C[2] | `0xeccc0e67` | C[7], TRZ | `0x00000207` |
| C[3] | `0x0ccd0000` | | |
| C[4] | `0x0000099a` | | |

Do not install the first result column over part of A while another product still needs A. Complete the three columns and translated origin in software-owned result storage, then install the finished matrix and translation. Also account for the input slots: coefficient columns and b have replaced any vertices that were previously loaded there. The original transforms and model vertices should remain in their normal cartridge data, available for subsequent command groups.

MVMVA itself leaves its selected matrix/control-vector words unchanged. It replaces MAC1–MAC3 and IR1–IR3 and starts a new FLAG; it does not push the screen, depth, or color FIFOs or update IR0/MAC0. Installing the completed transform is a separate set of writes. Keeping these roles explicit avoids mistaking a computed column in IR for an already updated matrix registerfile.

### 4.8.4 Composed and sequential integer transforms can disagree

The algebraic identity `A*(B*p+b)+a = (A*B)*p+(A*b+a)` is exact over real numbers. The integer command sequences have different places to discard fractions. Consider local vertex `(64,16,8)`.

If B and b are applied first with `SF=1`, `LM=0`, its complete row sums are `(282624,824,100768)`. The resulting integer IR vector is `(69,0,24)`. The Y coordinate's fraction and the Z coordinate's fraction have already been lost. Applying A and a to that IR vector gives:

```
X = (4096*40    + 3072*69 - 8192*0) ASR 12 = 91
Y = (4096*(-24) + 4096*69 + 6144*0) ASR 12 = 45
Z = (4096*512   + 4096*24) ASR 12 = 536
```

For this second command, MVMVA's v=3 selects the current signed IR vector; with mx=0, cv=0, SF=1, LM=0 its normal word is `0x00098012`. It snapshots all three input components before overwriting IR, so later rows do not accidentally consume earlier rows' outputs. This makes the two-command sequence well-defined, but does not restore the first command's lost fractions.

Now apply the stored C and t directly to the original vertex. Its row sums are `(371088,183504,2197920)`, giving **`(90,44,536)`**, not `(91,45,536)`. This path retains more of the vertex's local arithmetic until the final command, but it has already rounded a matrix coefficient and both fractional translation components during composition. Neither sequence has FLAG bits set in this case; quantization does not require an overflow or saturation flag.

Retaining the stored A/B coefficient fractions and all intermediate translation fractions in a wider software calculation, then rounding only the final vertex, is a third policy. The differences are not universally in favour of one of the two GTE sequences:

| Local vertex | B then A, integer IR between | Stored Q12 C and integer t | Stored A/B, fractions retained until final rounding |
|---|---|---|---|
| `(64,16,8)` | `(91,45,536)` | `(90,44,536)` | `(91,45,536)` |
| `(37,29,11)` | `(61,25,548)` | `(59,25,548)` | `(60,26,548)` |

Choose the policy appropriate to the program's coordinate precision and use it consistently. Precomposing once saves a transform per vertex, while the table shows the rounding differences from sequential integer transforms. Recompute moving transforms from the original data when accumulated rounding would otherwise change the required mapping.

### 4.8.5 Apply the same rule to view/model transforms

For a concrete view/model check, reuse only A's **linear** part as the model matrix, but choose model world origin `b_world=(116,28,600)` instead of the old a. Let viewing origin `c=(100,20,40)` and choose view axes with mapping `(X,Y,Z)->(-Y,X,Z)`. This is a specified software axis choice, not a GTE camera mode. Its matrix is V. Then:

```
b_world-c = (16,8,560)
t_view    = V*(b_world-c) = (-8,16,560)

            [ -4096  -6144     0 ]
(V*A)_raw = [  3072  -8192     0 ]
            [     0      0  4096 ]
```

For example, `V*A`'s first row is the negative of A's second row; its second row is A's first row. Local point `(64,16,0)` has world position `(132,116,600)`. Subtracting c gives `(32,96,560)` and applying V gives `(-96,32,560)`. The combined rows and t_view give the same result directly. These coefficients and translations are exact under the selected representation, so no intermediate rounding difference is hidden by this particular check.

For general non-exact view matrices, computing `V*(b_world-c)` before discarding fractions can differ from separately rounding `-V*c` and `V*b_world` and adding them. State that precision choice just as explicitly as for parent/local work.

Finally, respect the operand and result widths. A coefficient column fits the signed16 vector input because each source coefficient has that representation; a translation b does **not** necessarily fit, even though its own stored TR components are signed32. The one-vector origin-product method above applies when b fits the signed input components. Wider translations require an explicit software calculation in the owning coordinate representation, not silent loss to a vector halfword. Similarly, a composed coefficient can exceed signed16 even when both source matrices fit. IR limiting cannot turn it into the intended matrix. Visible MAC words can retain a wider shifted translation than IR, but still have signed32 wrap and cannot recover discarded fraction or earlier signed44 wrap. Plan intermediate widths and preserve each command's flags and needed outputs before consuming them in the next stage.

## 4.9 Perspective and geometric decisions

### 4.9.1 From transformed coordinates to screen coordinates

RTPS processes V0. RTPT processes V0, V1, and V2 in that order. Each vertex produces a transformed IR vector, a new unsigned depth, and a packed screen pair. The screen calculation uses the **limited IR1 and IR2**, not an unlimited copy of the matrix results.

The new depth is `A_Z ASR 12`, limited to 0–65535. Projection obtains a quotient `q` from H and this depth. Its scale is 65536 per unit ratio, so a quotient of `0x00008000` represents a factor of ½. The reciprocal-table divide is specified later in this chapter; it must not be replaced by an assumption of exact mathematical division. For the depths used below, H=256 gives exactly `q=32768` at depth 512 and `q=16384` at depth 1024.

```
SX = clamp((OFX + IR1*q) ASR 16, -1024, 1023)
SY = clamp((OFY + IR2*q) ASR 16, -1024, 1023)
```

H therefore controls how much transformed X and Y contribute at a given depth. OFX and OFY place the projected origin. For origin `(128,96)`, program OFX=`128×65536 = 0x00800000` and OFY=`96×65536 = 0x00600000`. At depth 512 with H=256, transformed `(64,32)` contributes `(32,16)` screen units, giving `(160,112)` after these offsets.

The offsets can contain fractional screen units: `0x00808000` represents 128½. The final arithmetic shift discards the fraction after the projected contribution is added. It rounds toward negative infinity, not toward zero: an offset of −½ (`0xffff8000`) with a zero projected contribution produces screen coordinate −1.

| Result | Scale and range | Where to read it |
|---|---|---|
| Transformed coordinates | Signed integer IR1–IR3, −32768–32767 with LM=0 | D[9]–D[11]; RTPT leaves the third vertex's values |
| Screen coordinates | Integer SX/SY, each limited to −1024–1023 | D[12]–D[14], oldest to newest; X low, Y high |
| Projection depth | Unsigned integer, 0–65535 | D[16]–D[19], oldest to newest |
| Depth-cue amount | Integer 0–4096, with 4096 representing a full cue in the `SF=1` color convention | IR0, D[8] |
| Scaled depth sum | Unsigned integer, 0–65535 | D[7], after AVSZ3 or AVSZ4 |

The screen FIFO holds three pairs. After RTPT, D[12], D[13], and D[14] correspond to V0, V1, and V2. The depth FIFO holds four depths; after RTPT, its newest three entries D[17]–D[19] correspond to those same vertices. D[16] retains the depth that preceded them. Reading the FIFOs does not consume entries. Another projection shifts their histories.

### 4.9.2 Clip a crossing edge before projecting it

Choose a usable spatial range in software before treating projected screen positions as a primitive. The numerical condition `H < 2*SZ3` avoids the divide- overflow branch, but it is not a clipping volume. A cartridge can choose a nearer working boundary with additional margin. Screen saturation also is not an intersection calculation: it changes a coordinate without constructing the missing geometric edge.

For an example, keep H=256 and choose the view-space near plane Z=256, retaining points with Z greater than or equal to 256. These are software choices. Consider one constant-color triangle, already transformed into view coordinates:

```
A = (-64,-64,128)    outside the chosen near plane
B = ( 64,-64,512)    inside
C = (  0, 64,512)    inside
```

Projecting A unchanged would take the divide-overflow branch because 256 equals twice its depth 128. Dropping the whole triangle would also discard its visible portion. Software can instead intersect the crossing edges with the chosen plane. For edge P→Q, linearly parameterized by `P+t*(Q-P)`, solve:

```
t = (nearZ - P.Z)/(Q.Z - P.Z)
```

A crossing edge has endpoints on opposite sides; the required t lies in 0–1. Here both edges from A have t=1/3. Their exact intersections are:

| Edge | View-space intersection |
|---|---|
| A→B | `(-64/3,-64,256)` |
| A→C | `(-128/3,-64/3,256)` |

Choose an explicit coordinate-rounding policy when supplying the new integer vertices. Rounding these intersection components to the nearest integer gives I_AB=`(-21,-64,256)` and I_AC=`(-43,-21,256)`. Both keep Z exactly on the chosen plane. This rounding is software work, not a projection mode or an action of FLAG. The retained polygon, in the original boundary order, is `I_AB,B,C,I_AC`. Split it into two triangles with the same winding: `(I_AB,B,C)` and `(I_AB,C,I_AC)`.

These vertices are **already view coordinates**. To project them directly, program matrix0 as Q12 identity and translation zero; do not apply the original model-to-view transform a second time. Use SF=1, LM=0, H=256, offsets(128,96) and zero DQA/DQB. At Z=256 the quotient is 65536; at Z=512 it is 32768.

| View point | Packed XY input | Z input | Screen pair | Packed screen word |
|---|---|---|---|---|
| I_AB `(-21,-64,256)` | `0xffc0ffeb` | `0x00000100` | `(107,32)` | `0x0020006b` |
| B `(64,-64,512)` | `0xffc00040` | `0x00000200` | `(160,64)` | `0x004000a0` |
| C `(0,64,512)` | `0x00400000` | `0x00000200` | `(128,128)` | `0x00800080` |
| I_AC `(-43,-21,256)` | `0xffebffd5` | `0x00000100` | `(85,75)` | `0x004b0055` |

Two RTPT commands can process the displayed triples. Each finishes with FLAG=0. The following NCLIP calculations give doubled areas 4416 and 3015, respectively, so this particular rounding preserves the chosen positive winding. Submit two drawing primitives, not the original triangle with its outside vertex silently replaced by a limiting screen coordinate. Retain each triple before another projection displaces its screen history.

This example uses one constant color, so it introduces no new per-vertex color values. A primitive with varying attributes needs software to carry the chosen attributes to the intersection vertices consistently with its interpolation policy. GTE does not create those vertices or infer their associations. Multiple planes require additional clipping steps and may produce more than one output primitive. After rounding intersection vertices, check the resulting winding and area before submitting each output primitive.

### 4.9.3 Area, depth, and the next decision

NCLIP evaluates the oriented screen area of the three screen FIFO entries:

```
MAC0 = (SX1-SX0)*(SY2-SY0) - (SX2-SX0)*(SY1-SY0)
```

Without signed MAC0 overflow, the magnitude is twice the triangle's area in square screen-coordinate units. Swapping two vertices reverses the determinant's sign. An unoverflowed zero result means that these integer screen positions are collinear, including coincident positions. Choose which sign is front-facing according to the cartridge's vertex order and screen-coordinate convention. NCLIP itself neither removes a vertex nor clips or rejects a primitive.

AVSZ3 multiplies the sum of the newest three depths by signed ZSF3, then shifts right twelve bits and limits the result to 0–65535. It is a **scaled sum**, not a hardwired division by three. ZSF3=`0x0555` represents `1365/4096`, slightly less than ⅓; ZSF4=`0x0400` represents ¼ for AVSZ4. These operations produce a depth value for the cartridge's use; they do not submit, sort, or draw primitives.

Read results before another command overwrites them. NCLIP and AVSZ3 both write MAC0. Every recognized command also starts a new FLAG result, so inspect the projection flags before issuing NCLIP, then inspect the area before issuing AVSZ3. RTPT combines the flags from all three vertices; its final FLAG does not identify which vertex caused an event.

A flag is not a primitive-rejection signal. For example, `H >= 2×SZ3`, including zero depth, produces DIV and quotient `0x1ffff`; a screen overflow produces the limiting coordinate −1024 or 1023. Both still produce FIFO results. The cartridge decides how to handle such geometry. Saturated coordinates need not preserve the original triangle's area or orientation. ERROR is a summary, not a substitute for the individual flag fields; some saturation fields do not contribute to ERROR.

## 4.10 Normals, light directions and color mixing

A position answers **where is the vertex?** A surface normal answers **which way does the surface face?** Both are supplied as three signed halfwords, but the lighting commands interpret the supplied vector as a direction, not as a position. They do not read three triangle positions, construct edge vectors, take their cross product, or normalize the result. Software must supply the normal it wants to use. An outward-facing direction, a unit-length normal and a particular fixed-point scale are software conventions, not properties enforced by the vector registers.

Choose the normal and light rows in a common coordinate frame. For example, when rotating a model, rotate its normal directions too, or express the light directions in the model's frame. A position's translation is not added to a normal direction. Projection does not make either adjustment for a later lighting command. With unit-length normals, a dot product can represent directional agreement; a longer normal also increases the result's magnitude. The GTE does not remove that extra factor. Likewise, a light row's magnitude may include the light's chosen strength; the row need not have unit length.

### 4.10.1 Keep a normal perpendicular after unequal scale

A normal represents a perpendicular direction, not just another position to send through the model's matrix. For example, choose tangent `e=(1,1,0)` and normal `n=(1,-1,0)`: their dot product is zero. If the model's linear transform doubles X but leaves Y/Z unchanged, the transformed tangent is `(2,1,0)`. Simply applying that same transform to n gives `(2,-1,0)`, whose dot product with the transformed tangent is 3, not zero.

For an invertible linear transform R, software can preserve perpendicularity by using the **inverse transpose**, `(R^-1)^T`, for normal directions. Its dot product with an edge transformed by R is the original normal/edge dot product. In the example it maps n to `(1/2,-1,0)`, which is perpendicular to `(2,1,0)`. If the lighting convention needs unit length, choose its normalization after this mapping; the inverse transpose preserves perpendicularity, not length. For a pure rotation, its inverse transpose equals the rotation itself.

This is a software construction rule. GTE does not derive this matrix from the position transform, invert a matrix, or normalize a direction automatically. Software supplies the desired coefficients and normal, using MVMVA if a separate matrix/vector product is required, then supplies the resulting direction to the lighting calculation. A transform that collapses an axis has no ordinary inverse; its surface/normal policy must also be chosen in software, not inferred from an arithmetic saturation flag. Translation is not part of this direction transform.

### 4.10.2 Three directional responses, then three color channels

The light matrix, C[8]–C[12], supplies three rows. Each row is dotted with the supplied normal. A useful convention points the row toward the light source: an outward normal facing that direction gives a positive dot product. If software stores the direction in which light travels instead, it must reverse that sign when constructing this setup. No light position, distance or attenuation is inferred from the vertex position.

For Q12 coefficients and a Q12 normal, select SF=1 so that each dot product is shifted right by twelve bits and remains Q12. LM=1 limits each resulting IR lane to 0–32767. Thus a negative dot product contributes zero to the following color matrix, with an IR saturation flag: this is a result clamp, not rejection of a negative normal component or matrix coefficient. LM=0 instead permits results down to −32768; negative directional responses can then subtract through the color matrix. Neither mode normalizes or otherwise repairs the inputs.

The color matrix, C[16]–C[20], maps these three directional responses to red, green and blue. **Its rows are output channels; its columns describe how each directional response contributes to all three channels.** The color stage also adds control vector 1, C[13]–C[15]. In the SF=1 convention, that vector is in the same scale as the color-stage result. It can supply a constant ambient contribution independent of normal direction. It is not a fourth dot product.

There are exactly three directional-response lanes in one light-matrix stage, not a register describing a physical number of lamps. One convenient model uses one directional light per row, but other software-defined bases are possible. More independent light contributions require software work, such as combining separately calculated contributions or using further passes and combining their results. Summing directions before the LM=1 clamp does not generally equal summing separately clamped light contributions. A triple lighting command processes three normals with the same setup; it does not provide three additional lights per normal, nor does it accumulate colors from earlier commands automatically.

### 4.10.3 A warm light, a green light and a back-facing blue light

Use the following non-identity setup. All displayed matrix entries are **raw signed coefficients**, with 4096 representing 1. The normal is the Q12 vector `N=(2458,0,3277)`, approximately `(0.6,0,0.8)`. Quantization makes it only approximately unit length; there is no subsequent normalization.

```
                light matrix L          color matrix K
                [ 3072    0   2048 ]     [ 4096  1024     0 ]
                [-2048    0   3072 ]     [ 2048  4096     0 ]
                [    0    0  -4096 ]     [ 1024  2048  4096 ]

ambient A = (256,512,128)       base RGB = (160,96,192)
```

Light row 0 points along `(3/4,0,1/2)` and row 1 along `(-1/2,0,3/4)`; their lengths include the chosen strength. Their color columns are respectively `(1,1/2,1/4)` and `(1/4,1,1/2)`, giving warm and green-biased contributions. Row 2 points along negative Z and contributes only blue. It faces away from this normal and will be clamped out. Ambient contributes gains `(1/16,1/8,1/32)` before the base-color multiplication.

Begin with zero color FIFO entries. Write the following complete set of operands, then issue NCCS with SF=1, LM=1. The high half of each packed pair is the next coefficient in row-major order; the final matrix coefficient occupies a signed low halfword. Ambient entries are full signed words. RGBC holds unsigned RGB bytes and CODE `0x34`.

| Destination | Address | Raw word |
|---|---|---|
| C[8], L00 / L01 | `0x080102e4` | `0x00000c00` |
| C[9], L02 / L10 | `0x080102e8` | `0xf8000800` |
| C[10], L11 / L12 | `0x080102ec` | `0x0c000000` |
| C[11], L20 / L21 | `0x080102f0` | `0x00000000` |
| C[12], L22 | `0x080102f4` | `0xfffff000` |
| C[13], ambient red | `0x080102f8` | `0x00000100` |
| C[14], ambient green | `0x080102fc` | `0x00000200` |
| C[15], ambient blue | `0x08010300` | `0x00000080` |
| C[16], K00 / K01 | `0x08010304` | `0x04001000` |
| C[17], K02 / K10 | `0x08010308` | `0x08000000` |
| C[18], K11 / K12 | `0x0801030c` | `0x00001000` |
| C[19], K20 / K21 | `0x08010310` | `0x08000400` |
| C[20], K22 | `0x08010314` | `0x00001000` |

| Destination | Address | Raw word |
|---|---|---|
| D[0], normal X / Y | `0x08010244` | `0x0000099a` |
| D[1], normal Z | `0x08010248` | `0x00000ccd` |
| D[6], RGBC | `0x0801025c` | `0x34c060a0` |
| COMMAND, NCCS | `0x08010344` | `0x0008041b` |

**First, form the three light responses.** Each row starts at zero and adds its X, Y and Z products in that order. The table shows those cumulative sums, the SF=1 arithmetic shift, and the subsequent LM=1 clamp. All accumulations in this example fit without 44-bit overflow or visible-word wrap.

| Light row | After X | After Y | After Z | MAC after ASR 12 | IR |
|---|---|---|---|---|---|
| 0 | 7550976 | 7550976 | 14262272 | 3482 | 3482 |
| 1 | −5033984 | −5033984 | 5032960 | 1228 | 1228 |
| 2 | 0 | 0 | −13422592 | −3277 | 0 |

The row-1 quotient is 1228.75, truncated by the arithmetic shift to 1228. Row 2 leaves MAC3 negative but writes zero to IR3, setting IR3 saturation. The next stage consumes **IR `(3482,1228,0)`**, not the unclamped MAC values.

**Next, mix the light responses and add ambient.** Each output row starts at `4096×A[channel]`, then adds its products with light responses 0, 1 and 2, in order. Shift by twelve and clamp again before applying the base color.

| Channel | Start | After response 0 | After response 1 | After response 2 | Gain |
|---|---|---|---|---|---|
| Red | 1048576 | 15310848 | 16568320 | 16568320 | 4045 |
| Green | 2097152 | 9228288 | 14258176 | 14258176 | 3481 |
| Blue | 524288 | 4089856 | 6604800 | 6604800 | 1612 |

Blue's pre-shift result represents 1612.5 and becomes 1612. No color-stage IR clamp is needed. The three gains are raw words `0x00000fcd`, `0x00000d99` and `0x0000064c`. Red contains both the warm and green light responses; it is not simply light row 0 copied into the red channel. The same distinction holds for green and blue.

**Finally, use each gain on its base byte.** NCCS forms `16×byte×gain`, shifts right by twelve, and writes MAC and clamped IR. The color FIFO then takes the signed MAC result shifted right by four and limits it to a byte.

| Channel | `16×byte×gain` | MAC / IR | Raw MAC / IR word | Output byte |
|---|---|---|---|---|
| Red | 10355200 | 2528 | `0x000009e0` | 158 |
| Green | 5346816 | 1305 | `0x00000519` | 81 |
| Blue | 4952064 | 1209 | `0x000004b9` | 75 |

Read RGB2, D[22] at `0x0801029c`, as **`0x344b519e`**: RGB `(158,81,75)` and CODE `0x34`. With the stated initial FIFO, RGB0 and RGB1 remain zero. MAC1–MAC3 and IR1–IR3 retain the final modulated values, not the light responses or gains. FLAG is **`0x00400000`**, recording the earlier negative third light response even though later IR3 is positive. IR3 saturation alone does not set ERROR. This command produces a color word, not a drawn pixel.

Ambient can be checked independently: replace the supplied normal with `(0,0,0)` and repeat. The light responses are all zero, but the color-stage gains remain `(256,512,128)`. The resulting RGB is `(10,12,6)`, giving RGB2 `0x34060c0a`. A constant contribution survives because it entered through control vector 1, not through a light row.

### 4.10.4 Select the color-stage scale

For the modulated calculation above, gain 4096 is unity because `(16×byte×4096) ASR 12 = 16×byte`. The FIFO's final four-bit shift restores the original byte. This is a **unit-gain convention**, not a universal color encoding. An unmodulated command such as NCS sends the color-matrix MAC result directly to the FIFO conversion: its useful output scale is byte×16, where 4080 produces 255. Feeding 4096 directly produces 256 before byte saturation, not a preserved base color. With this example's unchanged matrices and normal, NCS would emit `(252,217,100)` rather than `(158,81,75)` because it would divide the gains `(4045,3481,1612)` directly by sixteen. Choose and label the color-stage scale when constructing the setup; the registers do not distinguish a gain from an unmodulated intensity.

## 4.11 Choosing a color calculation

Choose a color command by **the values available at its input**. A supplied normal, three directional responses, three channel gains and an already calculated color represent different stages, although each uses three signed halfwords. Select the command whose first stage consumes the values currently held in the registers.

The following choices use the SF=1 conventions developed in Normals, light directions and color mixing. Light responses and modulation gains use 4096 as unity; an already calculated color held in IR uses byte×16 for the final color conversion. The command entries specify the exact arithmetic and other SF/LM settings.

| Values available | Required calculation | Choose |
|---|---|---|
| Supplied normal | Light matrix, then color matrix; emit its result without a base-color multiplication | NCS, or NCT for three normals |
| Supplied normal and a base RGB color | Both matrices, then apply the base color | NCCS, or NCCT for three normals |
| Supplied normal, base RGB, and a cue coefficient | Both matrices, base-color product and depth cue | NCDS, or NCDT for three normals |
| Directional responses already in IR1–IR3 | Color matrix and ambient, then base-color application | CC; CDP if depth cue is also required |
| Final channel gains already in IR1–IR3 | Base-color product and depth cue; do not apply either matrix again | DCPL |
| A finished RGB color in RGBC | Depth cue that byte color without matrix lighting or gain modulation | DPCS |
| A finished signed color vector in IR1–IR3 | Depth cue those values directly, using the same scale for source and far color | INTPL |
| Three finished byte colors in RGB0–RGB2 | Depth cue the three existing FIFO colors | DPCT |

NCS and NCT are **unmodulated**, not unlit: they still perform both matrices. Use them when the color-matrix result is itself the desired output intensity in byte×16 units. NCCS and NCCT instead interpret that result as channel gains applied to RGBC. Changing only the RGB bytes of RGBC therefore changes the modulated material color but not an NCS/NCT result. The CODE byte is copied to the output in either case.

The cued paths also differ by their starting point. CDP does not mean “add cue to the result of CC”: it starts by applying the color matrix and ambient to current IR. DCPL omits that matrix, but it still multiplies current IR by the RGBC bytes. DPCS does neither matrix lighting nor gain modulation. INTPL treats current IR as the color itself, not as gains to apply to RGBC. Select the command whose first stage matches the values actually retained.

### 4.11.1 Two material tints under the same light setup

Use the non-identity light matrix, color matrix and ambient from the preceding worked lighting example, with its original supplied normal restored. Its color-stage gains are `(4045,3481,1612)`. Do not renormalize those inputs. Suppose two surfaces use that same orientation but different chosen base tints:

| Material | Base RGB | RGBC word, CODE `0x34` | Output RGB | RGB2 word |
|---|---|---|---|---|
| A | `(160,96,192)` | `0x34c060a0` | `(158,81,75)` | `0x344b519e` |
| B | `(64,192,160)` | `0x34a0c040` | `(63,163,62)` | `0x343ea33f` |

For material B, the three unshifted application products are `(4142080,10693632,4126720)`. SF=1 changes these to MAC/IR `(1011,2610,1007)`; the final four-bit shift gives `(63,163,62)`. The changed material is supplied through RGBC, not by changing the normal to force a different lighting result.

For each material, write its RGBC word to D[6] at `0x0801025c`, then write NCCS command `0x0008041b` to `0x08010344`. V0 still holds the same supplied normal. The two commands each calculate the complete matrix-and-material path. If NCS were used instead, neither base tint would be applied: both operations would produce the preceding example’s unmodulated RGB `(252,217,100)`.

A three-normal batch is appropriate only when its shared inputs are appropriate for all three results. NCCT uses **one RGBC word for V0, V1 and V2**, not one material word per vector. NCT and NCDT have the same shared-matrix constraint; NCDT additionally uses one shared IR0 and far-color vector. Use separate commands or group work by shared material when different base colors are required.

### 4.11.2 Choose the fused cue path before discarding precision

For material A, now request a depth-cued color toward far RGB `(80,96,160)`. The coefficient is supplied explicitly here; selecting it from depth is a separate programming decision. With SF=1, program the far components as byte×16:

| Destination | Address | Raw word |
|---|---|---|
| D[6], restore material A | `0x0801025c` | `0x34c060a0` |
| D[8], IR0 = 2048 | `0x08010264` | `0x00000800` |
| C[21], far red = 1280 | `0x08010318` | `0x00000500` |
| C[22], far green = 1536 | `0x0801031c` | `0x00000600` |
| C[23], far blue = 2560 | `0x08010320` | `0x00000a00` |
| COMMAND, NCDS with SF=1, LM=1 | `0x08010344` | `0x00080413` |

The light and color matrices still produce gains `(4045,3481,1612)`. NCDS sends their **unshifted** base-color products into the cue calculation; it does not first push the NCCS byte result and then fetch it again. The products are (`10355200`, `5346816`, `4952064`). The signed, shifted cue differences are `(−1249,230,1351)`, all within the temporary difference limits. The final MAC/IR values are `(1903,1420,1884)`, and RGB2 becomes **`0x34755876`**, or RGB `(118,88,117)`. FLAG remains `0x00400000` from the third light response; none of these cue values adds a clamp flag.

Do not replace this path with an assumed byte-space average. In the red lane, the unshifted product represents 2528.125 at byte×16 scale. Its difference from far red 1280 becomes −1249 after the arithmetic shift. By contrast, copying the earlier NCCS output word `0x344b519e` into RGBC and issuing DPCS `0x00080410` starts from the already quantized red byte 158. Its red difference is −1248 and its final red byte is 119, producing **`0x34755877`** instead. Those are different programmed calculations. DPCS reads RGBC, not RGB2; that copy is necessary if an existing RGB2 color is to become its input.

If the intended source really is an already finished color, choosing DPCS, INTPL or DPCT is correct. INTPL avoids reducing a retained byte×16 IR color to whole bytes before cueing, but it cannot recover fractions or clamps discarded by an earlier command. No split path is automatically equivalent to a compound command's intermediate precision.

### 4.11.3 Keep the required stage alive

After NCCS, current IR contains the **modulated color**, not the earlier light responses or channel gains. After NCDS it contains the final cued color. Consequently, issuing CC or CDP immediately afterward applies the color matrix to those final values again. Issuing DCPL uses them as new gains and multiplies by RGBC again. Neither sequence means "continue where the previous compound command left off." A completed compound command does not expose all its earlier stages as separate retained results.

When software already has the earlier stages, it can instead load the appropriate IR values explicitly. For this setup:

| Available stage | Raw words for D[9], D[10], D[11] | Suitable continuation |
|---|---|---|
| Directional responses `(3482,1228,0)` | `0x00000d9a`, `0x000004cc`, `0x00000000` | CC for material A's uncued result; CDP for its cued result |
| Channel gains `(4045,3481,1612)` | `0x00000fcd`, `0x00000d99`, `0x0000064c` | DCPL for material A's cued result |

With the displayed matrix, base-color and cue operands, those continuations produce the corresponding RGB words above. Each overwrites IR again. Retaining values in software does not retain their old FLAG provenance: a new recognized command starts a new flag calculation. In particular, preloaded responses or gains do not recreate the earlier negative light-row event.

Manage the other inputs and histories separately:

- **Normal versus position slots.** V0–V2 retain their programmed words, but the same slots serve both purposes. Load normals before a normal-lighting command rather than assuming projected position words have become normals. These color calculations do not shift the screen or depth FIFOs.
- **Cue coefficient.** The listed cue commands consume current IR0 without updating it. Projection may replace it. RTPT leaves one final coefficient, from V2; NCDT does not select three coefficients from the depth FIFO. Program and preserve the coefficient required by the actual color calculation.
- **Color history.** Every calculated color pushes RGB0 ← old RGB1, RGB1 ← old RGB2, RGB2 ← new result. Starting with zero FIFO entries, material A, material B and then the cued material A leave precisely those three colors in RGB0, RGB1 and RGB2. Copy a result before further pushes replace it.
- **Three existing colors.** DPCT consumes the three original FIFO RGB colors in order and replaces them with three cued colors. All use the one current IR0 and far-color vector. Every new CODE comes from current RGBC, not from the consumed color's CODE.

Choose the input stage and scale, select the command, then preserve the operands and results needed by later work. Commands starting at different stages can produce different rounding and overwrite different histories.

## 4.12 Interpolation and general arithmetic

The arithmetic commands also serve general vector calculations. Choose the operand scales for the calculation, using SF to select the result shift and LM to select its lower limit. The following examples blend signed motion samples and form a triangle's perpendicular vector and squared-length components. Commands that push the color FIFO still do so when their arithmetic is used for these purposes.

### 4.12.1 Blend two signed motion samples

Suppose two motion samples use the same axes and the same integer coordinate counts:

```
a = (96,-48,32)
b = (-32,80,64)
wanted result = (3/4)*a + (1/4)*b
```

Choose Q12 scalar weights: 3/4 is raw 3072 (`0x0c00`) and 1/4 is raw 1024 (`0x0400`). Store a weight in signed IR0 and a vector in signed IR1–IR3. Select `SF=1` to remove the weight's factor of 4096 and `LM=0` to retain signed results. The weights add to one because software chose them that way; hardware does not require a convex blend or normalize the weights.

Use GPF to establish the weighted first term, then GPL to add the second. For these bounded operands, their relevant calculations are:

```
GPF: new MAC_i = (IR0*IR_i) ASR 12
GPL: new MAC_i = (IR0*IR_i + 4096*old signed MAC_i) ASR 12
```

Both commands then clamp each signed MAC result into IR. The command reference also specifies signed44 and visible-word wrapping; neither occurs in this example. GPL's old value is the **visible signed MAC word**, not a hidden unrounded product retained by GPF.

| Step | IR0 before command | IR1–IR3 before command | Normal command word | MAC1–MAC3 and IR1–IR3 afterwards |
|---|---|---|---|---|
| Establish first term with GPF | 3072 | `(96,-48,32)` | `0x0008003d` | `(72,-36,24)` |
| Add second term with GPL | 1024 | `(-32,80,64)` | `0x0008003e` | `(64,-16,40)` |

IR inputs consume signed low halfwords; for example −48 can be supplied with complete raw word `0xffffffd0`, and −32 with `0xfffffe0`.

After GPF, load b and the second weight into IR, **without replacing the MAC words**. Ordinary writes to the IR inputs do not erase the accumulated MAC values. An intervening arithmetic command that uses MAC1–MAC3 would, however, break this accumulation. The final row calculations are:

```
X: (1024*(-32) + 4096*72)  ASR 12 = 64
Y: (1024*80    + 4096*(-36)) ASR 12 = -16
Z: (1024*64    + 4096*24)  ASR 12 = 40
```

All three signed results fit IR. In particular, negative Y is part of the motion vector, not a color intensity to discard. LM=1 would not be the intended output convention for this signed result.

### 4.12.2 Account for the color pushes even when no color is wanted

GPF and GPL each push one RGB history entry. They take each **visible MAC**, shift it right four bits, clamp the result to a byte, and copy the current RGBC CODE byte. The lower RGB bytes of RGBC are not operands of this weighted sum.

For an explicit trace, let RGBC be `0x5a332211`, so CODE is `0x5a`, and let the initial color history be `0x10010203`, `0x20040506`, `0x30070809`:

| Point | MAC values | New RGB byte triple | RGB0, RGB1, RGB2 |
|---|---|---|---|
| After GPF | `(72,-36,24)` | `(4,0,1)` | `0x20040506`, `0x30070809`, `0x5a010004` |
| After GPL | `(64,-16,40)` | `(4,0,2)` | `0x30070809`, `0x5a010004`, `0x5a020004` |

The green byte is limited from a negative value in both commands. Each command's FLAG is consequently `0x00100000`, green color saturation, even though its signed MAC and IR vector results are correct. ERROR remains clear because color-channel flags are not part of that summary. Do not mistake the packed RGB result for a second representation from which the signed vector can be recovered: its fraction, sign, and range have already been discarded for byte output.

If color history was needed for drawing, consume or retain those entries before this calculation displaces them. Read a needed GPF FLAG before GPL starts a new one. Choosing non-color operands does not disable either the push or CODE copy; no command option in this sequence makes them vector-only arithmetic operations.

### 4.12.3 Interpolate towards a retained far vector with INTPL

There is another way to form this particular blend. As an **alternative** sequence, start again with a in IR1–IR3, store b in signed complete far-vector words C[21]–C[23], and put 1024 in IR0. Do not start from the already blended IR result left by GPL.

With `SF=1`, `LM=0`, and no arithmetic wrap, INTPL's two passes are:

```
d_i   = clamp(far_i - input_i, -32768, 32767)
MAC_i = (4096*input_i + IR0*d_i) ASR 12
IR_i  = clamp(MAC_i, -32768, 32767)
```

The temporary signed difference clamp is part of the operation. It is forced to the signed range even if the final result uses LM=1. Both passes use the selected SF; switching to SF=0 is not the same interpolation at another output format.

Here the differences are `(-128,128,32)`, all in range. INTPL command `0x00080011` gives `(64,-16,40)`, the same exact result as the two-command blend above. It also pushes `0x5a020004` and sets the same green color flag. Thus this bounded calculation can use a retained target vector and one arithmetic command, but it still has a color-history side effect. Reprogramming C[21]–C[23] for this purpose also changes the far-vector controls used by later depth-cue color commands.

The word "far" does not cause INTPL to fetch a scene endpoint, evaluate depth, or select an interpolation weight. Software supplies three target components and one signed IR0 scalar. For this convention IR0/4096 is the chosen weight; it is not a hardwired half-way operation or a general unbounded scalar lerp. Two qualifications matter:

- **Rounding occurs at different points.** For scalar a=5 and b=3 with the same weights, GPF first gives 3 from 3.75, then GPL gives 3 from `3+0.75`. INTPL instead evaluates `5+(1/4)*(-2)` before the final shift and gives 4. The exact blend is 4.5. The different shift points produce different integer results.

- **Even legal endpoints can have an unrepresentable difference.** Set the current first component to −30000, its far component to 30000, and IR0 to 4096, leaving the other components zero. Both endpoints fit signed16, but their difference 60000 is limited to 32767. INTPL therefore ends at `-30000+32767 = 2767`, not 30000. FLAG is `0x81000000`: IR1 saturation from the temporary difference and ERROR. The final 2767 is itself in range; the retained flag explains why the intended endpoint was not reached.

Choose INTPL when its actual difference range, precision, retained target, and side effects match the calculation. Do not replace a weighted sum by an ideal lerp formula without checking those stages.

### 4.12.4 Form a perpendicular vector, then measure its squared length

Consider a triangle whose software has already formed two edges from the same vertex:

```
u = (12,-8,4)
v = (-4,20,8)
```

OP can form the algebraic cross product `u × v`. Put u in the **diagonal of matrix 0**, and v in signed IR1–IR3. The needed diagonal fields are the low halves of C[0], C[2], and C[4]. A scratch setup using complete words `0x0000000c`, `0x0000fff8`, and `0x00000004` respectively supplies `(12,-8,4)`; the other matrix coefficients are not OP operands.

These diagonal values are raw integer edge components, **not Q12 rotation coefficients** in this calculation. Select `SF=0`, `LM=0`, normal OP word `0x0000000c`. No factor of 4096 needs removing. The three results are:

```
nX = uY*vZ - uZ*vY = (-8)*8 - 4*20       = -144
nY = uZ*vX - uX*vZ = 4*(-4) - 12*8       = -112
nZ = uX*vY - uY*vX = 12*20 - (-8)*(-4)   = 208
```

Both MAC and IR hold `(-144,-112,208)`; FLAG is zero. The result is perpendicular to both supplied edges: its dot products with u and v are zero. Swapping the edges reverses its sign. Software's edge order and coordinate convention must therefore match its intended triangle orientation. OP has not fetched triangle positions, formed the edges, chosen a front face, or normalized n.

If n is also needed for lighting or later geometry, retain it before the next step. SQR can square the current three IR components independently. With `SF=0`, `LM=0`, normal word `0x00000028`, it produces:

| Lane | Complete MAC result | Limited IR result |
|---|---|---|
| X squared | 20736 | 20736 |
| Y squared | 12544 | 12544 |
| Z squared | 43264 | 32767 |

The squares fit the visible MAC words, but the third does not fit IR. SQR sets FLAG=`0x00400000`, IR3 saturation only; ERROR is clear. Sum the three **MAC** results in software to obtain the squared length:

```
n dot n = 20736 + 12544 + 43264 = 76544
```

Summing the limited IR values would instead give 66047. SQR does not produce a sum, a square root, or a unit vector, and its output no longer retains n's signs. If the edges use spatial units, n has squared-spatial units and these squared components have fourth-power units. Producing a unit normal or a normal in a chosen lighting scale is further software work, not an automatic interpretation of IR.

OP and SQR do **not** push RGB history entries, unlike GPF, GPL, and INTPL. They do replace MAC1–MAC3 and IR1–IR3 and start new FLAG results. The OP setup also reuses fields of the projection matrix: supply the intended geometry transform again from normal cartridge data before projecting more vertices. The command itself does not install a separate normal matrix or preserve a hidden rotation.

Finally, SF is a shift choice, not a normalization request. Repeating OP with the same raw integer edges but SF=1 gives `(-1,-1,0)` after its twelve-bit shift, not a unit-length version of `(-144,-112,208)`. Match each multiply's input scales and required output range before selecting SF. Do not discard a valid MAC result merely because its narrower IR copy has been limited.

## 4.13 Choosing depth-cue coefficients

Depth cue changes a calculated color toward the programmed far color. Before choosing its coefficient, choose **what distance law software wants**. A constant blend, a blend linear in view depth and a projection-derived blend are different operations. The GTE color commands consume current IR0; they do not search the depth FIFO for a matching vertex or calculate a coefficient from a position.

In the SF=1 color convention, IR0=0 selects zero coefficient contribution and IR0=4096 selects full contribution. Evaluate the ordered cue arithmetic when choosing endpoint colors: temporary difference limits and discarded fractional bits can change the final bytes.

### 4.13.1 Program IR0 directly for a chosen blend

For an explicitly chosen fraction f in 0–1, software can encode approximately `4096×f` in IR0, choosing its integer rounding rule. The raw words for zero, one half and one are `0x00000000`, `0x00000800` and `0x00001000`, written to D[8] at `0x08010264`. No projection is required. The following cue command consumes that coefficient and leaves IR0 unchanged.

The port actually holds a **signed low-halfword operand**. Direct writes are not automatically restricted to 0–4096: negative values and values through 32767 remain representable, and the cue calculation consumes them as signed coefficients. Such inputs can extrapolate before its intermediate and final limits. The 0–4096 bound discussed below belongs to projection's IR0-producing stage, not to a universal unsigned IR0 register format.

For example, a fade linear in depth from Z=256 to Z=768 would request half strength at Z=512. Software can select IR0=2048 there. Within that interval, the ideal coefficient is `4096×(Z−256)/(768−256)`; choose the actual integer result in software and use the endpoint coefficients outside the interval. This is not the law produced by a single DQA/DQB setup.

### 4.13.2 Projection produces a reciprocal-depth law

RTPS, and the last vertex of RTPT, calculate:

```
q   = the specified reciprocal-table projection factor
A   = DQB + DQA*q
u   = A ASR 12
IR0 = clamp(u,0,4096)
```

DQA is a signed 16-bit multiplier; DQB is a signed 32-bit offset added **before** the twelve-bit shift. DQA is not a matrix coefficient with an independently implied Q12 meaning. The post-sum shift here is always twelve bits, irrespective of the projection command's SF setting. The full sum A is used for IR0 even if the visible low-word MAC0 has overflowed; MAC0 overflow and the IR0 clamp have separate flags.

Let Z denote the unsigned, limited projection depth SZ3, not a world-space distance or Euclidean distance from a viewing position. In the valid divide range, q approximates `65536×H/Z`. Therefore DQA/DQB describe a line in **q**, and approximately a line in **1/Z**, not a line in Z. For a fade from coefficient zero at near depth Zn to 4096 at far depth Zf, the ideal, unclamped fraction is:

```
f(Z) = (1/Zn - 1/Z) / (1/Zn - 1/Zf)
```

This gives a useful first design, but the actual reciprocal-table outputs, integer coefficients and arithmetic shifts determine the programmed result. Changing H changes q, so coefficients chosen for one H must not be assumed to give the same depth endpoints with another.

To fit desired raw IR0 counts kn and kf at calculated factors qn and qf, first solve the ideal two-point equations:

```
DQA = 4096*(kf-kn)/(qf-qn)
DQB = 4096*kn - DQA*qn
```

When the coefficient grows with distance, qf is smaller than qn and DQA is negative. The solved values need not be representable integers. After choosing an integer DQA, account for the output shift: obtaining count k **without an IR0 clamp** requires `4096*k <= DQB+DQA*q < 4096*(k+1)`. This interval, rather than equality to just its lower boundary, can make both endpoint counts attainable.

### 4.13.3 Fit near 256 and far 768 with H=128

Retain the non-identity lighting setup, original normal and material A from the preceding examples, and far RGB `(80,96,160)` from the color-selection example. Only the coefficient-selection calculation changes. Choose H=128, zero coefficient at projection depth 256, and full coefficient at depth 768.

The actual reciprocal calculation gives `qn=32768` and `qf=10923`. Thus:

```
DQA = 16777216/(10923-32768)
    = -768.011719...              choose the integer -768
```

Simply anchoring that integer slope at zero near depth would give `DQB=768×32768=25165824`, raw `0x01800000`. At depth 768, however, `(25165824−768×10923) ASR 12 = 4095`, not 4096.

Use the endpoint intervals instead. For near count 0, DQB may be 25165824–25169919. For far count 4096, it may be 25166080–25170175. Their overlap allows **DQB=25166080**, raw **`0x01800100`**. At the near endpoint A=256 shifts to zero; at the far endpoint A=16777216 shifts to 4096. Neither endpoint requires an IR0 clamp.

The resulting coefficient words are:

| Destination | Address | Raw word | Meaning |
|---|---|---|---|
| C[26], H | `0x0801032c` | `0x00000080` | Unsigned projection numerator 128 |
| C[27], DQA | `0x08010330` | `0xfffffd00` | Signed multiplier −768 |
| C[28], DQB | `0x08010334` | `0x01800100` | Signed pre-shift offset 25166080 |

To isolate the depth behavior, use matrix 0 identity, zero translation and zero screen offsets. The five matrix-0 word contents C[0]–C[4], in order, are `0x00001000`, `0x00000000`, `0x00001000`, `0x00000000`, `0x00001000`; C[5]–C[7] and C[24]–C[25] are zero. For each position `(0,0,Z)`, D[0] is zero and D[1] holds the displayed integer Z in its signed low halfword. Issue RTPS `0x00080001` with SF=1, LM=0.

Read IR0 and any projection flags before the following color command. Replace V0's position with the original supplied normal, retain IR0, and issue NCDS `0x00080413` with the earlier material and far-color operands. This separates the position that chooses the cue from the normal that chooses the lighting.

| Depth Z | Actual q | u before clamp | IR0 | Result RGB | RGB2 word |
|---|---|---|---|---|---|
| 128 | 65536 | −6144 | 0 | `(158,81,75)` | `0x344b519e` |
| 256, near | 32768 | 0 | 0 | `(158,81,75)` | `0x344b519e` |
| 384 | 21845 | 2048 | 2048 | `(118,88,117)` | `0x34755876` |
| 512 | 16384 | 3072 | 3072 | `(99,92,138)` | `0x348a5c63` |
| 768, far | 10923 | 4096 | 4096 | `(79,95,160)` | `0x34a05f4f` |
| 1024 | 8192 | 4608 | 4096 | `(79,95,160)` | `0x34a05f4f` |

The q column gives the intermediate reciprocal result used by projection. Calculate it with the reciprocal algorithm when reproducing these rows. For example, at Z=768 its normalized denominator is 49152, the seed-ROM index is 128 and its seed byte is `0x54`. Refinement gives reciprocal 87381; the rounded product with normalized numerator 8192 gives q=10923.

Half coefficient occurs at depth 384, not at the Z-linear midpoint 512. At 512 this setup already applies three quarters of the coefficient range. Depths 128 and 1024 set projection FLAG `0x00001000` because u is strictly outside 0–4096. The other displayed projections have FLAG zero, including both chosen endpoints. These clamps do not set ERROR by themselves. The following NCDS starts a new FLAG calculation, yielding the earlier light-row flag `0x00400000`; it does not retain the projection flags.

### 4.13.4 A full coefficient is not always an exact far-color byte

The calibrated far coefficient really is 4096, yet this NCDS example produces `(79,95,160)`, not the programmed far RGB `(80,96,160)`. Its red unshifted material product is 10355200. The cue difference calculation gives `(4096×1280−10355200) ASR 12 = −1249`. At IR0=4096, the final shifted red value is 1279, whose output byte is 79. Green likewise finishes at 1535, giving byte 95. The discarded first-pass fractions cannot be recovered by a full coefficient.

Larger differences can also hit the signed temporary-difference limits, preventing a full coefficient from reaching the requested far value. Distinguish three separate questions: did projection produce the desired IR0, did the cue arithmetic reach the desired scaled value, and did the final byte conversion produce the desired RGB? For DPCS with ordinary byte input and far byte×16 values, the differences are exact and in range, so SF=1 and IR0=4096 do reach those far bytes. That narrower property must not be generalized to every cue input path. Conversely, IR0=0 preserves the selected starting term at the final cue sum, but still performs the temporary difference calculation and can still set its flags.

### 4.13.5 Check whether the requested fade fits the coefficient fields

DQA must fit −32768–32767, DQB must fit a signed 32-bit word, and distinct desired endpoint counts require distinct q values. A narrower depth interval can demand an unrepresentable slope. With the same H and near depth, moving the far depth to 260 gives actual qf=32264 and an ideal integer-count fit of about **DQA=−33288.13**, outside the signed-halfword range. Even the largest negative representable slope changes the pre-shift sum by only `32768×(32768−32264)=16515072`, insufficient to span the two unclamped endpoint windows for counts 0 and 4096. Writing the low bits of the desired slope would program a different signed value, not an approximation of that fade.

Choose attainable endpoints, a smaller coefficient span, or calculate IR0 directly when that is the intended law. Also keep the projection divide range separate from the fade range: if `H >= 2×SZ3`, including zero depth, projection uses `q=0x1ffff` and sets DIV and ERROR. Depth and IR0 saturation do not clip geometry or make that projection an ordinary reciprocal sample. The fit above uses only valid, unsaturated depths for its specified endpoints.

### 4.13.6 Keep per-vertex coefficients associated with their vertices

RTPT computes IR0 only for V2. With depths `(256,384,768)`, the depth FIFO's newest three entries retain those three depths, but IR0 retains only **4096**. Replacing V0–V2 with three normals and issuing NCDT uses that one coefficient for all three colors. For three copies of the supplied example normal, all three outputs are `0x34a05f4f`; the batch does not recover coefficients `(0,2048,4096)` from the depth history.

For different per-vertex coefficients, use individually associated projection and color work, retain each separately obtained coefficient for later single color commands, or calculate the coefficients explicitly in software. Program each required IR0 before its color operation. Another projection may replace IR0; none of the cue commands discussed here consumes and advances a hidden coefficient FIFO. The batching choice must match the lifetime of the actual coefficient, not merely the presence of three retained depths.

## 4.14 Worked geometry sequences

### 4.14.1 Project a triangle, test its orientation, and form a depth value

This sequence prepares the screen triangle `(96,64)`, `(160,64)`, `(128,128)`. It uses an identity transform, integer model coordinates, H=256, and projected origin `(128,96)`. All three depths are 512. The coordinates fit a 256×192 drawing area, but GTE itself has no drawing-area setting and does not draw the triangle.

## Worked triangle after projection

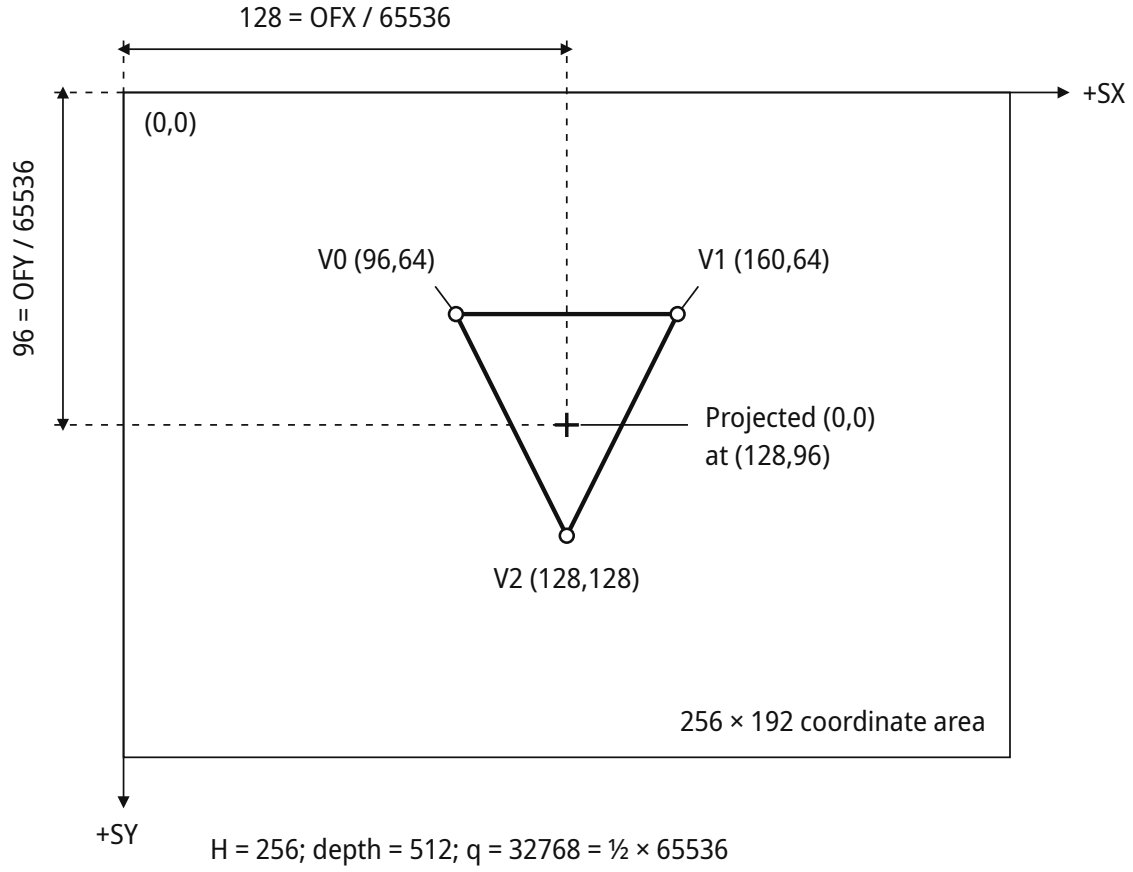



Figure 4.3: The worked triangle and its projected origin

The outline in the figure is the example coordinate area, not a GTE clipping window. Its arrows establish the screen-coordinate convention used in this drawing.

Start with reset-zero GTE state. Use the aligned 32-bit word writes below in the listed order. The complete transform setup includes explicit zero translation and zero depth-cue constants. For subsequent triangles sharing these settings, retain the controls and replace only the input vertices before the command.

| Step | Address / register | Raw word | Meaning |
|---|---|---|---|
| 1 | `0x080102c4` — C[0] | `0x00001000` | m00=4096, m01=0 |
| 2 | `0x080102c8` — C[1] | `0x00000000` | m02=0, m10=0 |
| 3 | `0x080102cc` — C[2] | `0x00001000` | m11=4096, m12=0 |
| 4 | `0x080102d0` — C[3] | `0x00000000` | m20=0, m21=0 |
| 5 | `0x080102d4` — C[4] | `0x00001000` | m22=4096 |
| 6 | `0x080102d8` — C[5] | `0x00000000` | TRX=0 |
| 7 | `0x080102dc` — C[6] | `0x00000000` | TRY=0 |
| 8 | `0x080102e0` — C[7] | `0x00000000` | TRZ=0 |
| 9 | `0x08010324` — C[24] | `0x00800000` | OFX=128 screen units |
| 10 | `0x08010328` — C[25] | `0x00600000` | OFY=96 screen units |
| 11 | `0x0801032c` — C[26] | `0x00000100` | H=256 |
| 12 | `0x08010330` — C[27] | `0x00000000` | DQA=0 |
| 13 | `0x08010334` — C[28] | `0x00000000` | DQB=0 |
| 14 | `0x08010338` — C[29] | `0x00000555` | ZSF3=1365 |

Pack the signed vertex coordinates next. The halfword for −64 is `0xffc0`.

| Step | Address / register | Raw word | Meaning |
|---|---|---|---|
| 15 | `0x08010244` — D[0] | `0xffc0ffc0` | V0.X=−64, V0.Y=−64 |
| 16 | `0x08010248` — D[1] | `0x00000200` | V0.Z=512 |
| 17 | `0x0801024c` — D[2] | `0xffc00040` | V1.X=64, V1.Y=−64 |
| 18 | `0x08010250` — D[3] | `0x00000200` | V1.Z=512 |
| 19 | `0x08010254` — D[4] | `0x00400000` | V2.X=0, V2.Y=64 |
| 20 | `0x08010258` — D[5] | `0x00000200` | V2.Z=512 |
| 21 | `0x08010344` — COMMAND | `0x00080030` | RTPT, SF=1, LM=0 |

The RTPT write completes the arithmetic. No CYCLES polling is required. Read FLAG at `0x08010340` now, before the next command. For this sequence it is zero. Each vertex has `q=32768`, so its screen displacement is half its transformed X/Y. The first result is therefore `(128−32,96−32)=(96,64)`, followed by `(160,64)` and `(128,128)`.

| Read immediately after RTPT | Address(es) | Raw result(s) |
|---|---|---|
| Screen FIFO, V0 then V1 then V2 | `0x08010274`, `0x08010278`, `0x0801027c` | `0x00400060`, `0x004000a0`, `0x00800080` |
| Depth FIFO, oldest to newest | `0x08010284`–`0x08010290` | `0x00000000`, `0x00000200`, `0x00000200`, `0x00000200` |
| IR1, IR2, IR3, from V2 | `0x08010268`–`0x08010270` | `0x00000000`, `0x00000040`, `0x00000200` |
| MAC1, MAC2, MAC3, from V2 | `0x080102a8`–`0x080102b0` | `0x00000000`, `0x00000040`, `0x00000200` |
| IR0; MAC0 | `0x08010264`; `0x080102a4` | `0x00000000`; `0x00000000` |
| Depth-average word; FLAG | `0x08010260`; `0x08010340` | `0x00000000`; `0x00000000` |
| CYCLES | `0x08010348` | `0x00000017` — nominal 23 |

RTPT preserves the input vertex words, RGBC, the color FIFO, the depth-average word, and C[0]–C[30]. It replaces IR0–IR3 and MAC0–MAC3 and pushes the screen and depth histories. D[15] reads the new D[14]; the packed IR aliases D[28]/D[29] reflect the new IR values. The three vertices have not been consumed from their input registers.

Continue with these commands and reads in order:

| Step | Operation | Raw word or result |
|---|---|---|
| 22 | Write `0x08010344` — COMMAND | `0x00000006` — NCLIP |
| 23 | Read `0x08010340` — FLAG | `0x00000000` |
| 24 | Read `0x080102a4` — MAC0 | `0x00001000` — doubled oriented area 4096 |
| 25 | Read `0x08010348` — CYCLES | `0x00000008` — nominal 8 |
| 26 | Write `0x08010344` — COMMAND | `0x0000002d` — AVSZ3 |
| 27 | Read `0x08010340` — FLAG | `0x00000000` |
| 28 | Read `0x08010260` — depth average | `0x000001ff` — 511 |
| 29 | Read `0x080102a4` — MAC0 | `0x001ffe00` — 2096640 |
| 30 | Read `0x08010348` — CYCLES | `0x00000005` — nominal 5 |

Neither NCLIP nor AVSZ3 uses SF or LM. NCLIP gives `(160−96)×(128−64) − (128−96)×(64−64) = 4096`, a positive doubled area. AVSZ3 gives `1365×(512+512+512) = 2096640`; the twelve-bit shift yields 511, not 512. This is the deliberate consequence of representing ⅓ by `1365/4096`, not an extra rounding stage.

NCLIP changes MAC0 and FLAG but preserves the screen and depth FIFOs, IR0–IR3, MAC1–MAC3, and D[7]. AVSZ3 then replaces MAC0 and D[7], again preserving those FIFOs and IR/MAC1–MAC3 results. Both preserve the programmed controls and input vertices. At the end, the screen words still contain the triangle, while MAC0 no longer contains its area. Retain the area decision before continuing.

## 4.15 From projected vertices to a visible image

A projected vertex is still only a pair of numbers. Reading SXY2 does not place a pixel in graphics memory. The cartridge must pass the projected coordinates to a GPU drawing command, and PCRTC must read the part of graphics memory into which that command draws.

| Stage | Data supplied by the cartridge | Result | What does not happen automatically |
|---|---|---|---|
| GTE geometry | Three-dimensional vectors, transformation matrix, translation, projection numerator and screen offsets | Screen pairs, depth history, arithmetic flags | No polygon or pixel is drawn. |
| GTE color arithmetic, when used | Direction vectors, light/color matrices, input color and optional depth-cue parameters | Color FIFO entries | Colors are not attached to a GPU packet automatically. |
| GPU drawing | A drawing command containing screen coordinates and colors | Pixel values in graphics memory | GTE results are not fetched by the GPU. |
| PCRTC display | Framebuffer base, stride, storage format, source window and circuit enable | Display samples from that framebuffer | It does not execute geometry or create the image being read. |

The cartridge connects these stages with register reads and writes. Geometry, shading and drawing are separate choices: projecting a triangle does not select its color; producing a color does not select a texture or submit a triangle. The example below deliberately chooses a solid, untextured red triangle so that no texture state is needed.

### 4.15.1 Continue the worked triangle

Continue immediately after **Project a triangle, test its orientation, and form a depth value**, before changing its transform or issuing another projection. Its retained screen words are:

| GTE word | Coordinate | Packed word |
|---|---|---|
| D[12] | (96, 64) | `0x00400060` |
| D[13] | (160, 64) | `0x004000a0` |
| D[14] | (128, 128) | `0x00800080` |

Start the continuation with GPU and PCRTC reset state and reset beam timing running. Serialize other GP0 writers while submitting it. The picture uses local-memory base zero, a 1024-word row stride, GX16 storage and a 256 × 192 display window.

All GPU command words in the following GP0 tables are aligned 32-bit CPU stores to **GP0 at `0x0801023c`**. Each table gives words in transmission order. Use ordinary CPU word stores; do not replace a complete command packet with a single floating-point store. Direct CPU stores wait for a free command FIFO slot when necessary. Preserve packet order and keep other writers off this port for the sequence.

### 4.15.2 Set drawing state and clear the destination

The drawing area bounds include their final coordinate. Here they cover X=0–255 and Y=0–191. The GPU drawing offset is zero: GTE has already placed the triangle using its screen offsets of (128, 96).

| Order | GP0 word | Meaning |
|---|---|---|
| 1 | `0xe1000400` | Draw mode: allow drawing to the displayed field; dithering off. No texture is used by this example. |
| 2 | `0xe3000000` | Drawing-area top-left = (0, 0). |

| Order | GP0 word | Meaning |
|---|---|---|
| 3 | `0xe402fcff` | Drawing-area bottom-right = (255, 191). |
| 4 | `0xe5000000` | Drawing offset = (0, 0). |
| 5 | `0xe6000000` | Do not test destination mask bits and do not force the mask bit on new pixels. |

Next send the complete three-word fill packet. This initializes the actual displayed storage; it does not rely on graphics-memory power-on contents.

| Order | GP0 word | Meaning |
|---|---|---|
| 6 | `0x02000000` | Fill rectangle with black RGB. |
| 7 | `0x00000000` | Destination (0, 0). |
| 8 | `0x00c00100` | Width 256; height 192. |

### 4.15.3 Submit the triangle

Opcode `0x20` selects a three-vertex polygon with one constant color, no texture and no semitransparency. The low 24 bits of its first word contain red, green and blue in successive low-to-high bytes. Therefore `0x200000ff` selects red (255, 0, 0).

| Order | GP0 word | Meaning |
|---|---|---|
| 9 | `0x200000ff` | Solid red triangle command and color. |
| 10 | Read D[12]: `0x00400060` | First screen coordinate. |
| 11 | Read D[13]: `0x004000a0` | Second screen coordinate. |
| 12 | Read D[14]: `0x00800080` | Third screen coordinate. |

Here the GTE screen format already supplies the coordinate word required by this GPU packet. Copy the words, not their byte addresses. The color FIFO is not required for this constant-color example. A cartridge using GTE-produced colors would explicitly place those result bytes into its chosen drawing packet.

The earlier geometry sequence reported positive doubled area 4096 before AVSZ3, then finished with OTZ=511. Those values do not themselves accept, reject or sort this GPU polygon. The example submits it explicitly. In particular, NCLIP is an orientation calculation, not a clipping engine.

Read **GP1/status at `0x08010240`** until bit 26 is one to wait for the complete drawing command stream to become idle. Do this after the full triangle packet, not between its coordinate words: an incomplete packet is not idle. A status read is not an additional GP0 packet word.

### 4.15.4 Select the framebuffer for display

Now perform these PCRTC word writes. The unchanged reset base/stride/format word is included to make the destination-to-display connection explicit. The source window begins at the same (0, 0) that was cleared and drawn above.

| Order | Address | Register | Word | Effect |
|---|---|---|---|---|
| 1 | `0x0801035c` | DISPFB1_LOW | `0x000fa000` | Base zero; width 16 × 64 = 1024 words; GX16. |
| 2 | `0x08010360` | DISPFB1_HIGH | `0x00000000` | Source origin (0, 0). |

| Order | Address | Register | Word | Effect |
|---|---|---|---|---|
| 3 | `0x08010368` | DISPLAY1_HIGH | `0x000bf3ff` | Horizontal extent 1023 and vertical extent 191. With reset horizontal signal step 4, this selects 256 × 192 output pixels. |
| 4 | `0x08010354` | PMODE_LOW | `0x0000ff21` | Enable circuit 1 only and select its RGB directly. |

Retain the reset signal position, magnification and timing words. The source origin is a framebuffer coordinate, while DISPLAY's position is a signal coordinate; do not copy the GTE screen offset into DISPLAY. PCRTC converts its selected signal window to output coordinates, so the example image begins at output (0, 0).

Complete all four writes before the intended **VBlank entry**, when the display latch takes the programmed configuration together. An edge between stores takes the intermediate values. Register readback shows programmed state; observe a subsequent VBlank entry to establish the displayed setting. The latch operates independently of PCRTC interrupt masks. See Section 3.2.3 for its full timing.

The observable result is a red triangle on a black 256 × 192 image, with vertices at (96, 64), (160, 64) and (128, 128). At the interior point (128, 96), the graphics-memory word is `0x001f`, which GX16 supplies as RGB (255, 0, 0). At (0, 0), the cleared word is `0x0000`, supplying black. With the selected direct-RGB composition, the samples' zero mask/alpha bit does not blend the red away.

The framebuffer stride is 1024 words even though the visible width is 256 pixels. Thus row y begins at local word address `1024 × y`, not `256 × y`; the center sample above is at word address 98432. The clearing command, GPU drawing coordinates and PCRTC source must agree on this organization. Changing only the displayed window width does not repack graphics memory.

## 4.16 Hardware frame sequence

The following sequence joins geometry, vertex-color, drawing and display work into a repeated frame loop. It draws the triangle first at `(96,64)`, `(160,64)`, `(128,128)`, then alternates its X translation between 0 and 32 every 32 completed loop iterations. At depth 512 and H=256, that translation moves its screen positions sixteen units horizontally. The display is 256×192.

### 4.16.1 Pack and transfer the operands

Use complete, aligned CPU word stores to the GTE register windows. Software may keep operands in ROM, but GTE does not fetch records or remember their source addresses. Each vector occupies two input words: signed X/Y in the low/high halves of the first, then signed Z in the low half of the second. The unused upper half of that second word is zero in this example. Matrix coefficients occupy successive signed halves in row order; the ninth coefficient occupies the low half of the fifth word.

| Destination | Words in increasing register order |
|---|---|
| Rotation C[0]–C[4] | `0x00001000`, `0x00000000`, `0x00001000`, `0x00000000`, `0x00001000` |
| Translation C[5]–C[7] | `0x00000000`, `0x00000000`, `0x00000000` |
| Light matrix C[8]–C[12] | Same five identity words as C[0]–C[4] |
| Ambient C[13]–C[15] | `0x00000000`, `0x00000000`, `0x00000000` |
| Color matrix C[16]–C[20] | Same five identity words as C[0]–C[4] |
| Projection C[24]–C[28] | `0x00800000`, `0x00600000`, `0x00000100`, `0x00000000`, `0x00000000` |
| Positions D[0]–D[5] | `0xffc0ffc0`, `0x00000200`, `0xffc00040`, `0x00000200`, `0x00400000`, `0x00000200` |

| Destination | Words in increasing register order |
|---|---|
| Lighting directions, replacing D[0]–D[5] after projection | `0x00001000`, `0x00000000`, `0x10000000`, `0x00000000`, `0x00000000`, `0x00001000` |
| Shared input color D[6] | `0x004080a0` |

All three matrices are Q12 identities. Position and translation words use integer spatial counts. The three vertex-lighting directions are deliberately supplied orthogonal directions, not normals computed from this planar triangle. They expose independent red, green and blue contributions so the GPU's color interpolation is visible. This is a data-flow example, not a physically smooth lighting model for a flat face.

Program the complete controls before issuing projection. Only the changing TRX word C[5] is rewritten independently in the loop. Keep the VBlank sequence, back-buffer index and completed-iteration count in software-owned writable storage; the interrupt handler and frame loop must use the same sequence word. After initialization, unmask only system VBlank bit 2 at `0x08000008`. The handler acknowledges that delivered source at `0x08000004` before advancing the sequence.

### 4.16.2 Keep the drawing buffer separate from the displayed buffer

The two drawing destinations occupy VRAM rows 0–191 and 256–447. Each has a 256×192 drawing area, using the GPU's 1024-word row stride. The drawing offset moves local triangle Y coordinates into the selected destination. The clear uses that same destination's absolute Y position. The two areas do not overlap, and both fit the installed video memory.

| Back-buffer index | Drawing origin Y | DISPFB1 low word | Displayed source |
|---:|---:|---|---|
| 0 | 0 | `0x000fa000` | 256×192 window starting at row 0 |
| 1 | 256 | `0x000fa040` | 256×192 window starting at row 256 |

The display source coordinates remain zero relative to the selected framebuffer base. The sequence retains the reset timing's horizontal signal step and sets DISPLAY1's extent for this 256×192 window. PMODE=`0x0000ff21` selects direct output from reading circuit 1.

The sequence first requests PCRTC RESET, which resets the drawing/display state; it does not use that as a GTE-reset operation. It explicitly initializes every GTE control consumed by its chosen calculations. The unused far-color and average-depth coefficients do not become inputs merely because their registers exist. Three screen and three color pushes replace the histories consumed by this sequence without a separate FIFO-clear sequence.

### 4.16.3 Observe the hand-offs in the loop

1. Select and clear the back buffer, never the currently displayed area.
2. Load the three position records and issue RTPT. Retain the three screen words for this primitive. NCLIP then evaluates that projected triangle.
3. Select positive signed doubled area, excluding zero. MAC0 is read as raw bits, so the sign-bit test is explicit; an unsigned `>0` comparison would accept a negative signed area too. These fixed operands cannot overflow MAC0. A general scene still needs its own overflow and clipping policy.
4. Load the supplied lighting directions into the same vector slots. NCCT applies the shared base color and leaves RGB0=`0x000000a0`, RGB1=`0x00008000`, RGB2=`0x00400000`. Submit the three matching coordinates and colors as one six-word Gouraud triangle packet. No implicit GTE-to-GPU transfer performs this step.
5. After the complete packet has been submitted, wait for GPU idle. Mask interrupt delivery, then program the finished buffer as the next display source.
6. After the final display store, acknowledge stale controller VBlank bit 2, sample the software sequence, then unmask VBlank and wait for a different sequence value. A pending request from before the stores cannot then masquerade as a new latch edge. An edge discarded by that acknowledgement can cost an extra refresh; the observed later edge has latched the source before the old front buffer is reused.
7. Exchange the back-buffer index and start another iteration.

Masking the system interrupt does not stop the PCRTC configuration-latch edge. The mask prevents an old request from advancing the software sequence during the stores and acknowledgement; it does not make the display stores atomic. Wait for the later observed edge before reusing storage. GPU completion does not follow from GTE CYCLES, nor display publication from GPU idle. These are separate ownership conditions.

### 4.16.4 Frame register and packet order

Before the loop, mask system interrupts, request PCRTC RESET with the word `0x00000200` at `0x080103d4`, set DISPFB1_HIGH to zero and DISPLAY1_HIGH to `0x000bf3ff`, then initialize the GTE controls above. Send GP0 words `0xe1000400` and `0xe6000000` for the chosen draw mode and mask behavior. Acknowledge stale interrupt sources with `0xffffffff` at `0x08000004`, then unmask VBlank with `0x00000004` at `0x08000008`.

For each iteration, let `b` be the back-buffer index (0 or 1), and let `y = 256 × b`. Send these six words in order to GP0 at `0x0801023c`:

| Order | GP0 word | Meaning |
|---|---|---|
| 1 | `0xe3000000 \| (y << 10)` | Drawing-area top-left (0, y) |
| 2 | `0xe40000ff \| ((y + 191) << 10)` | Drawing-area bottom-right (255, y+191) |
| 3 | `0xe5000000 \| (y << 11)` | Drawing offset (0, y) |
| 4 | `0x02000000` | Black fill command |
| 5 | `y << 16` | Absolute destination (0, y) |
| 6 | `0x00c00100` | Width 256, height 192 |

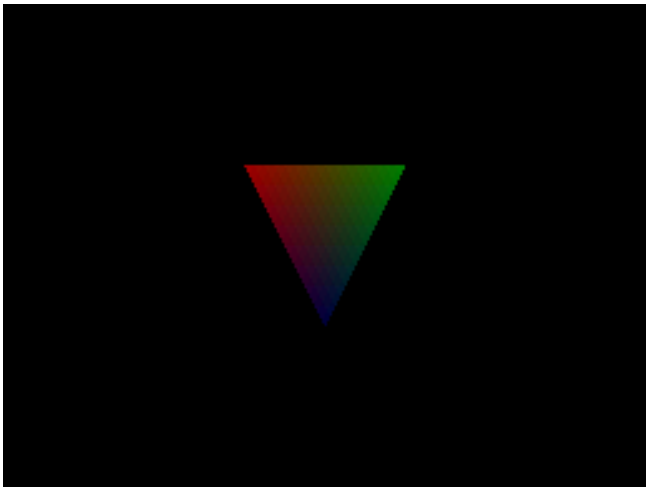
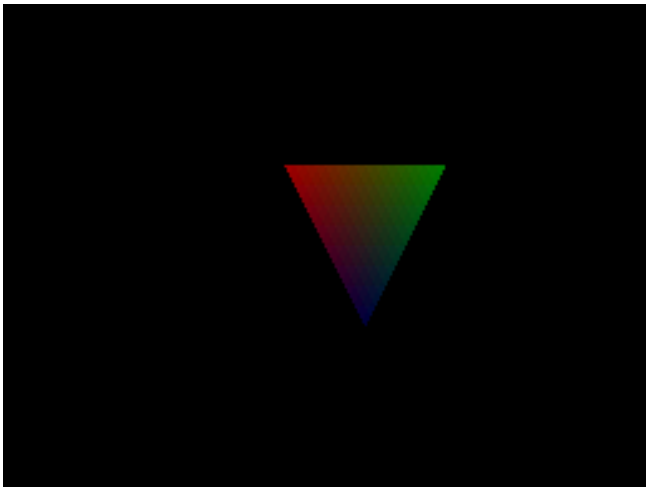
Write TRX as `((iteration >> 5) & 1) << 5`, transfer the position words, then issue RTPT `0x00080030`. Retain D[12]–D[14] before replacing the input vectors. Issue NCLIP `0x00000006` and select a nonzero MAC0 word whose bit 31 is clear. For this accepted triangle, transfer the lighting directions and input color, then issue NCCT `0x0008043f`. Submit the following complete six-word GP0 packet:

| Order | GP0 word |
|---|---|
| 1 | `0x30000000 \| (D[20] & 0x00ffffff)` |
| 2 | Retained first screen word |
| 3 | `D[21] & 0x00ffffff` |
| 4 | Retained second screen word |
| 5 | `D[22] & 0x00ffffff` |
| 6 | Retained third screen word |

Wait until GP1/status at `0x08010240` has bit 26 set. Write zero to IRQ_MASK at `0x08000008`, then `0x000fa000 | (b << 6)` to DISPFB1_LOW at `0x0801035c` and `0x0000ff21` to PMODE_LOW at `0x08010354`. After that final display store, write `0x00000004` to IRQ_ACK at `0x08000004`, sample the software VBlank sequence, then write `0x00000004` to IRQ_MASK. Wait for the sequence to change. Only then exchange `b` with `1-b`, increment the completed-iteration count and reuse the old displayed buffer.

### 4.16.5 Output plate: NCCT vertex-colour triangle

This sequence uses RTPT for projected coordinates, NCLIP for the winding result, and NCCT for three vertex colours. These plates show the complete 256 × 192 display images.

| Initial placement: `TRX = 0` | Translated placement: `TRX = 32` |
|---|---|
|  |  |
| **Initial output.** Projected vertices: `(96,64)`, `(160,64)`, `(128,128)`. | **Translated output.** Projected vertices: `(112,64)`, `(176,64)`, `(144,128)`. |

**Source and settings.** The position vectors are `(-64,-64,512)`, `(64,-64,512)` and `(0,64,512)`. The rotation is Q12 identity; translation is initially zero. Projection uses `H = 256` and offsets `(128 << 16, 96 << 16)`. Both lighting matrices are Q12 identity, with zero ambient. NCCT receives lighting directions `(4096,0,0)`, `(0,4096,0)` and `(0,0,4096)`, and input RGB word `0x004080a0`. RTPT uses shift-by-12; NCCT uses shift-by-12 and positive LM.

**Results.** At the initial placement, RGB0–RGB2 are `0x000000a0`, `0x00008000` and `0x00400000`. The visible triangle has red at upper left, green at upper right and blue at the bottom. Changing `TRX` from `0` to `32` moves all three projected vertices **16 pixels right**, with Y unchanged for these settings. The second image is the same visible gradient shifted exactly 16 pixels right.

**GTE and display output.** Software sends the GTE's three coordinate/colour pairs to GP0. The GTE supplies those vertex results, not the interior pixels. The plates show the subsequent display output, including the interior gradient; they do not specify the GPU's interpolation or edge-coverage rules.

## 4.17 A further geometry sequence

### 4.17.1 Translate before projection

Run the earlier **Project a triangle, test its orientation, and form a depth value** register sequence, not the repeated hardware frame sequence above. Continue from that sequence's retained state without resetting. Keep its identity matrix, H=256, screen origin `(128,96)`, and zero DQA/DQB. Change the translation to `(32,−16,512)` and project the new V0 `(64,32,512)`:

| Step | Address / register | Raw word | Meaning |
|---|---|---|---|
| 1 | `0x080102d8` — C[5] | `0x00000020` | TRX=32 |
| 2 | `0x080102dc` — C[6] | `0xfffffff0` | TRY=−16, signed complete word |
| 3 | `0x080102e0` — C[7] | `0x00000200` | TRZ=512 |
| 4 | `0x08010244` — D[0] | `0x00200040` | V0.X=64, V0.Y=32 |
| 5 | `0x08010248` — D[1] | `0x00000200` | V0.Z=512 |
| 6 | `0x08010344` — COMMAND | `0x00080001` | RTPS, SF=1, LM=0 |

The transformed position is `(64+32,32−16,512+512)=(96,16,1024)`. The new quotient is `16384`, a factor of ¼. Thus the screen result is `(128+96/4,96+16/4)=(152,100)`. Translation does not add a fixed number of screen units: its X/Y displacement is subject to perspective, while its Z displacement changes the denominator.

| Read immediately after RTPS | Raw result(s) |
|---|---|
| D[9], D[10], D[11] — IR1–IR3 | `0x00000060`, `0x00000010`, `0x00000400` |
| D[25], D[26], D[27] — MAC1–MAC3 | `0x00000060`, `0x00000010`, `0x00000400` |
| D[12], D[13], D[14] — screen history | `0x004000a0`, `0x00800080`, `0x00640098` |
| D[16], D[17], D[18], D[19] — depth history | `0x00000200`, `0x00000200`, `0x00000200`, `0x00000400` |
| D[8] — IR0; D[24] — MAC0 | `0x00000000`; `0x00000000` |
| D[7] — previous depth average | `0x000001ff`, unchanged |
| FLAG; CYCLES | `0x00000000`; `0x0000000f` — nominal 15 |

Only one new screen pair and one new depth were pushed. V1 and V2 remain programmed but were not processed by RTPS. The changed translation remains active for the next projection until the cartridge writes another translation.

## 4.18 Diagnosing numerical results

Check the arithmetic flags as well as the resulting screen coordinates. Follow the calculation in order: input halves and coefficient units, retained accumulator, visible MAC word, limited IR, depth and quotient, screen pair, then cue coefficient. A later stage can hide an earlier limit, and a later command replaces FLAG. Read the flag word belonging to the operation under investigation.

The cases below are independent. Begin each with matrix 0, translation, OFX/OFY, H, DQA/DQB and V0 components explicitly zero, then apply the stated changes. Other stored state is not an arithmetic input to these cases. Read the listed results immediately after the command. All numbers describe register contents or the signed arithmetic interpretation explicitly named; no case requires a register to hold a floating-point value.

### 4.18.1 An arithmetic shift is not rounding towards zero

Program C[0]=`0x00000800`, giving the X row coefficient 2048, and leave its other coefficients zero. MVMVA word `0x00086012` selects matrix 0, V0, no control vector, SF=1 and LM=0. Compare these two independent V0 inputs:

| X input | D[0] word, Y=0 | X product | Product ASR 12 | MAC1 and IR1 readback |
|---|---|---|---|---|
| 3 | `0x00000003` | 6144 | 1 | `0x00000001` |
| −3 | `0x0000fffd` | −6144 | −2 | `0xfffffffe` |

FLAG is zero in both cases. The negative result is not −1, and this difference is not saturation. With coefficient 1/2 under the Q12 convention, the exact results would be ±1.5; the fixed shift selects 1 and −2. Repeated transformations that discard fractions can accumulate directional error even while every FLAG is zero. Choose intermediate precision before assuming two algebraically equivalent command sequences will produce identical integers.

### 4.18.2 Visible-word wrap and accumulator overflow are different

First leave every matrix coefficient zero and program TRX=524288, C[5]=`0x00080000`. Issue MVMVA `0x00000012`: matrix0, V0, translation, SF=0, LM=0. The X accumulator is:

```
4096*524288 = 2147483648 = 0x000080000000
```

This fits signed44. The visible MAC1 word nevertheless becomes `0x80000000`, whose signed interpretation is −2147483648. IR1 clamps that signed value to −32768, read as `0xffff8000`. FLAG is `0x81000000`: ERROR and IR1 saturation, **without** either MAC1 signed44-overflow bit. Treating the visible word as a positive 2147483648, or assuming an IR clamp implies accumulator overflow, would diagnose the wrong stage.

Now change to a case with ordered signed44 overflow. Program C[0]= `0xf0001000`, meaning first-row coefficients `(4096,−4096,0)`, TRX= `0x7fffffff`, and V0 `(1,1,0)` using D[0]=`0x00010001`. Issue MVMVA `0x00080012`, selecting SF=1, LM=0 and translation. Track the additions:

| Stage | Retained signed44 value | New condition |
|---|---|---|
| `4096*TRX` | `2^43-4096` | None |
| Add `4096*1` | `-2^43` after wrap | MAC1 positive overflow |
| Add `-4096*1` | `2^43-4096` after wrap | MAC1 negative overflow |
| Add zero Z contribution | `2^43-4096` | No new condition |

The final mathematical contributions cancel, but the two intermediate wraps have occurred. MAC1 is `0x7fffffff` after SF=1; IR1 is `0x00007fff`. FLAG is **`0xc9000000`**: ERROR, both MAC1 overflow directions and IR1 saturation. Testing only the final row sum would lose both overflow events. The flags are cumulative within this command, not an attempt to classify the final word as exclusively positive-overflow or negative-overflow.

### 4.18.3 A limited IR can resemble an ordinary screen result

Use diagonal coefficients `(8192,4096,4096)`: C[0]=`0x00002000`, C[2]=C[4]=`0x00001000`, with the other matrix words zero. Set H=16 and V0 `(20000,0,512)`. Issue RTPS `0x00080001`, SF=1 and LM=0.

The transformed X is 40000, so MAC1=`0x00009c40`. IR1 becomes 32767, `0x00007fff`, because its signed-halfword range is narrower. Depth512 gives q=2048. Projection consequently uses **32767**, not 40000:

```
SX = (32767*2048) ASR 16 = 1023
```

SXY2 is `0x000003ff`. FLAG is `0x81000000`, reporting IR1 saturation but **not** screen-X saturation. The integer screen result is already at its upper endpoint, yet that stage did not clamp: its input shifted to exactly 1023. An endpoint coordinate therefore does not by itself identify which stage, if any, limited the calculation. The unclamped intended X would project to 1250, not 1023; the earlier IR limit has changed the geometry.

### 4.18.4 MAC0 can look harmless after a screen overflow

Use the Q12 identity matrix, H=256, V0 `(32767,0,512)`, and OFX= `0x7fffffff`; leave OFY, translation and cue constants zero. Issue RTPS `0x00080001`. The quotient is 32768 and the full screen-X expression is:

```
2147483647 + 32767*32768 = 3221192703
```

This exceeds signed32, setting MAC0 positive overflow. Its full fixed16 shift also exceeds the screen range, so SX becomes 1023 and screen-X saturation is set. Projection then calculates its final cue expression from zero DQA/DQB. The final visible MAC0 is therefore **zero**, and IR0 is zero, but FLAG is **`0x80014000`**: ERROR, MAC0 positive overflow and screen-X saturation. Reading final MAC0 alone misses the earlier event. The full screen expression, not its wrapped visible MAC0 word, supplied the screen shift.

### 4.18.5 The reciprocal boundary still publishes coordinates

Use the Q12 identity matrix, H=256, zero offsets/translation/cue constants, and V0.X=1, V0.Y=0. Issue RTPS `0x00080001` independently at each depth:

| V0.Z | New SZ3 | SXY2 | FLAG | Meaning |
|---|---|---|---|---|
| 129 | 129 | `0x00000001` | `0x00000000` | H is strictly less than twice depth. |
| 128 | 128 | `0x00000001` | `0x80020000` | Equality takes the divide-overflow path. |
| −1 | 0 | `0x00000001` | `0x80060000` | Depth clamps to zero; divide overflow also occurs. |

The screen pair is identical in these three cases. It does not establish that the input was in front of a usable near plane. At equality and at zero depth, q=`0x1ffff`; multiplication by X=1 followed by the fixed16 shift still gives SX=1. The negative-depth case also has IR3=−1 with LM=0; it is depth limiting, not a generic negative-IR rejection, that produces SZ3=0.

Every case pushes its result. None clips a triangle or prevents a subsequent NCLIP or draw packet. Preserve this FLAG before issuing NCLIP: NCLIP starts another flag calculation, and its area sign is a property of the already rounded/limited screen points. Arithmetic success, a usable spatial primitive, and a covered display pixel are three separate decisions.

## 4.19 Command reference

Choose a command by the result required, not by its nominal cycle count. Normal commands publish their results during the `COMMAND` write at `0x08010344`; the listed count is then readable from `CYCLES` at `0x08010348`. It does not specify a separate completion delay or busy period. VMAD3 uses the separate PLUS command window and delayed-publication protocol described in its entry.

For normal commands, the function is in bits 5:0. Add `0x00080000` for SF=1 and `0x00000400` for LM=1 to the function word shown in each entry. SF=0 performs no main-result shift; SF=1 shifts that result right by 12. LM=0 permits negative IR results down to −32768; LM=1 clamps them at zero. Both modes have an upper IR limit of 32767. Exceptions and forced intermediate clamps are stated below. Only MVMVA consumes the matrix, vector, and control-vector selectors; those fields do not redirect the operands of another command.

Every recognized normal command replaces normal FLAG, clearing it before calculation. Intermediate stages contribute flags even when a later stage overwrites their results. Triple commands collect flags across all three vectors or colors. See MAC, shift, and IR stages for the exact wrap, shift, clamp, and overflow tests, and normal FLAG for the summary mask. A color saturation flag, IR3 saturation flag, or IR0 saturation flag alone does not set ERROR.

The result lists below name stored registers. Other stored data and control words remain unchanged, apart from `COMMAND`, `CYCLES`, and FLAG. Derived register reads continue to follow their source registers. A color push shifts RGB0 ← old RGB1 and RGB1 ← old RGB2, then writes the new color to RGB2, occupying D[20]–D[22]. The new CODE byte always comes from RGBC, D[6]. A screen or depth push similarly updates the register history, not a queue of work awaiting execution.

### 4.19.1 Geometry commands

#### 4.19.1.1 RTPS — transform and project one vector

RTPS transforms V0 by the programmed rotation matrix and translation, projects its X/Y components, and appends one screen position and one depth to the histories. It also calculates IR0 for that vertex, ready for a subsequent depth-cue color calculation. Use it when processing positions individually. Use RTPT for a batch of the three vector slots, or MVMVA when only a matrix/vector result is required. RTPS neither clips a primitive nor submits drawing work; a saturated position is still published and pushed.

**Encoding and issue.** Write the command word to COMMAND, `0x08010344`:

```
word = 0x00000001 | (SF << 19) | (LM << 10)
```

| Bits | RTPS interpretation |
|---|---|
| 5:0 | Function `000001` |
| 19 — SF | 0: no shift of the three transformed MAC results; 1: arithmetic right shift by 12 |
| 10 — LM | Lower IR1–IR3 limit: −32768 when 0, zero when 1; upper limit always 32767 |
| 18:17, 16:15, 14:13 | Matrix, vector and control-vector selector fields are ignored: operands remain matrix 0, V0 and translation C[5]–C[7] |
| 31:20, 12:11, 9:6 | Ignored for execution |

The four zero-filled option words are `0x00000001` (SF=0, LM=0), `0x00000401` (SF=0, LM=1), `0x00080001` (SF=1, LM=0), and `0x00080401` (SF=1, LM=1). Ignored bits remain in COMMAND readback; they do not select a different vertex or suppress a stage.

**Inputs and a useful scale convention.** The following **integer-position/Q12-matrix convention** is a software choice: vector and translation words count position units, matrix coefficients represent their signed raw value divided by 4096, and screen offsets represent screen units multiplied by 65536. H is an unscaled unsigned projection numerator: under this equal-scale XYZ convention, it controls screen magnification, not a scene-length unit. Uniformly rescaling XYZ and translation leaves the intended picture unchanged with the same H, subject to the integer rounding and limits below. A cartridge may choose one screen unit to mean one pixel. With SF=1 and no overflow or saturation, the transform preserves the position unit; SF=0 leaves twelve additional fractional bits in its MAC/IR inputs instead. The hardware does not select a coordinate system, camera or handedness.

All addresses below accept aligned 32-bit register accesses. D[n] is at `0x08010244 + 4n`; C[n] is at `0x080102c4 + 4n`. A signed halfword uses two's complement, with range −32768…32767; a signed word has range −2147483648…2147483647.

| INPUT | Fields and raw representation | Interpretation under the named convention |
|---|---|---|
| D[0] — V0 X/Y | Signed X in bits 15:0; signed Y in bits 31:16 | Integer position units |
| D[1] — V0 Z | Signed Z in bits 15:0; upper halfword is not a component | Integer position units; a read sign-extends Z |
| C[0] | Signed R11 low / R12 high | Each coefficient is raw / 4096 |
| C[1] | Signed R13 low / R21 high | Same |
| C[2] | Signed R22 low / R23 high | Same |
| C[3] | Signed R31 low / R32 high | Same |
| C[4] | Signed R33 low | Same; writing this register sign-extends its low halfword |
| C[5], C[6], C[7] — TRX, TRY, TRZ | Three signed complete words | Integer translation in the vector's position units, not Q12 coefficients |
| C[24], C[25] — OFX, OFY | Two signed complete words | Raw / 65536 screen units; added before the fixed 16-bit screen shift |
| C[26] — H | Unsigned low halfword, 0…65535 | Unscaled unsigned projection numerator; with equal-scale XYZ it controls magnification in screen units, not spatial/depth units. The register **reads sign-extended** after a write, but RTPS uses its unsigned low half |
| C[27] — DQA | Signed low halfword; sign-extended on write/read | Multiplies the unsigned projection quotient; if IR0 / 4096 is a normalized cue weight, DQA / 256 is its gain on the approximate H/depth ratio |
| C[28] — DQB | Signed complete word | Added before the fixed 12-bit IR0 shift; DQB / 16777216 is the corresponding normalized cue-weight offset |
| Old D[13], D[14], D[17]–D[19] | Existing screen/depth history words | Retained history entries, not further vertices to process |

The old IR values, old MAC values, V1/V2, RGBC and color histories are not arithmetic operands. In particular, loading V1 does not make RTPS process it.

**Register footprint and history lifetime.** Each command replaces the following results, whether or not it sets a flag:

| MODIFIED | Effect |
|---|---|
| D[9]–D[11] — IR1–IR3 | Clamped transformed components, read as sign-extended 16-bit values |
| D[25]–D[27] — MAC1–MAC3 | Shifted and word-wrapped transform results; not necessarily equal to IR |
| D[12]–D[14] — SXY0–SXY2 | `(old SXY1, old SXY2, new screen pair)`; old SXY0 is discarded |
| D[16]–D[19] — SZ0–SZ3 | `(old SZ1, old SZ2, old SZ3, new depth)`; old SZ0 is discarded |
| D[8] — IR0 | New depth-cue coefficient, 0…4096 |

| MODIFIED | Effect |
|---|---|
| D[24] — MAC0 | Low word of the **depth-cue** expression, replacing both screen expressions |
| C[31] — FLAG | Cleared at command start, then accumulates conditions from every stage |
| COMMAND; CYCLES | Last raw command word; nominal duration `0x0000000f` |

D[15] reads the new SXY2; it is not a fourth retained screen position. D[28] and D[29] read a packed value derived from the new IR1–IR3, even though RTPS does not write a packed-color latch or push a color. Each derived channel is `clamp(IR ASR 7,0,31)`, packed red in bits 4:0, green in 9:5 and blue in 14:10.

**PRESERVED:** all other stored register state. This includes V0–V2, RGBC, OTZ, RGB0–RGB2, D[23], the D[28] input latch, the leading-sign source/count, all C[0]–C[30], and the complete PLUS register set. Reading the derived ports is not evidence that an RGB history entry changed. Save a needed FLAG or IR0 before issuing another normal command; save a needed screen/depth entry before subsequent pushes displace it.

**Ordered calculation.** The equations below use signed integer arithmetic. `ASR(x,k)` divides by `2^k` with negative results rounded toward minus infinity; it does not truncate negative fractions toward zero. `signed44(x)` means the signed two's-complement value of the low 44 bits, and `signed32(x)` means the signed value of the low 32 bits. `clamp(x,a,b)` limits x to the inclusive interval a…b. These are arithmetic definitions, not additional operands or programmable modes.

1. **Rotation and translation.** Obtain signed `(vx,vy,vz)` from V0. For each row i=1,2,3, start with `4096 × TR_i`, then add the X, Y and Z products **in that order**, wrapping to signed 44 bits after each addition:

   ```
   A1 = signed44(signed44(signed44(4096×TRX + R11×vx) + R12×vy) + R13×vz)
   A2 = signed44(signed44(signed44(4096×TRY + R21×vx) + R22×vy) + R23×vz)
   A3 = signed44(signed44(signed44(4096×TRZ + R31×vx) + R32×vy) + R33×vz)
   ```

   The initial translation term also has signed-44 interpretation. Retain any positive or negative 44-bit overflow condition from an intermediate addition, even if a later addition brings the result back into range. The unshifted A values have 4096 raw counts per transformed position unit under the named convention.

2. **Visible MAC and IR results.** For each i, write `MAC_i = signed32(ASR(A_i,12×SF))`. This is a word wrap, not a signed-32 saturation. IR1 and IR2 are their respective signed MAC words clamped to `[L,32767]`, where `L=−32768` for LM=0 and `L=0` for LM=1. Each actual IR1/IR2 clamp sets its saturation flag.

   IR3's **value** is likewise `clamp(signed MAC3,L,32767)`, but its **flag** is exceptional: set IR3 saturation precisely when `ASR(A3,12)` lies outside **−32768…32767**, regardless of SF and LM. An SF=0 IR3 output can therefore clamp without setting this flag. An LM=1 negative IR3 can clamp to zero without setting it. Conversely, IR3 saturation can be reported from the fixed-12 test even when a wrapped SF=0 MAC3 happens to fit the output range.

3. **Fixed-12 depth.** Form `z = ASR(A3,12)` from the retained 44-bit result, **not** from IR3 or the visible MAC3 word. Push `SZ3 = clamp(z,0,65535)` with the depth-history shift specified above. A clamp sets the depth saturation flag. This shift and the unsigned depth range do not change with SF or LM. With SF=1 under the named convention, z has integer position units even when the transform contained a fractional result.

4. **Integer reciprocal projection.** Let `n=unsigned H` and `d=new SZ3`. The quotient q has 65536 counts per unit ratio. It approximates `65536×H/d`, but is produced by a seed-ROM refinement, **not exact rational division**. If `n >= 2×d`, including `d=0` and even `n=d=0`, set divide overflow and use `q=131071` (`0x1ffff`). Otherwise normalize the nonzero depth and calculate:

   ```
   s          = number of leading zero bits in the 16-bit d
   r1         = (d << s) & 0x7fff
   r2         = T[(r1 + 0x40) >> 7] + 0x101
   r3         = ((low32(0x80 - r2×(r1 + 0x8000))) >> 8) & 0x1ffff
   reciprocal = (r2×r3 + 0x80) >> 8
   rawq       = (reciprocal×(n << s) + 0x8000) >> 16
   q          = min(rawq,131071)
   ```

   `low32` retains unsigned low 32 bits; its following shift is unsigned. The other shifts here act on nonnegative integers, and the final multiply retains its full product before shifting. T is the exact 257-byte table in Appendix: reciprocal seed ROM. It supplies a reciprocal seed for the normalized depth; the following products refine and rescale that seed. A `rawq` limit in this otherwise valid branch does **not** set divide overflow. For unsaturated positive depths, `H < 2×SZ3` avoids the divide-overflow branch; it is not a geometric clipping rule.

5. **Screen position.** Using the **clamped** signed IR1 and IR2, form the full expressions in X-then-Y order:

```
screenX = OFX + IR1×q
sx      = clamp(ASR(screenX,16),−1024,1023)
screenY = OFY + IR2×q
sy      = clamp(ASR(screenY,16),−1024,1023)
SXY2    = (sx & 0xffff) | ((sy & 0xffff) << 16)
```

   Each full expression is written through MAC0, testing signed-32 overflow and exposing its low word, but the screen shift consumes the **full expression**, not that low word. Under the SF=1 convention, `IR×q` and the offset both have 65536 raw counts per screen unit. Each out-of-range screen result sets its own saturation flag. Push the packed pair after shifting the old SXY1/SXY2 into SXY0/SXY1. Neither a flag nor a negative depth prevents this push.

6. **Single-vertex depth cue.** RTPS always performs the final-vertex calculation:

```
cue   = DQB + DQA×q
MAC0  = low32(cue)
IR0   = clamp(ASR(cue,12),0,4096)
```

   Test the full cue expression for signed-32 MAC0 overflow, and use that same full expression for the fixed-12 shift. Set IR0 saturation only if the shifted value is strictly below zero or above 4096. The final IR0 is a raw Q12 weight if software chooses to interpret it that way. With that interpretation, before quantization/clamping its weight is `DQB/16777216 + (DQA/256)×(q/65536)`. It is a function of the reciprocal quotient, not a linear function of the SZ3 word. SF and LM do not alter this stage. No color command is performed automatically.

**FLAG conditions.** RTPS starts a new FLAG word. Conditions are cumulative across all six stages, including the two MAC0 expressions subsequently overwritten by the cue expression; arithmetic continues after every condition.

| Bit(s) / mask | Set condition |
|---|---|
| 30, 29, 28 / `0x40000000`, `0x20000000`, `0x10000000` | Positive signed-44 overflow during the ordered row 1, 2 or 3 accumulation |
| 27, 26, 25 / `0x08000000`, `0x04000000`, `0x02000000` | Negative signed-44 overflow during the corresponding accumulation |
| 24 / `0x01000000`; 23 / `0x00800000` | Signed visible MAC1 or MAC2 lies outside the selected LM interval |
| 22 / `0x00400000` | `ASR(A3,12) < −32768` or `> 32767`; **not** a test of the selected IR3 clamp |
| 18 / `0x00040000` | Fixed-12 depth is below 0 or above 65535 |
| 17 / `0x00020000` | Unsigned `H >= 2×new SZ3` |
| 16 / `0x00010000`; 15 / `0x00008000` | Any full screen-X, screen-Y or cue expression exceeds 2147483647 or is below −2147483648 |
| 14 / `0x00004000`; 13 / `0x00002000` | Shifted screen X or Y is outside −1024…1023 |
| 12 / `0x00001000` | Fixed-12 cue result is outside 0…4096 |
| 31 / `0x80000000` — ERROR | Any condition in bits 30:23 or 18:13; equivalently, any bit selected by `0x7f87e000` |

The signed-44 interval is `−2^43…2^43−1`. Endpoints of all stated clamp intervals are legal and do not themselves set a flag. IR3 saturation alone and IR0 saturation alone do not set ERROR. RTPS does not set color-channel flags in bits 21:19; bits 11:0 remain zero. ERROR is an arithmetic summary, not a clipped-vertex or visibility classification.

**Worked transform with fractional offsets and an inexact reciprocal.** Program the following independent input state. The matrix is an exact quarter-turn in XY; its displayed positive-rotation direction depends on the software coordinate convention.

| Register(s) | Raw word(s) | Meaning |
|---|---|---|
| C[0]–C[4] | `0xf0000000`, `0x10000000`, `0x00000000`, `0x00000000`, `0x00001000` | Rows `(0,−4096,0)`, `(4096,0,0)`, `(0,0,4096)` |
| C[5]–C[7] | `0x0000000c`, `0xffffff8`, `0x0000012c` | Translation `(12,−8,300)` |
| D[0], D[1] | `0xffd3001e`, `0x000002bc` | V0 `(30,−45,700)` |
| C[24], C[25] | `0x00804000`, `0x00608000` | Screen origin `(128.25,96.5)` |
| C[26] | `0x0000014d` | H=333 |

| Register(s) | Raw word(s) | Meaning |
|---|---|---|
| C[27], C[28] | `0xffffff00`, `0x00800000` | DQA=−256; DQB=8388608 |
| D[12]–D[14] | `0x00020001`, `0x00040003`, `0x00060005` | Existing screen positions `(1,2)`, `(3,4)`, `(5,6)` |
| D[16]–D[19] | `0x0000006f`, `0x000000de`, `0x0000014d`, `0x000001bc` | Existing depths 111, 222, 333, 444 |
| COMMAND | `0x00080001` | RTPS, SF=1, LM=0 |

The rotation produces `(45,30,700)` before translation. The ordered accumulations finish at `(233472,90112,4096000)`, so MAC1–MAC3 and IR1–IR3 are `(57,22,1000)`. The new depth is 1000. The divide normalizes with `s=6`, `r1=31232`, table index 244, `T[244]=5`, `r2=262`, `r3=65572` and `reciprocal=67109`. Thus `rawq=(67109×21312+32768)>>16=21824`, with no divide overflow. Exact mathematical `65536×333/1000` is 21823.488; substituting it is not this datapath.

The screen expressions are `8404992+57×21824=9648960` and `6324224+22×21824=6804352`. Their fixed-16 shifts produce `(147,103)`. The cue expression is `8388608−256×21824=2801664`; its fixed-12 shift gives IR0=684, a normalized weight of 684/4096.

| Result | Raw readback |
|---|---|
| D[25]–D[27] — MAC1–MAC3 | `0x00000039`, `0x00000016`, `0x000003e8` |
| D[9]–D[11] — IR1–IR3 | `0x00000039`, `0x00000016`, `0x000003e8` |
| D[12]–D[14] — screen history | `0x00040003`, `0x00060005`, `0x00670093` |
| D[15] — newest-screen alias | `0x00670093` |
| D[16]–D[19] — depth history | `0x000000de`, `0x0000014d`, `0x000001bc`, `0x000003e8` |
| D[24] — MAC0; D[8] — IR0 | `0x002ac000`; `0x000002ac` |
| D[28], D[29] — derived packed reads | Both `0x00001c00`, from `(0,0,7)` after the three IR shifts |
| FLAG; CYCLES | `0x00000000`; `0x0000000f` |

**Boundary and option contrast.** Start each row below independently with an identity Q12 matrix (`C[0]`, `C[2]` and `C[4]` all equal `0x00001000`, C[1]/C[3]=0), zero translation, zero OFX/OFY and zero DQA/DQB. Load V0 `(−1,1,64)` using D[0]=`0x0001ffff`, D[1]=`0x00000040`; set H=128. The retained accumulations are (`−4096`, `4096`, `262144`) in every row. Depth remains 64 for **both** SF settings. Since `H=2×depth`, the equality takes the overflow branch and q is 131071, slightly less than a factor of two.

| Command / options | Signed MAC1–MAC3 | IR1–IR3 | New `(sx,sy)` / SXY2 word | FLAG |
|---|---|---|---|---|
| `0x00080001` / SF=1, LM=0 | `(−1,1,64)` | `(−1,1,64)` | `(−2,1)` / `0x0001fffe` | `0x80020000` — ERROR, divide |
| `0x00080401` / SF=1, LM=1 | `(−1,1,64)` | `(0,1,64)` | `(0,1)` / `0x00010000` | `0x81020000` — additionally IR1 |
| `0x00000001` / SF=0, LM=0 | (`−4096`, `4096`, `262144`) | (`−4096`, `4096`, `32767`) | `(−1024,1023)` / `0x03fffc00` | `0x80026000` — ERROR, divide, SX, SY |
| `0x00000401` / SF=0, LM=1 | (`−4096`, `4096`, `262144`) | `(0,4096,32767)` | `(0,1023)` / `0x03ff0000` | `0x81022000` — ERROR, IR1, divide, SY |

All four rows end with IR0=0 and MAC0=0. The first row's negative screen X rounds to −2 while positive Y truncates to 1; the quotient was not exactly two. The SF=0 rows clamp IR3 to 32767 **without** setting bit 22, because its flag test sees fixed-12 depth 64. SF=0 does not compensate the screen scale: its enlarged IR1/IR2 values are still multiplied by the same q and shifted by 16.

For the other IR3 asymmetry, change only V0.Z to −1 and use SF=1, LM=1. IR3 becomes zero, but its fixed-12 flag test sees −1, which is inside the signed range: IR3 saturation remains clear. SZ3 does clamp to zero, divide overflow still occurs, and FLAG becomes `0x81060000` (ERROR, IR1, depth saturation, divide). SXY2 remains `0x00010000`. A zero IR3 or a zero SZ3 must therefore not be taken as proof that the input was a usable visible vertex.

**Timing.** RTPS writes 15 to normal CYCLES. Results are available when its COMMAND write completes.

#### 4.19.1.2 RTPT — transform and project three vectors

RTPT transforms and projects V0, V1 and V2 in that order with one shared matrix, translation and projection setup. It leaves their three projected screen positions together in SXY0–SXY2 and their three depths together in SZ1–SZ3. Use it to prepare a triangle for subsequent screen-area or depth-average calculations. It does not test winding, clip, reject, draw, or retain three depth-cue coefficients. Only V2 produces the final IR0.

Choose RTPS instead when software must inspect transformed components, preserve a separate FLAG, or use a depth-cue coefficient between individual vertices. A batch's final IR/MAC values describe V2, whereas its FLAG can describe any of the three vertices. RTPT does not make a vertex usable merely by returning a packed screen position.

**Encoding and issue.** Write COMMAND at `0x08010344`:

```
word = 0x00000030 | (SF << 19) | (LM << 10)
```

| Bits | RTPT interpretation |
|---|---|
| 5:0 | Function `110000` |
| 19 — SF | 0: no main transform-result shift; 1: arithmetic right shift by 12 |
| 10 — LM | IR1–IR3 lower limit: −32768 when 0, zero when 1; upper limit always 32767 |
| 18:17, 16:15, 14:13 | Matrix/vector/control-vector selectors ignored; they cannot select another batch or transform |
| 31:20, 12:11, 9:6 | Ignored for execution |

The four option words with ignored bits zero are `0x00000030` (SF=0, LM=0), `0x00000430` (SF=0, LM=1), `0x00080030` (SF=1, LM=0), and `0x00080430` (SF=1, LM=1). One SF and LM setting applies to **every** vertex in the batch. Ignored bits remain readable as part of the last COMMAND word.

**Inputs and representation.** D[n] is at `0x08010244 + 4n`; C[n] is at `0x080102c4 + 4n`. Use aligned 32-bit register accesses. Signed halfwords range from −32768 to 32767 and signed complete words from −2147483648 to 2147483647, using two's complement; the halves of packed words are not floating-point fields. The upper halves of D[1], D[3] and D[5] are not extra components, and reads of those registers sign-extend Z.

| INPUT | Raw representation and use |
|---|---|
| D[0], D[1] — V0 | X signed low16 and Y signed high16 of D[0]; Z signed low16 of D[1] |
| D[2], D[3] — V1 | X signed low16 and Y signed high16 of D[2]; Z signed low16 of D[3] |
| D[4], D[5] — V2 | X signed low16 and Y signed high16 of D[4]; Z signed low16 of D[5] |
| C[0] — matrix 0 | Signed R11 low / R12 high |
| C[1] | Signed R13 low / R21 high |
| C[2] | Signed R22 low / R23 high |
| C[3] | Signed R31 low / R32 high |
| C[4] | Signed R33 low; writing this register sign-extends the low halfword |
| C[5]–C[7] — translation | Signed complete words TRX, TRY, TRZ, shared by all three vertices |
| C[24], C[25] — OFX, OFY | Signed complete screen-offset words |
| C[26] — H | Unsigned low16 projection numerator; although register readback is sign-extended after a write, the projection consumes an unsigned value |
| C[27] — DQA | Signed low16 cue gain, sign-extended on write/read; consumed only after projecting V2 |
| C[28] — DQB | Signed complete cue offset, consumed only after projecting V2 |
| Old D[19] — SZ3 | Retained history value that becomes final SZ0 |

Under the **integer-position/Q12-matrix convention**, all vector and translation components count the same position unit, matrix coefficients represent raw/4096, and SF=1 returns transformed integer-position components. OFX/OFY represent raw/65536 screen units; software may choose pixels for those screen units. H is an unscaled unsigned numerator controlling screen magnification under this convention, **not** a value required to share scene/depth units. Uniformly rescaling XYZ and translation leaves the intended picture unchanged with H held fixed, subject to integer rounding and range limits.

DQA multiplies the integer projection quotient, not SZ3. DQB is added before the fixed 12-bit cue shift. Interpreting IR0/4096 as a normalized weight gives gain DQA/256 on the numerical quotient q/65536 and offset DQB/16777216. These are useful software scales; the registers contain raw signed integers.

Old IR values, old MAC values, RGBC and RGB histories are not operands. The old screen history does not affect the three new screen positions. Initial SZ0–SZ2 are displaced; only the old SZ3 survives in the final depth history. All vertices share the same controls: the command offers no per-vertex matrix, H, offset or cue-constant change inside the batch.

**Ordered calculation.** The following sequence is performed for j=0, then j=1, then j=2. FLAG is cleared **once before j=0**, not between vertices. `ASR(x,k)` divides by `2^k`, rounding negative fractions toward minus infinity. `signed44` retains and interprets the low44 bits; `signed32` does the same for low32. `low32` is the unsigned low-word bit pattern. Clamp bounds are inclusive.

1. **Transform Vj.** For the current signed `(vx,vy,vz)`, accumulate each row in translation, X, Y, Z order, wrapping signed44 after each addition:

   ```
   A1 = signed44(signed44(signed44(4096×TRX + R11×vx) + R12×vy) + R13×vz)
   A2 = signed44(signed44(signed44(4096×TRY + R21×vx) + R22×vy) + R23×vz)
   A3 = signed44(signed44(signed44(4096×TRZ + R31×vx) + R32×vy) + R33×vz)
   ```

   The initial translation term also has signed44 interpretation. Any signed44 overflow at an intermediate addition contributes its lane/direction flag even if later terms cancel it. Under the named convention A has 4096 counts per transformed position unit.

2. **Publish this vertex's transformed MAC/IR.** Write `MAC_i=signed32(ASR(A_i,12×SF))` for i=1…3. Clamp signed MAC1 and MAC2 to `[L,32767]` for IR1/IR2, where L is −32768 for LM=0 and zero for LM=1; actual clamps set their respective flags. IR3's value is the same selected-LM clamp of signed MAC3, but its saturation flag instead tests `ASR(A3,12)` against **−32768…32767**, independently of SF and LM. Thus SF=0 may clamp IR3 without its flag, and LM=1 may clamp a negative IR3 to zero without its flag. MAC word wrap itself is not signed32 saturation.

3. **Push this vertex's depth.** Calculate `d_j=clamp(ASR(A3,12),0,65535)`, setting depth saturation if a clamp occurs. Shift `(SZ0,SZ1,SZ2,SZ3)` to `(old SZ1,old SZ2,old SZ3,d_j)`. Use the retained A3, **not** clamped IR3 or the public MAC3 word. The depth shift remains exactly 12 for both SF settings and both LM settings.

4. **Obtain this vertex's projection quotient.** Use unsigned H and the just-produced d_j. If `H >= 2×d_j`, including zero depth, set divide overflow and use `q_j=131071` (`0x1ffff`). Otherwise apply the exact normalized-depth seed-ROM reciprocal refinement specified in Reciprocal divide, then limit its rounded unsigned result to 131071. This valid-branch limiting does not set divide overflow. There are 65536 quotient counts per unit numerical ratio; `65536×H/d_j` is an approximation to the intent, **not** an interchangeable implementation of the prescribed divide. Each vertex obtains its own quotient from its own clamped depth, not from an average depth or the previous vertex's result.

5. **Push this vertex's screen pair.** With the current clamped signed IR1/IR2, calculate X then Y:

   ```
   screenX = OFX + IR1×q_j
   sx_j    = clamp(ASR(screenX,16),−1024,1023)
   screenY = OFY + IR2×q_j
   sy_j    = clamp(ASR(screenY,16),−1024,1023)
   P_j     = (sx_j & 0xffff) | ((sy_j & 0xffff) << 16)
   (SXY0,SXY1,SXY2) = (old SXY1,old SXY2,P_j)
   ```

   Each full screen expression passes through MAC0: signed32 overflow contributes flags and its low32 bits become the visible word, but the fixed-16 screen shift uses the **full expression**. Screen X/Y clamps have separate flags. The pair is pushed even after overflow or saturation. Under SF=1 and the named convention, the offset and IR×q terms have 65536 counts per screen unit.

6. **Only after j=2, calculate the cue coefficient.** Neither j=0 nor j=1 performs this stage or replaces IR0. After V2:

   ```
   cue  = DQB + DQA×q_2
   MAC0 = low32(cue)
   IR0  = clamp(ASR(cue,12),0,4096)
   ```

The full cue expression supplies both the signed32 MAC0 overflow test and the fixed-12 IR0 calculation. A shifted value strictly outside 0…4096 sets IR0 saturation. SF and LM do not change this stage. RTPT has not retained the cue expressions or coefficients for V0 and V1, and has not applied a color operation to any vertex.

**Register footprint and exact FIFO trace.** Let initial screen words be `(A,B,C)`, initial unsigned depth words be `(a,b,c,d)`, and new screen/depth results be `(P0,z0)`, `(P1,z1)`, `(P2,z2)` for V0/V1/V2. The trace is:

| Point in command | SXY0, SXY1, SXY2 | SZ0, SZ1, SZ2, SZ3 | MAC1–MAC3 / IR1–IR3 | IR0 |
|---|---|---|---|---|
| Before V0 | A, B, C | a, b, c, d | Previous state | Previous state |
| After V0 | B, C, P0 | b, c, d, z0 | V0 results | Unchanged |
| After V1 | C, P0, P1 | c, d, z0, z1 | V1 results | Unchanged |
| After V2 | P0, P1, P2 | d, z0, z1, z2 | V2 results | Cue from V2 |

The intermediate rows specify calculation order and flag accumulation, not separate asynchronous result-publication events. All normal command results are available when the one COMMAND write completes.

| MODIFIED | Final state |
|---|---|
| D[12]–D[14] | V0/V1/V2 screen pairs in that order; all initial screen pairs displaced |
| D[16]–D[19] | Old SZ3, then V0/V1/V2 depths in that order |
| D[25]–D[27]; D[9]–D[11] | V2's shifted/wrapped MAC words; V2's clamped IR components |
| D[24]; D[8] | Low32 of V2 cue expression; V2 cue coefficient |
| C[31] | Combined conditions from all vertices and the one final cue stage |
| COMMAND; CYCLES | Last raw command word; nominal count `0x00000017` |

D[15] reads final SXY2, hence V2's screen pair, not an additional vertex. Reads of D[28]/D[29] derive from final IR1–IR3: clamp each `IR ASR 7` to 0…31 and pack the results into bits 4:0, 9:5 and 14:10. These derived read values may change without a color-history push or a write to the D[28] input latch.

**PRESERVED:** every other stored register word, including the three input vectors, RGBC, OTZ, RGB0–RGB2, D[23], the packed-input latch, leading-sign source/count, C[0]–C[30] and all PLUS state. There is no per-vertex FLAG or retained V0/V1 MAC/IR result. NCLIP can consume the three screen pairs next; AVSZ3 can consume SZ1–SZ3. Preserve the RTPT FLAG before either following command replaces it, and preserve the histories before later projection pushes displace them.

**FLAG conditions.** A condition from V0 remains set even if V1/V2 produce in-range results. The batch does not identify which vertex set a flag.

| Bit(s) / mask | RTPT condition |
|---|---|
| 30:28 / `0x40000000`, `0x20000000`, `0x10000000` | Positive signed44 overflow in row 1/2/3 of any vertex |
| 27:25 / `0x08000000`, `0x04000000`, `0x02000000` | Negative signed44 overflow in row 1/2/3 of any vertex |
| 24 / `0x01000000`; 23 / `0x00800000` | Any vertex's signed visible MAC1/MAC2 outside the selected IR clamp interval |
| 22 / `0x00400000` | Any vertex's full A3 ASR12 outside −32768…32767, regardless of selected SF/LM |
| 18 / `0x00040000` | Any fixed-12 depth below 0 or above 65535 |
| 17 / `0x00020000` | H is at least twice any vertex's newly clamped depth |
| 16 / `0x00010000`; 15 / `0x00008000` | Any full screen-X/screen-Y expression, or V2's cue expression, above 2147483647 or below −2147483648 |
| 14 / `0x00004000`; 13 / `0x00002000` | Any shifted screen X/Y outside −1024…1023 |
| 12 / `0x00001000` | **V2's** shifted cue value outside 0…4096 |
| 31 / `0x80000000` | Summary of bits selected by `0x7f87e000`: bits 30:23 and 18:13 |

Signed44 range is `-2^43…2^43-1`; equality at a clamp endpoint does not set a saturation flag. IR3 or IR0 saturation alone does not set ERROR. RTPT leaves color-channel flags 21:19 and low bits 11:0 clear. An SF=0 transform still uses fixed-12

depth and fixed-16 screen stages; it does not automatically restore the SF=1 screen scale. LM=1 can remove negative X/Y components before projection rather than merely changing a final sign flag.

**Worked mixed-depth triangle.** Use the integer-position/Q12-matrix convention, SF=1 and LM=0. Program the following state; no particular old MAC/IR, color or OTZ values are required.

| Register(s) | Raw word(s) | Decoded setup |
|---|---|---|
| C[0]–C[4] | `0xf0000000`, `0x10000000`, `0x00000000`, `0x00000000`, `0x00001000` | Rows `(0,−4096,0)`, `(4096,0,0)`, `(0,0,4096)` |
| C[5]–C[7] | `0x0000000c`, `0xffffff8`, `0x0000012c` | Translation `(12,−8,300)` |
| D[0], D[1] | `0xffd3001e`, `0x00000064` | V0 `(30,−45,100)` |
| D[2], D[3] | `0xff88ff74`, `0x0000012c` | V1 `(−140,−120,300)` |
| D[4], D[5] | `0x00500096`, `0x000004b0` | V2 `(150,80,1200)` |
| C[24], C[25]; C[26] | `0x00804000`, `0x00608000`; `0x0000014d` | Offset `(128.25,96.5)`; H=333 |
| C[27], C[28] | `0xffffff00`, `0x00800000` | DQA=−256; DQB=8388608 |
| D[12]–D[14] | `0x00020001`, `0x00040003`, `0x00060005` | Old screen pairs `(1,2)`, `(3,4)`, `(5,6)` |
| D[16]–D[19] | `0x0000006f`, `0x000000de`, `0x0000014d`, `0x000001bc` | Old depths 111, 222, 333, 444 |
| COMMAND | `0x00080030` | RTPT, SF=1, LM=0 |

The quarter-turn followed by translation gives the three transformed vectors below. Every transform MAC and IR is in range; each depth satisfies `H < 2×depth`.

| Vertex | Unshifted `(A1,A2,A3)` | Transformed MAC/IR | New depth | Integer q | Full screen X / Y | Screen pair / raw word |
|---|---|---|---|---|---|---|
| V0 | (`233472`, `90112`, `1638400`) | `(57,22,400)` | 400 | 54559 | 11514855 / 7524522 | `(175,114)` / `0x007200af` |
| V1 | (`540672`, `−606208`, `2457600`) | (`132`, `−148`, `600`) | 600 | 36372 | 13206096 / 941168 | `(201,14)` / `0x000e00c9` |
| V2 | (`−278528`, `581632`, `6144000`) | (`−68`, `142`, `1500`) | 1500 | 14549 | 7415660 / 8390182 | `(113,128)` / `0x00800071` |

For example, V1 screen Y is `6324224−148×36372=941168`; ASR16 gives 14. Its negative transformed Y is retained with LM=0 rather than clamped away. The final cue is calculated only from q2: `8388608−256×14549=4664064`, and ASR12 gives IR0=1138. The analogous expressions for V0/V1 would be negative, but RTPT does **not** calculate or clamp them. They cannot set IR0 saturation.

| Final readback | Raw result |
|---|---|
| D[12]–D[14]; D[15] | `0x007200af`, `0x000e00c9`, `0x00800071`; alias `0x00800071` |
| D[16]–D[19] | `0x000001bc`, `0x00000190`, `0x00000258`, `0x000005dc` — depths 444, 400, 600, 1500 |
| D[25]–D[27]; D[9]–D[11] | Both groups `0xfffffbc`, `0x0000008e`, `0x000005dc` — V2 only |
| D[24]; D[8] | `0x00472b00`; `0x00000472` |
| D[28], D[29] | Both `0x00002c20`, derived from final five-bit components `(0,1,11)` |
| FLAG; CYCLES | `0x00000000`; `0x00000017` |

**Contrasts that expose batch lifetime.** Change only V0.Z to −200 (`D[1]=0xffffff38`) and issue the same command from the same initial histories. Its transformed depth is now 100; since `333 >= 2×100`, V0 sets divide overflow and uses q0=131071.

Its screen pair is `(242,140)`, word `0x008c00f2`. The later two projections still produce their original results. Final FLAG is `0x80020000`, while final IR1–IR3, MAC1–MAC3, IR0 and MAC0 are **identical** to the all-in-range example. Final depth history is `(444,100,600,1500)`. Looking only at V2's final registers would miss V0's divide condition.

Alternatively, retain the original three vertices but select LM=1 with `0x00080430`. V1's negative Y and V2's negative X clamp to zero. The screen pairs become `(175,114)`, `(201,96)`, `(128,128)`; depths and quotients do not change. Final MAC is still (`−68`, `142`, `1500`), but final IR is `(0,142,1500)` and FLAG is `0x81800000` (ERROR, IR1 and IR2 saturation). This is a changed projection, not a harmless sign-suppression option.

**Timing.** RTPT writes 23 (`0x00000017`) to normal CYCLES. All three projections and the final depth-cue result are available when its COMMAND write completes.

#### 4.19.1.3 NCLIP — calculate doubled oriented screen area

NCLIP calculates the signed two-dimensional determinant of the three positions already in SXY0–SXY2. Use it after projecting a triangle when its screen-coordinate orientation or degeneracy is needed. Despite its name, it does **not** clip a primitive, decide visibility, reject vertices, or draw anything. It neither reads depth nor transforms a vector.

**Encoding.** Write `0x00000006` to COMMAND at `0x08010344`. Function bits 5:0 are `000110`; **all bits 31:6 are ignored**, including SF, LM and every matrix/vector/control selector. Thus `0x00080406` and even `0xffffffc6` perform the same calculation. Ignored bits remain in the last COMMAND readback word. There is no shifted-result or positive-only-area option.

**INPUT.** The three raw words are read without changing their history order:

| Register | Address | Fields consumed |
|---|---|---|
| D[12] — SXY0 | `0x08010274` | Signed X0 in bits 15:0; signed Y0 in bits 31:16 |
| D[13] — SXY1 | `0x08010278` | Signed X1 low; signed Y1 high |
| D[14] — SXY2 | `0x0801027c` | Signed X2 low; signed Y2 high |

Each coordinate is a two's-complement signed16 value, −32768…32767. A whole-word read of SXY preserves its unsigned raw bits; that does not make the individual coordinates unsigned. For example, `0x80007fff` means `(32767,−32768)`, not `(32767,32768)`. NCLIP always consumes these three screen-history words, not V0–V2 or a vector selected by command bits. D[15] is a read alias of SXY2, not a fourth input.

Direct writes to D[12]–D[14] program their complete raw words and can supply the full signed16 coordinate range. They do not invoke projection's −1024…1023 coordinate clamp. A write to the separate push port D[15] changes history before NCLIP; NCLIP itself performs no push.

**Calculation and scale.** Decode the six signed coordinates, then evaluate the complete determinant:

```
area2 = X0×Y1 + X1×Y2 + X2×Y0
      − X0×Y2 − X1×Y0 − X2×Y1
```

Equivalently, `area2=(X1−X0)×(Y2−Y0)−(Y1−Y0)×(X2−X0)`. The result is **twice the triangle's oriented Euclidean area** in squared units of the supplied screen coordinates. If one coordinate unit means a pixel, area2 has squared-pixel units; it is not a normalized coefficient or a Q12 result. No SF shift, intermediate IR clamp or fixed-point rescaling occurs.

Use the full determinant to test signed32 overflow, then publish its low32 bits in MAC0. The calculation is not a succession of signed32-wrapped partial products. No arithmetic saturation repairs an overflowing determinant, and no separate area register retains an unwrapped result for software readback.

**MODIFIED and PRESERVED.**

| Modified register | Final effect |
|---|---|
| D[24] — MAC0, `0x080102a4` | Low32 bits of area2 |
| C[31] — FLAG, `0x08010340` | New overflow record; previous FLAG is cleared first |
| COMMAND; CYCLES | Last raw command word; nominal count `0x00000008` |

All other stored register state is preserved: MAC1–MAC3, IR0–IR3, all three screen words, all four depths, OTZ, RGB histories, vectors, coefficients, leading-sign ports and all PLUS state. The D[15] alias and IR-derived D[28]/D[29] reads therefore retain their previous values. There is no FIFO mutation or automatic hand-off to the GPU.

**FLAG.** Only the following bits can be set:

| Bit / mask | Condition on the **full** determinant |
|---|---|
| 16 / `0x00010000` — MAC0 positive overflow | area2 > 2147483647 |
| 15 / `0x00008000` — MAC0 negative overflow | area2 < −2147483648 |
| 31 / `0x80000000` — ERROR | Either overflow condition |

The signed32 endpoints themselves are in range. A negative in-range area and a zero area do not set any flag. SF/LM cannot change these conditions. Bits 30:17 and 14:0 remain clear.

Preserve a projection command's FLAG **before** issuing NCLIP: the new word does not retain projection overflow or saturation. Its area describes the stored, possibly quantized or saturated positions, not the original unclipped geometric triangle. A zero result can therefore reflect coincident projected points or lost precision as well as a geometrically collinear input.

**Winding is a software decision.** With no MAC0 overflow, interpret its raw word as signed32: positive, negative and zero represent the two possible orientations and degeneracy. Which orientation is front-facing depends on the cartridge's vertex order and coordinate convention. With X increasing right and Y down, a positive determinant appears clockwise on the displayed image; reversing Y reverses that visual convention. A nonzero area alone does not establish that a triangle is on-screen or in front of the viewpoint.

Raw word comparison with `>0` is not a signed test: an in-range negative result such as `0xffffec14` is still a positive unsigned readback number. For the chosen positive-winding convention, reject overflow first, then require a nonzero word with bit31 clear. For negative winding, require bit31 set after rejecting overflow. See Diagnosing numerical results for the surrounding flag/result lifetime.

**Ordinary, reversed and degenerate cases.** Program the words directly below; retain the same register order when issuing `0x00000006`.

| Case | SXY0 | SXY1 | SXY2 | Full area2 | MAC0 | FLAG |
|---|---|---|---|---|---|---|
| Ordinary | `(10,20)` / `0x0014000a` | `(100,30)` / `0x001e0064` | `(40,80)` / `0x00500028` | 5100 | `0x000013ec` | `0x00000000` |
| Reverse last two | Same | `(40,80)` / `0x00500028` | `(100,30)` / `0x001e0064` | −5100 | `0xffffec14` | `0x00000000` |
| Collinear | Same | `(100,30)` / `0x001e0064` | `(55,25)` / `0x00190037` | 0 | `0x00000000` | `0x00000000` |

For the ordinary triangle, `(100−10)×(80−20)−(30−20)×(40−10)=90×60−10×30=5100`; its unsigned Euclidean area is 2550 square screen units. Swapping two vertices changes the determinant's sign, not its magnitude. The degenerate third point lies halfway along the first edge and gives `90×5−10×45=0`. None of these commands changes a screen word or consumes a history entry.

**Representable direct-screen overflow.** The full signed16 input range admits a determinant larger than signed32, even though projected outputs normally use a smaller range:

| Case | SXY0 | SXY1 | SXY2 |
|---|---|---|---|
| Positive overflow | `0x80008000` | `0x80007fff` | `0x7fff8000` |
| Reverse last two | `0x80008000` | `0x7fff8000` | `0x80007fff` |

| Case | Full area2 | Raw MAC0 | Signed32 MAC0 | FLAG |
|---|---|---|---|---|
| Positive overflow | 4294836225 | `0xfffe0001` | −131071 | `0x80010000` |
| Reverse last two | −4294836225 | `0x0001ffff` | 131071 | `0x80008000` |

The two perpendicular edge differences are 65535, so the positive determinant is `65535×65535`. Its low word has the **opposite signed interpretation** from the full area; the reverse case fails in the other direction. Checking only bit31 would therefore classify both incorrectly. Use the overflow flags, not an assumption that every screen word came unchanged from projection.

If every coordinate really remains within projection's −1024…1023 output interval, the absolute determinant is at most `2047²=4190209`, safely inside signed32. That arithmetic bound does not undo a previous screen clamp or establish geometric visibility.

**Timing.** NCLIP writes 8 to normal CYCLES. MAC0 and FLAG are available when its COMMAND write completes.

#### 4.19.1.4 MVMVA — selected matrix-vector calculation

MVMVA multiplies one selected signed three-component vector by one selected matrix and applies the selected control-vector rule. It returns three MAC words and three clamped IR components without projection or a color push. Use it for matrix/vector arithmetic, a transform without screen coordinates, or the column calculations used in Composing transforms. Choose RTPS/RTPT when screen/depth histories are required, or a lighting/color command when its additional stages and RGB history output are required.

All selector encodings are representable. In particular, mx=3 is a defined synthetic matrix, and cv=2 is a two-pass operation whose first result is discarded but whose flags remain. Neither selection is an invalid-command condition or an ordinary substitute for the other selectors.

**Encoding.** Write COMMAND at `0x08010344`:

```
word = 0x00000012 | (SF << 19) | (mx << 17)
     | (v << 15) | (cv << 13) | (LM << 10)
```

| Bits | MVMVA interpretation |
|---|---|
| 5:0 | Function `010010` |
| 19 — SF | 0: no MAC-result shift; 1: arithmetic right shift by 12 |
| 18:17 — mx | Matrix selector, 0…3 |
| 16:15 — v | Vector selector, 0…3 |
| 14:13 — cv | Control-vector selector, 0…3 |
| 10 — LM | Final IR lower bound: −32768 when 0, zero when 1; upper bound always 32767 |
| 31:20, 12:11, 9:6 | Ignored for execution; retained in the last COMMAND word |

For example, `0x00080012` selects SF=1, matrix 0, V0, translation/control 0 and LM=0. `0x00086012` selects the same matrix/vector with no control-vector addition. `0x0009e012` selects matrix 0 and current IR with no control-vector addition. Set LM by adding `0x00000400`; change SF by adding or removing `0x00080000`. The selector fields do not change meaning with SF or LM.

**Inputs and representations.** D[n] is at `0x08010244 + 4n`; C[n] is at `0x080102c4 + 4n`. Access registers as aligned 32-bit words. `lo` and `hi` below mean signed two's-complement halfwords, with range −32768…32767. A signed complete word has range −2147483648…2147483647.

| mx | Matrix consumed |
|---|---|
| 0 | C[0]–C[4], conventionally the rotation matrix |
| 1 | C[8]–C[12], conventionally the light-direction matrix |
| 2 | C[16]–C[20], conventionally the color matrix |
| 3 | Synthetic matrix from RGBC.R, signed IR0 and parts of C[1]/C[2], specified below |

For mx=0,1,2, let b=8×mx. The exact row-major packing is:

```
          column 1       column 2       column 3
row 1     lo(C[b])       hi(C[b])       lo(C[b+1])
row 2     hi(C[b+1])     lo(C[b+2])     hi(C[b+2])
row 3     lo(C[b+3])     hi(C[b+3])     lo(C[b+4])
```

The upper half of the last matrix word is not a tenth coefficient. Writes to C[4], C[12] and C[20] sign-extend their low halfword, without changing the coefficient consumed.

| v | Signed `(x,y,z)` consumed |
|---|---|
| 0 | `(lo(D[0]),hi(D[0]),lo(D[1]))` — V0 |
| 1 | `(lo(D[2]),hi(D[2]),lo(D[3]))` — V1 |
| 2 | `(lo(D[4]),hi(D[4]),lo(D[5]))` — V2 |
| 3 | `(lo(D[9]),lo(D[10]),lo(D[11]))` — current IR1–IR3 |

All three vector components are sampled **before any row result is written**. Consequently v=3 uses one original IR vector, not the IR1 produced by row 1 followed by a mixture of old and new values. This remains true for the preliminary IR writes of cv=2. D[1]/D[3]/D[5] use only their signed low halves and read sign-extended.

| cv | Complete signed control words | Calculation rule |
|---|---|---|
| 0 | C[5], C[6], C[7] | Ordinary matrix plus control vector; conventionally translation |
| 1 | C[13], C[14], C[15] | Ordinary matrix plus control vector; conventionally background terms |
| 2 | C[21], C[22], C[23] | Special two-pass rule; these are the far-control words, **not** an ordinary retained addition |
| 3 | No control words consumed | Supply three zeros |

Matrix and control-vector selection are independent. For example, selecting matrix 1 does not select C[13]–C[15] automatically. Choosing cv=3 removes the control term without clearing any programmed register.

**Synthetic mx=3.** Let `r=D[6]&255`, `k=lo(D[8])`, `a=lo(C[1])` and `b=lo(C[2])`. The effective coefficients are:

```
       [ −16×r    +16×r       k ]
M  =   [     a        a       a ]
       [     b        b       b ]
```

Only RGBC's unsigned red byte is used; its green, blue and CODE bytes do not participate. IR0 is a **signed** halfword here, even if it previously held a nonnegative depth-cue weight. C[1]/C[2] are from the first stored matrix regardless of the selected control vector: their low halves ordinarily hold R13 and R22. Their upper halves are not consumed by this synthetic matrix. The three rows therefore do not depend on new IR1/IR2 results. IR0, RGBC and those control words are preserved throughout MVMVA. This is a coefficient generator, not a stored fourth 3×3 matrix or an implicit lighting calculation.

**A useful scale convention.** With matrix coefficients representing raw/4096, choose SF=1 to preserve the selected vector's numerical scale. If a vector uses `2^f` raw counts per unit, ordinary control-vector words must use the same scale for the addition to have that interpretation. The accumulated expression has `4096×2^f` counts per output unit, and its fixed-12 shift returns to `2^f`. Thus integer positions plus integer translation produce integer-position results, while a Q12 matrix column multiplied with cv=3 produces another Q12 column, subject to rounding and limits.

SF=0 does not remove the coefficient scale: it leaves the products unshifted before word wrap and IR clamping. LM=1 discards negative final IR components; it is generally unsuitable when the signs of a transform or composed matrix must be retained. Neither option changes the input field widths or chooses a physical coordinate system. The synthetic matrix's coefficients have the raw values shown above; interpreting all of them as Q12 is a software convention, not an additional rescaling performed by mx=3.

**Ordered arithmetic.** `ASR(x,k)` divides by `2^k`, rounding negative fractions toward minus infinity. `signed44(x)` retains the signed interpretation of low44 bits; `signed32(x)` does the same for low32. A clamp includes its endpoints.

For cv=0,1,3, process rows i=1,2,3 in order. With selected control component c_i and selected coefficients m_i1, m_i2, m_i3:

```
A_i = signed44(4096×c_i)
A_i = signed44(A_i + m_i1×x)
A_i = signed44(A_i + m_i2×y)
A_i = signed44(A_i + m_i3×z)
MAC_i = signed32(ASR(A_i,12×SF))
IR_i  = clamp(signed MAC_i,L,32767)
```

Here L=−32768 for LM=0 and L=0 for LM=1. Test every ordered signed44 addition for positive or negative overflow **before wrapping** and retain those conditions. An intermediate overflow is not cancelled by later terms. The public MAC word is a shifted/wrapped result, not a signed32 saturation. The IR clamp consumes that signed public word, not the original 44-bit accumulator.

MVMVA uses this ordinary IR rule for **all three lanes**. IR3 saturation tests the selected-SF, signed public MAC3 against the selected LM interval. It does **not** use RTPS/RTPT's exceptional fixed-12 IR3 flag test.

For **cv=2**, perform both passes for row 1 before proceeding to row 2, then both passes for row 3:

```
-- First pass of row i: selected far-control component and X only.
F_i = signed44(4096×C[20+i])
F_i = signed44(F_i + m_i1×x)
MAC_i = signed32(ASR(F_i,12×SF))
IR_i  = clamp(signed MAC_i,−32768,32767)

-- Second pass of the same row: start again at zero; use Y then Z only.
B_i = signed44(0 + m_i2×y)
B_i = signed44(B_i + m_i3×z)
MAC_i = signed32(ASR(B_i,12×SF))
IR_i  = clamp(signed MAC_i,L,32767)
```

The first IR clamp **forces the signed lower bound −32768**, even when the command selects LM=1. Both passes use the command's selected SF; each accumulation and clamp contributes flags. The second pass overwrites both MAC and IR for that lane. Therefore the final values contain **only the Y/Z products**: the far-control and X terms are not present in the final sum. There is no rounding-preserving recombination of the two passes. With mx=3 and/or v=3, use the synthetic coefficients and the original sampled vector in both passes exactly as specified above.

**Register footprint.** MVMVA always replaces the same arithmetic result words, regardless of selectors:

| MODIFIED | Effect |
|---|---|
| D[25]–D[27] — MAC1–MAC3 | Final shifted/wrapped row results; second-pass results for cv=2 |
| D[9]–D[11] — IR1–IR3 | Corresponding final clamped components, read as sign-extended halfwords |
| C[31] — FLAG | Cleared before arithmetic; accumulates every row and both passes when cv=2 |
| COMMAND; CYCLES | Last raw command word; nominal duration `0x00000008` |

Reads of D[28]/D[29] derive from the new IR components: clamp each `IR ASR 7` to 0…31 and pack into bits 4:0, 9:5 and 14:10. MVMVA does not write the D[28] packed-input latch or push a color merely because those read values change.

**PRESERVED:** all other stored register state, including V0–V2, RGBC, OTZ, IR0, MAC0, all screen/depth/color histories, the packed-input latch, leading-sign source/count, C[0]–C[30] and all PLUS state. D[15]'s screen alias therefore continues to read the unchanged SXY2. With v=3, the original IR input is overwritten by the output; preserve it separately if needed for another calculation. The selected matrix/control words remain programmed for later commands.

**FLAG conditions.** Arithmetic continues after a condition. Flags from cv=2's discarded first pass remain visible even if every final MAC/IR is in range.

| Bit(s) / mask | Set condition |
|---|---|
| 30, 29, 28 / `0x40000000`, `0x20000000`, `0x10000000` | Positive overflow beyond `2^43−1` during ordered row 1/2/3 accumulation, in either pass |

| Bit(s) / mask | Set condition |
|---|---|
| 27, 26, 25 / `0x08000000`, `0x04000000`, `0x02000000` | Negative overflow below `-2^43` during the corresponding accumulation |
| 24 / `0x01000000`; 23 / `0x00800000`; 22 / `0x00400000` | Row 1/2/3 signed public MAC lies outside its applicable IR interval; for cv=2 this includes the forced-LM=0 preliminary clamp |
| 31 / `0x80000000` — ERROR | Any of the six MAC overflow bits, IR1 saturation or IR2 saturation; IR3 saturation alone does not set ERROR |

There is no separate flag for signed32 word wrap. No screen, depth, divide, color, MAC0 or IR0 condition is generated; bits 21:0 remain zero. Exactly representable clamp endpoints do not set saturation. FLAG is an arithmetic record, not a validity check on the chosen matrix or coordinate convention.

**Worked ordinary matrix with fractional results.** Select matrix 0, V1 and control 0 with SF=1, LM=0:

| Input | Raw word(s) | Interpretation |
|---|---|---|
| C[0]–C[4] | `0x08001000`, `0xf800fc00`, `0x04001000`, `0xfc000400`, `0x00001000` | Rows `(4096,2048,-1024)`, `(-2048,4096,1024)`, `(1024,-1024,4096)` |
| C[5]–C[7] | `0x00000003`, `0xfffffffe`, `0x0000000a` | Control/translation `(3,-2,10)` |
| D[2], D[3] | `0xfff9000c`, `0x00000005` | V1 `(12,-7,5)` |
| COMMAND | `0x00088012` | SF=1, mx=0, v=1, cv=0, LM=0 |

The three ordered sums, with no signed44 overflow, are:

```
A1 = 3×4096 + 4096×12 + 2048×(−7) − 1024×5 = 41984
A2 = −2×4096 − 2048×12 + 4096×(−7) + 1024×5 = −56320
A3 = 10×4096 + 1024×12 − 1024×(−7) + 4096×5 = 80896
```

The unrounded mathematical results are `(10.25,-13.75,19.75)`. ASR12 produces `(10,-14,19)`, not `(10,-13,19)`. Both MAC1–MAC3 and IR1–IR3 read `0x0000000a`, `0xfffffff2`, `0x00000013`. FLAG is zero and CYCLES is 8. The same operands expose the option differences:

| Command | Signed MAC1–MAC3 | Final IR1–IR3 | FLAG |
|---|---|---|---|
| `0x00088012` — SF1, LM0 | `(10,-14,19)` | `(10,-14,19)` | `0x00000000` |
| `0x00088412` — SF1, LM1 | `(10,-14,19)` | `(10,0,19)` | `0x80800000` — ERROR, IR2 |
| `0x00008012` — SF0, LM0 | `(41984,-56320,80896)` | (32767, −32768, 32767) | `0x81c00000` — ERROR, IR1/2/3 |
| `0x00008412` — SF0, LM1 | `(41984,-56320,80896)` | `(32767,0,32767)` | `0x81c00000` |

**Worked synthetic matrix and current-IR input.** Program RGBC D[6]=`0x7faabb05`, IR0 D[8]=`0xffffff00` (−256), C[1]=`0x12340080` and C[2]=`0xabcdffc0`. Load IR1–IR3 with `(3,-4,5)` using `0x00000003`, `0xfffffffc`, `0x00000005`. Issue `0x0007e012`: SF=0, mx=3, v=3, cv=3, LM=0.

The synthetic rows are `(-80,80,-256)`, `(128,128,128)`, `(-64,-64,-64)`. The original vector is sampled once:

```
A1 = −80×3 + 80×(−4) − 256×5 = −1840
A2 = 128×3 + 128×(−4) + 128×5 = 512
A3 = −64×3 − 64×(−4) − 64×5 = −256
```

MAC and IR both become `(-1840,512,-256)`, raw `0xfffff8d0`, `0x00000200`, `0xffffff00`; FLAG is zero. Row 2 did **not** use the new IR1=−1840. IR0, RGBC and C[1]/C[2] retain their programmed words; D[28]/D[29] read `0x00000080` from final packed components `(0,4,0)`, without an RGB push.

Keeping the operands but setting SF=1 (`0x000fe012`) gives MAC/IR `(-1,0,-1)` and FLAG zero. Keeping SF=0 but setting LM=1 (`0x0007e412`) retains MAC `(-1840,512,-256)`, clamps IR to `(0,512,0)` and gives FLAG `0x81400000` (ERROR, IR1 and IR3 saturation). There is no implicit Q12 conversion or unsigned interpretation of the negative IR0 coefficient.

**Worked cv=2: discarded data, retained flag.** Put the ordinary example's five matrix words in C[8]–C[12] instead, selecting matrix 1. Load V0 `(12,7,5)` using D[0]=`0x0007000c`, D[1]=`0x00000005`. Program C[21]/C[22]/C[23]=`0x00000000`, `0x00000000`, `0x00009c40` — far-control words `(0,0,40000)`. Issue `0x000a4412`: SF=1, mx=1, v=0, cv=2, LM=1.

| Row | First-pass unshifted F | First MAC / forced signed IR | Final Y/Z sum B | Final MAC / selected-LM IR |
|---|---|---|---|---|
| 1 | `0×4096 + 4096×12 = 49152` | 12 / 12 | `2048×7 − 1024×5 = 9216` | 2 / 2 |
| 2 | `0×4096 − 2048×12 = −24576` | −6 / −6 | `4096×7 + 1024×5 = 33792` | 8 / 8 |
| 3 | `40000×4096 + 1024×12 = 163852288` | 40003 / 32767 | `−1024×7 + 4096×5 = 13312` | 3 / 3 |

The final MAC and IR read `(2,8,3)`, raw `0x00000002`, `0x00000008`, `0x00000003`. FLAG is **`0x00400000`**: first-pass IR3 saturated, but ERROR is clear because IR3 alone is not summarized. The preliminary −6 in row 2 is legal because that clamp forces LM=0, despite the command's LM=1. Nothing in the final three values reveals the discarded 40003. Selecting LM=0 instead (`0x000a4012`) gives the same final values and FLAG for these operands. This command did not add `(0,0,40000)` to a full matrix product.

**A signed44 boundary is not repaired by SF.** With matrix 0 as Q12 identity, V0 `(1,0,0)` and translation `(2147483647,0,0)`, row 1 starts at `2^43-4096`. Adding `4096×1` reaches `2^43`, sets positive row-1 overflow and wraps to `-2^43`. With SF=1, LM=0 (`0x00080012`), MAC1 reads `0x80000000`, IR1 reads `0xffff8000`, and FLAG is `0xc1000000` (ERROR, MAC1 positive overflow, IR1 saturation). With SF=0, LM=0 (`0x00000012`), low32 of that same wrapped accumulator is zero: MAC1 and IR1 both read zero, but FLAG is `0xc0000000`. Rows 2/3 remain zero in both cases. A small-looking public MAC word is not proof that ordered accumulation stayed in range.

**Timing.** Every MVMVA selector and option combination described above writes 8 to normal CYCLES, including the synthetic-matrix and two-pass cases. Results are available when its COMMAND write completes.

#### 4.19.1.5 OP — cross-product components from the matrix diagonal

OP combines the three diagonal coefficients of matrix 0 with the current signed IR vector. Its three expressions have the form of a cross product. Use it when software has already supplied two vectors in those locations; it does not obtain edges from triangle positions or calculate a unit normal. It does not use the rest of the matrix, translation, or the V0–V2 input slots.

**Encoding.** Write normal COMMAND with:

```
word = 0x0000000c | (SF << 19) | (LM << 10)
```

Bits 5:0 select function 001100. SF bit19 selects no shift or arithmetic shift12; LM bit10 selects IR lower bound −32768 or zero, with upper bound 32767. Matrix/vector/control-vector selectors in bits18:13 and all other option bits are ignored. The four zero-filled words are 0000000c, 0000040c, 0008000c and 0008040c for SF/LM=0/0, 0/1, 1/0 and 1/1 respectively.

**Inputs.** Define d=(d1,d2,d3) from signed low halves of C[0], C[2], C[4], the matrix0 diagonal. Define v=(v1,v2,v3) from signed low halves of D[9]–D[11]. Each component ranges −32768…32767. These are raw signed operands: OP does not assume that d is a Q12 rotation or that v is normalized. RGBC, IR0, other matrix fields, translation, old MAC and all screen/depth/color histories are not operands.

**Ordered calculation.** Snapshot v before any IR output write, then compute:

```
x1 = d2*v3 - d3*v2
x2 = d3*v1 - d1*v3
x3 = d1*v2 - d2*v1
MAC_i = signed32(x_i ASR (12*SF))
IR_i  = clamp(MAC_i, L, 32767)
L = -32768 when LM=0; 0 when LM=1
```

ASR rounds negative fractions towards minus infinity. MAC is the visible signed word; IR is its separately limited copy. Each row uses the original snapshot, not an IR component just written by a preceding row. OP supplies no signed44 MAC-overflow flags. In fact, the differences of these signed16 products fit signed32; large valid results can nevertheless exceed the much narrower IR range.

**Footprint and flags.** OP replaces MAC1–MAC3 (D[25]–D[27]), IR1–IR3 (D[9]–D[11]) and FLAG. It sets I1/I2/I3, masks 01000000/00800000/00400000, when a shifted signed MAC lies strictly outside its selected clamp interval. I1 or I2 also sets ERROR; I3 alone does not. It does not set MAC44, MAC0, divide, depth, screen, IR0 or color flags. Clamp endpoints are legal.

All other stored GTE words are preserved, including IR0, MAC0, the diagonal inputs, V0–V2, RGBC, and every FIFO. No RGB entry is pushed. D[28]/D[29] derived packed reads can change because IR changed, without a color-history update. COMMAND retains the issued word and CYCLES reports nominal6. Subsequent commands replace FLAG and may replace MAC/IR; retain a needed cross product before SQR or another arithmetic stage overwrites it.

**Worked signed cross product.** Put d=(12,−8,4) in the diagonal and v=(−4,20,8) in IR. Complete scratch words C[0]=0000000c, C[2]=0000fff8, C[4]=00000004 provide the diagonal; other coefficients are not consumed. With SF=0, LM=0:

```
x1 = (-8)*8 - 4*20 = -144
x2 = 4*(-4) - 12*8 = -112
x3 = 12*20 - (-8)*(-4) = 208
```

MAC and IR both become (−144,−112,208), FLAG=0. The result's dot product with each supplied operand is zero. Swapping the operands changes its sign; software chooses the edge ordering. SF=0 is appropriate here because both inputs are integer edge components, not a Q12 weight multiplied by a coordinate.

| Options | Signed MAC | IR | FLAG |
|---|---|---|---|
| SF0, LM0 | (−144,−112,208) | Same | 00000000 |
| SF0, LM1 | (−144,−112,208) | (0,0,208) | 81800000 |
| SF1, LM0 | (−1,−1,0) | Same | 00000000 |

The last row is a twelve-bit shift, not normalization. The LM1 row is no longer the signed perpendicular vector in IR even though MAC retains the signed terms.

For a range contrast, d=(−32768,−32768,−32768) and v=(−32768,32767,32767) produce MAC=(0,2147450880,−2147450880) with SF0/LM0. IR becomes (0,32767,−32768), FLAG=80c00000: ERROR and IR2/IR3 saturation, not a MAC-overflow condition. Read MAC if those wider components are needed; reading only the limited IR would change a later squared-length calculation.

**Timing.** OP writes 6 to normal CYCLES. Results are available when its COMMAND write completes.

#### 4.19.1.6 AVSZ3 — scale the newest three depths

Use AVSZ3 to form an unsigned depth quantity from SZ1, SZ2 and SZ3, for example after projecting a triangle. It calculates a **scaled sum**, not an automatic arithmetic mean. Choose ZSF3 to supply the intended normalization. AVSZ4 instead includes SZ0 and uses ZSF4; neither command sorts, rejects or submits a primitive.

**Encoding and issue.** Write `0x0000002d` to normal COMMAND at `0x08010344`. Bits 5:0 select function `101101`; bits 31:6 do not affect AVSZ3, including SF, LM and the matrix/vector/control-vector selectors. Ignored bits remain in the raw COMMAND readback. There is no selectable shift or signed output mode.

**Inputs and scale.** SZ1, SZ2 and SZ3 are the unsigned low halfwords of D[17]–D[19], each 0–65535. ZSF3 is the signed low halfword of C[29], −32768–32767. Its fixed denominator in the output calculation is 4096. Thus ZSF3=4096 preserves the depth sum before output limiting, ZSF3=1365 approximately divides it by three, and a negative coefficient produces a negative product unless the sum is zero. SZ0, D[16], is not an operand.

**Ordered calculation.** With exact integer intermediates:

```
S    = unsigned SZ1 + unsigned SZ2 + unsigned SZ3
P    = signed ZSF3 * S
MAC0 = low 32 bits of P
u    = P ASR 12
OTZ  = clamp(u,0,65535)
```

The sum has range 0–196605. The full product P is retained for the fixed twelve-bit arithmetic shift. MAC0's visible word wraps to its low 32 bits; that word is **not** the input to the OTZ shift. No division, reciprocal-table lookup, IR clamp, matrix calculation or FIFO push occurs.

**Flags.** AVSZ3 clears the previous normal FLAG before calculating. A full product above 2147483647 sets MAC0-positive overflow and ERROR; below −2147483648 sets MAC0-negative overflow and ERROR. Independently, u below zero or above 65535 sets SZ/OTZ saturation and ERROR, and OTZ receives the corresponding endpoint. Exactly representable endpoints do not set that saturation flag. Input depth zero is allowed; AVSZ3 does not set DIV merely because a depth is zero.

**Written and preserved state.** AVSZ3 replaces OTZ at D[7], MAC0 at D[24] and its retained full accumulator, normal FLAG at C[31], last COMMAND and normal CYCLES. All other stored data/control words are preserved, including IR0–IR3, MAC1–MAC3, all input vectors, RGBC, screen/depth/color histories, ZSF3 itself and all PLUS state. Reading the depth entries does not consume them. Retain a needed MAC0 or FLAG before another command replaces it.

**Timing.** AVSZ3 writes 5 (`0x00000005`) to normal CYCLES. OTZ, MAC0 and FLAG are available when its COMMAND write completes.

**Worked case: an approximate triangle mean.** Supply these word contents:

| Destination | Address | Raw word |
|---|---|---|
| D[16], older SZ0 | `0x08010284` | `0x0000ea60` |
| D[17], SZ1=1000 | `0x08010288` | `0x000003e8` |
| D[18], SZ2=2000 | `0x0801028c` | `0x000007d0` |
| D[19], SZ3=3000 | `0x08010290` | `0x00000bb8` |
| C[29], ZSF3=1365 | `0x08010338` | `0x00000555` |

Issue AVSZ3. S=6000 and P=8190000, so MAC0 reads `0x007cf830`. The fixed shift gives 1999, and OTZ at `0x08010260` reads `0x000007cf`; FLAG is zero. The exact arithmetic mean would be 2000, but `1365/4096` is slightly less than one third. The older depth 60000 has no effect, and all four depth words retain their original contents.

**Word-overflow boundary.** Replace SZ1–SZ3 with `(65535,4,0)` and ZSF3 with `0x00007fff` (32767), then issue again. S=65539 and the full product is 2147516413, although visible MAC0 reads `0x80007ffd`, a negative signed word. The full positive product shifts to 524295, so OTZ is **65535**, not zero. FLAG is `0x80050000`: MAC0-positive overflow plus SZ/OTZ saturation and ERROR. Using the wrapped MAC0 word for the output calculation would predict the wrong endpoint.

A small negative product also follows the signed shift exactly: with depths `(1,0,0)` and ZSF3=`0xffffffff` (−1), P=−1 and `P ASR 12=-1`, not zero. MAC0 reads `0xffffffff`, OTZ is zero, and FLAG is `0x80040000` from the output clamp alone. The coefficient's sign is arithmetic input, not an invalid setting.

#### 4.19.1.7 AVSZ4 — scale all four depth-history entries

AVSZ4 forms a scaled sum of SZ0, SZ1, SZ2 and SZ3, then writes its unsigned16 limited result to OTZ. Use it when four retained depths belong to the primitive being processed, for example after projecting a quadrilateral's four vertices. Use AVSZ3 for only the newest three entries. Neither command sorts or draws a primitive, and AVSZ4 does not divide by four automatically.

**Encoding.** Write `0x0000002e` to COMMAND at `0x08010344`; function bits 5:0 are `101110`. All bits 31:6 are ignored, including SF, LM and every selector. `0x0008042e` and `0xffffffee` therefore perform the same arithmetic as `0x0000002e`. Ignored bits remain in COMMAND readback. The output shift is always 12; SF cannot change it.

**INPUT.**

| Register | Address | Consumed representation |
|---|---|---|
| D[16]–D[19] — SZ0–SZ3 | `0x08010284`, `0x08010288`, `0x0801028c`, `0x08010290` | Four unsigned low16 depths, each 0…65535 |
| C[30] — ZSF4 | `0x0801033c` | Signed low16 coefficient, −32768…32767 |

Depth-register writes retain only low16 and reads zero-extend them: writing `0xffffffff` supplies depth 65535, not −1. ZSF4 instead sign-extends its low halfword on write/read, and the datapath consumes that signed coefficient. C[29]'s ZSF3 value, IR values, screen positions, matrices and projection H are not operands.

SZ0 is the **oldest** retained depth and SZ3 the newest. AVSZ4 does not determine which primitive produced them. After RTPT alone, only SZ1–SZ3 hold that command's three new depths; SZ0 still holds the previously newest depth. A four-vertex sequence can use RTPT for the first three vertices and RTPS for the fourth, leaving all four depths together. The three-entry screen history then no longer retains the first screen position, so preserve coordinates needed for drawing separately. AVSZ4 consumes no history entry and performs no push.

**Ordered calculation and scale.** Decode the four unsigned depths and signed ZSF4, then calculate:

```
sum     = SZ0 + SZ1 + SZ2 + SZ3
product = signed ZSF4 × sum
MAC0    = low32(product)
scaled  = ASR(product,12)
OTZ     = clamp(scaled,0,65535)
```

`ASR` divides by 4096 with negative fractions rounded toward minus infinity. Overflow tests and the OTZ calculation use the **full product**, not the signed interpretation of the public MAC0 word. Sum ranges from 0 to 262140; the signed16 coefficient can make the product exceed signed32 in either direction. MAC0 wraps to its low word rather than saturating. OTZ has a separate unsigned clamp.

Under an equal-scale depth convention, ZSF4 represents a numerical multiplier of raw/4096. Programming 1024 (`0x00000400`) produces the mean of four depths, rounded down to an integer. Programming 4096 instead produces their sum, subject to the OTZ clamp. Other values deliberately change the scale of the chosen depth key. No ordering-table address or geometric visibility decision is implied by the name OTZ.

**MODIFIED and PRESERVED.**

| Modified register | Final effect |
|---|---|
| D[24] — MAC0, `0x080102a4` | Low32 of the full signed product |
| D[7] — OTZ, `0x08010260` | Limited unsigned16 scaled result, read zero-extended |
| C[31] — FLAG | Previous flags cleared; new product/depth conditions recorded |
| COMMAND; CYCLES | Last raw command word; nominal duration `0x00000006` |

All other stored state is preserved: SZ0–SZ3 themselves, all screen/color histories, vectors, IR0–IR3, MAC1–MAC3, coefficients including ZSF4, leading-sign ports and all PLUS state. Screen/packed-color aliases retain their prior read values. Save a previous command's FLAG before AVSZ4 replaces it; save OTZ/MAC0 before a later command overwrites those results.

**FLAG.**

| Bit / mask | Set condition |
|---|---|
| 16 / `0x00010000` — MAC0 positive overflow | Full product > 2147483647 |
| 15 / `0x00008000` — MAC0 negative overflow | Full product < −2147483648 |
| 18 / `0x00040000` — depth saturation | Full product ASR12 < 0 or > 65535 |
| 31 / `0x80000000` — ERROR | Any of these three conditions |

Exactly in-range endpoints do not set a flag. A negative ZSF4 is not itself an error: with a zero sum its product is zero, OTZ is zero and FLAG is clear. With a positive sum, even a small negative product can set depth saturation after the arithmetic shift. No MAC1–MAC3, IR, color, screen or divide flags are generated.

**Four unequal depths.** Program SZ0–SZ3 with `0x0000012d`, `0x000002be`, `0x0000044f`, `0x000005e0` — depths 301, 702, 1103 and 1504 — and ZSF4=`0x00000400`. Issue `0x0000002e`.

The sum is 3610 and the full product is `1024×3610=3696640` (`0x00386800`). Dividing by 4096 gives 902.5, so OTZ becomes **902**, raw `0x00000386`, not 903. MAC0 reads `0x00386800`, FLAG reads zero and CYCLES reads 6. All four depths remain in their original slots. The oldest 301 participated; using only the newest three depths is a different command and calculation.

**Boundary and word-wrap cases.** Start each row independently with all four SZ entries set to `0x0000ffff`, hence sum=262140:

| Raw ZSF4 / signed value | Full product | Product ASR12 | MAC0 raw word | OTZ | FLAG |
|---|---|---|---|---|---|
| `0x00000400` / 1024 | 268431360 | 65535 | `0x0fff0000` | `0x0000ffff` | `0x00000000` |
| `0x00007fff` / 32767 | 8589541380 | 2097056 | `0xfffa0004` | `0x0000ffff` | `0x80050000` — ERROR, positive MAC0 overflow, depth saturation |
| `0xffff8000` / −32768 | −8589803520 | −2097120 | `0x00020000` | `0x00000000` | `0x80048000` — ERROR, negative MAC0 overflow, depth saturation |

The normalizing 1024 coefficient covers the full input-depth range without saturating OTZ. In the positive-overflow row, public MAC0 looks negative as signed32, but OTZ still saturates **upward**, using the full positive product. In the negative-overflow row, public MAC0 looks positive, but OTZ saturates **downward**. Reconstructing OTZ from the read MAC0 would be wrong in both cases.

For a small negative boundary, use depths `(0,0,0,1)` and ZSF4=`0xffffffff` (signed low16 −1). Product is −1, MAC0 reads `0xffffffff`, and ASR12 gives −1 rather than zero. OTZ clamps to zero and FLAG is `0x80040000`; no signed32 product overflow occurred.

**Timing.** AVSZ4 writes 6 to normal CYCLES. OTZ, MAC0 and FLAG are available when its COMMAND write completes.

### 4.19.2 Lighting and color commands

The lighting/color family selects stages by command, not by the MVMVA selector bits. The light stage takes three dot products of a signed vector with matrix 1, C[8]–C[12]. The color stage combines those three intermediate values through matrix 2, C[16]–C[20], adding signed control vector 1, C[13]–C[15]. These are the actual programmable operations: there is no separate command field for a number of lights. A triple command processes three input vectors, not three successive lights.

For SF=1, Q12 matrix coefficients preserve the scale of their input vector at each matrix stage. In a modulated-color calculation, an intermediate value of 4096 is a unit gain: multiplying `RGB byte×16×4096` and shifting by 12 produces `RGB byte×16`, which the color FIFO's final four-bit shift converts back to a byte. By contrast, a command without color modulation sends its final matrix MAC results directly to that four-bit color conversion; program that result scale accordingly. The equations and intermediate clamps are in Shared matrix and color stages.

#### 4.19.2.1 NCS — one supplied normal to an unmodulated color

NCS processes V0 through the light matrix and then the color matrix with ambient addition, and pushes the resulting color. Choose it when those matrix results already represent the desired output intensity. Unlike NCCS it does **not** multiply by RGBC's base RGB bytes; unlike NCDS it does not depth-cue. NCT is the corresponding three-input operation. No positions are projected or drawn.

**Encoding.** `word = 0x0000001e | (SF<<19) | (LM<<10)`. Function bits5:0 are 011110. SF bit19 chooses no shift or arithmetic shift12 at **both** matrix stages. LM bit10 chooses IR lower bound −32768 or zero, upper bound32767. Bits18:13 selectors and every remaining bit are ignored. Zero-filled option words are 0000001e, 0000041e, 0008001e, 0008041e for SF/LM=0/0,0/1,1/0,1/1.

**Inputs.** V0 has signed X/Y halves in D[0] and signed Z low half in D[1]. Light matrix L is nine signed16 coefficients packed row-major in C[8]–C[12]; color matrix K is similarly packed in C[16]–C[20]. Ambient A is three signed32 words C[13]–C[15]. Only RGBC.CODE, its unsigned high byte in D[6], is used. There is no translation, light position, automatic edge/normal calculation or normalization. Old IR/MAC, IR0, V1/V2, RGBC's RGB bytes and histories are not arithmetic operands.

**Ordered calculation.** Clear FLAG, then perform these stages:

1. Snapshot V0. For each row r of L, start accumulator at zero, add `L[r,0]*X`, then `L[r,1]*Y`, then `L[r,2]*Z`. Wrap signed44 after each addition, retaining every positive/negative overflow event. Shift by12 if SF1, otherwise not; expose the low signed32 result in MAC_(r+1), then clamp it into IR_(r+1) using LM. Complete the row accumulations from the snapshot before publishing the IR outputs.

2. Snapshot the three **limited** IR results. For each row r of K, start at `4096*A[r]` with signed44 interpretation, then add its X/Y/Z response products in that order, with the same ordered wraps, selected shift, signed32 visible MAC and selected-LM IR clamp. This overwrites the first stage's MAC/IR but retains its flags.
3. For each final signed MAC, arithmetic-shift4 and clamp to0…255. Shift `(RGB0,RGB1,RGB2)` to `(old RGB1,old RGB2,new color)`, packing red low, green next, blue next and **current RGBC.CODE** high.

ASR rounds negative fractions towards minus infinity. The final byte push consumes MAC, not limited IR. With Q12 L, K and a Q12 input normal, SF1 keeps Q12 intermediate responses; for unmodulated output, byte255 corresponds to final MAC4080, not4096. The unit does not choose these scales for software.

**Footprint and flags.** Replaces MAC1–MAC3, IR1–IR3, RGB0–RGB2 and FLAG; COMMAND/CYCLES retain the raw issue word and nominal14. All other stored words are preserved, including V0–V2, IR0, MAC0, matrices/ambient/RGBC, OTZ, screen and depth histories, packed-input latch, leading-sign words and PLUS state. D[28]/D[29] derived reads reflect the new IR, without another color push.

Either matrix stage can set signed44 positive/negative lane flags (30:28/27:25) and corresponding ERROR. An IR clamp sets bits24/23/22; I1/I2 contribute to ERROR, I3 alone does not. Byte limiting sets R/G/B bits21/20/19, not ERROR by itself. All conditions accumulate across both stages; a later positive final IR does not erase an earlier clamp. NCS does not set MAC0, depth, divide, screen or IR0 flags. Exact clamp endpoints are legal. Read needed flags/results before subsequent commands replace them or shift RGB.

**Worked unmodulated result.** Reuse the warm/green/back-facing-light setup in "Normals, light directions and color mixing": N=(2458,0,3277), ambient (256,512,128), and its nonidentity light/color matrices. Set CODE=`0x34`; the base RGB bytes may be any values. With SF1/LM1, light MAC=(3482,1228,−3277), limited IR=(3482,1228,0). The color rows then give:

| Row | Full sum | Final MAC/IR | Byte after ASR4 |
|---|---:|---:|---:|
| Red | 16568320 | 4045 | 252 |
| Green | 14258176 | 3481 | 217 |
| Blue | 6604800 | 1612 | 100 |

RGB2 is **3464d9fc**. FLAG=00400000 remembers the negative third light response, although final IR3=1612; ERROR is clear. NCCS with the same setup and its stated base color instead produces 344b519e because it includes material modulation. NCS has intentionally omitted that stage.

Selecting LM0 for NCS retains the third light response −3277. The color-stage blue MAC becomes −1665, so RGB2=3400d9fc and FLAG=00080000, a blue byte clamp. Thus LM changes the inputs to the color mix, not just an output sign flag.

For an independent scale boundary, set V0 and both matrices to zero and ambient to (4080,0,0), SF1/LM1. The ambient-only output is red255 with no flags. Changing only ambient red to4096 still produces255 but sets R saturation00200000: 4096 ASR4 is256. A unit gain appropriate to a modulated command is not an unsaturated maximum unmodulated color.

**Timing.** NCS writes 14 to normal CYCLES. The color and arithmetic results are available when its COMMAND write completes.

#### 4.19.2.2 NCT — light and color three normals without modulation

NCT runs the light matrix and color matrix for V0, V1 and V2, in that order, and pushes three colors. Choose it when all three normals share the matrices and ambient contribution and the color-matrix result is already in output byte×16 units. RGBC's RGB bytes do not modulate that result. Use NCCT for a shared base-color multiplication, NCDT for that multiplication plus cue, or separate NCS commands when software must change the setup between normals. Three normals do not mean three additional lights per normal.

**Encoding.** Write the word to normal COMMAND, `0x08010344`:

```
word = 0x00000020 | (SF << 19) | (LM << 10)
```

Function bits 5:0 are `100000`. SF=0 selects no main-result shift; SF=1 selects ASR 12 **at each matrix stage**. LM=0 selects IR limits −32768–32767; LM=1 selects 0–32767. Only those fields affect NCT; selectors and all other command bits are ignored for execution and retained in raw COMMAND readback. The four zero-filled words are `0x00000020`, `0x00000420`, `0x00080020` and `0x00080420` for SF/LM combinations 0/0, 0/1, 1/0 and 1/1.

**Inputs.** V0, V1 and V2 occupy D[0]/D[1], D[2]/D[3] and D[4]/D[5]. Each XY word supplies signed low/high halfwords; Z is the signed low half of the following word. The light matrix is the nine signed coefficients in C[8]–C[12]; the color matrix is

C[16]–C[20]. Signed full words C[13]–C[15] are the color-stage offsets. D[6] supplies CODE only. IR0, far color and the projection matrix are not operands. The normal direction, length and scaling are supplied conventions; no normal is generated or normalized.

With Q12 matrices and SF=1, each matrix stage preserves its input scale. For NCT's direct color output, arrange the final MAC values in byte×16 units; a final value 4096 converts to 256 and saturates, rather than representing a unit modulation gain. See Normals, light directions and color mixing.

**Ordered calculation.** Repeat the following complete sequence for V0, V1, then V2. Each matrix reads a snapshot of its three inputs.

1. For each light row r, start at zero and add `L[r,0]×nx`, `L[r,1]×ny`, `L[r,2]×nz` in that order with signed-44 wrapping and cumulative overflow tests. Write visible `MAC[r+1]=signed32(ASR(sum,12×SF))`, then clamp it to the selected IR range. All three rows are calculated before their IR outputs replace the input state.
2. Snapshot those limited light responses. Each color row starts at `4096×offset[r]`, then adds the three color-coefficient products in order, using the same signed-44 accumulation, selected shift, signed32 visible word and selected IR clamp. This overwrites the light-stage MAC/IR values.
3. Convert each **signed visible MAC**, not its limited IR copy, by ASR 4 and clamp to 0–255. Pack R, G and B into bytes 0, 1 and 2, and current RGBC CODE into byte 3. Push RGB0 ← old RGB1, RGB1 ← old RGB2, RGB2 ← new color.

The exact signed-44 and visible-word definitions are in MAC, shift, and IR stages. No base-color application or depth-cue calculation intervenes between the color matrix and the push.

**State and flags.** NCT clears normal FLAG once at the start and accumulates all three vectors' matrix-overflow, IR-limit and RGB-limit conditions. It does not clear flags between vectors. MAC positive/negative overflow, IR1/IR2 saturation and their normal summary fields can set ERROR; IR3 or RGB saturation alone does not. Flags from an overwritten light-stage result remain visible.

At completion, RGB0–RGB2 contain V0/V1/V2's new colors in order. MAC1–MAC3 and IR1–IR3 retain only V2's final color-matrix values. Normal FLAG, COMMAND and CYCLES are replaced. All other stored state is preserved: the vector slots, RGBC, matrices/offsets, IR0, MAC0, OTZ, screen/depth histories and PLUS state. The derived packed-color read ports D[28]/D[29] reflect the changed IR values; NCT does not write a separate packed-color result latch.

**Timing.** NCT writes 30 (`0x0000001e`) to normal CYCLES. All three colors are available when its COMMAND write completes.

**Worked case: three directions, not three materials.** Keep the preceding non-identity light/color matrices and offsets, but supply these three normals:

| Slot | Signed normal | XY word | Z word |
|---|---|---|---|
| V0 | `(2458,0,3277)` | `0x0000099a` | `0x00000ccd` |
| V1 | `(−2458,0,−3277)` | `0x0000f666` | `0xfffff333` |
| V2 | `(4096,0,0)` | `0x00001000` | `0x00000000` |

Retain RGBC=`0x34c060a0` and issue `0x00080420` (SF=1, LM=1). The RGB bytes `(160,96,192)` in that word are deliberately not applied. The matrix stages and direct color conversion give:

| Input | Light MAC after shift | Limited light IR | Color MAC / IR | Output RGB |
|---|---|---|---|---|
| V0 | `(3482,1228,−3277)` | `(3482,1228,0)` | `(4045,3481,1612)` | `(252,217,100)` |
| V1 | `(−3482, −1229, 3277)` | `(0,0,3277)` | `(256,512,3405)` | `(16,32,212)` |
| V2 | `(3072,−2048,0)` | `(3072,0,0)` | `(3328,2048,896)` | `(208,128,56)` |

RGB0–RGB2 therefore read **`0x3464d9fc`, `0x34d42010`, `0x343880d0`**. Any three initial FIFO entries have been displaced. The final MAC/IR values are `(3328,2048,896)`, not V0's or V1's results. FLAG is `0x81c00000`: IR1, IR2 and IR3 saturation occurred across the light stages, and IR1/IR2 make ERROR set. No output byte saturated. V1's negative row-1 value is −1229, not −1228, because the signed shift rounds toward negative infinity.

Changing only LM to zero, raw command `0x00080020`, preserves the negative light responses instead of removing them. Final V2 MAC/IR becomes `(2816,0,−128)` and RGB2 becomes `0x340000b0`. Across all three colors, FLAG is `0x00380000`, reporting RGB clamps with ERROR clear. This is a different signed-lighting calculation, not an alternative way to obtain the LM=1 colors.

#### 4.19.2.3 NCCS — light one vector and modulate the base color

NCCS converts supplied V0 into three light responses, mixes those responses and ambient into three channel gains, then applies those gains to the RGB bytes of RGBC. It pushes one resulting color. Choose it when the color-matrix result is a **gain on a material color**. NCS omits the base-color multiplication; CC starts from light responses already in IR rather than from V0; NCDS adds depth cue; NCCT processes three supplied vectors. NCCS performs no projection, normalization, visibility test or drawing.

**Encoding.** Write `0x0000001b | (SF<<19) | (LM<<10)` to COMMAND at `0x08010344`. Function bits 5:0 are `011011`. SF selects no main-result shift when zero and ASR12 when one. LM selects final IR lower bound −32768 when zero or zero when one; the upper bound is 32767. Both options apply at **each** of the light, color-matrix and modulation stages, not once at the end.

The four option words are `0x0000001b`, `0x0000041b`, `0x0008001b`, `0x0008041b` for SF/LM `(0,0)`, `(0,1)`, `(1,0)`, `(1,1)`. Bits 18:13 containing matrix/vector/control selectors are ignored: they cannot redirect this command from V0 or the two prescribed matrices. Bits 31:20, 12:11 and 9:6 are also ignored for execution and retained in COMMAND readback.

**INPUT.** D[n] is at `0x08010244+4n`; C[n] is at `0x080102c4+4n`. Use aligned whole-word register accesses.

| Input | Raw representation |
|---|---|
| D[0], D[1] — V0 | Signed X low16 / Y high16 in D[0]; signed Z low16 in D[1] |
| C[8]–C[12] — L | Nine signed16 light-matrix coefficients |
| C[13]–C[15] — B | Three signed complete-word background/ambient components |
| C[16]–C[20] — K | Nine signed16 color-matrix coefficients |
| D[6] — RGBC | Unsigned R/G/B bytes in bits 7:0/15:8/23:16; CODE in 31:24 |
| Old D[21], D[22] | Existing RGB1/RGB2 words retained by the one color-history push |

For either matrix, with base b=8 for L or b=16 for K, rows are `(lo(C[b]), hi(C[b]), lo(C[b+1]))`, `(hi(C[b+1]), lo(C[b+2]), hi(C[b+2]))`, `(lo(C[b+3]), hi(C[b+3]), lo(C[b+4]))`, where every half is signed two's complement. C[b+4]'s upper half is not another coefficient. Signed16 range is −32768…32767; ambient words are signed32. D[1], C[12] and C[20] sign-extend their low half on write/read.

No value is taken from old IR1–IR3, IR0, far color, the geometric rotation/translation, H or screen offsets. V1/V2 are not processed. CODE is copied as a byte; it is not interpreted or applied as an extra color channel.

**Useful SF=1 lighting convention.** Supply normal components and L/K coefficients at 4096 counts per unit. L's outputs then remain Q12 directional responses. The color stage combines those responses with ambient B in Q12 gain units; gain4096 means unity. Modulation converts byte×gain to byte×16, and the color-output shift restores a byte. The unit neither normalizes V0 nor checks that the chosen rows represent normalized directions. Raw signed values outside that software convention still follow the arithmetic below.

**Ordered calculation.** `ASR(x,k)` divides by `2^k`, rounding negative fractions toward minus infinity. Each matrix row begins at its stated initial value, adds X/Y/Z terms in that order, and wraps signed44 after every addition. A signed44 overflow condition is retained even if later terms cancel it. After a stage's full row result A_i, write `MAC_i=signed32(ASR(A_i,12×SF))`, then clamp that signed word to `[Lmin,32767]` for IR_i, where Lmin is −32768 for LM=0 or zero for LM=1.

1. **Light responses.** Sample signed `(nx,ny,nz)` from V0. For each row i=1…3:

   ```
   A_i = L_i1×nx + L_i2×ny + L_i3×nz
   response_i = clamp(signed32(ASR(wrapped44 A_i,12×SF)),Lmin,32767)
   ```

   Start each accumulation at zero. Finish the three rows from the original vector before publishing their IR results. These **clamped responses**, not their unclamped MAC words, become the next stage's inputs.

2. **Channel gains plus ambient.** Sample those three responses. For each color row i:

   ```
   A_i = 4096×B_i + K_i1×response_1
                  + K_i2×response_2 + K_i3×response_3
   gain_i = clamp(signed32(ASR(wrapped44 A_i,12×SF)),Lmin,32767)
   ```

   Wrap the initial control term and each ordered addition signed44. Publish these MAC/IR results after evaluating the three rows from the same response vector.

3. **Material modulation.** For R/G/B independently, use its unsigned RGBC byte and the corresponding clamped gain:

```
product_i = signed44(16×base_byte_i×gain_i)
MAC_i = signed32(ASR(product_i,12×SF))
IR_i  = clamp(signed MAC_i,Lmin,32767)
```

   These are the command's final MAC/IR values. They replace the earlier responses and gains. With SF=1 and gain4096, MAC becomes `16×base_byte`; SF=0 does not preserve this unit-gain interpretation.

4. **Color conversion and push.** Calculate each byte as `clamp(ASR(signed MAC_i,4),0,255)`. This uses the **public MAC word, not clamped IR**. Shift RGB0 ← old RGB1 and RGB1 ← old RGB2, then write RGB2=`R | (G<<8) | (B<<16) | (RGBC.CODE<<24)`. The push occurs even after saturation.

**Footprint.** Modified words are MAC1–MAC3 (D[25]–D[27]), IR1–IR3 (D[9]–D[11]), RGB0–RGB2 (D[20]–D[22]), FLAG C[31], last COMMAND and CYCLES=`0x00000011`. The final newest color is RGB2. One old color is displaced; the old FIFO colors are not lighting operands.

Preserved stored state includes V0–V2, RGBC, OTZ, IR0, MAC0, every screen/depth entry, all C[0]–C[30], leading-sign ports, the packed-input latch and all PLUS state. D[15] still aliases unchanged SXY2. D[28]/D[29] derive from final IR, clamping each `IR ASR7` to 0…31 and packing those components into bits 4:0/9:5/14:10; a changed derived read is not another color push.

**FLAG.** Clear previous FLAG before the first matrix and accumulate conditions across all four stages. Signed44 positive/negative row overflow sets the corresponding MAC1/2/3 bits 30/29/28 or 27/26/25. An actual selected-LM IR clamp at **any** stage sets IR1/2/3 bits24/23/22. IR3 uses the ordinary selected-SF/LM clamp rule, not projection's special test. Final byte clamps set R/G/B bits21/20/19 (`0x00200000`, `0x00100000`, `0x00080000`). ERROR bit31 summarizes the MAC-overflow and IR1/IR2 bits; IR3 or color saturation alone does not set it. No depth, divide, screen, MAC0 or IR0 flags are generated. Legal endpoints do not set saturation.

**Non-identity worked lighting and material.** Program the following complete inputs. The normal is approximately `(0.6,0,0.8)` in Q12; one chosen light faces away from it.

| Register(s) | Raw words |
|---|---|
| C[8]–C[12] — L | `0x00000c00`, `0xf8000800`, `0x0c000000`, `0x00000000`, `0xfffff000` |
| C[13]–C[15] — ambient | `0x00000100`, `0x00000200`, `0x00000080` |
| C[16]–C[20] — K | `0x04001000`, `0x08000000`, `0x00001000`, `0x08000400`, `0x00001000` |
| D[0], D[1] — V0 | `0x0000099a`, `0x00000ccd` — `(2458,0,3277)` |
| D[6] — material and CODE | `0x34c060a0` — RGB `(160,96,192)`, CODE `0x34` |
| D[20]–D[22] — old FIFO | `0x11112222`, `0x33334444`, `0x55556666` |
| COMMAND | `0x0008041b` — SF=1, LM=1 |

L rows are `(3072,0,2048)`, `(−2048,0,3072)`, `(0,0,−4096)`. Their unshifted sums are (`14262272`, `5032960`, `−13422592`), yielding MAC `(3482,1228,−3277)` and IR `(3482,1228,0)`. The last clamp sets IR3 saturation.

K rows are `(4096,1024,0)`, `(2048,4096,0)`, `(1024,2048,4096)`. With ambient `(256,512,128)`, their full sums become `(16568320,14258176,6604800)` and gains become `(4045,3481,1612)`. The blue sum represents 1612.5 and truncates to 1612.

Material products are (`10355200`, `5346816`, `4952064`). Their fixed12 shifts give final MAC/IR `(2528,1305,1209)`, raw `0x000009e0`, `0x00000519`, `0x000004b9`. Fixed4 color conversion produces `(158,81,75)`.

Final FIFO is `0x33334444`, `0x55556666`, **`0x344b519e`**. FLAG is `0x00400000`, retaining the earlier negative light-response clamp even though final IR3 is positive; ERROR is clear. CYCLES reads 17. With the same setup but a zero normal, ambient survives: final RGB2 is `0x34060c0a` — RGB `(10,12,6)` — and FLAG is zero.

**Options change intermediate values, not just the final sign.** Restore the original normal/material before each row:

| SF/LM | Final signed MAC | Final IR | RGB2 | FLAG |
|---|---|---|---|---|
| 1/1 | `(2528,1305,1209)` | Same | `0x344b519e` | `0x00400000` |
| 1/0 | `(2528,1305,−1249)` | Same | `0x3400519e` | `0x00080000` — blue byte saturation only |

| SF/LM | Final signed MAC | Final IR | RGB2 | FLAG |
|---|---|---|---|---|
| 0/1 | (83883520, 50330112, 100660224) | (32767, 32767, 32767) | 0x34ffffff | 0x81f80000 |
| 0/0 | (83883520, 50330112, −100663296) | (32767, 32767, −32768) | 0x3400ffff | 0x81f80000 |

With LM=0 the back-facing negative response survives into color mixing; it is not replaced by the LM=1 blue gain. With SF=0, every matrix and modulation stage omits its shift, saturating IRs and final bytes in this setup. Neither setting is a substitute for choosing the input/output scales correctly.

**Timing.** NCCS writes 17 to normal CYCLES. Results are available when its COMMAND write completes.

#### 4.19.2.4 NCCT — three normals with one shared base color

NCCT processes V0, V1 and V2 through lighting, color mixing/ambient and RGBC modulation, pushing one color per input. Use it for three supplied directions that share matrices, ambient and material color. NCCS processes one input; NCT omits base-color modulation; NCDT additionally depth-cues. NCCT does not project positions or calculate the normals from a triangle.

**Encoding.** `word=0x0000003f | (SF<<19) | (LM<<10)`. Function bits5:0 are 111111. SF bit19 selects no shift or ASR12 at every matrix/application stage; LM bit10 selects signed or nonnegative IR clamp, with upper bound32767. All selector fields18:13 and other unused bits are ignored. The four zero-filled option words are 0000003f,0000043f,0008003f,0008043f for SF/LM=0/0,0/1,1/0,1/1. The one SF/LM choice applies to all three vectors.

**Inputs.** Each Vj uses signed X/Y halves of D[2j] and signed Z low half of D[2j+1]. Light matrix L is nine signed16 coefficients in C[8]–C[12]; color matrix K is nine signed16 coefficients in C[16]–C[20], both packed row-major. Ambient A is signed32 C[13]–C[15]. D[6] supplies unsigned RGB bytes and CODE. These controls are shared for the complete batch. IR0, far vector, projection controls, translation and old MAC/IR/color entries are not arithmetic operands.

**Ordered calculation.** Clear FLAG once. For j=0,1,2, complete the following sequence before starting the next input:

1. Snapshot Vj. For every L row, accumulate its X, then Y, then Z products, starting at zero. Interpret/wrap each accumulator as signed44 after every addition and retain every overflow condition. Publish `MAC=signed32(A ASR(12*SF))`; limit that signed word to IR range `[LM?0:-32768,32767]`.
2. Snapshot those limited IR responses. For every K row start with signed44 `4096*ambient`, then add the three response products in order, using the same wraps/shift/word publication/selected-LM clamp. This overwrites light MAC/IR, not the already accumulated flags.
3. For each channel form `16*RGBC_byte*current signed IR`. Wrap signed44, record its signed44 overflow, apply the selected SF shift and signed32 MAC publication, then clamp IR with LM. These are the final MAC/IR values for this input.
4. Arithmetic-shift each signed final MAC by4, clamp to byte0…255, and push packed RGB plus **current RGBC.CODE** into the RGB history.

ASR rounds negative fractions towards minus infinity. With the Q12 gain convention and SF1, gain4096 preserves a base color byte because modulation produces byte×16 before the push. SF0 changes all stages; it does not simply relabel the same output. No per-input RGBC or matrix replacement occurs inside the batch.

**Footprint and flags.** Final RGB0/1/2 correspond to V0/V1/V2; all three old entries are displaced. Final MAC1–MAC3 and IR1–IR3 describe V2 only. FLAG combines conditions from every stage of every input, with no vertex identifier. COMMAND/CYCLES retain the raw issue and nominal39. All other stored words are preserved, including V0–V2, RGBC and controls, IR0/MAC0, OTZ and all screen/depth histories, packed-input and leading-sign state, and PLUS. Derived D[28]/D[29] reads reflect V2's final IR; they are not another retained color.

Signed44 overflow in either matrix or application sets lane flags30:28/27:25 and ERROR. Any IR clamp sets I1/I2/I3 (24/23/22); I1/I2 also summarize into ERROR, I3 alone does not. Byte limiting sets R/G/B bits21/20/19 without ERROR by itself. Endpoints are legal; flags survive later in-range calculations. No MAC0, depth, divide, screen or IR0 stage runs. Preserve a needed batch FLAG before another command, and colors before later pushes displace them.

**Worked three-result lifetime.** Program L rows `(3072,0,2048)`, `(−2048,0,3072)`, `(0,0,−4096)` and K rows `(4096,1024,0)`, `(2048,4096,0)`, `(1024,2048,4096)`. Set ambient `(256,512,128)` and RGBC=`0x34c060a0`. Supply

N0=(2458,0,3277), N1=(4096,0,0), N2=(0,0,0); their raw vector words are 0000099a/00000ccd, 00001000/00000000, and 00000000/00000000. With SF1/LM1:

| Input | Light IR | Color-matrix gains | Final modulated MAC/IR | Pushed word |
|---|---|---|---|---|
| V0 | (3482,1228,0) | (4045,3481,1612) | (2528,1305,1209) | 344b519e |
| V1 | (3072,0,0) | (3328,2048,896) | (2080,768,672) | 342a3082 |
| V2 | (0,0,0) | (256,512,128) | (160,192,96) | 34060c0a |

Starting with any RGB history `(A,B,C)`, the pushes leave `(B,C,color0)`, then `(C,color0,color1)`, then the three listed colors. The zero final normal still produces ambient-modulated RGB(10,12,6). Final MAC/IR therefore look entirely in range, but FLAG is **80c00000**: V0's third light response was negative and clamped, and V1's second response was negative and clamped. The latter sets ERROR; the former alone would not. Final V2 registers do not reveal those earlier conditions.

With the same inputs but LM0, RGB0/1/2 become 3400519e,3400006e,34060c0a. Negative light responses now subtract in K; blue byte limits set FLAG=00080000. Final V2 MAC/IR are unchanged, yet the first two colors differ. This option changes the per-vertex calculation, not just how a batch flag is reported.

**Timing.** NCCT writes 39 to normal CYCLES. Its three colors are available together when the single COMMAND write completes.

#### 4.19.2.5 NCDS — light, modulate and depth-cue one normal

NCDS takes one supplied normal through the light matrix and color matrix, applies RGBC's base-color bytes, depth-cues the products using current IR0, and pushes one color. Choose it for the complete single-normal cued path. NCCS omits cue; CDP starts with directional responses already in IR; DCPL starts with final channel gains. NCDT repeats the NCDS path for three normals with one shared RGBC, IR0 and far-color vector.

**Encoding.** Write to normal COMMAND, `0x08010344`:

```
word = 0x00000013 | (SF << 19) | (LM << 10)
```

Bits 5:0 select `010011`. SF=0 selects no main-result shift; SF=1 selects ASR 12 at **both matrices and both cue passes**. LM=0 permits final/stage IR −32768–32767; LM=1 permits 0–32767. The temporary cue-difference clamp always permits −32768–32767, independently of LM. All other fields, including selectors, are ignored for execution and retained in COMMAND readback. The zero-filled option words are `0x00000013`, `0x00000413`, `0x00080013` and `0x00080413` for SF/LM 0/0, 0/1, 1/0 and 1/1. SF=0 is not the same blend with extra precision.

**Inputs and representations.**

| Input | Representation and role |
|---|---|
| V0, D[0]/D[1] | Signed X/Y low/high halves, signed Z low half; supplied normal/vector |
| C[8]–C[12] | Nine signed16 light-matrix coefficients |
| C[16]–C[20] | Nine signed16 color-matrix coefficients |
| C[13]–C[15] | Signed32 color-stage offsets, added before the selected shift |
| RGBC, D[6] | Unsigned R/G/B bytes and CODE high byte |
| IR0, D[8] | Signed low-halfword cue coefficient; not automatically clamped on input |
| C[21]–C[23] | Signed32 far red/green/blue components |

With SF=1 and Q12 matrices/normal, the matrices can produce Q12 channel gains: 4096 is unit modulation gain. For ordinary color output the far vector is byte×16, so target byte b is encoded as signed word `16×b`. These are chosen scales; other representable signed inputs follow the same arithmetic and limits. No position, depth FIFO entry, DQA or DQB is read. Software must supply the coefficient belonging to this color; NCDS does not derive one from a depth.

**Ordered calculation.** Let `s=12×SF`, and let the selected IR clamp be `I(q)=clamp(q,LM?0:−32768,32767)`.

1. Snapshot V0. Each light row starts at zero and accumulates its X, Y and Z products in order with signed-44 wrap and cumulative overflow tests. Write `MAC_i=signed32(ASR(sum_i,s))`, then `IR_i=I(MAC_i)`.

2. Snapshot those three limited light responses. Each color row starts at `4096×offset_i`, then accumulates its three matrix products in order under the same wrap/shift/IR rules. The resulting IR values are the channel gains.
3. Form all three unshifted products `P_i=16×RGBC.byte_i×signed IR_i`. **Do not first shift, clamp or push a separate base-color result.**
4. Compute all three temporary differences:

   ```
   D_i = clamp(signed32(ASR(signed44(4096*far_i-P_i),s)), -32768,32767)
   ```

   Each difference updates MAC and flags; its clamp does not replace the IR register words. Signed-44 overflow is tested on the raw difference before wrapping. This pass permits a negative difference even with LM=1.
5. For each channel, form `P_i+signed IR0×D_i`, test/wrap that full sum to signed44, shift by s and expose its signed32 MAC word. Write selected-clamped IR from that MAC result. The original P terms, not shifted approximations of them, enter this final sum.
6. Take each final signed MAC ASR 4, clamp to 0–255, and push one RGB color with current RGBC CODE. The push shifts the previous RGB1/RGB2 into RGB0/RGB1.

The common wrap definitions are in MAC, shift, and IR stages. The two cue passes are actual limited integer operations, not an ideal RGB interpolation substituted after lighting.

**Written state and flags.** Final cued MAC1–MAC3 and IR1–IR3 replace the matrix intermediates. RGB0–RGB2 shift once. FLAG is cleared at command start and accumulates matrix overflow, matrix IR limits, temporary-difference limits, final-result limits and output-byte limits. Even a temporary value later overwritten can set a flag. Normal ERROR summarizes its specified fields: IR3 and RGB saturation alone do not set it; IR1/IR2 saturation does.

COMMAND and CYCLES are replaced. All other stored state is preserved, including IR0, MAC0, OTZ, all vector slots, RGBC, matrices/offsets/far color, screen/depth histories and PLUS. D[28]/D[29]'s derived packed-color reads change with final IR, but no separate packed-color result latch is written. Read needed final values/FLAG before another normal command replaces them.

**Timing.** NCDS writes 19 (`0x00000013`) to normal CYCLES. Results are available when its COMMAND write completes.

**Worked case: cued material under non-identity lighting.** Use the matrices, offsets and raw control packing from Normals, light directions and color mixing. Supply V0 XY=`0x0000099a`, Z=`0x00000ccd`, RGBC=`0x34c060a0`, IR0=`0x00000800` and far words `(0x00000500,0x00000600,0x00000a00)`. Thus the supplied normal is `(2458,0,3277)`, base RGB is `(160,96,192)`, and far RGB is `(80,96,160)`. Issue `0x00080413` (SF=1, LM=1).

The limited light responses are `(3482,1228,0)` and the channel gains are `(4045,3481,1612)`. The cue stages proceed as follows:

| Channel | Unshifted P | Shifted signed difference | Limited D | Final MAC / IR | Byte |
|---|---|---|---|---|---|
| Red | 10355200 | −1249 | −1249 | 1903 | 118 |
| Green | 5346816 | 230 | 230 | 1420 | 88 |
| Blue | 4952064 | 1351 | 1351 | 1884 | 117 |

RGB2 reads **`0x34755876`**; final MAC/IR words are `0x0000076f`, `0x0000058c`, `0x0000075c`. IR0 remains `0x00000800`. FLAG is `0x00400000`, the earlier negative third light response; none of the cue or byte values requires limiting. Copying NCCS's already reduced RGB bytes into RGBC and then using DPCS would instead produce red 119, not 118, because it changes the cue input's precision. See Choosing a color calculation.

**Zero-coefficient boundary.** Keep those operands, but set IR0=0 and far red C[21]=`0x00010000` (65536); leave the other far components unchanged. The red temporary difference shifts to 63007 and clamps to 32767. The final sum still uses zero times that difference, so RGB2 is the uncued result `0x344b519e`, while FLAG is **`0x81400000`**: IR1 saturation and ERROR, plus the light-stage IR3 flag. IR0=0 did not bypass the difference pass.

With IR0=4096 and the original far vector, this example produces `0x34a05f4f`, RGB `(79,95,160)`, because the signed first-pass shift already discarded fractions. Endpoint selection and this qualification are developed in Choosing depth-cue coefficients.

#### 4.19.2.6 NCDT — light, modulate, and depth-cue three vectors

NCDT processes V0, V1 and V2 in order. Each supplied vector passes through the light matrix, color matrix plus ambient, base-color modulation and depth cue, then pushes one RGB result. Use it for three normals sharing **one** light/color setup, material RGBC, IR0 and far-color vector. NCDS performs one such calculation; NCCT omits depth cue; DPCT instead cues three already-finished FIFO colors without lighting or material gain calculation.

NCDT does not project positions or derive three cue coefficients from their depths. In particular, a preceding RTPT leaves only its final IR0; NCDT then uses that **same coefficient for all three colors**. Use separate appropriately programmed calculations when the vertices require different materials or cue weights.

**Encoding.** Write `0x00000016 | (SF<<19) | (LM<<10)` to COMMAND, `0x08010344`. Function bits 5:0 are `010110`. SF selects no main shift when zero or ASR12 when one; LM selects the final IR lower bound −32768 when zero or zero when one, with upper bound 32767. Both options apply at each light/color matrix stage and at the cue arithmetic stages; the temporary cue-difference clamp always forces LM=0.

The four words are `0x00000016`, `0x00000416`, `0x00080016`, `0x00080416` for SF/LM `(0,0)`, `(0,1)`, `(1,0)`, `(1,1)`. All selectors in bits 18:13 are ignored, as are bits 31:20,12:11,9:6. They cannot choose different normal slots, matrices or per-vertex cue inputs. Ignored bits remain in last COMMAND readback.

**INPUT.** D[n] is at `0x08010244+4n`; C[n] at `0x080102c4+4n`. Access whole aligned words.

| Input | Representation |
|---|---|
| D[0]/D[1], D[2]/D[3], D[4]/D[5] | V0/V1/V2: signed16 X low and Y high in the first word, Z signed low16 in the second |
| C[8]–C[12] — L | Nine signed16 light coefficients |
| C[13]–C[15] — B | Three signed32 ambient components |
| C[16]–C[20] — K | Nine signed16 color coefficients |
| D[6] — RGBC | R/G/B unsigned bytes at bits7:0/15:8/23:16; CODE at 31:24, shared by all outputs |
| D[8] — IR0 | Signed low16 cue coefficient, shared by all three calculations |
| C[21]–C[23] — F | Three signed complete far-color words |

With matrix base b=8 or16, rows are (`lo(C[b])`, `hi(C[b])`, `lo(C[b+1])`), (`hi(C[b+1])`, `lo(C[b+2])`, `hi(C[b+2])`), (`lo(C[b+3])`, `hi(C[b+3])`, `lo(C[b+4])`), with signed halves. Signed16 range is −32768…32767, signed32 range −2147483648…2147483647. The upper half of each Z word or last matrix word supplies no extra component. IR0 is consumed as signed16 even when software normally treats 0…4096 as cue weights; NCDT does not clamp or replace IR0 itself.

In the useful SF=1 convention, normal and matrix coefficients use 4096 per unit; the first stage produces Q12 light responses and the color stage Q12 material gains. Ambient has the same gain scale. RGBC is byte color; far F is byte×16. IR0/4096 is the nominal interpolation weight. These scales are software choices, not runtime tags or normalization operations. Negative or larger-than-unit IR0 extrapolates through the same limited integer arithmetic.

**Ordered per-vector calculation.** Execute the entire sequence below for V0, then V1, then V2. Clear FLAG only before V0. `ASR` rounds negative fractions toward minus infinity; signed44/signed32 mean the signed interpretations of the retained low44/32 bits.

1. **Light responses.** Sample the current supplied normal `(nx,ny,nz)`. Each row starts at zero and adds `L_i1×nx`, `L_i2×ny`, `L_i3×nz` in that order, wrapping signed44 after each addition. Publish `MAC_i=signed32(ASR(A_i,12×SF))`; clamp that signed word into the selected LM interval for IR_i. Complete all three rows from the same normal before writing their IR outputs.
2. **Color gains and ambient.** Sample those clamped responses `(r1,r2,r3)`. Each row begins at signed44 `4096×B_i`, then adds `K_i1×r1`, `K_i2×r2`, `K_i3×r3`, in that order with signed44 wrap after each. Again publish selected-SF MAC and selected-LM IR. These clamped IR values are channel gains `g_i`, not final colors.
3. **Unshifted modulation source.** For each channel, form `source_i=16×RGBC.byte_i×g_i`. Keep this **unshifted product** for both following cue passes. NCDT does not first perform NCCS's shifted MAC/color push and then read a quantized color back.
4. **Signed cue differences.** For all three channels, calculate:

```
difference_full_i = signed44(4096×F_i − source_i)
MAC_i = signed32(ASR(difference_full_i,12×SF))
difference_i = clamp(signed MAC_i,−32768,32767)
```

This clamp forces the signed interval even with LM=1, so cueing can move a channel downward. It contributes MAC/IR flags and writes temporary MAC words, but does **not** replace the IR register words with these differences.

5. **Final cue result.** Using signed IR0 and the limited differences:

```
final_full_i = signed44(source_i + IR0×difference_i)
MAC_i = signed32(ASR(final_full_i,12×SF))
IR_i = clamp(signed MAC_i,selected_lower_bound,32767)
```

The selected lower bound is −32768 for LM=0 or zero for LM=1. Final IR now replaces the gains. Both cue passes use selected SF, including the difference pass; there is no final-only SF option.

6. **Push color.** Limit `ASR(signed public MAC_i,4)` to 0…255 for each channel, independently of final IR clamping. Push RGB0 ← old RGB1, RGB1 ← old RGB2, RGB2 ← new RGB with CODE copied from RGBC. Continue to the next normal without clearing flags.

Every signed44 stage retains its overflow conditions before wrapping. An IR or byte clamp does not stop later arithmetic or suppress the push. All three normals use the same preserved IR0 and far words.

**Footprint and lifetime.** MAC1–MAC3 (D[25]–D[27]) and IR1–IR3 (D[9]–D[11]) finish with **V2's final cued** values. RGB0–RGB2 (D[20]–D[22]) finish with the three new colors for V0/V1/V2 in that order. Initial RGB histories `(A,B,C)` evolve as `(B,C,P0)`, `(C,P0,P1)`, `(P0,P1,P2)`; no old color survives. FLAG combines every stage of every normal. COMMAND is the last raw word and CYCLES becomes `0x0000002c`.

Preserved stored state includes V0–V2, RGBC, IR0, MAC0, OTZ, screen/depth histories, leading-sign ports, packed-input latch, all C[0]–C[30] and all PLUS state. D[15] still reads unchanged SXY2; D[28]/D[29] derive from final IR by clamping each `IR ASR7` to 0…31 and packing bits 4:0/9:5/14:10. No separate light responses, gains, cue differences, per-normal flags or per-normal IR0 values are retained for software.

**FLAG.** Signed44 positive/negative overflow sets row1/2/3 MAC bits 30/29/28 or 27/26/25. Every actual selected-LM matrix/final IR clamp and forced-signed temporary-difference clamp sets the corresponding IR bit 24/23/22. Final RGB byte clamps set R/G/B bits 21/20/19. ERROR bit 31 summarizes the MAC-overflow and IR1/IR2 conditions; IR3/color saturation alone does not set it. No MAC0, screen, depth, divide or IR0 saturation flags are generated. Conditions from earlier normals remain even if V2 is in range; neither legal endpoints nor a negative value inside its applicable signed interval sets saturation.

**Worked shared-material batch.** Program these independent inputs and issue `0x00080416` (SF=1, LM=1):

| Registers | Raw words |
|---|---|
| C[8]–C[12] — L | `0x00000c00`, `0xf8000800`, `0x0c000000`, `0x00000000`, `0xfffff000` |
| C[13]–C[15] — ambient | `0x00000100`, `0x00000200`, `0x00000080` |
| C[16]–C[20] — K | `0x04001000`, `0x08000000`, `0x00001000`, `0x08000400`, `0x00001000` |
| D[0],D[1] — V0 | `0x0000099a`, `0x00000ccd` — `(2458,0,3277)` |
| D[2],D[3] — V1 | `0x10000000`, `0x00000000` — `(0,4096,0)` |
| D[4],D[5] — V2 | `0x00000000`, `0x00001000` — `(0,0,4096)` |
| D[6]; D[8] | `0x34c060a0` — RGB `(160,96,192)`, CODE34; `0x00000800` — IR0=2048 |
| C[21]–C[23] | `0x00000500`, `0x00000600`, `0x00000a00` — far `(1280,1536,2560)`, target bytes `(80,96,160)` |

The two matrices' rows decode respectively to `(3072,0,2048)`, `(−2048,0,3072)`, `(0,0,−4096)` and `(4096,1024,0)`, `(2048,4096,0)`, `(1024,2048,4096)`.

| Vector | Clamped light responses | Color-stage gains | Unshifted material source |
|---|---|---|---|
| V0 | `(3482,1228,0)` | `(4045,3481,1612)` | `(10355200, 5346816, 4952064)` |
| V1 | `(0,0,0)` | `(256,512,128)` | `(655360,786432,393216)` |

| Vector | Clamped light responses | Color-stage gains | Unshifted material source |
|---|---|---|---|
| V2 | `(2048,3072,0)` | `(3072,4608,2176)` | `(7864320,7077888, 6684672)` |

V1 receives ambient despite having no positive directional response. V0/V2's negative third light responses set IR3 saturation. Cueing each source with the **shared** IR0 gives:

| Vector | Limited cue differences | Final MAC/IR | Output RGB / packed word |
|---|---|---|---|
| V0 | `(−1249,230,1351)` | `(1903,1420,1884)` | `(118,88,117)` / `0x34755876` |
| V1 | `(1120,1344,2464)` | `(720,864,1328)` | `(45,54,83)` / `0x3453362d` |
| V2 | `(−640,−192,928)` | `(1600,1632,2096)` | `(100,102,131)` / `0x34836664` |

For example, V2 red uses `(7864320+2048×(−640)) ASR12=1600`, then ASR4 gives 100. Final MAC/IR are V2's raw `0x00000640`, `0x00000660`, `0x00000830`; IR0 remains `0x00000800`. FLAG is `0x00400000`, ERROR clear; all final gains and colors can be positive while an earlier light clamp remains recorded.

**Weight and option contrasts.** Restore all operands before each command. With IR0=0 and SF=1/LM=1, the FIFO is `0x344b519e`, `0x34060c0a`, `0x34666c78`. With IR0=4096, it becomes `0x34a05f4f`, `0x34a06050`, `0x34a06050`. V0 does **not** reach exact far red/green bytes 80/96: its source fractions and the independently shifted difference lose precision, yielding 79/95. An endpoint promise based only on the ideal interpolation formula would be wrong even without cue clamps.

At IR0=2048, choosing LM=0 (`0x00080016`) preserves negative back-facing responses and produces `0x34285876`, `0x3453362d`, `0x34236664`, with FLAG zero. Choosing SF=0, LM=1 (`0x00000416`) omits the shift at every matrix and cue pass: all three colors become `0x34ff00ff`, FLAG=`0x81f80000`. These are different staged calculations, not alternative encodings of one byte-space blend.

**Timing.** NCDT writes 44 to normal CYCLES. Its three colors are available together when the single COMMAND write completes.

#### 4.19.2.7 CC — mix a supplied response vector and apply base color

CC begins with current IR1–IR3, applies the color matrix and ambient vector, then modulates by RGBC and pushes one color. Use it when the response vector has already been calculated or supplied. NCCS additionally obtains that response from V0 and the light matrix. CC does **not** skip the color matrix and is not merely a base-color multiply on already mixed gains. CDP uses the same starting point but adds depth cueing.

**Encoding.** `word=0x0000001c | (SF<<19) | (LM<<10)`. Function bits5:0 are 011100. SF bit19 selects no shift or ASR12 at both arithmetic stages. LM bit10 sets the IR lower limit to−32768 or0, upper32767. Selector bits18:13 and other unused bits are ignored. Zero-filled option words are 0000001c,0000041c, 0008001c,0008041c for SF/LM=0/0,0/1,1/0,1/1.

**Inputs.** Signed low halves of D[9]–D[11] supply the three responses. Color matrix K is nine signed16 coefficients packed row-major in C[16]–C[20]; ambient A is signed32 C[13]–C[15]. D[6] supplies unsigned base RGB bytes and CODE. V0–V2, the light/rotation matrices, translation, old MAC, IR0 and far vector are not operands. The input IR vector is snapshotted before output writes; its components need not be a unit normal or a position.

**Ordered calculation.** Clear FLAG and:

1. For each K row r, start with signed44 `4096*A[r]`, then add `K[r,0]*input1`, `K[r,1]*input2`, `K[r,2]*input3` in that order. Wrap signed44 after the initial term and every addition, retaining positive and negative overflow conditions. Publish `MAC=signed32(accumulator ASR(12*SF))` and clamp that signed word into IR range `[LM?0:-32768,32767]`. All row accumulations use the original input snapshot.
2. Form each `16*RGBC_byte*current signed IR`, wrap signed44 with its overflow test, apply the selected SF shift and visible signed32 word wrap, and clamp IR using selected LM. These overwrite the matrix-stage MAC/IR.
3. Shift each final signed MAC right4, clamp to0…255, and push packed RGB with **current RGBC.CODE**: old RGB1 becomes RGB0, old RGB2 becomes RGB1, and the new color becomes RGB2.

All right shifts are arithmetic, rounding negative fractions towards minus infinity. The push reads MAC rather than limited IR. For Q12 K/gains and SF1, unity gain4096 preserves a base byte; the final MAC is then byte×16. SF0 applies neither twelve-bit shift and therefore needs deliberately chosen input scales, not merely another label on the same result.

**Footprint and flags.** Replaces MAC1–MAC3, IR1–IR3, RGB0–RGB2 and FLAG. COMMAND/CYCLES retain the issue word and nominal11. All other stored words are preserved, including V0–V2, all controls and RGBC, IR0/MAC0, OTZ, screen/depth histories, packed-input latch, leading-sign words and PLUS. D[28]/D[29] derived reads reflect the new final IR, not an additional RGB push.

Ordered matrix overflow sets MAC lane positive/negative flags30:28/27:25 and ERROR. Selected IR clamps at either arithmetic stage set I1/I2/I3 bits24/23/22; I1/I2 summarize into ERROR, I3 alone does not. Byte limits set R/G/B bits21/20/19 without ERROR by themselves. Conditions accumulate even when later stages write in-range results; exact endpoints are legal. No MAC0/depth/divide/screen/IR0 stage runs. Preserve a needed FLAG before another command starts one, and RGB before subsequent pushes displace it.

**Worked preloaded responses.** Program K rows `(4096,1024,0)`, `(2048,4096,0)`, `(1024,2048,4096)`, ambient `(256,512,128)` and RGBC=`0x34c060a0`. Preload IR=(3482,1228,0): these are limited light responses, not final gains. No light matrix or V0 setup is needed for CC. With SF1/LM1:

| Channel | Color-row full sum | Gain in IR | Application full product | Final MAC/IR | Byte |
|---|---|---|---|---|---|
| Red | 16568320 | 4045 | 10355200 | 2528 | 158 |
| Green | 14258176 | 3481 | 5346816 | 1305 | 81 |
| Blue | 6604800 | 1612 | 4952064 | 1209 | 75 |

RGB2 is **0x344b519e**, FLAG=0. NCCS's earlier negative light response does not magically accompany these preloaded values: CC starts a new flag calculation and never runs that light stage. Software must retain any earlier light flags separately if they matter.

If IR is instead preloaded with the already mixed gains(4045,3481,1612), CC applies K and ambient **again**, producing new gains(5171,6015,4491), then RGB2=0x34d28cc9. This illustrates the command's actual starting boundary.

For a signed option contrast, preload IR=(0,−4096,0) with the same controls. K produces (−768,−3584,−1920). With LM0 the final MAC is (−480,−1344,−1440); the RGB push emits black and FLAG=00380000, three byte limits with ERROR clear. With LM1 the K outputs first clamp to zero, final MAC/IR are zero, the same black is emitted, and FLAG=81c00000 records all three IR clamps plus ERROR. Identical packed color does not imply identical arithmetic.

**Timing.** CC writes 11 to normal CYCLES. Results are available when its COMMAND write completes.

#### 4.19.2.8 CDP — color-matrix, material and cue from current IR

CDP starts with the three signed values already in IR1–IR3. It applies the color matrix and offsets, modulates RGBC, depth-cues the unshifted products and pushes one color. Choose it when directional responses are already available and the light matrix must not run again. NCDS adds that light stage and V0; DCPL instead omits the color matrix because its input IR already contains final channel gains. CC performs CDP's matrix/material calculation without cue.

**Encoding.** Write to normal COMMAND, `0x08010344`:

```
word = 0x00000014 | (SF << 19) | (LM << 10)
```

Function bits 5:0 are `010100`. SF=1 selects ASR 12 at the color matrix and at **both** cue passes; SF=0 selects no main-result shift at those stages. LM=0 selects IR limits −32768–32767; LM=1 selects 0–32767. The temporary cue difference always uses the signed −32768–32767 limits. All remaining bits, including matrix/vector/control selectors, are ignored for execution and retained in raw COMMAND readback. The four zero-filled option words are `0x00000014`, `0x00000414`, `0x00080014`, `0x00080414` for SF/LM 0/0, 0/1, 1/0 and 1/1 respectively.

**Inputs and scale.** IR1–IR3, D[9]–D[11], supply signed low-halfword inputs to the nine signed16 color coefficients C[16]–C[20]. The signed32 offset vector C[13]–C[15] is added by that matrix stage. RGBC at D[6] supplies unsigned RGB bytes and CODE. Signed low-halfword IR0 at D[8] and signed32 far components C[21]–C[23] supply cue operands. Direct IR0 inputs are not automatically restricted to 0–4096.

Under the SF=1 convention, Q12 matrix coefficients preserve the input response scale; program offsets in the resulting gain scale. A color-stage IR value of 4096 is unit material gain. Program ordinary far-color components as byte×16. These are

software scale choices. The light matrix, V0–V2, projection depths, DQA and DQB are not inputs. LM does not clamp the incoming signed IR values before the matrix; it limits the matrix results.

**Ordered calculation.** Snapshot the original IR vector x. With `s=12×SF`:

1. For each color row r, start at `4096×offset_r` and add `K[r,0]×x1`, `K[r,1]×x2`, `K[r,2]×x3` in order, with signed-44 wrap and cumulative overflow tests. Shift by s, wrap to signed32 MAC, then apply the selected IR clamp. All three row inputs come from the original snapshot, not a partly overwritten IR vector.
2. Form all three unshifted material products `P_i=16×RGBC.byte_i×signed IR_i` using those limited matrix results.
3. For each channel, test/wrap the full difference `4096×far_i−P_i` to signed44, shift by s, expose signed32 MAC and clamp to −32768–32767. Retain these three signed temporary differences D. They update MAC and flags but do not replace the IR register words.
4. Form `P_i+signed IR0×D_i`, test/wrap to signed44, shift by s and expose signed32 MAC. Apply the selected IR clamp. Use P unshifted, without a separate color-application shift.
5. Convert final signed MAC by ASR 4 and limit to an RGB byte. Push one color into RGB2, shifting old RGB1/RGB2 into RGB0/RGB1, with CODE from current RGBC.

See MAC, shift, and IR stages for the common wrap definitions. A negative temporary difference is permitted even with LM=1. SF=0 changes the matrix/cue scales and limited differences, not merely accuracy.

**State and flags.** CDP clears normal FLAG, then accumulates the color-matrix overflow/IR limits, both cue passes' overflow/limits and RGB output limits. The temporary-difference clamp is flagged even if IR0 is zero. IR1/IR2 saturation and signed44 overflow can set ERROR; IR3 or RGB saturation alone does not. Final cued MAC1–MAC3 and IR1–IR3 replace the input responses, and RGB0–RGB2 shift once. COMMAND and normal CYCLES are replaced.

All other stored state is preserved, including IR0, MAC0, OTZ, input vectors, RGBC, light/color matrices, offsets/far color, screen/depth histories and PLUS. The derived packed-color ports D[28]/D[29] reflect final IR. To process another material from the same original responses, reload or otherwise supply those responses again; the post-CDP IR values are already finished colors. CDP after CC does not merely add cue: it applies the color matrix to CC's final IR again.

**Timing.** CDP writes 13 (`0x0000000d`) to normal CYCLES. Results are available when its COMMAND write completes.

**Worked case: signed responses and a forced-signed cue difference.** Use the non-identity color matrix and offsets from Normals, light directions and color mixing, but start directly from the following IR words; no normal or light transform participates:

| Operand | Raw word(s) | Meaning |
|---|---|---|
| D[9]–D[11] | `0xfffffc00`, `0x00000800`, `0x00001000` | Responses `(−1024,2048,4096)` |
| D[6], RGBC | `0x34c060a0` | Base `(160,96,192)`, CODE `0x34` |
| D[8], IR0 | `0x00000800` | Coefficient 2048 |
| C[21]–C[23] | `0x00000500`, `0x00000600`, `0x00000a00` | Far RGB `(80,96,160)` in byte×16 |

Issue `0x00080414`, SF=1, LM=1. The ordered color-row accumulations finish at `(−1048576,8388608,20447232)`. Shifting gives MAC `(−256,2048,4992)`; selected limiting gives gains `(0,2048,4992)`. The negative incoming first response was used in the row products; it was the resulting red gain that clamped to zero.

| Channel | Unshifted P | Signed cue difference D | Final MAC / IR | Output byte |
|---|---:|---:|---:|---:|
| Red | 0 | 1280 | 640 | 40 |
| Green | 3145728 | 768 | 1152 | 72 |
| Blue | 15335424 | −1184 | 3152 | 197 |

RGB2 reads **`0x34c54828`**. Final MAC/IR words are `0x00000280`, `0x00000480`, `0x00000c50`. FLAG is `0x81000000`: the earlier red color-matrix IR clamp and ERROR, even though final red is positive. Blue's negative difference was retained despite LM=1 and reduced its amplified material term toward the lower far-blue value. IR0 remains 2048.

**LM boundary.** Reload the same original response words and change only LM to zero, command `0x00080014`. The red matrix gain remains −256, producing P=−655360 and signed difference 1440. Final red MAC is 560 and byte 35; RGB2 is `0x34c54823` and FLAG is zero. The result differs before cue because the matrix-stage clamp changed, not because the final cued color needed limiting.

### 4.19.3 Interpolation and general arithmetic commands

The commands in this group can operate without a vector transform or light matrix. For the depth-cue commands, IR0 is consumed as a signed low-halfword value. A coefficient of 2048 represents one half only in the SF=1, correctly scaled, unclamped calculation; directly programmed negative or larger coefficients remain representable inputs. For SF=1, far-color components are signed 32-bit words in the same scale as the final IR/MAC color values, normally byte×16 for a color FIFO result. Intermediate shift truncation, the temporary difference clamp, and arithmetic wrap remain part of the result even at nominal endpoint coefficients.

#### 4.19.3.1 DPCS — depth-cue a programmed RGB color

DPCS cues the finished RGB bytes in RGBC toward a programmed far color using current signed IR0, then pushes one RGB result. It performs neither matrix lighting nor base-color gain modulation. Use it when the source is already a byte color. INTPL starts from signed IR color components; DPCT consumes three existing FIFO colors; NCDS/NCDT retain the full modulation products of their lighting paths before cueing. DPCS does not read RGB2 automatically: copy a desired FIFO color into RGBC explicitly before using it as this command's source.

**Encoding.** Write `0x00000010 | (SF<<19) | (LM<<10)` to COMMAND at `0x08010344`. Function bits 5:0 are `010000`. SF selects no shift when zero or ASR12 when one, in **both** cue passes. LM selects final IR lower bound −32768 when zero or zero when one; upper bound is 32767. The temporary cue-difference clamp always permits negative values regardless of LM.

Option words are `0x00000010`, `0x00000410`, `0x00080010`, `0x00080410` for SF/LM `(0,0)`, `(0,1)`, `(1,0)`, `(1,1)`. Selectors bits 18:13 and bits 31:20,12:11,9:6 are ignored for execution and retained in COMMAND readback. They do not select a matrix, vector or FIFO source.

**INPUT.**

| Register | Address | Representation |
|---|---|---|
| D[6] — RGBC | `0x0801025c` | R/G/B unsigned bytes at 7:0/15:8/23:16; CODE at 31:24 |
| D[8] — IR0 | `0x08010264` | Signed low16 coefficient, −32768…32767, read sign-extended |
| C[21]–C[23] — far R/G/B | `0x08010318`, `0x0801031c`, `0x08010320` | Three complete signed32 words |
| Old D[21],D[22] | `0x08010298`, `0x0801029c` | Retained history words only, not source colors |

Use aligned whole-word accesses. Neither old IR1–IR3 nor a supplied normal, matrix, H, DQA/DQB or depth history is an arithmetic input. IR0 is consumed as programmed; DPCS does not derive it from depth, constrain it to 0…4096 or write it back.

**Useful SF=1 convention.** Program far components as target bytes×16 and IR0 as weight×4096. The source is converted internally from byte to byte×65536; dividing its cue result by 4096 produces byte×16 MAC units, and the final four-bit output shift produces bytes. With ordinary byte targets and IR0 in 0…4096, this supports interpolation between the source and far color. Larger/negative signed weights extrapolate through the same limited arithmetic; they are not rejected.

**Exact ordered calculation.** Sample IR0 and the three source bytes. For each channel i, form `source_i=65536×RGBC.byte_i`. `ASR` rounds negative fractions toward minus infinity; signed44/signed32 mean the signed interpretations of low44/low32 bits.

First calculate all three limited differences:

```
wide_difference_i = 4096×far_i − source_i
wrapped_i = signed44(wide_difference_i)
MAC_i = signed32(ASR(wrapped_i,12×SF))
difference_i = clamp(signed MAC_i,−32768,32767)
```

The full difference is tested for signed44 overflow before wrap. This pass writes temporary MAC words and contributes IR saturation flags, but the limited differences are temporary operands, **not writes to IR1–IR3**. Its lower bound is always −32768, even when LM=1.

Then calculate each final channel:

```
wide_final_i = source_i + signed IR0×difference_i
MAC_i = signed32(ASR(signed44(wide_final_i),12×SF))
IR_i  = clamp(signed MAC_i,LM ? 0 : −32768,32767)
byte_i = clamp(ASR(signed MAC_i,4),0,255)
```

Test the full final expression for signed44 overflow before wrapping. SF applies again here. The color byte comes from the **public MAC word**, not final clamped IR. A negative MAC can therefore set a color-byte flag even if LM=1 has already made its IR zero. Neither a clamp nor an overflow suppresses output.

Push RGB0 ← old RGB1, RGB1 ← old RGB2 and write RGB2 with the new R/G/B bytes and **RGBC's CODE**. CODE is copied, not interpolated. A second DPCS with unchanged RGBC repeats the same source calculation rather than cueing the previous RGB2 again.

**Footprint.** Modified words are MAC1–MAC3 D[25]–D[27], IR1–IR3 D[9]–D[11], RGB0–RGB2 D[20]–D[22], FLAG C[31], last COMMAND and CYCLES=`0x00000008`. Final MAC/IR contain the cued result, not the temporary differences. One color history entry is displaced.

All other stored state is preserved, including RGBC, IR0, MAC0, OTZ, V0–V2, screen/depth histories, far/control words C[0]–C[30], leading-sign ports, packed-input latch and all PLUS state. D[15] retains unchanged SXY2. D[28]/D[29] derive from final IR by clamping each `IR ASR7` to 0…31 and packing bits 4:0/9:5/14:10; this is not another RGB push.

**FLAG.** Start with a clear word and accumulate both passes and byte conversion. Signed44 positive/negative overflow sets the corresponding MAC1/2/3 bits 30/29/28 or 27/26/25. Temporary forced-signed or final selected-LM IR clamps set IR1/2/3 bits 24/23/22. Byte clamps set R/G/B bits 21/20/19. ERROR bit 31 summarizes MAC overflow and IR1/IR2 saturation, but not IR3 or color saturation alone. No IR0 saturation flag is generated, even for an extrapolating weight; MAC0/screen/depth/divide flags are also absent. Legal endpoints do not set saturation. IR0=0 does **not** skip the difference pass or its flags.

**Worked half cue with a downward blue correction.** Program RGBC=`0x56604020` — RGB `(32,64,96)`, CODE `0x56`; IR0=`0x00000800` (2048); far words C[21]–C[23]=`0x00000a00`, `0x00000800`, `0x00000400`, corresponding to target RGB `(160,128,64)`. Issue `0x00080410` (SF=1, LM=1).

| Channel | Source byte×65536 | Full difference | ASR12 limited difference | Final full expression | Final MAC/IR | Output byte |
|---|---|---|---|---|---|---|
| R | 2097152 | 8388608 | 2048 | 6291456 | 1536 | 96 |
| G | 4194304 | 4194304 | 1024 | 6291456 | 1536 | 96 |
| B | 6291456 | −2097152 | −512 | 5242880 | 1280 | 80 |

The temporary negative blue correction is allowed despite LM=1. No clamp occurs. Final MAC/IR read `0x00000600`, `0x00000600`, `0x00000500`; new RGB2 is **`0x56506060`**, FLAG is zero and CYCLES is 8. If initial FIFO words were `0x11112222`, `0x33334444`, `0x55556666`, they become `0x33334444`, `0x55556666`, `0x56506060`.

**Endpoints, extrapolation and options.** With the same source/far setup, independently initialize each row:

| Command / IR0 | Final signed MAC | RGB2 | FLAG |
|---|---|---|---|
| SF1/LM1, IR0=0 | `(512,1024,1536)` | `0x56604020` | `0x00000000` |
| SF1/LM1, IR0=4096 | `(2560,2048,1024)` | `0x564080a0` | `0x00000000` |
| SF1/LM1, IR0=8192 (`0x2000`) | `(4608,3072,512)` | `0x5620c0ff` | `0x00200000` — red byte saturation only |
| SF1/LM0, IR0=−4096 (`0xfffff000`) | `(−1536,0,2048)` | `0x56800000` | `0x00200000` |
| SF1/LM1, IR0=−4096 | Same MAC; IR1 becomes 0 | `0x56800000` | `0x81200000` — ERROR, IR1 and red byte saturation |
| SF0/LM1 (command `0x00000410`), IR0=2048 | (69203968, 71301120, −60817408) | `0x5600ffff` | `0x81f80000` |

The byte-target setup reaches exact source/far endpoints with SF=1 because source byte×16 and far words have integral differences and no clamp. For signed far words or fused modulation, include the temporary clamp and retained source fractions in the endpoint calculation. SF=0 in the last row leaves both passes unshifted, changing the scale of both calculations.

**Flags from a result that contributes zero.** Restore the main source, use IR0=0 and change only far red to 40000 (`C[21]=0x00009c40`). The temporary red difference becomes 39488, clamps to 32767 and sets IR1 saturation. Final color still

equals the original RGBC, but FLAG is `0x81000000`. Changing far red instead to signed32 minimum (`0x80000000`) makes its full difference `-2^43-2097152`: signed44 wrap makes the temporary shifted value 2147483136, while negative MAC1 overflow and temporary IR1 saturation remain recorded. Final source color is again unchanged, with FLAG=`0x89000000`. Zero weight is not a bypass or a flag-clearing shortcut.

**Timing.** DPCS writes 8 to normal CYCLES. Results are available when its COMMAND write completes.

#### 4.19.3.2 INTPL — interpolate a supplied vector towards the far vector

INTPL starts from signed IR1–IR3, forms a limited difference towards the three far-vector words, scales that difference by signed IR0, and pushes a color. Use it when those actual difference limits and scales match the calculation. It does not fetch positions, evaluate depth, select a half-way weight, or apply RGBC's base RGB bytes. DPCS starts from byte color; DCPL starts from a modulated product; INTPL starts from the supplied signed IR vector.

**Encoding.** `word=0x00000011 | (SF<<19) | (LM<<10)`. Function bits5:0 are 010001. SF bit19 selects no shift or ASR12 at **both** passes. LM bit10 sets the final IR lower bound−32768 or0, upper32767. Selectors18:13 and every other unused bit are ignored. The four zero-filled words are 00000011,00000411, 00080011,00080411 for SF/LM=0/0,0/1,1/0,1/1.

**Inputs.** D[9]–D[11] supply signed low16 starting components; D[8] supplies signed low16 IR0. Far components C[21]–C[23] are signed32 complete words. RGBC supplies only its unsigned high CODE byte. Other RGBC bytes, V0–V2, matrices, translation, old MAC, screen/depth histories and projection controls are not arithmetic inputs. Q12 weight interpretation is a software choice; INTPL does not constrain its signed IR0 input to0…4096.

**Ordered calculation.** Clear FLAG. Snapshot each starting component a_i and IR0, and form `in_i=4096*a_i`. Define the per-lane result stage as: test the complete expression against signed44 range$-2^{43\ldots2}43-1$, record its positive/ negative overflow, wrap to signed44, shift by12 if SF1, expose its low signed32 word in MAC_i. Then:

```
first_i = result_stage(4096*far_i - in_i)
d_i     = clamp(first_i, -32768,32767)       // forced signed clamp
final_i = result_stage(in_i + signed IR0*d_i)
IR_i    = clamp(final_i, LM?0:-32768,32767)
```

Calculate all three temporary differences before final results. The first-pass clamps update flags but do not install their temporary values in the IR words. The second pass overwrites MAC and installs final IR; earlier flags survive. ASR rounds negative fractions towards minus infinity. The temporary signed32 word and clamp, not an unlimited far-minus-start value, feed the final product.

Finally, arithmetic-shift each signed **final MAC** by4 and clamp to0…255. Shift RGB1→RGB0 and RGB2→RGB1, then put the new RGB bytes plus current RGBC.CODE in RGB2. The push does not use final IR as its byte source.

With SF1 and no wrap, the first pass is limited `far-a`, and the final expression is `a+(IR0/4096)*limited_difference` rounded down. That useful interpretation must not be generalized to unrestricted endpoints, or to SF0 where both passes have changed scale.

**Footprint and flags.** Replaces MAC1–MAC3, IR1–IR3, RGB0–RGB2 and FLAG. COMMAND/CYCLES retain the issue word and nominal8. Preserves all other stored words, including IR0, MAC0, input vectors, far/matrix/projection controls, RGBC, OTZ, screen/depth histories, packed-input latch, leading-sign state and PLUS. D[28]/D[29] derived reads follow final IR; they are not another color push.

Either expression can set MAC44 positive/negative lane flags30:28/27:25 and ERROR. The temporary forced-signed or final selected-LM clamp can set I1/I2/I3 bits24/23/22. I1/I2 also set ERROR; I3 alone does not. Byte limiting sets R/G/B bits21/20/19 without ERROR alone. Flags accumulate across both passes; endpoints are legal. There are no MAC0/depth/divide/screen/IR0 flag stages. Read needed FLAG and results before another command replaces or shifts them.

**Worked signed vector interpolation.** Reuse the motion samples in "Interpolation and general arithmetic": a=(96,−48,32), far=(−32,80,64), IR0=1024, RGBC=0x5a332211, SF1/LM0. Differences are(−128,128,32), all legal. The final full expressions are(262144,−65536,163840); after ASR12 both MAC and IR are **(64,−16,40)**. RGB2=0x5a020004 and FLAG=00100000 because the negative middle MAC is limited to green byte0. Signed vector arithmetic remains correct; the color push has lost sign and precision.

For a full-weight boundary, start at−30000, far=30000, IR0=4096 in one lane, other lanes zero. The difference60000 limits to32767, so the result is2767, not the legal far endpoint30000. FLAG=81000000 identifies the temporary IR1 clamp even though the final result fits. A weight of4096 is not an unconditional endpoint-selection instruction.

**Option contrast.** Independently start at(5,−3,2), far=(9,1,−2), IR0=1024, CODE=0x5a:

| SF/LM | Difference after forced clamp | Final signed MAC | Final IR | FLAG |
|---|---|---|---|---|
| 1/0 | (4,4,−4) | (6,−2,1) | Same | 00100000 |
| 1/1 | (4,4,−4) | (6,−2,1) | (6,0,1) | 80900000 |
| 0/0 | (16384, 16384, −16384) | (16797696, 16764928, −16769024) | (32767, 32767, −32768) | 81f80000 |
| 0/1 | (16384, 16384, −16384) | Same | (32767, 32767, 0) | 81f80000 |

SF1 pushes CODE with black RGB; SF0 pushes 0x5a00ffff. The large SF0 results are not a preserved version of the same interpolation with extra output fractional bits: the difference also lacked its shift before multiplication.

**Timing.** INTPL writes 8 to normal CYCLES. Results are available when its COMMAND write completes.

#### 4.19.3.3 DCPL — apply current gains and depth-cue the material

DCPL treats current IR1–IR3 as channel gains, multiplies them by RGBC's base bytes and applies depth cue before pushing one color. No light or color matrix is evaluated. Choose it when the final gains are already available. CDP still applies the color matrix/offsets; NCDS also calculates directional responses from V0. DPCS cues RGBC directly without gains, while INTPL treats IR itself as the source color rather than multiplying it by RGBC.

**Encoding.** Write to normal COMMAND, `0x08010344`:

```
word = 0x00000029 | (SF << 19) | (LM << 10)
```

Function bits 5:0 are `101001`. SF=1 shifts by twelve at **both** cue passes; SF=0 does not shift their main results. LM selects the final IR range: −32768–32767 for zero, 0–32767 for one. The temporary difference always has signed limits −32768–32767. All other fields, including selectors, are ignored for execution and retained in raw COMMAND readback. Zero-filled option words are `0x00000029`, `0x00000429`, `0x00080029`, `0x00080429` for SF/LM 0/0, 0/1, 1/0, 1/1.

**Inputs and useful scale.** IR1–IR3, D[9]–D[11], are signed low-halfword gains. RGBC, D[6], supplies unsigned base RGB bytes and CODE. IR0, D[8], is a signed low-halfword coefficient, not an automatically limited unsigned fraction. Far red/green/blue C[21]–C[23] are signed32 words. All matrices, offset vector C[13]–C[15], vector slots and geometric histories are unused.

For SF=1, gain 4096 is unity: `16×byte×4096` shifts to `16×byte`, which the final ASR 4 converts back to the base byte. Ordinary far colors therefore use byte×16. These conventions do not restrict the signed input format: negative gains and gains above unity participate directly. LM does not first clamp the supplied gains. A zero coefficient affects the final cue sum, but does not bypass its temporary difference calculation.

**Ordered calculation.** Read all three signed gains and form `P_i=16×RGBC.byte_i×gain_i`. Keep those products unshifted. Let `s=12×SF`.

1. For all three channels, calculate the full difference `4096×far_i−P_i`. Test signed44 overflow, wrap to signed44, shift by s and expose its signed32 MAC word. Clamp that word to −32768–32767 to obtain temporary D_i. These clamps update flags but do not replace the IR register words.
2. For each channel, form the full sum `P_i+signed IR0×D_i`. Test/wrap to signed44, shift by s and expose signed32 MAC. Write its selected-clamped IR copy. The unshifted P term is used here, not a separately shifted, limited or byte-converted material result.
3. Convert each final signed MAC by ASR 4 and clamp to 0–255. Shift RGB0 ← old RGB1, RGB1 ← old RGB2, and write the new RGB2 with current RGBC CODE.

The wrap/shift definitions are in MAC, shift, and IR stages. This is the cue calculation's two-pass integer path, not an ideal blend applied after a separately executed color multiplication. SF=0 changes both passes' scales and their limited differences.

**State and flags.** DCPL replaces MAC1–MAC3 and IR1–IR3 with the final cued values, shifts RGB0–RGB2 once, and replaces normal FLAG, COMMAND and CYCLES. FLAG is cleared initially and accumulates signed44 overflow, temporary-difference limits, selected final IR limits and output-byte limits. Temporary saturation can be reported even if IR0 is zero. Signed44 overflow and IR1/IR2 saturation can set ERROR; IR3 and RGB saturation alone do not.

All other stored state is preserved: IR0, MAC0, OTZ, RGBC, all vector and control operands, screen/depth histories and PLUS. D[28]/D[29] derive their packed-color reads from the new IR values, without a separate packed-result write. Retain or reload gains when reusing them: post-DCPL IR contains finished color values, not the original gain vector.

**Timing.** DCPL writes 8 (`0x00000008`) to normal CYCLES. Results are available when its COMMAND write completes.

**Worked case: amplified, reduced and negative gains.** Program these word contents; no matrix setup is needed:

| Destination | Raw word(s) | Interpretation |
|---|---|---|
| D[9]–D[11] | `0x00002000`, `0x00000800`, `0xfffffc00` | Gains `(2,1/2,-1/4)` in Q12 |
| D[6], RGBC | `0x3480a060` | Base `(96,160,128)`, CODE `0x34` |
| D[8], IR0 | `0x00000400` | Coefficient one quarter |
| C[21]–C[23] | `0x00000200`, `0x00000400`, `0x00000c00` | Far RGB `(32,64,192)` in byte×16 |

Issue `0x00080429`, SF=1, LM=1. The calculation is:

| Channel | Unshifted P | Signed D after first pass | Final MAC / IR | Byte |
|---|---|---|---|---|
| Red | 12582912 | −2560 | 2432 | 152 |
| Green | 5242880 | −256 | 1216 | 76 |
| Blue | −2097152 | 3584 | 384 | 24 |

RGB2 reads **`0x34184c98`**, final MAC/IR words are `0x00000980`, `0x000004c0`, `0x00000180`, and FLAG is zero. Although the incoming blue gain and starting product were negative, no input clamp removed them. Cue brought the final blue value into positive range before the selected LM=1 output clamp. IR0 remains `0x00000400`.

**Zero-coefficient boundary.** Reload the original gain words, set IR0=0 and repeat the same command. Final MAC is `(3072,1280,-512)`, while final IR is `(3072,1280,0)`. The pushed RGB is `(192,80,0)`, word `0x340050c0`, and FLAG is **`0x00480000`**: IR3 saturation plus blue-byte saturation, with ERROR clear. The byte conversion used the negative MAC, not a silently prelimited input gain.

With the original gains reloaded and IR0=4096 instead, this particular example reaches exact far RGB `(32,64,192)`, word `0x34c04020`, because its first-pass differences are exact and unclamped. Fractional material products or limited differences need not have that endpoint property; see Choosing depth-cue coefficients.

#### 4.19.3.4 DPCT — depth-cue the three FIFO colors

DPCT cues the three existing colors in RGB0–RGB2 and replaces that FIFO with three new colors in the same source order. Each uses the same signed IR0, far color, SF and LM. Use it when the source colors are already finished byte colors. DPCS instead starts from RGBC's RGB bytes; NCDT starts from three normals and performs lighting/modulation before cueing. DPCT performs no lighting, projection or drawing, and does not obtain three different weights from depth.

**Encoding.** Write `0x0000002a | (SF<<19) | (LM<<10)` to COMMAND at `0x08010344`. Function bits 5:0 are `101010`. SF selects no shift or ASR12 in both cue passes. LM selects the final IR lower bound −32768 or zero, with upper bound 32767; the temporary difference clamp always forces the signed interval.

Option words are `0x0000002a`, `0x0000042a`, `0x0008002a`, `0x0008042a` for SF/LM `(0,0)`, `(0,1)`, `(1,0)`, `(1,1)`. Selectors bits 18:13 and bits 31:20,12:11,9:6 are ignored for execution and retained in COMMAND readback. They do not choose a different FIFO or input count.

**INPUT.** Use aligned whole-word accesses.

| Register | Address | Consumed fields |
|---|---|---|
| D[20]–D[22] — RGB0–RGB2 | `0x08010294`, `0x08010298`, `0x0801029c` | Three initial RGB byte triples; each word's R/G/B are bits 7:0/15:8/23:16 |
| D[6] — RGBC | `0x0801025c` | **Only CODE**, bits 31:24, used for every new output |
| D[8] — IR0 | `0x08010264` | Signed low16 weight, −32768…32767 |

| Register | Address | Consumed fields |
|---|---|---|
| C[21]–C[23] — far R/G/B | `0x08010318`, `0x0801031c`, `0x08010320` | Three complete signed32 components |

The input FIFO's three CODE bytes are ignored. RGBC's RGB bytes are also ignored: this is not three repetitions of the same RGBC color. IR0 reads sign-extended and remains unchanged; it is not clamped to 0…4096 or recalculated between colors. Old IR1–IR3, normals, matrices, screen/depth histories and DQA/DQB are not operands.

Under the useful SF=1 convention, far components are target bytes×16 and IR0/4096 is the numerical cue weight. Each source byte is promoted to byte×65536 before the cue passes. The final MAC has byte×16 units before its four-bit byte conversion. These are software scales, not normalization or validity checks performed by the device.

**Consume and push in order.** Let initial FIFO words be A,B,C. Each of three iterations reads the **then-current RGB0**, cues its RGB fields, and pushes a new result:

| Iteration | Source read from RGB0 | FIFO after push |
|---|---|---|
| 0 | Original A | B, C, P(A) |
| 1 | Original B | C, P(A), P(B) |
| 2 | Original C | P(A), P(B), P(C) |

No newly produced color is cued again within this command. The trace describes ordered arithmetic/history updates during one normal command, not three separately published asynchronous operations. FLAG is cleared once before iteration 0 and accumulates through iteration 2.

**Per-color arithmetic.** Sample that iteration's three source bytes. For each channel, use `S_i=65536×source_byte_i`. `ASR` rounds negative fractions toward minus infinity. Test each full expression for signed44 overflow before taking its signed44 wrapped value; selected-SF results are then wrapped to signed32 public MAC words.

First calculate all three differences:

```
MAC_i = signed32(ASR(signed44(4096×far_i − S_i),12×SF))
d_i   = clamp(signed MAC_i,−32768,32767)
```

The forced signed clamp permits negative corrections even with LM=1. Temporary MAC writes and clamp flags are real, but d_i is not written into the IR register words. Then calculate the final channel:

```
MAC_i = signed32(ASR(signed44(S_i + signed IR0×d_i),12×SF))
IR_i  = clamp(signed MAC_i,LM ? 0 : −32768,32767)
byte_i = clamp(ASR(signed MAC_i,4),0,255)
```

The byte calculation uses **MAC, not clamped IR**. Push the three bytes together with RGBC's CODE. All colors execute both passes, even at zero weight; flags from a discarded or zero-weight difference remain. A negative or greater-than-unit IR0 extrapolates through the same limits rather than raising an input-validity exception.

Because LM affects only final IR here, not the MAC-based output or the next original FIFO source, changing LM alone does not change the three pushed RGB values. It can change final IR and the accumulated flags. SF changes both cue passes and therefore generally changes the colors.

**Footprint.** Modified: RGB0–RGB2 D[20]–D[22] (all three new outputs), MAC1–MAC3 D[25]–D[27] and IR1–IR3 D[9]–D[11] (final **third source's** cued values), FLAG C[31], last COMMAND and CYCLES=`0x00000011`. No old FIFO color or per-source FLAG remains separately accessible.

Preserved stored state includes RGBC, IR0, MAC0, OTZ, all vectors, screen/depth histories, all C[0]–C[30], leading-sign ports, the packed-input latch and all PLUS state. D[15] still reads unchanged SXY2. D[28]/D[29] derive from final IR by clamping each `IR ASR7` to 0…31 and packing bits 4:0/9:5/14:10; they do not hold three independent color results. Preserve the batch's FLAG before a later normal command replaces it.

**FLAG.** Positive/negative signed44 overflow sets MAC1/2/3 bits 30/29/28 or 27/26/25. Any temporary forced-signed or final selected-LM IR clamp sets bits 24/23/22. Byte clamps set R/G/B bits 21/20/19. ERROR bit 31 summarizes MAC overflow and IR1/IR2 saturation, not IR3 or color saturation alone. All three iterations contribute to the same word; final MAC/IR need

not expose the input that caused a flag. No IR0, MAC0, depth, screen or divide flags are generated, and legal clamp endpoints do not set saturation.

**Worked three-source FIFO.** Program these inputs, then issue `0x0008042a` (SF=1, LM=1):

| Register(s) | Raw input | Meaning |
|---|---|---|
| D[20] | `0x11604020` | A: RGB `(32,64,96)`, old CODE `0x11` |
| D[21] | `0x224080a0` | B: RGB `(160,128,64)`, old CODE `0x22` |
| D[22] | `0x338010f0` | C: RGB `(240,16,128)`, old CODE `0x33` |
| D[6] | `0x7fabcdef` | Output CODE `0x7f`; RGB `(239,205,171)` deliberately irrelevant |
| D[8] | `0x00000800` | Shared weight 2048 |
| C[21]–C[23] | `0x00000500`, `0x00000600`, `0x00000a00` | Far `(1280,1536,2560)`, target bytes `(80,96,160)` |

For these integral source/far values, the SF=1 difference equals far minus source-byte×16:

| Source | Limited differences | Final MAC/IR | Output RGB / word |
|---|---|---|---|
| A | `(768,512,1024)` | `(896,1280,2048)` | `(56,80,128)` / `0x7f805038` |
| B | `(−1280,−512,1536)` | `(1920,1792,1792)` | `(120,112,112)` / `0x7f707078` |
| C | `(−2560,1280,512)` | `(2560,896,2304)` | `(160,56,144)` / `0x7f9038a0` |

For C red, `(240×65536+2048×(−2560)) ASR12=2560`, then ASR4 gives 160. Final FIFO is exactly the three output words above, with CODE `0x7f` in **all** entries. Final MAC/IR raw words are `0x00000a00`, `0x00000380`, `0x00000900`; IR0 and RGBC are unchanged. FLAG is zero; CYCLES reads 17.

**Endpoint, extrapolation and flag-lifetime contrasts.** Restore the original three FIFO words before every command; otherwise the next DPCT consumes the already-cued results.

- With SF=1, LM=1 and IR0=0, RGB bytes are unchanged but CODE is replaced: FIFO becomes `0x7f604020`, `0x7f4080a0`, `0x7f8010f0`.
- With IR0=4096, all three become `0x7fa06050`, the programmed byte target, with no flag in this ordinary setup.
- With IR0=8192, SF=1 and LM=1, outputs are `0x7fe08080`, `0x7fff4000`, `0x7fc0b000`. B's blue byte saturates and C's red MAC−1280 gives both IR1 and red-byte saturation: FLAG=`0x81280000`. With LM=0 the **same RGB words** are produced, but final IR1 retains −1280 and FLAG is only `0x00280000` (red/blue color clamps, ERROR clear).
- With SF=0, LM=1 and IR0=2048, outputs are `0x7fffffff`, `0x7fff0000`, `0x7fffff00`; FLAG=`0x81f80000`. Omitting both shifts is not an alternative encoding of the half cue.
- With SF=1, LM=1, IR0=0 and only far red changed to 35000 (`0x000088b8`), A's red difference 34488 clamps, while B/C's differences 32440/31160 fit. Final FIFO still has the original RGB bytes, but FLAG=`0x81000000` retains **A's** temporary IR1 clamp. Final C MAC/IR are in range.

**Timing.** DPCT writes 17 to normal CYCLES. All three colors are available when its COMMAND write completes.

#### 4.19.3.5 SQR — square the three current IR components

SQR independently squares signed IR1, IR2 and IR3. It is useful for component energy or squared-length work after software has chosen the scales. It does not sum the squares, calculate a square root, or normalize the original vector. It does not use matrix coefficients or push a color.

**Encoding.** `word=0x00000028 | (SF<<19) | (LM<<10)`. Function bits5:0 are 101000. SF bit19 selects no shift or ASR12. LM bit10 selects the ordinary IR lower bound−32768 or0, upper32767. Other bits, including selectors18:13, are ignored. Zero-filled option words are 00000028,00000428,00080028,00080428 for SF/LM=0/0,0/1,1/0,1/1.

**Inputs and calculation.** Snapshot the signed low halves of D[9]–D[11] into a1,a2,a3. For each lane:

```
product_i = a_i*a_i
MAC_i     = signed32(product_i ASR(12*SF))
IR_i      = clamp(MAC_i, LM?0:-32768,32767)
```

Every row reads the snapshot, not a preceding squared output. Legal inputs are −32768…32767, so their squares are0…1073741824. All are nonnegative signed32 values before either shift: visible-word publication cannot wrap in SQR, and LM0 and LM1 produce the same values and flags. The selected IR upper limit can still discard a large valid square. SQR supplies no signed44 overflow flags.

**Footprint and flags.** Replaces MAC1–MAC3, IR1–IR3 and FLAG; COMMAND/CYCLES retain the issue and nominal5. An IR result strictly above32767 sets that lane's I1/I2/I3 mask01000000/00800000/00400000. I1 or I2 also sets ERROR; I3 alone does not. No negative clamp is possible from these squares. There are no MAC44, MAC0, RGB, depth, divide, screen or IR0 flag stages.

All other stored state is preserved, including IR0/MAC0, V0–V2, controls, RGBC and every FIFO, OTZ, packed-input/leading-sign state and PLUS. D[28]/D[29] derived packed reads can change with IR without a color push. Retain an original signed vector before SQR if it is needed later: signs are not recoverable from its squares. Read needed MAC/FLAG before later commands replace them.

**Worked squared length.** Use the perpendicular vector from the practical arithmetic lesson, IR=(−144,−112,208). With SF0/LM0, normal word00000028:

| Component | Exact square / MAC | IR copy |
|---|---|---|
| −144 | 20736 | 20736 |
| −112 | 12544 | 12544 |
| 208 | 43264 | 32767 |

FLAG=00400000. Software can sum the valid complete MAC results to76544, the squared length in the selected component units. Summing limited IR instead gives66047 and changes the calculation. ERROR being clear does not mean every IR output is unmodified.

For a Q12 input vector(4096,−2048,1024), choose SF1 instead. Its full products have two factors of4096; shifting twelve leaves Q12 squared components (4096,1024,256), representing(1,1/4,1/16). Their software sum5376 represents 21/16. FLAG is zero. Neither the positive squared vector nor that scalar sum is the original vector normalized.

**Range and option boundary.** Independently supply(181,182,−32768):

| SF | Signed MAC1–MAC3 | IR1–IR3, either LM | FLAG |
|---|---|---|---|
| 0 | (32761,33124,1073741824) | (32761,32767,32767) | 80c00000 |
| 1 | (7,8,262144) | (7,8,32767) | 00400000 |

$181^2$ fits IR and$182^2$ does not at SF0. At SF1 the first two fit, but the third still exceeds the IR upper limit. Its MAC value262144 remains available. The absence of MAC-overflow flags is correct, not evidence of a missing signed32 wrap signal: all legal products fit the word.

**Timing.** SQR writes 5 to normal CYCLES. Results are available when its COMMAND write completes.

#### 4.19.3.6 GPF — multiply three IR lanes by one scalar

GPF multiplies current IR1–IR3 by the common signed scalar IR0. Use it for a fresh scalar/vector product when its color-FIFO side effect is acceptable. It does not add old MAC values; use GPL when that accumulation is intended. It is not a dot product: the three lanes remain independent and are not summed. For an independent multiply-add path without a color push, consider PLUS VMAD3 with its separate operands, publication timing and result limits.

**Encoding.** Write to normal COMMAND, `0x08010344`:

```
word = 0x0000003d | (SF << 19) | (LM << 10)
```

Bits 5:0 select `111101`. SF=0 retains the integer products; SF=1 applies ASR 12. LM=0 selects IR limits −32768–32767; LM=1 selects 0–32767. Selectors and all other bits are ignored for execution and retained in raw COMMAND readback. The four zero-filled words are `0x0000003d`, `0x0000043d`, `0x0008003d`, `0x0008043d` for SF/LM 0/0, 0/1, 1/0, 1/1. No option suppresses the color push or selects a different scalar.

**Inputs and scale.** IR0 at D[8] and IR1–IR3 at D[9]–D[11] are signed low-halfword operands, −32768–32767. IR0 is not restricted to a nonnegative depth-cue fraction. RGBC at D[6] supplies CODE only: its RGB bytes do not multiply the numeric result. No matrix, control vector, V slot, prior MAC, far color or geometric FIFO is read.

For two Q12 operands, SF=1 yields a Q12 product. Alternatively, an integer vector multiplied by a Q12 scalar returns integer-vector units under SF=1. SF=0 retains the unshifted product scale; it does not make the limited IR result wider. For direct color output, the visible MAC values must be in byte×16 units, independently of any numeric scale assigned to IR.

**Ordered calculation.** Read signed IR0 and the three signed IR inputs. For each lane i=1,2,3:

```
P_i   = signed IR0 * signed IR_i
MAC_i = P_i ASR (12*SF)
IR_i  = clamp(MAC_i, LM?0:-32768, 32767)
byte_i = clamp(MAC_i ASR 4, 0, 255)
```

The MAC path uses signed44 arithmetic and a signed32 visible word as defined in MAC, shift, and IR stages. Here every signed16 product already fits both widths, so wrapping changes none of these products and GPF cannot generate MAC-overflow flags from its operands. Arithmetic shift rounds negative fractions toward minus infinity: for example, `2048×(−3) ASR 12 = −2`, not −1.

After the three numeric results, push the bytes as one color: RGB0 ← old RGB1, RGB1 ← old RGB2, RGB2 ← new RGB with current RGBC CODE. Byte conversion uses the **signed MAC**, not its selected-clamped IR copy. No previous MAC value enters P, even if its stored bits were written immediately before GPF.

**State and flags.** GPF clears normal FLAG, writes MAC1–MAC3 and IR1–IR3, pushes RGB0–RGB2 once, and replaces COMMAND/CYCLES. IR-limit and RGB-limit flags accumulate. IR1 or IR2 saturation contributes to ERROR; IR3 and RGB saturation alone do not. A numeric product can be acceptable for IR while still setting a color saturation flag because the color interpretation has a different range.

All other stored state is preserved, including the scalar IR0, MAC0, OTZ, RGBC, all vectors/control words, screen/depth histories and PLUS. Derived packed-color read ports D[28]/D[29] reflect the new IR values; no separate packed-result latch is written. Retain numeric results before a later command overwrites them, and account for the RGB push even when no color is wanted.

**Timing.** GPF writes 5 (`0x00000005`) to normal CYCLES. Results are available when its COMMAND write completes.

**Worked case: a signed Q12 vector product with an unwanted color clamp.** Program these word contents:

| Operand | Raw word(s) | Interpretation |
|---|---|---|
| D[8], IR0 | `0x00000c00` | Q12 scalar 3/4 |
| D[9]–D[11] | `0x00001000`, `0xfffff800`, `0x00000400` | Q12 vector `(1,−1/2,1/4)` |
| D[6], RGBC | `0x5a7f2040` | CODE `0x5a`; RGB bytes ignored |

Issue `0x0008003d`, SF=1, LM=0. The three products and results are:

| Lane | Unshifted product P | Final MAC / IR | Q12 meaning | Byte before limit |
|---|---|---|---|---|
| 1 | 12582912 | 3072 | 3/4 | 192 |
| 2 | −6291456 | −1536 | −3/8 | −96 |
| 3 | 3145728 | 768 | 3/16 | 48 |

MAC/IR words are `0x00000c00`, `0xfffffa00`, `0x00000300`. The numeric vector is unsaturated, but the negative green-byte value clamps to zero. RGB2 reads **`0x5a3000c0`** and FLAG is **`0x00100000`**, green color saturation with ERROR clear. IR0 remains 3072. Seeding MAC1–MAC3 with `(123,456,789)` before issue does not change any result: GPF did not accumulate those words.

**LM boundary.** Reload the original vector, retain scalar 3072 and issue `0x0008043d`, LM=1. MAC remains `(3072,−1536,768)`, but IR becomes `(3072,0,768)`. RGB2 is still `0x5a3000c0`; FLAG becomes `0x80900000` from IR2 saturation, green-byte saturation and ERROR. The green color flag remains because conversion used negative MAC2, not the zero IR2 copy.

**SF boundary.** Reload the same original inputs and issue `0x0000003d`, SF=0, LM=0. MAC retains (`12582912`, `−6291456`, `3145728`), while IR clamps to (`32767`, `−32768`, `32767`). RGB2 becomes `0x5aff00ff`; FLAG is `0x81f80000`, all three IR-limit and RGB-limit fields plus ERROR. This shows why a Q12 input convention must be paired with the chosen shift and why neither IR saturation nor the color push can be omitted when budgeting the command's effects.

#### 4.19.3.7 GPL — add a scalar product to the visible MAC words

GPL adds signed IR0×IR1/2/3 products to the corresponding existing **public MAC1–MAC3 words**, with SF-dependent alignment. It publishes new MAC/IR values and pushes one RGB color. Use it for a three-lane multiply-accumulate when those retained MAC words are the intended accumulators. GPF omits the old-MAC term. GPL is not a matrix calculation, depth cue, or a way to retain all earlier hidden accumulator precision.

**Encoding.** Write `0x0000003e | (SF<<19) | (LM<<10)` to COMMAND, `0x08010344`. Function bits 5:0 are `111110`. SF=0 leaves the complete sum unshifted; SF=1 scales the old public MAC by 4096 before adding the product and shifts the resulting sum right 12. LM selects final IR lower bound −32768 when 0 or zero when 1, upper bound 32767.

Option words are `0x0000003e`, `0x0000043e`, `0x0008003e`, `0x0008043e` for SF/LM `(0,0)`, `(0,1)`, `(1,0)`, `(1,1)`. Selectors bits 18:13 and bits 31:20,12:11,9:6 are ignored for execution and retained in COMMAND readback; none selects a vector slot or matrix.

**INPUT.** Use aligned whole-word register accesses.

| Register | Address | Consumed representation |
|---|---|---|
| D[8] — IR0 | `0x08010264` | Signed low16 scalar, −32768…32767 |
| D[9]–D[11] — old IR1–IR3 | `0x08010268`, `0x0801026c`, `0x08010270` | Three signed low16 multiplicands |
| D[25]–D[27] — old MAC1–MAC3 | `0x080102a8`, `0x080102ac`, `0x080102b0` | Three complete signed32 public words |
| D[6] — RGBC | `0x0801025c` | Only CODE bits 31:24 for the new color; RGB bytes unused |
| Old RGB1/RGB2, D[21]/D[22] | `0x08010298`, `0x0801029c` | Retained color-history words, not arithmetic operands |

IR inputs read sign-extended halfwords. MAC inputs are the signed interpretations of their raw 32 words, not their unsigned readback magnitudes and not private unshifted values retained by a previous command. Software may explicitly preload D[25]–D[27]. MAC0, vectors, matrices, far colors, screen/depth histories and projection constants are not operands. IR0 is not restricted to the usual cue interval 0…4096.

**Exact arithmetic.** Process the three independent lanes in order. Capture each lane's old signed IR and public MAC before replacing that lane. `ASR` rounds negative fractions toward minus infinity; signed44/signed32 retain the signed low44/low32 bits:

```
full_i = signed IR0×old signed IR_i
       + old signed MAC_i×(SF ? 4096 : 1)
wrapped_i = signed44(full_i)
MAC_i = signed32(ASR(wrapped_i,12×SF))
IR_i  = clamp(signed MAC_i,LM ? 0 : −32768,32767)
```

Test the **full sum** for signed44 positive/negative overflow before wrapping. Public MAC writes wrap to 32 bits rather than saturating. IR clamps consume that signed public word, not the wide value. Updating lane 1 cannot change lane 2/3's separate operands. IR0 remains unchanged.

Then convert each signed public MAC to a color byte with `clamp(ASR(MAC_i,4),0,255)`, independently of IR clamping. Push RGB0 ← old RGB1, RGB1 ← old RGB2, RGB2 ← new R/G/B plus RGBC.CODE. This color push is unavoidable even when GPL is being used for non-color arithmetic. A zero scalar does not skip readout, IR clamps, flags or the push.

**Scale choice.** With SF=1, use Q12 IR0 and a common unit for old MAC and IR multiplicands. Before overflow/clamping, the result is `old_MAC + ASR(IR0×IR,12)`. Multiplying old MAC by 4096 aligns the sum; it does not recover discarded fractional bits.

With SF=0, IR0 acts as an unscaled integer multiplier: `signed32(old_MAC+IR0×IR)`, subject to signed44 checking first. Keeping raw IR0=2048 does not keep a factor of one-half in this mode. The final byte converter always shifts by 4, so arbitrary arithmetic results still cause deterministic color saturation and history changes.

**Footprint and repeated use.** Modified: MAC1–MAC3 D[25]–D[27], IR1–IR3 D[9]–D[11], RGB0–RGB2 D[20]–D[22], FLAG C[31], last COMMAND and CYCLES=`0x00000005`. Final IR is the clamped accumulated result, **not the original multiplicand**. Repeatedly issuing GPL without reloading IR therefore does not add a fixed increment vector each time.

Preserved stored state includes IR0, RGBC, MAC0, OTZ, V0–V2, screen/depth histories, all C[0]–C[30], leading-sign ports, packed-input latch and all PLUS state. D[15] still reads unchanged SXY2. D[28]/D[29] derive from final IR by clamping each `IR ASR7` to 0…31 and packing bits 4:0/9:5/14:10; they do not preserve old multiplicands or old-MAC precision. Save any required FLAG before another recognized command replaces it.

**FLAG.** Clear old FLAG before calculation. A full lane sum above `2^43-1` or below `-2^43` sets its MAC1/2/3 positive bits 30/29/28 or negative bits 27/26/25. The final selected-LM IR clamps set bits 24/23/22; IR3 uses this ordinary rule, not projection's special test. Final byte clamps set R/G/B bits 21/20/19. ERROR bit 31 summarizes MAC overflow and IR1/IR2 clamps; IR3/color saturation alone does not set it. No MAC0, depth, screen, divide or IR0 flags are generated. Signed32 word wrap has no separate MAC overflow bit, and legal clamp endpoints do not set saturation.

**Worked signed increment with SF=1.** Program the following independent state and issue `0x0008003e` (SF=1, LM=0):

| Register(s) | Raw word(s) | Value |
|---|---|---|
| D[8] | `0x00000800` | Q12 scalar 2048, one-half |
| D[9]–D[11] | `0x00000140`, `0xffffff60`, `0x00000060` | Multiplicands `(320,-160,96)` |
| D[25]–D[27] | `0x00000320`, `0x00000280`, `0x00000460` | Old MAC `(800,640,1120)` |
| D[6] | `0x7fabcdef` | Output CODE `0x7f`; RGB bytes irrelevant |
| D[20]–D[22] | `0x11112222`, `0x33334444`, `0x55556666` | Old FIFO |

| Lane | Old MAC×4096 | IR0×IR | Full sum | New MAC/IR | Output byte |
|---|---|---|---|---|---|
| 1 / red | 3276800 | 655360 | 3932160 | 960 | 60 |
| 2 / green | 2621440 | −327680 | 2293760 | 560 | 35 |
| 3 / blue | 4587520 | 196608 | 4784128 | 1168 | 73 |

The negative input contribution lowers the second accumulated result without making its final value negative. Final MAC/IR read `0x000003c0`, `0x00000230`, `0x00000490`. FIFO becomes `0x33334444`, `0x55556666`, **`0x7f49233c`**; FLAG is zero, IR0 unchanged and CYCLES reads 5.

Issue GPL again **without** restoring IR: multiplicands are now `(960,560,1168)`, so new MAC/IR become `(1440,840,1752)` and RGB2=`0x7f6d345a`. If instead software restores original IR `(320,-160,96)` before that second command, MAC/IR become `(1120,480,1216)` and RGB2=`0x7f4c1e46`. This is operand lifetime, not a choice of accumulation mode.

Restore the original complete setup for each option comparison:

| Options / scalar | New signed MAC | New IR | RGB2 | FLAG |
|---|---|---|---|---|
| SF1/LM1, IR0=2048 | `(960,560,1168)` | Same | `0x7f49233c` | `0x00000000` |
| SF1/LM0, IR0=−2048 (`0xfffff800`) | `(640,720,1072)` | Same | `0x7f432d28` | `0x00000000` |
| SF0/LM0, IR0=2048 | (656160, −327040, 197728) | (32767, −32768, 32767) | `0x7fff00ff` | `0x81f80000` |
| SF0/LM1, IR0=2048 | Same MAC | `(32767,0,32767)` | `0x7fff00ff` | `0x81f80000` |

**Visible-word and signed44 boundaries.** Independently set MAC1=`0x7fffffff`, IR1=1, MAC2/MAC3/IR2/IR3=0 and CODE `0x7f`:

- With SF=0, LM=0 and IR0=1, full lane 1 sum is 2147483648, which fits signed44 but wraps to public MAC1=`0x80000000`. IR1 becomes −32768 and red byte clamps to 0; FLAG=`0x81200000` (ERROR, IR1 and red saturation). **No MAC1-positive44 overflow bit** is set.
- With SF=1, LM=0 and IR0=4096, rescaled old MAC1 is `2^43−4096`; adding 4096 crosses the signed44 limit. The wrapped/shifted public MAC1 is again `0x80000000`, IR1 again −32768, but FLAG=`0xc1200000` additionally records MAC1 positive overflow. RGB2 is `0x7f000000` in both cases.

The same public result can therefore have different overflow provenance. Neither case should be diagnosed from a raw MAC word alone.

**Timing.** GPL writes 5 to normal CYCLES. Results and the color push are available when its COMMAND write completes.

#### 4.19.3.8 VMAD3 — timed vector multiply-add through PLUS

VMAD3 computes a vector plus a scalar multiple of another vector. It uses the separate PLUS registerfile, pending calculation and published result set. Choose it for this three-lane arithmetic without matrix/projection or a color push. It is not issued through normal GTE COMMAND and does not use normal MAC, IR, FLAG or FIFO state.

**Encoding and access.** Write an aligned **direct CPU word store** to PLUS COMMAND at `0x080103a4`. Low bits7:0 must be `0x01`. Bits31:8 are ignored for arithmetic and remain in accepted COMMAND readback. There is no SF or LM option: normal GTE's bit19 and bit10 meanings do not apply here. A low byte other than01 is an invalid PLUS command, including values with low6 bits01 but bit6/7 set.

| PLUS index/address | Word | Representation/access |
|---|---|---|
| 0 / 08010384 | ADD_XY | Signed X low16, Y high16; raw-word read/write |
| 1 / 08010388 | ADD_Z | Signed Z low16, upper16 ignored by arithmetic; raw read/write |
| 2 / 0801038c | MUL_XY | Signed X low16, Y high16; raw read/write |
| 3 / 08010390 | MUL_Z | Signed Z low16, upper16 ignored; raw read/write |
| 4 / 08010394 | SCALAR | Shared signed low16, upper16 ignored; raw read/write |
| 5 / 08010398 | RESULT_XY | Published signed X/Y halves |
| 6 / 0801039c | RESULT_Z | Published signed Z low16; **upper16 zero**, even for negative Z |
| 7 / 080103a0 | FLAG | Published PLUS flags, not normal C[31] |
| 8 / 080103a4 | COMMAND | Last accepted raw word; direct CPU write admission interlocked |
| 9 / 080103a8 | CYCLES | Busy bit31 plus retained duration, not a countdown |

PLUS ignores writes to result, FLAG and CYCLES words. Results are replaced by the next completion or cleared by reset. Issue COMMAND with a direct CPU word store so that its busy interlock controls acceptance; DMA command writes are ignored.

**Calculation and scales.** At acceptance, snapshot the five current operand words, decode their signed components and calculate each lane:

```
full_i    = 4096*ADD_i + MUL_i*signed SCALAR
shifted_i = full_i ASR 12
result_i  = clamp(shifted_i,-32768,32767)
```

ASR rounds negative fractions towards minus infinity. These legal signed16 operands keep the full sum within signed32. There is no normal signed44 accumulator, MAC publication, selected shift or selected lower limit. A useful software convention uses A, M and result in the same integer coordinate counts and scalar/4096 as a dimensionless weight. The signed scalar supports negative weights too; it is not restricted to0…4096.

Pack the limited X/Y halves into pending RESULT_XY and Z's low half into pending RESULT_Z. A shifted lane strictly above32767 sets its positive flag and ERROR; a lane below−32768 sets its negative flag and ERROR. Exact integer endpoints are legal. This comparison occurs **after** the shift, not against an ideal fractional output.

| PLUS flag | Mask |
|---|---|
| ERROR | 80000000 |
| X+, Y+, Z+ saturation | 40000000, 20000000, 10000000 |
| X−, Y−, Z− saturation | 08000000, 04000000, 02000000 |
| Invalid command | 01000000 |

No other flag bits are set. Every VMAD3 starts a new **pending** flag result; the readable FLAG still belongs to the previous published job until completion.

**Pending versus published footprint.** Accepted VMAD3 starts a five-system-cycle interval and immediately exposes CYCLES=80000005. The input words and accepted COMMAND remain readable. Until completion, result words and FLAG retain the older published set. Input writes while busy prepare later work; they do not alter the captured operands. There is one pending job, not a command FIFO.

At completion, the two results and PLUS FLAG publish together; CYCLES clears busy and retains5. Any PLUS read, or an operand/COMMAND write, at or after the deadline synchronizes that completion; a waiting CPU command store also receives the ready edge. There is no completion interrupt or result-consumed acknowledgement. Individual result reads are not one multiword snapshot: another publication between reads can mix jobs. Retain every needed result and its flag before allowing the following job to replace them. Three scalar word loads alone have a six-cycle CPU base cost, longer than one pending five-cycle interval.

All normal GTE data/control words, MAC/IR/FLAG and color/screen/depth FIFOs are preserved. No automatic writeback into ADD, a normal vector slot, or a drawing packet occurs.

**Worked fractional displacement.** Supply these raw words:

```
ADD_XY=0xffec000a   ADD_Z=0x0000001e       // A=(10,-20,30)
MUL_XY=0x0005fffd   MUL_Z=0x0000fff9       // M=(-3,5,-7)
SCALAR=0x00000800                         // one half
```

Full sums are(34816,−71680,108544). After ASR12 the result is(8,−18,26), not (9,−17,27): each fractional result rounds towards negative infinity. Issue 00000001, or00080401 with ignored upper option bits, and after completion read RESULT_XY=ffee0008, RESULT_Z=0000001a, FLAG=0. Rewriting ADD_XY while the job is pending does not change this result.

For a rounding/clamp boundary, A=(32767,−32768,0), M=(1,−1,−1), scalar2048 produces fractional mathematical values(32767.5,−32768.5,−0.5). Their shifted integers are(32767,−32769,−1), so only Y saturates. Published words are 80007fff/0000ffff and FLAG=84000000. The positive half-count beyond32767 did not survive the shift; the negative half-count did cross the lower limit. The larger saturation case in Repeated PLUS work produces both X+ and Y−, FLAG=c4000000.

**Invalid command.** A low byte other than01 starts a one-cycle pending result, CYCLES=80000001. It retains the previous published result words as its pending output and publishes FLAG=81000000 at completion. It does not rerun VMAD3 with a fallback opcode, reset inputs, clear results or change normal GTE FLAG. Until that completion, the old published flag remains visible. Busy input and command-admission rules still apply.

**CPU issue and timing.** A direct CPU command store attempted while busy is blocked before acceptance. The old completion publishes and releases that instruction, which can retry and issue the next job. This prevents pending-work overwrite; it does not retain the old result for software.

For one concrete sequence with operands already supplied, two LOADK instructions cost two cycles, the first command store accepts at cycle4, and an immediately following store attempts at6 and blocks. The pending completion at9 releases it; the retried store accepts at11, and RET finishes at13. The second job is still busy until16. Those are ten charged CPU cycles, three elapsed wait cycles, and an outstanding result at return—not two jobs fully prepared and consumed in ten cycles. Preparation, polling/readout and result retention must be budgeted as well; the repeated-work example below counts the complete sequence.

## 4.20 Budgeting a complete geometry work interval

The following instruction schedule prepares one triangle for drawing. It sets geometry and lighting controls, transfers vertices and lighting directions from ROM, projects the triangle, tests winding and calculates vertex colors. It finishes with a six-word Gouraud packet, three saved FLAG words and the oriented area in RAM. GP0 submission and display waiting follow this interval.

Count from the first instruction of the entered function through its return. The operands match the frame example, with unrolled transfers and the register allocation below. Include any additional loops, address calculations or spills when applying this budget to another sequence.

### 4.20.1 State the operands and setup assumptions

The function is already entered and its cartridge ROM workset is selected; startup, cartridge selection and the caller's work are outside the interval. Reserve 40 bytes at the illustrative RAM output address for the six packet words, three FLAG words and oriented area; this is not GTE-owned scratch RAM. There are no pending interrupts, competing bus masters, PLUS operations, or faults. No GTE matrix, projection or lighting setup is assumed to be preloaded: this interval supplies all of it.

The workset is stored in cartridge slot 0 ROM at `0x10001000`, not in the BIOS ROM. It contains 38 whole words, 152 bytes:

| ROM word indices | Contents | Destination |
|---|---|---|
| 0–7 | Q12 identity rotation and zero translation | C[0]–C[7] |
| 8–12 | OFX=`0x00800000`, OFY=`0x00600000`, H=256, DQA=0, DQB=0 | C[24]–C[28] |
| 13–17 | Q12 identity light matrix | C[8]–C[12] |
| 18–22 | Q12 identity color matrix | C[16]–C[20] |
| 23–25 | Zero ambient vector | C[13]–C[15] |
| 26–31 | Positions `(-64,-64,512)`, `(64,-64,512)`, `(0,64,512)` | D[0]–D[5] |
| 32–37 | Supplied directions `(4096,0,0)`, `(0,4096,0)`, `(0,0,4096)` | D[0]–D[5], after geometry |

These directions deliberately produce different vertex colors; they are not normals automatically derived from the planar triangle. RGBC is `0x004080a0`. The selected normal commands are RTPT=`0x00080030`, NCLIP=`0x00000006`, and NCCT=`0x0008043f`.

Twelve LOADK instructions establish all function constants:

| Registers | Values |
|---|---|
| r0–r3 | Cartridge-ROM workset address `0x10001000`; control base `0x080102c4`; data base `0x08010244`; RAM output address `0x080c1000` |
| r4–r7 | RTPT word; NCLIP word; NCCT word; RGBC word |
| r8–r11 | Packet tag `0x30000000`; RGB mask `0x00ffffff`; sign mask `0x80000000`; zero |

Every subsequent memory operation uses the scalar `LOAD_MEM_D` or `STORE_MEM_D`, Word access, with address `base + 4*displacement`. For example, ROM position word 26 is one displaced load from r0; its destination D[0] is one displaced store through r2. The address addition belongs to that instruction; there is no hidden separate ADD in this specified sequence. Conversely, these counts must not be applied to an instruction sequence with additional address arithmetic, loop tests, spills, or calls.

### 4.20.2 Count the complete accepted-triangle path

The current scalar ISA charges two base cycles per displaced word load/store, one per LOADK, BAND, BOR or EQ, and two for RET. The function's accepted path executes:

| Work | Executed instructions | Charged CPU cycles |
|---|---:|---:|
| Load the twelve addresses/constants | 12 LOADK | 12 |
| Copy all 26 control words from ROM | 26 loads + 26 stores | 104 |
| Copy six position words from ROM | 6 loads + 6 stores | 24 |
| Issue RTPT; retain projection FLAG in RAM; read three screen pairs | 1 command store + 1 flag load/store pair + 3 loads | 12 |
| Issue NCLIP; retain area and its FLAG in RAM | 1 command store + 2 load/store pairs | 10 |
| Select nonzero positive raw area | 2 EQ + 1 BAND | 3 |
| Copy six supplied lighting-direction words from ROM | 6 loads + 6 stores | 24 |
| Supply RGBC | 1 store | 2 |
| Issue NCCT and retain its FLAG in RAM | 1 command store + 1 flag load/store pair | 6 |
| Read/mask three colors, add the first-word opcode, write six packet words | 3 loads + 3 BAND + 1 BOR + 6 stores | 22 |
| Return | 1 RET | 2 |
| **Complete interval** | **120 instructions** | **221 cycles** |

The area selection is real control flow, not a free assumption. An EQ test rejects zero; BAND extracts the sign bit; a second EQ rejects negative area. Each test is followed by a JMP to the return. For this positive-area input the two JMPs are skipped rather than issued. The program has 122 stored instructions, but only 120 execute on the accepted path. A rejected path has a different budget and does not prepare the six packet words.

The three screen pairs are read once into r13–r15 before lighting replaces the input vectors and arithmetic results. A temporary register carries each retained FLAG into RAM; the oriented area is likewise retained before later arithmetic. The color reads go to r19–r21, are masked to RGB24, and receive the Gouraud tag only in the first packet word. No input transfer, result preservation, mask or packet RAM store is omitted from the table.

An independent count is 100 scalar memory instructions at two cycles each, 12 LOADK at one cycle each, seven bit-wise/comparison instructions at one cycle each, and RET at two cycles: `200+12+7+2 = 221`. The resulting six RAM words are:

```
0x300000a0  0x00400060
0x00008000  0x004000a0
0x00400000  0x00800080
```

The three retained FLAG words are zero and the retained doubled area is 4096. Thus the 221-cycle schedule prepares the stated geometry/color packet, including its retained arithmetic results.

RTPT, NCLIP and NCCT report nominal counts 23, 8 and 39. Their arithmetic completes during the issuing register writes, whose two-cycle CPU base costs are already included in 221. The final CYCLES readback is 39; retain the full instruction budget when calculating this work interval.

### 4.20.3 Keep frame waits outside the fixed preparation number

This interval ends with a packet in RAM. Sending its words to GP0, establishing GPU drawing state, clearing a buffer, waiting for GPU completion, publishing the display source, servicing interrupts and observing a later VBlank are additional work. None has been assigned an invented fixed cost here.

Separate the following when making a frame budget:

- **CPU issue work:** the actual instruction sequence used to read a prepared packet and submit it, including addresses, loops and stores. A RAM packet is not an automatic transfer to GPU.
- **GPU-completion observation:** a polling loop has a measured cost per issued iteration, multiplied by a state-dependent number of iterations. GPU work may already overlap earlier CPU work; its nominal drawing work cannot simply be added again as if it always began at the first poll.
- **Display-edge waiting:** time halted or polling for the required VBlank edge, plus the actual interrupt-handler and counter-check instructions. The wait depends on the scanout phase and on when the counter was sampled. It is not a constant that can be inferred from this GTE work unit.

Report charged CPU work and elapsed device/scheduler waits separately, then construct the actual timeline if their overlap matters. The cartridge's actual loops, changing translation, GP0 traffic and frame-publication path require their own accounting; **221 is not their full-frame BLua budget**. This cold work unit also includes control setup that a cartridge can retain across many triangles or frames. Reuse changes the work interval and its count; it does not retroactively turn a lower-bound transfer estimate into a measured program budget.

## 4.21 Timing and bus programming

### 4.21.1 Normal commands: nominal cost and CPU cost

A normal COMMAND write performs the complete arithmetic operation during that write. A following data/control read sees its results without polling. Normal CYCLES records the last nominal command count; it has no busy bit and does not count down. For example, RTPS reports 15, MVMVA reports 8, and RTPT reports 23. These values do not cause waits of 15, 8 or 23 system cycles.

An admitted direct CPU word-store instruction has a two-system-cycle base cost, whether it writes an ordinary operand word or normal COMMAND. Normal execution does not add the reported GTE CYCLES count to that cost. Address and operand preparation, subsequent result loads, and other CPU instructions have their own costs. Count these instructions and any actual device waits when budgeting a complete operation.

An unrecognized normal function performs no arithmetic, leaves normal FLAG unchanged and reports zero CYCLES. The raw command word is still retained for COMMAND readback. Zero CYCLES does not mean that the issuing CPU instruction itself was free.

### 4.21.2 PLUS: timed publication and store interlock

PLUS measures its completion interval in system cycles. At the installed 33,868,800 Hz frequency:

| Interval | Time |
|---|---|
| One system cycle | Approximately 29.526 ns |
| VMAD3 completion interval: five cycles | Approximately 147.628 ns |
| Invalid-command completion interval: one cycle | Approximately 29.526 ns |

An accepted command takes all operands from the current PLUS input words and forms one pending result set. Before its completion deadline, RESULT_XY, RESULT_Z and PLUS FLAG retain the previously published values. PLUS CYCLES reads `0x80000005` for pending VMAD3 or `0x80000001` for a pending invalid command. The low 31 bits retain the nominal interval; they are not the remaining time.

At or after the deadline, a PLUS read, accepted input/command write, or CPU command-store readiness check observes completion: both results and PLUS FLAG publish together, and busy clears. CYCLES then reads `0x00000005` or `0x00000001`. A CPU store already waiting on PLUS COMMAND is released at the completion deadline. Normal GTE accesses do not wait for or synchronize PLUS.

PLUS inputs may be rewritten while busy. Such writes prepare later work; they do not modify the operands already taken by the pending command. There is no input queue. Write the intended inputs before issuing each command. A direct 32-bit CPU store to PLUS COMMAND waits if busy, then issues the new command; this interlock governs command acceptance, not the readout of an old result. Read any result that must be retained before the next completion replaces it.

DMA cannot issue PLUS commands. A DMA write to PLUS COMMAND is denied even when the unit is idle and changes neither the command nor its operands or results. This restriction does not specify the DMA channel's completion or fault policy.

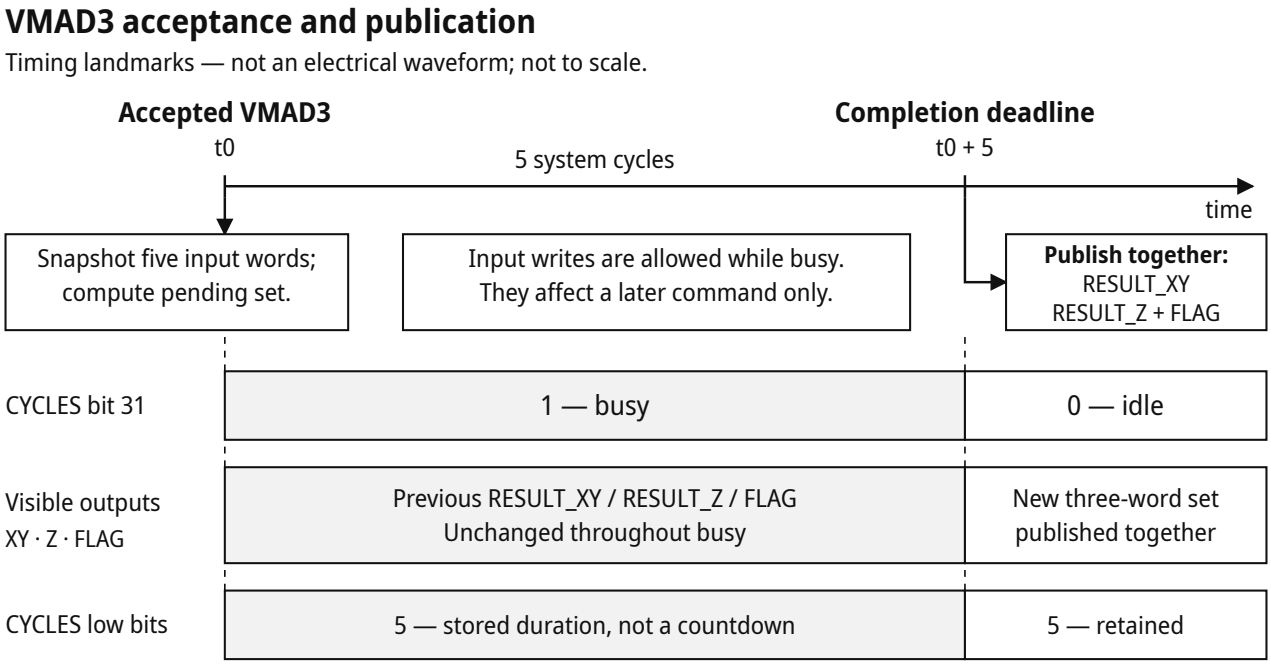



Figure 4.4: PLUS command admission and result publication

### 4.21.3 Register access and bus faults

Use aligned 32-bit word accesses to program the register windows. CPU word, U32, F32 and F64 accesses require four-byte alignment; U16 requires two-byte alignment. A CPU alignment rejection raises an address-error load or store condition, records the bad address, and does not reach GTE.

At the mapped IO interface, halfword reads and writes, byte writes, and accesses between word slots report `BUS_FAULT_UNALIGNED_IO`. Only a byte read at a word-aligned address is supported; it returns the low eight bits of that register's read value. A new mapped-memory bus fault raises the CPU data-bus-error condition; a failed CPU load leaves its destination unchanged. This is distinct from rejection by the CPU's initial alignment check.

F32 accesses interpret one raw word as floating-point bits; they do not select floating-point GTE arithmetic. F64 accesses interpret two words, low word at A followed by high word at A+4. They require four-byte, not eight-byte, alignment and perform two ordered register-word accesses. A later fault does not undo an earlier word's effects. Scalar-store readiness checks only A: an F64 store starting at PLUS FLAG (`0x080103a0`) can reach PLUS COMMAND as its second word without that address's store interlock. If PLUS is still busy and not due, that command word is ignored, rather than waiting; if idle after synchronization, it is accepted. Use a direct word store for the command protocol.

CPU word-sequence stores instead check every addressed IO word's readiness before writing any sequence word. Once admitted, they write in increasing four-byte addresses and stop at the first bus fault; earlier writes remain effective. There is no GTE-specific user/supervisor or enable-bit prerequisite in this mapped access protocol.

Saturation, arithmetic overflow and invalid commands use the flag behavior described in the arithmetic and register sections; they are not alignment faults. Writes documented as having no GTE effect do not become read-only bus faults.

### 4.21.4 Interrupts and reset

Neither normal completion nor PLUS completion signals a GTE interrupt. There are no GTE interrupt-enable, pending or acknowledgement registers. PLUS completion makes results visible and releases a waiting command store; it is not an IRQ acknowledgement protocol.

Device reset clears all register words, accumulators and cycle counts to zero and discards pending PLUS work. It also releases a CPU store waiting on PLUS COMMAND. This release is not publication of the discarded result. There is no separate mapped GTE reset register.

## 4.22 Register reference

All addresses below are byte addresses. Each word occupies four bytes:

- `D[n] = 0x08010244 + 4n`, for `n=0–31`.
- `C[n] = 0x080102c4 + 4n`, for `n=0–31`.
- `PLUS[n] = 0x08010384 + 4n`, for `n=0–9`.

A register word is a raw 32-bit pattern. Signed interpretation occurs at the arithmetic input, not by changing the word's representation. In the tables, **low** means bits 15:0 and **high** means bits 31:16. Matrix rows and columns are numbered 0–2.

| Access notation | Read and mapped-write operation |
|---|---|
| RW | Read and write the complete word. |
| S16 | Writes sign-extend the low halfword; reads return that 32-bit word. |
| U16 | Writes keep only bits 15:0; reads return those bits with a zero upper halfword. |
| Push | Read the newest screen pair; write pushes a screen pair. |
| Derived | Read a computed word; see the entry for its write behavior. |
| Output | Read the result; a write has no GTE effect and does not fault as a read-only write. |

### 4.22.1 Data words

#### 4.22.1.1 D[0]–D[7] — vectors, input color, and depth average

| Word | Bits | Field |
|---|---|---|
| Vector | 31:16 | Y, signed 16-bit |
| Vector | 15:0 | X, signed 16-bit |

| Word | Bits | Field |
|---|---|---|
| D[6] | 31:24 | CODE |
| D[6] | 23:16 | Blue |
| D[6] | 15:8 | Green |
| D[6] | 7:0 | Red |

| Word | Address | Function and fields | Access | Reset |
|---|---|---|---|---|
| D[0] | `0x08010244` | V0.X low / V0.Y high | RW | `0x00000000` |
| D[1] | `0x08010248` | V0.Z low; high is sign extension | S16 | `0x00000000` |
| D[2] | `0x0801024c` | V1.X low / V1.Y high | RW | `0x00000000` |
| D[3] | `0x08010250` | V1.Z low; high is sign extension | S16 | `0x00000000` |

| Word | Address | Function and fields | Access | Reset |
|---|---|---|---|---|
| D[4] | `0x08010254` | V2.X low / V2.Y high | RW | `0x00000000` |
| D[5] | `0x08010258` | V2.Z low; high is sign extension | S16 | `0x00000000` |
| D[6] | `0x0801025c` | RGBC input: unsigned RGB bytes and CODE | RW | `0x00000000` |
| D[7] | `0x08010260` | Depth-average output, low unsigned 16-bit; high zero | U16 | `0x00000000` |

#### 4.22.1.2 D[8]–D[15] — intermediate results and screen FIFO

| Word | Bits | Field |
|---|---|---|
| IR | 31:16 | Sign extension |
| IR | 15:0 | Signed intermediate result |

| Word | Bits | Field |
|---|---|---|
| Screen | 31:16 | SY, signed 16-bit |
| Screen | 15:0 | SX, signed 16-bit |

| Word | Address | Function | Access | Reset |
|---|---|---|---|---|
| D[8] | `0x08010264` | IR0 | S16 | `0x00000000` |
| D[9] | `0x08010268` | IR1 | S16 | `0x00000000` |
| D[10] | `0x0801026c` | IR2 | S16 | `0x00000000` |
| D[11] | `0x08010270` | IR3 | S16 | `0x00000000` |
| D[12] | `0x08010274` | Screen FIFO pair SX0/SY0, oldest | RW | `0x00000000` |
| D[13] | `0x08010278` | Screen FIFO pair SX1/SY1 | RW | `0x00000000` |
| D[14] | `0x0801027c` | Screen FIFO pair SX2/SY2, newest | RW | `0x00000000` |
| D[15] | `0x08010280` | Screen read alias / push port | Push | `0x00000000` |

Reading D[15] returns D[14]. Writing a word `w` to D[15] performs `D[12]←old D[13]`, `D[13]←old D[14]`, `D[14]←w`. Writing D[12], D[13], or D[14] changes only that entry.

#### 4.22.1.3 D[16]–D[23] — depth and color FIFOs

| Word | Bits | Field |
|---|---|---|
| Depth | 31:16 | Zero |
| Depth | 15:0 | Unsigned depth |

| Word | Bits | Field |
|---|---|---|
| RGB | 31:24 | CODE |
| RGB | 23:16 | Blue |
| RGB | 15:8 | Green |
| RGB | 7:0 | Red |

| Word | Address | Function | Access | Reset |
|---|---|---|---|---|
| D[16] | `0x08010284` | SZ0, oldest depth | U16 | `0x00000000` |
| D[17] | `0x08010288` | SZ1 | U16 | `0x00000000` |
| D[18] | `0x0801028c` | SZ2 | U16 | `0x00000000` |
| D[19] | `0x08010290` | SZ3, newest depth | U16 | `0x00000000` |
| D[20] | `0x08010294` | RGB0, oldest color | RW | `0x00000000` |
| D[21] | `0x08010298` | RGB1 | RW | `0x00000000` |
| D[22] | `0x0801029c` | RGB2, newest color | RW | `0x00000000` |
| D[23] | `0x080102a0` | Unassigned storage, bits 31:0 retained | RW | `0x00000000` |

Depth and color pushes are defined with the arithmetic stages below. Direct writes to these FIFO entries do not push the other entries.

#### 4.22.1.4 D[24]–D[31] — MAC words, RGB5 conversion, and leading-sign count

| Word | Bits | Field |
|---|---|---|
| RGB5 | 31:15 | Zero |
| RGB5 | 14:10 | Blue |
| RGB5 | 9:5 | Green |
| RGB5 | 4:0 | Red |

| Word | Address | Function and fields | Access | Reset |
|---|---|---|---|---|
| D[24] | `0x080102a4` | MAC0 visible word, bits 31:0 | RW | `0x00000000` |
| D[25] | `0x080102a8` | MAC1 visible word, bits 31:0 | RW | `0x00000000` |
| D[26] | `0x080102ac` | MAC2 visible word, bits 31:0 | RW | `0x00000000` |
| D[27] | `0x080102b0` | MAC3 visible word, bits 31:0 | RW | `0x00000000` |
| D[28] | `0x080102b4` | RGB5 input / packed IR output | Derived | `0x00000000` |
| D[29] | `0x080102b8` | Packed IR output alias | Derived | `0x00000000` |
| D[30] | `0x080102bc` | Leading-sign-bit source, bits 31:0 | RW | `0x00000000` |
| D[31] | `0x080102c0` | Leading-sign-bit count, bits 31:0 | Output | `0x00000000` |

Both D[28] and D[29] read `R | (G<<5) | (B<<10)`, where each channel is the corresponding signed IR1–IR3 value arithmetic-shifted right seven and clamped to 0–31. This read does not set flags. Writing D[28] takes bits 4:0, 9:5, and 14:10, multiplies each by 128, and writes IR1, IR2, and IR3 respectively; bits 31:15 do not contribute. Writing D[29] or D[31] has no GTE effect.

Writing D[30] also updates D[31] to the number of leading bits equal to bit 31: leading zeroes for a nonnegative source, or leading ones for a negative source. The result is 32 for both `0x00000000` and `0xffffffff`.

Direct MAC-word writes alter the visible words, not the full arithmetic accumulators. GPL explicitly uses these visible words as its old-MAC operands.

#### 4.22.1.5 Conversion and leading-sign examples

Writing `0xfffffc1f` to D[28] supplies five-bit components `(31,0,31)`; bits 31:15 are ignored. IR1–IR3 become `(3968,0,3968)`, each component multiplied by 128. Both D[28] and D[29] then read `0x00007c1f`.

Now write IR1=`0xffffff80` (−128), IR2=`0x00000880` (2176) and IR3=`0x00001000` (4096), without another D[28] write:

| Channel | Signed IR | IR ASR 7 | Limited five-bit value |
|---|---|---|---|
| R | −128 | −1 | 0 |
| G | 2176 | 17 | 17 |
| B | 4096 | 32 | 31 |

Both packed ports now read `0x00007e20`, not the previously written color. The reads derive from current IR; they neither change IR nor set FLAG, even when conversion limits a channel. Retain a packed result before another calculation changes its IR source. This conversion does not push the RGB FIFO.

The leading-sign ports can expose the headroom in a raw signed word without issuing an arithmetic command:

| Write to D[30] at `0x080102bc` | Matching leading bits | Read D[31] at `0x080102c0` |
|---|---|---|
| `0x00000000` | 32 zeroes | 32 |
| `0xffffffff` | 32 ones | 32 |
| `0x00000f00` | 20 zeroes | 20 |
| `0xfff00000` | 12 ones | 12 |
| `0x80000000` | One one | 1 |
| `0x7fffffff` | One zero | 1 |

The count includes the sign bit. For a nonzero signed32 source, count minus one gives the number of successive doublings whose exact result still fits signed32; zero remains zero. This is a software use of the count, not an automatic shift or normalization operation. Neither port changes FLAG.

### 4.22.2 Control words

For matrix selectors 0–2, nine signed 16-bit coefficients occupy five words in row-major order. The last coefficient is sign-extended on write. Each following three-word control vector contains signed 32-bit elements.

| Word | Slot | Bits | Field |
|---|---|---|---|
| Matrix | Base+0 | 31:16 | m[0,1] |
| Matrix | Base+0 | 15:0 | m[0,0] |
| Matrix | Base+1 | 31:16 | m[1,0] |
| Matrix | Base+1 | 15:0 | m[0,2] |
| Matrix | Base+2 | 31:16 | m[1,2] |
| Matrix | Base+2 | 15:0 | m[1,1] |
| Matrix | Base+3 | 31:16 | m[2,1] |
| Matrix | Base+3 | 15:0 | m[2,0] |
| Matrix | Base+4 | 31:16 | Sign extension |
| Matrix | Base+4 | 15:0 | m[2,2] |

“Base +” in this layout counts control words, not bytes. The bases are C[0], C[8], and C[16].

#### 4.22.2.1 C[0]–C[7] — rotation matrix and translation

| Word | Address | Function and fields | Access | Reset |
|---|---|---|---|---|
| C[0] | `0x080102c4` | Matrix 0: m[0,0] low / m[0,1] high | RW | `0x00000000` |
| C[1] | `0x080102c8` | Matrix 0: m[0,2] low / m[1,0] high | RW | `0x00000000` |
| C[2] | `0x080102cc` | Matrix 0: m[1,1] low / m[1,2] high | RW | `0x00000000` |
| C[3] | `0x080102d0` | Matrix 0: m[2,0] low / m[2,1] high | RW | `0x00000000` |
| C[4] | `0x080102d4` | Matrix 0: m[2,2] low | S16 | `0x00000000` |

| Word | Address | Function and fields | Access | Reset |
|---|---|---|---|---|
| C[5] | 0x080102d8 | Control vector 0, X; signed bits 31:0 | RW | 0x00000000 |
| C[6] | 0x080102dc | Control vector 0, Y; signed bits 31:0 | RW | 0x00000000 |
| C[7] | 0x080102e0 | Control vector 0, Z; signed bits 31:0 | RW | 0x00000000 |

Projection uses matrix 0 for rotation and control vector 0 for translation.

#### 4.22.2.2 C[8]–C[15] — light matrix and control vector 1

| Word | Address | Function and fields | Access | Reset |
|---|---|---|---|---|
| C[8] | 0x080102e4 | Matrix 1: m[0,0] low / m[0,1] high | RW | 0x00000000 |
| C[9] | 0x080102e8 | Matrix 1: m[0,2] low / m[1,0] high | RW | 0x00000000 |
| C[10] | 0x080102ec | Matrix 1: m[1,1] low / m[1,2] high | RW | 0x00000000 |
| C[11] | 0x080102f0 | Matrix 1: m[2,0] low / m[2,1] high | RW | 0x00000000 |
| C[12] | 0x080102f4 | Matrix 1: m[2,2] low | S16 | 0x00000000 |
| C[13] | 0x080102f8 | Control vector 1, X; signed bits 31:0 | RW | 0x00000000 |
| C[14] | 0x080102fc | Control vector 1, Y; signed bits 31:0 | RW | 0x00000000 |
| C[15] | 0x08010300 | Control vector 1, Z; signed bits 31:0 | RW | 0x00000000 |

The lighting stage uses matrix 1. The color-matrix stage adds control vector 1.

#### 4.22.2.3 C[16]–C[23] — color matrix and far-color vector

| Word | Address | Function and fields | Access | Reset |
|---|---|---|---|---|
| C[16] | 0x08010304 | Matrix 2: m[0,0] low / m[0,1] high | RW | 0x00000000 |
| C[17] | 0x08010308 | Matrix 2: m[0,2] low / m[1,0] high | RW | 0x00000000 |
| C[18] | 0x0801030c | Matrix 2: m[1,1] low / m[1,2] high | RW | 0x00000000 |
| C[19] | 0x08010310 | Matrix 2: m[2,0] low / m[2,1] high | RW | 0x00000000 |
| C[20] | 0x08010314 | Matrix 2: m[2,2] low | S16 | 0x00000000 |
| C[21] | 0x08010318 | Control vector 2 / far red; signed bits 31:0 | RW | 0x00000000 |
| C[22] | 0x0801031c | Control vector 2 / far green; signed bits 31:0 | RW | 0x00000000 |
| C[23] | 0x08010320 | Control vector 2 / far blue; signed bits 31:0 | RW | 0x00000000 |

#### 4.22.2.4 C[24]–C[30] — projection and depth coefficients

| Word | Bits | Field |
|---|---|---|
| Coeff. | 31:16 | Sign extension |
| Coeff. | 15:0 | Coefficient |

| Word | Address | Function and fields | Access | Reset |
|---|---|---|---|---|
| C[24] | `0x08010324` | OFX, signed 32-bit screen X offset | RW | `0x00000000` |
| C[25] | `0x08010328` | OFY, signed 32-bit screen Y offset | RW | `0x00000000` |
| C[26] | `0x0801032c` | H, unsigned low 16-bit projection numerator | S16 | `0x00000000` |
| C[27] | `0x08010330` | DQA, signed low 16-bit depth-cue coefficient | S16 | `0x00000000` |
| C[28] | `0x08010334` | DQB, signed 32-bit depth-cue offset | RW | `0x00000000` |
| C[29] | `0x08010338` | ZSF3, signed low 16-bit depth scale | S16 | `0x00000000` |
| C[30] | `0x0801033c` | ZSF4, signed low 16-bit depth scale | S16 | `0x00000000` |

H has a deliberate read/datapath distinction: a write of `0x00008000` reads back as `0xffff8000`, but projection consumes the unsigned numerator 32768. OFX/OFY are added before the screen calculation’s 16-bit right shift; DQB is added before the depth-cue calculation’s 12-bit right shift.

#### 4.22.2.5 C[31] — normal FLAG

| Word | Address | Access | Reset |
|---|---|---|---|
| C[31] — FLAG | `0x08010340` | Read / masked write | `0x00000000` |

| Word | Bits | Field |
|---|---|---|
| FLAG | 31 | ERROR |
| FLAG | 30 | M1+ |
| FLAG | 29 | M2+ |
| FLAG | 28 | M3+ |
| FLAG | 27 | M1− |
| FLAG | 26 | M2− |
| FLAG | 25 | M3− |
| FLAG | 24 | I1 |
| FLAG | 23 | I2 |
| FLAG | 22 | I3 |

| Word | Bits | Field |
|---|---|---|
| FLAG | 21 | R |
| FLAG | 20 | G |
| FLAG | 19 | B |
| FLAG | 18 | SZ |
| FLAG | 17 | DIV |
| FLAG | 16 | M0+ |
| FLAG | 15 | M0− |
| FLAG | 14 | SX |

| Word | Bits | Field |
|---|---|---|
| FLAG | 13 | SY |
| FLAG | 12 | I0 |
| FLAG | 11:0 | Zero |

M1–M3 indicate positive/negative 44-bit MAC overflow; I1–I3 indicate IR saturation. R/G/B indicate color-output saturation; SZ indicates depth or depth-average saturation; DIV indicates the projection divide overflow condition. M0+/M0− indicate signed 32-bit MAC0 overflow; SX/SY indicate screen-coordinate saturation; I0 indicates IR0 saturation. Exact arithmetic conditions are given with the stages below.

A write keeps `word & 0x7ffff000`, then sets ERROR if any bit selected by `0x7f87e000` is set. Bit 31 is therefore recomputed, not directly writable. Bits 11:0 read zero. This is not write-one-to-clear behavior.

Every recognized normal command clears FLAG before arithmetic and forms ERROR afterward. Triple operations accumulate flags across all three iterations. ERROR summarizes bits 30:23 and 18:13; IR3, color-channel, and IR0 saturation alone do not set ERROR. An unrecognized normal function leaves FLAG unchanged.

### 4.22.3 Normal COMMAND and CYCLES

| Register | Address | Access | Reset |
|---|---|---|---|
| COMMAND | `0x08010344` | Write executes; read last raw command write | `0x00000000` |
| CYCLES | `0x08010348` | Read last nominal duration; write has no GTE effect | `0x00000000` |

| Word | Bits | Field |
|---|---|---|
| COMMAND | 31:20 | Ignored |
| COMMAND | 19 | SF |
| COMMAND | 18:17 | mx |
| COMMAND | 16:15 | v |
| COMMAND | 14:13 | cv |
| COMMAND | 12:11 | Ignored |
| COMMAND | 10 | LM |
| COMMAND | 9:6 | Ignored |
| COMMAND | 5:0 | Function |

| Word | Bits | Field |
|---|---|---|
| CYCLES | 31:0 | Last normal-command nominal cycle count |

Only MVMVA uses mx, v, and cv. SF selects a 12-bit result shift when one, or no shift when zero. LM selects the lower IR1–IR3 clamp bound: zero when one, −32768 when zero; the upper bound is always 32767. NCLIP, AVSZ3, and AVSZ4 use neither SF nor LM. Other bits are ignored for execution but remain in the COMMAND readback word.

An unknown function executes no arithmetic, leaves data/control and accumulator values unchanged, and sets normal CYCLES to zero. It still becomes the last COMMAND readback word.

### 4.22.4 GTE_PLUS words

#### 4.22.4.1 PLUS[0]–PLUS[4] — multiply-add operands

| Word | Bits | Field |
|---|---|---|
| XY | 31:16 | Y, signed 16-bit |

| Word | Bits | Field |
|---|---|---|
| XY | 15:0 | X, signed 16-bit |

| Word | Bits | Field |
|---|---|---|
| Z/scalar | 31:16 | Retained, ignored by VMAD3 |
| Z/scalar | 15:0 | Signed 16-bit operand |

| Word / name | Address | Function | Access | Reset |
|---|---|---|---|---|
| PLUS[0] — ADD_XY | `0x08010384` | Addend X/Y | RW | `0x00000000` |
| PLUS[1] — ADD_Z | `0x08010388` | Addend Z | RW | `0x00000000` |
| PLUS[2] — MUL_XY | `0x0801038c` | Multiplicand X/Y | RW | `0x00000000` |
| PLUS[3] — MUL_Z | `0x08010390` | Multiplicand Z | RW | `0x00000000` |
| PLUS[4] — SCALAR | `0x08010394` | Shared scalar | RW | `0x00000000` |

#### 4.22.4.2 PLUS[5]–PLUS[9] — published results and command status

| Word / name | Address | Access | Reset |
|---|---|---|---|
| PLUS[5] — RESULT_XY | `0x08010398` | Output | `0x00000000` |
| PLUS[6] — RESULT_Z | `0x0801039c` | Output | `0x00000000` |
| PLUS[7] — FLAG | `0x080103a0` | Output | `0x00000000` |
| PLUS[8] — COMMAND | `0x080103a4` | Read last accepted raw word; direct CPU word write starts when idle | `0x00000000` |
| PLUS[9] — CYCLES | `0x080103a8` | Output | `0x00000000` |

| Word | Bits | Field |
|---|---|---|
| RESULT_XY | 31:16 | Result Y, signed 16-bit |
| RESULT_XY | 15:0 | Result X, signed 16-bit |

| Word | Bits | Field |
|---|---|---|
| RESULT_Z (VMAD3) | 31:16 | Zero |
| RESULT_Z (VMAD3) | 15:0 | Result Z, signed 16-bit |

| Word | Bits | Field |
|---|---|---|
| COMMAND | 31:8 | Retained, ignored for selection |
| COMMAND | 7:0 | Function: 0x01 = VMAD3; all others invalid |

| Word | Bits | Field |
|---|---|---|
| CYCLES | 31 | Busy |
| CYCLES | 30:0 | Nominal duration: 5 for VMAD3; 1 for invalid command |

Busy clears when the pending results and flag are published together. The nominal duration remains. RESULT_Z has a zero upper halfword even when its signed low-halfword result is negative.

#### 4.22.4.3 PLUS FLAG fields

| Word | Bits | Field |
|---|---|---|
| PLUS FLAG | 31 | ERROR |
| PLUS FLAG | 30 | X+ |
| PLUS FLAG | 29 | Y+ |
| PLUS FLAG | 28 | Z+ |
| PLUS FLAG | 27 | X− |
| PLUS FLAG | 26 | Y− |
| PLUS FLAG | 25 | Z− |
| PLUS FLAG | 24 | INVALID |
| PLUS FLAG | 23:0 | Zero |

These flags are independent of C[31]. A valid command starts with a pending flag word of zero; strict lane overflow or underflow sets the corresponding direction bit and ERROR. An invalid command publishes `0x81000000` (ERROR and INVALID) and preserves both previously published result words. It does not execute VMAD3 or raise an exception.

## 4.23 Normal-command arithmetic

The following definitions specify the integer datapath. Multiplication and addition use mathematical integers except where a word wrap or explicit shift is specified. `s16lo` and `s16hi` interpret the low and high halfwords as signed; `u16lo` is the unsigned low halfword; `s32` interprets a complete word as signed. `U32(x)` keeps its low 32 bits as unsigned, while `W32(x)` keeps them as signed. `ASR(x,n)` divides by `2^n`, rounding toward negative infinity. `clamp(x,a,b)` limits x to the inclusive interval a–b.

Matrix rows r and columns c run from 0 to 2; MAC/IR lanes i run from 1 to 3. Row r produces lane `i=r+1`, writing MAC at D[25+r] and IR at D[9+r]. Control-vector and translation components `CV_r` and `TR_r` use X/Y/Z order. Lane-indexed color components use lane 1 for red, 2 for green, and 3 for blue.

### 4.23.1 MAC, shift, and IR stages

The three main accumulators have the signed 44-bit range −2^43 to 2^43−1. Wrapping is not saturation:

```
MIN44 = -0x80000000000       MAX44 = 0x7fffffffff
W44(x) = x - 2^44  if x > MAX44
         x + 2^44  if x < MIN44
         x         otherwise
S(x)   = ASR(x,12) if SF=1; x if SF=0
```

`M_i(a,p,n)`, for lane i=1–3, retains the unshifted accumulator a and writes `D[24+i]=W32(S(a))`. It sets the lane's positive/negative MAC flags when p/n are true. Those flags also set ERROR. The return value is the signed visible MAC word, not a.

`M44_i(x)` first wraps x with W44 and then performs M_i, using `x>MAX44` and `x<MIN44` as its overflow conditions. Dot products instead use the following ordered accumulation; its flags cannot be replaced by checking only the final sum:

```
A44(initial; b0,b1,b2):
  a = W44(initial)
  positive = (initial > MAX44); negative = (initial < MIN44)
  for b in b0,b1,b2, in that order:
    t = W44(a+b)
    positive |= (t<0 and a>=0 and b>=0)
    negative |= (t>=0 and a<0 and b<0)
    a = t
  deliver a and the accumulated positive/negative conditions to M_i
```

Thus cancellation after overflow does not erase its flag. The ordered wrap applies after the initial term and after each addend.

`I_i(q,LM)` clamps the signed visible MAC result q to −32768…32767 for LM=0 or 0…32767 for LM=1. A strictly out-of-range q sets that IR lane's saturation flag. Unless a stage explicitly says otherwise, this clamped value is written to D[9]–D[11]. The clamp consumes the shifted, 32-bit-wrapped word, not the full 44-bit value.

`M0(x)` retains full x and writes its low 32 bits to D[24]. It sets M0+ and ERROR for `x>2147483647`, or M0− and ERROR for `x<−2147483648`. Projection and depth-average calculations that follow M0 use full x, not the wrapped word.

### 4.23.2 Operand selection

MVMVA's selectors have these complete encodings:

| Selector | 0 | 1 | 2 | 3 |
|---|---|---|---|---|
| mx | Matrix 0, C[0]–C[4] | Matrix 1, C[8]–C[12] | Matrix 2, C[16]–C[20] | Synthetic matrix below |
| v | V0, D[0]/D[1] | V1, D[2]/D[3] | V2, D[4]/D[5] | Current signed IR1–IR3 |
| cv | Signed C[5]–C[7] | Signed C[13]–C[15] | Signed C[21]–C[23] | (0,0,0) |

The synthetic matrix for mx=3 is defined, not rejected. With `r` the unsigned red byte of D[6], `a=s16lo(C[1])`, and `b=s16lo(C[2])`, it is:

```
[ -16r   +16r   s16lo(D[8]) ]
[   a      a         a      ]
[   b      b         b      ]
```

For V0–V2, X and Y are the signed low/high halves of D[2v], and Z is the signed low half of D[2v+1]. IR0 is also signed at arithmetic inputs. RGBC color components are unsigned bytes.

### 4.23.3 Shared matrix and color stages

The exact command calculations use these shared stages. Each stage uses the selected SF and LM, except for explicitly forced LM=0 clamps.

**Matrix stage.** Snapshot all three input components `(x,y,z)`. For each row r=0–2 of matrix m, let `i=r+1` and form `A44(4096×CV_r; m[r,0]×x, m[r,1]×y, m[r,2]×z)`. Write that lane's MAC through M_i and IR through I_i. All three row accumulations finish before any output IR is written. `L(v)` selects matrix 1, vector v, and a zero control vector. `K` selects matrix 2, the current IR vector, and control vector 1.

**MVMVA with cv=2.** This selection has a two-pass result. Snapshot x, y, and z. For each row r=0–2, let `i=r+1`. First compute `A44(4096×CV_r; m[r,0]×x,0,0)`, write MAC_i through M_i, and clamp IR_i with LM=0. Then compute `A44(0; m[r,1]×y,m[r,2]×z,0)`, overwriting that lane's MAC and IR using the selected LM. Flags from the first pass remain, but its control-vector and X contributions are absent from the final result. Other cv selections use the full matrix stage.

**Color application, CA.** For each lane, multiply the corresponding RGBC byte by 16 and by the current signed IR value. Pass that product through M44_i and I_i.

**Depth cue, DC(in).** Snapshot signed IR0 and use the signed far-color words C[21]–C[23]. First compute all three temporary differences, then all three final results:

```
d_i  = I_i(M44_i(4096*far_i - in_i), LM=0)
IR_i = I_i(M44_i(in_i + IR0*d_i), selected LM)
```

The temporary differences update MAC and flags but not the IR register words. Both passes use the selected SF, including the first pass. The command's specified input scaling is part of the operation; do not substitute an unscaled color.

**Color FIFO push, P.** For each lane, take `ASR(s32(D[24+i]),4)` and clamp it to 0–255, setting the corresponding R/G/B saturation bit if outside that range. Shift D[20] ← old D[21], D[21] ← old D[22], then pack the new RGB bytes into D[22]. Its CODE byte always comes from current D[6], including DPCT; it never comes from the consumed FIFO color.

### 4.23.4 Direct arithmetic functions

OP snapshots signed IR values `(u1,u2,u3)` and the signed rotation diagonal `(d1,d2,d3)=(m[0,0],m[1,1],m[2,2])` from matrix 0. Its three inputs to M_i are:

```
lane 1: d2*u3 - d3*u2
lane 2: d3*u1 - d1*u3
lane 3: d1*u2 - d2*u1
```

SQR snapshots signed IR1–IR3 and supplies each square to M_i. OP and SQR do not supply MAC-overflow conditions; they still apply the selected shift, visible-word wrap, and IR clamp.

GPF supplies `IR0×IR_i` to M44_i. GPL instead supplies `IR0×IR_i + s32(D[24+i])×2^(12×SF)`. Both use signed IR operands, clamp the resulting IR values with LM, then perform P. GPL's old-MAC operand is the visible word, including a directly programmed MAC word.

NCLIP uses signed halves of the three screen FIFO words and performs:

```
M0(sx0*sy1 + sx1*sy2 + sx2*sy0
 - sx0*sy2 - sx1*sy0 - sx2*sy1)
```

It does not push a FIFO. AVSZ3 and AVSZ4 multiply their unsigned depth sum by signed ZSF3 or ZSF4 respectively. For the full product x, they perform M0(x) and write `D[7]=clamp(ASR(x,12),0,65535)`. A depth clamp sets SZ and ERROR. This output uses full x, not the wrapped MAC0 word.

## 4.24 Projection stage RT(v, last)

RTPS projects V0 with `last=true`. RTPT projects V0, V1, and V2 in order, setting last only for V2. The following sequence applies to each vector.

### 4.24.1 Rotation, intermediate results, and depth

For each rotation row r=0–2, let `i=r+1` and accumulate `A44(4096×TR_r; m[r,0]×vx,m[r,1]×vy,m[r,2]×vz)` with matrix 0 and translation C[5]–C[7], then write lane i through M_i. IR1 and IR2 use the normal selected-LM clamp of the visible MAC result.

IR3 has a different flag test. Its output is the visible signed MAC3 clamped according to LM, but its saturation flag tests `ASR(full MAC3,12)` against −32768…32767, independently of SF and LM. Consequently an LM=1 clamp of a negative value need not set I3; with SF=0, IR3 can clamp while its fixed-12 flag test remains in range.

Push depth `clamp(ASR(full MAC3,12),0,65535)`: D[16]←old D[17], D[17]←old D[18], D[18]←old D[19], D[19]←new depth. An out-of-range depth sets SZ and ERROR. The depth shift is always 12, regardless of SF.

### 4.24.2 Reciprocal divide

The projection factor q is produced by the seed-ROM calculation below, not by substituting a general H/SZ division. Let `n=u16lo(C[26])` and `d=u16lo(D[19])`. If `n>=2d`, including d=0, set DIV and ERROR and use `q=0x1ffff`.

Otherwise d is nonzero. `CLZ16(d)` counts leading zeroes in its 16-bit representation, and T is the exact byte table in the appendix:

```
s          = CLZ16(d)
r1         = (d << s) & 0x7fff
r2         = T[(r1 + 0x40) >> 7] + 0x101
r3         = (U32(0x80 - r2*(r1 + 0x8000)) >> 8) & 0x1ffff
reciprocal = (r2*r3 + 0x80) >> 8
rawq       = (reciprocal*(n << s) + 0x8000) >> 16
q          = min(rawq,0x1ffff)
```

The shift after U32 is unsigned; the other divide shifts act on nonnegative integers. The final multiply uses the full integer product before shifting. A rawq clamp in this valid branch does not set DIV.

### 4.24.3 Screen coordinates and last-vertex depth cue

Shift the screen FIFO's older entries, then calculate X and Y in this order:

```
D[12] = old D[13]; D[13] = old D[14]
M0(OFX + signed IR1*q)
sx = clamp(ASR(full MAC0,16),-1024,1023)
M0(OFY + signed IR2*q)
sy = clamp(ASR(full MAC0,16),-1024,1023)
D[14] = (sx & 0xffff) | ((sy & 0xffff) << 16)
```

An out-of-range coordinate sets SX or SY and ERROR. Saturated coordinates are still pushed; projection does not reject or clip the vertex.

Only for the last vertex, calculate `M0(DQB+DQA×q)` and then `IR0=clamp(ASR(full MAC0,12),0,4096)`. A strict IR0 clamp sets I0. DQA is signed low 16-bit; DQB is signed 32-bit. Thus RTPT computes IR0 only for its third vertex, and its final visible MAC0 word is that last depth-cue result, not a screen-coordinate product.

## 4.25 GTE_PLUS multiply-add

VMAD3 selects three signed addends, three signed multiplicands, and one signed scalar from PLUS[0]–PLUS[4]. For each lane i:

```
t_i      = ASR(4096*addend_i + multiplicand_i*scalar,12)
result_i = clamp(t_i,-32768,32767)
```

There is no rounding bias, intermediate saturation, or 44-bit MAC stage. The pre-shift expression fits the signed 32-bit range for these operands. A strict `t_i>32767` sets the lane's positive flag; a strict `t_i<−32768` sets its negative flag. Either also sets PLUS ERROR.

The accepted command retains its full raw word, but selection uses only bits 7:0. VMAD3 is `0x01`; all other values are invalid. PLUS does not decode the normal SF, LM, or matrix/vector selectors. Its result, flag, and cycle words are independent of every normal GTE result and FIFO.

## 4.26 PLUS programming example

### 4.26.1 Three-lane multiply-add with delayed publication

Start from device reset. Use signed integers scaled by 4096: addend (1,2,3), multiplicand (2,1,−1), and scalar 1/2. Perform the following aligned 32-bit writes in order:

| Order | Address | Register | Raw word |
|---|---|---|---|
| 1 | `0x08010384` | PLUS ADD_XY | `0x20001000` |
| 2 | `0x08010388` | PLUS ADD_Z | `0x00003000` |
| 3 | `0x0801038c` | PLUS MUL_XY | `0x10002000` |
| 4 | `0x08010390` | PLUS MUL_Z | `0x0000f000` |
| 5 | `0x08010394` | PLUS SCALAR | `0x00000800` |
| 6 | `0x080103a4` | PLUS COMMAND — VMAD3 | `0x00000001` |

The lane integers are (8192,10240,10240), representing (2,2.5,2.5). Before the five-cycle completion deadline, PLUS CYCLES at `0x080103a8` is `0x80000005`; the result and flag words still read their reset values. At publication:

| Address | Register | Raw result |
|---|---|---|
| 0x08010398 | RESULT_XY | 0x28002000 |
| 0x0801039c | RESULT_Z | 0x00002800 |
| 0x080103a0 | FLAG | 0x00000000 |
| 0x080103a8 | CYCLES | 0x00000005 |

Software can poll busy until it clears, or issue the next direct CPU word store to PLUS COMMAND and let the store interlock wait. No GTE interrupt is generated. The normal GTE words and flags remain unchanged.

## 4.27 Repeated PLUS work

PLUS is useful when the calculation is simply a vector plus a scaled vector, without projection, a matrix transform or a color FIFO push. For example, use a chosen fraction of a displacement to update a position. With integer-coordinate addend A and multiplicand M, and scalar s scaled by 4096, VMAD3 calculates each lane as:

```
result = clamp((4096*A + M*s) ASR 12, -32768, 32767)
```

The three lanes share s and use a fixed twelve-bit shift with signed saturation. A, M and the result share the chosen coordinate unit; s is a dimensionless Q12 fraction in this example.

### 4.27.1 Prepare three independent calculations

Use the following work items. The first adds half a displacement. The second subtracts a quarter of a different displacement. The third deliberately exceeds the coordinate limits in opposite directions so that its result and flag must be retained together.

| Work | Addend A | Multiplicand M | Signed scalar s | Result |
|---|---|---|---|---|
| 1 | (120,−48,256) | (24,16,−32) | 2048, one half | (132,−40,240) |
| 2 | (−96,80,512) | (−20,12,8) | −1024, negative one quarter | (−91,77,510) |
| 3 | (32760, −32760, 1024) | (32,−32,−2048) | 2048, one half | (32767,−32768,0) |

The following table gives the complete operand words. X occupies the low half of each XY word and Y the high half. ADD_Z, MUL_Z and SCALAR consume signed low halves; their supplied upper halves are zero here.

| PLUS word | Address | Work 1 | Work 2 | Work 3 |
|---|---|---|---|---|
| ADD_XY | 0x08010384 | 0xffd00078 | 0x0050ffa0 | 0x80087ff8 |
| ADD_Z | 0x08010388 | 0x00000100 | 0x00000200 | 0x00000400 |
| MUL_XY | 0x0801038c | 0x00100018 | 0x000cffec | 0xffe00020 |
| MUL_Z | 0x08010390 | 0x0000ffe0 | 0x00000008 | 0x0000f800 |
| SCALAR | 0x08010394 | 0x00000800 | 0x0000fc00 | 0x00000800 |

After supplying an item's five operands, issue it with an aligned CPU word store of 0x00000001 directly to PLUS COMMAND at 0x080103a4. An accepted command takes the current five words for one pending calculation. Its publication deadline is five system cycles after acceptance.

The first two results are exact in the chosen integer units. Work 3's full sums are (134250496,−134250496,0), shifting to (32776,−32776,0) before the signed-halfword clamp. It sets positive-X saturation, negative-Y saturation and PLUS ERROR. Ordinary negative results, such as work 1's Y=−40, do not themselves set saturation flags.

| Work | RESULT_XY at 0x08010398 | RESULT_Z at 0x0801039c | PLUS FLAG at 0x080103a0 |
|---|---|---|---|
| 1 | 0xffd80084 | 0x000000f0 | 0x00000000 |
| 2 | 0x004dffa5 | 0x000001fe | 0x00000000 |

| Work | RESULT_XY at `0x08010398` | RESULT_Z at `0x0801039c` | PLUS FLAG at `0x080103a0` |
|---|---|---|---|
| 3 | `0x80007fff` | `0x00000000` | `0xc4000000` |

RESULT_Z's upper half is zero. Decode its low half as signed when consuming a negative Z result. These outputs and PLUS FLAG are independent of normal GTE MAC, IR, FLAG and FIFO state.

### 4.27.2 Separate current inputs, pending work and published results

There is one writable input set, one pending calculation and one published result set. They are not a queue of jobs or a history of completed vectors. While work 1 is pending, its published result ports still contain the previous result, initially zero. Rewriting ADD_XY for work 2 changes the input word for later work; it does not change work 1's already taken operands.

At completion, RESULT_XY, RESULT_Z and PLUS FLAG publish together. PLUS CYCLES, at `0x080103a8`, changes from `0x80000005` to `0x00000005`. The low bits are the retained interval, **not a countdown**. Until publication, PLUS FLAG also belongs to the older published result. A zero result or a zero flag is not a completion indication for the current job.

Atomic publication is not a multi-read snapshot. If software reads RESULT_XY, then another job publishes, then software reads RESULT_Z and FLAG, it can associate words from different jobs. Completion does not wait for readout, and there is no result-consumed acknowledgement. Retain every required result word and its flag **before the following job can replace them**.

A straightforward repeated-work order is:

1. Program work 1's operands and issue work 1.
2. Program work 2's operands while work 1 is pending. Ensure work 1 has completed, then copy its RESULT_XY, RESULT_Z and FLAG to CPU-owned storage.
3. Issue work 2 only after those three words have been retained.
4. Program work 3's operands while work 2 is pending; complete and retain work 2.
5. Issue work 3, wait for its completion, then retain its two results and flag.

There is no need to reload unchanged operands, but every changed operand must be ready before its command is accepted. Writing next-job inputs while busy does not itself issue that job.

### 4.27.3 Use the command interlock or poll for the purpose intended

A direct CPU word store to COMMAND waits if a previous job is still pending. When that completion becomes due, the old result is published and the waiting store can resume to issue the new job. This prevents overwriting pending work; it does **not** retain the completed result for software. DMA cannot issue this command, and an arbitrary wider store is not a substitute for the direct word store's admission protocol.

Use the interlock when the next command is the required synchronization point. Use a CYCLES poll when software needs the current result without issuing another command: read until bit 31 is clear, then read the results while no later job can publish. PLUS generates no completion interrupt. Independent CPU work can cover the pending interval instead, provided the resulting schedule establishes the same result lifetime.

Do not issue work n+1 and then assume there is enough time to copy all of work n. Three direct word loads for XY, Z and FLAG have a six-system-cycle base cost in this CPU, longer than VMAD3's five-cycle pending interval. Starting those loads immediately after the new issue can therefore cross its publication deadline. Copying before the next issue avoids relying on such a narrow window.

A direct-store interlock also is not an instruction that advances an abstract five-cycle arithmetic counter for free. For example, with operands already prepared, consider two one-cycle constant/address loads followed by consecutive two-cycle command stores. The first command is accepted at cycle 4; the next store reaches the busy port at cycle 6. The cycle-9 completion releases the waiting instruction, and its resumed two-cycle store accepts the second command at cycle 11, with deadline 16. Instruction issue and the intervening wait are distinct from the five-cycle arithmetic interval.

### 4.27.4 Budget preparation and readout, not just arithmetic

A schedule for the three displayed work items can retain the next inputs while the previous job is pending, but must copy the previous output before issuing the next command. With five scalar operand stores per item, preparation alone costs ten base CPU cycles, already longer than VMAD3's five-cycle interval. Only the final item then needs an explicit completion wait.

| CPU work in a specified three-item schedule | Charged cycles |
|---|---|
| Nineteen constant/address loads | 19 |
| Fifteen operand stores | 30 |
| Three command stores | 6 |
| Nine result/flag loads | 18 |
| Nine RAM result stores | 18 |
| Final CYCLES polling, tests and branches | 7 |
| Return | 2 |
| **Total** | **100** |

For this schedule, commands are accepted at cycles 31, 55 and 79, with result publication due at 36, 60 and 84. Next-item operand stores occupy 33…41 and 57…65; previous result loads and copies occupy 41…53 and 65…77. Final CYCLES reads finish at 81 (busy) and 85 (ready), result copying ends at 98 and return at 100. The retained words are the three result/flag triples listed above.

This count applies to the stated instruction schedule. Operand calculation, different storage, unchanged-input reuse and useful independent work change the budget. Five cycles remains the interval for one accepted three-lane operation, separate from preparation, issue, readout and use.

## 4.28 Depth-cued color: four equal swatches

Four equal-size, equal-shape screen-space swatches make the depth-cue color change easy to compare. They are not projections of equal-size world geometry. Each sample uses position `(0,0,Z)`; only `Z` varies. Keep the normal, light and color matrices, ambient light, material, far color, transform and projection controls fixed. The normal is `(2458,0,3277)` in Q12, the material RGB is `(160,96,192)`, and the programmed far RGB is `(80,96,160)`.

### 4.28.1 Fixed register operands

Program these controls with complete word stores before the first sample. The light and color matrices use the Q12 convention. The table gives the actual input words, including the zero high half of each final matrix word; C[12] sign-extends its low half on write.

| Destination | Programmed values | Raw words in increasing register order |
|---|---|---|
| Rotation C[0]–C[4] | Identity | `0x00001000`, `0x00000000`, `0x00001000`, `0x00000000`, `0x00001000` |
| Translation C[5]–C[7] | (0, 0, 0) | `0x00000000`, `0x00000000`, `0x00000000` |
| Light matrix L, C[8]–C[12] | Rows (3072, 0, 2048), (−2048, 0, 3072), (0, 0, −4096) | `0x00000c00`, `0xf8000800`, `0x0c000000`, `0x00000000`, `0x0000f000` |
| Ambient C[13]–C[15] | (256, 512, 128) | `0x00000100`, `0x00000200`, `0x00000080` |
| Color matrix K, C[16]–C[20] | Rows (4096, 1024, 0), (2048, 4096, 0), (1024, 2048, 4096) | `0x04001000`, `0x08000000`, `0x00001000`, `0x08000400`, `0x00001000` |
| Far color C[21]–C[23] | (80×16, 96×16, 160×16) | `0x00000500`, `0x00000600`, `0x00000a00` |
| Screen offsets C[24]–C[25] | (0, 0) | `0x00000000`, `0x00000000` |
| H, DQA, DQB, C[26]–C[28] | 128, −768, 25166080 | `0x00000080`, `0xfffffd00`, `0x01800100` |

| Destination | Programmed values | Raw words in increasing register order |
|---|---|---|
| Material and CODE, D[6] | RGB (160, 96, 192), CODE 0x34 | `0x34c060a0` |
| Normal, D[0]–D[1] when lighting | (2458, 0, 3277) | `0x0000099a`, `0x00000ccd` |
| Position, D[0]–D[1] when projecting | (0, 0, Z), Z=256, 384, 512, 768 | First word zero; second word `0x00000100`, `0x00000180`, `0x00000200`, or `0x00000300` |

The register address formulas are D[n]=`0x08010244 + 4n` and C[n]=`0x080102c4 + 4n`. The retained high byte of RGBC is the color CODE, not a GPU command. The four positions choose four coefficients under the same lighting and material setup.

For each row, use this sequence:

1. Put the position in `V0` and issue `RTPS` word `0x00080001` with `SF=1, LM=0`. Retain `IR0` and the projection `FLAG` immediately.
2. Replace `V0` with the same fixed normal. Do not write `IR0`.
3. Issue `NCDS` word `0x00080413` with `SF=1, LM=1`. Retain `RGB2` and the new `FLAG` immediately.

`IR0` is a single coefficient register, not a per-vertex FIFO. `NCDS` consumes it without changing it. Do not replace this sequence with `RTPT` followed by `NCDT`: `RTPT` leaves only its last vertex's coefficient, which `NCDT` then shares across all three normals.

| Position Z | IR0 | GTE RGB2 | GTE RGB8 |
|---|---|---|---|
| 256 | 0 | `0x344b519e` | (158,81,75) |
| 384 | 2048 | `0x34755876` | (118,88,117) |
| 512 | 3072 | `0x348a5c63` | (99,92,138) |
| 768 | 4096 | `0x34a05f4f` | (79,95,160) |

Thus `Z=384` gives half cue; `Z=512` gives three-quarter cue. Full cue at `IR0=4096` produces `(79,95,160)`, not exactly the programmed far bytes: ordered fixed-point shifts retain this precision difference. `RGB2` preserves `CODE=0x34`.

Every projection `FLAG` is `0x00000000`. Every `NCDS` `FLAG` is `0x00400000`: the negative third light component saturates `IR3` under `LM=1`. This is not the `ERROR` bit. Each command resets `FLAG`, so retain the two results separately.

### 4.28.2 Displaying the retained color in GX16

Retain the complete GTE RGB2 result, including its CODE byte. For drawing, remove `CODE=0x34` and form the opaque, untextured rectangle command as `0x60000000 | (RGB2 & 0x00ffffff)`. The retained CODE byte is not the rectangle opcode.

Disable dithering with `0xe1000400` and force stored bit 15 with `0xe6000001`; mask checking is off. Each RGB8 channel is truncated to five bits by shifting right three places. GX16 stores `R5 | (G5 << 5) | (B5 << 10) | 0x8000`. PCRTC expands each channel as `(c5 << 3) | (c5 >> 2)`.

Consequently, the GTE result bytes and the visible output bytes differ. Keep the original RGB2 values as calculation results; do not substitute the displayed RGB values.

| Z | Raw GX16 | Displayed RGB8 |
|---|---|---|
| 256 | `0xa553` | (156, 82, 74) |
| 384 | `0xb96e` | (115, 90, 115) |
| 512 | `0xc56c` | (99, 90, 140) |
| 768 | `0xd169` | (74, 90, 165) |



Figure 4.5: Four equal GTE depth-cued color swatches

*Figure: Actual 256×192 picture on black. Four equal 44×96 rectangles start at X=16, 76, 136 and 196, all at Y=48; left to right, Z=256, 384, 512 and 768. These are equal screen-space swatches, not projected geometry.*

Use raw regions (0,0)–(255,191) and (0,256)–(255,447) for the two pictures; both lie outside the queried firmware reservation. Serialize their drawing and display operations through one GTE/GPU sequence and one VBlank handler. After enabling a picture, acknowledge stale controller VBlank bit 2 and wait for the handler's counter to advance. The following register sequence gives the source selection and readback coordinates.

### 4.28.3 Display and readback sequence

With system interrupt delivery masked, request PCRTC RESET by writing `0x00000200` to `0x080103d4`. Keep PMODE_LOW zero while preparing the first image, and program these display words:

| Register | Address | Word |
|---|---|---|
| DISPFB1_LOW | `0x0801035c` | `0x000fa000` |
| DISPFB1_HIGH | `0x08010360` | `0x00000000` |
| DISPLAY1_LOW | `0x08010364` | `0x018252a8` |
| DISPLAY1_HIGH | `0x08010368` | `0x000bf3ff` |

Write GP1 word `0x09000000`. For the first surface, send GP0 draw-state words `0xe1000400`, `0xe2000000`, `0xe3000000`, `0xe402fcff`, `0xe5000000`, `0xe6000001` in that order. Clear with the opaque rectangle packet `0x60000000`, `0x00000000`, `0x00c00100`; unlike the fill command, this rectangle follows the selected drawing offset and stored-mask setting. For each swatch, send the color command described above, then its packed position `(X,48)`, then `0x0060002c` (44×96). Use X=16, 76, 136 and 196.

Probe the first surface at absolute centers (38,96), (98,96), (158,96) and (218,96). Each C0 packet is `0xc0000000`, the packed absolute coordinate, `0x00010001`. Wait for GP1/status bit 27 (READY), then drain its one GPUREAD word. After all readbacks, separately wait for GP1/status bit 26 (GPU idle).

Enable the first image with PMODE_LOW=`0x0000ff21`. Acknowledge stale VBlank bit 2 at `0x08000004` after that store, sample the VBlank counter, unmask bit 2 at `0x08000008`, and wait for the counter to change. Keep the first image selected while inspecting it, then prepare the material comparison in the other surface.

### 4.28.4 Lighting reference and material tints

Use the same supplied normal `(2458, 0, 3277)`, light matrix L, color matrix K, and ambient vector for all three commands:

- L rows: `(3072, 0, 2048)`, `(-2048, 0, 3072)`, `(0, 0, -4096)`.
- K rows: `(4096, 1024, 0)`, `(2048, 4096, 0)`, `(1024, 2048, 4096)`.
- Ambient: `(256, 512, 128)`.

With SF=1 and LM=1, NCS applies the light transform, then the color matrix and ambient contribution. Its output is the unmodulated lighting reference: the RGB bytes of RGBC do not participate, but CODE is retained in RGB2.

NCCS performs those same stages, then modulates the result with the RGB bytes of RGBC. Material A supplies RGB `(160, 96, 192)`; material B supplies `(64, 192, 160)`. Both retain CODE `34H`.

| Command | Material word (RGBC) | Raw RGB2 | RGB8 (R, G, B) |
|---|---|---|---|
| NCS `0008041EH` | `34C060A0H` (RGB ignored) | `3464D9FCH` | `(252, 217, 100)` |
| NCCS `0008041BH` | `34C060A0H` (A) | `344B519EH` | `(158, 81, 75)` |
| NCCS `0008041BH` | `34A0C040H` (B) | `343EA33FH` | `(63, 163, 62)` |

IR0 remains `4096` from the preceding depth stage. Neither command consumes IR0, so that retained value does not tint these results.

Each command starts with FLAG cleared and finishes with `00400000H`. The third light component is `-3277`; LM=1 limits it to zero, recording IR3 saturation. This flag is neither material-byte clipping nor summary ERROR. Retain RGB2 and FLAG immediately after each command, before issuing the next.

The second 256×192 picture contains three equal 56×96 swatches at X=20, 100 and 180, Y=48: reference, material A and material B, from left to right. Stage A remains selected and untouched while drawing Stage B is confined to raw rows 256–447. Use opaque, untextured 60H rectangles for both the black clear and the swatches; a 02H fill does not follow the drawing offset or stored-mask setting.

Keep the previous light, color and ambient controls. Mask interrupt delivery while preparing the second surface. Send GP0 words `0xe3040000`, `0xe406fcff`, `0xe5080000` to select its bounds and offset. Clear it with `0x60000000`, `0x00000000`, `0x00c00100`. Reload the normal into D[0]–D[1] before the three lighting commands. Each swatch packet contains `0x60000000 | (RGB2 & 0x00ffffff)`, its local packed position `(X,48)`, then `0x00600038` (56×96).

At a pitch of 1024 words, framebuffer row 256 means base word 00040000H, FBP=64 and DISPFB1_LOW=000FA040H, with source X/Y still zero. This framebuffer base is separate from the relative drawing offset Y=256: swatch Y=48 becomes raw Y=304. C0 transfers ignore that offset; read the centers at absolute (48,352), (128,352) and (208,352). The next table distinguishes stored GX16 words, the complete C0 register word and displayed RGB; the preceding table retains the original GTE bytes.

After the three material readbacks, read the four first-surface centers again to check their unchanged GX16 words. Drain readback and wait for GPU idle. With interrupt delivery masked, write the new framebuffer selection, acknowledge stale controller VBlank pending state and record the interrupt handler's VBlank counter. Unmask VBlank and wait for the counter to advance before using the new display selection. Keep both surfaces intact for this comparison. Both lie outside the firmware region reported by the BIOS reservation query.

| Swatch | Stored GX16 | C0 word | Display R,G,B |
|---|---|---|---|
| Reference | `0xb37f` | `0x0000b37f` | 255,222,99 |
| Material A | `0xa553` | `0x0000a553` | 156,82,74 |
| Material B | `0x9e87` | `0x00009e87` | 57,165,57 |



Figure 4.6: GTE unmodulated lighting reference and two material tints

*Second picture: reference, material A and material B on black. The displayed RGB values include RGB555 storage truncation and display expansion.*

## 4.29 Near-plane clipping: actual output

The cartridge explicitly constructs the crossing vertices in this illustration. The GTE projects supplied vertices; it does not clip the triangle or produce replacement geometry.

Use identity raw-Q12 rotation, zero translation, SF=1 and LM=0. LM=0 permits negative IR1/IR2 values for signed geometry. Set H=256, OFX=128«16, OFY=96«16 and DQA=DQB=0. These positions are already in view space.

The cartridge retains Z>=256. A is in front of the eye but outside this chosen positive view-space plane; B and C are inside. On both A→B and A→C, the intersection parameter is exactly t=(256−128)/(512−128)=1/3. Thus D=A+(B−A)/3 and E=A+(C−A)/3. Replace ABC with the ordered quad D,B,C,E and submit triangles DBC and DCE.

| Vertex | View-space (X,Y,Z) | Screen (SX,SY) |
|---|---|---|
| A | (0,−32,128) | (128,32) |
| B | (96,64,512) | (176,128) |
| C | (−96,64,512) | (80,128) |
| D | (32,0,256) | (160,96) |
| E | (−32,0,256) | (96,96) |

This plane is not a GTE setting. The divider instead tests H>=2×SZ3: A meets equality and raises division overflow, but still produces screen coordinates. That condition is distinct from the cartridge's Z>=256 rule. FLAG=0 does not establish membership in that half-space.

| RTPT triple | Projection FLAG | NCLIP MAC0 (doubled area) | NCLIP FLAG |
|---|---|---|---|
| ABC | 0x80020000 | 9216 | 0 |
| DBC | 0 | 3072 | 0 |
| DCE | 0 | 2048 | 0 |

Retain projection FLAG before issuing NCLIP. NCLIP clears FLAG at entry and computes signed doubled screen-space area; its new zero FLAG does not erase the earlier projection condition. Neither command constructs intersections or chooses replacement triangles. Every RTPT here reports IR0=0 and nominal CYCLES=23; 23 is not measured elapsed execution or frame time.

Keep signed geometry in the software calculation until the intersections have been constructed. For transfer, pack each vertex into two raw words: signed X/Y halfwords in the first, then signed Z in the low half of the second; this example sets the

unused high half to zero. Transfer those complete words to D[0]–D[5]. The GTE datapath decodes the signed components; raw halfword bit patterns are not signed working values for the clipping calculation.

These intersections are exact integers. For other geometry, choose a rounding rule and interpolate any vertex attributes at the same intersections.

Transfer ABC, then DBC and DCE as the triples in the table. Issue RTPT `0x00080030` for each triple and NCLIP `0x00000006` afterwards. Seed D[19] (SZ3) with zero before the first RTPT, so the older retained depth becomes zero in SZ0. Rotation C[0]–C[4] uses the identity words listed in the hardware frame sequence; C[5]–C[7] are zero. C[24]–C[28] contain `0x00800000`, `0x00600000`, `0x00000100`, `0x00000000`, `0x00000000`.

### 4.29.1 Displaying the result

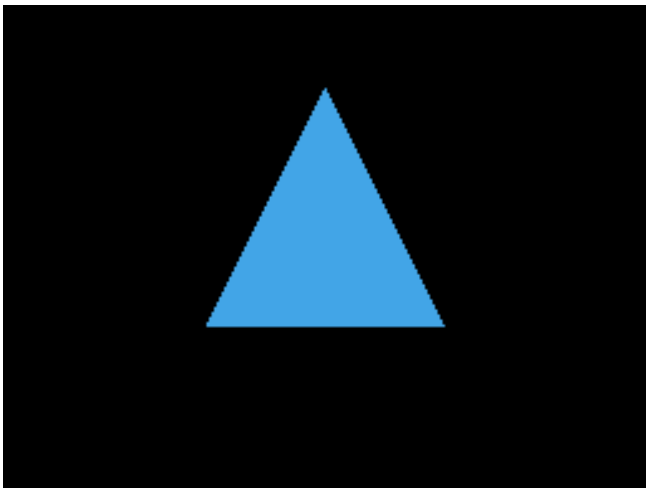
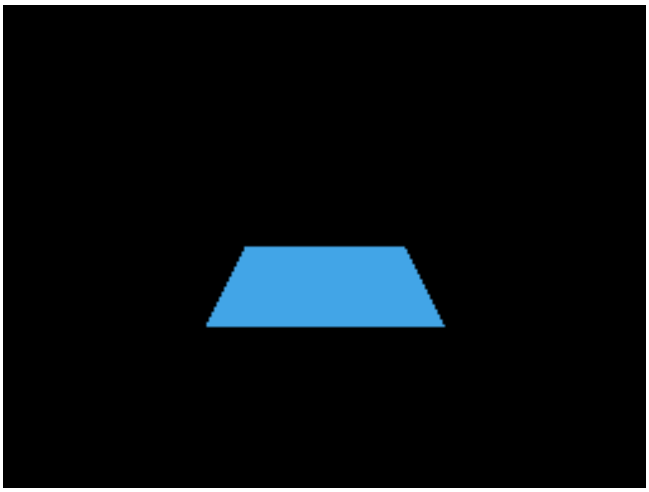
Both images use the same flat source RGB (64, 160, 224), supplied directly in GP0 triangle command `0x20e0a040`; no GTE colour command is involved. With dithering disabled and the stored mask bit set, the GX16 word is F288H, displayed as RGB (66, 165, 231).

The two 256 × 192 raw surfaces have a 1024-word pitch: A occupies rows 0–191 and B rows 256–447. Drawing bounds select each surface, and E5 adds the relative Y offset 256 for B. Display selection is different: FBP 64 selects base word 40000H, while the display source remains 0. A remains selected while B is drawn; neither surface is subsequently reused.

Use the display initialization words in the preceding depth-cue sequence: PCRTC RESET=`0x00000200`, initial PMODE_LOW=0, DISPFB1_LOW=`0x000fa000`, DISPFB1_HIGH=0, DISPLAY1_LOW=`0x018252a8`, DISPLAY1_HIGH=`0x000bf3ff`, and GP1=`0x09000000`. Send GP0 `0xe1000400`, `0xe2000000`, `0xe6000001` before drawing. For stage index `s` (0 or 1), select bounds and offset with `0xe3000000 | ((s << 8) << 10)`, `0xe40000ff | (((s << 8) + 191) << 10)`, and `0xe5000000 | ((s << 8) << 11)`. Clear with the three-word opaque rectangle packet `0x60000000`, `0x00000000`, `0x00c00100`, then send each triangle's command and its three retained screen words. After readback and GPU idle, select DISPFB1_LOW=`0x000fa000 | (s << 6)` and PMODE_LOW=`0x0000ff21`. Keep the first surface selected for two additional VBlank counter increments before drawing the second, so the two results can be viewed in sequence.

C0 probes use absolute raw coordinates, independent of E5. The probes at (128, 64)/(128, 112) return F288H/F288H for A; (128, 320)/(128, 368) return 8000H/F288H for B.

Drain each read after READY, then wait separately for GPU idle before the display stores. With interrupts masked, acknowledge stale VBlank after the final display store, sample the VBlank counter, unmask VBlank and wait for it to advance. Keep other users from changing GTE/GPU state or the two surfaces during this comparison, and preserve the BIOS reservation reported by the firmware query.

| Original ABC triangle | Clipped DBC/DCE trapezoid |
|---|---|
|  |  |

The actual images show ABC versus DBC/DCE: the new top edge is E(96, 96) → D(160, 96), with the same B/C base.

## 4.30 Appendix: reciprocal seed ROM

T contains 257 unsigned bytes. These are hexadecimal table indices, not mapped addresses. Each row gives sixteen consecutive values; the singleton index 100 follows the 00–FF range.

| Index | +0 | +1 | +2 | +3 | +4 | +5 | +6 | +7 | +8 | +9 | +a | +b | +c | +d | +e | +f |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 00 | ff | fd | fb | f9 | f7 | f5 | f3 | f1 | ef | ee | ec | ea | e8 | e6 | e4 | e3 |
| 10 | e1 | df | dd | dc | da | d8 | d6 | d5 | d3 | d1 | d0 | ce | cd | cb | c9 | c8 |
| 20 | c6 | c5 | c3 | c1 | c0 | be | bd | bb | ba | b8 | b7 | b5 | b4 | b2 | b1 | b0 |
| 30 | ae | ad | ab | aa | a9 | a7 | a6 | a4 | a3 | a2 | a0 | 9f | 9e | 9c | 9b | 9a |
| 40 | 99 | 97 | 96 | 95 | 94 | 92 | 91 | 90 | 8f | 8d | 8c | 8b | 8a | 89 | 87 | 86 |
| 50 | 85 | 84 | 83 | 82 | 81 | 7f | 7e | 7d | 7c | 7b | 7a | 79 | 78 | 77 | 75 | 74 |
| 60 | 73 | 72 | 71 | 70 | 6f | 6e | 6d | 6c | 6b | 6a | 69 | 68 | 67 | 66 | 65 | 64 |
| 70 | 63 | 62 | 61 | 60 | 5f | 5e | 5d | 5d | 5c | 5b | 5a | 59 | 58 | 57 | 56 | 55 |
| 80 | 54 | 53 | 53 | 52 | 51 | 50 | 4f | 4e | 4d | 4d | 4c | 4b | 4a | 49 | 48 | 48 |
| 90 | 47 | 46 | 45 | 44 | 43 | 43 | 42 | 41 | 40 | 3f | 3f | 3e | 3d | 3c | 3c | 3b |
| a0 | 3a | 39 | 39 | 38 | 37 | 36 | 36 | 35 | 34 | 33 | 33 | 32 | 31 | 31 | 30 | 2f |
| b0 | 2e | 2e | 2d | 2c | 2c | 2b | 2a | 2a | 29 | 28 | 28 | 27 | 26 | 26 | 25 | 24 |
| c0 | 24 | 23 | 22 | 22 | 21 | 20 | 20 | 1f | 1e | 1e | 1d | 1d | 1c | 1b | 1b | 1a |
| d0 | 19 | 19 | 18 | 18 | 17 | 16 | 16 | 15 | 15 | 14 | 14 | 13 | 12 | 12 | 11 | 11 |
| e0 | 10 | 0f | 0f | 0e | 0e | 0d | 0d | 0c | 0c | 0b | 0a | 0a | 09 | 09 | 08 | 08 |
| f0 | 07 | 07 | 06 | 06 | 05 | 05 | 04 | 04 | 03 | 03 | 02 | 02 | 01 | 01 | 00 | 00 |

| Index | Value |
|---|---|
| 100 | 00 |

# 5 GPU Drawing Engine

## 5.1 Function and data flow

The GPU converts screen-space polygons, lines and rectangles into pixels in its 2 MiB local memory. Use polygons for scaled or transformed surfaces, rectangles for sprites and tiles, and lines for outlines. Drawing commands supply positions, colors and texture coordinates. A retained drawing environment supplies the drawing area, coordinate offset, texture page, blending and mask controls.

Commands and their parameters form a stream of 32-bit words written to GP0. The GPU assembles complete packets and executes them in order. CPU stores feed a sixteen-word command FIFO. DMA supplies the same stream through a separate sixteen-word ingress buffer, handing each admitted block to the command FIFO at block end. Packet framing continues across those block boundaries. GP1 is the separate control port for reset, DMA direction, state queries and GPU mode settings.

Local memory also holds textures and palettes. Fill, upload and copy commands prepare this storage; GPUREAD returns requested raw pixel words to the CPU or DMA. These operations share the ordered drawing stream, allowing an upload, its draws and a later copy to be placed in the required sequence.

The programmer manages three hand-offs: supplying packets, completing GPU work, and selecting a finished image for display. FIFO readiness permits further input; GPU idle identifies a drained command path; readback READY permits image-data consumption. PCRTC's VBlank-entry latch separately selects the settings used to display local memory. Use the indication belonging to the hand-off being performed.

For three-dimensional scenes, first transform, light and project the geometry with GTE or software. Clip and order its primitives, then submit their screen positions and colors. Triangle color and texture interpolation are affine in screen space; a quad is processed as two triangles. The GPU draws in command order, so overlapping surfaces must be submitted in the intended order.

PCRTC reads the resulting local-memory image using its programmed source layout, display windows and composition settings. GPU drawing always uses the raw destination-word layout described below. Select a matching PCRTC layout for that storage. The frame hand-off sequence shows how to alternate drawing and display regions.

## 5.2 Technical specifications

The following specifications distinguish the **32-bit GP0 programming word**, the **16-bit local-memory word**, and the **command tick** used for execution costs. Drawing, texture lookup and display sampling share local memory.

| Resource or interface | Specification |
|---|---|
| Installed main RAM | 4 MiB, separate from GPU local memory |
| GPU local memory | 2 MiB: 1,048,576 sixteen-bit words; low byte first |
| GP0 — `0x0801023c` | Write command/input words; read GPUREAD |
| GP1 — `0x08010240` | Write control words; read GPUSTAT, not the last control word |
| GP0 input FIFO | 16 thirty-two-bit words |
| Separate DMA ingress | 16 thirty-two-bit words; not another 16 CPU FIFO slots |
| Ordinary programming access | Thirty-two-bit words, four-byte alignment |
| Input coordinates | Each supplied X and Y is signed eleven-bit: −1024…1023 |
| Drawing offset | Separate signed eleven-bit X and Y values |
| Input color | Independent unsigned eight-bit R, G and B components: 0–255 |
| Rendered color storage | R in bits 4:0, G in 9:5, B in 14:10; bit 15 stored separately |
| Installed CPU/system-cycle frequency | 33,868,800 Hz; approximately 29.526 ns per cycle |
| Command-cost conversion | Two command ticks per system cycle |

Polygon vertices receive the drawing offset as integer sums. Before triangle raster shifting, a triangle is rejected if its inclusive span exceeds **1024 in X or 512 in Y**: `maxX−minX+1 > 1024` or `maxY−minY+1 > 512`. The test applies separately to each triangle, including each half of a quad.

Triangle color and texture-coordinate planes use supplied screen X/Y. Their affine interpolation and ordered local-memory writes are detailed in the primitive sections. Display-window sizes are programmed separately through PCRTC.

**Issue cost and command completion.** An ordinary scalar word load or store, including its displacement form, has a **two-system-cycle base cost**. Address and value preparation, control flow and readiness waits are additional. A word-sequence store instead charges two base cycles plus its word count divided by four, rounded upward. The sequence advances the destination by four bytes per word: starting at GP0 reaches GP1 next, not another GP0 stream word.

Accepting a GP0 word and completing an assembled command are separate events. For a command with positive cost N ticks starting execution at cycle C:

```
Completion deadline = C + (N/2 rounded upward)
```

Zero-cost work completes immediately. Queued execution continues from the prior command's deadline. CPU issue and GPU execution can overlap; construct their timeline when budgeting a frame. GP1 controls have no queued raster-cost interval. Use GPUSTAT to observe progress.

For a fill with rounded width 32 and height 4, the command cost is `46 + (32/8 + 9)×4 = 98` ticks, or **49 system cycles**. A normalized 3×5 local-memory copy costs `2×3×5 = 30` ticks, or **15 cycles**. These intervals begin at execution, not at preparation of the first packet word. With the GPU IRQ request initially clear, an IRQ marker queued behind a fill starting at cycle zero executes and raises its request at cycle 49; its own one-tick interval completes at cycle 50. The IRQ request can therefore be set while the GPU is not yet idle.

An image-upload or readback command's one-tick cost does **not** mean its whole payload transfers in one cycle. Readback becomes pending at command execution completion and becomes consumable only when ready; GPUSTAT bit 27 reports that ready condition. Until then, GP0 reads return the retained GPUREAD value. Neither command completion, idle status nor an IRQ marker acknowledges that a new picture has reached the display latch.

**Access-width cautions.** Use complete word transfers for commands. An aligned byte load runs the full GP0 read operation before returning its low byte; it is not a byte-wise peek into a ready readback word. Byte stores and halfword IO accesses fault. F32 uses one word's bit pattern; F64 accesses the word at A and then A+4, stopping if the first faults. Thus an F64 access at GP0 reaches **GP0 then GP1**, not two words of the GP0 stream, and is not an atomic GPU transaction. Misaligned accesses fault before the corresponding device operation.

## 5.3 Program a complete picture in BLua

This sequence prepares one static 256×192 picture, reads three stored texels back, and displays those same raw words in a small strip. Its texture and command words are fixed; two writable words retain the readback. Each scalar stream store addresses GP0 again, while indexing advances through the source data.

The picture contains two enlarged copies of one 4×4 texture, a Gouraud triangle, additive rectangles and protected overlapping rectangles. A small unscaled textured rectangle demonstrates persistent page selection. A separate three-pixel strip receives the actual readback result. The bottom strip displays sixteen uploaded palette4 texels. No GTE command is needed: all positions are already screen coordinates.

### 5.3.1 Ports, texture and command storage

The following listings form one entry sequence. GP0/GP1 are the GPU ports listed in the specification table. PCRTC's CSR, framebuffer, source, extent and PMODE formats are specified in the PCRTC chapter.

```
module<entry>

local irq_ack<const>: *word = 0x08000004
local irq_mask<const>: *word = 0x08000008
local gp0<const>: *word = 0x0801023c
local gp1<const>: *word = 0x08010240
local pmode<const>: *word = 0x08010354
```

```
local framebuffer<const>: *word = 0x0801035c
local source<const>: *word = 0x08010360
local display_extent<const>: *word = 0x08010368
local pcrtc_csr<const>: *word = 0x080103d4

-- Sixteen raw RGB555/mask words, low halfword first on upload.
rodata texels: u16[4][4] = {
    { 0x001f, 0x001f, 0x03e0, 0x03e0 },
    { 0x001f, 0x001f, 0x03e0, 0x03e0 },
    { 0x7c00, 0x7c00, 0x7fff, 0x7fff },
    { 0x7c00, 0x7c00, 0x8000, 0x0000 },
}
```

### 5.3.2 Texture uploads and drawing packets

The halfword array is a sixteen-pixel image. Its last row deliberately includes both `8000H` and `0000H`: the former draws black; the latter leaves the background. The upload sends two adjacent halfwords per command word, low half first.

The packet array below contains **55 words**. Its first two packets are nine-word textured quads. T0 supplies U/V with CLUT zero; direct16 ignores that palette. T1 supplies page/depth fields `010AH`: X=10×64=640, Y=0, direct16. The persistent update preserves the separately programmed draw-to-displayed-field bit. The four-word textured rectangle after them supplies no new page.

Each quad maps byte U/V from 0 to 3 across its vertices, magnifying the texture by affine interpolation. Raw RGB gain is ignored. The modulated quad halves red and preserves green; its supplied blue gain saturates already-full-blue texels. The later six-word triangle instead interleaves three colors with three positions and has no UV words.

```
-- Palette entry zero is red; entry three is transparent; entry four is black.
rodata palette_colors: u16[16] = {
    0x001f, 0x03e0, 0x7c00, 0x0000, 0x8000, 0x7fff, 0x03ff, 0x7fe0,
    0x7c1f, 0x4210, 0x0010, 0x0200, 0x4000, 0x0210, 0x4200, 0x4010,
}
rodata palette_indices: u16[4] = { 0x3210, 0x7654, 0xba98, 0xfedc }

-- These are complete GP0 packets, not an array of register values.
rodata drawing_commands: word[] = {
    -- Raw textured quad. T1 selects direct16, page X=640, page Y=0.
    0x2d808080,
    16 | (24 << 16), 0,
    80 | (24 << 16), 3 | (0x010a << 16),
    16 | (88 << 16), 3 << 8,
    80 | (88 << 16), 3 | (3 << 8),
    -- Modulated quad: halve red, preserve green; blue gain 1.5 saturates.
    0x2cc08040,
    96 | (24 << 16), 0,
    160 | (24 << 16), 3 | (0x010a << 16),
    96 | (88 << 16), 3 << 8,
    160 | (88 << 16), 3 | (3 << 8),
    -- No E1 here: this rectangle inherits the preceding polygon's T1 page.
    0x65000000, 24 | (112 << 16), 0, 4 | (4 << 16),
    -- A separate RGB-interpolated triangle; no texture words.
    0x300000ff, 176 | (24 << 16),
    0x0000ff00, 240 | (24 << 16),
    0x00ff0000, 208 | (88 << 16),
    -- Two opaque rectangles, then an additive rectangle crossing both.
    0x60402020, 16 | (128 << 16), 64 | (40 << 16),
    0x60204020, 80 | (128 << 16), 64 | (40 << 16),
    0xe1000420,
    0x62202080, 48 | (144 << 16), 64 | (24 << 16),
    -- Set+check: the first rectangle marks its pixels; the second protects them.
    0xe6000003,
```

```
    0x600080c0, 176 | (128 << 16), 32 | (32 << 16),
    0x60c08000, 192 | (144 << 16), 32 | (32 << 16),
    0xe6000000,
    -- Sixteen palette4 texels across the bottom strip. CLUT is (768,256).
    0x2d000000,
    16 | (176 << 16), (16 << 8) | (0x4030 << 16),
    240 | (176 << 16), 15 | (16 << 8) | (0x000b << 16),
    16 | (192 << 16), 16 << 8,
    240 | (192 << 16), 15 | (16 << 8),
}
```

The final quad uses palette4 page X=704,Y=0 and constant V=16. Its four source halfwords hold sixteen four-bit indices, lowest nibble first: `3210H` selects entries 0,1,2,3. The CLUT attribute `4030H` selects raw (768,256); entry 0 is red, entry 3 is zero, and entry 4 is nonzero black. The palette and indices are uploaded separately through GP0. Raw texturing ignores the quad's zero color field. The sixteen colors are magnified across the bottom strip by interpolating U=0…15 while the source remains four packed words.

### 5.3.3 Readback storage and interrupts

The two BSS words receive the readback result. The interrupt entry acknowledges delivered requests. GPU IRQ delivery is masked during drawing; its request latch is observed explicitly later.

```
bss readback_words: word[2]

function irq(flags)
    *irq_ack = flags
end
```

### 5.3.4 Establish the drawing and display domains

PCRTC CSR RESET establishes both PCRTC configuration and GPU stream/register defaults; it does not initialize local-memory pixels. The Y-address gate is explicitly closed, and E1–E6 establish the drawing environment. The background fill covers only the 256×192 picture; the texture's later upload initializes its separate source words.

The drawing area includes (0,0)…(255,191), with zero offset. E1=`0400H` sets draw-to-displayed-field and disables dither. E6 clears destination check/force. The picture's background RGB bytes `(8,16,24)` reduce to raw word `0C41H`. PCRTC reading circuit 1 selects GX16 storage with a 1,024-word stride; PMODE remains disabled while the GPU prepares this single, never-reused picture.

```
*irq_mask = 0
*pcrtc_csr = 0x00000200
*pmode = 0
*framebuffer = 0x000fa000
*source = 0
*display_extent = 0x000bf3ff
*gp1 = 0x09000000

-- Explicit drawing state; RESET does not clear local memory.
*gp0 = 0xe1000400
*gp0 = 0xe2000000
*gp0 = 0xe3000000
*gp0 = 0xe40000ff | (191 << 10)
*gp0 = 0xe5000000
*gp0 = 0xe6000000
*gp0 = 0x02181008
*gp0 = 0
*gp0 = 256 | (192 << 16)
```

### 5.3.5 Upload and issue the stream through one port

A0 selects 4×4 raw pixels at `(640,0)`: exactly eight payload words follow its three-word header. The word view reads two adjacent raw halfwords at a time. Both loops write to `*gp0` every iteration; indexing applies to the source only. The scalar-store admission mechanism supplies any necessary wait; DMA-ready bit 28 is not its admission test. A CPU word-sequence store advances the destination and must not replace these fixed-port stores.

```
-- A0 header followed by eight whole-word payload stores to the SAME port.
*gp0 = 0xa0000000
*gp0 = 640
*gp0 = 4 | (4 << 16)
local texture_words<const>: *word = texels
for index = 0, sizeof(u16[4][4]) // 4 - 1 do
    *gp0 = texture_words[index]
end
-- A palette is raw local-memory data, uploaded exactly like the direct image.
*gp0 = 0xa0000000
*gp0 = 768 | (256 << 16)
*gp0 = 16 | (1 << 16)
local palette_words<const>: *word = palette_colors
for index = 0, sizeof(u16[16]) // 4 - 1 do
    *gp0 = palette_words[index]
end
*gp0 = 0xa0000000
*gp0 = 704 | (16 << 16)
*gp0 = 4 | (1 << 16)
local index_words<const>: *word = palette_indices
for index = 0, sizeof(u16[4]) // 4 - 1 do
    *gp0 = index_words[index]
end
for index = 0, #drawing_commands - 1 do
    *gp0 = drawing_commands[index]
end
```

The direct image occupies eight payload words. The sixteen-entry palette occupies another eight, and its four packed index halfwords occupy two. All three uploads finish before their drawing packets. The final quad's T1 selects palette4 rather than the preceding direct16 page; its T0 locates the uploaded palette. A zero index therefore draws red, while index 3 leaves the background because the **fetched palette word** is zero. Index 4 draws black from `8000H`. No direct CPU local-memory aperture or palette upload helper is needed.

### 5.3.6 Wait for readback readiness and consume its full count

The C0 request selects the first three texture pixels: red, red, green. GPUSTAT bit 27—not bit 26—qualifies consumption. Once ready and without a DMA reader owning the port, the two reads return `001F001FH` and `000003E0H`. The final word's high half is zero and consuming it releases readback. The preceding draw commands must reach the ordered readback fence before its result is supplied. Continue waiting until bit 27 reports READY.

```
-- Three pixels produce two reads. The final read has a zero high half.
*gp0 = 0xc0000000
*gp0 = 640
*gp0 = 3 | (1 << 16)
while (*gp1 & 0x08000000) == 0 do
end
readback_words[0] = *gp0
readback_words[1] = *gp0
```

### 5.3.7 Use the returned raw representation directly

Upload those two words into the three-pixel strip at `(16,112)`. There is no conversion to RGB bytes or display pixels. The final high half is unused because the destination area contains three pixels, not four. The following marker is a new **whole** GP0 word, never a value hidden in that unused halfword.

```
-- Re-upload those exact words as a three-pixel strip at (16,112).
*gp0 = 0xa0000000
*gp0 = 16 | (112 << 16)
*gp0 = 3 | (1 << 16)
for index = 0, #readback_words - 1 do
    *gp0 = readback_words[index]
end
```

### 5.3.8 Separate the marker, idle and display hand-off

The marker sets GPUSTAT bit 24 after earlier queued commands. GP1(02h) clears that GPU latch; the controller acknowledgement clears pending GPU IRQ bit 6. Waiting for bit 26 is separate because the marker can be set before its own one-tick command interval completes. Only then does PMODE enable reading circuit 1. The later VBlank publication, not either GPU status bit, accepts the picture. The cartridge subsequently acknowledges VBlank requests and sleeps without modifying the displayed pixels.

```
-- IRQ is an in-order marker, not an idle or display acknowledgement.
*gp0 = 0x1f000000
while (*gp1 & 0x01000000) == 0 do
end
*gp1 = 0x02000000
*irq_ack = 0x00000040
while (*gp1 & 0x04000000) == 0 do
end
*pmode = 0x0000ff21
*irq_ack = 0xffffffff
*irq_mask = 0x00000004
while true do
    halt_until_irq
end
```

### 5.3.9 Stored-word results

These expected samples distinguish the controls without relying on display-color expansion. Mask bits remain part of each stored word.

| Raw position | Expected word | What it distinguishes |
|---|---|---|
| (16,24) | `001FH` | Raw full red |
| (96,24) | `000FH` | Red reduced by gain 64 |
| (138,68) | `7FEFH` | Modulated white: red reduced, green unchanged, blue saturated |
| (24,112) | `001FH` | Following rectangle inherits polygon T1 page |
| (26,115) | `8000H` | Nonzero black texel writes its bit 15 |
| (27,115) | `0C41H` | Exact-zero texel leaves the initialized background |
| (50,150) | `3114H` | Additive rectangle over first destination color |
| (90,150) | `2194H` | Same source over a different destination color |
| (200,150) | `8218H` | A later checked draw preserves the first rectangle’s marked pixel |

| Raw position | Expected word | What it distinguishes |
|---|---|---|
| (220,165) | `E200H` | Second rectangle writes and marks an unprotected pixel |
| (16,180) | `001FH` | Palette index 0 fetches a nonzero red word |
| (31,180) | `03E0H` | Next packed nibble selects palette entry 1 |
| (46,180) | `7C00H` | Palette entry 2 is blue |
| (61,180) | `0C41H` | Index 3 fetches zero and preserves the background |
| (76,180) | `8000H` | Index 4 fetches nonzero black |
| (91,180) | `7FFFH` | Index 5 fetches white |

The picture remains in local memory after drawing. PCRTC's VBlank-entry latch makes its display settings active. For animation, alternate storage using Render successive frames; include packet issue, GPU execution and the display wait in the frame budget.

### 5.3.10 GPU output plate

This plate brings several packet types into one 256 × 192 image. The two large upper quads sample the same 4 × 4 direct-colour texture: the left uses raw texture colour, while the middle applies the packet's RGB colour. Their red and white areas show the contrast. The bottom strip instead reads four-bit indices and looks up their colours in a CLUT; index zero is red here, not transparent.

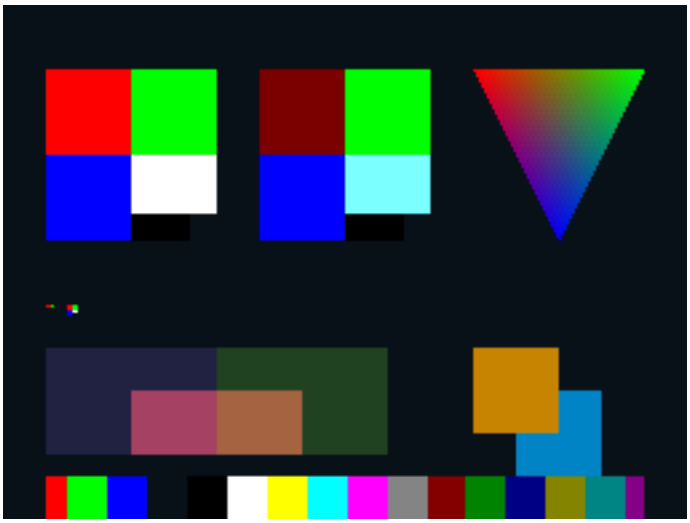


Figure 5.1: Full 256 by 192 GPU output plate

**GPU output plate.** Direct and indexed textures, an untextured RGB-gradient triangle, a blending overlay, and a mask-protected overlap share a dark background. The small marks near the left edge are the readback strip and the unscaled 4 × 4 texture.

The black texture areas and the background-coloured gaps are deliberately different. The source word `0x8000` supplies black and retains bit 15 in VRAM; a sampled `0x0000` leaves the background unchanged. This distinction appears in both the direct-colour texture and the indexed strip, whose entries 4 and 3 contain those words respectively.

The lower-left overlay selects semi-transparency with background-plus-foreground blending. At the lower right, set-mask and check-mask are enabled together: the first rectangle sets destination bit 15, so the second cannot replace it in their overlap. These are different effects; the protected overlap is not a blend.

**Picture regions.** The table identifies each packet's placement and effect, with selected raw VRAM words from the completed drawing.

| Packet location | Picture and packet interpretation | Selected VRAM readback |
|---|---|---|
| Background: fill at `(0,0)`, `256 × 192` | Dark field and margins; fill header `0x02181008`. | Background visible through the upper-left texture at `(78,84)` is `0x0c41`. |
| Quad: `(16,24)`, `(80,24)`, `(16,88)`, `(80,88)` | Raw red/green/blue/white texture, plus black and unchanged-background areas. Header `0x2d808080`; UV `(0,0)` to `(3,3)`; T1 `0x010a` selects direct16 at `(640,0)`. | `(16,24)` = `0x001f`; black at `(59,84)` = `0x8000`. |
| Quad: `(96,24)`, `(160,24)`, `(96,88)`, `(160,88)` | Modulated counterpart: darker red and tinted white. Header `0x2cc08040`; same UVs and T1 as the raw quad. | `(96,24)` = `0x000f`; `(140,24)` = `0x03e0`; `(138,68)` = `0x7fef`. |
| Rectangle: `(24,112)`, `4 × 4` | Small raw view of the same direct-colour image; header `0x65000000`, UV `(0,0)`. | `(26,114)` = `0x7fff`; `(26,115)` = `0x8000`; `(27,115)` = `0x0c41`. |

**Nine-region legend, continued.** Packet positions, visible results and selected raw words remain separate.

| Packet location | Picture and packet interpretation | Selected VRAM readback |
|---|---|---|
| Triangle: `(176,24)`, `(240,24)`, `(208,88)` | Separate untextured triangle with red, green and blue vertex colours. | No triangle pixel word is listed. |
| Bases: `(16,128)` and `(80,128)`, each `64 × 40`; overlay: `(48,144)`, `64 × 24` | The overlay crosses both bases. Header `0x62202080` with E1 `0xe1000420` selects background-plus-foreground blending. | Intersections: `(50,150)` = `0x3114`; `(90,150)` = `0x2194`. |
| Rectangles: `(176,128)` and `(192,144)`, each `32 × 32` | Mustard/cyan pair with E6 `0xe6000003`: the first rectangle remains in the protected overlap. E6 then returns to `0xe6000000`. | Overlap `(200,150)` = `0x8218`; second-only `(220,165)` = `0xe200`. Both retain bit 15. |
| Strip: `(16,176)`, `(240,176)`, `(16,192)`, `(240,192)` | Raw indexed texture, U `0..15`, V `16`; T1 `0x000b` selects palette4 at page `(704,0)`. CLUT `0x4030` selects `(768,256)`. Indices come from `0x3210`, `0x7654`, `0xba98`, `0xfedc`, low nibble first. | Red `(16,180)` = `0x001f`; unchanged background `(61,180)` = `0x0c41`; black `(76,180)` = `0x8000`; white `(91,180)` = `0x7fff`. |
| Witness: `(16,112)`, `3 × 1` | Three pixels read from `(640,0)` and uploaded at this destination. | The three-pixel row reads as two GPUREAD words: `0x001f001f`, then `0x000003e0`. |

## 5.4 GP0, GPUREAD, GP1 and GPUSTAT

The GPU has two thirty-two-bit programming ports. Their read and write meanings are different; neither address is an ordinary read/write register variable.

| Address | Write operation | Read operation | Full-reset read at beam epoch zero |
|---|---|---|---|
| `0x0801023c` | GP0 command or payload word | GPUREAD latch or next permitted ready readback word | `0x00000000` |
| `0x08010240` | GP1 control word | GPUSTAT | `0x14922000` |

Writing GP0 does not make a following read return that command word. Writing at the GPUSTAT address sends a GP1 control; it does not assign a status-bit mask. The reset status above includes the reset display-mode fields. Beam-derived bits subsequently change with the scanout position.

### 5.4.1 Access and ordering

Use aligned word operations for normal programming. Ordinary scalar word loads and stores, including their displacement forms, have a two-system-cycle base cost; preparation, control flow and a blocked store's wait/retry are separate.

| Access | Port effect |
|---|---|
| Word or U32 | Four-byte aligned; one complete thirty-two-bit port operation |
| U8 load | Only at a word-aligned IO address; perform the full word read, then return its low byte |
| U8 store; U16 load/store | Fault rather than partially access the port |
| F32 | Four-byte aligned; one word's bit pattern |
| F64 | Four-byte aligned; low word at A, then high word at A+4; a first-word fault prevents the second operation |

A byte read is not a peek into one byte of GPUREAD: it can consume the same complete readback word as a word load. Misaligned word/F32/F64 accesses and odd halfword accesses are rejected before a GPU operation occurs.

An F64 instruction is still **one CPU instruction**, but its two ordered word operations are not a paired GPU register transaction. At GP0 they address **GP0 then GP1**, not two GP0 stream words. An F64 store checks admission for its first address only; it does not reserve both ports. Do not infer a sixty-four-bit GPU command or atomic GP0/GP1 update from the CPU instruction boundary.

A CPU word-sequence store also advances the destination by four bytes for each word. Starting it at GP0 therefore reaches GP1 next; it is not a burst of repeated writes to the GP0 stream port.

### 5.4.2 GPUSTAT fields

| Bits | Field | Value and producer |
|---|---|---|
| 31 | Displayed-line parity | Beam-derived; calculation below. Not command readiness. |
| 30:29 | DMA direction | GP1 direction low two bits: 0 off, 1 FIFO, 2 CPU → GP0, 3 GPUREAD → CPU. |
| 28 | DMA packet acceptance | The packet-state test below; not a general CPU-store-ready bit. |
| 27 | Readback ready | One exactly when a readback stream is READY, not merely when its header has been accepted. |
| 26 | Idle | One only under all idle conditions below. |
| 25 | DMA request | Direction 0: zero. Direction 1: DMA ingress empty and GP0 FIFO has room. Direction 2: bit 28. Direction 3: bit 27. A system ingress stop suppresses the write-side requests for directions 1 and 2. |
| 24 | GPU interrupt request | Latched by GP0(1Fh); GP1(02h) clears this GPU request latch. |
| 23 | Display disabled | GP1 display-disable low bit; one means disabled. |
| 22 | Vertical-interlace mode | GP1 display-mode bit 5. |
| 21 | 24-bit display-color mode | GP1 display-mode bit 4. |
| 20 | PAL-mode flag | GP1 display-mode bit 3. |
| 19 | Vertical-resolution mode | GP1 display-mode bit 2. |
| 18:17 | Horizontal-resolution mode 1 | GP1 display-mode bits 1:0. |
| 16 | Horizontal-resolution mode 2 | GP1 display-mode bit 6. |
| 15 | Texture-page Y high | GP0 draw-mode bit 11, moved here rather than copied to bit 11. |
| 14 | Reverse mask | Cleared by a display-mode update. Retained display-mode bit 7 is not mapped here and specifies no reverse-image action. |
| 13 | Interlaced-field flag | One if GPU vertical-interlace mode is clear or the PCRTC field is zero; otherwise zero. |
| 12 | Destination-mask check | Stored mask-mode bit 1. |
| 11 | Set output mask | Stored mask-mode bit 0. |
| 10:0 | Draw-mode readback | Stored GP0 draw-mode bits 10:0, without re-encoding. |

A GPUSTAT read first synchronizes already-due GPU command completions, then updates beam-derived fields and recomputes command/DMA conditions. Stored configuration readback, a retained request latch and current readiness are thus parts of the same returned word, not interchangeable kinds of completion.

The GP1 mode fields are GPU register readback, not PCRTC beam-clock selectors. For bit 31, use the decoded display-start Y and the current PCRTC field. When both GPU vertical-resolution and vertical-interlace bits are set, its parity is `(displayStartY + displayedField) AND 1`, where displayedField is zero during VBlank and otherwise the current field. In other modes it is `(displayStartY + integer-quotient(currentHalfLine,2)) AND 1`. If the beam is stopped, bits 13 and 31 retain their existing values.

### 5.4.3 Idle, DMA acceptance and CPU admission are different tests

**Bit 26** is one only when readback is idle, no command-completion deadline is pending, both input FIFOs are empty, and no fixed-packet assembly, image-upload payload or open polyline remains. Retained completed commands awaiting presentation do not prevent this idle indication. It does not acknowledge a new displayed picture.

**Bit 28** first requires idle readback, no system ingress stop and empty DMA ingress. After those requirements:

- During an image-upload payload, it is one when the GP0 FIFO has room.
- During a polyline payload, it is zero.
- With no partial packet and no FIFO words, it is one.
- Otherwise it is one only for a nonpolygon, nonline packet with fewer accumulated words than its target length.

**A direct CPU GP0 word store** instead requires no system ingress stop, empty DMA ingress, a physical GP0 FIFO count below sixteen, and no DMA ownership of the GP0 write port. It need not wait for GPU idle or readback completion if a word can still be admitted. A polygon or line prefix can therefore clear bit 28 while a CPU word store remains admissible. Do not substitute bit 28 for this store-admission protocol.

The scalar CPU store checks admission before writing. If blocked, it waits and retries; no command word has yet been accepted. GP1's store-admission gate is the system supervisor-quiesce condition, not GP0 busy status. A full GP0 FIFO, ordinary command execution or an active readback does not by itself block GP1.

### 5.4.4 GPUREAD and status do not return the same result

GPUREAD retains the most recent info-query result or consumed readback word. An info query that produces a result updates this same latch; it does not make the query result appear at the GP1 read address.

| State at a CPU read | GP0 result and side effect | GP1 result |
|---|---|---|
| No READY readback | Existing GPUREAD value, including a retained query result; no stream advance | GPUSTAT with bit 27 clear |
| READY, and DMA does not own GP0's read port | Consume the next complete readback word and replace GPUREAD | GPUSTAT; bit 27 indicates readiness, not the data word |
| READY, but DMA owns GP0's read port | Return the existing GPUREAD latch without consuming the stream | GPUSTAT may still have bit 27 set |

A permitted read packs the first sixteen-bit pixel in the low half and the next in the high half. An odd final pixel has a zero high half. Consuming the final word returns readback to idle and allows queued command processing to resume. Check GPU idle separately when more commands are queued. The next permitted READY read replaces any intervening query result with image data.

### 5.4.5 Reset and display visibility

The other programmed defaults associated with full GPU reset are:

| State | Raw default |
|---|---|
| Draw mode, texture window, drawing bounds, drawing offset, mask mode | Zero |
| Display start | Zero |
| Display mode | `0x00000009` |
| Horizontal display range | `0x00c60260` |
| Vertical display range | `0x00044c23` |

| State | Raw default |
|---|---|
| VRAM Y-address extension | Zero |
| DMA direction; GPU interrupt-request latch | Zero |

These control values are selected through GP1 commands. **GP1(00h)** restores the defaults above while preserving GPUREAD, the VRAM Y-extension latch and PCRTC configuration. It does not initialize or clear local memory.

Full-device reset initializes VRAM with a deterministic pattern. GP1(00h) and PCRTC CSR RESET instead perform the command/reset transitions described below; accepted drawing and consumed upload data can still change pixels. At cartridge entry the BIOS has already uploaded its system texture and used the terminal. Initialize the program's drawing and texture regions, preserving the firmware VRAM reservation.

Accepted GP1 changes appear immediately in programmed GPU state after due command completions have synchronized. Presented display/status/start/range and Y-extension values are copied at VBlank presentation instead of on every control write. Immediate GPUSTAT readback proves programmed state, not presentation-latch completion. PCRTC's separately programmed reading circuits, windows, timing and displayed latch remain a distinct interface.

## 5.5 GP1 control commands

Write each control as one aligned word to **GP1, `0x08010240`**. A read from that address returns GPUSTAT, not the control word. GP1 is separate from the GP0 packet stream: a control is not inserted between a GP0 packet's operands.

| Bits | Encoding |
|---|---|
| 31:24 | Complete eight-bit opcode |
| 23:0 | Payload; each command below states which bits it consumes |

The word is `(opcode << 24) OR (payload AND 0x00ffffff)`. Decode uses all eight opcode bits, not the low six. A payload bit outside a command's mask is ignored unless explicitly described as retained. An accepted GP1 write first synchronizes already-due GPU work, then applies its control to programmed state. It does not join the GP0 FIFO or acquire a separate queued raster-completion interval.

An ordinary scalar word store has a two-system-cycle base cost, separate from instruction preparation and any admission wait. GP0 busy status does not by itself prevent a GP1 store: GP1 admission is the system supervisor-quiesce gate. The reset and clear controls can therefore be issued while GP0 has an incomplete stream; they do not wait for idle.

### 5.5.1 GP1(00h) — reset GPU command and register state

**Word:** `0x00000000`; all payload bits are ignored.

Perform the GP0 stream clear described under GP1(01h), then restore these GPU register defaults:

| Programmed state | Reset value |
|---|---|
| Draw mode, texture window, drawing-area corners, drawing offset, mask mode | Zero |
| Display start | Zero |
| Display mode | `0x09` |
| Horizontal display range | `0x00c60260` |
| Vertical display range | `0x00044c23` |
| Display-disabled latch | One |
| DMA direction; GPU interrupt-request latch | Zero |

This is **not a full machine or display reset**. It preserves GPUREAD and the VRAM Y-address-extension latch, does not initialize local memory, and does not reset PCRTC registers or its beam timing. It does not acknowledge the system IRQ controller's pending GPU bit. A later GPUSTAT read is recomputed from these control values and current beam/command conditions; do not substitute a fixed status constant for that read.

### 5.5.2 GP1(01h) — clear the GP0 stream

**Word:** `0x01000000`; all payload bits are ignored.

Clear the GP0 FIFO, separate DMA ingress FIFO, incomplete fixed-packet/image header and polyline assembly, and cancel the pending command deadline. Drawing, display and DMA-direction registers are not reset. Neither GPUSTAT's interrupt request latch nor the system IRQ pending latch is acknowledged.

A clear is **not a rollback of every previously accepted operation**. An accepted pending raster/upload target is completed where present. If an A0 upload has already consumed at least one payload word, its accepted partial payload is flushed; a header-only upload is discarded. The unaccepted remainder is not manufactured. Do not resume that abandoned image payload as though its header were still active: subsequent GP0 words begin a new stream.

Readback cancellation depends on its phase:

| Readback boundary | Clear effect |
|---|---|
| C0 has not activated | Remove an unexecuted C0 at the executed-command prefix boundary |
| Readback pending | Remove C0 and following retained commands; preserve the prior command prefix |
| Readback submitted or READY | Clear retained command state and reset the readback stream |

Thus GP1(01h) can abandon a readback barrier; it is not equivalent to reading and consuming its remaining image words. Choose GP1(00h) instead only when the GPU register defaults are also wanted.

### 5.5.3 GP1(02h) — acknowledge the GPU request latch

**Word:** `0x02000000`; all payload bits are ignored.

Clear **GPUSTAT bit 24**. This does not clear the independent GPU pending bit in the system interrupt controller. The in-order GP0(1Fh) marker sets bit 24 and raises `IRQ_GPU`, bit 6 (`0x00000040`), only when the GPU request latch was clear; markers received while it remains set do not raise a fresh request. A complete acknowledgement therefore services the GPU latch with this command and the controller latch through its own acknowledgement register. This command is not an idle, VBlank or displayed-picture acknowledgement. Becoming idle does not itself raise `IRQ_GPU`.

### 5.5.4 GP1(03h) — set display-disable state

**Word:** `0x03000000 OR d`; **payload mask:** `0x000001`.

| d | Programmed GPU display state | GPUSTAT bit 23 |
|---|---|---|
| 0 | Enabled | 0 |
| 1 | Disabled | 1 |

All other payload bits are ignored. This controls the GPU display-disabled latch, not PCRTC's circuit enables or timing registerfiles. Its immediate status readback does not prove that a new displayed presentation has latched.

### 5.5.5 GP1(04h) — select DMA direction and request source

**Word:** `0x04000000 OR direction`; **payload mask:** `0x000003`.

| Direction | Selection | GPUSTAT bit 25, DMA request |
|---|---|---|
| 0 | Off | Zero |
| 1 | FIFO | DMA ingress empty and GP0 FIFO has room |
| 2 | CPU → GP0 | GPUSTAT bit 28, DMA packet acceptance |
| 3 | GPUREAD → CPU | GPUSTAT bit 27, readback READY |

GPUSTAT bits 30:29 return the selected direction. A system ingress stop suppresses the write-side requests for directions 1 and 2. Entering direction 3 enables readback DMA request generation; leaving it disables that generation. The command selects GPU-side direction/request behavior only: it does not program a DMA channel's addresses or count, start a channel transfer, or grant a CPU read ownership of a DMA-owned GP0 read port. In particular, direction 2's request source is not a general CPU-store-ready predicate.

### 5.5.6 GP1(05h) — set the GPU display-start word

**Word:** `0x05000000 OR start`; **payload mask:** `0x0fffe`.

The retained word is `start AND 0x000fffe`: bit 0 is discarded. Decode it as

- source X = `retained AND 0x3ff`, an even raw-word X address;
- source Y = `(retained >> 10) AND Ymask`, where GP1(09h) selects `Ymask` as `0x1ff` or `0x3ff`.

This is GPU display-start state. It neither writes PCRTC's DISPFB/DISPLAY registers nor restarts the beam. The stored high Y bit remains available when the Y gate is closed; changing the gate changes its decoded address rather than requiring that the stored start word be rewritten.

For example, `0x05ffffff` retains `0x000fffe`: X is 1022, and Y is 511 with the nine-bit gate or 1023 with the ten-bit gate. Program PCRTC's source layout and window separately for the required displayed picture.

### 5.5.7 GP1(06h) — retain horizontal-display-range state

**Word:** `0x06000000 OR range`; **payload mask:** `0xffffff`.

Retain all twenty-four payload bits as the GPU horizontal-display-range word. For example, `0x06ffffff` retains `0x00ffffff`. These are GPU control values, not writes to PCRTC's horizontal synchronization or reading-window registers; this command does not establish a physical pixel clock or beam interval.

### 5.5.8 GP1(07h) — set vertical-display-range state

**Word:** `0x07000000 OR range`; **payload mask:** `0x0fffff`.

| Retained bits | Field |
|---|---|
| 9:0 | Range start |
| 19:10 | Range end |

The retained word is `range AND 0x000fffff`; payload bits 23:20 are ignored. Start and end are stored without ordering validation: equal or reversed values are not silently replaced by a preferred range. This is not a write to PCRTC's vertical timing or DISPLAY window. `0x07ffffff` therefore stores `0x000fffff`, with both ten-bit fields equal to 1023, rather than selecting a maximum valid monitor mode.

### 5.5.9 GP1(08h) — set GPU display-mode fields

**Word:** `0x08000000 OR mode`; **payload mask:** `0x0000ff`.

All eight mode bits are retained. Their GPUSTAT mapping is:

| Mode bits | Field | GPUSTAT destination |
|---|---|---|
| 1:0 | Horizontal-resolution mode 1 | 18:17 |
| 2 | Vertical-resolution mode | 19 |
| 3 | PAL-mode flag | 20 |
| 4 | 24-bit display-color mode | 21 |
| 5 | Vertical-interlace mode | 22 |
| 6 | Horizontal-resolution mode 2 | 16 |

| Mode bits | Field | GPUSTAT destination |
|---|---|---|
| 7 | Retained raw bit | None; no reverse-image action |

A mode update clears GPUSTAT's bit-14 reverse mask; retained mode bit 7 is not copied there. For example, `0x08ffffff` retains mode `0xff`, sets GPUSTAT bits 22:16 to `0x7f`, and leaves bit 14 clear. Do not reinterpret that raw high bit as a horizontal-reversal command.

These GPU mode/status fields do not program PCRTC's clock, synchronization, reading-circuit layout or display window. The vertical-resolution/interlace flags also participate in the beam-derived GPUSTAT fields described in the status reference; they are not command-completion flags.

### 5.5.10 GP1(09h) — select the VRAM Y-address gate

**Word:** `0x09000000 OR extension`; **payload mask:** `0x000001`.

| Extension | Decoded Y bits | Ymask |
|---|---|---|
| 0 | Nine | `0x1ff` |
| 1 | Ten | `0x3ff` |

This selects address interpretation; it does not resize the installed local memory. It is a different latch from GP0 draw-mode's texture-page-Y-high bit. GP1(00h) preserves it; a full device reset clears it. Draw commands retain the Y-extension value when emitted, so a later gate change does not retroactively reinterpret already emitted drawing operands.

### 5.5.11 Opcode boundaries and display visibility

Opcodes `0x10`–`0x1f` are info-query aliases, not additional controls in this catalog; the payload's low four bits select the query. Other eight-bit opcodes have no selected control action and do not raise an unknown-opcode exception. An opcode whose low six bits resemble a listed command does not alias it.

GP1 control changes are visible immediately in programmed GPU state after due work synchronizes. Corresponding presented display/status/start/range and Y-extension values are copied at VBlank presentation, not by each GP1 write. GPUSTAT readback, an IRQ marker and GPU idle therefore do not acknowledge a newly displayed picture. PCRTC remains the separate interface for the actual reading circuits, windows and beam timing.

## 5.6 Query drawing state through GPUREAD

Write an information query to **GP1**, then obtain its result through **GP0/GPUREAD**. Reading GP1 instead returns GPUSTAT, not the query answer. All sixteen GP1 opcodes `10H–1FH` select the same query operation. The **payload's low four bits**, not the opcode's low four bits, select the requested result; payload bits 23:4 are ignored.

For example, `10000002H` and `1F000002H` both request selector `02H`. So does `1F345672H`: its opcode is `1FH`, but its selector is `2`. Changing the query opcode within that range does not change which register is requested.

| Payload selector | Value placed in GPUREAD |
|---|---|
| `02H` | Stored texture-window word, low 20 bits; mask `000FFFFFH` |
| `03H` | Stored drawing-area top-left word, low 20 bits; mask `000FFFFFH` |
| `04H` | Stored drawing-area bottom-right word, low 20 bits; mask `000FFFFFH` |
| `05H` | Stored drawing-offset word, low 22 bits; mask `003FFFFFH` |
| `07H` | Constant `00000002H` |
| `08H` | Constant `00000000H` |
| `00H,01H,06H,09H–0FH` | Preserve the previous GPUREAD latch |

The register answers are **raw stored words**, not expanded texture-window masks, separately sign-extended offset components or a displayed-picture snapshot. A supported query updates the shared GPUREAD latch immediately after already-due work synchronizes; it does not build a queue of answers. In particular, selector `00H` does not reset the GPU, and selector `01H` does not clear its stream: within a query they are unsupported selectors and leave the latch unchanged. An unsupported selector does not force zero or raise a query exception. Conversely, selector `08H` deliberately writes zero.

GPUREAD also carries image-readback data. With no READY readback, a GP0 read returns the current latch. With a READY stream, a permitted read instead consumes the next packed image word and **replaces** that latch. Issuing a query does not consume, cancel or reserve the readback stream, and does not make its answer take priority over image data.

Consider this sequence. Initially no readback is READY, and the stored texture-window word is `00010C62H`. Later, an already-requested readback becomes READY with three raw pixel words `001FH,03E0H,7C00H`. Throughout this example no DMA owns the GP0 read port, and no subsequent readback is queued:

| Action before the CPU's GP0 word read | Latch just after query | GP0 read returns | READY after read |
|---|---|---|---|
| Write GP1 `1F345672H` | `00010C62H` | `00010C62H` | 0 |
| Write GP1 `10000006H` | Unchanged | `00010C62H` | 0 |
| Readback becomes READY; write GP1 `10000007H` | `00000002H` | `03E0001FH` | 1 |
| No further query; read next word | — | `00007C00H` | 0 |
| No new query or readback; read again | — | `00007C00H` | 0 |

The third read returns two pixels, low halfword first, rather than the freshly queried constant. The next word has zero in its unused high half because only one pixel remains. Consuming that last word returns readback to idle and allows queued GP0 processing to resume. The final read then repeats the last GPUREAD value; it does not manufacture another pixel.

If DMA owns GP0's read port, a CPU read returns the current latch **without consuming** a ready image word; DMA reads consume the stream. Finish that DMA ownership before taking over image reads with the CPU. A query uses the same latch, so a permitted READY image read replaces its answer. Use word reads: even a byte read performs the port-read operation and can consume an entire packed image word.

These queries report programmed drawing state. They do not acknowledge VBlank presentation or prove that a newly displayed picture has latched. Keep state inspection, image-data consumption and display acknowledgement separate.

## 5.7 Render successive frames

Reserve two regions of shared local memory for drawing and display. While PCRTC reads the front buffer, draw the next picture into the back buffer. After drawing completes, program PCRTC to select the finished buffer and wait for the display latch before reusing the former front buffer. The regions are chosen by the program; a source switch changes the addresses being read.

| Step | Required work and completion |
|---|---|
| Choose storage | Keep the drawing destination separate from storage still needed by display, texture/CLUT reads, copies or readback. Preserve the firmware VRAM reservation. Match the raw drawing layout to the PCRTC source layout. |
| Submit the next picture | Complete every packet, upload payload and polyline terminator. Order changes to texture and palette storage with their uses; queued commands do not retain private copies of those pixels. |
| Finish GPU-side work | Drain any readback result and finish its downstream DMA writes when used. Observe GPU IDLE separately from DMA DONE or a GP0 IRQ marker. None of these acknowledges display publication. |
| Program the display source | Write the required PCRTC source/configuration words. They remain programmed words until the VBlank-entry latch; separate stores are not an atomic update. See Section 3.2.3. |

| Step | Required work and completion |
|---|---|
| Observe the new selection | Use a fresh-after-write VBlank observation. The Memory display-switch sequence gives the IRQ/counter setup and exact order. |

Before rebuilding a former display region, also finish its other uses: the second reading circuit, texture or palette sampling, copy sources and readback can still refer to it. Keep the pixels intact until all those uses have ended.

The two-surface example demonstrates A → B → rebuilt A with one reading circuit and a VBlank counter. It gives the interrupt masking, acknowledgement and fresh-edge sequence used to establish each display hand-off.

## 5.8 Hand a GP0 stream between CPU and DMA

The GP0, GPUREAD, GP1 and GPUSTAT reference gives the complete CPU admission tests, status fields and access-width rules. The additional DMA boundary matters even when both producers supply the same sequence of GP0 words.

### 5.8.1 A stalled CPU store is not an admitted DMA block

A direct scalar CPU store checks admission before writing. If blocked, that instruction waits and retries; its word has not entered the stream. A DMA transfer owning GP0's write port excludes direct CPU writes even if the physical GP0 FIFO has room. Do not use the FIFO's free space or GPUSTAT bit 28 as permission to interleave CPU words into that DMA-owned stream.

DMA writes first enter a **separate sixteen-word ingress FIFO**, not sixteen extra CPU FIFO slots. For an admitted DMA block:

1. Words before its `BLOCK_END` indication are staged without running the normal GP0 pipeline for those writes.
2. At block end, the GPU synchronizes already-due work and drains the staged words toward the GP0 command FIFO.
3. The GP0 parser interprets those words under its current packet phase, subject to command-completion and readback barriers.

The DMA write-ready check at this boundary tests whether ingress is stopped. The DMA controller owns block sizing and admission; that limited check is not permission to submit an arbitrarily large block to sixteen-word storage. Channel addresses, counts and transfer controls belong to the DMA interface, not an additional GPU register map.

### 5.8.2 A DMA block boundary is not a GP0 packet boundary

`BLOCK_END` is a transfer indication, not a GP0 opcode or polyline terminator. It does not supply missing packet fields, end an image payload or close an open polyline. The next supplied word still has the role required by the current GP0 command-stream phase. Conversely, a DMA hand-off is not an execution or pixel-completion fence: queued command intervals and a non-idle readback can still prevent the parser from progressing. Acceptance, packet completion and execution completion remain different events.

### 5.8.3 Keep ownership separate from READY and access width

The same distinction applies in the other direction: GPUSTAT bit 27 can be set while DMA owns GP0's read port. A CPU read then returns the existing GPUREAD latch without consuming image data; READY does not transfer ownership to the CPU. The query/readback reference describes that latch interaction.

A system ingress stop gates both direct CPU and DMA GP0 writes; GP1 also has its supervisor-quiesce gate. This is not another name for ordinary GPU busy status. Use the word operations specified under **Access and ordering** in the two-port reference: a byte read can consume a whole readback word, narrow writes are not partial command writes, and a contiguous CPU word sequence does not repeatedly address fixed GP0. DMA ownership does not redefine those CPU access side effects or turn either port into ordinary RAM.

## 5.9 Organize local memory for drawing and display

GPU local memory contains **2 MiB: 1,048,576 sixteen-bit words**. Divide this shared store among drawing destinations, textures, palettes and display sources. A **VRAM word** stores sixteen bits; a **GP0 input word** carries thirty-two bits. PCRTC decodes a **source sample** from local memory according to its selected storage format. Choose the drawing layout and display interpretation together before placing a framebuffer.

Raw GPU coordinates address rows of **1,024 words**. For an already decoded raw coordinate, the word index and byte offset within local memory are:

```
Word index  = (1024 × Y + (X AND 03FFH)) AND 000FFFFFH
Byte offset = 2 × word index
```

X wraps within its row. Thus `(1024,16)` addresses word `00004000H`, the same as `(0,16)`, not word `00004400H` at `(0,17)`. The final word-index mask wraps the installed memory. Distinguish that physical storage period from the **Y-address gate**: transfer Y, drawing-area Y, texture-page Y and palette-row Y decode with a nine-bit mask when the gate is closed, or a ten-bit mask when it is open. For a raw image transfer, logical `(32,512)` therefore selects word `00000020H` with the gate closed and `00080020H` with it open. Closing the gate does not remove the upper half of local memory.

Rendered RGB555 words place red in bits 4:0, green in 9:5 and blue in 14:10; bit 15 is stored separately. Eight-bit color components reduce to five bits by discarding their low three bits at storage conversion. Raw image data can instead supply complete sixteen-bit words. Within each stored word, the low byte comes first.

For a small placement example, arrange a two-by-two RGB555 tile at raw origin `(32,16)`:

| Raw coordinate | Word index | Byte offset | Stored word |
|---|---|---|---|
| (32,16) | `00004020H` | `00008040H` | `001FH` |
| (33,16) | `00004021H` | `00008042H` | `03E0H` |
| (32,17) | `00004420H` | `00008840H` | `7C00H` |
| (33,17) | `00004421H` | `00008842H` | `FFFFH` |

The first row is red then green; the second is blue then white. The white word has bit 15 set; that bit is not an extra blue-color bit. The second row starts **1,024 words**, or **2,048 bytes**, after the first—not immediately after its two pixels. These indices and offsets describe local-memory placement, not a GP0 packet or a display-window position.

For a framebuffer drawn by ordinary primitives, GX16 provides the corresponding linear display layout. PCRTC also supports page, block and column mappings described under **Local-memory address layout** in its chapter. To use one of those layouts, arrange the stored words for that mapping; primitive drawing continues to use the raw address calculation above. DISPFB's source origin and DISPLAY's window then select the visible samples.

## 5.10 Choose a primitive for the required image

Choose the primitive's coverage and attributes before constructing its packet. Supply screen positions after projection and clipping, and submit overlapping primitives in the intended drawing order. The following choices determine the packet fields and interpolation available for the image.

| Required operation | Primitive | Supplied attributes |
|---|---|---|
| Solid axis-aligned area | Untextured rectangle | One RGB color and an origin; fixed or variable dimensions |
| Axis-aligned texture image | Textured rectangle | One RGB color, initial U/V and dimensions; raw or modulated texture |
| Straight segment or connected segments | Line or polyline | One RGB color, or endpoint RGB colors for a color ramp; no texture lookup |
| Three-corner shape | Triangle | One RGB color or three vertex colors; optional U/V at each vertex |

| Required operation | Primitive | Supplied attributes |
|---|---|---|
| Four-corner shape | Quad | Four positions, with the corresponding flat/vertex-color and texture choices; executed as two triangles |

A rectangle has no four-corner color interpolation. Its variable width uses ten bits and its variable height uses nine bits: 0–1023 by 0–511. Either zero extent covers no samples. Fixed rectangles are 1×1, 8×8 or 16×16. These are drawing rectangle rules, not the raw image-transfer size encoding or fill operation.

### 5.10.1 A small area with explicit sample limits

Suppose the drawing area contains the entire region, the drawing offset is zero, no row is suppressed, and no texture, blending or destination protection removes or changes a write. A rectangle at `(40,24)` with width 4 and height 3 covers:

```
Y = 24:  X = 40, 41, 42, 43
Y = 25:  X = 40, 41, 42, 43
Y = 26:  X = 40, 41, 42, 43
```

Its right boundary is 44 and bottom boundary is 27; those boundaries are excluded. The result is twelve destination samples, not a five-by-four inclusive rectangle. A quad with vertices `(40,24)`, `(44,24)`, `(40,27)`, `(44,27)` describes the same rectangular coverage under these conditions, but takes a different arithmetic path:

| Triangle | Vertex indices | Positions |
|---|---|---|
| First | 0, 1, 2 | `(40,24)`, `(44,24)`, `(40,27)` |
| Second | 2, 1, 3 | `(40,27)`, `(44,24)`, `(44,27)` |

The common diagonal joins vertices 1 and 2. Each triangle applies its own coverage and attribute calculations. The edge rule assigns a normal shared diagonal to one half, but it does not prevent twice-written pixels when the two halves overlap. A crossed or overlapping four-corner input is not repaired into a nonoverlapping rectangle.

### 5.10.2 Color and texture choices are independent of the corner count

Flat color supplies one RGB value for the primitive. Vertex coloring supplies RGB at triangle or quad vertices, or at line endpoints. Triangle color and U/V attributes are affine functions of screen X/Y, evaluated with the specified integer precision. A quad therefore has two attribute planes, meeting at its chosen diagonal; it is not a bilinear blend of four corners. A shape with the same outline can have different interior colors when its diagonal or vertex attributes change.

Textured triangles and rectangles fetch their source words from GPU local memory. Raw texturing keeps the sampled RGB555 color; modulated texturing combines it with the supplied or interpolated RGB components. Lines have no texture lookup. The later texture and color sections give the actual window, palette, transparency, modulation and dither rules; choosing a textured primitive does not bypass them.

A line uses a different coverage rule from an area. An accepted unclipped segment with endpoint difference `(dx,dy)` generates `max(abs(dx),abs(dy))+1` samples, including its two endpoints; a zero-length segment generates one sample. Adjacent polyline segments can therefore write their common endpoint twice. That matters when blending or destination-mask protection makes a second write observable.

## 5.11 Place primitives across coordinate boundaries

A drawing coordinate uses **signed eleven-bit** X and Y values, each ranging from −1024 through 1023. Only the low eleven bits of each sixteen-bit coordinate component participate: `03FFH` means 1023, `0400H` means −1024, and both `07E0H` and `FFE0H` mean −32. Drawing-offset X uses bits 10:0 of its word; drawing-offset Y uses bits 21:11, with the same signed interpretation. The offset moves primitives, not the drawing area. Use each primitive's coordinate rule—not the raw VRAM X-address mask—to determine where it reaches that area.

Suppose the already-programmed drawing-offset word is `00008400H`: X field `0400H` supplies −1024 and Y field `0010H` supplies +16. Select a drawing area from `(0,0)` through `(1023,511)`, inclusive. Consider this triangle:

| Raw X / Y components | Supplied coordinate | Ordinary offset sum | Triangle coordinate after common shift |
|---|---|---|---|
| `07E0H / 0000H` | (−32,0) | (−1056,16) | (992,16) |
| `0020H / 0000H` | (32,0) | (−992,16) | (1056,16) |
| `0000H / 0020H` | (0,32) | (−1024,48) | (1024,48) |

Triangles first form the **ordinary integer sums**. Their inclusive span is tested before shifting or drawing-area clipping: `maxX−minX+1` must not exceed 1024, and `maxY−minY+1` must not exceed 512. This example spans 65 by 33 and passes. Separately on each axis, a minimum sum **below −1024** adds 2048 to all three coordinates on that axis; otherwise no shift is added. Here the X minimum is −1056, so every X receives 2048; Y receives no shift. The threshold is strict: a minimum of exactly −1024 does not trigger it. Positive results such as 1056 are not individually wrapped back into the signed range. The drawing area limits which part can produce pixels.

The span limit is not a clipping limit or a display-resolution rating. For example, X extremes −512 and 511 have inclusive span 1024 and pass the X test. Change the second extreme to 512 and the span becomes 1025: that triangle is rejected before clipping could hide the excess. The corresponding raw X components are `0600H`, `01FFH` and `0200H`. The size check applies to each triangle, including each half of a quad.

**Rectangles wrap their origin instead.** Decode the supplied origin, add the offset, then decode each sum again as signed eleven bits. With the same state, a rectangle anchored at `(−32,0)` starts at `(992,16)`, while one anchored at `(32,0)` starts at `(−992,16)`—not at the triangle's `(1056,16)`. A width of 16 extends the first X interval to `[992,1008)` and the second to `[−992,−976)`. The endpoint is not repeatedly wrapped as the rectangle extends. Variable rectangle widths are 0…1023 and heights 0…511; a zero extent covers nothing, rather than selecting a maximum extent.

**Lines wrap generated samples.** Their offset endpoint sums undergo the same span limits as triangles, but each generated sample is decoded as signed eleven bits before the drawing-area test. Do not wrap the endpoints first and generate a different line between them. For the two sums −1056 and −992 above, the ordinary X span is only 65. At the sample-conversion boundary, −1056 becomes 992, −1025 becomes 1023, −1024 stays −1024 and −992 stays −992. This is neither the triangle's common shift nor a rectangle's origin-only wrap; the line calculation determines which samples occur between its endpoints.

Drawing-area right and bottom fields are **inclusive**: `(0,0)` through `(1023,511)` permits coordinate tests `0≤X<1024` and `0≤Y<512`. X area fields use ten bits. Y area fields use nine bits with the Y-address gate closed and ten with it open. Thus raw top/bottom Y fields `0200H/03FFH` decode to `0/511` when closed but `512/1023` when open; this does not apply a blanket nine-bit wrap to supplied vertex Y. Reversed **decoded** area endpoints make that dimension empty: left 900 and right 100 do not exchange places or select a wrapped interval.

Row suppression is separate from this area. When the vertical-resolution and interlace status bits are both set and draw-to-displayed-field is clear, samples on the retained active-line parity are skipped; otherwise this parity rule skips no rows. It does not protect an entire displayed rectangle. The GP0 drawing-environment and GPU control references define how to program the named state used here.

## 5.12 Predict shared-edge and joint writes

Triangle coverage tests **integer pixel coordinates**, not half-pixel centers. After placement, its candidate bounds include the minimum X/Y and exclude the maximum X/Y, intersected with the drawing area. Both nonzero windings are accepted; reversing the winding does not by itself reject a triangle. A zero-area triangle produces no pixels.

For vertices `(x0,y0)`, `(x1,y1)`, `(x2,y2)`, calculate:

```
A = (x2−x0)×(y1−y0) − (y2−y0)×(x1−x0)
s = +1 when A > 0; −1 when A < 0
```

For each directed edge `v1→v2`, `v2→v0`, `v0→v1`, with endpoints `(ax,ay)` and `(bx,by)`, use:

```
E(X,Y) = s × ((X−ax)×(by−ay) − (Y−ay)×(bx−ax))
stepX  = s × (by−ay)
stepY  = −s × (bx−ax)
minimum = 0 if stepX > 0, or if stepX = 0 and stepY > 0
          1 otherwise
```

A candidate sample must satisfy `E≥minimum` for **all three edges**. An exact edge value of zero therefore belongs to some directed boundaries but not their opposites. Do not replace this rule with “include every boundary pixel.” A rectangle instead covers its clipped left/top inclusive and right/bottom exclusive; it does not run these triangle edge tests.

To make write counts visible, use flat untextured RGB bytes `(32,0,0)`, dithering off, and **enabled additive blending, selection 1**. The source quantizes to RGB555 word `0004H`. Start each case with the shown destination words at `0000H`; disable destination-mask rejection and force-mask, skip no row parity, and include X=0…63 and Y=0…31 in the drawing area. Each write then adds four to the five-bit red channel: one write gives `0004H`, two give `0008H`. Neither result saturates, and stored bit 15 remains zero.

Choose quad vertices `v0=(40,24)`, `v1=(46,24)`, `v2=(40,26)`, `v3=(46,26)`. The two triangles execute as `(0,1,2)` then `(2,1,3)`. Put `i=X−40`, `j=Y−24`; candidate samples have `0≤i<6`, `0≤j<2`. Both triangles have A=−12. Their three tests reduce to:

| Triangle (0,1,2) | Triangle (2,1,3) |
|---|---|
| `12−2i−6j ≥ 1` | `12−2i ≥ 1` |
| `2i ≥ 0` | `12−6j ≥ 1` |
| `6j ≥ 0` | `2i+6j−12 ≥ 0` |

The complete sample sets are:

| Y | First triangle: covered X | Second triangle: covered X |
|---|---|---|
| 24 | 40, 41, 42, 43, 44, 45 | None |
| 25 | 40, 41, 42 | 43, 44, 45 |

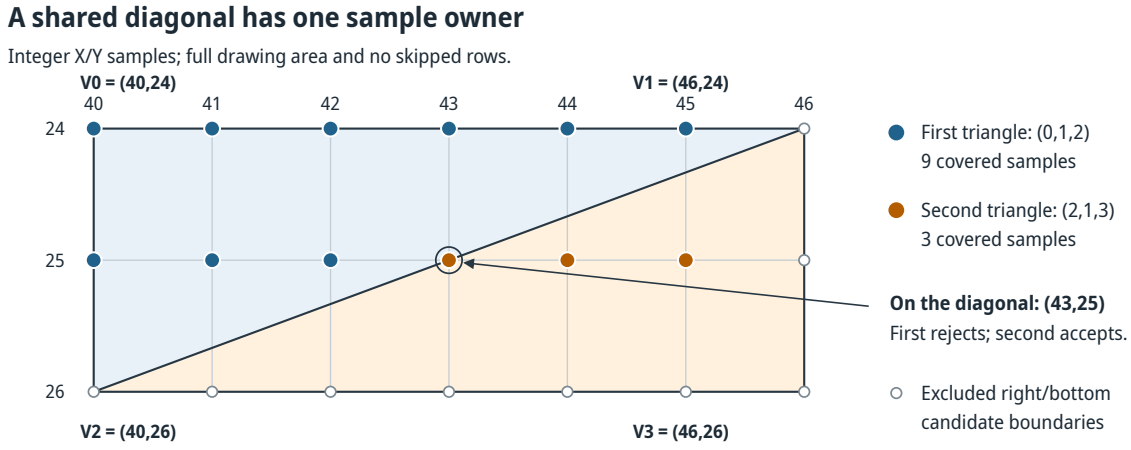



Figure 5.2: Integer sample ownership on a quad shared diagonal

All **twelve** pixels receive exactly one write and finish at `0004H`. Sample `(43,25)` is exactly on the diagonal: `2i+6j=12`. The first triangle requires its opposite edge value to be at least one, so excludes it; the second accepts zero, so owns it. It does not become `0008H`. Swapping the first triangle’s second and third vertices changes its area sign, but normalization preserves its nine covered samples. By contrast, collinear vertices `(40,24)`, `(43,25)`, `(46,26)` give A=0 and no triangle writes.

There is no once-only coverage operation over an arbitrary four-vertex outline. Move only `v3` to `(41,25)`: the first triangle is unchanged, while the second now has A=+4 and overlapping coverage. Its tests are `i+5j−6≥0`, `i+j−2≥0`, `12−2i−6j≥1`; only `(41,25)` and `(42,25)` pass within its bounds. Both already belong to the first triangle, so each finishes at **`0008H`**. The first

triangle's other seven samples finish at `0004H`; `(43,25)` is now untouched at `0000H`. Overlapping halves can blend twice even though an ordinary shared diagonal is owned once.

A polyline joint is different again. Two accepted, unclipped segments `(40,30)`→`(43,30)`→`(46,30)` each include **both endpoints** and generate four samples. The shared endpoint `(43,30)` is consequently written by both segments and finishes at `0008H` under the same state; either outer endpoint finishes at `0004H`. A zero-length accepted line generates one sample, rather than following the triangle's zero-area rule. Plan additive or other repeated-write effects around the primitive's actual coverage, not around its apparent outline.

### 5.12.1 See shared edges and repeated writes

Additive blending makes repeated writes visible. This example clears to black and uses the brighter drawing colour RGB=(96,64,32), rather than the preceding red material. One write stores VRAM word `110CH`; two writes store `2218H`.



Figure 5.3: Shared polygon edges, overlapping triangles, and a repeated polyline endpoint

*Actual 256 × 192 output. Bright regions identify two additive writes, not a different drawing colour.*

| Region | Geometry | What to observe |
|---|---|---|
| Upper left | Quad vertices (16,24), (112,24), (16,56), (112,56) | The 96 × 32 rectangle is written once throughout. Its shared diagonal is not doubled. |
| Upper right | Quad vertices (144,24), (240,24), (144,56), (160,40) | Moving only the fourth vertex produces a bright overlap wedge. Black parts of the 96 × 32 comparison area remain uncovered. |
| Lower left | Twenty-four polylines at Y=112…135, each X=16 → 64 → 112 | These actually drawn rows form a 97 × 24 band. Both segments include X=64, producing the one-pixel-wide bright joint. |

The polygon vertex differences are 16 times those of the preceding numerical example. These larger polygons are rasterized afresh; the picture is not an enlargement of its small pixel result. The upper-left result is not a guarantee that an arbitrary quad writes every covered position only once: the upper-right case shows why.

#### 5.12.1.1 Submit the coverage examples

Use the 256×192 drawing environment from the first programming sequence, with destination-mask checking and force-mask disabled. Set E1=`0420H` for additive blending, dither off and both row parities. Send these complete GP0 packets to clear the picture and draw the two quads:

```
rodata drawing_commands: word[] = {
    0x02000000, 0, 256 | (192 << 16),
    0x2a204060,
    16 | (24 << 16), 112 | (24 << 16),
    16 | (56 << 16), 112 | (56 << 16),
    0x2a204060,
    144 | (24 << 16), 240 | (24 << 16),
    144 | (56 << 16), 160 | (40 << 16),
}
*gp0 = 0xe1000420
for index = 0, #drawing_commands - 1 do
    *gp0 = drawing_commands[index]
end
for y = 112, 135 do
    *gp0 = 0x4a204060
    *gp0 = 16 | (y << 16)
    *gp0 = 64 | (y << 16)
    *gp0 = 112 | (y << 16)
    *gp0 = 0x50005000
end
```

The last word terminates each polyline at its coordinate boundary. Complete the stream and wait for GPU idle before enabling its PCRTC source; use the separate display-publication sequence, not an extra drawing delay.

## 5.13 Predict interpolated colors and texture coordinates

Triangle interpolation supplies a byte value for each covered pixel. Gouraud color uses three separate planes, one for each RGB component. Textured triangles use two planes for U and V. Flat color uses its supplied RGB bytes without a color plane; raw texturing ignores supplied RGB for the output. Coverage is a separate calculation: the attribute's rounding bias does not move a triangle's edges.

Each plane is affine in the final screen X/Y coordinates. There is no Z or reciprocal-depth term, texture filtering or mip-level selection in this sample path. Supplying projected vertices therefore does not make the U/V calculation perspective-correct. Software must choose geometry and texture coordinates that produce the required approximation.

### 5.13.1 Form a finite plane, not an ideal interpolator

Let the triangle's shifted coordinates be `(x0,y0)`, `(x1,y1)`, `(x2,y2)` and let one supplied byte attribute be `a0`, `a1`, `a2`. Use the nonzero determinant:

```
D = (x1−x0)×(y2−y0) − (x2−x0)×(y1−y0)
Nx = (a1−a0)×(y2−y1) − (a2−a1)×(y1−y0)
Ny = (x1−x0)×(a2−a1) − (x2−x1)×(a1−a0)
```

The X/Y step quotients are `trunc(4096×Nx/D)` and `trunc(4096×Ny/D)`. Here `trunc` discards a fractional part **toward zero**, including for a negative quotient. Retain only the low twenty bits of each quotient. Decode each retained step as signed twenty-bit two's complement when forming the intercept:

```
raw step range       = 00000H…FFFFFH
signed interpretation = raw, if bit 19 is clear
                      = raw − 100000H, if bit 19 is set
```

Thus the raw twenty-bit plane retains twelve fractional bits. A large derivative wraps in that field; it is not saturated to a plausible color slope. For example, a quotient of 983040 has raw value `F0000H` and signed interpretation −65536, not a saturated positive step.

The intercept is anchored at one supplied vertex. The selection depends on X and its exact tie rules:

| Condition | Anchor |
|---|---|
| `x1≤x0` and `x2≤x1` | 2 |
| `x1≤x0` and `x2>x1` | 1 |
| `x1>x0` and `x2<x0` | 2 |
| `x1>x0` and `x2≥x0` | 0 |

Using the signed retained steps `sx`, `sy`, calculate:

```
intercept = (4096×aAnchor + 2048 − xAnchor×sx − yAnchor×sy)
            AND FFFFFH
value(X,Y) = ((intercept + X×sx + Y×sy) >> 12) AND FFH
```

Only bits 19:12 supply the final byte. Attribute overflow therefore wraps rather than clamping to 0 or 255. Adding 2048 biases the **attribute** by one half unit: one half of a color count or texel-coordinate count, not one half screen pixel. The quantized steps and anchor determine rounding at each pixel, as the following half-count example shows.

### 5.13.2 A half-count that the quantized slope does not reach

Use zero drawing offset and these positions and attributes:

| Vertex | X | Y | Attribute a |
|---|---|---|---|
| 0 | 0 | 0 | 10 |
| 1 | 7 | 0 | 250 |
| 2 | 0 | 6 | 11 |

The determinant is 42. X1 exceeds X0 while X2 equals X0, so the anchor is **vertex 0**, not vertex 2. The arithmetic is:

| Quantity | Exact expression | Retained value |
|---|---|---|
| X step | `983040/7 = 140434 + 2/7` | 140434, raw `22492H` |
| Y step | `4096/6 = 682 + 2/3` | 682, raw `002AAH` |
| Intercept | `10×4096+2048` | 43008, raw `0A800H` |

At `(0,3)`, the ideal unquantized attribute is 10.5. The actual retained plane instead evaluates `43008+3×682=45054`, two raw counts below `11×4096=45056`. Its byte is therefore **10**, not 11. Moving along that row gives:

| Sample (X,3) | Raw plane expression | Extracted byte |
|---|---|---|
| (0,3) | 45054 | 10 |
| (1,3) | 185488 | 45 |
| (2,3) | 325922 | 79 |
| (3,3) | 466356 | 113 |

These four integer positions pass this triangle's edge tests when the drawing area contains them and no row suppression is active. Other pixel-write controls still apply after attribute evaluation. If a is a Gouraud color component, its byte proceeds to the color/texture calculation and eventual RGB555 conversion; if it is U or V, the byte proceeds to texture-window replacement and lookup. It is not itself a stored framebuffer word.

Using vertex 2 as an arbitrary substitute anchor would give intercept 43012 and byte 11 at `(0,3)`: the difference is observable. The exact anchor rule is part of the arithmetic, not a choice left to software.

For the signed contrast, retain the same positions but use attributes `(250,10,249)`. The quotients become −140434 and −682, not −140435 and −683; raw steps are `DDB6EH` and `FFD56H`. Anchor 0 gives intercept `FA800H`. At `(0,3)` its expression is 1024002 and its byte is 250. Do not replace the specified toward-zero derivative quotients with downward-rounded division.

A quad evaluates these rules separately for triangles `(0,1,2)` and `(2,1,3)`. The common diagonal's coverage ownership does not turn their attribute planes into one bilinear four-vertex interpolator.

## 5.14 Predict line samples and color ramps

Start with the ordinary offset endpoint sums after the line's span check. If the first X exceeds the second, exchange **both endpoint coordinates and their RGB colors**. Do not exchange colors independently of their positions. Equal-X vertical lines retain the supplied order; there is no corresponding Y-order swap.

After this ordering, let `ΔX=x1−x0`, `ΔY=y1−y0`, and `N=max(abs(ΔX),abs(ΔY))`. The line generates samples numbered 0 through N, including both endpoints of an accepted unclipped segment. For N=0, all steps are zero and one sample uses the first supplied color. Consecutive polyline segments perform this calculation separately, so their common endpoint can be processed twice.

**Position equations—mathematical notation.** Q32 denotes thirty-two fractional bits, not a thirty-two-bit total word width. Let `S=2^32`, and let `trunc0` mean signed division rounded toward zero. For N>0:

```
dX = trunc0((ΔX×S + sign(ΔX)×(N−1)) / N)
dY = trunc0((ΔY×S + sign(ΔY)×(N−1)) / N)
```

Here `sign` is −1, 0 or +1. These steps round a nonintegral step magnitude **outward**, not to nearest. The initial Q32 positions are:

```
PX(0) = x0×S + S/2 − 1024
PY(0) = y0×S + S/2 − bY
bY    = 1024 when dY < 0; otherwise 0
PX(k) = PX(0) + k×dX
PY(k) = PY(0) + k×dY
```

The bias 1024 is measured in Q32 units: it is `2^−22` of a pixel, not 1,024 pixels. Convert each position by the mathematical floor of `P/S`, including downward rounding for negative values. Then decode the resulting X/Y as signed eleven-bit coordinates and apply drawing-area and row-parity tests. This floor conversion differs from the `trunc0` used to obtain steps.

Color uses a separate **twelve-fractional-bit** calculation. For each RGB byte C, N>0 gives:

```
dC      = trunc0((C1−C0)×4096 / N)
Cfixed(k) = C0×4096 + 2048 + k×dC
Cbyte(k)  = trunc0(Cfixed(k) / 4096)
```

For N=0, dC is zero. The half-unit color bias and truncated color step are distinct operations; replacing them with an ideal color interpolation rounded once at the end can change a tie. The resulting RGB bytes then undergo eligible dithering and RGB555 reduction. Lines use the draw-mode dither bit for **both flat and Gouraud color**, unlike the more restricted polygon rule. The following source-color calculations have dithering disabled; they do not include later blending or destination protection.

For a Gouraud line, supply `(14,5)` with RGB `(20,200,40)`, followed by `(10,12)` with RGB `(220,20,180)`. The X-order swap makes the calculation start at `(10,12)` with its associated color. Now `ΔX=4`, `ΔY=−7`, N=7:

```
dX = 2454267027 = 92492493H Q32 units
dY = −4294967296 = −100000000H Q32 units
(dR,dG,dB) = (−117028,105325,−81920) Q12 units
```

The steep, negative-Y line produces:

| k | Sample (X,Y) | Interpolated RGB bytes | RGB555 source word |
|---|---|---|---|
| 0 | (10,12) | (220,20,180) | `585BH` |
| 1 | (11,11) | (191,46,160) | `50B7H` |
| 2 | (11,10) | (163,71,140) | `4514H` |
| 3 | (12,9) | (134,97,120) | `3D90H` |
| 4 | (12,8) | (106,123,100) | `31EDH` |
| 5 | (13,7) | (77,149,80) | `2A49H` |
| 6 | (13,6) | (49,174,60) | `1EA6H` |
| 7 | (14,5) | (20,200,40) | `1722H` |

If the drawing area admits only rows 7 through 10 and odd rows are suppressed, only k=2 and k=4 pass those tests. They retain source words `4514H` and `31EDH`. Clipping and row suppression do **not** restart position or color progression: the first admitted sample is not assigned the original starting color, and N is not recomputed for the surviving portion.

Equal-X order can be observable at a color tie. Supply `(20,20)` with RGB `(7,64,128)` and `(20,26)` with `(8,64,128)`. N=6 and the red step is `trunc0(4096/6)=682`. At the middle sample `(20,23)`, red fixed value is `7×4096+2048+3×682=32766`, giving byte 7. Reverse the two complete endpoint/color pairs: equal X prevents a swap, the red step becomes −682, and the same middle coordinate has `8×4096+2048−3×682=32770`, giving byte 8. With dithering off, the RGB555 source words are respectively **`4100H` and `4101H`**. Keep this specified order and precision when reproducing a vertical ramp; geometric reversal is not a promise of identical quantized interior colors.

## 5.15 Locate texture texels and palette colors

Texture U and V are **unsigned eight-bit coordinates**. Apply the texture window first; the resulting U/V select a texel within its page. Page X is measured in **raw sixteen-bit words**, not texels: draw-mode bits 3:0 select X in units of 64 words. Page Y combines bit 4 in units of 256 rows and bit 11 in units of 512 rows, then passes through the Y-address gate. Depth bits 8:7 determine how many texels occupy each raw word:

| Depth field | Texels per raw word | Source-word X | Selection from that word |
|---|---|---|---|
| 0: palette4 | 4 | `pageX + (U>>2)` | Nibble `(word>>(4×(U AND 3))) AND 0FH` |
| 1: palette8 | 2 | `pageX + (U>>1)` | Byte `(word>>(8×(U AND 1))) AND FFH` |
| 2: direct16 | 1 | `pageX + U` | Complete sixteen-bit color word |
| 3: direct16 | 1 | `pageX + U` | Same direct lookup as depth 2 |

Source-word Y is `pageY+V`. A 256-texel row therefore occupies 64, 128 or 256 raw words; a 64-word page-base unit does not mean every depth occupies only 64 words. All lookups use raw 1,024-word rows: X wraps within its row, and the installed word-index mask is `000FFFFFH`.

Palette depths use the extracted index for a second local-memory read. The CLUT attribute supplies base X `(attribute AND 003FH)×16` and base Y `attribute>>6`, with Y passed through the address gate. Palette4 has sixteen possible indices; palette8 has 256. The CLUT base is sixteen-word-aligned in either case, so a palette8 lookup can cross X=1023 and wrap to X=0 **on the same palette row**.

### 5.15.1 A packed palette8 lookup across both row ends

Use already-selected page/depth fields `0000089FH`, an **open Y-address gate**, texture-window fields `00000000H`, and CLUT attribute `E13FH`. These are field values, not complete command packets. They decode to page `(960,768)`, depth 1, an identity window, and CLUT base `(1008,900)`. Suppose local memory contains:

| Purpose | Raw coordinate | Word index | Stored word |
|---|---|---|---|
| Packed indices | (1023,791) | `000C5FFFH` | `100FH` |
| Packed indices | (0,791) | `000C5C00H` | `1100H` |
| Palette entry | (1023,900) | `000E13FFH` | `001FH` |
| Palette entry | (0,900) | `000E1000H` | `03E0H` |
| Palette entry | (1008,900) | `000E13F0H` | `7C00H` |
| Palette entry | (1,900) | `000E1001H` | `0000H` |

For V=23, source Y is `768+23=791`. Four lookups produce:

| U | Source-word index | Selected byte/index | CLUT word index | Fetched color word |
|---|---|---|---|---|
| 126 | `000C5FFFH` | `0FH` | `000E13FFH` | `001FH` |
| 127 | `000C5FFFH` | `10H` | `000E1000H` | `03E0H` |

| U | Source-word index | Selected byte/index | CLUT word index | Fetched color word |
|---|---|---|---|---|
| 128 | 000C5C00H | 00H | 000E13F0H | 7C00H |
| 129 | 000C5C00H | 11H | 000E1001H | 0000H |

U=126 selects the low byte of 100FH; U=127 selects its high byte. U=128 advances the raw source X from 1023 to 0, not to the next row. Independently, palette index 16 advances CLUT X from base 1008 to wrapped X=0. Index zero returns the nonzero color word 7C00H: **palette index zero is not automatically transparent**. Index 17 returns the zero color word, which is transparent. These are fetched source colors, before later color and destination-write controls.

With the Y gate closed, the same page fields select page Y=256 and the same CLUT attribute selects row 388. The reads use different words; the depth and palette-index range are unchanged. Depth 3 is also deterministic: with page/depth fields 0000099FH, the same page base and U=63,V=23 directly return 100FH from (1023,791). That word is a color, not two palette indices. Depth 2 performs the same direct lookup.

Program a complete picture in BLua supplies the complete palette4 route: typed readonly colors and packed indices, separate raw uploads, T0 CLUT selection and T1 palette-depth/page selection. Its result includes a nonzero entry 0, a transparent entry and a nonzero black entry, so palette-index interpretation can be distinguished from fetched-color rules.

### 5.15.2 Replace coordinate bits with a texture window

The E2 window fields are X mask bits 4:0, Y mask bits 9:5, X offset bits 14:10 and Y offset bits 19:15. Independently for U and V, the mathematical transformation is:

```
A = NOT(mask×8) AND FFH
O = (offset AND mask)×8
windowed coordinate = (coordinate AND A) OR O
```

Only masked offset bits participate; the low three coordinate bits are preserved. This is bit replacement, not clipping to a pixel rectangle or adding an ordinary offset. For raw window value 00010C62H, the four fields are (2,3,3,2): U uses AND/OR masks EFH/10H, and V uses E7H/10H. Input (U,V)=(6FH,1FH) becomes (7FH,17H), or (127,23). Reusing the example's page and CLUT, it selects the high byte 10H from 100FH and fetches color word 03E0H. Both the texture window and the palette placement must therefore be included when predicting a texture read.

## 5.16 Keep a clipped or flipped rectangle on its texture

A textured rectangle anchors its initial U/V at its **unclipped destination origin**. X-flip independently selects a U step of −1 instead of +1; Y-flip selects a V step of −1 instead of +1. Neither bit changes the destination rectangle. It also does not move the texture anchor automatically to the opposite edge: choose the initial U/V for the traversal you intend.

For already-placed destination origin (X0,Y0) and supplied byte coordinates (U0,V0), the mathematical texture-coordinate calculation is:

```
U = U0 + (X−X0)×stepU
V = V0 + (Y−Y0)×stepV
stepU = −1 with X-flip set; otherwise +1
stepV = −1 with Y-flip set; otherwise +1
```

Use the original destination coordinates in these differences, not the number of pixels which survive clipping. The texture window then transforms the stepped U/V; clipping does not restart them at U0/V0.

Consider a rectangle covering X=10…17 and Y=20…25 before clipping. Let the drawing area remove its first three columns and first two rows, leaving X=13…17 and Y=22…25. Use initial U0=2,V0=1. The first surviving coordinate is therefore three U steps and two V steps from the anchor—not a new texture origin.

Select these texture-window fields: X mask=1, Y mask=3, X offset=0, Y offset=1. They give U AND/OR masks F7H/00H and V masks E7H/08H. The first surviving pixel (13,22) demonstrates independent flip choices:

| X-flip | Y-flip | Stepped U/V before window | Windowed U/V |
|---|---|---|---|
| 0 | 0 | (5,3) | (05H,0BH) |
| 1 | 0 | (−1,3) | (F7H,0BH) |
| 0 | 1 | (5,−1) | (05H,EFH) |
| 1 | 1 | (−1,−1) | (F7H,EFH) |

The negative values are traversal intermediates, not negative command U/V bytes. The window consumes the eight-bit coordinate domain through its bit replacement. For example, an identity window would map −1 to FFH; the selected U window instead gives `(-1 AND F7H) OR 00H = F7H`. Do not substitute raw VRAM's 1,024-word X-row wrap for this eight-bit UV transformation. The texture placement rules determine the later raw-word address.

Keep **both flip bits set**. Across X=13,14,15,16,17, stepped U is −1,−2,−3,−4,−5 and windowed U is F7H,F6H,F5H,F4H,F3H. Each row starts with this same first U; it does not continue from the end of the preceding row.

For row suppression, assume the GPU vertical-resolution and interlace status bits are both set, draw-to-displayed-field is clear, and the retained active-line parity is odd. Odd destination rows are then skipped:

| Destination Y | Stepped V | Windowed V | Row qualification |
|---|---|---|---|
| 22 | −1 | EFH | Admitted |
| 23 | −2 | EEH | Skipped |
| 24 | −3 | EDH | Admitted |
| 25 | −4 | ECH | Skipped |

The next admitted row after Y=22 uses EDH, **not EEH**: skipped Y=23 still advances V. Its first lookup coordinate is (F7H,EDH), while Y=22 starts at (F7H,EFH). These are texture-lookup inputs; later source-color and destination-write rules determine the stored result. Neither drawing-area clipping nor row suppression compresses the texture into fewer rows. The rectangle flip bits do not reverse a triangle's interpolated U/V.

## 5.17 Choose texture gain and dither deliberately

Choose **raw texturing** when the fetched texture color itself is the desired source RGB555. The supplied RGB color does not tint that result, and raw texturing bypasses dithering. Choose **modulated texturing** to multiply the texture's five-bit channels by supplied RGB bytes; a Gouraud polygon uses its interpolated RGB bytes at each pixel. These bytes are gain controls, not replacement texture colors. In particular, 80H (128), not FFH (255), is exact neutral gain with dithering off.

A fetched texture word of 0000H is skipped before either calculation. A nonzero word, including black 8000H, proceeds. The words calculated below specify the **source RGB555**, before later blending, destination protection and stored-bit-15 selection; they are not unconditional final-memory results.

For each modulated channel, let T be the texture component (0…31), C the supplied or interpolated byte (0…255), and d the eligible dither offset, or zero with dithering off. The mathematical calculation is:

```
P = (T × C) >> 4            P is in 0…494
S = P + d
F = 0                      if S < 0
F = min(S >> 3, 31)         otherwise
```

Both right shifts discard low bits. Keep the two stages and their order: do not expand the five-bit texture channel by bit replication, clamp P to a byte before adding d, or add d after producing F. Saturation belongs after the dither addition.

For texture word 6208H, RGB5=(8,16,24), the following choices give different material behavior. Dithering is off:

| Supplied RGB bytes | Intermediate P for R/G/B | Source RGB5 | Source word |
|---|---|---|---|
| (128,128,128) | (64,128,192) | (8,16,24) | 6208H |
| (64,128,192) | (32,128,288) | (4,16,31) | 7E04H |
| (255,255,255) | (127,255,382) | (15,31,31) | 7FEFH |

The second choice halves red, preserves green and saturates blue: blue reaches 36 before the limit of 31. The third is not neutral white modulation. Selecting raw texturing instead preserves source `6208H` for all three supplied colors.

Dither uses the **final destination coordinate**, with the following offset at row `Y AND 3`, column `X AND 3`:

| Y phase | X phase | 0 | 1 | 2 | 3 |
|---|---|---|---|---|---|
| 0 | | −4 | 0 | −3 | +1 |
| 1 | | +2 | −2 | +3 | −1 |
| 2 | | −3 | +1 | −4 | 0 |
| 3 | | +3 | −1 | +2 | −2 |

The same offset is added separately to all three components. It is not indexed by U/V or restarted at a primitive's origin. Moving an otherwise identical sample to a different destination phase can therefore change its source RGB555. Dither acts before five-bit reduction and blending; it is not a PCRTC display effect.

Take an eligible modulated textured polygon with dithering enabled. Suppose its samples fetch texture word `3527H`, RGB5=(7,9,13), with RGB bytes=(124,100,160). The first products reduce to P=(54,56,130):

| Destination / setting | d | S for R/G/B | Source RGB5 | Source word |
|---|---|---|---|---|
| Dithering off | 0 | (54,56,130) | (6,7,16) | `40E6H` |
| (18,21), phase (2,1) | +3 | (57,59,133) | (7,7,16) | `40E7H` |
| (16,20), phase (0,0) | −4 | (50,52,126) | (6,6,15) | `3CC6H` |

Neutral gain 128 does not cancel these offsets. At the lower limit, a nonzero fetched word `0001H` with RGB bytes=(1,0,0) gives P=(0,0,0); offset −4 clamps every source channel to zero. That resulting black source does **not** trigger the earlier fetched-zero skip.

Finally, the draw-mode dither setting is not universal. It applies to modulated textured polygons, untextured Gouraud polygons, and both flat and Gouraud lines. Flat untextured polygons, raw-textured polygons, and all rectangles do not dither. Without texturing, eligible dithering adds d to each RGB byte, clamps to 0…255, then discards three bits to obtain RGB555. Thus a modulated textured rectangle can use the same gain calculation, but not the polygon's dither variation.

### 5.17.1 Compare polygon dither with a rectangle control

Use the four panels below to separate polygon dithering from texture sampling. Every panel uses texture material `3527H`, whose RGB5 channels are `(7, 9, 13)`, modulated by RGB bytes `(124, 100, 160)`. The two columns differ in the dither control: OFF on the left, ON on the right. Dither is draw-mode bit 9 (`0200H`).

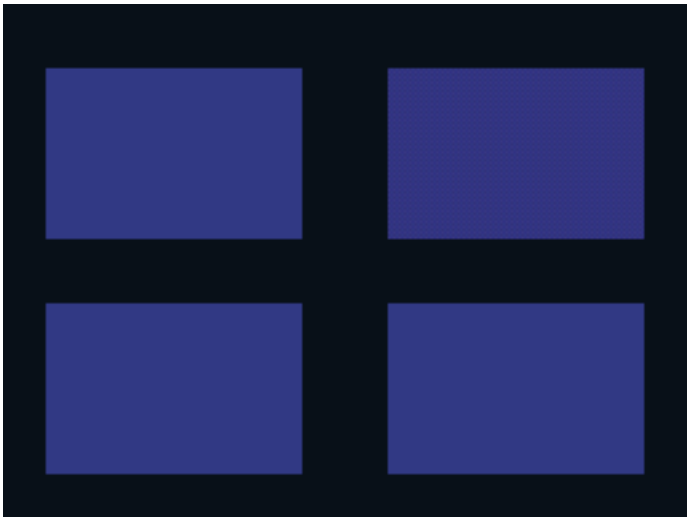


Figure 5.4: Four panels comparing textured quads and textured rectangles with dither off and on.

| Position | Primitive | Dither control | Destination start |
|---|---|---|---|
| Upper left | Modulated textured quad | OFF | `(16, 24)` |
| Upper right | Modulated textured quad | ON | `(144, 24)` |
| Lower left | Modulated textured rectangle | OFF | `(16, 112)` |
| Lower right | Modulated textured rectangle | ON | `(144, 112)` |

In the displayed picture, only the upper-right panel has grain; the other three panels are uniform. Both quads give every vertex UV `(0, 0)`, so they repeatedly sample the single texture word at `(640, 0)`. Each rectangle instead advances one texel per destination pixel and row, traversing the entire initialized `96 × 64` texture patch. That patch contains `3527H` throughout, so advancing UV does not introduce a texture pattern.

The distinction is the primitive's dither behavior. A modulated textured polygon honors the enabled dither bit. A textured rectangle does not apply dither, even with that bit enabled. Dither is therefore not a universal effect on all drawing commands.

Without dither, modulation produces RGB5 `(6, 7, 16)`, stored as `40E6H`. The upper-right panel instead stores the following raw VRAM words at its top-left:

| Y / X | 144 | 145 | 146 | 147 |
|---|---|---|---|---|
| 24 | `3CC6H` | `40E6H` | `3CC6H` | `40E6H` |
| 25 | `40E7H` | `40C6H` | `40E7H` | `40C6H` |
| 26 | `3CC6H` | `40E6H` | `3CC6H` | `40E6H` |
| 27 | `40E7H` | `40C6H` | `40E7H` | `40C6H` |

These are stored RGB555 words, not displayed-picture RGB bytes. The dither offsets come from a `4 × 4` matrix indexed by destination coordinates. For these particular texture and color operands, quantization maps those offsets into the repeated `2 × 2` word pattern shown here. It does **not** make the GPU's dither matrix universally `2 × 2`.

Use the GP0 port and 256×192 drawing environment from the first programming sequence, with mask controls cleared and E1=`0400H` initially. The complete packets below first fill the source texture uniformly, then draw all four panels. No image decoder or filtered sampler is involved.

```
rodata drawing_commands: word[] = {
    0x02181008, 0, 256 | (192 << 16),
    0x02684838, 640, 96 | (64 << 16),
    -- Modulated quad, dither off; every UV selects the same texel.
    0x2ca0647c,
    16 | (24 << 16), 0,
    112 | (24 << 16), 0x010a << 16,
    16 | (88 << 16), 0,
    112 | (88 << 16), 0,
    -- The same quad with draw-mode bit 9 set.
    0xe100070a,
    0x2ca0647c,
    144 | (24 << 16), 0,
    240 | (24 << 16), 0x010a << 16,
    144 | (88 << 16), 0,
    240 | (88 << 16), 0,
    -- Rectangle UVs advance across the uniform patch.
    0xe100050a,
    0x64a0647c, 16 | (112 << 16), 0, 96 | (64 << 16),
    0xe100070a,
    0x64a0647c, 144 | (112 << 16), 0, 96 | (64 << 16),
}
for index = 0, #drawing_commands - 1 do
    *gp0 = drawing_commands[index]
end
```

After submission, use the ordinary GPU-completion and PCRTC-publication sequence. The rectangle panels remain uniform despite the enabled dither bit; changing the texture page does not change which primitive classes apply dither.

## 5.18 Separate texture transparency, blending and destination protection

Four decisions act at different stages. A fetched texture word of **exactly `0000H`** skips the pixel. The command's semitransparency selection enables a possible blend. For textured drawing, fetched **source bit 15** must also be one for that blend to occur. Finally, mask-mode controls can reject a write using the **old destination bit 15**, or force the **new output bit 15**. These are not interchangeable meanings of transparency.

For an untextured primitive, its semitransparency selection enables blending of eligible covered pixels without a texture-bit condition. For a textured primitive, a nonzero fetched word with bit 15 clear remains opaque even under a semitransparent command. Setting source bit 15 alone is insufficient: an opaque command still replaces the destination RGB. Neither source nor destination bit 15 supplies a fractional alpha coefficient.

When blending is enabled, let F be one already-quantized five-bit source channel and B the current five-bit destination channel. The draw-mode selection chooses the following calculation independently for red, green and blue:

| Blend selection | Result channel |
|---|---|
| 0: half sum | `(B + F) >> 1` |
| 1: saturated addition | `min(B + F, 31)` |
| 2: saturated subtraction | `max(B − F, 0)` |
| 3: quarter-source addition | `min(B + (F >> 2), 31)` |

The right shifts discard low bits. In selection 0, add **before** halving: B=5,F=3 gives 4, not `(5>>1)+(3>>1)=3`. An odd sum such as 2+7 gives 4. In selection 3, reduce the source **before** adding it; F=3 contributes zero, while F=7 contributes one.

For destination RGB5=(5,2,29), RGB word `7445H`, and source RGB5=(3,7,7), RGB word `1CE3H`, all four choices are distinguishable:

| Selection | Result RGB5 | Result RGB word, excluding bit 15 |
|---|---|---|
| 0 | (4,4,18) | `4884H` |
| 1 | (8,9,31) | `7D28H` |
| 2 | (2,0,22) | `5802H` |
| 3 | (5,3,30) | `7865H` |

Addition clips blue's 29+7 to 31. Subtraction clips green's 2−7 to zero; it does not wrap. Quarter-source addition instead adds (0,1,1), preserving those integer truncations.

The following independent raw-texture samples all start with destination `7445H` and use blend selection 0. Check-mask is off. "Force" means mask-mode bit 0, which sets the output bit 15 without changing blend eligibility:

| Fetched word | Semitransparent command | Force | Effect and resulting stored word |
|---|---|---|---|
| `0000H` | Yes | 1 | Skipped; destination stays `7445H` |
| `8000H` | Yes | 0 | Black source blends; stores `B822H` |
| `8000H` | No | 0 | Opaque black; stores `8000H` |
| `1CE3H` | Yes | 0 | Source bit 15 clear: opaque; stores `1CE3H` |
| `9CE3H` | No | 0 | Command does not blend; stores `9CE3H` |
| `9CE3H` | Yes | 0 | Both blend conditions met; stores `C884H` |
| `1CE3H` | Yes | 1 | Still opaque; forced output stores `9CE3H` |

`8000H` is therefore not the transparent word: it is nonzero black. In the blended row its RGB becomes (2,1,14), while the output bit 15 comes from the fetched sample. Conversely, force-mask cannot turn `0000H` into a write or turn a bit-15-clear texel into a blended source.

**Check-mask**, mask-mode bit 1, tests the destination's old bit 15 before blending or storing. If that bit is one, the complete write is suppressed, including any forced output-bit change. Starting instead with destination `F445H`, check-mask leaves `F445H` unchanged, whether force-mask is zero or one. With both controls clear, an opaque write of `1CE3H` replaces it and clears its old bit 15.

To mark newly drawn pixels and protect them on later checked draws, enable both controls: mask-mode low bits=`3`. Starting at `7445H`, a raw `1CE3H` sample passes the check and stores `9CE3H`. A second checked draw then fails against that newly set bit. Clearing check-mask permits overwriting it again; force-mask does not override protection.

On an admitted write, output bit 15 is one if forced. Otherwise textured drawing copies the fetched bit, untextured drawing writes zero, and raw upload/copy preserves its source bit. Blended RGB never determines that output bit, and the old destination bit is not implicitly retained. Raw upload/copy also uses these check/set controls, but not the drawing blend calculation.

## 5.19 Choose fill, drawing or a raw transfer

A fill, a drawing rectangle and an image transfer can touch a rectangular region, but they do not have the same dimensions, address rules or write controls. Choose **fill** for an aligned solid-color memory operation; choose a **drawing rectangle** for the drawing-area, offset and pixel-write rules; choose **upload or VRAM copy** to move raw sixteen-bit storage words.

For a variable drawing rectangle, width is the low ten size bits and height the low nine: zero is empty. Fill uses those same low-bit fields but rounds width upward to a multiple of 16 and aligns destination X downward to a multiple of 16. Upload, copy and readback instead normalize each dimension with:

```
W = ((rawWidth  − 1) AND 3FFH) + 1
H = ((rawHeight − 1) AND 1FFH) + 1
```

Thus these raw width fields have different meanings, before clipping or write suppression:

| Raw width | Variable drawing rectangle | Fill | Upload/copy/readback |
|---|---|---|---|
| 0 | Empty | Empty | 1024 |
| 1 | 1 | 16 | 1 |
| 16 | 16 | 16 | 16 |
| 1023 | 1023 | 1024 | 1023 |

Zero height likewise means empty for fill/drawing, but 512 rows for an image transfer. A zero image-size word requests **1024×512**, not no work. Fixed 1×1, 8×8 and 16×16 drawing rectangles have no variable size word.

The write controls are also different:

| Operation | Drawing offset/area | Check-mask and force-mask | Retained skipped-row parity |
|---|---|---|---|
| Fill | Bypassed | Bypassed | Applied |
| Drawing rectangle | Applied | Applied | Applied |
| Raw upload/copy | Bypassed | Applied | Bypassed |

Fill truncates its RGB bytes to RGB555 and writes bit 15 clear; it does not blend or dither. Raw upload/copy preserves the supplied RGB bits and source bit 15 unless mask controls reject the write or force bit 15. It does not blend or dither. Drawing rectangles follow their color/texture path, but neither kind dithers. Their origin follows the drawing-coordinate rule; transfer X wraps within a 1,024-word row and transfer Y follows the selected address gate. Do not substitute transfer wrapping for drawing placement.

For a small control contrast, start both `(31,20)` and `(31,21)` at `8010H`, use zero drawing offset, an including drawing area, and retained suppression of odd rows. Each following operation starts from those same contents:

| Operation and controls | Word at (31,20) | Word at (31,21) |
|---|---|---|
| Red fill, raw width=1,height=2; check and force set | `001FH` | `8010H` |
| Untextured red drawing rectangle, width=1,height=2; check and force set | `8010H` | `8010H` |

| Operation and controls | Word at (31,20) | Word at (31,21) |
|---|---|---|
| Upload 001FH,0000H, width=1,height=2; check and force set | 8010H | 8010H |
| Same upload; check clear, force set | 801FH | 8000H |

Red here is supplied RGB=(255,0,0). The fill also writes X=16…30 on admitted row 20: its requested X=31 aligns to 16. It ignores both mask controls, but skips row 21. Drawing cannot replace the protected row-20 destination, while upload ignores row suppression but still obeys destination protection. With checking disabled, upload writes even its zero source word and forces the stored high bit.

**Horizontal overlap is handled per row.** A copy traverses columns backward when `((targetX−sourceX) AND 3FFH)` is nonzero and less than W; otherwise it traverses forward. With both mask controls clear, copy width=3 from X=1022 to X=1023 on one row. In column order X=(1022,1023,0,1), let the before-words be `(0011H,0022H,0033H,0044H)`. The reads run X=0,1023,1022 and write X=1,0,1023, producing `(0011H,0011H,0022H,0033H)`. The X wrap stays in that row.

**Vertical overlap is not a rectangle snapshot.** Rows always advance in logical order, reading each source word immediately before its write. Select the 512-row Y gate and clear both mask controls. Copy 3×3 from `(1023,510)` to `(1023,511)`; the three columns wrap to X=(1023,0,1):

| Physical Y | Before words at X=(1023,0,1) | After words |
|---|---|---|
| 510 | 0011H,0022H,0033H | 0011H,0022H,0033H |
| 511 | 0044H,0055H,0066H | 0011H,0022H,0033H |
| 0 | 0077H,0088H,0099H | 0011H,0022H,0033H |
| 1 | 00AAH,00BBH,00CCH | 0011H,0022H,0033H |

The first row writes 510→511. The second reads that newly written row 511 and writes it to row 0; the third reads the newly written row 0 and writes row 1. Retained row suppression does not prevent these raw-copy writes. To preserve the original rows in this shift, copy logical rows 2,1,0 as separate one-row commands, or use a nonoverlapping intermediate region; do not expect a hidden snapshot.

Upload data is row-major raw words, packed low halfword first in each GP0 payload word. A 3×3 upload needs five payload words: only the last high halfword is unused, with no padding after each odd-width row. Supply the counted payload, not an end-image marker. Readback reports raw storage words, not PCRTC-converted or composed display pixels.

## 5.20 Frame the GP0 command stream

Supply GP0 input as complete **32-bit words**. A command's first word selects its operation with bits 31:24; bits 23:0 have the meaning assigned by that command. Later words belong to the selected packet until its required fields or payload have been consumed. Their high bytes are not independently decoded as new opcodes.

### 5.20.1 Select the family before interpreting option bits

The complete opcode-byte map is:

| Opcode byte | Operation family |
|---|---|
| 02H | Local-memory fill |
| 1FH | GPU interrupt request |
| 20H–3FH | Polygons |
| 40H–5FH | Finite lines or streamed polylines |
| 60H–7FH | Rectangles |
| 80H–9FH | Local-memory copy |
| A0H–BFH | Image upload to local memory |
| C0H–DFH | Local-memory readback |
| E1H–E6H | Drawing environment |

Every opcode within a listed range selects that family. The option bits do not have one universal meaning:

| Family | Opcode bits within that family |
|---|---|
| Polygon | Bit 4 selects Gouraud colors; bit 3 selects a quad rather than a triangle; bit 2 enables texture; bit 1 requests semi-transparency; bit 0 selects raw rather than modulated texture when textured |
| Line | Bit 4 selects Gouraud colors; bit 3 selects a streamed polyline; bit 1 requests semi-transparency. Bits 2 and 0 do not insert texture fields |
| Rectangle | Bits 4:3 select the size form; bit 2 enables texture; bit 1 requests semi-transparency; bit 0 selects raw rather than modulated texture when textured |

In particular, rectangle bit 4 does not request Gouraud colors. Bit 0 does not create a raw-color mode for an untextured primitive. Use the selected family's word list to determine the exact packet length; do not infer it from an option bit borrowed from another family.

At a **command boundary**, opcode bytes `00H`, `04H–1EH`, `E0H` and `E7H–EFH` are discarded NOPs without a command-cost interval. The remaining unlisted values—`01H`, `03H` and `F0H–FFH`—are one-word ignored operations with a one-command-tick cost, not rejected opcodes. `01H` does not perform a texture-cache operation. These rules do not turn matching high bytes inside a packet into NOPs or new commands.

A packed word's role also determines its numerical interpretation. RGB consumes three unsigned bytes in bits 7:0, 15:8 and 23:16. Primitive positions consume signed eleven-bit X/Y fields within their two halfwords. Image addresses instead use raw local-memory coordinates, including the selected Y-address mask. The common 32-bit container does not make these representations interchangeable.

### 5.20.2 Finish the current packet before starting another

A fixed packet waits for its exact field count. An image-upload header selects a counted pixel payload: words in that payload remain image data even when they look like opcodes or polyline terminators. If a field is omitted, the next word is consumed in its place—even when that word looks like the start of another command.

A polyline has a different boundary. Its initial two-endpoint header contains three words for flat color or four for Gouraud color. Subsequent flat vertices supply one position each; Gouraud vertices supply a color followed by a position. Each complete new vertex defines another segment from the retained previous endpoint. The initial segment and later complete segments need not wait for the terminator before executing.

After that header, the polyline terminator is any word satisfying:

```
(word AND F000F000H) = 50005000H
```

Test this only where a new continuation vertex begins: the next position for a flat polyline, or the next **color** for a Gouraud polyline. A matching word in a Gouraud continuation's position slot is position data, not a terminator. The terminator adds no segment. Without it, the stream remains a polyline; an unfinished Gouraud color/position pair remains unfinished.

For example, supply this Gouraud polyline followed by a drawing-environment command. The high byte `58H` selects the line family with both Gouraud and polyline bits set:

| Word | Raw input | Role in the stream |
|---|---|---|
| 1 | `58000020H` | Command and first RGB `(32,0,0)` |
| 2 | `00180028H` | First position `(40,24)` |
| 3 | `E10000FFH` | Second RGB `(255,0,0)`; **not** an E1 command |
| 4 | `0018002EH` | Second position `(46,24)`; completes the initial header |
| 5 | `0000FF00H` | Continuation RGB `(0,255,0)` |
| 6 | `001A002EH` | Continuation position `(46,26)`; completes the next segment |
| 7 | `5ABC5DEFH` | Terminator at the next continuation-color boundary |

| Word | Raw input | Role in the stream |
|---|---|---|
| 8 | `E1000000H` | A new E1 drawing-environment command |

Word 3 belongs to the header, so its `E1H` high byte cannot change drawing state. Word 7 qualifies because masking it gives `50005000H`; the other bits need not be zero. If that same word occupied word 6's position slot, it would not end the polyline. Only after a legal termination can word 8 be interpreted as a new command.

Finally, **word framing is not execution timing**. A positive-cost command of N ticks starting at system cycle C has an execution-completion deadline of `C + (N/2 rounded upward)`; zero-cost work completes immediately. Queued execution continues from the prior completion deadline, not from a later observation. These intervals do not tell how many words a packet contains or the cost of preparing and supplying them.

## 5.21 Flat untextured polygons — GP0(20H–23H,28H–2BH)

Use a flat untextured polygon to cover a triangle or quad with **one supplied RGB color**. There is no texture lookup or per-vertex color interpolation. "Flat" describes the color input, not an axis-aligned shape.

### 5.21.1 Opcode and packet

| Opcode byte | Shape | Color operation | Total words |
|---|---|---|---|
| `20H`, `21H` | Triangle | Opaque | 4 |
| `22H`, `23H` | Triangle | Semi-transparent | 4 |
| `28H`, `29H` | Quad | Opaque | 5 |
| `2AH`, `2BH` | Quad | Semi-transparent | 5 |

Bit 3 selects four rather than three vertices; bit 1 requests semi-transparency. Bit 0 is ignored for these untextured polygons: it neither changes the color calculation nor adds a field. Bits 4 and 2 are clear in this entry's variants; setting them selects Gouraud or textured polygon formats instead.

Supply these thirty-two-bit words to GP0 in order:

| Word | Triangle | Quad |
|---|---|---|
| 1 | `(opcode << 24) OR C` | `(opcode << 24) OR C` |
| 2 | `P0` | `P0` |
| 3 | `P1` | `P1` |
| 4 | `P2` | `P2` |
| 5 | — | `P3` |

`C` contains red in bits 7:0, green in 15:8 and blue in 23:16. Each is an unsigned byte. Every `Pi` contains X in bits 10:0 and Y in bits 26:16, decoded as signed eleven-bit coordinates, −1024…1023. The remaining five bits of each halfword do not extend that range. One direct packing equation is:

```
C = R OR (G << 8) OR (B << 16)
P = (X AND 07FFH) OR ((Y AND 07FFH) << 16)
```

For example, X=−32 has low field `07E0H`; a sign-extended component `FFE0H` decodes to the same X, not a different sixteen-bit position. See Place primitives across coordinate boundaries for drawing-offset and placement rules.

There is no second color, U/V word, size word or terminator. The last required position completes the packet; the following word begins the next command. Omitting a position does not shorten the packet: the next word is consumed as that missing position even if its high byte resembles another opcode.

### 5.21.2 Drawing state and geometry

The quad executes triangles **(0,1,2), then (2,1,3)**, both using C. Supply the corners for that split, not an arbitrary perimeter order. Each triangle has its own size, area and coverage tests. Overlapping halves can write the same pixel twice; the GPU does not repair the four positions into a nonoverlapping outline.

The packet uses the current drawing environment without replacing its values. At draw emission it retains the draw mode, texture window, drawing bounds, drawing offset, mask mode, Y-address gate and row-suppression parity. Later changes do not reinterpret that emitted draw. No texture-page field is present in this packet. The GP0 packet also has no Z, W or GTE operation: projection, lighting, any three-dimensional clipping and draw ordering must already have provided the desired positions, color and sequence.

Apply the named **coordinate placement** and **shared-edge** rules, rather than assuming a bounding rectangle is filled. In particular:

- Each triangle's inclusive source span must fit 1024 by 512; larger triangles are rejected before drawing-area clipping can hide the excess.
- Accepted triangles receive the common coordinate shift described under Place primitives across coordinate boundaries, then drawing-area clipping. The drawing area's programmed right and bottom limits are inclusive.
- A zero-area triangle writes nothing. Both nonzero windings are accepted; reversing winding is not a backface-culling request.
- Coverage tests integer samples with the shared-edge tie rule. Mask protection or retained row suppression can subsequently remove otherwise covered writes.

For this family, a set dither-enable bit does **not** dither the color. Drop three low bits from each RGB byte to obtain the five-bit source channels. Opaque variants replace the eligible destination RGB. Semi-transparent variants blend that quantized source using the draw-mode blend selection; no texture bit is needed to enable their blend. Destination-mask checking can suppress an entire write, and force-mask controls stored bit 15 separately from RGB.

### 5.21.3 Complete packet and exact result

Assume the drawing area is `(0,0)` through `(63,31)`, the drawing offset is zero, no row parity is suppressed, and destination protection and force-mask are off. Select additive blend mode 1; leave dither enabled to demonstrate that it does not affect this flat family. Before each trial below, initialize the affected destination words to `1882H`, RGB5=(2,4,6). These are programmed-state and memory preconditions, not extra fields in the polygon packet.

This opaque quad supplies RGB bytes (124,100,160):

| Word | Raw input | Meaning |
|---|---|---|
| 1 | `28A0647CH` | Opaque flat untextured quad; C=`A0647CH` |
| 2 | `00180028H` | P0=(40,24) |
| 3 | `0018002EH` | P1=(46,24) |
| 4 | `001A0028H` | P2=(40,26) |
| 5 | `001A002EH` | P3=(46,26) |

The source channels are `(124>>3,100>>3,160>>3)`=(15,12,20), so the stored RGB word is `15 OR (12<<5) OR (20<<10)` = **`518FH`**. Bit 15 is zero. The opaque opcode ignores the selected additive blend operation, and enabled dithering does not vary this result between destination phases.

Using the sample ownership derived in Predict shared-edge and joint writes:

| Y | Triangle (0,1,2) writes X | Triangle (2,1,3) writes X |
|---|---|---|
| 24 | 40,41,42,43,44,45 | None |
| 25 | 40,41,42 | 43,44,45 |

All twelve samples finish at `518FH`; other samples remain unchanged. The on-diagonal sample (43,25) belongs only to the second triangle. For a triangle instead, change word 1 to `20A0647CH` and omit word 5: only the first triangle's nine samples are written.

In a separate trial from the same `1882H` background, change only word 1 to `2AA0647CH`. Additive blending gives RGB5=(17,16,26), hence **`6A11H`** at each of the twelve samples. The shared diagonal does not receive a second addition. `29H` is

equivalent to `28H`, and `2BH` to `2AH`, because bit 0 is ignored. For an opaque trial with force-mask enabled the result is `D18FH`; if instead destination protection is enabled and an old destination is `9882H`, that entire old word remains unchanged.

### 5.21.4 Command cost

Flat untextured setup is **46 command ticks for a triangle, 82 for a quad**, plus one geometric work estimate for each triangle in its split. For timing, use ordinary decoded vertex-plus-offset sums, not the later raster common coordinate shift. An oversized source span or empty drawing area contributes zero work. Otherwise clamp each vertex to the inclusive drawing bounds and start with `p = absolute-double-area >> 1`. If semi-transparency **or** destination-mask checking is enabled, add `(p+1)>>1`; if a row parity is suppressed, halve the resulting estimate by a right shift. This is an estimate, not a count of covered or successfully stored pixels.

Neither clipping nor those modifiers apply in the opaque packet's example. Each triangle has absolute double area 12 and contributes p=6:

| Variant under the example's state | Tick calculation | Command interval |
|---|---|---|
| Opaque triangle | `46+6=52` | 26 system cycles |
| Opaque quad | `82+6+6=94` | 47 system cycles |
| Semi-transparent quad | `82+9+9=100` | 50 system cycles |

The first triangle actually covers nine samples and the second three, although their work estimates are both six. An in-area collinear triangle contributes zero work but still has its 46-tick setup. Convert ticks to the execution completion deadline using Frame the GP0 command stream. These intervals do not include CPU packet preparation or input-store waits, and do not acknowledge local-memory presentation or display completion.

## 5.22 Gouraud untextured polygons — GP0(30H–33H,38H–3BH)

Use this family when each vertex supplies its own RGB color and no texture lookup is required. The GPU interpolates three color planes across each triangle, quantizes the resulting RGB bytes to RGB555, then applies the selected pixel-write rules.

| Opcode bytes | Primitive | Blend request | Packet words |
|---|---|---|---|
| `30H,31H` | Triangle | Off | 6 |
| `32H,33H` | Triangle | On | 6 |
| `38H,39H` | Quad | Off | 8 |
| `3AH,3BH` | Quad | On | 8 |

Within each pair, bit 0 has no color-mode effect because texturing is absent. Bit 4 selects Gouraud color, bit 3 selects the quad, bit 2 is clear, and bit 1 requests semitransparency. Do not insert texture or CLUT words.

The complete word order is:

```
Triangle: (opcode<<24) OR C0, P0, C1, P1, C2, P2
Quad:     (opcode<<24) OR C0, P0, C1, P1, C2, P2, C3, P3
```

This is packet notation, not cartridge-language code. Each C contains R in bits 7:0, G in 15:8 and B in 23:16, as unsigned bytes. Only the first word's high byte selects an opcode; bits 31:24 of C1–C3 are ignored as color data, even when they resemble a drawing-environment command. The next word after the sixth/eighth starts a new command; there is no terminator.

Each P consumes signed eleven-bit X in bits 10:0 and Y in 26:16, range −1024…1023. The remaining five bits in each halfword do not extend its coordinate. Drawing offsets, per-triangle span rejection, triangle coordinate shifting, drawing-area clipping and retained row suppression follow Place primitives across coordinate boundaries. A quad uses triangles (0,1,2) then (2,1,3), with each half constructing its own color planes; it is not four-corner bilinear shading.

The calculation follows Predict interpolated colors and texture coordinates: derivatives are determinant quotients truncated toward zero, retained as twenty-bit words with twelve fractional bits. Signed twenty-bit derivatives form the biased intercept; readout extracts bits 19:12 as the RGB byte. Attribute overflow wraps in that finite field rather than clamping to a byte. Do not replace it with a fresh floating-point interpolation. Gouraud untextured polygons are eligible for draw-mode dithering; after any eligible dither, byte saturation and the three-bit reduction produce the five-bit blend source.

### 5.22.1 Worked six-word triangle

Use zero drawing offset, inclusive drawing area X=40…44,Y=24…28, no skipped-row parity, dithering off and mask-mode low bits zero. Supply these GP0 words in order:

| Word | Raw word | Operand |
|---|---|---|
| 1 | `30C8280AH` | Triangle, opaque; C0 RGB=(10,40,200) |
| 2 | `00180028H` | P0=(40,24) |
| 3 | `E12878FAH` | C1 RGB=(250,120,40) |
| 4 | `0018002CH` | P1=(44,24) |
| 5 | `E250C832H` | C2 RGB=(50,200,80) |
| 6 | `001C0028H` | P2=(40,28) |

Words 3 and 5 do **not** execute E1/E2 commands. For this triangle, determinant D=16 and the plane anchor is vertex 0. The plane construction gives these raw twenty-bit quantities:

| Channel | X-step word | Y-step word | Intercept |
|---|---|---|---|
| R | `3C000H` | `0A000H` | `BA800H` |
| G | `14000H` | `28000H` | `48800H` |
| B | `D8000H` | `E2000H` | `D8800H` |

The blue steps represent −40×4096 and −30×4096. These intercepts include the half-color-unit bias, not a half-pixel coverage shift. At covered pixel `(41,25)`, plane evaluation gives `50800H,64800H,82800H`; extracting bits 19:12 yields RGB=(80,100,130). With dithering off, reduction gives RGB5=(10,12,16), packed `418AH`. The opaque, unchecked, unforced write replaces an existing `FFFFH` there with **`418AH`**, clearing its old bit 15.

The command-work setup is `46+288=334` ticks. Its unclipped determinant gives estimated work `16>>1=8`, so total cost is **342 command ticks**, or 171 scheduled system cycles. Selecting `32H` or `33H` instead requests blending and makes the work estimate `8+((8+1)>>1)=12`, for 346 ticks/173 cycles with the other state unchanged. Its stored RGB then follows the configured blend calculation, not the opaque result above. These intervals exclude CPU packet preparation/issue and do not count exact stored pixels.

### 5.22.2 State and result boundary

The command retains draw mode, the Y-address-extension value, texture window, drawing bounds/offset, mask mode and skipped-row parity when emitted. Later state changes do not reinterpret that retained command. This untextured packet contains no environment update or texture-page selection. Semitransparent variants blend the five-bit source; mask checking can reject the write, and force-mask determines whether untextured output bit 15 is set instead of zero.

For general costs, the Gouraud setup is 334 ticks per triangle command or 370 per quad command, plus the separate triangle work estimates described in Budget command work separately from issuing words. Timing uses ordinary signed-eleven-bit position-plus-offset sums, not the raster triangle shift.

A GTE result is not an extra packet field: supply the resulting screen XY and RGB in the listed words. The first color word still needs the selected GP0 opcode in its high byte; a packed color result is not already a GPU command. Projection, lighting, near-plane clipping and depth-order selection precede this primitive; it has no Z/W input, perspective correction or depth comparison.

## 5.23 Flat textured polygons — GP0(24H–27H,2CH–2FH)

These commands supply one RGB color and a U/V pair at each vertex. “Flat” means one supplied color, **not** one texture coordinate or one fetched texel. U/V still use the finite affine planes described in Predict interpolated colors and texture coordinates. Each covered sample performs one point lookup.

### 5.23.1 Opcode and complete packet

| Triangle opcode | Quad opcode | Texture color | Semi-transparency request |
|---|---|---|---|
| 24H | 2CH | Modulated by supplied RGB | Off |
| 25H | 2DH | Raw fetched RGB555 | Off |
| 26H | 2EH | Modulated by supplied RGB | On |
| 27H | 2FH | Raw fetched RGB555 | On |

Bit 3 selects a quad, bit 2 is set for texture, bit 1 requests semi-transparency, and bit 0 selects raw rather than modulated texture. Unlike untextured polygon bit 0, it is meaningful here. Bit 4 is clear: no per-vertex colors are supplied.

| Word | Triangle: seven words | Quad: nine words |
|---|---|---|
| 1 | (opcode << 24) OR C | (opcode << 24) OR C |
| 2 | P0 | P0 |
| 3 | T0 | T0 |
| 4 | P1 | P1 |
| 5 | T1 | T1 |
| 6 | P2 | P2 |
| 7 | T2 | T2 |
| 8 | — | P3 |
| 9 | — | T3 |

C contains unsigned R/G/B bytes in bits 7:0/15:8/23:16. Each P contains signed eleven-bit X in bits 10:0 and Y in 26:16; the other five bits of each halfword do not extend the coordinate. Each T contains unsigned U in bits 7:0, V in 15:8, and an attribute in 31:16. Raw texture still requires the complete packet, including C and every T; ignoring C for color does not remove its word.

There is no terminator. T2, or T3 for a quad, finishes the packet. A quad uses triangles (0,1,2), then (2,1,3), carrying each vertex's own UV. The coordinate, size-rejection and shared-edge rules are the same as in Flat untextured polygons; an arbitrary four-corner outline is not repaired into a different split.

### 5.23.2 T0 selects the palette; T1 changes persistent drawing state

T0's upper half is the CLUT attribute:

```
CLUT base X = (attribute AND 003FH) × 16 raw words
CLUT base Y = (attribute >> 6), decoded through the Y-address gate
```

T1's upper half supplies these drawing-mode fields:

| Attribute bits | Meaning |
|---|---|
| 3:0 | Texture-page base X in units of 64 raw words |
| 4 | Add 256 to texture-page base Y |
| 6:5 | Blend selection |
| 8:7 | Texture depth: 0 palette4, 1 palette8, 2 or 3 direct sixteen-bit |
| 11 | Add 512 to texture-page base Y |

The page Y result is decoded through the Y-address gate. Other T1 attribute bits do not participate in the update:

```
new draw mode = (old draw mode AND NOT 09FFH)
                OR (T1 attribute AND 09FFH)
```

This changes the **persistent** draw mode before taking this polygon's own state snapshot. Bits 9,10,12,13 survive: dither enable, draw-to-displayed-field and rectangle X/Y flips are not supplied anew by T1. T2/T3 upper halves are ignored; their low U/V bytes remain required. None of these words starts an independent GP0 command, even if its high byte resembles E1.

The emitted polygon retains its resulting draw mode and the current texture window, area, offset, mask mode, Y gate and skipped-row parity. Later state changes do not reinterpret it. Conversely, its T1 page update remains current for later commands. A textured rectangle's T0 selects its CLUT, **not** a new page: it inherits the current page, depth, blend selection and flip settings.

After U/V plane extraction, apply texture-window replacement and the lookup rules in Locate texture texels and palette colors. Palette4 selects a nibble from a packed word, palette8 selects a byte, and depths 2/3 fetch a direct word. Both packed-source X and CLUT X wrap within their respective raw rows. A palette index of zero is not intrinsically transparent; the fetched color word decides.

### 5.23.3 Worked seven-word triangle

Use zero drawing offset, identity texture window, an open Y-address gate, drawing area `(0,0)` through `(63,31)`, no row suppression and mask mode zero. Initially program draw mode **`3742H`**. Its dither, draw-to-displayed-field and both rectangle-flip bits are set; its old page/depth/blend fields will change. Initialize the destination region used below to **`0C41H`**, RGB5=(1,2,3).

Declare these existing raw local-memory contents before drawing. They are source words, not an upload packet or a suggested loading API:

| Raw coordinate | Sixteen-bit source word |
|---|---|
| (61,Y), for every Y=773…776 | `1514H` |
| (62,Y), for every Y=773…776 | `1716H` |
| (61,772) | `1415H` |
| (4,500) | `6208H` |
| (5,500) | `E208H` |
| (6,500) | `0000H` |
| (7,500) | `8000H` |

No sampled source coordinate is left unspecified, and these source regions do not overlap the destination. Supply this complete polygon packet:

| Word | Raw input | Meaning |
|---|---|---|
| 1 | `26C08040H` | Modulated, semi-transparent triangle; RGB=(64,128,192) |
| 2 | `00180028H` | P0=(40,24) |
| 3 | `7D3F05FAH` | T0: U0=250,V0=5; CLUT attribute=`7D3FH` |
| 4 | `0018002CH` | P1=(44,24) |
| 5 | `08BF05FEH` | T1: U1=254,V1=5; page attribute=`08BFH` |
| 6 | `001C0028H` | P2=(40,28) |
| 7 | `E10009FAH` | T2: U2=250,V2=9; ignored upper half=`E100H` |

The CLUT attribute gives base (1008,500). T1 selects page (960,768), palette8 and additive blend selection 1. The merge is `(3742H AND NOT 09FFH) OR 08BFH = 3EBFH`; preserved bits are `3600H`. Word 7 does **not** overwrite this state with an E1 command.

The UV planes have exact steps of one texel per destination pixel: at covered integer samples, U=`250+(X-40)` and V=`5+(Y-24)`. The inherited rectangle flips do not reverse polygon planes. On row Y=24 the lookups are:

| Destination X | U,V | Packed raw coordinate | Selected index | CLUT coordinate | Fetched word |
|---|---|---|---|---|---|
| 40 | 250,5 | (61,773) | 20 | (4,500) | `6208H` |

| Destination X | U,V | Packed raw coordinate | Selected index | CLUT coordinate | Fetched word |
|---|---|---|---|---|---|
| 41 | 251,5 | (61,773) | 21 | (5,500) | `E208H` |
| 42 | 252,5 | (62,773) | 22 | (6,500) | `0000H` |
| 43 | 253,5 | (62,773) | 23 | (7,500) | `8000H` |

For X=40, `960+(250>>1)=1085`, which wraps to raw X=61 without changing row 773. Its low byte selects index 20. Then `1008+20=1028` wraps to CLUT X=4 without changing row 500. The packed `1514H` holds indices 20 and 21, not two RGB555 colors.

Sample `6208H` has RGB5=(8,16,24). Modulation by (64,128,192) first gives products `(32,128,288)`. At (40,24), dither −4 gives source RGB5=(3,15,31), word **`7DE3H`**. This fetched sample has bit 15 clear, so it stays opaque even though the command requests semi-transparency. At (41,24), fetched `E208H` has the same RGB but bit 15 set: dither zero gives source (4,16,31), additive blending over (1,2,3) gives (5,18,31), and retained sample bit 15 makes the final word **`FE45H`**.

`0000H` skips the write entirely. `8000H` is nonzero black: at (43,24) its modulated source remains black, additive blending preserves background RGB, but stored sample bit 15 changes the result to **`8C41H`**. The exact completed triangle is:

| Y | X=40 | X=41 | X=42 | X=43 |
|---|---|---|---|---|
| 24 | `7DE3H` | `FE45H` | `0C41H` (zero sample) | `8C41H` |
| 25 | `7E04H` | `FE24H` | `0C41H` (zero sample) | — |
| 26 | `7DE3H` | `FE45H` | — | — |
| 27 | `7E04H` | — | — | — |

Dashes are outside coverage and remain unchanged. There are ten covered samples, two fetched-zero skips and eight writes. For the full gain/dither arithmetic and write-protection rules, use **Choose texture gain and dither deliberately** and **Separate texture transparency, blending and destination protection**. In particular, force-mask does not make a bit-15-clear texture sample blend, and even force-mask cannot turn fetched `0000H` into a write.

In a separate trial from the original background, change only word 1 to `27C08040H`. Raw texture ignores the supplied gain and bypasses dithering: (40,24) becomes `6208H`; (41,24) blends raw RGB5=(8,16,24) over the background and becomes **`EE49H`**. Semi-transparency and stored-bit selection still apply. Use modulation color (128,128,128) for exact unit gain **with dithering off**; (255,255,255) is not neutral.

### 5.23.4 A later rectangle inherits the changed page and preserved flips

With no intervening drawing-mode change, send:

```
66C08040H  00180030H  7D3F05FBH  00020002H
```

This is a two-by-two semi-transparent modulated textured rectangle at (48,24), starting U/V=(251,5), using the same CLUT and RGB. Its sole texture word contains no page replacement. Current `3EBFH` therefore supplies palette8, page(960,768), additive blending and both flips. U advances 251→250; V advances 5→4. Row V=4 uses declared raw word `1415H`, reversing the two palette indices. Rectangles do not dither, despite retained dither enable:

| Destination row | X=48 | X=49 |
|---|---|---|
| 24 | `FE45H` | `7E04H` |
| 25 | `7E04H` | `FE45H` |

Thus T1 was not a temporary material choice, and the polygon did not clear flip state merely because it did not use it. Program the intended drawing mode before a later material that requires different state.

### 5.23.5 Command cost

Flat textured setup is 226 ticks for a triangle or 262 for a quad, followed by the per-triangle work estimates in **Budget command work separately from issuing words**. Raw texture receives no setup/work discount. This triangle has absolute double area 16, giving p=8, then 16 for texture and 24 for semi-transparency: `226+24=250` ticks, a 125-system-cycle command interval. The opaque `24H`/`25H` variants instead take `226+16=242` ticks, or 121 cycles, under the same state. Neither fetched-zero skips nor the actual eight stores reduce that estimate. CPU issue work, readback readiness and presentation completion remain separate.

## 5.24 Gouraud textured polygons — GP0(34H–37H,3CH–3FH)

Use this family when each vertex supplies RGB material gains as well as texture coordinates. Modulated commands interpolate RGB and U/V across each triangle; raw commands retain the same packet layout but take the unmodulated source color from the texture.

### 5.24.1 Opcode and complete packet

| Opcode | Primitive | Texture color | Blend request | Words |
|---|---|---|---|---|
| `34H` | Triangle | Modulated | Off | 9 |
| `35H` | Triangle | Raw | Off | 9 |
| `36H` | Triangle | Modulated | On | 9 |
| `37H` | Triangle | Raw | On | 9 |
| `3CH` | Quad | Modulated | Off | 12 |
| `3DH` | Quad | Raw | Off | 12 |
| `3EH` | Quad | Modulated | On | 12 |
| `3FH` | Quad | Raw | On | 12 |

The opcode has Gouraud bit 4 and texture bit 2 set. Bit 3 selects a quad, bit 1 requests semitransparency, and bit 0 selects raw texture. Supply complete GP0 words in this order:

```
Triangle: (opcode<<24) OR C0, P0,T0, C1,P1,T1, C2,P2,T2
Quad:     (opcode<<24) OR C0, P0,T0, C1,P1,T1, C2,P2,T2, C3,P3,T3
```

Each C contains unsigned R/G/B bytes in bits 7:0, 15:8 and 23:16. Only the first word's high byte selects a command; the high byte of C1–C3 is ignored. Each P contains signed eleven-bit X in bits 10:0 and Y in 26:16; the other five bits in each halfword do not extend the coordinate. Each T contains unsigned U in bits 7:0 and V in 15:8, with the upper half interpreted below.

**Raw mode does not remove the color words.** C1 and C2 remain words 4 and 7; in a quad C3 remains word 10. Omitting them makes subsequent positions and texture words occupy the wrong fields. There is no terminator after word 9 or 12. These are packet fields, not a BLua data-loading API.

A quad draws triangles `(0,1,2)` then `(2,1,3)`, preserving each vertex's C/P/T association. Each half has its own affine planes; this is not bilinear shading of four corners. Geometry, inclusive drawing-area clipping and row suppression follow Place primitives across coordinate boundaries and Predict shared-edge and joint writes.

### 5.24.2 CLUT, persistent T1 state and retained operands

T0's upper half is the CLUT attribute: base X is `(attribute AND 3fH)×16` raw words, and base Y is `attribute>>6` through the retained Y-address gate. Palette lookup uses that base; direct16 depth does not use the CLUT.

T1's upper half is the texture-page attribute. It is word **6** here, not word 5 as in a flat-textured packet. Its selected fields are:

| Attribute bits | Selection |
|---|---|
| 3:0 | Page X in units of 64 raw words |
| 4 | Page Y contribution of 256 |

| Attribute bits | Selection |
|---|---|
| 6:5 | RGB blend mode 0–3 |
| 8:7 | Texture depth: 0 palette4; 1 palette8; 2 or 3 direct16 |
| 11 | Page Y contribution of 512, subject to the Y-address gate |

The update is persistent:

```
new draw mode = (old draw mode AND NOT 09ffH) OR (T1 attribute AND 09ffH)
```

It preserves draw-mode bits 9, 10, 12 and 13. Other T1 attribute bits do not change them; T2/T3 upper halves do not select further pages. The update happens **before this polygon's own environment snapshot**, so it affects both this polygon and later commands that use current draw mode, including textured rectangles. Restore the desired page/blend/depth state explicitly when the next material differs.

At emission, the polygon retains the resulting draw mode, Y extension, texture window, drawing-area corners, drawing offset, mask mode and skipped-row parity. Later state writes do not reinterpret those operands. The command changes no other drawing-environment register; eligible covered pixels change local memory through the retained mask/blend rules.

### 5.24.3 Attribute and pixel calculation

The finite planes are those in Predict interpolated colors and texture coordinates: determinant quotients truncate toward zero, steps/intercepts occupy twenty bits with twelve fractional bits, and readout takes bits 19:12. The half-attribute bias does not move coverage. Modulated mode evaluates all five R/G/B/U/V planes; raw mode ignores RGB for output but still evaluates U/V. There is no Z/W term, perspective correction, filtering or mip-level selection in this point sample.

U/V then undergo texture-window replacement and the page/CLUT lookup in Locate texture texels and palette colors. A fetched `0000H` skips the pixel; `8000H` is nonzero black, not transparent. Modulation uses each five-bit texel component t and interpolated byte c:

```
p = (t×c) >> 4
source component = 0,                          if p+d < 0
                 = min((p+d)>>3, 31),          otherwise
```

Here d is the eligible destination-position dither offset, or zero with dithering off. Byte 128 is exact unit gain **with dithering off**, not byte 255. Modulated Gouraud-textured polygons are eligible for draw-mode dithering; raw texturing bypasses both modulation and dithering. See Choose texture gain and dither deliberately.

A semitransparent opcode blends only when the fetched nonzero texel also has bit 15 set. Otherwise that pixel is opaque. Blending uses the quantized RGB5 source, not interpolated RGB bytes. Destination check-mask can suppress the write; force-mask sets output bit 15 but does not make an unmarked texel blend. Without force-mask, output bit 15 copies the fetched texel. The independent zero, blend and destination-protection rules are in Separate texture transparency, blending and destination protection.

### 5.24.4 Worked nonconstant nine-word triangle

Declare the following initial state and memory; no upload packet is implied:

- Drawing offset zero; inclusive drawing area X=0…255,Y=0…191; no skipped rows; Y-address extension zero; texture-window word zero; mask mode zero.
- Current draw mode `3709H`. Its dither/draw-to-displayed-field/flip bits are set; the flip bits are preserved but do not flip triangle U/V planes.
- Every raw texture word at X=192…200,Y=5…23 is `2525H`. Raw CLUT word `(229,100)` is `F194H`, RGB5=(20,12,28), bit 15 set.
- All destination words covered by this triangle initially contain `A505H`, RGB5=(5,8,9), bit 15 set. Check-mask is off, so that old bit does not protect them.

Supply these nine words:

| Word | Raw word | Operand |
|---|---|---|
| 1 | 36C08040H | Triangle, modulated, semitransparent; C0=(64,128,192) |
| 2 | 00180028H | P0=(40,24) |
| 3 | 190C0503H | T0: U/V=(3,5), CLUT attribute 190cH |
| 4 | E18040C0H | C1=(192,64,128); high E1H is not a command |
| 5 | 0018002FH | P1=(47,24) |
| 6 | 00A30711H | T1: U/V=(17,7), page attribute 00a3H |
| 7 | E240C080H | C2=(128,192,64); high E2H is not a command |
| 8 | 001E0028H | P2=(40,30) |
| 9 | BEEF1709H | T2: U/V=(9,23); upper half ignored |

T1 changes draw mode to `(3709H AND NOT 09ffH) OR 00a3H = 36a3H`: page X=192, Y=0, palette8, additive blend mode 1. CLUT attribute 190cH gives base `(192,100)`. The triangle determinant is 42 and its plane anchor is vertex 0. The retained plane quantities are:

| Attribute | X-step word | Y-step word | Intercept |
|---|---|---|---|
| R | 12492H | 0AAAAH | 65140H |
| G | F6DB7H | 0AAAAH | EE378H |
| B | F6DB7H | EAAABH | 2E360H |
| U | 02000H | 01000H | 9B800H |
| V | 00492H | 03000H | B2130H |

For example, the R steps are `trunc(4096×128/7)=74898` and `trunc(4096×64/6)=43690`; the negative G/B steps use the same toward-zero rule. Evaluating covered pixel `(42,26)` gives **RGB=(122,131,131), U/V=(9,12)**. The identity texture window leaves U/V unchanged. Palette8 fetches raw word `(192+(9>>1),12)=(196,12)`, takes its high byte `25H`, then reads CLUT `(192+37,100)=(229,100)`: sample `F194H`.

The destination's dither phase is `(2,2)`, giving d=−4. The three first-stage products are `(20×122)>>4=152`, `(12×131)>>4=98` and `(28×131)>>4=229`. After adding −4 and shifting right three, the source is RGB5=(18,11,28). Both the opcode request and sample bit 15 permit blending. Additive mode gives `min((5,8,9)+(18,11,28),31)=(23,19,31)`, and the copied sample bit 15 produces **`FE77H`**.

A neighboring covered pixel proves the gains are not constant: `(43,26)` has RGB=(140,122,122), U/V=(11,12), d=0 and foreground RGB5=(21,11,26). It fetches the same CLUT color through packed word `(197,12)` but stores **`FE7AH`** over the declared destination.

Starting each contrast from the same initial destination:

| Change at (42,26) | Stored result | Reason |
|---|---|---|
| Opcode `34H` instead | F172H | Modulated/dithered source replaces RGB; no blend request |
| Opcode `37H` instead | FE99H | RGB fields still present but ignored; raw source (20,12,28) blends |
| CLUT color `7194H` instead, opcode 36H | 7172H | Same modulated RGB, but fetched bit 15 clear prevents blending |

### 5.24.5 Command cost

This Gouraud-textured family has setup `46+288+162=496` ticks for a triangle or `82+288+162=532` for a quad, followed by the separate triangle work estimates in Budget command work separately from issuing words. Raw mode does not remove the textured or Gouraud setup charges.

For the worked triangle there is no clipping or row skipping. Estimated work is `abs(42)>>1=21`, doubled for texturing to 42, then increased for the blend request to `42+((42+1)>>1)=63`. Total cost is **559 command ticks**, or **280 scheduled system cycles** from execution start. The raw 37H contrast has that same cost. The opaque 34H contrast instead costs `496+42=538`

ticks, or 269 cycles. Clearing the fetched sample bit 15 does not remove the semitransparent opcode's work surcharge. These estimates are not counts of stored pixels and exclude CPU packet preparation/issue; timing coordinates use ordinary signed-eleven-bit position-plus-offset sums, not the raster triangle's common coordinate shift.

## 5.25 Finite flat and Gouraud lines — GP0(40H–47H,50H–57H)

A finite line supplies exactly two endpoints. Flat color uses one RGB value for both; Gouraud color supplies a separate RGB value with each endpoint. Neither form has texture operands.

| Opcode bytes | Color form | Blend request | Packet words |
|---|---|---|---|
| `40H,41H,44H,45H` | Flat | Off | 3 |
| `42H,43H,46H,47H` | Flat | On | 3 |
| `50H,51H,54H,55H` | Gouraud | Off | 4 |
| `52H,53H,56H,57H` | Gouraud | On | 4 |

Bit 4 selects Gouraud color, bit 3 is clear for these finite forms, and bit 1 requests semitransparency. Bits 2 and 0 do not add texture fields or select raw color: all four opcodes in each row have the same operand layout and color behavior.

```
Flat:    (opcode<<24) OR C0, P0, P1
Gouraud: (opcode<<24) OR C0, P0, C1, P1
```

This is packet notation, not cartridge-language code. C uses unsigned R/G/B bytes in bits 7:0/15:8/23:16. C1's high byte is ignored, not decoded as another command. P consumes signed eleven-bit X in bits 10:0 and Y in 26:16; the other bits in each halfword do not extend its range. There is **no terminator**. After the third/fourth word, the next whole word starts a new command.

### 5.25.1 Endpoint order, samples and color lifetime

The calculation is the one in Predict line samples and color ramps. If the first endpoint has greater X, the GPU exchanges **both endpoints and their associated colors**. Equal-X vertical lines retain their supplied order: there is no corresponding normalization to upward or downward traversal. Reversing such a line can change a quantized color tie, as the vertical reversal example in that lesson demonstrates.

After ordinary drawing-offset sums, reject source spans exceeding 1024×512 inclusive. The accepted segment generates `max(abs(dx),abs(dy))+1` samples, including both endpoints; an accepted zero-length segment generates one. Positions use the specified outward-rounded Q32 steps/biases, while Gouraud bytes use biased Q12 accumulators and truncated color steps. Generated positions wrap to signed eleven bits before drawing-area and row-suppression tests. Neither clipping nor skipped rows restarts the original color ramp.

For a concrete clipped, descending-Y Gouraud line, use zero offset, inclusive area X=10…14,Y=7…10, retained suppression of odd rows, dithering off and mask-mode low bits zero:

| Word | Raw word | Operand |
|---|---|---|
| 1 | `5528C814H` | Opaque Gouraud; first RGB=(20,200,40) |
| 2 | `0005000EH` | First endpoint (14,5) |
| 3 | `E1B414DCH` | Second RGB=(220,20,180), not an E1 command |
| 4 | `000C000AH` | Second endpoint (10,12) |

Opcode `55H` deliberately has both ignored bits set. X ordering makes the working start `(10,12)` with RGB=(220,20,180), and the end `(14,5)` with RGB=(20,200,40). N=7 gives eight generated samples. The Q12 RGB steps are −117028,+105325,−81920. Of the samples at original indices 0…7, the area admits indices 2…5; odd-row suppression removes indices 3 and 5:

| Original sample index | Destination | RGB byte readout | Stored RGB555 word |
|---|---|---|---|
| 2 | (11,10) | (163,71,140) | `4514H` |

| Original sample index | Destination | RGB byte readout | Stored RGB555 word |
|---|---|---|---|
| 4 | (12,8) | (106,123,100) | `31EDH` |

The intervening rejected samples still advance every accumulator. These two opaque, unchecked, unforced writes replace their old words and write bit 15 clear.

### 5.25.2 Flat lines also dither

The draw-mode dither setting applies to **flat as well as Gouraud lines**. As a separate example, enable dithering, disable row suppression and use area X=16…19,Y=20, zero offset and mask-mode low bits zero. The complete flat packet is:

```
450F0807H, 00140010H, 00140013H
```

Its RGB=(7,8,15) is constant, but absolute destination phases produce different words:

| Destination | Dither offset | RGB555 word |
|---|---|---|
| (16,20) | −4 | `0400H` |
| (17,20) | 0 | `0420H` |
| (18,20) | −3 | `0400H` |
| (19,20) | +1 | `0821H` |

The common offset is added to each byte, saturated to 0…255, then reduced by three bits. It is not a per-line phase restarted at the first endpoint, and blending uses the resulting five-bit source.

### 5.25.3 State and command cost

Draw state is retained when the complete command is emitted; later environment changes do not reinterpret it. Semitransparent forms blend admitted source pixels against their current destination. Mask checking can suppress the write; force-mask sets output bit 15, otherwise these untextured lines write zero there. The packet itself contains no drawing-environment update.

Finite-line command work is the clipped **source bounding-box** estimate, not the generated or stored-sample count. Use ordinary signed-eleven-bit position-plus-offset sums, reject excessive source spans, and intersect the inclusive source bounding rectangle with the drawing area. Empty work costs zero ticks. Otherwise, with clipped dimensions W/H, active row suppression changes H to `max(1,H>>1)` and work is `max(W,H)`. Flat/Gouraud choice does not add a separate setup term.

For the first packet, the source box is 5×8, clipped to 5×4. Skipping changes the timing height to 2, but `max(5,2)=5`: **five ticks/three scheduled system cycles**, despite only two stored samples. Without skipping it still costs five ticks, although four samples survive clipping. The flat dither example costs four ticks/two cycles. A nonempty zero-length finite line costs one tick, even if row suppression prevents its one store. These are queued command intervals, not CPU packet-supply time or physical raster bandwidth; finite lines do not acquire a polyline's initial 16-tick setup.

## 5.26 Flat and Gouraud polylines — GP0(48H–4FH,58H–5FH)

A polyline starts with two endpoints, then adds one segment for each complete continuation vertex. Flat color retains one RGB value throughout; Gouraud color supplies a new RGB value with each new endpoint.

| Opcode bytes | Color form | Blend request | Initial words | Words per added vertex |
|---|---|---|---|---|
| `48H,49H,4CH,4DH` | Flat | Off | 3 | 1 |
| `4AH,4BH,4EH,4FH` | Flat | On | 3 | 1 |
| `58H,59H,5CH,5DH` | Gouraud | Off | 4 | 2 |
| `5AH,5BH,5EH,5FH` | Gouraud | On | 4 | 2 |

Bit 4 selects Gouraud color, bit 3 selects continuation, and bit 1 requests semitransparency. Bits 2 and 0 neither add texture operands nor select raw color. The word order is:

```
Flat:    (opcode<<24) OR C0, P0, P1, P2, P3, ... , terminator
Gouraud: (opcode<<24) OR C0, P0, C1, P1, C2, P2, C3, P3, ... , terminator
```

This is packet notation. Colors consume unsigned RGB bytes in bits 7:0/15:8/23:16; positions consume signed eleven-bit X/Y fields in bits 10:0/26:16. The complete initial header emits segment P0–P1 **without waiting for termination**. Flat P2 emits P1–P2 using C0 again; Gouraud C2/P2 emits the same segment using C1/C2. The previous endpoint and its color remain available for the following segment.

### 5.26.1 Terminate at the right field boundary

After the initial header, any word satisfying this predicate is a terminator:

```
(word AND F000F000H) = 50005000H
```

Test it only at the start of a continuation vertex: the next **position** for flat color, or the next **color** for Gouraud color. `5ABC5DEFH` qualifies. The identical word in a Gouraud continuation's position slot is position data, not a terminator. Initial-header fields are not termination boundaries either.

The complete eight-word example in Frame the GP0 command stream makes the distinction concrete:

| Word in that example | Input | Boundary meaning |
|---|---|---|
| 3 | `E10000FFH` | Initial second color; high byte ignored, not an E1 command |
| 6 | `001A002EH` | Continuation position, completing the second segment |
| 7 | `5ABC5DEFH` | Next continuation-color slot: terminates without adding a segment |
| 8 | `E1000000H` | Now a new drawing-environment command |

Moving the marker into word 6 would make it position data. Conversely, a matching full word in a continuation-color slot terminates **before** its low 24 bits can become a color. An opcode-looking or NOP-looking continuation word is not an escape to a command boundary.

There is no fixed total polyline word count. Omitting termination leaves the continuation phase open; supplying a Gouraud color without its position leaves that pair unfinished. A pause while buffered words drain does not discard the retained endpoint. Termination does not undo segments already emitted. Each complete segment retains the drawing state present at its own emission; it does not wait for a final whole-polyline state snapshot.

### 5.26.2 Count the joint twice when blending

Every segment uses the finite-line calculation in Predict line samples and color ramps: X ordering exchanges endpoints and their associated colors, equal-X order is retained, and an accepted segment generates `max(abs(dx),abs(dy))+1` samples. Generated positions wrap to signed eleven bits before area/row tests. Clipping or row suppression does not restart a color ramp. Dithering, if enabled, applies to flat as well as Gouraud segments at each absolute destination phase.

For a complete flat example, use zero offset, inclusive area X=40…46/Y=30, no row suppression, dithering off, mask checking/forcing off and saturated additive blend selection 1. Initialize all seven destination words to `0000H` and supply:

| Word | Raw input | Meaning |
|---|---|---|
| 1 | `4A000020H` | Flat semitransparent polyline; RGB=(32,0,0) |
| 2 | `001E0028H` | P0=(40,30) |
| 3 | `001E002BH` | P1=(43,30); completes the initial segment |
| 4 | `001E002EH` | P2=(46,30); completes the next segment |
| 5 | `5ABC5DEFH` | Terminates at the next flat position boundary |

Each segment generates four samples, including both ends. RGB=(32,0,0) becomes RGB5=(4,0,0), word `0004H`, before blending. The first segment writes X=40…43; the second writes X=43…46. Their common endpoint receives **two additions**, not polygon-style once-only seam ownership:

| Destination X at Y=30 | 40 | 41 | 42 | 43 | 44 | 45 | 46 |
|---|---|---|---|---|---|---|---|
| Write count | 1 | 1 | 1 | 2 | 1 | 1 | 1 |
| Final word | `0004H` | `0004H` | `0004H` | `0008H` | `0004H` | `0004H` | `0004H` |

These are the results after both segments' pixel writes. The stated mask settings matter: destination protection could reject an old protected word, while forcing would set output bit 15. No sample bit 15 is needed to enable blending for these untextured lines.

### 5.26.3 Command cost

Use the finite-line source-bounding-box estimate for each segment: ordinary decoded position-plus-offset sums, reject inclusive spans over 1024×512, then intersect with drawing bounds. Empty intersection gives zero work. Otherwise, with clipped dimensions W/H, row suppression changes H to `max(1,H>>1)` and work is `max(W,H)`.

The **initial polyline segment adds 16 setup ticks**; later continuation segments use work alone. Each example box is 4×1, giving **20 ticks/10 scheduled system cycles** for the initial segment and **4 ticks/2 cycles** for the continuation. The terminator adds no segment. An initial segment with zero work still has its 16-tick setup; a zero-work continuation does not repeat it. Gouraud color, blending and dithering add no separate line-cost term. These are queued command intervals, not CPU word-supply costs, physical raster bandwidth or presentation acknowledgement.

## 5.27 Drawing rectangles — GP0(60H–7FH)

A drawing rectangle supplies one RGB color, one destination origin and either a fixed size or an explicit width/height. Textured forms add one U/V and CLUT word. There is no Gouraud rectangle: opcode bits 4:3 select size, not color interpolation.

| Size | Color source | Opaque opcode bytes | Semitransparent opcode bytes | Words |
|---|---|---|---|---|
| Variable | Untextured | `60H,61H` | `62H,63H` | 3 |
| Variable | Modulated texture | `64H` | `66H` | 4 |
| Variable | Raw texture | `65H` | `67H` | 4 |
| 1×1 | Untextured | `68H,69H` | `6AH,6BH` | 2 |
| 1×1 | Modulated texture | `6CH` | `6EH` | 3 |
| 1×1 | Raw texture | `6DH` | `6FH` | 3 |
| 8×8 | Untextured | `70H,71H` | `72H,73H` | 2 |
| 8×8 | Modulated texture | `74H` | `76H` | 3 |
| 8×8 | Raw texture | `75H` | `77H` | 3 |
| 16×16 | Untextured | `78H,79H` | `7AH,7BH` | 2 |
| 16×16 | Modulated texture | `7CH` | `7EH` | 3 |
| 16×16 | Raw texture | `7DH` | `7FH` | 3 |

Bit 2 enables texture, bit 1 requests semitransparency, and bit 0 selects raw rather than modulated texture **only when textured**. The paired untextured opcodes ignore bit 0. Raw texture ignores the supplied RGB color; modulation uses it as channel gain.

```
Variable untextured: (opcode<<24) OR C0, P0, S
Variable textured:   (opcode<<24) OR C0, P0, T0, S
Fixed untextured:    (opcode<<24) OR C0, P0
Fixed textured:      (opcode<<24) OR C0, P0, T0
```

This is packet notation. C0 consumes unsigned RGB bytes in bits 7:0/15:8/23:16; P0 consumes signed eleven-bit X/Y in bits 10:0/26:16. S supplies width in bits 9:0 and height in bits 24:16: **0…1023 by 0…511**, with either zero extent empty. This is not an image transfer's zero-means-maximum convention. Fixed forms have **no S word**: an extra size word would instead occupy the next command boundary.

T0 has U in bits 7:0, V in 15:8 and a CLUT attribute in 31:16. CLUT X is `(attribute AND 3FH)×16`; CLUT Y is `attribute>>6` through the Y-address gate. The rectangle uses the **current E1 texture page and depth**. It has no polygon T1 word and does not overwrite E1 with its CLUT attribute.

### 5.27.1 Placement and texture lifetime

Add the signed drawing offset to the decoded origin, then wrap that origin to signed eleven bits. Extend right/down by the selected size; do not repeatedly wrap the destination extent. Clip against the inclusive drawing area. Textured rectangles anchor U/V at the original, unclipped origin. Independent X/Y flip settings change their steps to −1 instead of +1; they neither reverse destination coverage nor move the anchor automatically to the opposite texture edge. Left/top clipping advances U/V, and suppressed rows still advance V. See Keep a clipped or flipped rectangle on its texture for the four independent flip choices.

Every complete rectangle retains its drawing-state snapshot. Later environment changes do not reinterpret it. Neither untextured nor textured rectangles dither, even with the draw-mode dither setting enabled. The normal transparent-sample, blending and destination-mask rules still apply: fetched `0000H` skips the write, while `8000H` is processed black; textured semitransparency needs both the opcode request and sample bit 15. Mask checking tests the old destination; forcing sets output bit 15.

### 5.27.2 Complete untextured and textured packets

For both trials below, use zero offset, inclusive drawing area X=13…17/Y=22…25, retained suppression of odd rows, mask checking/forcing off and the dither setting on. Initialize the twenty destination words in that area to `6B5AH` before **each** trial.

The complete opaque fixed-8×8 packet is only two words:

```
70A0647CH, 0014000AH
```

It supplies RGB=(124,100,160) and origin (10,20). Clipping leaves five columns and four rows; odd-row suppression leaves ten writes on Y=22 and 24. With no rectangle dither, every write is RGB5=(15,12,20), **`518FH`**. Rows 23 and 25 retain `6B5AH`. There is no trailing size word.

For the textured trial, let current E1 select page (64,256), eight-bit palette sampling and both rectangle flips. Use texture-window fields X-mask=1/Y-mask=3/X-offset=0/Y-offset=1, giving U AND/OR masks `F7H/00H` and V masks `E7H/08H`. Declare these raw source words; all other texture words and palette entries used by this example are `0000H`:

| Role | Raw VRAM coordinate | Word |
|---|---|---|
| Packed texture | (187,495) | `A512H` |
| Packed texture | (187,493) | `3612H` |
| Palette color | (133,300) | `25ABH` |
| Palette color | (22,300) | `8000H` |

Supply the complete variable, modulated, opaque packet:

| Word | Raw input | Meaning |
|---|---|---|
| 1 | `64808080H` | RGB=(128,128,128), exact unit modulation gain |
| 2 | `0014000AH` | Origin (10,20) |
| 3 | `4B3E0102H` | U0=2,V0=1; CLUT base (992,300) |
| 4 | `00060008H` | Width 8, height 6 |

The first admitted pixel (13,22) has stepped U/V=(−1,−1), transformed to `F7H/EFH`. Palette8 reads raw (64 + 123, 256 + 239) = (187, 495); odd U selects `A5H`. CLUT X wraps within its row: `(992+165) AND 3FFH=133`, yielding `25ABH`. Neutral gain preserves its RGB5=(11,13,9).

| Destination | Windowed U/V | Selected index | Fetched color | Final destination |
|---|---|---|---|---|
| (13,22) | F7H/EFH | A5H | 25ABH | 25ABH |
| (14,22) | F6H/EFH | 12H | 0000H | Unchanged 6B5AH |
| (13,24) | F7H/EDH | 36H | 8000H | 8000H |

Y=23 still consumes V=EEH despite being suppressed, so Y=24 uses EDH, not EEH. Its index 36H selects CLUT X=(992+54) AND 3FFH=22. Only the two nonzero-color destinations shown change; every other destination retains 6B5AH. The 8000H write is opaque processed black, not a transparent skip. Changing opcode 64H to raw 65H keeps the packet layout and lookup; this particular neutral-gain case then has the same output without using C0.

### 5.27.3 Command cost

Use ordinary signed-eleven-bit position-plus-offset sums for timing, **not** the raster origin's rewrapped sum. Both examples have clipped W=5,H=4 before row suppression and H′=2 after it. The untextured row cost is q=5: **16+5×2=26 ticks, 13 scheduled system cycles**. Palette8 is below both texture thresholds, adding W=5 row ticks: q=10 and **16+10×2=36 ticks, 18 cycles**. Raw texture does not remove that overhead. The cost remains 36 ticks despite only two successful texture writes.

In general add (W+1)>>1 to row cost when semitransparency **or** mask checking is enabled, once even when both are enabled. The wider/greater-area texture branches are listed in Budget command work separately from issuing words; use clipped H **before** row halving to select them. An empty rectangle still costs 16 ticks/eight cycles. For example, replacing the textured packet's S with 00060000H gives width zero and no samples, not a 1024-column image. These intervals exclude CPU word preparation/supply and do not acknowledge presentation.

## 5.28 Fill raw local memory — GP0(02H)

Use fill for a solid RGB color written in sixteen-word horizontal units, not as a drawing rectangle with the usual pixel-write controls. **Only opcode 02H** selects this operation. Its complete packet is three GP0 words:

| Word | Contents |
|---|---|
| 1 | 02000000H OR RGB888: R in bits 7:0, G in 15:8, B in 23:16 |
| 2 | Destination XY: raw X in the low halfword, raw Y in the high halfword |
| 3 | Size S: width in the low halfword, height in the high halfword |

There is no pixel payload or terminator. The next word starts a new command; its bits are not additional fill pixels. The low twenty-four header bits are color, not raw/modulated or semitransparency options.

### 5.28.1 Decode alignment, dimensions and color

With Ymask=1ffH for the nine-bit gate or 3ffH for the ten-bit gate:

```
X0 = XY AND 3f0H
Y0 = (XY >> 16) AND Ymask
W  = ((S AND 3ffH) + 15) AND NOT 0fH
H  = (S >> 16) AND 1ffH
```

X is aligned **down** to a multiple of 16; the selected width is rounded **up** to a multiple of 16. Width 1…16 therefore writes 16 columns, while width 1009…1023 writes 1024. Width zero remains empty, and height zero remains empty: neither selects the maximum transfer dimension. Height uses only nine bits even when the Y-address gate is open. Unselected position/size bits do not extend these fields.

For each admitted row j and column i, raw coordinates are X=(X0+i) AND 3ffH, Y=(Y0+j) AND Ymask. X wraps within its row; an overrun past 1023 does not carry into the next row. The Y gate also applies to each generated row. These are raw-word addresses, not signed drawing coordinates or PCRTC storage-format addresses; see Organize local memory for drawing and display.

Each byte loses its low three bits. The stored word is `(R>>3) OR ((G>>3)<<5) OR ((B>>3)<<10)`, with **bit 15 clear**. Fill bypasses drawing offset, drawing-area clipping, dithering, blending, destination-mask checking and force-mask. It can therefore overwrite an old marked word and clear its bit 15 even while both mask controls are set. It does not change those configured controls.

The exception is **retained skipped-row parity**. A row whose parity matches the retained suppression value is not written. Such suppression can be selected when vertical-resolution and interlace mode are both set and draw-to-displayed- field is clear; otherwise no row parity is suppressed. Do not assume fill always writes every row merely because it bypasses the drawing area. The differences from drawing, upload and copy are collected in Choose fill, drawing or a raw transfer.

### 5.28.2 Complete nonaligned fill with both address wraps

Declare nine-bit Y addressing and retained suppression of odd rows. Drawing area X=100…103,Y=200…202 excludes the target; drawing offset is `(13,-7)`; dithering and subtractive blend mode are selected; mask mode has check and force both set. The locations named below initially contain `FFFFH`. These are declared example contents, not a reset-memory guarantee.

Supply:

| Word | Raw word | Requested operand |
|---|---|---|
| 1 | `024964F1H` | RGB=(241,100,73) |
| 2 | `03FF03FFH` | X=1023,Y=1023 |
| 3 | `00040011H` | Width 17,height 4 |

X aligns to 1008, width rounds to 32, and the nine-bit Y gate selects 511. The four row addresses are 511,0,1,2; each row's columns are 1008…1023 followed by 0…15. The output color is RGB5=(30,12,9), packed **`259EH`**:

| Raw Y | Retained parity test | After-fill words at X=1008…1023 and X=0…15 |
|---|---|---|
| 511 | Odd: suppressed | `FFFFH`, unchanged |
| 0 | Even: admitted | `259EH` |
| 1 | Odd: suppressed | `FFFFH`, unchanged |
| 2 | Even: admitted | `259EH` |

Exactly 64 distinct words change here. Neighbors such as `(16,0)` and `(1007,2)` remain unchanged. The excluding drawing area and nonzero offset do not relocate or clip these writes; old bit 15 does not protect them, and force-mask does not mark the result. The selected dither/blend state likewise does not alter `259EH`.

### 5.28.3 Command cost, not stored-pixel count

For decoded W/H, the cost is:

```
ticks = 46 + ((W >> 3) + 9)×H
```

The worked packet costs `46+(4+9)×4=98` command ticks, or **49 scheduled system cycles** from execution start. Disabling row suppression would write 128 words instead of 64 but would not change that cost. Likewise, changing only S to `00040000H` makes width zero and writes nothing, yet still costs 82 ticks/41 cycles; S=`00000011H` makes height zero and costs 46 ticks/23 cycles.

These intervals exclude preparation and issue of the three words and are not a physical fill-rate specification. Use the execution-deadline conversion in Budget command work separately from issuing words. Do not apply the signed-position-plus-offset or clipped-area work rules of drawing primitives to this fill calculation.

## 5.29 Raw image upload — GP0(A0H–BFH)

Upload supplies raw **sixteen-bit local-memory words**, two per GP0 payload word. Every opcode byte from `A0H` through `BFH` selects the same operation; the opcode's low five bits and the header's low 24 bits do not select formats or colors.

```
(opcode<<24) OR ignored24, destinationXY, S, payload0, payload1, ...
W = (((S AND FFFFH) − 1) AND 3FFH) + 1
H = ((((S >> 16) AND FFFFH) − 1) AND 1FFH) + 1
payload words = (W×H + 1) >> 1
```

This is packet notation and integer mathematics. The header is exactly three 32-bit words. Destination X uses its low ten bits; destination Y comes from the upper half through the selected Y-address gate. These are raw addresses, not signed primitive positions. W is 1…1024 and H is 1…512: **zero does not request an empty upload**.

| Raw S | Normalized W×H | Payload words | Total GP0 words |
|---|---|---|---|
| `00030003H` | 3×3 | 5 | 8 |
| `00000000H` | 1024×512 | 262144 | 262147 |
| `02010401H` | 1×1 | 1 | 4 |

Consume each payload word's **low sixteen bits first, then high sixteen bits**, advancing a row-major pixel cursor. Packing does not restart at each row. With an odd width, one payload word can supply the last pixel of one row and the first pixel of the next. For an odd total pixel count, ignore the final payload word's upper half; the **next whole GP0 word** begins the next command. There is no end-image opcode in place of the count. Opcode-looking or polyline-marker-looking data remains image data throughout that payload.

### 5.29.1 Raw addressing and mask state

For each pixel, X wraps within its 1024-word row, without carrying into Y. Generated rows wrap through the selected 512- or 1024-row Y-address period. This is raw local-memory addressing, not drawing-area clipping.

Upload applies the retained destination-mask mode but ignores drawing offset, drawing area, row suppression, blending and dithering. Check-mask tests the destination's **old** bit 15 and rejects the entire write when set. Otherwise force-mask sets output bit 15; without force-mask, all sixteen source bits pass through. In particular, source `0000H` is an ordinary raw write, not texture transparency, and `8000H` is ordinary raw data rather than a blend request.

A normal upload becomes one retained command after the counted payload has been assembled. Its mask mode and Y-address state are snapshotted when the complete upload—or a flushed partial upload—is emitted, not supplied by an extra payload flag. Apparent E commands inside the image cannot change that state.

### 5.29.2 A 3×3 upload across both address boundaries

Select the 512-row Y-address period and mask mode `3`: checking and forcing both enabled. The following table declares the old destination words. Other draw-state settings do not affect this raw upload. Supply all eight words:

| GP0 word | Raw input | Role |
|---|---|---|
| 1 | `BF123456H` | Upload alias BF; low 24 bits ignored |
| 2 | `01FF03FEH` | Destination X=1022,Y=511 |
| 3 | `00030003H` | Width 3,height 3 |
| 4 | `E1800000H` | Pixels 0/1: `0000H`,`E180H` |
| 5 | `5ABC5DEFH` | Pixels 2/3: `5DEFH`,`5ABCH`; not a polyline terminator |
| 6 | `00008000H` | Pixels 4/5: `8000H`,`0000H` |
| 7 | `A0007C00H` | Pixels 6/7: `7C00H`,`A000H`; not a new upload |
| 8 | `E1001234H` | Pixel 8: `1234H`; upper `E100H` ignored |

The complete masked result is:

| Pixel index | Destination | Source | Before | After |
|---|---|---|---|---|
| 0 | (1022,511) | 0000H | 001FH | 8000H |
| 1 | (1023,511) | E180H | 03E0H | E180H |
| 2 | (0,511) | 5DEFH | 8001H | Unchanged 8001H |
| 3 | (1022,0) | 5ABCH | 1111H | DABCH |
| 4 | (1023,0) | 8000H | 2222H | 8000H |
| 5 | (0,0) | 0000H | 8333H | Unchanged 8333H |
| 6 | (1022,1) | 7C00H | 4444H | FC00H |
| 7 | (1023,1) | A000H | 8555H | Unchanged 8555H |
| 8 | (0,1) | 1234H | 6666H | 9234H |

Payload word 5 crosses the logical row boundary: its low half targets (0,511), while its high half targets (1022,0). X overflow alone did **not** move pixel 2 to Y=0; the completed three-pixel row advances Y and the selected gate wraps 512 to zero. Pixel 0 proves that raw zero really writes: force-mask turns it into 8000H. Pixels 2,5,7 retain their protected old words regardless of their source bits. The ignored upper half of word 8 cannot supply another pixel or start an E1 command.

### 5.29.3 An interrupted upload retains only its consumed prefix

GP1(01h) clears the stream, not every earlier accepted pixel operation. If upload assembly has consumed at least one payload word, the clear flushes that partial payload; a header-only upload is discarded. Only up to min(W×H,2×consumedPayloadWords) pixels are represented. It does not manufacture the missing rectangle pixels, and words still merely queued in the cleared FIFOs are not promised as part of that prefix.

As a separate trial from the same old-memory table, suppose only payload words 4 and 5 have been consumed before the GP1 clear. The retained prefix represents **four pixels**, not one complete row or two rows. After its masked writes, indices 0,1,3 have the table's new values; index 2 remains protected; indices 4…8 retain their **Before** values. Subsequent GP0 words start a new stream: do not send the abandoned remainder as though its old header were active. GP1(00h) performs the same stream-clear operation before restoring register defaults; it is not a local-memory rollback. See GP1 control commands for the complete clear/reset and readback distinctions.

### 5.29.4 Supply cost is separate from command cost

A normal upload has **one command tick** of scheduled command cost, giving one system cycle, independently of image area. That is not a bound on preparing, admitting or supplying its header/payload, nor on making the resulting pixels available for presentation.

Supply CPU words to the **same fixed GP0 port**, or transfer the stream by DMA as described in Hand a GP0 stream between CPU and DMA. Use separate fixed-port CPU stores: a contiguous word sequence advances its destination. Keep supplying image words through the end of the counted payload, across DMA block boundaries when used, and finish DMA ownership before taking over the port with CPU stores.

## 5.30 Read raw local memory — GP0(C0H–DFH)

Use this command to return raw sixteen-bit local-memory words through GPUREAD. It does not return a PCRTC-composed, magnified or storage-format-converted picture, and it does not expose a CPU-addressable local-memory aperture.

### 5.30.1 Three-word request and normalized dimensions

Every opcode from C0H through DFH selects the same operation. The opcode's low five bits and the header's low twenty-four bits do not select another format. Supply:

| Word | Contents |
|---|---|
| 1 | opcode << 24; low twenty-four bits ignored |
| 2 | Source XY: raw X in the low halfword, raw Y in the high halfword |

| Word | Contents |
|---|---|
| 3 | Size S: raw width in the low halfword, raw height in the high halfword |

There is **no GP0 pixel payload or terminator**. Image data comes back on reads; a following GP0 write belongs to the next command, subject to the readback barrier below.

```
W = (((S AND ffffH) − 1) AND 3ffH) + 1
H = ((((S >> 16) AND ffffH) − 1) AND 1ffH) + 1
GPUREAD word count = (W×H + 1) >> 1
```

W is 1…1024 and H is 1…512, including with the ten-bit Y-address gate. A zero size word requests 1024×512 pixels and 262144 reads, not an empty operation. Raw size halfwords 1025/513 instead select 1×1. These are image-transfer dimensions, not fill or drawing-rectangle zero-size rules.

Source X uses ten bits. Source Y and every generated row use the request's nine-bit or ten-bit Y mask. X wraps within its 1024-word row, without carrying into the next row. The source coordinates, normalized dimensions and Y-extension value belong to the retained request; a later gate write does not retarget an already emitted request. The raw word representation and address distinction are described in Organize local memory for drawing and display.

### 5.30.2 Execution, submission and READY are separate

The command has a **one-command-tick** execution interval. At its scheduled completion, it activates a pending readback with cursor zero and a command-count fence identifying the required command prefix through this request. That scheduled interval is not the time to return all W×H pixels.

The request progresses through three distinct states:

| State | Meaning for the request |
|---|---|
| Pending | Waiting for read work to be submitted at its exact completed-command fence |
| Submitted | The ordered read work has been claimed; its matching completion is still required |
| READY | Data is available for permitted GPUREAD consumption; GPUSTAT bit 27 is set |

The fence ensures the required earlier work is performed before pixel extraction. The count and these phases are ordering state, not extra request words or additional directly addressed registers. Later packets cannot execute past a non-idle readback. Do not replace READY with the command's one-tick deadline, GPU idle, or acceptance of the third header word. No fixed C0-to-READY latency or sustained readout rate is specified by that command cost.

### 5.30.3 Read ownership, latch and counted release

Use complete word reads and the port rules in GP0, GPUREAD, GP1 and GPUSTAT. A GP0 read first synchronizes already-due command work. If no readback is READY, it returns the retained GPUREAD latch without consuming a pixel; it does not necessarily return zero. If READY and DMA does not own GP0's read port, a CPU read consumes the next packed word and replaces the latch. A DMA read may consume its stream; a CPU read while DMA owns that port instead returns the latch without advancing. READY alone does not grant CPU ownership.

Packing is one continuous row-major pixel sequence: first pixel low halfword, next pixel high halfword, continuing across row boundaries. There is no padding after an odd-width row. Only the final odd pixel has an unused **zero high halfword**. Count `(W×H+1)>>1` permitted word reads; neither a zero word nor a particular pixel value is an end marker.

Consuming the final word returns readback to idle and resumes queued command processing. GPUREAD keeps that final word until another query or permitted data read replaces it. The next no-READY read repeats the latch, not a new pixel. A byte read performs the same port-read operation and can consume an entire packed word. Query/latch interactions are detailed in Query drawing state through GPUREAD; a query does not reserve its answer against a ready image read.

### 5.30.4 Complete odd 3×3 readback across both address wraps

Declare nine-bit Y addressing, GPUREAD=`DEADBEEFH`, no DMA read ownership and no intervening info query. The source words are already present as follows; no upload or reset-memory contents are implied:

| Raw Y | X=1023 | X=0 | X=1 |
|---|---|---|---|
| 511 | `0011H` | `8022H` | `0033H` |
| 0 | `0044H` | `8055H` | `0066H` |
| 1 | `0077H` | `8088H` | `0099H` |

Issue:

| Word | Raw word | Meaning |
|---|---|---|
| 1 | `D9ABCDEFH` | Readback alias D9H; ABCDEFH ignored |
| 2 | `03FF03FFH` | X=1023; supplied Y=1023 masks to 511 |
| 3 | `00030003H` | W=3,H=3 |

Each row addresses X=1023,0,1; generated rows address 511,0,1. While bit 27 remains clear, a GP0 read returns `DEADBEEFH` and does not advance this image. Once READY, the five permitted word reads are:

| Read | GPUREAD result | Low pixel | High pixel | READY after read |
|---|---|---|---|---|
| 1 | `80220011H` | (1023,511) | (0,511) | 1 |
| 2 | `00440033H` | (1,511) | (1023,0) | 1 |
| 3 | `00668055H` | (0,0) | (1,0) | 1 |
| 4 | `80880077H` | (1023,1) | (0,1) | 1 |
| 5 | `00000099H` | (1,1) | Unused zero | 0 |

Read 2 crosses a logical row boundary without padding. Source bit 15 passes through unchanged; these are storage words, not newly quantized RGB values. With no next readback or query, another GP0 read returns `00000099H` again. An intervening CPU read under DMA read ownership would instead repeat the current latch without consuming one of these five words.

### 5.30.5 A queued marker waits for the final read

For the same request, start with the GPU request latch clear and place the whole GP0 marker word `1F000000H` immediately after the three-word header. It is the next command, not data for this readback. It cannot execute while the readback is pending, submitted or READY, including after only four of the five reads.

Let R be the cycle when the permitted final read resumes processing, with this marker the only queued follow-up. The marker executes in that resumed processing, sets its request, and has its own one-tick interval ending at R+1 system cycle. The final read releases the image barrier; the marker's request and GPU idle are still different conditions. Acknowledgement of GPU and controller latches follows GP1 control commands.

### 5.30.6 Clearing is an abort, not counted consumption

GP1(01h) can abandon readback; retained-command effects depend on whether the request is unactivated, pending, submitted or READY. It is not equivalent to consuming the remaining words and is not a rollback of all earlier accepted work. Use the phase-dependent clear description in GP1 control commands. GP1(00h) also clears the stream but preserves the GPUREAD latch and Y-extension value; neither resetting that GPU register state nor reading an old latch proves local memory was erased.

## 5.31 Raw local-memory copy — GP0(80H–9FH)

Copy moves sixteen-bit words directly between two local-memory regions. Every opcode byte from `80H` through `9FH` has the same four-word layout; the low five opcode bits and low 24 header bits are ignored. There is no GP0 pixel payload or CPU readback:

```
(opcode<<24) OR ignored24, sourceXY, destinationXY, S
W = (((S AND FFFFH) − 1) AND 3FFH) + 1
H = ((((S >> 16) AND FFFFH) − 1) AND 1FFH) + 1
```

This is packet notation and integer mathematics. Each XY has raw X in its low ten bits and raw Y in its upper half through the selected Y-address gate. They are not signed primitive positions. W is 1…1024 and H is 1…512: S=`00000000H` requests **1024×512**, not an empty copy. The 1024-row Y-address setting does not enlarge that 512-row maximum transfer height. After the fourth packet word, the next word is another command, not image data.

The complete packet retains its mask mode and Y-address state when emitted; later environment changes do not reinterpret it. Source/destination X wrap within their respective 1024-word rows, and generated Y values wrap through the selected 512- or 1024-row period. Drawing offset, drawing area, row suppression, blending and dithering do not affect this raw operation.

Check-mask rejects a write when the destination's **old** bit 15 is set. Otherwise force-mask sets output bit 15; with forcing clear, all sixteen source bits are copied. Source `0000H` is writable raw data, not texture transparency. A set source bit 15 is also data: it does not prevent reading or request blending.

### 5.31.1 Determine traversal before relying on overlap

Rows always advance in increasing **logical** row order, even when Y wraps to a smaller physical coordinate. For every row calculate:

```
D = (destinationX − sourceX) AND 3FFH
```

If `0 < D < W`, visit columns W−1…0; otherwise visit 0…W−1. Read each source word immediately before its destination mask test/write. The operation does **not** first capture a whole source rectangle. Horizontal reverse traversal therefore does not promise source preservation for vertical overlap.

For a complete wrapped horizontal example, clear both mask controls and initialize row 20 as follows. Supply:

```
9F123456H, 001403FEH, 001403FFH, 00010003H
```

This copies 3×1 from (1022,20) to (1023,20); the first word deliberately uses alias `9FH` and ignored payload bits. D=1, so columns execute 2,1,0:

| Column | Source X/read word | Destination X |
|---|---|---|
| 2 | 0 / `0033H` | 1 |
| 1 | 1023 / `0022H` | 0 |
| 0 | 1022 / `0011H` | 1023 |

| X at Y=20 | 1022 | 1023 | 0 | 1 |
|---|---|---|---|---|
| Before | `0011H` | `0022H` | `0033H` | `0044H` |
| After | `0011H` | `0011H` | `0022H` | `0033H` |

X wrapping stays on row 20; it does not carry into row 21.

### 5.31.2 A downward copy reads the rows it just wrote

Select the 512-row Y-address period, clear both mask controls and supply this complete packet:

| Word | Raw input | Meaning |
|---|---|---|
| 1 | `80000000H` | Raw local-memory copy |
| 2 | `01FE03FFH` | Source (1023,510) |
| 3 | `01FF03FFH` | Destination (1023,511) |
| 4 | `00030003H` | Width 3,height 3 |

Use the following before-words at the three columns X=(1023,0,1):

| Physical Y | Before | After |
|---|---|---|
| 510 | `0011H,0022H,0033H` | `0011H,0022H,0033H` |
| 511 | `0044H,0055H,0066H` | `0011H,0022H,0033H` |
| 0 | `0077H,0088H,0099H` | `0011H,0022H,0033H` |
| 1 | `00AAH,00BBH,00CCH` | `0011H,0022H,0033H` |

D=0 selects forward columns. Logical row 0 copies 510→511. Logical row 1 then reads the **new** row 511 into physical row 0, and logical row 2 reads the new row 0 into row 1. The result repeats the original first source row; it does not shift three independently preserved original rows. Choose fill, drawing or a raw transfer describes software ordering when those originals must be preserved.

Destination protection can change which source values cascade. As a separate trial, restore the Before values, change only (0,511) from `0055H` to **`8055H`**, and enable check-mask with forcing clear: mask mode `2`. The first copied row cannot replace that protected middle word, leaving row 511 as `0011H,8055H,0033H`. The next row reads those actual contents, so row 0 also becomes `0011H,8055H,0033H`; the third repeats them into row 1. Source bit 15 does not block these reads or writes to still-unprotected destinations. There are eight successful stores rather than nine, but no preserved-source snapshot appears.

### 5.31.3 Command cost

Copy costs **2×W×H command ticks**, using normalized dimensions, without drawing-area clipping or a mask-hit reduction. Thus the horizontal packet costs six ticks/three scheduled system cycles. The 3×3 packet costs **18 ticks/nine cycles**, both with and without the protected-word contrast. S=zero costs 1,048,576 ticks/524,288 cycles, although the packet still contains only four words.

Use the normalized raw-image dimensions for this cost. Budget CPU issue or DMA transport separately, and use the traversal rules above when the source and destination overlap.

## 5.32 Drawing mode — GP0(E1H)

E1 is one GP0 word: `E1000000H OR parameter`. It retains parameter bits **13:0**, mask `00003FFFH`; bits 23:14 are ignored. The draw-mode word's reset value is zero, not a zero value for the whole GPUSTAT register.

| Draw-mode bits | Meaning |
|---|---|
| 3:0 | Texture-page X: field×64 **raw sixteen-bit words** |
| 4 | Texture-page Y low selector: bit×256 rows |
| 6:5 | Blend selection: 0=half-sum, 1=saturated addition, 2=clamped subtraction, 3=quarter-foreground addition |
| 8:7 | Texture depth: 0=four-bit palette, 1=eight-bit palette, 2=direct sixteen-bit, **3=the same direct-sixteen-bit sampling path** |
| 9 | Enable dithering for eligible primitive/color paths |
| 10 | Draw-to-displayed-field control: a set bit disables the conditional row-parity suppression described below |
| 11 | Texture-page Y high selector: bit×512 rows |

| Draw-mode bits | Meaning |
|---|---|
| 12 | Textured-rectangle X flip: U step −1 when set, +1 when clear |
| 13 | Textured-rectangle Y flip: V step −1 when set, +1 when clear |

Page Y is the sum of the two selectors through the current Y-address gate. Bit 11 is a page-address bit, **not a texture-disable bit**. Depth 3 is representable and reads direct words; do not reject it or interpret it as another palette format.

The blend selector chooses the calculation, not whether a pixel blends. An untextured primitive still needs its semitransparency opcode request; a textured pixel needs both that request and fetched sample bit 15. Dithering applies to untextured Gouraud polygons, modulated textured polygons and lines, but not flat untextured polygons, raw texture or rectangles. The numerical color paths are described in Choose texture gain and dither deliberately and Separate texture transparency, blending and destination protection.

Bit 10 does **not** describe an arbitrary protected display rectangle. One retained row parity is suppressed only when the GPU vertical-resolution and interlace flags are both set **and bit 10 is clear**; the retained active-line parity selects the skipped rows. Otherwise no row parity is suppressed. Rectangle flip bits independently change UV traversal, not destination geometry or the initial UV anchor, and do not reverse triangle interpolation.

### 5.32.1 State changes at parser execution, not admission

E1 changes current draw mode when its word reaches execution in the ordered GP0 stream. The change occurs at the start of its **one-command-tick interval**, with completion one system cycle later. Each drawing command latches the mode and other drawing state used for that primitive. Place E1 before the primitive whose mode it selects; later E1 commands leave that primitive's latched state unchanged.

Textured polygons have another persistent mode producer. T1's upper attribute half updates current mode **before that polygon takes its snapshot**:

```
new mode = (old mode AND NOT 09FFH) OR (T1 attribute AND 09FFH)
```

This replaces bits 8:0 and 11: page, depth and blend. It preserves bits **9,10,12,13**, regardless of those bits in the attribute. T1 is word 5 of a flat textured polygon or word 6 of a Gouraud textured polygon, counting the command word as 1. It is an operand, not a standalone E1 command. T0 supplies the CLUT, while T2/T3 upper halves do not make further page changes. Textured rectangles have no T1 and use the current mode.

### 5.32.2 A complete E1 word and a later T1 overwrite

Select the 1024-row Y-address period and issue the complete word:

```
E1ABDBDFH
```

Only `1BDFH` is retained; the result is identical to `E1001BDFH`. It selects page (960,768), direct depth 3, subtractive blend selection 2, dithering on, bit 10 clear, X flip on and Y flip off. With the 512-row gate instead, the same page selectors yield Y=256, not 768.

In a later textured polygon, use **`E4A2078CH` in its T1 slot**. Its U/V bytes are (140,7); its attribute is `E4A2H`. The persistent merge is:

```
old preserved bits = 1BDFH AND NOT 09FFH = 1200H
new selected bits  = E4A2H AND 09FFH     = 00A2H
new current mode   = 1200H OR 00A2H      = 12A2H
```

All four preserved flags deliberately disagree with the attribute:

| Bit | Old mode | Attribute | Result |
|---|---|---|---|
| 9, dither | 1 | 0 | 1 |
| 10, draw-to-displayed-field | 0 | 1 | 0 |

| Bit | Old mode | Attribute | Result |
|---|---|---|---|
| 12, rectangle X flip | 1 | 0 | 1 |
| 13, rectangle Y flip | 0 | 1 | 0 |

The new mode selects page (128,0), palette8 and additive blend selection 1; the old dither/field/flip flags survive. For an illustrative lookup whose **post-window** U/V are (140,7):

| Mode | First raw texture read |
|---|---|
| `1BDFH`, direct depth 3 | X=(960+140) AND 3FFH=76, Y=768+7=775 |
| `12A2H`, palette8 | X=128+(140»1)=198, Y=7; even U selects the low index byte, then the T0-selected CLUT |

These are lookup inputs, not a claim that a particular polygon vertex is covered. A draw emitted before the merge retains `1BDFH`; the polygon carrying T1 retains `12A2H`; a following textured rectangle also uses `12A2H` until another mode producer changes it.

### 5.32.3 Check only the status bits which echo draw mode

GPUSTAT bits 10:0 echo mode bits 10:0, while mode bit 11 appears at **GPUSTAT bit 15**. GPUSTAT bits 11/12 belong to destination mask mode, not the rectangle flips. Do not read GPUSTAT as a complete fourteen-bit E1 echo.

After the respective mode update executes, the example therefore gives:

| Current mode | `GPUSTAT AND 000087FFH` |
|---|---|
| `1BDFH` | `000083DFH` |
| `12A2H` | `000002A2H` |

Other status fields still reflect their own controls, beam and command state. These masked observations do not establish drawing completion or expose a separate draw-mode query port.

## 5.33 Texture window, drawing bounds, offset and masks — GP0(E2H–E6H)

Each command below is **one GP0 word**, formed by ORing its opcode word with the parameter. All five retained parameter words reset to zero. Bits outside the listed mask are ignored; they do not add packet fields.

| Command word | Retained mask | Parameter fields |
|---|---|---|
| `E2000000H OR parameter` | `000FFFFFH` | Texture window: X mask 4:0, Y mask 9:5, X offset 14:10, Y offset 19:15 |
| `E3000000H OR parameter` | `000FFFFFH` | Drawing-area top-left: X 9:0, Y 19:10 |
| `E4000000H OR parameter` | `000FFFFFH` | Drawing-area bottom-right: X 9:0, Y 19:10 |
| `E5000000H OR parameter` | `003FFFFFH` | Drawing offset: signed X 10:0, signed Y 21:11 |
| `E6000000H OR parameter` | `00000003H` | Bit 0 force output bit 15; bit 1 reject writes over old destination bit 15 |

These update programmed drawing state when the ordered GP0 parser **executes** the command, not merely when its word is accepted at the port. The effect occurs at the start of a one-command-tick interval; its completion deadline is one system cycle later. An environment command is not itself a retained drawing primitive. CPU preparation/admission waits and display completion are separate costs/events.

### 5.33.1 E2 replaces texture-coordinate bits

Each texture-window mask/offset field is five bits. For either axis, derive the byte-sized operands:

```
AND byte = (NOT (mask×8)) AND FFH
OR byte  = (offset AND mask)×8
result   = (coordinate AND AND-byte) OR OR-byte
```

Apply X fields to U and Y fields to V. This is bit replacement, not an arbitrary clipping rectangle or texture width/height. Offset bits outside the mask do not participate; the low three coordinate bits are unchanged. Reset E2=zero gives AND bytes `FFH` and OR bytes `00H`: identity on byte coordinates, not a forced lookup at zero. For page/CLUT placement after this transformation, see Locate texture texels and palette colors.

### 5.33.2 E3/E4 are inclusive limits; E5 is a signed displacement

Drawing-area X is unsigned ten-bit. The retained corner words keep ten Y bits, but a draw decodes them through **its retained Y-address gate**. Right and bottom are inclusive: a decoded area `(left,top)…(right,bottom)` admits the half-open ranges `[left,right+1)` and `[top,bottom+1)`. A reversed decoded X or Y axis is empty, not automatically exchanged or wrapped. Both reset corner words zero describe the single-sample area (0,0), not a full-screen drawing area.

E5 decodes each eleven-bit offset as signed two's complement, −1024…1023. It moves primitive coordinates, not the drawing-area limits or raw image-transfer addresses. The primitive families' different boundary rules remain those in Place primitives across coordinate boundaries; E5 does not replace them with one universal wrapping rule.

### 5.33.3 E6 controls a destination write, not a source color

With bit 1 set, test the destination's **old** bit 15 before blend/store and reject the entire write if that bit is one. A permitted write has output bit 15 forced to one when bit 0 is set. Otherwise untextured drawing writes zero there, textured drawing copies the sample bit, and raw upload/copy preserves the source bit. Force-mask does not enable texture blending. Fill bypasses both controls; raw upload/copy honors them.

GPUSTAT bits **11/12** echo E6 bits **0/1**, respectively. Thus mask mode 3 gives `GPUSTAT AND 00001800H = 00001800H`; mode zero gives zero under that mask. This says nothing about the other status fields or drawing completion. The distinct transparency/blend cases are described in Separate texture transparency, blending and destination protection.

### 5.33.4 Program concrete state, then preserve its lifetime

Select the 512-row Y-address period and supply these five complete GP0 commands. Their deliberately nonzero ignored bits do not change the retained values:

| Input word | Retained parameter | Effective state |
|---|---|---|
| `E2F08061H` | `00008061H` | Window masks X=1/Y=3, offsets X=0/Y=1 |
| `E3F8580DH` | `0008580DH` | Top-left raw X=13/Y=534, decoded (13,22) |
| `E4F86411H` | `00086411H` | Bottom-right raw X=17/Y=537, decoded (17,25) |
| `E5FFEFFEH` | `003FEFFEH` | Offset X=−2,Y=−3 |
| `E6FFFF03H` | `00000003H` | Check old destination and force permitted output bit 15 |

E2's derived byte operands are:

| Axis | Mask field | Offset field | AND byte | OR byte |
|---|---|---|---|---|
| U | 1 | 0 | `F7H` | `00H` |
| V | 3 | 1 | `E7H` | `08H` |

For example, U=ADH,V=09H becomes **A5H,09H**: U changes while V already has the selected replacement bits. No packed byte mask is substituted for the original five-bit command fields.

The drawing area is five columns by four rows, half-open X=[13,18)/Y=[22,26). Raw Y values 534/537 are retained, not destroyed by the 512-row decode. With the 1024-row gate, those same corner words instead select Y=534…537. The offset's raw fields are 7FEH/7FDH, decoding to −2/−3; a supplied coordinate (16,27) has ordinary offset sum (14,24), inside the selected 512-period area.

After a drawing packet has actually been **emitted** under this state, execute:

```
E2000000H, E400640CH, E5000000H, E6000000H
```

Current state now has an identity window, zero offset and no destination-mask controls. The unchanged left limit is 13, but the new right limit is 12, so its X drawing interval is empty. A later drawing primitive that uses the area sees that empty interval.

The already-emitted draw still retains window 8061H, the earlier two corner words, offset 3FEFFEH, mask mode 3 and its 512-row Y gate. Later environment writes do not retroactively change those operands. For a pending draw, use its actual command or segment emission as the snapshot boundary, not merely acceptance of its first word. This distinction is why a current-state status observation is not a readout of every queued draw's retained state.

## 5.34 Budget command work separately from issuing words

A command's work cost is measured in **command ticks**, not packet words or stored pixels. For N positive ticks starting execution at system cycle C, its completion deadline is:

```
deadline = C + ((N + 1) >> 1)
```

Zero-tick work completes immediately. Round each command's interval separately: two one-tick commands occupy two system cycles. Queued work continues from the preceding deadline even when completion is observed late.

The following equations use integer arithmetic. >>1 and >>2 discard low bits; drawing-area right/bottom limits are inclusive. A retained skipped-row parity is either active or inactive. The cost equations do not count the exact rows, transparent texels, destination-mask hits or stored pixels.

For polygon, line/polyline and rectangle costs, decode each supplied position component and offset as signed eleven-bit values, then add them as ordinary integers. Do **not** wrap the sum back to signed eleven bits or apply the triangle's common 2048-coordinate shift. Drawing-area Y still uses the retained Y-address gate. These are timing inputs; they do not replace the primitive-specific raster placement rules.

**Polygons.** Add a setup cost and one work estimate for triangle (0,1,2), plus triangle (2,1,3) for a quad:

| Setup term | Ticks |
|---|---|
| Triangle / quad base | 46 / 82 |
| Additional Gouraud cost | 288 |
| Additional texture cost, flat / Gouraud | 180 / 162 |

For each triangle, return zero work if its source bounding span exceeds 1024 columns or 512 rows, or the drawing area is empty. The spans include both endpoints: max−min+1. Otherwise clamp each vertex's X/Y separately to the inclusive drawing-area limits and calculate:

```
D = (x1−x0)×(y2−y0) − (x2−x0)×(y1−y0)
p = abs(D) >> 1
if textured:                        p = 2×p
if semitransparent OR mask-check:   p = p + ((p+1) >> 1)
if skipped-row parity is active:    p = p >> 1
```

Apply these steps in order. Semitransparency and mask checking together add the surcharge **once**, not twice. Raw texturing still pays the textured setup and work terms. Zero work does not remove polygon setup.

For an untextured flat quad at `(38,22)`, `(46,22)`, `(38,29)`, `(46,29)`, use zero offset and drawing area X=40…43,Y=24…26, with no skipping or mask checking. Its two clamped triangles are `(40,24)`, `(43,24)`, `(40,26)` and `(40,26)`, `(43,24)`, `(43,26)`. Each has |D|=6 and p=3: total cost is `82+3+3=88` ticks, or 44 cycles. Yet the original quad covers **12 integer samples** inside that drawing area, not six. Clamping vertices for this estimator is not geometric polygon clipping or a raster sample count.

For the same geometry with Gouraud texturing and both semitransparency and mask checking, setup is `82+288+162=532`. Each work term becomes `3→6→9`, giving 550 ticks/275 cycles. Active row skipping makes each term `9>>1=4`, giving 540 ticks/270 cycles. Do not halve setup.

**Lines and polylines.** Reject an excessive source span using the same 1024×512 inclusive limits. Intersect the source bounding rectangle with the drawing area; an empty intersection has zero work. Otherwise let W/H be that intersection's inclusive width/height. With active row skipping replace H by `max(1,H>>1)`, then work is `max(W,H)`. This bounding-box calculation is not a count of line samples surviving clipping.

A finite line costs just that work, whether flat or Gouraud. A polyline's initial two-point header costs `16+work`; later continuation segments cost their individual work without another 16-tick setup. For a vertical line `(40,20)→(40,28)` clipped to Y=22…27, W=1,H=6 gives six ticks/three cycles. Active skipping makes it three ticks/two cycles. The same first segment as a polyline costs 22 ticks/11 cycles, or 19 ticks/10 cycles with skipping. A horizontal line's width can dominate the maximum, so skipping does not generally halve line cost. A zero-work finite line completes immediately; a zero-work initial polyline still costs 16 ticks.

**Drawing rectangles.** Select the fixed or variable dimensions first, then clip the offset rectangle against the drawing area. An empty result costs 16 ticks. For a nonempty result, let W/H be the clipped dimensions **before** row halving. Set row cost `q=W`, add the texture overhead below when textured, then add `(W+1)>>1` when semitransparency **or** mask checking is enabled. Finally use H′=H without skipping, or `max(1,H>>1)` with skipping:

```
N = 16 + q×H'
Q = (128 − (128 remainder W)) / W
```

Q is an exact integral quotient used only in the narrow-width branches. The texture overhead added to q is:

| Texture format | First matching condition | Additional row ticks |
|---|---|---|
| 4-bit palette | Always | W |
| 8-bit palette | W>128 | `(W>>2)×8` |
| 8-bit palette | Otherwise W×H>2048 | `(W>>2)×4×Q` |
| 8-bit palette | Otherwise | W |
| Direct 16-bit, including format 3 | W>128 | `(W>>1)×8` |
| Direct 16-bit, including format 3 | Otherwise W×H>1024 | `(W>>2)×8×Q` |
| Direct 16-bit, including format 3 | Otherwise | W |

The comparisons are strict, and the wide-row test comes first. Use the **original clipped H**, not H′, when choosing a texture branch. For a palette4 rectangle at `(6,6)`, size 11×6, drawing area X=8…15,Y=7…10 clips it to W=8,H=4. With both semitransparency and mask checking, q=`8+8+4=20`; active skipping gives H′=2 and N=`16+20×2=56` ticks/28 cycles. The two flags still add only one four-tick row surcharge.

With no semitransparency, mask check or skipping, these nonempty examples expose the texture boundaries:

| Format | Clipped W×H | Texture overhead | Total ticks | Cycles |
|---|---|---|---|---|
| 8-bit palette | 64×32 | 64 | 4112 | 2056 |
| 8-bit palette | 64×33 | 128 | 6352 | 3176 |
| Direct 16-bit | 64×16 | 64 | 2064 | 1032 |
| Direct 16-bit | 64×17 | 256 | 5456 | 2728 |
| 8-bit palette | 128×1 | 128 | 272 | 136 |
| 8-bit palette | 129×1 | 256 | 401 | 201 |
| Direct 16-bit | 129×1 | 512 | 657 | 329 |
| 4-bit palette | 129×1 | 129 | 274 | 137 |

At W=64, Q=2. Enabling skipping on the 64×33 palette8 case keeps its overhead 128 and q=192, but uses H′=16: `16+192×16=3088` ticks/1544 cycles. It must not switch to the lower-area branch. These thresholds specify command cost, not a physical texture-cache capacity. Even H=1 remains H′=1 when skipping is active; do not infer one stored row from that minimum cost.

**Fill and raw transfers.** For fill, decode rounded width `w=((rawWidth AND 3FFH)+15) AND NOT 15` and height `h=rawHeight AND 1FFH`:

```
fill ticks = 46 + ((w >> 3) + 9)×h
copy ticks = 2×W×H
```

Copy W/H use image normalization: `W=((rawWidth−1) AND 3FFH)+1`, `H=((rawHeight−1) AND 1FFH)+1`. A 32×4 fill costs 98 ticks/49 cycles; a 3×5 copy costs 30 ticks/15 cycles. Fill cost is unchanged by suppressed rows. Even raw fill width=0,height=4 costs 82 ticks/41 cycles although it writes no pixels. Conversely, zero image dimensions select a 1024×512 copy, costing 1,048,576 ticks/524,288 cycles—not an empty operation.

Upload emission and the readback command each cost one tick, independently of area. That is not the time to supply an upload payload, move the data or make readback ready. Readback has separate pending/submitted/ready phases and stops queued parsing while nonidle.

E1–E6, IRQ request and non-NOP unmatched GP0 operations also cost one tick; their selected register/IRQ effect occurs at parser execution, before that completion deadline. Boundary-discarded NOPs have no queued cost interval. GP1 controls have no separate queued raster-cost interval. For example, an IRQ behind a 32×4 fill starting at cycle 0 executes and raises at 49, then completes at 50; the IRQ does not prove presentation has finished.

Finally, budget the **CPU work**. A scalar word issue has two base CPU cycles; include address/value preparation, control flow and admission waits as well. CPU issue and GPU execution can overlap. The following complete timeline shows how their costs combine for one fill-and-fence sequence.

## 5.35 Count a complete issue-and-fence interval

The following complete instruction sequence supplies a 32×4 red fill, queues an IRQ marker, waits for it, saves the observed status, acknowledges both IRQ latches, checks idle separately, saves that status and returns. **Its cost is 83 system cycles**, not 83 plus the fill's command interval. CPU issue/polling and GPU work overlap.

The schedule includes address setup, loads and stores, status tests, taken branches and return. Use the instruction sequence of the program being timed when applying the same method to another implementation.

### 5.35.1 Define the interval and initial state

Start counting at the first instruction of an already-entered function. The GPU is idle, with empty input, no partial packet or readback, its IRQ request clear, and no pending competing work. The reset drawing/display state supplies no row suppression. DMA does not own either port, ingress is not stopped, the controller pending and mask words are zero, and no IRQ, fault or other bus master interrupts the interval. The beam remains on reset line parity zero throughout these 83 cycles. Startup, the caller and initialization that establishes these preconditions are outside this stated interval.

Reserve eight writable bytes at illustrative RAM address `0x080c1000` for the two status snapshots. This is cartridge-owned result storage, not GPU scratch RAM. The function loads every address/value it uses; none of its CPU registers is assumed preloaded:

| Register | Constant loaded by LOADK |
|---|---|
| r0 | GP0 address `0x0801023c` |
| r1 | GP1/GPUSTAT address `0x08010240` |
| r2 | Red fill word `0x020000ff` |
| r3 | Zero: destination `(0,0)` and the comparison operand |
| r4 | Fill size word `0x00040020`:32×4 |
| r5 | IRQ marker word `0x1f000000` |
| r6 | GPU IRQ status mask `0x01000000`, bit 24 |
| r7 | GP1 acknowledge word `0x02000000` |
| r8 | Controller acknowledge address `0x08000004` |

| Register | Constant loaded by LOADK |
|---|---|
| r9 | Controller GPU IRQ bit `0x00000040` |
| r10 | Result-storage address `0x080c1000` |
| r11 | GPU idle status mask `0x04000000`, bit 26 |

r12 holds a GPUSTAT read; r13 holds its masked test. The twelve LOADK instructions cost twelve cycles. All memory operations below are scalar displaced **Word** loads/stores with two-cycle base cost. An address `base+4` uses displacement 1; no extra address ADD is hidden in this schedule. Every GP0 store uses the same r0 address, not an incrementing destination.

### 5.35.2 Specify every stored instruction

The operation column describes ISA effects, not BLua source syntax. EQ uses A=1: if the masked value differs from zero, skip the following JMP. Thus a clear status bit executes the backward branch; a set bit exits the polling loop. BAND, EQ and an executed JMP each cost one cycle. A skipped JMP is not issued.

| Instruction index | Operation |
|---|---|
| 0…11 | LOADK: load the twelve constants above into r0…r11 |
| 12 | STORE_MEM_D: r2 → Word[r0], fill header |
| 13 | STORE_MEM_D: r3 → Word[r0], destination |
| 14 | STORE_MEM_D: r4 → Word[r0], size; complete the fill packet |
| 15 | STORE_MEM_D: r5 → Word[r0], IRQ marker |
| 16 | LOAD_MEM_D: Word[r1] → r12 |
| 17 | BAND: r12 AND r6 → r13 |
| 18 | EQ: compare r13 with r3; skip instruction 19 when unequal |
| 19 | JMP: jump to instruction 16 |
| 20 | STORE_MEM_D: r12 → Word[r10], retain IRQ observation |
| 21 | STORE_MEM_D: r7 → Word[r1], acknowledge GPU request |
| 22 | STORE_MEM_D: r9 → Word[r8], acknowledge controller pending bit |
| 23 | LOAD_MEM_D: Word[r1] → r12 |
| 24 | BAND: r12 AND r11 → r13 |
| 25 | EQ: compare r13 with r3; skip instruction 26 when unequal |
| 26 | JMP: jump to instruction 23 |
| 27 | STORE_MEM_D: r12 → Word[r10+4], retain idle observation |
| 28 | RET: return, two cycles |

There are 29 stored instruction words, but loops change the executed count. In this case instructions 16…19 execute nine unsuccessful passes, followed by one successful pass 16…18. Instructions 23…25 succeed at the first idle check; JMP at instruction 26 is not issued.

This is an execution fence and status readout, not image readback or a displayed frame. A C0 image read requires its separate READY/ownership/count protocol and has no fixed whole-transfer latency established by the one-tick command cost. Neither the 83-cycle function nor a sum of drawing costs establishes physical bus bandwidth, sustained fill rate or the CPU cost of a complete game frame.

### 5.35.3 Follow issue, command completion and readout

Count cycles relative to function entry. Each mapped access occurs at its instruction's charged endpoint:

| Cycle | Event |
|---|---|
| 12 | All constants are loaded |
| 14,16,18 | The three fill words are accepted; the complete fill starts at 18 |
| 20 | IRQ marker accepted behind the active fill, without waiting for idle |
| 22,27,…,62 | Nine completed status reads find GPU IRQ bit 24 clear |
| 67 | Fill reaches `18+49`; marker executes and sets IRQ. The tenth read sees IRQ set but idle clear |

| Cycle | Event |
|---|---|
| 68 | The marker’s own one-tick interval completes |
| 69 | BAND/EQ finish selecting the successful IRQ path; JMP at instruction 19 is skipped |
| 71 | First retained status is written to result word 0 |
| 73,75 | GPU request and controller pending latches are acknowledged separately |
| 77 | First separate idle read completes with bit 26 set |
| 79 | Idle BAND/EQ complete; JMP at instruction 26 is skipped |
| 81,83 | Second status is retained, then RET completes |

The fill costs 98 command ticks, 49 cycles. The marker adds its own one-cycle interval: completion of the queued work is at 68, not at the first IRQ observation at 67. Under the stated reset/parity conditions, the saved words are `11922000H` (IRQ set, idle clear) and `14922000H` (IRQ clear, idle set). In general, beam and configuration fields must be interpreted rather than compared with these whole example constants.

| Executed work | Instructions | CPU cycles |
|---|---:|---:|
| Address/value setup | 12 LOADK | 12 |
| Supply fill and marker | 4 stores | 8 |
| Nine unsuccessful IRQ passes | 9×(load, BAND, EQ, JMP) | 45 |
| Successful IRQ pass | load, BAND, EQ | 4 |
| Retain IRQ status; acknowledge GPU and controller | 3 stores | 6 |
| Successful idle check | load, BAND, EQ | 4 |
| Retain idle status | 1 store | 2 |
| Return | 1 RET | 2 |
| **Complete entered-function interval** | **63** | **83** |

No direct CPU store stalls in this sequence; the marker fits in the input while the fill executes. The 49 IRQ-poll cycles are still charged CPU work, not free waiting. Replacing polling with an interrupt handler changes setup, entry, acknowledgement and resumption costs and needs its own budget.

## 5.36 Move a raw image through DMA

Move an image along this route: RAM → fixed GP0 port → GPU local memory → fixed GPUREAD port → RAM. GP0 and GPUREAD share address `0x0801023c`; GP1 and GPUSTAT share `0x08010240`. GP1 opcode `04` selects the GPU DMA direction, but does not start a DMA channel.

This example owns DMA0 and the GPU ports, masks interrupts, and polls completion. It uploads a raw 9 × 9 image at local-memory coordinate (96, 64), then reads the same rectangle into a separate RAM buffer. It does not program PCRTC or display the image.

| Transfer | GP1 direction | DMA CONTROL | Read endpoint | Write endpoint | Requests |
|---|---|---|---|---|---|
| RAM → GP0 | 2 (`0x04000002`) | `0x00003c41` | RAM, incrementing | GP0, fixed | Read FORCE; write GX_WRITE |
| GPUREAD → RAM | 3 (`0x04000003`) | `0x00003c0a` | GPUREAD, fixed | RAM, incrementing | Read GX_READ; write FORCE |

Both controls select up to 16 words per block. COUNT counts words, not pixels or bytes. Each admitted block captures its addresses, count, and control; requests are sampled at admission. Both request selectors must permit admission. FORCE permits it unconditionally.

### 5.36.1 Upload, read back and release the ports

Start with GPU idle, DMA0 inactive, no partial packet/readback, and exclusive ownership of both units. Mask interrupt delivery for this polling sequence. The source and destination below are RAM arrays; COUNT is in 32-bit words. The 81 source pixels have raw values 1 through 81. The last high half is not part of the image.

```
module<entry>

data upload: word[44] = {
    0xa0000000, 0x00400060, 0x00090009, 0x00020001,
    0x00040003, 0x00060005, 0x00080007, 0x000a0009,
    0x000c000b, 0x000e000d, 0x0010000f, 0x00120011,
    0x00140013, 0x00160015, 0x00180017, 0x001a0019,
    0x001c001b, 0x001e001d, 0x0020001f, 0x00220021,
    0x00240023, 0x00260025, 0x00280027, 0x002a0029,
    0x002c002b, 0x002e002d, 0x0030002f, 0x00320031,
    0x00340033, 0x00360035, 0x00380037, 0x003a0039,
    0x003c003b, 0x003e003d, 0x0040003f, 0x00420041,
    0x00440043, 0x00460045, 0x00480047, 0x004a0049,
    0x004c004b, 0x004e004d, 0x0050004f, 0xffff0051,
}

bss returned: word[41]
local dma<const>: *word = 0x0800000c
local gp0<const>: *word = 0x0801023c
local gp1<const>: *word = 0x08010240
local irq_ack<const>: *word = 0x08000004
local irq_mask<const>: *word = 0x08000008

*irq_mask = 0
*gp1 = 0x00000000                  -- Reset stream/register state, not VRAM
*gp1 = 0x09000000                  -- Closed Y-address gate
*gp0 = 0xe6000000                  -- No mask check or force-mask
*gp1 = 0x04000002                  -- CPU-to-GP0 DMA request selection
dma[0] = upload                   -- READ_ADDR
dma[1] = 0x0801023c                -- WRITE_ADDR: fixed GP0
dma[2] = 44                        -- TRANSFER_COUNT
dma[3] = 0x00003c41                -- CONTROL: up to sixteen words/block
dma[5] = 1                         -- TRIGGER
while (dma[4] & 2) == 0 do         -- STATUS.DONE
end
while (*gp1 & 0x04000000) == 0 do  -- GPU completion is separate
end

*gp0 = 0xc0000000
*gp0 = 96 | (64 << 16)
*gp0 = 9 | (9 << 16)
*gp0 = 0x1f000000                  -- Marker waits behind the counted readback
while (*gp1 & 0x08000000) == 0 do  -- READY, not idle
end
*gp1 = 0x04000003
dma[0] = 0x0801023c                -- Fixed GPUREAD
dma[1] = returned                  -- Incrementing RAM destination
dma[2] = 41
dma[3] = 0x00003c0a
dma[5] = 1
while (dma[4] & 2) == 0 do
end
while (*gp1 & 0x01000000) == 0 do  -- Ordered marker
end
*gp1 = 0x02000000                  -- GPU request latch
*irq_ack = 0x00000040              -- Separate central pending latch
```

```
while (*gp1 & 0x04000000) == 0 do  -- Marker interval must finish too
end
*gp1 = 0x04000000                  -- DMA request selection off
```

The upload contains **44 words**: three A0 header words and 41 payload words, 176 bytes in all. Packing is continuous, low halfword first; it does not restart at a nine-pixel row boundary. Payload word 4 is `000A0009H`, whose two halves belong to different rows. The last source word is FFFF0051H; only 0051H is a pixel. The unused FFFFH neither writes a pixel nor begins a command.

Readback fills **41 words**, 164 bytes, without headers. Returned words 0, 4, 39 and 40 are respectively `00020001H`, `000A0009H`, `0050004FH` and `00000051H`. The final unused halfword reads zero. At DONE, each DMA COUNT is zero; upload advanced its RAM source by 176 bytes and readback advanced its RAM destination by 164 bytes. The GP0/GPUREAD endpoint addresses did not advance. The final GPUREAD latch is 00000051H, and another CPU read with no READY stream returns that latch rather than an extra pixel.

### 5.36.2 Account for admitted transfer time

Installed RAM timing charges one cycle per word plus one per burst; the I/O side charges zero. Transfers between these regions use the larger side charge.

| Transfer | Words | Admitted blocks | Block charges | Total |
|---|---|---|---|---|
| Upload packet | 44 | 16 + 16 + 12 | 17 + 17 + 13 | 47 cycles |
| READY readback | 41 | 16 + 16 + 9 | 17 + 17 + 10 | 44 cycles |

These are admitted DMA charges, not end-to-end measurements. They exclude CPU preparation, request waits, other channels, readback preparation, and display timing. The final blocks are partial blocks, not padded sixteen-word transfers.

### 5.36.3 Keep completion boundaries separate

DMA **DONE** means the channel finished its transfers. GP0 DMA words first enter a separate sixteen-word staging buffer; block-end processing advances the pipeline. GP0 parsing consumes the counted A0 payload and emits the retained upload command. Neither channel DONE nor packet acceptance alone establishes command completion.

After the readback drain, an ordered GPU marker and GPU idle are polled separately. GPUSTAT bit 28 reports packet acceptance, bit 27 reports pixel **READY**, and bit 26 reports idle; idle excludes pending deadlines, FIFO contents, partial packets, image transfers, polylines, and readback. GPU and controller acknowledgements remain separate.

Wait for READY before reading image data. Consuming the last word clears READY and its DMA request, allowing parsing to advance to the queued marker. Use PCRTC's VBlank-entry sequence separately when selecting these pixels for display.

DMA owns its selected endpoint throughout BUSY, even before admission. Do not interleave CPU GP0 writes or consume GPUREAD pixels as a second reader. A CPU GPUREAD access while DMA owns readback returns the retained latch, not the next pixel. Writing START while DMA is BUSY does not restart it.

No display setup occurs in this sequence. To show the uploaded rectangle, select a PCRTC source and use its VBlank-publication sequence separately. DONE, READY, counted drain, marker and idle are not display acknowledgements.

# 6 Interrupt controller

Interrupts let a cartridge service device completion and video events while the CPU performs other work or waits in HALT. The controller records each pending source in FLAGS and combines FLAGS with MASK to assert the CPU's maskable interrupt line. Each source has one pending bit, so repeated arrivals can coalesce before service.

Sources include DMA completion, PCRTC events, VBlank, GEO completion/errors, APU events, GPU markers, IMGDEC events and the two cartridge sockets. Program a source's local event controls, install its service routine, then enable its MASK bit. The source chapter defines any local acknowledgement or re-arming.

The CPU accepts the line when current interrupt enable `STATUS.IEc` is set. All enabled sources share an IRQ entry; software reads FLAGS, services the selected sources and acknowledges them. The controller and CPU entry protocol are specified below.

## 6.1 IRQ specifications

| Resource | Specification |
|---|---|
| Register interface | Three 32-bit word ports, four-byte spacing; 12 bytes of address space |
| Pending storage | One 32-bit pending word, not an event queue |
| Delivery selection | One 32-bit enable word, reset to zero |
| Named source bits | Eleven, bits 0–10; the remaining word bits are not removed by controller operations |

These register-address bytes are not a payload RAM capacity. The full address/access/reset and named-source tables appear in the register reference.

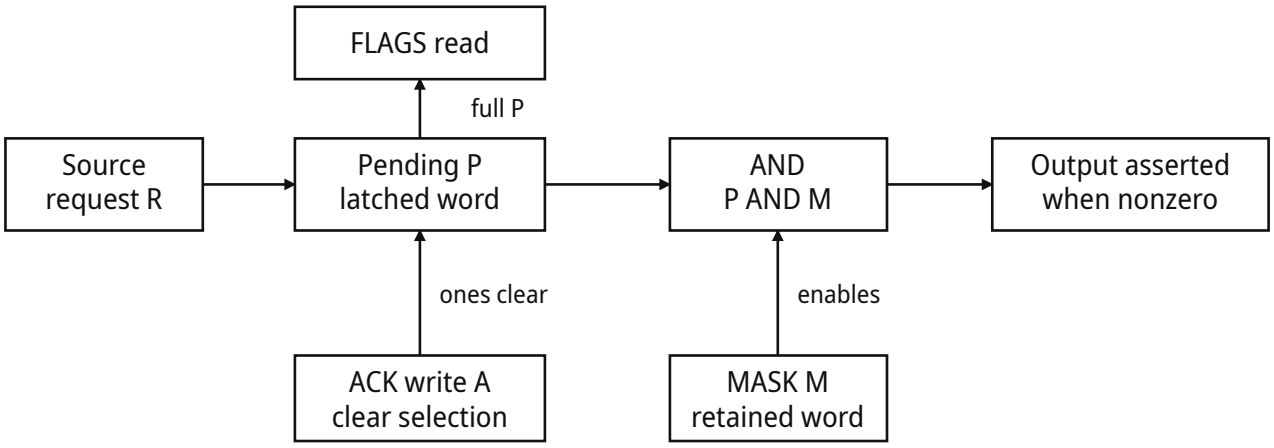



Figure 6.1: Pending latches and delivery mask

FLAGS reads the full pending word. The diagram ends at the controller output; the CPU interrupt-enable gate is separate.

### 6.1.1 Latching, masking and acknowledgement

Let **P** be the pending word, **M** the mask, **R** a raised-bit word and **A** an accepted ACK word. All bitwise operations below are 32-bit operations:

```
Raise R:       P ← P OR R         M is unchanged
Write MASK W:  M ← W              P is unchanged
Write ACK A:   P ← P AND NOT A    M is unchanged

Interrupt indication asserted ⇔ (P AND M) ≠ 0
```

Raising a bit latches it regardless of MASK. Raising it again while it remains pending leaves the same bit set; the controller does not record how many times it was raised. This is coalescing, not a backlog of events.

MASK selects which pending bits can assert the output. Each MASK write replaces the previous mask rather than adding to it. Consequently, enabling an already-pending bit can assert the output without another raise, while disabling it leaves its pending state intact.

ACK clears exactly the pending bits selected by its written ones. Written zeros preserve the corresponding pending bits, so writing `00000000H` changes nothing. Reading FLAGS also changes nothing. An ACK need not clear every pending bit: the output remains asserted whenever another bit remains set in both P and M. The same equation applies to unnamed bits retained in those words.

An accepted ACK or MASK write changes the controller state before that write completes. This is the register-write visibility edge; it does not specify CPU interrupt delivery, a priority order, or a fixed interrupt-entry delay.

### 6.1.2 Worked operation trace

The following sequence uses `IRQ_GPU` (`00000040H`) and `IRQ_VBLANK` (`00000004H`) only as bit names. “Raise” means that the indicated word is presented to the controller; no device trigger is implied. Each row shows state after the operation.

| Operation | P | M | Indication |
|---|---|---|---|
| Reset | `00000000H` | `00000000H` | Clear |
| Raise `00000040H` | `00000040H` | `00000000H` | Clear |
| Raise `00000040H` again | `00000040H` | `00000000H` | Clear |
| Write MASK `00000044H` | `00000040H` | `00000044H` | Asserted |
| Raise `00000004H` | `00000044H` | `00000044H` | Asserted |
| Write ACK `00000040H` | `00000004H` | `00000044H` | Asserted |
| Write MASK `00000040H` | `00000004H` | `00000040H` | Clear |
| Write ACK `00000000H` | `00000004H` | `00000040H` | Clear |
| Write MASK `00000004H` | `00000004H` | `00000004H` | Asserted |
| Write ACK `00000004H` | `00000000H` | `00000004H` | Clear |

The repeated GPU raise coalesces into one pending bit. The first ACK clears GPU but leaves VBlank pending and enabled. Replacing MASK with `00000040H` hides VBlank without clearing it; enabling VBlank again exposes that same pending bit. Only the final ACK clears it.

## 6.2 CPU delivery and waiting

### 6.2.1 From pending requests to `irq(flags)`

A maskable interrupt is accepted only when both conditions hold:

- At least one bit is set in `IO_IRQ_FLAGS & IO_IRQ_MASK`.
- The CPU’s current interrupt-enable bit, `STATUS.IEc`, is set.

Masking a source does not discard its pending request. Source requests accumulate in `IO_IRQ_FLAGS` independently of `IO_IRQ_MASK`; repeated requests for an already-pending source retain one bit, not a count.

The CPU does not choose the highest-priority source or take one vector per pending bit. An accepted maskable interrupt takes the shared IRQ entry for the current execution context: the active program's entry in user mode, or the system entry in non-user mode. Software determines which pending sources to service.

The CPU enters the IRQ function whose absolute function-record address is in the selected ROM header. It supplies no function arguments. That entry routine runs as the exception frame and is responsible for reading source state and executing RFE when service is complete.

The standard BLua IRQ entry reads the **complete `IO_IRQ_FLAGS` word** and, when it is nonzero, calls `irq(flags)`. This argument reflects the later FLAGS read; it can include masked pending bits as well as the bit that enabled delivery. The callback returns normally to the entry routine, which executes RFE. A separately authored IRQ entry can dispatch differently, but must perform its own source reads and exception return.

Neither exception entry nor ordinary interrupt return acknowledges a source. Software must write the appropriate bits to `IO_IRQ_ACK`. Writing one clears the corresponding pending bit; writing zero leaves it unchanged. A source left pending and enabled can request delivery again once `IEc` is restored.

> **Caution — delivery is not source selection.** Do not assume that every bit supplied to `irq(flags)` was enabled, or that the CPU has already acknowledged any of them.

### 6.2.2 CPU state on entry and return

The relevant CP0 registers are `STATUS` (register 12), `CAUSE` (13), and `EPC` (14). In `STATUS`, bit 0 is current interrupt enable and bit 1 selects current user mode. Cartridge entry begins with `STATUS = 00000003H`: user mode, interrupts enabled. System entry uses `00000001H`: non-user mode, interrupts enabled.

On accepted IRQ entry, the CPU records the interrupted PC in `EPC`, writes `00000400H` to `CAUSE`, and shifts the six low mode-stack bits left by two:

```
STATUS := (STATUS & ~0000003fH) | ((STATUS << 2) & 0000003fH)
```

Both current bits become zero. The handler therefore executes in non-user mode with maskable interrupts disabled; entry does not establish automatic priority-based nesting.

For a cartridge beginning at `00000003H`, the complete worked sequence is:

```
Before IRQ:       STATUS = 00000003H  user mode; IEc = 1
After IRQ entry:  STATUS = 0000000cH  non-user mode; IEc = 0
After RFE:        STATUS = 00000003H  user mode; IEc = 1
```

`RFE` returns to the address in `EPC` and restores the low four status bits as follows:

```
STATUS := (STATUS & ~0000000fH) | ((STATUS >> 2) & 0000000fH)
```

`RFE` is an exception-return instruction, not a general procedure return. It requires non-user mode and an exception to return from. User-mode execution takes the coprocessor-unusable exception; execution without an exception to return from hard-halts.

Privilege also matters for CP0 access. In user mode, `MFC0` and `MTC0` take the coprocessor-unusable exception. In non-user mode, `MFC0` can read these three registers; `MTC0` replaces `STATUS` or `EPC`, but does not change `CAUSE`. Writing `STATUS` replaces the whole word, not just `IEc`.

### 6.2.3 Wait for service, not merely for a wake-up

`halt_until_irq` is a statement that executes the CPU's `HALT` instruction. It is not a function call. HALT and controller acknowledgement are separate operations.

An accepted interrupt can leave a one-bit accepted-event indication when the CPU was not already waiting. If this indication is present, the next HALT consumes it and continues without establishing a wait. Otherwise HALT waits for interrupt acceptance. This closes the ordinary interval between checking software state and executing HALT, but it does **not** make HALT an event counter: several accepted interrupts can collapse into the same retained indication.

Use an explicit software predicate. For example, reserve a mutable service sequence in BSS, whose initial value is zero:

```
bss service_count: word
```

In the selected service path of `irq(flags)`, complete that source's work and required acknowledgement, then advance the sequence:

```
*service_count = *service_count + 1
```

The caller snapshots the sequence **before** starting the operation that will request this service, then waits for it to change:

```
local before<const> = *service_count
-- Start the operation that will request the selected service here.
while *service_count == before do
    halt_until_irq
end
```

These are fragments of a source-specific protocol, not a replacement IRQ handler. Only the selected completed service advances this counter. Other interrupts may wake the caller without satisfying its predicate, and the selected handler may already have completed before HALT executes. The loop handles both cases. The software counter records completed services; it cannot reconstruct source requests that already coalesced in a controller pending bit.

> **Caution — a pending bit alone need not wake HALT.** Maskable acceptance still requires a matching mask bit and `IEc = 1`. Do not wait for a source while disabling its delivery and expect its raw pending bit to suffice.

Device timers continue to progress during HALT; parked CPU instructions do not execute. No fixed request-to-handler latency or guaranteed acceptance boundary after every source instruction is specified here.

## 6.3 Timing and service cost

Interrupt service instructions use the 33.8688 MHz system timebase: 33,868,800 cycles per second, approximately 29.525699 ns per cycle. A service interval includes time pending, exception entry, source dispatch, handler work and exception return.

There is no delayed-enable instruction after a MASK write. An already-pending request is eligible at the next CPU execution boundary that satisfies both gates; an NMI or another CPU hold can still prevent maskable acceptance.

ACK and MASK change controller state on their accepted word writes. A pending enabled bit can therefore assert the line as a result of a MASK write, without another source request. CPU acceptance also requires IEc; assertion of the controller output and execution of a cartridge handler are not the same edge.

| Instruction | ISA base cost, system cycles |
|---|---|
| `HALT` | 1 |
| `MFC0` | 1 |
| `MTC0` | 1 |
| `RFE` | 1 |
| `LOAD_MEM`, including the IRQ entry's FLAGS read | 2 |

Add these costs to the executed entry and handler instructions when budgeting service. Device timing continues during HALT and while a request is masked. Request-to-handler time therefore depends on those waits as well as on the instruction sequence. The example below accounts for the entry instructions separately from the handler.

### 6.3.1 Worked IRQ entry cost

Consider an IRQ entry that reads FLAGS and compares it directly with an immediate zero constant. Its instruction-base charge is as follows:

| Entry operation | Nonzero FLAGS path | Zero FLAGS path |
|---|---|---|
| Load FLAGS address | 1 | 1 |
| Read FLAGS word | 2 | 2 |
| Compare with zero | 1 | 1 |
| Branch past the call | Skipped | 1 |
| Load the `irq` function | 1 | — |
| Move FLAGS into its argument | 1 | — |
| Call `irq(flags)` | 2 | — |
| `RFE` | 1 | 1 |
| **Entry-sequence total, cycles** | **9** | **6** |

Handler execution, including its own return instruction, is extra. If the entry instead loads zero into a register before comparing, add one cycle to each path: 10 and 7 cycles. The actual entry instructions determine this budget; it is not a fixed interrupt-entry charge or a request-to-service delay. In particular, it does not include time spent pending behind a closed MASK or disabled IEc.

## 6.4 Acknowledge the source and the controller separately

Controller ACK removes a pending bit. It is not a command to the unit that raised that bit. The GPU provides a concrete example of the distinction:

- GP0 word `1f000000H` is an in-order interrupt marker. When it executes with GPUSTAT24 clear, it sets that local bit and raises controller bit 6.
- GP1 word `02000000H` clears GPUSTAT24. It does not clear controller bit 6.
- Writing `00000040H` to IO_IRQ_ACK clears controller bit 6. It does not clear GPUSTAT24.

The marker's low 24 bits do not select an interrupt number. A marker while GPUSTAT24 remains set does not raise another GPU request. Nor does GPU idle alone raise an interrupt.

### 6.4.1 Controller clear alone does not re-arm the GPU

Start with both GPU-local and controller pending bits clear. Keep delivery masked while following this sequence:

| Ordered operation | GPUSTAT24 | FLAGS bit 6 | Meaning |
|---|---|---|---|
| Execute first GP0 marker | 1 | 1 | Both latches record the request |
| Write ACK=`00000040H` | 1 | 0 | Controller cleared; GPU still latched |
| Execute another GP0 marker | 1 | 0 | No new request while the GPU latch stays set |
| Write GP1=`02000000H` | 0 | 0 | GPU re-armed |
| Execute a further GP0 marker | 1 | 1 | A new independent request is recorded |

The opposite partial acknowledgement is different: clearing only GPUSTAT24 leaves controller bit 6 pending. A new marker can then raise bit 6 again, but OR-ing it into an already-set pending bit still leaves just one pending bit. The programming example below demonstrates that coalescence before enabling delivery.

Follow the local acknowledgement sequence of each source. The GPU command reference defines marker ordering and GPU completion; PCRTC defines display publication. A marker service establishes that the marker was reached, while the display has its own publication edge.

### 6.4.2 A pending bit cannot count arrivals during service

For one selected source, let P be its pending bit. A raise sets P to 1; ACK sets P to 0. The order matters:

| Operation order | Final P |
|---|---|
| Raise, then ACK | 0 |
| ACK, then raise | 1 |
| Raise, raise, then ACK | 0 |

Consequently a flags read followed later by an acknowledgement is not an atomic read-and-consume transaction. A second request that arrives before that clear can coalesce with the first and be removed by the same clear. A request after the clear becomes pending again. No acknowledgement order can turn this one-bit latch into a count of all source events.

Keep a software service count distinct from a source-event count. Several requests can coalesce into one service; no software counter recovers arrivals already lost in the pending latch.

### 6.4.3 Install the cartridge handler

For the standard BLua entry sequence, define `function irq(flags)` before enabling interrupt delivery. The handler selects the sources it supports, services their local state, and clears their controller bits. Clear only the supported subset: `flags & 00000040H` clears GPU pending while preserving other sources supplied in the same full FLAGS argument.

Prepare each source's service state before enabling its MASK bit. The following example supplies the global callback and performs both GPU and controller acknowledgement.

## 6.5 Program and service a GPU marker

### 6.5.1 Starting conditions

Use ordinary BIOS cartridge entry with the inherited beam running and CPU interrupt enable set. Own GP0, the GPU interrupt handler and MASK exclusively; no unfinished packet, readback or DMA transfer may remain. This example leaves VBlank pending but masked so that the full handler argument can be distinguished from the enabled subset.

```
bss gpu_services: word
bss delivered_flags: word
local irq_flags<const>: *word = 0x08000000
local irq_ack<const>: *word = 0x08000004
local irq_mask<const>: *word = 0x08000008
local gp0<const>: *word = 0x0801023c
local gp1<const>: *word = 0x08010240

function irq(flags)
    if (flags & 0x00000040) ~= 0 then
        *delivered_flags = flags
        *gp1 = 0x02000000
        *irq_ack = 0x00000040
        *gpu_services = *gpu_services + 1
    end
end
```

With the handler installed, establish the masked requests, then enable GPU delivery and wait for the two services:

```
*irq_mask = 0
while (*gp1 & 0x04000000) == 0 do
end
*gp1 = 0x02000000
*irq_ack = 0xfffffffb       -- Clear stale requests except VBlank.
```

```
while (*irq_flags & 4) == 0 do
end

*gp0 = 0x1f000000          -- First masked GPU request.
while (*gp1 & 0x01000000) == 0 do
end
*gp1 = 0x02000000          -- Re-arm GPU; central pending is still set.
*irq_ack = 0               -- No pending bit is cleared.
*gp0 = 0x1f000000          -- A second request coalesces into bit 6.
while (*gp1 & 0x01000000) == 0 do
end

*irq_mask = 0x00000040
while *gpu_services == 0 do
    halt_until_irq
end
*gp0 = 0x1f000000          -- The handler cleared both required latches.
while *gpu_services == 1 do
    halt_until_irq
end
*irq_mask = 0
```

Under the stated ownership conditions, the ordered states are as follows. GPU24 is shown as a masked word. Rows are operation boundaries, not equal-duration time slots or atomic multi-register samples.

| Operation | FLAGS | MASK | GPU24 | Completed GPU services |
|---|---|---|---|---|
| VBlank pending, before a marker | 04 | 00 | 00000000 | 0 |
| First marker executes | 44 | 00 | 01000000 | 0 |
| GPU-local clear only | 44 | 00 | 00000000 | 0 |
| Second marker executes | 44 | 00 | 01000000 | 0 |
| First handler, before acknowledgement | 44 | 40 | 01000000 | 0 |
| First service complete | 04 | 40 | 00000000 | 1 |
| Third marker serviced | 04 | 40 | 00000000 | 2 |
| Delivery masked again | 04 | 00 | 00000000 | 2 |

Both services receive `44H`, although the enabled subset is only `40H`. The handler clears GPU only, so disabled VBlank remains pending. ACK always reads zero. The two service increments count these controlled services after re-arming; they do not count all possible source arrivals or establish an interrupt-latency bound.

## 6.6 Register reference

Use four-byte-aligned, 32-bit word accesses. The three addresses name separate operations: a pending-word read, a selective clear and an enable-word replacement.

| Address | Name | Access | Bits | Reset |
|---|---|---|---|---|
| `08000000H` | `IO_IRQ_FLAGS` | Read only | 31:0 pending requests | `00000000H` |
| `08000004H` | `IO_IRQ_ACK` | Read zero / write-one-to-clear | 31:0 pending-bit clear selection | `00000000H` |
| `08000008H` | `IO_IRQ_MASK` | Read/write | 31:0 delivery enables | `00000000H` |

### 6.6.1 IO_IRQ_FLAGS — pending requests

Each set bit records a pending request. Reading the word leaves every bit unchanged. A source request sets its pending bit even when the corresponding MASK bit is zero. Another request while the bit is already set does not add a second entry or increment a count.

FLAGS is not the enabled subset. For example, FLAGS=00000044H and MASK=00000040H means that both bit 2 and bit 6 are pending, but only bit 6 enables the controller output. Writing FLAGS does not acknowledge it; the write is rejected with READ_ONLY bus fault 3.

### 6.6.2 IO_IRQ_ACK — selective pending clear

A written one clears the corresponding pending bit; a written zero preserves it. The operation is:

```
pending := pending AND NOT written_word
```

The clear takes effect on the accepted register write. It does not replace MASK, clear a source unit's local status, or record the word that was written. ACK reads 00000000H after reset and after each accepted write.

With FLAGS=00000044H, writing ACK=00000040H leaves FLAGS=00000004H. Writing ACK=00000000H changes nothing. Writing ACK=ffffffffH clears every pending bit, including disabled ones; it is not selective service of the source that caused an interrupt.

### 6.6.3 IO_IRQ_MASK — delivery enables

A set bit enables that pending bit to contribute to the output line. A clear bit excludes it from the output line without erasing it. Reading MASK returns the current enable word. An accepted write replaces all 32 bits immediately; it is not a set-bits command.

For example, replacing MASK=00000040H with 00000004H disables bit 6 and enables bit 2. To keep both enabled the replacement word must be 00000044H. MASK=0 closes delivery from this controller; it neither clears pending requests nor acknowledges their producers. Enabling an already-pending bit can assert the output without waiting for another source request.

### 6.6.4 Controller reset

Reset clears pending and MASK and returns ACK readback to zero. Writing MASK=0 alone does not reproduce reset: pending requests remain. Writing ACK with all ones alone does not reproduce reset either: the enable word remains unchanged. Source-unit reset and re-arming remain that unit's separate operations.

### 6.6.5 Named source bits

The same bit position is used in FLAGS, ACK and MASK. The names identify the source; they do not make all source units use the same request or re-arm rule.

| Bit | Mask | Source name |
|---|---|---|
| 0 | 00000001H | IRQ_DMA0_DONE |
| 1 | 00000002H | IRQ_GX_PCRTC |
| 2 | 00000004H | IRQ_VBLANK |
| 3 | 00000008H | IRQ_GEO_DONE |
| 4 | 00000010H | IRQ_GEO_ERROR |
| 5 | 00000020H | IRQ_APU |
| 6 | 00000040H | IRQ_GPU |
| 7 | 00000080H | IRQ_IMGDEC |
| 8 | 00000100H | IRQ_DMA1_DONE |
| 9 | 00000200H | IRQ_CARTRIDGE_SLOT0 |
| 10 | 00000400H | IRQ_CARTRIDGE_SLOT1 |

Bits 31:11 are unnamed, not hardwired-zero fields. MASK retains their written values, and the controller's pending, clear and output equations operate on the complete 32-bit words. This does not assign a cartridge-programmable request source to an unnamed bit. Cartridge-source device programming belongs to Book III.

## 6.7 Register access and faults

The following rules concern the three controller words. A word access still obeys the read-only rule for FLAGS.

| Mapped data access | Address condition | Result |
|---|---|---|
| Register-word or U32 read/write | Four-byte aligned | One complete register operation |
| U8 read | At a register's word address | Full register read, returning bits 7:0 |
| U8 read | Word offset 1,2 or 3 | UNALIGNED_IO bus fault 2; no register operation |
| U8 write | Any controller byte address | UNALIGNED_IO bus fault 2; no register operation |
| U16 read/write | Even controller address | UNALIGNED_IO bus fault 2; no register operation |
| Word write to FLAGS | At `08000000H` | READ_ONLY bus fault 3; pending unchanged |

CPU alignment rejection precedes these mapped-access rules. See Memory — Bus-fault status and CPU exceptions for the shared report and exception behavior. Clear that report through its BUS_FAULT_ACK port; IRQ_ACK clears controller pending bits only.

F32 transfers one word's bits. F64 transfers the low word at A and the high word at A+4, stopping if the first access faults. Thus an F64 store starting at FLAGS faults before ACK is reached. Starting at ACK instead performs a pending clear followed by a MASK replacement; it is not a combined controller operation. Use separate word stores when those are the operations intended.

# 7 Direct memory access

The direct memory access controller moves 32-bit words between main RAM, ROM, cartridge storage and device ports. Its two channels have identical capabilities and can maintain two outstanding transfers. A shared transfer path serves one block at a time, alternating eligible channels at block boundaries. A channel can therefore wait for a device request while the other makes progress.

Use DMA to copy buffers, supply GP0 command and image words, collect GPUREAD results, or supply and drain APU and IMGDEC FIFOs. Each device publishes request signals indicating when a block can begin. The read/write addresses select the ports or storage; the request selectors synchronize their use.

Transfer time depends on the source and destination regions and the block size. With otherwise equal conditions, cartridge-ROM-to-RAM costs more system cycles than RAM-to-RAM, while system-ROM-to-RAM costs fewer. The route-rate comparison gives the numerical rates. For GPU uploads, DMA delivers words to GP0 and the GPU performs the VRAM work. For APU uploads, DMA fills the transfer FIFO and the APU subsequently commits a sample-RAM batch. Observe transport and device completion separately.

## 7.1 DMA specifications

| Resource | Specification |
|---|---|
| Channels | Two, numbered 0 and 1 |
| Register interface | Six 32-bit word ports per channel; the two banks are not adjacent |
| Transfer word | 32 bits, four bytes, little-endian |
| Address and remaining-count words | 32 bits each; addresses count bytes, transfer count counts words |
| Programmed block length | 1–16 words, independently selected by each channel |
| Shared transfer capacity | One admitted block across both channels |
| Block configuration | Captured read address, write address, full remaining count and CONTROL |
| Completion requests | One IRQ source per channel |
| Timing unit | 33,868,800 system cycles per second |

An admitted block retains its addresses, count and CONTROL until its service edge. Its source words are read at that edge. Keep the source storage stable until its transfer completes. The receiving unit may continue processing after DMA has delivered the last word.

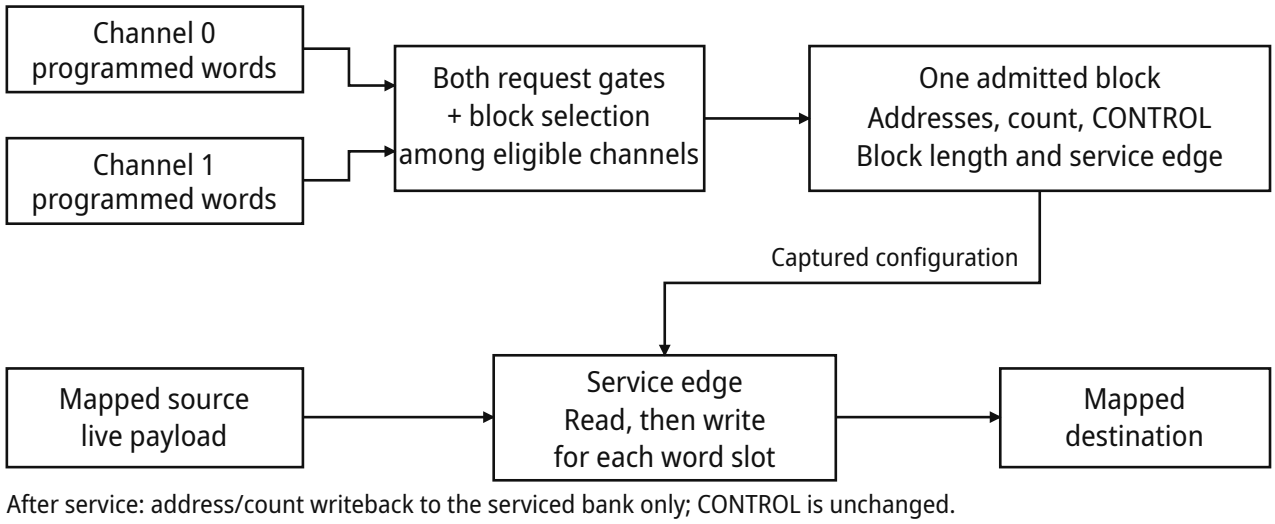



Figure 7.1: Two programmed channels share one admitted block; payload is read at service

The upper path selects and captures configuration. The lower path moves data at the service edge. Address/count writeback belongs to the serviced channel; CONTROL is not rewritten by block completion.

## 7.2 Follow a buffer through a transfer

Consider the twenty-word RAM copy used in the BLua example below. The source array `source` and destination array `copy0` each occupy 80 bytes of ordinary main RAM. They are four-byte aligned and do not overlap. Let S and D denote their first byte addresses.

To copy the array in order, DMA must read successive source words and write successive destination words:

| Word transferred | Source byte address | Destination byte address |
|---|---|---|
| First | S | D |
| Second | S + 4 | D + 4 |
| Sixteenth | S + 60 | D + 60 |
| Seventeenth | S + 64 | D + 64 |
| Twentieth | S + 76 | D + 76 |

Program READ_ADDR=S and WRITE_ADDR=D, enable both increments, and program TRANSFER_COUNT=20. Addresses count **bytes**; the count records **words**. After the copy, the next addresses are S+80 and D+80, and the remaining count is zero. Programming a count of 80 would request 320 bytes, not this array.

The **block length** controls how much of that transfer is served at once. With sixteen-word blocks, the controller first admits sixteen words, leaving four for a second block. After the first block, the addresses are S+64 and D+64 and the remaining count is 4. The final block transfers those four words; software does not pad the array to sixteen more words. A smaller block changes the service and arbitration boundaries, not the address units or total count.

### 7.2.1 A device port is a fixed endpoint

A command stream differs from the RAM copy at its destination. Every command word goes to the **same GP0 port**, `0801023cH`; the GPU interprets the words as a stream. Enable the source increment to walk the command array, but disable the destination increment to keep writing GP0. Advancing this destination by four bytes would address GP1 instead of supplying the next word to GP0. The pixel coordinates within a command are not DMA destination addresses.

The address and the request condition answer different questions. WRITE_ADDR identifies GP0; GX_WRITE says when the GPU can accept a DMA block. Selecting GX_WRITE does not fill in WRITE_ADDR, and writing GP0's address does not select GX_WRITE. For an ordinary RAM copy, FORCE on both sides permits admission without waiting for a device request. In either route, the shared transfer path must also be available, and the device's block and payload rules still apply.

### 7.2.2 Source reuse and result use

Suppose software starts copying the source array, then replaces its final four words with data for the next frame before their block is served. DMA reads those new words, so the destination can contain sixteen words from the old array followed by four from the new one. Block admission captures the addresses and control settings, not a private copy of the source bytes.

Keep the source unchanged through DMA completion, then reuse it. For an ordinary memory copy with successful bus accesses, completion also makes the whole destination available. The polling loops in Copy words in BLua wait for BUSY to clear before the next copy uses `source`. The shared bus-fault report must also be checked; DONE is a count result, not proof of successful accesses.

For a device route, receiving the words and finishing their work are separate events. A GPU command buffer can be released after DMA finishes reading it while the GPU is still processing the submitted commands. Publishing the target image at that point can expose unfinished drawing. First wait for DMA before reusing the source; then use the GPU's ordered completion protocol before using its rendering result, and the PCRTC publication edge before displaying it. Send a command stream to GP0 places the DMA wait before its separate GPU-marker and GPU-idle waits.

## 7.3 Program a transfer

1. Reserve an idle channel for the transfer and prepare the source/destination storage or device protocol.
2. Write READ_ADDR and WRITE_ADDR as byte addresses and TRANSFER_COUNT as the number of 32-bit words.
3. Write CONTROL with the address increments, request selectors and block size.
4. Write START to TRIGGER. BUSY replaces the previous DONE state, and a block can be admitted immediately when both requests are asserted.
5. Observe DMA completion and the shared bus-fault report. For a device route, observe any additional device completion before using its result.

Program all configuration words before START. Select increments for the actual endpoints:

| Route | Read increment | Write increment | Read request | Write request |
|---|---|---|---|---|
| Memory → memory | 1 | 1 | FORCE | FORCE |
| Memory → fixed GP0 port | 1 | 0 | FORCE | GX_WRITE |
| GPUREAD port → memory | 0 | 1 | GX_READ | FORCE |

A fixed address is used again for every word; an enabled increment adds four bytes after each read or write. For the twenty-word RAM example, select both increments, FORCE on both requests and a programmed block length of sixteen. Count remains 20, not an address, byte length, packet count or block count.

Both selected requests must be asserted before a nonzero block is admitted. For a GP0 route, program GP0's address explicitly and select GX_WRITE on that side. FORCE is suitable for an ordinary memory endpoint; a device endpoint must also satisfy its own packet, readiness and block-size rules.

### 7.3.1 Assemble CONTROL

The following expression describes its defined fields. Each request is its four-bit encoding, and block length is 1–16:

```
CONTROL = read_increment
        | (write_increment << 1)
        | (read_request << 2)
        | (write_request << 6)
        | ((block_length - 1) << 10)
```

For that RAM copy, the request encodings are both zero, the two increment bits contribute 3, and `(16−1) << 10` contributes `00003c00H`. Their combination is `00003c03H`, used for the `source → copy0` sequence below. The following CONTROL words select this and the other example routes:

| CONTROL | Operation |
|---|---|
| `00003c03H` | Both increments, FORCE/FORCE, 16-word blocks |
| `00000c03H` | Both increments, FORCE/FORCE, four-word blocks |
| `00003c41H` | Read increment, fixed destination, FORCE/GX_WRITE, 16-word blocks |
| `00003c0aH` | Fixed source, write increment, GX_READ/FORCE, 16-word blocks |

### 7.3.2 START and zero count

TRIGGER bit 0 is START. Writing it while idle replaces STATUS with BUSY and starts the transfer protocol. A zero count instead reaches DONE and requests its IRQ immediately, without a data block and without waiting for either request gate. Do not assume a minimum time between START and completion.

START written while BUSY is ignored: it is not queued, does not restart the channel and does not cancel an admitted block. A write with START clear also does not start anything. TRIGGER reads zero after either kind of write.

## 7.4 Copy words in BLua

The example first copies twenty ROM words into main RAM, then uses both channels to copy that RAM source into two separate destinations. It begins after normal BIOS handoff with both channels idle and reserved for this work. The buffers do not overlap.

Delivery is masked during the polling waits, so the final ACK clears the completion requests raised by the two channels. Array indices select the six consecutive mapped registers in each channel.

```
rodata seed: word[20] = {
    0, 1, 2, 3, 4, 5, 6, 7, 8, 9,
    10, 11, 12, 13, 14, 15, 16, 17, 18, 19,
}
bss source: word[20]
bss copy0: word[20]
bss copy1: word[20]
local dma0<const>: *word = 0x0800000c
local dma1<const>: *word = 0x080103e4
local irq_ack<const>: *word = 0x08000004
local irq_mask<const>: *word = 0x08000008

*irq_mask = 0
*irq_ack = 0x00000101
-- Cartridge ROM -> RAM, both endpoints increment, sixteen-word blocks.
dma0[0] = seed
dma0[1] = source
dma0[2] = 20
dma0[3] = 0x00003c03
dma0[5] = 1
while (dma0[4] & 1) ~= 0 do
end

-- Prepare both RAM copies before starting either channel.
dma0[0] = source
dma0[1] = copy0
dma0[2] = 20
dma0[3] = 0x00003c03
dma1[0] = source
dma1[1] = copy1
dma1[2] = 20
dma1[3] = 0x00000c03
dma0[5] = 1
```

```
dma1[5] = 1
while (dma0[4] & 1) ~= 0 or (dma1[4] & 1) ~= 0 do
end
*irq_ack = 0x00000101
```

After successful bus accesses, `source`, `copy0` and `copy1` contain the twenty seed words in order. Each completed channel has count zero, STATUS=`00000002H` and TRIGGER zero. Its incrementing addresses have advanced 80 bytes. Check the shared bus-fault report as well: DONE alone does not certify successful accesses.

The ROM-to-RAM transfer costs 168 admitted DMA cycles. For the two RAM copies, the round-robin trace gives the 94-cycle schedule when channel 1 starts before channel 0's first block completes. Those costs exclude programming instructions, arbitration waits and interrupt service; sequential START stores are not an atomic simultaneous launch.

### 7.4.1 Send a command stream to GP0

Retain `dma0` and `irq_ack` from above. Own GP0 exclusively, with no partial packet or unread readback. This five-word stream sets mask mode, fills a red 64 × 64 rectangle at (0,0), then requests an in-order GPU interrupt.

```
rodata gpu_commands: word[5] = {
    0xe6000000, 0x020000ff, 0x00000000, 0x00400040, 0x1f000000,
}
local gp1<const>: *word = 0x08010240
while (*gp1 & 0x04000000) == 0 do
end
*gp1 = 0x02000000
*irq_ack = 0x00000041
*gp1 = 0x04000002           -- CPU_TO_GP0 direction; enable GX_WRITE.
dma0[0] = gpu_commands
dma0[1] = 0x0801023c
dma0[2] = 5
dma0[3] = 0x00003c41
dma0[5] = 1
while (dma0[4] & 1) ~= 0 do
end
while (*gp1 & 0x01000000) == 0 do
end
while (*gp1 & 0x04000000) == 0 do
end
*gp1 = 0x02000000
*irq_ack = 0x00000041
```

DMA holds GP0 fixed and advances its ROM source by 20 bytes; its one short block costs 44 cycles. The DMA wait releases the command-stream source; the following waits separately observe the GPU marker and GPU idle. The local fill's RGB555 result is `001fH`, but observing those status bits is not a pixel readback. The sequence neither programs a display mode nor publishes a new scanout source. Display latching remains a PCRTC operation.

## 7.5 Register reference

Use whole-word accesses at the addresses below. Each reset word is zero. READ_ADDR, WRITE_ADDR, TRANSFER_COUNT and CONTROL are ordinary read/write words. STATUS is read-only. TRIGGER accepts command writes and reads zero. Reading these words does not acknowledge an IRQ or clear DONE.

| Register | Channel 0 address | Channel 1 address | Access | Reset |
|---|---|---|---|---|
| READ_ADDR | `0800000cH` | `080103e4H` | R/W | `00000000H` |
| WRITE_ADDR | `08000010H` | `080103e8H` | R/W | `00000000H` |
| TRANSFER_COUNT | `08000014H` | `080103ecH` | R/W | `00000000H` |
| CONTROL | `08000018H` | `080103f0H` | R/W | `00000000H` |

| Register | Channel 0 address | Channel 1 address | Access | Reset |
|---|---|---|---|---|
| STATUS | 0800001cH | 080103f4H | R | 00000000H |
| TRIGGER | 08000020H | 080103f8H | Command / R=0 | 00000000H |

Do not derive channel 1 by adding one channel-0 bank length. READ_ADDR and WRITE_ADDR retain complete byte-address words. TRANSFER_COUNT retains a complete unsigned word. Neither the address nor count writes clear STATUS.

### 7.5.1 CONTROL fields

| Bits | Field | Meaning | Reset |
|---|---|---|---|
| 31:14 | Unassigned retained bits | Stored and read back; not decoded as transfer controls | 0 |
| 13:10 | BLOCK_WORDS | Programmed block length minus one | 0: one word |
| 9:6 | WRITE_REQUEST | Destination-side admission request selector | 0: FORCE |
| 5:2 | READ_REQUEST | Source-side admission request selector | 0: FORCE |
| 1 | WRITE_INCREMENT | 1 adds four bytes per word; 0 repeats the write address | 0 |
| 0 | READ_INCREMENT | 1 adds four bytes per word; 0 repeats the read address | 0 |

The complete CONTROL word is retained, including bits 31:14. Those bits are not cleared by a write or admission. No transfer-control meaning is assigned to them here.

### 7.5.2 Request-selector encodings

Both selector fields use the same encodings. The fields gate admission independently; their meanings are not reversed when used on the other side.

| Encoding | Name |
|---|---|
| 0 | FORCE |
| 1 | GX_WRITE |
| 2 | GX_READ |
| 3 | APU_WRITE |
| 4 | APU_READ |
| 5 | IMGDEC_WRITE |
| 6 | IMGDEC_READ |
| 7 | CARTRIDGE_SLOT0_WRITE |
| 8 | CARTRIDGE_SLOT0_READ |
| 9 | CARTRIDGE_SLOT1_WRITE |
| 10 | CARTRIDGE_SLOT1_READ |
| 11–14 | Unnamed; test their corresponding request-line bits |
| 15 | DISABLED |

FORCE is always asserted and DISABLED is never asserted. Do not equate the unnamed encodings with either one. For cartridge socket selection and card protocols, see Book III, “Cartridge sockets and mapped card devices”, under “DMA between sockets”.

### 7.5.3 STATUS and TRIGGER

| Word | Bits | Name | Meaning |
|---|---|---|---|
| STATUS | 0 | BUSY | Accepted transfer is in progress, including a wait for requests |
| STATUS | 1 | DONE | Transfer count reached completion |
| STATUS | 31:2 | Unassigned | Reset, accepted START and completion write these bits zero |
| TRIGGER write | 0 | START | Start only if the channel is idle |
| TRIGGER write | 31:1 | No start action | Do not start a channel without bit 0 |
| TRIGGER read | 31:0 | Zero | Written command bits are not retained for readback |

Accepted START sets STATUS to exactly `00000001H`; finish sets it to exactly `00000002H`. DONE is not write-one-to-clear. Writing STATUS faults instead of clearing it. A later accepted START replaces DONE, and reset clears it.

Address/count/CONTROL writes take effect in the programmed words on their accepted writes. This is distinct from the admission edge and the service edge described next. A register read is not a peek into the current word slot of an admitted block.

## 7.6 Programmed state and an admitted block

### 7.6.1 Configuration is captured; source data is not

A transfer proceeds one block at a time. At admission the channel captures READ_ADDR, WRITE_ADDR, the full remaining TRANSFER_COUNT and CONTROL, and sets the service edge. The actual block contains the smaller of the remaining count and the programmed 1–16 words.

These captured values remain in force through service, including when software rewrites the programmed registers. Payload is read from the captured source address at service. This is the old/new-array hazard described under Source reuse and result use: admission captures the addresses, not the bytes stored at them.

### 7.6.2 What happens at service

The complete admitted block is serviced at its service edge; there are no separate per-word service edges. Within that service, each word has this order:

1. Read the mapped source word.
2. Write that word to the mapped destination.
3. Update the read address.
4. Update the write address.
5. Decrement the remaining count.

CONTROL bit 0 enables a four-byte source step; bit 1 enables a four-byte destination step. A disabled step leaves that address unchanged. Address addition and count subtraction wrap as unsigned 32-bit arithmetic. For example, a four-byte step from `0xFFFFFFFC` produces `0x00000000`.

Only after the entire block are the resulting read address, write address and remaining count written back to the channel registers. Register values are therefore not a per-word progress display. Destination writes occur during service, not at admission. When the transfer ends, the final address/count words are published before DONE, and DONE is published before the completion IRQ.

Keep source contents unchanged until completion if the transfer must reproduce their original values. Admission alone neither releases the source nor makes the destination final. Because each write precedes the next read, overlapping source and destination ranges have the additional effect shown next.

### 7.6.3 Worked example: three-word forward overlap

Let A be four-byte aligned, with A through A + 15 in one ordinary writable RAM span, outside I/O. All addresses below are relative to A. Initially:

| Address | Word |
|---|---|
| A | 0x11111111 |
| A + 4 | 0x22222222 |
| A + 8 | 0x33333333 |

Admit one three-word block with read address A, write address A + 4, count 3 and both increments enabled.

- Word 1 reads 0x11111111 at A, then writes it to A + 4.
- Word 2 reads the **new** 0x11111111 at A + 4, then writes it to A + 8.
- Word 3 reads the **new** 0x11111111 at A + 8, then writes it to A + 12.

The three destination words all become 0x11111111. The channel finishes with read address A + 12, write address A + 16 and count zero. Forward overlap does not preserve the original source sequence: each write precedes the next read.

### 7.6.4 Programming while BUSY

BUSY does not reject address, count or CONTROL programming. It also does not turn those writes into edits of an admitted block.

**Schematic active-block example.** Suppose admission captured read address A, write address B, count 5, three-word blocks and both increments enabled. Before service, software programs read address X, write address Y, count 9, and changes CONTROL to two-word blocks with a fixed source and incrementing destination. Assume both request gates remain asserted.

The admitted block still transfers three words using the old configuration. Its writeback replaces X, Y and 9 with A + 12, B + 12 and 2. The changed CONTROL word remains. The next admission for this channel can therefore capture those resulting addresses and count together with the new CONTROL: two words, fixed source, incrementing destination.

When a BUSY channel has no admitted block, newly programmed words instead supply subsequent admission; programming count zero can complete the channel even with closed request gates. START while BUSY is ignored and does not restart, recapture or cancel a block. Reasserting START therefore cannot rescue address/count words that the admitted block will overwrite.

## 7.7 Requests and device-port ownership

A device transfer has four distinct conditions: both request gates permit admission; DMA owns the applicable port against conflicting CPU use; each port accepts its mapped access; and the block completes. Request selectors provide the first condition. The device chapter supplies its port and payload protocol, including any work continuing after DMA completion.

### 7.7.1 Request gates

CONTROL selects a read request and a write request independently. Both must be asserted before a nonzero block is admitted. The selectors are not inferred from READ_ADDR or WRITE_ADDR. FORCE (0) is always asserted; DISABLED (15) is always deasserted. Unnamed selectors 11–14 test their respective request bits; they are not aliases for DISABLED and have no assigned device meaning here.

| Selector | Request | Asserted condition |
|---|---|---|
| 1 | GX_WRITE | GPU direction is FIFO or CPU_TO_GP0, and the corresponding write request is asserted as described below. |

| Selector | Request | Asserted condition |
|---|---|---|
| 2 | GX_READ | DMA read is enabled by GPUREAD_TO_CPU direction, and readback is READY. Consuming the last packed readback word returns readback to IDLE. |
| 3 | APU_WRITE | APU transfer mode is DMA_WRITE and the transfer FIFO is empty. |
| 4 | APU_READ | APU transfer mode is DMA_READ and the 16-word transfer FIFO is full. |
| 5 | IMGDEC_WRITE | IMGDEC is active, not quiescing, has nonzero input words remaining, and has at least `min(input words remaining, 16)` free input-FIFO words. |
| 6 | IMGDEC_READ | IMGDEC is active, not quiescing, has nonzero output words remaining, and holds at least `min(output words remaining, 16)` output-FIFO words. |

In GPU FIFO direction, GX_WRITE requires ingress not stopped, empty DMA ingress, and room in the command FIFO. In CPU_TO_GP0 direction, packet acceptance requires readback IDLE, ingress not stopped, and empty DMA ingress. With image-load words outstanding, it tests command-FIFO space. During polyline payload it is false. Otherwise, no partial fixed packet and an empty FIFO permit acceptance; the remaining case requires an incomplete packet that is neither a polygon nor a line. This packet request is not the CPU's one-word write-ready condition.

IMGDEC's remaining quantities belong to its stream, not to the DMA channel. APU requests describe transfer-FIFO batches, not completion of sample-memory filling or draining. Selectors 7–10 are cartridge-slot 0 write/read and slot 1 write/read respectively; their device protocols belong to Book III.

### 7.7.2 Admission is a block boundary

Once admitted, a block retains its captured endpoints, count and CONTROL. A request dropping during a port access does not stop the remaining words of that block. Requests are tested again for a later admission, not between the words of an admitted block. Choose the block length to suit the destination's transfer contract; reducing the count or selecting FORCE does not make an otherwise invalid payload valid.

The last read and last write of each admitted block carry BLOCK_END. GPU DMA ingress processes its staged words at this boundary; APU batch scheduling and IMGDEC decode/request updates also use their applicable end-of-block paths. The boundary is not necessarily channel completion.

Source contents and overlapping ranges obey the ordered read/write behavior under Programmed state and an admitted block.

### 7.7.3 CPU access while BUSY

DMA ownership compares an exact port address. An admitted channel owns its captured starting endpoint; another BUSY channel uses its programmed endpoint. This is not a range lock or a reservation of every register in the device. A channel waiting for a request can still own its configured port. Rewriting an admitted channel's address does not move that block's ownership to the new address.

CPU writes to GP0 require no DMA write owner, empty DMA ingress, command-FIFO room, and ingress not stopped. APU transfer-data CPU writes require neither read nor write DMA ownership of that port. IMGDEC CPU writes require no DMA write owner, an active nonquiescing stream, input still expected, and FIFO room. Their DMA write-ready conditions differ: GP0 requires ingress not stopped; DMA write-ready is always asserted at the APU and IMGDEC data ports.

A refused CPU store waits before writing and retries the instruction after release. DMA ownership release requires the same blocked address and no remaining read or write owner there; release itself does not deposit a word. Reads are different: GP0 and IMGDEC CPU reads while DMA read-owned return their retained data latches rather than consuming DMA's data. APU CPU reads return its data latch. Do not use these reads as evidence that the DMA source FIFO is empty or that a transfer has stopped.

### 7.7.4 Supervisor boundaries

A supervisor request first closes new TRIGGER admission. Existing channels can continue until producer quiescence, after which a second gate prevents another block from being admitted. The already admitted block completes; a BUSY channel waiting indefinitely for a request does not prevent this boundary. The cartridge channel registers and arbitration turn are then retained while supervisor channels start clear. Leave restores them and resumes arbitration; it does not restore overwritten payload RAM. See SYSTEM, Device boundaries during supervisor service.

## 7.8 Faults and recovery

DMA bus faults and DMA completion are separate reports. An unmapped source read records UNMAPPED and supplies zero; an unmapped destination write records UNMAPPED without writing. An unaligned I/O read records UNALIGNED_IO and supplies zero; an unaligned I/O write records it without writing. A write to read-only STATUS records READ_ONLY without changing that register. Program DMA registers with whole-word writes.

The first unacknowledged fault retains the CODE, ADDR and ACCESS details; later faults leave that report in place. DMA has no corresponding error field or error-completion interrupt: it still attempts the destination write after a faulting source read, advances enabled addresses, decrements count, and continues the admitted block. Earlier read side effects and accepted writes are not undone.

A DMA destination can also refuse a write through its DMA write-ready condition. That refusal raises no bus fault by itself, and counting still proceeds. DONE therefore means that the channel's count reached zero, not that every write was accepted, that no bus fault occurred, or that GPU, APU or IMGDEC work has finished. START while BUSY is ignored; DISABLED prevents a later admission but does not cancel an admitted block.

DMA-unit reset cancels DMA service, discards the admitted block, clears all twelve mapped DMA words and request latches, and returns the next arbitration turn to channel 0. It preserves the installed timing configuration. These are reset effects, not a DMA CONTROL command or a device-port reset operation.

## 7.9 Completion and interrupts

At ordinary completion, the last block first publishes its final read address, write address and zero remaining count. Finish then replaces STATUS with DONE, and only then raises the channel's interrupt request. There is no DMA-local interrupt-enable bit in CONTROL.

| Channel | Controller source | Pending/mask bit | ACK word |
|---|---|---|---|
| 0 | DMA0_DONE | 0 | `00000001H` |
| 1 | DMA1_DONE | 8 | `00000100H` |

The interrupt-controller chapter specifies FLAGS, MASK, ACK and CPU acceptance. Masking completion IRQs neither prevents DMA completion nor clears DONE. STATUS reads do not clear pending; controller ACK does not clear STATUS. Repeated unacknowledged completions coalesce in one pending bit, not a transfer counter.

For interrupt-driven service, reserve one software sequence per channel. The handler acknowledges the selected controller source and advances that channel's sequence. Snapshot the sequence before issuing START, then wait for it to change in a predicate loop containing `halt_until_irq`. Other accepted IRQs may wake the CPU without completing that channel. Completion can also occur before the caller first executes HALT. Do not treat one wake-up as a completion test.

DMA DONE means that the programmed word slots have been processed. It does not certify that every bus access succeeded, nor that downstream work has finished. For a GP0 command stream, transfer completion, execution of an in-order GPU marker, GPU idle and publication of a displayed picture are different conditions. The programming example waits for DMA and GPU separately; the PCRTC chapter covers display latching.

## 7.10 Transfer timing and sharing the bus

DMA transfer times use the **33,868,800 Hz system timebase**. A block charged C cycles has an admitted interval of C / 33,868,800 seconds. To calculate elapsed time, add request waits and competing-channel service. CPU programming and IRQ service consume additional instructions. Device processing after word delivery is timed by the receiving unit.

### 7.10.1 Blocks are the unit of admission

The two DMA channels share **one admitted block**. They do not have two independently progressing transfer buses. A channel programs a block size of 1–16 words using CONTROL bits 10–13, which contain the block size minus one. Each word is four bytes. The admitted length is

```
admitted_words = min(remaining_word_count, programmed_block_length)
```

A short final block therefore uses its actual length, not the programmed maximum. Source and destination increment are independently enabled; an enabled endpoint advances four bytes per word, and a disabled endpoint repeats its address. Address arithmetic wraps at 32 bits.

Admission captures the addresses, remaining count, complete control word, and actual block length, then establishes the block's completion deadline. An admitted block is nonpreemptible: another channel becoming ready, or a request dropping, does not interrupt it. Programming new values does not retime that block. At its deadline, the block performs its ordered word transfers and publishes the resulting addresses and remaining count. No intermediate per-word completion boundaries are provided.

### 7.10.2 Calculating a block's charge

Calculate source and destination costs over paired timing spans. A span ends at the first mapping boundary reached by either incrementing endpoint, or at the block end. A mapping crossing splits the timing calculation, **not** the admitted block.

The installed region costs for a span of n words are:

| Endpoint region | Word charge | Setup charge at that endpoint's region start |
|---|---|---|
| RAM | n | 1 |
| System ROM | n | 0 |
| Cartridge region | 8n | 4 |
| I/O | 0 | 0 |
| Other | 0 | 0 |

Both endpoints begin a new region span at every block admission. Thus each RAM or cartridge endpoint pays setup again in the next block. Within a block, splitting a span because the *opposite* endpoint crosses a boundary does not repeat setup for the continuing endpoint. An endpoint pays setup again when its own mapping span ends, including crossings between cartridge ROM, RAM, and MMIO subregions. A fixed endpoint has one region span for the entire block: repeated accesses to its address do not repeatedly incur setup.

For each paired span, let R and W include the applicable endpoint setup charges. Its contribution is

```
Span charge = R + W       if both endpoints are RAM or both are Cartridge
            = max(R,W)    otherwise
Block charge = max(1, sum of all span charges)
```

"Both" refers to region kind, not matching addresses or cartridge slots. System-ROM-to-system-ROM uses the maximum rule, not addition. The one-cycle minimum applies once to the **whole block**, not to each word or each span. Consequently an all-I/O block costs one cycle regardless of its admitted word count. The cartridge charge applies throughout the cartridge aperture, including cartridge RAM and MMIO, not only ROM storage. Both sockets share the external transfer path: cartridge-to-cartridge costs add even when different socket request selectors choose its source and destination. Separate chip selects do not provide a second transfer bus.

### 7.10.3 Single-channel examples

Assume uninterrupted request eligibility, no mapping crossings, and no programming changes during these transfers. All source addresses increment; destination addresses increment except for the fixed GP0 port.

| Transfer | Count | Programmed block | Actual blocks | Calculation | Total cycles |
|---|---|---|---|---|---|
| RAM → RAM | 20 | 16 | 16, 4 | (17 + 17) + (5 + 5) | **44** |
| RAM → RAM | 20 | 4 | 4, 4, 4, 4, 4 | 5 × (5 + 5) | **50** |
| Cartridge-region ROM → RAM | 20 | 16 | 16, 4 | max(132,17) + max(36,5) | **168** |
| Cartridge-region ROM → fixed GP0 I/O port | 5 | 16 | 5 | max(44,0) | **44** |

For RAM copies, a block of n words costs 2n + 2: the two endpoints share the RAM charge additively, and each has one setup charge. Smaller blocks increase repeated setup. In the cartridge examples the source cost dominates the maximum. **System ROM is different:** the corresponding 20-word system-ROM-to-RAM transfer with block size 16 costs 17 + 5 = 22 cycles, not 168.

### 7.10.4 Transfer rates by route

The following rates convert the admitted DMA charge into transferred words and payload bytes. They assume one eligible channel, complete 16-word blocks, continuously asserted requests, successful endpoint accesses, no mapping crossings and no register changes. CPU setup, request gaps and work performed by the destination are outside this calculation. A fixed I/O address identifies a port, not a directly addressed VRAM word.

For a block of 16 words charged C cycles:

```
Words per second = 16 × 33,868,800 / C
Bytes per second = 4 × words per second
```

Count each payload byte once, not once for its read and again for its write. The table uses decimal millions: Mword/s is 1,000,000 words/s and MB/s is 1,000,000 bytes/s. Rates are rounded to three decimal places.

| Source → destination | Cycles per 16 words | Mword/s | MB/s |
|---|---|---|---|
| RAM → RAM | 34 | 15.938 | 63.753 |
| System ROM → RAM | 17 | 31.877 | 127.506 |
| Cartridge ROM → RAM | 132 | 4.105 | 16.421 |
| Cartridge region → cartridge region | 264 | 2.053 | 8.211 |
| RAM → fixed I/O | 17 | 31.877 | 127.506 |
| System ROM → fixed I/O | 16 | 33.869 | 135.475 |
| Cartridge ROM → fixed I/O | 132 | 4.105 | 16.421 |
| Fixed I/O → RAM | 17 | 31.877 | 127.506 |
| Fixed I/O → fixed I/O | 1 | 541.901 | 2167.603 |

The table expresses the DMA controller's admitted transfer rates. The all-I/O row follows from the one-cycle block minimum. A usable device-to-device route also requires both ports to supply and accept its complete block. Use the assigned device requests and count only successful accesses as useful payload; FORCE by itself supplies no device readiness.

### 7.10.5 Block size, tails and waiting time

For RAM → RAM, repeated setup changes the rate even without contention:

| Words per full block | Cycles per block | Payload MB/s |
|---|---|---|
| 1 | 4 | 33.869 |
| 4 | 10 | 54.190 |
| 16 | 34 | 63.753 |

Use the actual final block when budgeting a finite transfer. The 20-word, 16-word-block RAM copy above costs 44 cycles, so its 80-byte payload averages **61.580 MB/s** over admitted transfer time, not the full-block 63.753 MB/s. A larger block reduces repeated setup, but keeps the shared DMA path until that block completes. It does not make both channels progress simultaneously.

For elapsed throughput, include intervals in which no block can be admitted. Suppose a 64-word RAM copy with 16-word blocks starts at cycle zero, but a selected request remains false until cycle 100. Its first block completes at 134. If that request drops at 133, the admitted block still completes, but another block cannot start. If the request returns at 200 and remains true, the remaining blocks complete at 234, 268 and 302. The transfer has 136 charged cycles and 166 waiting cycles: 256 bytes over 302 cycles, or **28.710 MB/s**. Those request edges are conditions of this example, not prescribed device response times.

When both channels are eligible, include the competing blocks in each channel's elapsed interval. The following round-robin trace shows why equal channel count does not imply an equal split of bandwidth: the channels can have different block lengths, endpoint costs and remaining counts.

### 7.10.6 Worked block-round-robin trace

A channel is eligible only when BUSY, its count is nonzero, and **both** its read and write requests are asserted. FORCE is always asserted. Selection scans from the retained next-channel turn, skipping ineligible channels. Admission gives the next turn to the other channel; skips and completions do not themselves advance it. Thus round-robin is at block granularity, not word interleaving or a promise to wait for an unready channel.

Take two incrementing RAM-to-RAM copies, each with count 20 and FORCE on both requests. Channel 0 programs block size 16; channel 1 programs block size 4. Assume no earlier bus work, continuously true requests, no mapping crossings, and no register rewrites. Let **A** be channel 0's first admission. Channel 1 starts after A but before channel 0's first deadline, so it cannot preempt that first block.

| Block | Channel | Words | Charge | Service edge | Remaining count |
|---|---|---|---|---|---|
| 1 | 0 | 16 | 34 | A + 34 | 4 |
| 2 | 1 | 4 | 10 | A + 44 | 16 |
| 3 | 0 | 4 | 10 | A + 54 | 0: complete |
| 4 | 1 | 4 | 10 | A + 64 | 12 |
| 5 | 1 | 4 | 10 | A + 74 | 8 |
| 6 | 1 | 4 | 10 | A + 84 | 4 |
| 7 | 1 | 4 | 10 | A + 94 | 0: complete |

Each continuously eligible successor is anchored to the preceding block's deadline. Channel 0 completes at **A + 54**; channel 1 completes at **A + 94**. The complete sequence costs 34 + 6 × 10 = 94 cycles. After channel 0 completes, it is skipped: channel 1 continues without waiting, while the retained turn remains channel 0. Final addresses and count are published before DONE and the channel's completion IRQ; these completion boundaries do not specify when the CPU handles that IRQ.

### 7.10.7 Budgeting transfers to and from VRAM

VRAM is GPU-local storage, not a separate mapped DMA endpoint. Uploads send a command and packed image words to **GP0 at 0801023cH**. Downloads read packed image words from **GPUREAD at that same address**. The DMA timing classification of that address is I/O. Select the proper GPU DMA direction and GX_WRITE or GX_READ request; the I/O row alone says nothing about the GPU's readiness.

For an image containing P pixels, an A0–BF upload sends three command words followed by P/2 payload words, rounded upward. Each payload word contains two sixteen-bit pixels; an odd final pixel leaves one unused halfword. When DMA sends both command and image, its count includes the three command words.

A C0–DF readback command also contains three words, but its returned data is a separate stream of P/2 words, rounded upward. Wait for **GPUSTAT bit 27, readback ready**, before consuming that stream. A word read before readiness returns retained GPUREAD data, not a newly transferred image word.

For example, an 8×8 image has 64 pixels and 32 packed payload words. Assume 16-word DMA blocks, no competing channel, no mapping crossing and uninterrupted request eligibility throughout each data transfer:

| Operation | DMA words | Actual blocks | DMA charge |
|---|---|---|---|
| Upload command + image, RAM → fixed GP0 | 35 | 16, 16, 3 | 17 + 17 + 4 = **38 cycles** |
| Same upload, cartridge ROM → fixed GP0 | 35 | 16, 16, 3 | 132 + 132 + 28 = **292 cycles** |
| Ready readback data, fixed GPUREAD → RAM | 32 | 16, 16 | 17 + 17 = **34 cycles** |

The readback row excludes submission of its three-word command and all waiting for readiness. The upload rows count command overhead, not just image bytes. All three rows describe DMA work, not a fixed interval to completed VRAM writes or a displayed result. Request gaps add elapsed time; an unready endpoint cannot be made ready by substituting FORCE.

The GPU has a separate sixteen-word DMA ingress and a sixteen-word GP0 FIFO; they are not thirty-two interchangeable CPU input slots. A block's final word releases its captured ingress block for processing. Upload assembly and ordered GPU work are separate from DMA completion. The upload/readback command's one-tick charge is not a pixel-transfer bandwidth specification. A local VRAM copy uses no DMA payload path at all, so its command charge must not be substituted for a RAM-to-VRAM bus rate.

For downloads, observe readback readiness and consume the full returned count. For other GPU work, use the command ordering and completion protocol in the GPU chapter. Neither DMA DONE, GPU idle nor a GPU IRQ marker certifies that new pixels have reached the display latch; display publication belongs to PCRTC.

# 8 Audio processing unit

The BMSX has sixteen independently programmed playback slots feeding one stereo mixer. Each slot holds one PCM voice, BADP compressed voice or square generator, with its own source, cursor, rate, gain, filter history and playback state. A stereo source still occupies one slot; PLAY on an occupied slot replaces its voice. All playing slots contribute at the common 44,100-frame-per-second output cadence, even when their source rates differ. The mixer sums their left and right contributions separately, then saturates each final output to signed sixteen-bit range.

Sampled voices read installed system ROM, cartridge ROM or the shared 512 KiB sample RAM. Several slots can read the same bytes with independent playback positions. Main RAM is not a direct playback source, and the square generator needs no sample payload.

Use the sample-transfer port to load private sample RAM. For a CPU upload, MANUAL_WRITE stores each word directly. For DMA, a sixteen-word FIFO buffers the transfer while the APU moves words to or from sample RAM. This path operates alongside playback.

To start a voice, write its source and playback parameters to the shared staging bank, then issue PLAY. The command FIFO holds that configuration until execution. Later gain, pause and stop commands use the same path; a selected-slot register window also permits live parameter changes. The programming sequence below separates sample loading, command execution and playback.

## 8.1 APU specifications

| Resource | Specification |
|---|---|
| Playback slots | 16, numbered 0–15 |
| Slot parameter banks | 21 retained 32-bit words per slot |
| Command staging bank | One separate set of 21 parameter words |
| Command FIFO | 16 commands, each retaining its command and 21 parameter words |
| Selected-slot window | 21 word ports addressing one of the sixteen slot banks |
| Private sample RAM | 512 KiB; sample-source addresses `40000000H`–`4007ffffH` |
| Sample-transfer FIFO | 16 words, 64 bytes |
| Output cadence | 44,100 stereo frames per second |
| Output components | Signed sixteen-bit left and right values |
| System-cycle timebase | 33,868,800 cycles per second; 768 cycles per output frame |
| Sample-RAM transfer budget | 1,058,400 words per second; 32 cycles per scheduled word |

The `40000000H` source range addresses private RAM for voice playback. CPU and DMA transfers use TRANSFER_DATA instead of accessing that range directly.

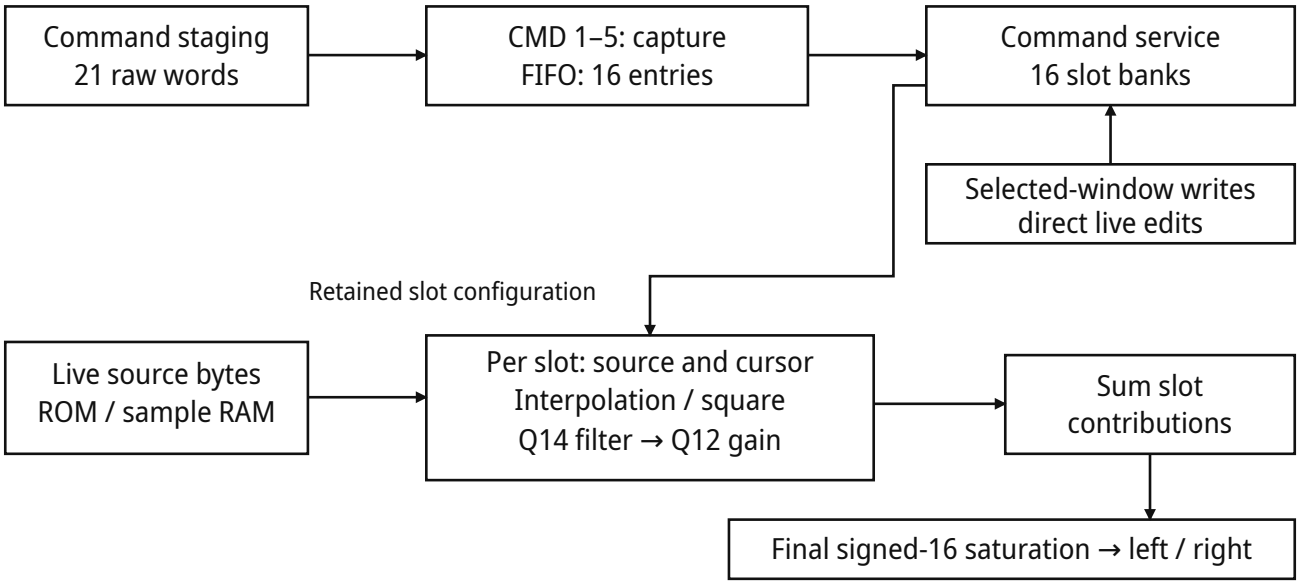



Figure 8.1: Command capture, live parameters and the per-slot sample path

The command path installs a voice's parameters. At each output frame, the voice path reads or generates samples, applies interpolation, filtering and gain, and contributes to the stereo mixer. The separate transfer path supplies sample RAM.

## 8.2 Programming sequence

First choose where the samples will reside and how they will be decoded. ROM samples can play in place. Samples produced or modified in RAM must be loaded into private sample RAM before playback.

| Starting material | Hardware route to playback |
|---|---|
| PCM or BADP in installed system ROM or the selected cartridge ROM | Bind the complete region with SOURCE_ADDR and SOURCE_BYTES, and program its source metadata. PCM metadata is not read from a container header. |
| PCM or BADP in private sample RAM | Bind the source at `40000000H + byte_offset`, after the sample bytes have committed to RAM. This range is not a CPU load/store window. |
| Samples in main RAM, cartridge RAM or cartridge mailbox space | Transfer the bytes into private sample RAM first; these are not direct playback sources. |
| Square generator | Program GENERATOR_KIND=1, rate, loop and duty. No sample upload or source-buffer binding is needed. |

For source decoding, SOURCE_BITS_PER_SAMPLE values 4 and 16 select BADP and PCM16 respectively; other values select PCM8. SOURCE_DATA_OFFSET counts bytes within the bound region, whereas START_SAMPLE and loop bounds count source frames.

1. **Own the required resources.** Reserve a playback slot and serialize users of the shared staging bank. Own the shared transfer ports for a sample-RAM transfer; for DMA, also reserve the chosen system-DMA channel. Either channel can supply or drain the APU FIFO with the same capability. Finish earlier owned transfers before reprogramming their resources.
2. **Prepare the sample storage.** TRANSFER_ADDRESS programs a moving RAM byte offset from bits 18:2, not a playback source address. Offset zero corresponds to source address `40000000H`; readback retains the programmed word rather than following transfer progress. MANUAL_WRITE CPU stores through TRANSFER_DATA write RAM directly. DMA_WRITE and DMA_READ instead use the sixteen-word sample-transfer FIFO, a fixed TRANSFER_DATA endpoint and the corresponding APU_WRITE or APU_READ block request.
3. **Separate DMA transport from RAM commit.** Keep an upload's DMA source unchanged until channel DONE, then separately wait for STATUS.TRANSFER_BUSY to clear before playback or a mode change. DONE reports the final system-DMA accesses, not the final sample-RAM commit. For download, channel DONE marks receipt of the requested

words; select transfer STOP afterward to discard unwanted prefetch. The transfer unit has no separate completion IRQ. See Sample RAM transfer.

4. **Capture coherent configuration.** Assemble the 21-word staging bank before writing CMD. A recognized nonzero command captures all 21 words, restores staging defaults and selects live bank zero. Check CMD_FREE before a burst that must retain every command: the sixteen-entry command FIFO replaces its oldest entry when full. It is separate from the sample-transfer FIFO. Source extents and offsets are bytes; start positions and loop bounds are source frames. Unity rate is `00010000H`, unity gain `00001000H`.
5. **Distinguish capture, execution and live access.** PLAY binds at execution, not capture; keep the intended cartridge socket selected until then. Reselect through staging SLOT after capture before using the selected-slot window. Its stores change that bank directly, while other staging stores only prepare a command. A live edit is not an ordered follow-up to a still-queued PLAY. Use PLAY for a coherent source replacement rather than consecutive live source-field stores.
6. **Retain playback storage for the voice's lifetime.** PLAY does not copy its sample payload, and pause retains the voice and its source requirement. Natural ends and completed nonzero fades publish SLOT_ENDED; an immediate zero-fade STOP_SLOT does not. The APU retains only the latest end event, not one FIFO entry per end. Its central interrupt request is distinct from DMA completion, and an interrupt wakeup alone does not identify which condition completed. See End events and interrupts.

The worked download shows the transfer-port sequence and the timing of DMA receipt and surplus prefetch.

### 8.2.1 Start a PCM loop or square voice

These two configurations show the complete staging bank. Start with the selected slot idle, no queued commands, no outstanding APU fault, and exclusive ownership of the staging bank and that slot. No DMA or transfer-port operation is needed: PCM is read from cartridge ROM, while the square generator has no sample buffer.

For the PCM case, place the following mono PCM16 samples in cartridge ROM. Let **A** be their mapped byte address; the declaration occupies 64 bytes. Keep the cartridge containing these bytes selected until PLAY executes.

```
rodata pcm: i16[32] = {
  0, 3135, 5793, 7568, 8192, 7568, 5793, 3135,
  0, -3135, -5793, -7568, -8192, -7568, -5793, -3135,
  0, 3135, 5793, 7568, 8192, 7568, 5793, 3135,
  0, -3135, -5793, -7568, -8192, -7568, -5793, -3135,
}
```

Choose one column and write **all 21 words** with aligned CPU stores. Do not interleave another producer's staging writes. Parameter order below follows increasing address; CMD is separate and must be written last.

| Staging address | Parameter | PCM loop | Square |
|---|---|---|---|
| `08000120H` | SOURCE_ADDR | A | `00000000H` |
| `08000124H` | SOURCE_BYTES | 64 | 0 |
| `08000128H` | SOURCE_SAMPLE_RATE_HZ | 7040 | 440 |
| `0800012CH` | SOURCE_CHANNELS | 1 | 1 |
| `08000130H` | SOURCE_BITS_PER_SAMPLE | 16 | 16 |
| `08000134H` | SOURCE_FRAME_COUNT | 32 | 1 |
| `08000138H` | SOURCE_DATA_OFFSET | 0 | 0 |
| `0800013CH` | SOURCE_DATA_BYTES | 64 | 0 |
| `08000140H` | SOURCE_LOOP_START_SAMPLE | 0 | 0 |
| `08000144H` | SOURCE_LOOP_END_SAMPLE | 16 | 1 |
| `08000148H` | SLOT | 2 | 4 |
| `0800014CH` | RATE_STEP_Q16 | `00010000H` | `00010000H` |
| `08000150H` | GAIN_Q12 | `00000800H` | `00000400H` |
| `08000154H` | START_SAMPLE | 0 | 0 |
| `08000158H` | FILTER_CONTROL | 0 | 0 |
| `0800015CH` | FILTER_B0_B1 | `00004000H` | `00004000H` |
| `08000160H` | FILTER_B2_A1 | 0 | 0 |
| `08000164H` | FILTER_A2 | 0 | 0 |
| `08000168H` | FADE_SAMPLES | 0 | 0 |
| `0800016CH` | GENERATOR_KIND | 0 | 1 |
| `08000170H` | GENERATOR_DUTY_Q12 | `00000800H` | `00000800H` |

Write 00000001H to CMD at 08000174H to submit PLAY. Capture restores staging defaults and the visible selector to zero; it does not mean that the voice has started. For these idle-slot starting conditions, ACTIVE_MASK at 08000190H gains bit 2 (00000004H) for PCM or bit 4 (00000010H) for the square when PLAY executes. Check the fault words as well as the active bit.

The PCM loop repeats sixteen source frames at 7040 frames/s, giving a 440 Hz waveform at one-half gain. The square uses 440 as its source rate, a one-frame loop and one-half duty; it is a continuous 440 Hz generator, not a sound lasting one output frame. Its quarter-gain contribution is mixed at Q12 precision; with no other contributing voices, the final channel levels are +8191 and −8192. Leave the PCM ROM bytes intact for the voice's complete lifetime, including pauses.

To stop either voice without a fade, write its slot number to 08000148H, zero to FADE_SAMPLES at 08000168H, then 00000002H to CMD. These are new staging writes, not edits to the selected live bank. The slot's active bit clears when STOP_SLOT executes; submitting it does not release the source immediately. This zero-fade stop does not publish SLOT_ENDED.

## 8.3 Commands and slot ownership

### 8.3.1 Stage, capture, execute

Write the parameters to the shared staging bank, then write CMD last. CMD decodes the entire word; the command table defines values zero through five. Serialize access to staging so another routine cannot alter a partly prepared command.

A valid nonzero command captures **all 21 staging words** into one FIFO entry. Subsequent staging writes cannot change that entry. Capture resets the visible staging words to their defaults, returns CMD readback to zero and selects slot 0 for the selected-slot window. The queued command still targets its captured slot; clearing the visible selector does not retarget it.

NONE captures nothing and leaves the staged parameters intact. An unrecognized word records BAD_CMD and clears staging to defaults. A latched APU fault does not itself block later valid commands.

At command execution, the APU first completes audio due through that cycle, then executes the queued commands in FIFO order. Consecutive commands can execute without an output frame between them. Use SAMPLE_SEQUENCE when an operation requires an elapsed number of output frames.

### 8.3.2 Preserve FIFO entries when every command matters

The FIFO retains sixteen entries. CMD_QUEUED reports its count, CMD_FREE reports sixteen minus that count, and CMD_CAPACITY reports sixteen. With a full FIFO, another accepted command replaces the oldest entry. It neither waits for free space nor reports a FIFO-full fault. Use the reported space before submitting a burst that must preserve every command.

Queue-counter reads return the current queue state. They leave commands queued; STATUS and selected-slot accesses synchronize audio through their access cycle as described below.

### 8.3.3 Command words

| CMD word | Name | Captured inputs and operation |
|---|---|---|
| 00000000H | NONE | No FIFO entry; staging parameters remain |
| 00000001H | PLAY | Bind captured source, replace the target voice and copy all parameters into its bank on successful binding |
| 00000002H | STOP_SLOT | Target SLOT and FADE_SAMPLES choose immediate stop or a sample-based fade |
| 00000003H | SET_SLOT_GAIN | Target SLOT and GAIN_Q12 update the raw gain through the live-parameter path |
| 00000004H | PAUSE_SLOT | Pause a non-idle slot without clearing its parameter bank |

| | | |
|---|---|---|
| 00000005H | RESUME_SLOT | Resume a paused slot without reloading its source |

PLAY and the other slot commands use bits 3:0 of captured SLOT. The complete word remains retained data. PLAY binds at execution, not capture. A failed source bind does not replace the old slot's parameters, phase or output; it records SOURCE_RANGE with the source address as detail.

STOP_SLOT with zero fade, or with an inactive target, clears the target bank and stops it immediately. This immediate explicit stop does **not** produce a slot-ended event. A nonzero fade of an active slot instead retains its bank while fading. Pause and resume do not clear it. SET_SLOT_GAIN can also retain a gain word in an inactive bank; that write does not start playback.

### 8.3.4 Live slot parameters

Write staging SLOT to select one bank through its low four bits. Only that staging write immediately changes selection; the other staging parameter writes merely prepare a later command. The selected window reads and writes the bank itself, after synchronizing preceding output to the access cycle. A live write does not enqueue CMD, reset staging or activate an idle slot.

Select a bank through staging SLOT at 08000148H. The selected-window SLOT word at 080001bcH is ordinary bank data and leaves the selection unchanged.

ACTIVE_MASK bit n means slot n is non-idle. Paused and fading slots still have active bits. STATUS.BUSY is any active slot or queued command, so it can remain set while paused sound produces no samples. It is not TRANSFER_BUSY. SELECTED_SOURCE_ADDR is the selected bank's original source address while that slot is active, otherwise zero. It is not the moving playback cursor.

Every active source-field write can rebind that voice independently, using the other words currently in its bank. Several source writes are not an atomic source replacement. The source/playback sections specify the resulting cursor, filter and binding effects. Use captured PLAY when a new source description must be supplied together rather than assembled through intermediate live states.

## 8.4 Register reference

The APU exposes sixty distinct word ports: 21 staging words, 21 selected-window words and eighteen command/status/event/transfer/fault ports. Fault ports are not adjacent to the main bank. All addresses below are byte addresses.

Use word accesses. STATUS, fault code/detail, event words, selected-source and ACTIVE_MASK writes raise system READ_ONLY without mutation. Queue-counter and SAMPLE_SEQUENCE writes are accepted but do not change their device-produced values.

### 8.4.1 Staging and selected-window parameters

All rows below have read/write word access. The two reset columns intentionally differ. Staging defaults are also restored after each recognized nonzero command; reset slot banks contain zero, not a ready-to-play copy of those defaults. The selected addresses alias the bank chosen by staging SLOT bits 3:0.

#### 8.4.1.1 Source-description words

| Index | Parameter | Staging address | Selected address | Staging reset | Selected reset |
|---|---|---|---|---|---|
| 0 | SOURCE_ADDR | 08000120H | 08000194H | 00000000H | 00000000H |
| 1 | SOURCE_BYTES | 08000124H | 08000198H | 00000000H | 00000000H |
| 2 | SOURCE_SAMPLE_RATE_HZ | 08000128H | 0800019cH | 00000000H | 00000000H |
| 3 | SOURCE_CHANNELS | 0800012cH | 080001a0H | 00000000H | 00000000H |
| 4 | SOURCE_BITS_PER_SAMPLE | 08000130H | 080001a4H | 00000000H | 00000000H |
| 5 | SOURCE_FRAME_COUNT | 08000134H | 080001a8H | 00000000H | 00000000H |
| 6 | SOURCE_DATA_OFFSET | 08000138H | 080001acH | 00000000H | 00000000H |
| 7 | SOURCE_DATA_BYTES | 0800013cH | 080001b0H | 00000000H | 00000000H |

| Index | Parameter | Staging address | Selected address | Staging reset | Selected reset |
|---|---|---|---|---|---|
| 8 | SOURCE_LOOP_START_SAMPLE | 08000140H | 080001b4H | 00000000H | 00000000H |
| 9 | SOURCE_LOOP_END_SAMPLE | 08000144H | 080001b8H | 00000000H | 00000000H |

#### 8.4.1.2 Slot and playback words

| Index | Parameter | Staging address | Selected address | Staging reset | Selected reset |
|---|---|---|---|---|---|
| 10 | SLOT | 08000148H | 080001bcH | 00000000H | 00000000H |
| 11 | RATE_STEP_Q16 | 0800014cH | 080001c0H | 00010000H | 00000000H |
| 12 | GAIN_Q12 | 08000150H | 080001c4H | 00001000H | 00000000H |
| 13 | START_SAMPLE | 08000154H | 080001c8H | 00000000H | 00000000H |
| 14 | FILTER_CONTROL | 08000158H | 080001ccH | 00000000H | 00000000H |
| 15 | FILTER_B0_B1 | 0800015cH | 080001d0H | 00004000H | 00000000H |
| 16 | FILTER_B2_A1 | 08000160H | 080001d4H | 00000000H | 00000000H |
| 17 | FILTER_A2 | 08000164H | 080001d8H | 00000000H | 00000000H |
| 18 | FADE_SAMPLES | 08000168H | 080001dcH | 00000000H | 00000000H |
| 19 | GENERATOR_KIND | 0800016cH | 080001e0H | 00000000H | 00000000H |
| 20 | GENERATOR_DUTY_Q12 | 08000170H | 080001e4H | 00000800H | 00000000H |

Each port retains the complete parameter word. The following sections give the source units, signed rate and gain formats, filter fields and live-write effects.

### 8.4.2 Command, status, event and queue ports

| Address | Name | Access | Reset read |
|---|---|---|---|
| 08000174H | CMD | Command write / read zero | 00000000H |
| 08000178H | STATUS | Read; writes fault | 00000020H |
| 0800017cH | EVENT_KIND | Read; writes fault | 00000000H |
| 08000180H | EVENT_SLOT | Read; writes fault | 00000000H |
| 08000184H | EVENT_SOURCE_ADDR | Read; writes fault | 00000000H |
| 08000188H | EVENT_SEQ | Read; writes fault | 00000000H |
| 0800018cH | SE-LECTED_SOURCE_ADDR | Read; writes fault | 00000000H |
| 08000190H | ACTIVE_MASK | Read; writes fault | 00000000H |
| 080001f4H | CMD_QUEUED | Read; writes have no queue effect | 00000000H |
| 080001f8H | CMD_FREE | Read; writes have no queue effect | 00000010H |
| 080001fcH | CMD_CAPACITY | Read; writes have no queue effect | 00000010H |
| 08000200H | SAMPLE_SEQUENCE | Read; writes have no clock effect | 00000000H |

STATUS reads synchronize preceding audio to their access cycle. SAMPLE_SEQUENCE also synchronizes and returns the low 32 bits of elapsed output frames, even with no active slots. Event words, ACTIVE_MASK and SELECTED_SOURCE_ADDR are retained mirrors, not read-to-clear operations.

### 8.4.3 Transfer and fault ports

| Address | Name | Access | Reset read |
|---|---|---|---|
| 080001e8H | TRANSFER_ADDRESS | R/W full programmed word | 00000000H |
| 080001ecH | TRANSFER_DATA | R/W data port | 00000000H |

| Address | Name | Access | Reset read |
|---|---|---|---|
| 080001f0H | TRANSFER_CONTROL | R/W full programmed word | 00000000H |
| 08010218H | FAULT_CODE | Read; writes fault | 00000000H |
| 0801021cH | FAULT_DETAIL | Read; writes fault | 00000000H |
| 08010220H | FAULT_ACK | Nonzero write acknowledges / read zero | 00000000H |

### 8.4.4 STATUS fields

| Bit | Name | Condition |
|---|---|---|
| 0 | FAULT | APU first-fault latch is set |
| 1 | SELECTED_SLOT_ACTIVE | Selected slot is active |
| 2 | BUSY | Any slot active or command FIFO nonempty |
| 3–4 | Unassigned | Read zero |
| 5 | CMD_FIFO_EMPTY | No queued commands |
| 6 | CMD_FIFO_FULL | Sixteen queued commands |
| 7 | DMA_REQUEST | DMA read request OR DMA write request |
| 8 | DMA_READ_REQUEST | DMA_READ mode and full transfer FIFO |
| 9 | DMA_WRITE_REQUEST | DMA_WRITE mode and empty transfer FIFO |
| 10 | TRANSFER_BUSY | A sample-RAM transfer batch is scheduled |
| 11–31 | Unassigned | Read zero |

TRANSFER_CONTROL bits 1:0 select STOP=0, MANUAL_WRITE=1, DMA_WRITE=2 or DMA_READ=3. Upper bits remain retained/readable but are not transfer controls. TRANSFER_ADDRESS retains its full programmed word, while the moving internal byte offset starts from its bits 18:2 with bits 1:0 zero. Its readback is not a progress counter. TRANSFER_DATA reads return the existing data latch to the CPU; DMA accesses have the distinct FIFO effects described in sample RAM transfer.

## 8.5 Voice sources and their lifetime

Program a source description for each sampled voice. The description identifies the byte region, decoder, channel layout, frame count and loop. PCM uses these register values directly, so a PCM source needs only sample bytes rather than a file header.

| Source field | Meaning |
|---|---|
| SOURCE_ADDR | Base address of the complete bound source region |
| SOURCE_BYTES | Byte extent of that region |
| SOURCE_SAMPLE_RATE_HZ | Source rate in Hz, used with RATE_STEP_Q16 |
| SOURCE_CHANNELS | Channel count used by the decoder |
| SOURCE_BITS_PER_SAMPLE | Decoder selection: 4 for BADP, 16 for PCM16, otherwise the PCM8 branch |
| SOURCE_FRAME_COUNT | Number of source frames used for playback duration |
| SOURCE_DATA_OFFSET | Byte displacement from SOURCE_ADDR to sample or compressed-block data |
| SOURCE_DATA_BYTES | Retained data-byte metadata; not a decoder read bound |
| SOURCE_LOOP_START_SAMPLE / SOURCE_LOOP_END_SAMPLE | Source-frame loop bounds |
| GENERATOR_KIND | Zero for source-buffer playback; one for the square generator |
| GENERATOR_DUTY_Q12 | Square-generator duty threshold |

A frame contains the channel samples at one source-time position. Thus a stereo frame is two samples, not two frames. START_SAMPLE and loop bounds count frames; byte offsets count bytes.

For source-buffer playback, PLAY binds the entire region identified by SOURCE_ADDR and SOURCE_BYTES when the command executes. Every accepted binding requires a nonzero byte count and the complete requested extent within the applicable storage.

| Source region | Admitted storage |
|---|---|
| System ROM | Actual installed firmware-image bytes, starting at 00000000H |
| Selected cartridge ROM | Entire extent within 10000000H–2FFFFFFFH, with a ROM device present and the extent within that card's actual ROM bytes |
| Private sample RAM | 512 KiB at 40000000H–4007FFFFH |

For samples held in main RAM, card RAM or other storage, transfer the bytes into private sample RAM first. A bound source must fit within actual installed storage, including the end of private RAM or a ROM. The zero-filled tail of a CPU ROM read is outside the bindable source extent.

PLAY binds the cartridge ROM selected at execution. Keep that socket selected until the command executes; later SELECT writes leave the voice on its bound ROM. Live source-field writes bind again using the selection at that access. See Source fields are individually live and Book III, "Cartridge sockets and mapped card devices", under "Cartridge controller".

The voice reads the bound storage while playing; PLAY does not copy the sample payload. Keep that storage available for the voice's lifetime. PCM reads use the current bytes. BADP caches its current and previous decoded frames; edits to compressed bytes take effect when those bytes are decoded again.

A failed binding raises SOURCE_RANGE (`00000102H`) with SOURCE_ADDR as detail; a failed PLAY leaves the old voice and slot bank intact. Binding checks the source region. The decoder then uses the programmed rate, channels, frame count, data offset and bit depth directly.

**SOURCE_DATA_BYTES does not limit decoding.** SOURCE_BYTES sets the bound region, while decode addresses follow the format and frame metadata. Keep the header, seek table and sample payload consistent with those values.

GENERATOR_KIND=1 selects the square generator and bypasses source binding. Other nonzero values also bypass binding but have no defined waveform; use one for square output and zero for sampled output.

## 8.6 PCM byte layouts and interpolation

PCM data starts at SOURCE_ADDR + SOURCE_DATA_OFFSET. For frame n, the first sample index is n multiplied by SOURCE_CHANNELS. Channel zero supplies left; channel one supplies right. Mono repeats its one decoded sample on both output channels.

| Format | Bytes in frame n, relative to the data start |
|---|---|
| PCM8 mono | n: mono byte |
| PCM8 stereo | 2n: left byte; 2n+1: right byte |
| PCM16 mono | 2n: low byte; 2n+1: high byte |
| PCM16 stereo | 4n, 4n+1: left low/high; 4n+2, 4n+3: right low/high |

PCM8 is unsigned offset-binary. Its decoded value is (byte minus 128) multiplied by 256. Bytes 00H, 80H, FFH and 40H therefore decode to −32768, 0, 32512 and −16384. In particular, FFH is not +32767 and 00H is not silence.

PCM16 is signed little-endian. Bytes 00H 80H mean −32768; FFH 7FH mean +32767; 00H C0H mean −16384. A stereo data sequence `00 80 FF 7F 00 00 00 C0` contains two frames: (−32768, +32767), then (0, −16384).

Playback uses linear interpolation for PCM and for decoded BADP samples. Let n be the current integer source-frame position and f its 16-bit fractional part. For either channel:

```
sample = first + ASR16((second − first) × f)
```

Here first is frame n, second is its interpolation neighbor, and ASR16 divides by 65536 with rounding toward negative infinity. Interpolate left and right independently. With first=−10000, second=10000 and f=16384, the result is −5000. With first=1000, second=−1001 and f=32768, the difference contributes −1001, giving −1; truncation toward zero would give the wrong result.

Without a loop, the last source frame is also its own interpolation neighbor. Interpolation does not read frame SOURCE_FRAME_COUNT beyond that last frame. With a loop, a neighbor reaching or passing the loop end is replaced by the loop-start frame. For loop [4,8), position 7.5 interpolates frame 7 with frame 4, not frame 8.

For example, mono PCM16 frames [0,10000,−10000,0], a source rate of 22050 Hz and unity RATE_STEP_Q16 produce eight output samples:

```
0, 5000, 10000, 0, −10000, −5000, 0, 0
```

The final pair is zero because the last source frame is held for interpolation until the cursor reaches the nonloop end.

### 8.6.1 Size a PCM source

For mono or stereo PCM, count storage from source frames, not output frames:

```
bytes per source frame = channels × bytes per channel sample
PCM payload bytes      = SOURCE_FRAME_COUNT × bytes per source frame
```

SOURCE_DATA_OFFSET locates that payload within the bound SOURCE_BYTES region. Keep the complete payload inside the region. The descriptor words are programmed separately; no PCM file header is required in the sample bytes.

At a source rate of 44,100 Hz, the following sizes hold one second of source material. With unity RATE_STEP_Q16, no loop and START_SAMPLE=0, each produces 44,100 output frames:

| PCM format | Bytes per source frame | One second of payload | Frames in all 512 KiB of sample RAM |
|---|---:|---:|---:|
| 8-bit mono | 1 | 44,100 | 524,288 |
| 8-bit stereo | 2 | 88,200 | 262,144 |
| 16-bit mono | 2 | 88,200 | 262,144 |
| 16-bit stereo | 4 | 176,400 | 131,072 |

The last column assumes one payload starting at RAM offset zero, with no other sample storage allocated. For example, 131,072 stereo PCM16 frames last about 2.972 seconds at 44,100 Hz and unity rate. Slots share this RAM: sixteen slots do not provide sixteen private 512-KiB banks. Several slots can instead bind the same stored sample and retain independent cursors and playback settings.

## 8.7 BADP compressed source format

BADP is selected by SOURCE_BITS_PER_SAMPLE=4. Its source metadata still comes from the slot fields. The decoder consumes the following header words relative to SOURCE_ADDR, not relative to SOURCE_DATA_OFFSET:

| Source byte offset | Consumed field |
|---:|---|
| 28 | Little-endian 32-bit seek-entry count |
| 32 | Little-endian 32-bit seek-table byte offset, relative to SOURCE_ADDR |

Each seek entry occupies eight bytes.

| Entry byte offset | Field |
|---:|---|
| +0 | Little-endian 32-bit source-frame number |
| +4 | Little-endian 32-bit compressed-block byte offset, relative to the data start |

The data start is SOURCE_ADDR + SOURCE_DATA_OFFSET. Header and seek-table addresses use the source base; compressed-block offsets use the data base. Only the header fields listed above are consumed.

Seeking chooses the last entry whose frame number is no greater than the requested frame. An empty table starts at frame zero and compressed offset zero. If every entry frame exceeds the request, entry zero is selected.

### 8.7.1 Blocks and channel state

At a block offset relative to the data start, the block has this layout:

| Block byte offset | Field |
|---:|---|
| +0 | Little-endian 16-bit BLOCK_FRAMES |
| +2 | Little-endian 16-bit total BLOCK_BYTES |
| +4 + 4c | Channel c signed little-endian 16-bit predictor |
| +6 + 4c | Channel c unsigned byte step-table index |
| +7 + 4c | One byte not read by the decoder |
| +4 + 4×SOURCE_CHANNELS | Nibble payload begins |

Channel records are stored in increasing channel order. BLOCK_BYTES includes the block header and payload and locates the next block: next block offset equals current block offset plus BLOCK_BYTES. Loading a block replaces each channel's predictor and index and resets the block frame index and nibble cursor to zero.

Payload bytes are consumed **high nibble first, then low nibble**. Within a frame, channel zero is decoded before the next channel. The nibble cursor continues across frames.

| Layout | Payload byte consumption |
|---|---|
| Mono | Byte 0 high: frame 0; byte 0 low: frame 1; byte 1 high: frame 2; and so on |
| Stereo | Byte 0 high: frame 0 left; byte 0 low: frame 0 right; byte 1 high/low: frame 1 left/right |

The header predictor is a starting accumulator, **not an extra first output frame**. Every emitted frame first applies a nibble to its channel predictor. Mono duplicates the decoded result to right. After BLOCK_FRAMES decoded frames, the next frame loads the next block using BLOCK_BYTES.

### 8.7.2 Exact nibble recurrence

The step table has 89 entries, indexed 0 through 88.

| Index | Step | Index | Step | Index | Step | Index | Step |
|---:|---:|---:|---:|---:|---:|---:|---:|
| 0 | 7 | 23 | 66 | 46 | 598 | 69 | 5358 |
| 1 | 8 | 24 | 73 | 47 | 658 | 70 | 5894 |
| 2 | 9 | 25 | 80 | 48 | 724 | 71 | 6484 |
| 3 | 10 | 26 | 88 | 49 | 796 | 72 | 7132 |
| 4 | 11 | 27 | 97 | 50 | 876 | 73 | 7845 |
| 5 | 12 | 28 | 107 | 51 | 963 | 74 | 8630 |
| 6 | 13 | 29 | 118 | 52 | 1060 | 75 | 9493 |
| 7 | 14 | 30 | 130 | 53 | 1166 | 76 | 10442 |
| 8 | 16 | 31 | 143 | 54 | 1282 | 77 | 11487 |
| 9 | 17 | 32 | 157 | 55 | 1411 | 78 | 12635 |

| Index | Step | Index | Step | Index | Step | Index | Step |
|---|---|---|---|---|---|---|---|
| 10 | 19 | 33 | 173 | 56 | 1552 | 79 | 13899 |
| 11 | 21 | 34 | 190 | 57 | 1707 | 80 | 15289 |
| 12 | 23 | 35 | 209 | 58 | 1878 | 81 | 16818 |
| 13 | 25 | 36 | 230 | 59 | 2066 | 82 | 18500 |
| 14 | 28 | 37 | 253 | 60 | 2272 | 83 | 20350 |
| 15 | 31 | 38 | 279 | 61 | 2499 | 84 | 22385 |
| 16 | 34 | 39 | 307 | 62 | 2749 | 85 | 24623 |
| 17 | 37 | 40 | 337 | 63 | 3024 | 86 | 27086 |
| 18 | 41 | 41 | 371 | 64 | 3327 | 87 | 29794 |
| 19 | 45 | 42 | 408 | 65 | 3660 | 88 | 32767 |
| 20 | 50 | 43 | 449 | 66 | 4026 | — | — |
| 21 | 55 | 44 | 494 | 67 | 4428 | — | — |
| 22 | 60 | 45 | 544 | 68 | 4871 | — | — |

For code c, read step s from the current index. Start difference at s shifted right by three. Add s when bit 2 is set, s shifted right by one when bit 1 is set, and s shifted right by two when bit 0 is set. These shifts discard low bits. If bit 3 is set, subtract the difference from the predictor; otherwise add it. Saturate the new predictor to −32768 through +32767.

Then change the index using the low three code bits, independently of bit 3:

| Code bits 2:0 | Index change |
|---|---|
| 0 | −1 |
| 1 | −1 |
| 2 | −1 |
| 3 | −1 |
| 4 | +2 |
| 5 | +4 |
| 6 | +6 |
| 7 | +8 |

Saturate the updated index to 0 through 88. The header’s index byte is used for the initial lookup before that update saturation; it is not checked or clipped first.

As a byte-level example, this complete mono block has two frames:

```
02 00 09 00 E8 03 00 00 17
```

BLOCK_FRAMES=2, BLOCK_BYTES=9, initial predictor=1000 and index=0. Code 1 comes first: step 7 gives difference 1, so the first output is 1001 and the index remains 0 after saturation. Code 7 gives difference 11, so the second output is 1012 and the index becomes 8. The initial value 1000 is never emitted by itself.

### 8.7.3 Seeking and reverse playback

A seek clears both decoded-frame caches. Starting from the seek-table choice, decoding advances through block headers to the block containing the requested frame, then applies the forward nibble recurrence through that frame. A request for the current or previous cached frame reuses that decoded pair. An earlier uncached request seeks again and decodes forward.

With negative RATE_STEP_Q16, the cursor requests earlier frames. An uncached frame is obtained by seeking to an earlier entry and decoding forward through it. Closely spaced seek entries reduce the amount of compressed data needed for these backward requests.

## 8.8 Source phase, rate and duration

PLAY copies the captured parameters into the target slot, makes it PLAYING, clears fade progress and filter histories, and sets the cursor to START_SAMPLE multiplied by 65536. START_SAMPLE is an unsigned frame index, not a signed word. Its initial fractional position and accumulated rate remainder are zero.

RATE_STEP_Q16 is a complete signed 32-bit Q16 word. 00010000H is +1, FFFF0000H is −1, and zero freezes phase. SOURCE_SAMPLE_RATE_HZ is an unsigned word. Convert the source-rate/rate combination as follows:

```
P = signed(RATE_STEP_Q16) × SOURCE_SAMPLE_RATE_HZ
P = q × 44100 + r
```

Choose q by division truncated toward zero; r is signed and has magnitude less than 44100. q is a cursor increment in Q16 units, not an integer source-frame count. Retain r rather than rounding it away on every output frame.

For each output frame, render the current cursor, filter if enabled, and contribute the current-gain product to the mix. Then add r to the accumulated rate remainder. At +44100 or above, subtract 44100 and add a +1 cursor-unit carry. At −44100 or below, add 44100 and use a −1 carry. Advance the cursor by q plus that carry. A cursor unit is 1/65536 of a source frame.

For 8000-Hz source data at unity rate, P=524288000, q=11888 and r=27200. Starting with remainder zero, the first two advances are 11888 and 11889 cursor units: the remainders become 27200 and 10300. The second advance includes the carry. Negative unity rate gives q=−11888 and r=−27200, with correspondingly negative carries. Zero rate, or zero source rate, causes no phase advancement; neither is rejected or clamped by this conversion.

### 8.8.1 Loops apply before the first sample

Looping exists exactly when SOURCE_LOOP_END_SAMPLE is greater than SOURCE_LOOP_START_SAMPLE as unsigned words. Otherwise it is disabled. Bounds are converted directly to Q16 and are not restricted to SOURCE_FRAME_COUNT.

An enabled loop wraps the cursor **before the first output sample**, and again after every phase advance. It is not an introduction followed by a loop. The exact wrap is:

```
loopStart + positiveModulo(cursor − loopStart, loopEnd − loopStart)
```

The loop interval includes its start and excludes its end. Positive modulo produces a nonnegative result even for negative input. This also handles an advance longer than the entire loop.

For loop [4,8), START_SAMPLE=1 wraps to frame 5 before playback, not to frame 1 or necessarily frame 4. A forward advance from 7.75 by 0.5 wraps to 4.25; a reverse advance from 4.25 by −1 wraps to 7.25. The interpolation neighbor wraps at the same loop boundary.

### 8.8.2 Nonloop completion

Before an output sample, an unlooped cursor outside [0, SOURCE_FRAME_COUNT×65536) ends without contributing a sample. Otherwise the current frame is rendered before phase advancement. Crossing either boundary then ends the voice after that rendered sample, including while a fade is active.

At source rate 44100 Hz and unity rate, a nonloop source of N frames played from zero emits frames 0 through N−1. At negative unity rate, start at N−1 to emit N−1 down through zero. Starting at N is already outside the interval. An in-range zero-rate voice does not naturally end by phase motion alone.

Natural source completion and fade completion use the same output-sample edge. The earlier condition ends the voice; coincident completion produces one end notification. A fade does not extend an unlooped source.

## 8.9 Square generator

GENERATOR_KIND=1 selects the square generator. It reads no sample bytes and supplies the same sample to left and right. Let f be the cursor's fractional position modulo 65536:

```
if f × 4096 < GENERATOR_DUTY_Q12 × 65536: sample = +32767
otherwise:                                sample = −32768
```

Duty is an unsigned Q12 threshold, unlike signed GAIN_Q12.

| Duty word | Result for an in-range fractional cursor |
|---|---|
| 00000000H | Always −32768 |
| 00000800H | Positive for the first half of the phase cycle |
| 00001000H or greater | Always +32767 |

The phase uses the same signed rate conversion as source playback. One integer source-frame advance corresponds to one complete square phase cycle. The unwrapped phase rate in cycles per second is SOURCE_SAMPLE_RATE_HZ multiplied by signed(RATE_STEP_Q16)/65536; the actual 44,100-Hz output sequence is determined by the comparison above.

For example, source rate 441 Hz, unity rate, half duty and loop [0,1) give a 100-output-frame pattern: 50 positive samples followed by 50 negative samples. FRAME_COUNT and loop bounds still govern duration and wrapping; selecting a generator does not remove those controls.

## 8.10 Gain, stereo mixing and clipping

GAIN_Q12 is the signed value of the entire word. It is not an unsigned volume percentage.

| Gain word | Multiplier |
|---|---|
| 00000000H | 0 |
| 00000800H | +0.5 |
| 00001000H | +1 |
| 00002000H | +2 |
| FFFFF000H | −1 |

Negative gain inverts polarity. Gain values above unity are not clipped when written. During a fade the retained current gain changes without rewriting the programmed slot GAIN_Q12 word.

For each channel separately, sum the products of every running voice's filtered sample and current Q12 gain. Only then shift and saturate:

```
mix = sum(sample × currentGainQ12)
output = signed16Saturate(ASR12(mix))
```

ASR12 rounds negative results toward negative infinity. There is no per-voice gain-product saturation. Two samples of +20000 at unity therefore yield a pre-clipped result of +40000 and final output +32767. One +20000 sample at unity plus one at half gain yields +30000 without clipping. Two samples of +1 at half gain combine to +1 because their fractional contributions are summed before shifting.

Minus-unity gain applied to −32768 produces +32768 before final clipping and thus +32767 at the output. This is the signed-16 asymmetry, not a gain-write restriction. An enabled filter has a separate signed-16 saturation before gain and mixing.

## 8.11 Per-voice Q14 filter

FILTER_CONTROL bit 0 enables the stereo biquad. Other bits have no decoded filter effect. The two channels share five coefficients but have independent two-word histories.

| Register halfword | Coefficient |
|---|---|
| FILTER_B0_B1 bits 15:0 | b0 |
| FILTER_B0_B1 bits 31:16 | b1 |
| FILTER_B2_A1 bits 15:0 | b2 |
| FILTER_B2_A1 bits 31:16 | a1 |
| FILTER_A2 bits 15:0 | a2 |
| FILTER_A2 bits 31:16 | Not decoded |

Each coefficient is signed 16-bit Q14: 4000H is +1, 2000H is +0.5, C000H is −1 and 8000H is −2. Decode each halfword independently; the sign of the whole packed register is not the sign of both coefficients.

For one channel, x is the interpolated source or generator sample and z1,z2 are retained signed 32-bit history words. Use the old histories for the right-hand sides:

```
y      = ASR14(b0 × x + z1)
nextZ1 = WRAP32(b1 × x − a1 × y + z2)
nextZ2 = WRAP32(b2 × x − a2 × y)
output = signed16Saturate(y)
```

ASR14 rounds toward negative infinity. WRAP32 retains the low 32 bits and interprets them as signed: 2147483648 wraps to −2147483648. History overflow wraps; it does not saturate. Crucially, history updates use the unsaturated y, not the clipped filter output. Left and right execute this recurrence with separate histories.

For a two-sample average, program FILTER_B0_B1=20002000H, FILTER_B2_A1=0 and FILTER_A2=0, then enable bit 0. Starting from zero histories, input [8192,0,−8192,0] yields [4096,4096,−4096,−4096]. On the first sample, y=(8192×8192)/16384=4096 and nextZ1=67108864. On the next zero input, that history alone gives 4096; the new history becomes zero. The negative pair follows the same recurrence.

PLAY clears histories and filter output before installing its programmed coefficients. The filter datapath's reset setting is disabled, with b0=4000H and the other coefficients zero. Live control or coefficient writes install all five coefficients from the slot words without clearing histories. Disabling bypasses processing entirely: histories remain frozen, and re-enabling resumes them. Live source replacement, live START_SAMPLE seeks and pause also retain histories.

## 8.12 Stop and fade

STOP_SLOT with FADE_SAMPLES=0 stops immediately and clears all 21 slot words. Stopping an idle slot also clears its bank. With a non-idle slot and a positive duration, STOP_SLOT instead enters FADING for at most that many rendered output frames: an unlooped source can end sooner. A paused voice remains paused, so its fade waits for resume. Writing FADE_SAMPLES alone does not begin a fade.

For initial retained signed gain G0 and duration N, configure:

```
Q = truncTowardZero(G0 / N)
R = G0 − Q × N
error = N − 1
remaining = N
```

Each frame uses the current gain before any fade adjustment. After phase advancement, first apply the loop wrap or nonloop source-end check. If the source has ended, stop. Otherwise decrement remaining. If it becomes zero, end immediately without another gain update. Otherwise add the magnitude of R to error. If error is at least N, subtract N and use carry=sign(R); otherwise carry=0. The new current gain is WRAP32(currentGain−Q−carry).

At G0=4096 and N=4, with sufficient remaining source data or an enabled loop, the four frame gains are 4096,3072,2048,1024, then the voice stops. There is no fifth frame at zero gain. A live GAIN_Q12 write sets current gain immediately and replans this arithmetic over the remaining frame count, not the original duration.

An unlooped source stops when its cursor first leaves its declared frame interval, even with fade frames remaining. No sample is fetched beyond that interval to finish the fade. For example, three remaining source frames at 44100 Hz and unity rate permit only three output frames of an eight-frame fade. With G0=4096 their gains are 4096,3584,3072, then playback ends. Provide enough remaining data or a valid loop when the complete fade is required.

Natural completion and fade completion end and clear the slot and produce the slot-ended notification described in the event section. An immediate explicit stop does not emit that notification.

## 8.13 Pause, resume and live programming

PAUSE_SLOT changes a non-idle, unpaused voice to paused without discarding it. RESUME_SLOT acts on a paused voice without replaying it. The active-mask bit remains set while paused.

A paused voice contributes no samples and consumes no phase, fade frames or filter history. Resume continues the retained cursor, gain, fade and histories. Selected-slot writes can nevertheless modify a paused voice, because it remains non-idle.

Selected-slot reads and writes first advance audio through the current access cycle. A write then takes effect for later output; it cannot change samples already due through that cycle. These accesses are distinct from writing command staging words.

### 8.13.1 Source fields are individually live

The following active-slot stores each replace the source description:

- SOURCE_ADDR and SOURCE_BYTES;
- SOURCE_SAMPLE_RATE_HZ, SOURCE_CHANNELS and SOURCE_BITS_PER_SAMPLE;
- SOURCE_FRAME_COUNT, SOURCE_DATA_OFFSET and SOURCE_DATA_BYTES;
- GENERATOR_KIND.

Each store first changes its raw slot word, then reloads all source metadata and rebinds using the cartridge selection at that access. Consecutive stores are **not an atomic source update**.

A successful replacement preserves cursor, accumulated phase remainder, current gain, fade progress and filter histories. It reconfigures the rate conversion, loop bounds and filter coefficients. BADP state is reset and sought to the retained source position. Thus even a change to a rate or extent word in this group takes the full source-replacement route.

If a live replacement cannot bind, it stops and deactivates the slot and raises SOURCE_RANGE with the newly programmed SOURCE_ADDR. Unlike a failed PLAY, this failure occurs after the raw bank write. The bank words are retained rather than cleared.

| Field written | Active or paused voice effect |
|---|---|
| Loop start or end | Recompute loop bounds |
| RATE_STEP_Q16 | Recompute q and r; retain accumulated phase remainder |
| GAIN_Q12 | Replace signed current gain; replan any remaining fade |
| START_SAMPLE | Seek immediately; clear accumulated phase remainder; retain filter histories |
| GENERATOR_DUTY_Q12 | Change the square duty threshold |
| FILTER_CONTROL or a coefficient word | Reload all five coefficients; retain histories |
| FADE_SAMPLES | Store the word only; do not start a fade |

Writing a raw bank word does not activate an idle slot. Use the command programming route when a new voice, reset histories or a coherent replacement of all captured parameters is required.

## 8.14 Clocks and transfer timing

### 8.14.1 Installed rates

The installed system clock is 33,868,800 Hz. The APU sample clock runs at 44,100 output frames per second. Sample-RAM transfer service has a separate budget of 1,058,400 words per second: 24 four-byte words per output frame.

| Quantity | Installed value |
|---|---|
| System cycles per output frame | 33,868,800 / 44,100 = 768 |
| System cycles per sample-RAM transfer word | 33,868,800 / 1,058,400 = 32 |
| Sample-RAM service for 16 words | 16 × 32 = 512 cycles |
| Sample-RAM service for 4 words | 4 × 32 = 128 cycles |

These costs cover the sample-RAM side of a DMA transfer. Add CPU setup and system-DMA transport separately. MANUAL_WRITE stores take effect at the CPU port-write edge, bypassing the FIFO and its timed batches.

All transfers share the RAM/FIFO path. Playback reads its bound sources independently of that path, so all sixteen slots retain the 44,100-Hz output cadence during sample transfers. Transfer-mode changes leave the output-frame clock running.

For a scheduled batch of N words:

**batch completion cycle = batch start cycle + 32 × N**

All N words transfer together at completion. They do not become visible one at a time during the interval. Rewriting the same mode preserves a pending batch; changing mode cancels it.

### 8.14.2 Sample sequence and edge order

SAMPLE_SEQUENCE at `08000200H` is the low 32 bits of the output-frame count. Reset establishes its phase and clears the count. It advances once per 768 system cycles even when every slot is idle. A read includes all frames due through that access cycle. Writes leave the running count unchanged and cause no read-only-port fault.

A transfer completing at an output-sample edge is visible to voice processing for that sample. For an access to TRANSFER_ADDRESS, TRANSFER_DATA or TRANSFER_CONTROL at cycle t, the order is:

1. Complete earlier transfer deadlines, with intervening voice progress.
2. Advance voice processing through cycle t − 1.
3. Complete a transfer batch due exactly at t.
4. Perform the register access.
5. Advance voice processing through t.

Thus a control write at the batch's exact deadline cannot cancel that already-due batch. An address write at that deadline affects subsequent transfers, not the batch just completed. The same transfer-before-sample rule applies when no register access occurs.

### 8.14.3 DMA transport and RAM commit are separate

DMA transports words between its source and the APU DATA port. The APU then transfers those words between its FIFO and private sample RAM. Each stage has its own timing and completion condition; adding only the sample-RAM cost does not give the complete upload time.

For a contiguous cartridge-ROM source and a fixed I/O destination, an admitted DMA block of N words costs **8 × N + 4 cycles**. The setup cost applies afresh to each admitted block. This cost and the APU's **32 × N** service cost describe separate stages.

The following controlled upload transfers 20 words. Before cycle 0, the address and DMA registers are programmed, DMA_WRITE is selected, and the FIFO is empty. The DMA source increments through readable cartridge ROM without a region crossing; its destination remains TRANSFER_DATA. Its blocks contain at most 16 words, with FORCE read admission and APU_WRITE write admission. There are no intervening mode/address writes or competing work. Cycle 0 is the first DMA block admission, not the start of CPU setup.

| Cycle | Event |
|---|---|
| 0 | Admit the first 16-word DMA block. Its cost is 8 × 16 + 4 = 132 cycles. |
| 132 | DMA supplies 16 FIFO words. BLOCK_END schedules 16 words of RAM service: 512 cycles. |
| 644 | The first 16 words commit together. The empty FIFO requests the remaining four DMA words; their block costs 8 × 4 + 4 = 36 cycles. |
| 680 | DMA supplies the four-word tail, schedules 128 cycles of RAM service, and reaches channel DONE. The tail is not yet in sample RAM. |
| 808 | The four-word tail commits; TRANSFER_BUSY clears. All 20 words are now in sample RAM. |

Here, DMA DONE occurs at cycle 680, while final RAM commit occurs at cycle **132 + 512 + 36 + 128 = 808**. Switching to STOP at DMA DONE would cancel the tail's scheduled commit.

## 8.15 Sample RAM transfer

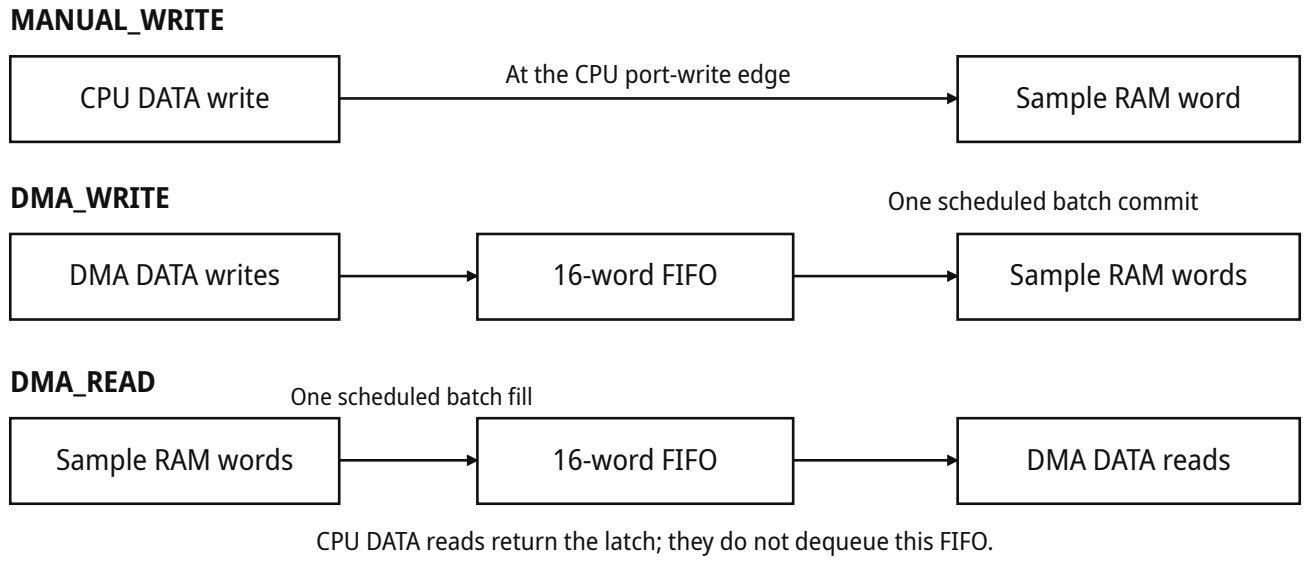



Figure 8.2: Three transfer modes share the private sample RAM and one FIFO

The rows are alternative mode paths through the same RAM. They are not three separate memories. All DATA writes update the latch; CPU DATA reads return only that latch and do not dequeue the FIFO.

### 8.15.1 Private RAM and the two address forms

The APU contains 512 KiB of private sample RAM: 524,288 bytes or 131,072 four-byte words. Reset clears it to zero. A sample-source descriptor addresses these bytes at 40000000H–4007FFFFH inclusive. A nonempty descriptor must fit entirely within this range; descriptor binding neither wraps an overlong range nor requires word alignment.

This source-address range is **not a CPU load/store aperture**. An ordinary CPU word read there returns zero and raises system bus fault UNMAPPED = 1. A word write raises that fault without changing sample RAM. Use the transfer port to write or read back the private RAM.

| Address | Register | Function |
|---|---|---|
| 080001E8H | TRANSFER_ADDRESS | Program transfer offset; read raw programmed word |

| Address | Register | Function |
|---|---|---|
| 080001ECH | TRANSFER_DATA | CPU data latch or DMA FIFO port |
| 080001F0H | TRANSFER_CONTROL | Program transfer mode; read raw programmed word |
| 08000178H | STATUS | Read transfer requests and scheduled-service state |

Writing TRANSFER_ADDRESS retains the entire word for readback and loads a separate moving byte offset:

**moving offset = programmed word AND 0007FFFCH**

Upper bits and bits 1:0 do not enter that offset. For example, programming 40000003H selects offset zero but still reads back as 40000003H. Offset zero corresponds to descriptor address 40000000H. Each actual RAM transfer reads or writes a little-endian four-byte word, then advances the offset:

**next offset = (current offset + 4) AND 0007FFFFH**

Transfers therefore wrap from the last RAM word to the first. TRANSFER_ADDRESS readback does not follow this progress.

### 8.15.2 Modes, latch and FIFO

TRANSFER_CONTROL retains the entire programmed word. Only bits 1:0 select the mode; upper bits have no transfer function.

| Bits 1:0 | Mode | Sample-RAM operation |
|---|---|---|
| 0 | STOP | No transfer batches |
| 1 | MANUAL_WRITE | CPU DATA writes store directly into RAM |
| 2 | DMA_WRITE | FIFO-to-RAM batches |
| 3 | DMA_READ | RAM-to-FIFO batches |

Changing the decoded mode cancels the pending batch and empties the FIFO. It preserves sample RAM, the data latch, TRANSFER_ADDRESS readback and the moving offset. Rewriting the same mode, even with different upper bits, preserves the FIFO and pending batch. Either write updates the request levels and starts an eligible batch when none is pending.

CPU DATA reads always return the existing data latch. They never fetch RAM, dequeue FIFO data or advance the offset. CPU DATA writes always replace the latch. In MANUAL_WRITE only, they also write RAM immediately and advance the offset. In every other mode a CPU DATA write changes only the latch; it cannot feed the DMA FIFO.

DMA DATA writes also replace the latch. In DMA_WRITE they append to the FIFO if space exists; a full FIFO discards the new word while retaining its value in the latch. Outside DMA_WRITE they neither enqueue nor write RAM. DMA DATA reads in DMA_READ dequeue the oldest available word into the latch and return it. An empty FIFO, or any other mode, returns the existing latch without dequeue or address movement. These full, empty and wrong-mode cases do not themselves raise an APU fault.

### 8.15.3 Requests and admitted blocks

| DMA request selector | Asserted condition |
|---|---|
| APU_WRITE = 3 | DMA_WRITE mode and FIFO empty |
| APU_READ = 4 | DMA_READ mode and FIFO full with 16 words |

The controller admits a block only when both its read and write request selectors pass. FORCE = 0 always passes; DISABLED = 15 never passes. Once admitted, every word in that block executes even if the APU request drops after the first word. Requests gate **whole blocks**, not individual ready words. Use the appropriate APU selector, the intended transfer mode and a fixed TRANSFER_DATA endpoint; FORCE cannot override the latch, overflow or empty-read behavior above.

The final source access and final destination access of each admitted block carry BLOCK_END. In DMA_WRITE, that edge attempts to schedule FIFO-to-RAM service; in DMA_READ, it attempts a RAM-to-FIFO refill. It occurs for intermediate blocks as well as the block that finishes a channel. It is not synonymous with channel DONE.

With no batch pending, DMA_WRITE schedules the FIFO count then present. DMA_READ schedules the free capacity, 16 minus that count. Zero words schedule nothing. The scheduled count and deadline are fixed; later scheduling attempts neither enlarge nor reprice them. At completion, all admitted words transfer in FIFO order, the offset advances per word, requests update, and another batch may be scheduled from that completion deadline.

The offset is **not captured with the batch**. Writing TRANSFER_ADDRESS while service is pending retargets that batch to the new offset, unless the batch is already due at the access cycle. Changing mode instead cancels it.

### 8.15.4 Status and CPU port ownership

| STATUS bit | Mask | Meaning |
|---|---|---|
| 7 | 00000080H | DMA_REQUEST: read request OR write request |
| 8 | 00000100H | DMA_READ_REQUEST |
| 9 | 00000200H | DMA_WRITE_REQUEST |
| 10 | 00000400H | TRANSFER_BUSY: a RAM batch is scheduled |

TRANSFER_BUSY is not a mode indicator. DMA_WRITE with an empty FIFO can request a block while TRANSFER_BUSY is clear. DMA_READ with a full FIFO can likewise request a block without pending RAM service. STATUS BUSY, bit 2, concerns active voices or queued commands; it is not transfer completion.

A CPU DATA write waits while either busy DMA channel owns DATA as its exact read or write port, even when that channel is waiting for requests. For an admitted block, ownership uses its captured base address; otherwise it uses the channel's current programmed address. The write proceeds only after neither direction owns that port. This is not FIFO-full backpressure. CPU DATA reads are not blocked by this ownership rule. DMA DATA writes are always accepted by the port without an occupancy stall; their actual effect follows the FIFO rules above.

### 8.15.5 Completing uploads and downloads

For upload, program the offset, select DMA_WRITE, and DMA to the fixed DATA port using APU_WRITE admission. DMA channel DONE reports completion of the controller's final mapped accesses and count/address updates; its DMA IRQ does not report sample-RAM commit. Wait for the final scheduled batch to finish before changing mode or using the new bytes. In particular, a mode change immediately after DONE can discard the final upload batch.

For download, program the offset before selecting DMA_READ. Filling the FIFO costs RAM-service time; APU_READ asserts only when all 16 words are available. DMA reads dequeue those prefetched words. Final BLOCK_END can schedule further prefetch even when the channel reaches DONE. After receiving the desired words, select STOP to cancel unwanted prefetch. CPU DATA reads cannot drain the FIFO. The transfer unit has no separate START/DONE register or transfer-completion IRQ.

A nonzero DMA count sets channel BUSY at START. Zero count reaches DONE at START without accessing DATA. Neither DONE nor TRANSFER_BUSY = 0 proves that the intended RAM bytes received valid data. An admitted DMA block is not aborted by a system bus fault and can still finish with DONE and its IRQ after accesses to an unmapped or read-only destination.

Use aligned word accesses for these ports. I/O byte writes, halfword accesses and unaligned word accesses raise system bus fault UNALIGNED_IO = 2 without performing the intended port operation. An aligned byte read invokes the word read and returns its low eight bits; it does not select a byte lane. STATUS rejects word writes with READ_ONLY = 3. These system bus faults are distinct from the APU fault latch.

Reset clears the programmed transfer words, offset, latch, FIFO contents/count and timing state, selects STOP, cancels service and deasserts both request lines. Reset clears sample RAM as well; ordinary STOP and mode changes preserve it.

## 8.16 End events and interrupts

An output-reported natural or faded end clears the ended slot, then publishes one SLOT_ENDED event. EVENT_KIND becomes `00000001H`, EVENT_SLOT identifies the slot, and EVENT_SOURCE_ADDR records that slot's source address from before its bank was cleared. EVENT_SEQ advances modulo $2^{32}$. All event words are published before the central APU interrupt request, bit 5 (`00000020H`), is raised.

The event registers hold the latest end. If slots 0 and 1 end during the same audio advancement, EVENT_SEQ advances twice and EVENT_SLOT becomes 1. The central pending bit can combine both requests. Compare the sequence to detect new ends and inspect ACTIVE_MASK to determine which slots remain occupied; intermediate event addresses are overwritten.

Reading event words does not clear them. There is no separate APU event-ACK port. Acknowledge central bit 5 through the interrupt controller; that does not erase the latest event payload. FAULT_ACK is not an event or central interrupt acknowledgement.

Immediate explicit STOP, pause, resume, gain changes and command/source faults do not emit SLOT_ENDED. In particular, a program waiting for an end IRQ after STOP with zero fade will wait for an event that command does not produce. A nonzero fade needs output progress to reach its end; pausing is not a way to complete it. The playback section states its exact pause/fade behavior.

When waiting for an end interrupt, have the handler record the event sequence and service the central APU source. Retain a baseline before starting the voice or fade, then recheck the completion predicate after every `halt_until_irq`. An unrelated interrupt can also wake the CPU. When several voices may end, account for the latest-event overwrite rule rather than assigning one handler call to each voice.

## 8.17 Faults and reset

APU faults are retained separately from system memory bus faults and central interrupt pending. An APU fault does not raise an interrupt request. The first unacknowledged APU fault sets STATUS.FAULT and retains FAULT_CODE/FAULT_DETAIL. Later raises preserve that first report. A nonzero FAULT_ACK clears code, detail and the fault bit; zero changes nothing. Neither operation clears an event, acknowledges central IRQ, flushes commands or stops all other voices. FAULT_ACK reads zero.

| Code | Name | Condition and detail |
|---|---|---|
| `00000000H` | NONE | No latched fault |
| `00000001H` | BAD_CMD | Unrecognized complete CMD word; detail is that word |
| `00000102H` | SOURCE_RANGE | Source binding failed; detail is SOURCE_ADDR |

A failed PLAY leaves the previous slot unchanged. A failed live source-field write retains the newly written parameter but stops and deactivates the voice; its bank remains readable. Use a staged PLAY for a coherent replacement.

Source metadata controls decode addresses directly. Match it to the supplied format and payload. Command-queue overflow is separate from these faults: a full queue replaces its oldest entry, so check CMD_FREE before a required burst.

Ordinary CPU access to `40000000H` does not reach sample RAM; it raises the system UNMAPPED fault. Invalid mapped I/O widths or alignment use the system UNALIGNED_IO fault. Mapped writes to the read-only APU ports use system READ_ONLY and do not change those ports. These accesses report through the system bus-fault registers, separately from the APU BAD_CMD/SOURCE_RANGE latch.

An APU reset clears sample RAM, transfer FIFO/state, commands, slots, output voices, faults, events and sample-clock sequence. It restores the staging parameter defaults and clears all slot-bank words. No global-reset command is encoded among CMD zero through five. STOP_SLOT affects its target; transfer STOP changes transfer mode. Neither is a private-sample-RAM clear operation.

## 8.18 Worked download: requested data and surplus prefetch

The following examples download twenty words and four words from sample RAM. Let **O** be the word-aligned starting byte offset and **Wi** the word at **O + 4i**. The tables show the moving offset; TRANSFER_ADDRESS continues to read O. Both examples stay within sample RAM without wrapping.

With the transfer unit stopped and FIFO empty, program TRANSFER_ADDRESS and arm DMA0 for a fixed TRANSFER_DATA source, an aligned incrementing RAM destination, count 20 or 4, APU_READ/FORCE requests and sixteen-word blocks (`CONTROL=00003C12H`). The channel waits for prefill. Cycle 0 is the write changing TRANSFER_CONTROL from STOP to DMA_READ. The timelines use the installed 33,868,800-Hz clock, with no intervening CPU accesses, other DMA activity, system pause or destination-region crossing.

Sample transfer costs 32 cycles per word. The initial prefill always contains 16 words and therefore takes 512 cycles, even for a four-word DMA request. APU_READ is asserted by a full FIFO, not by the requested count. Once admitted, a DMA block completes without rechecking that request between words. Fixed-port-to-RAM transport costs **N + 1** cycles per block: 17 for 16 words, then 5 for four. Prefill and transport are separate stages; the tables show states after each completed operation, not individual word-access edges.

**Twenty-word request: 16 + 4 DMA words**

| Cycle | Completed work | FIFO contents | Moving offset | Next work |
|---|---|---|---|---|
| 0 | DMA_READ selected; DMA waits | Empty | O | Prefill 16 |
| 512 | Prefill 16 | W0–W15 | O + 64 | DMA 16, 17 cycles |
| 529 | DMA writes W0–W15 | Empty | O + 64 | Prefill 16, 512 cycles |
| 1041 | Prefill 16 | W16–W31 | O + 128 | DMA 4, 5 cycles |
| 1046 | DMA writes W16–W19; DONE | W20–W31: 12 words | O + 128 | Surplus prefill 4, 128 cycles |
| 1174 | Surplus prefill 4 | W20–W35: 16 words | O + 144 | None |

**Four-word request**

| Cycle | Completed work | FIFO contents | Moving offset | Next work |
|---|---|---|---|---|
| 0 | DMA_READ selected; DMA waits | Empty | O | Prefill 16 |
| 512 | Prefill 16 | W0–W15 | O + 64 | DMA 4, 5 cycles |
| 517 | DMA writes W0–W3; DONE | W4–W15: 12 words | O + 64 | Surplus prefill 4, 128 cycles |
| 645 | Surplus prefill 4 | W4–W19: 16 words | O + 80 | None |

DMA DONE marks receipt of all requested destination words and raises the DMA interrupt. It does not mean sample transfer is idle: the final block schedules prefill into four free FIFO slots, leaving `TRANSFER_BUSY` set. Twenty requested words therefore require two 16-word prefills before DONE; the subsequent surplus is four words, not sixteen. Do not wait for `TRANSFER_BUSY` to recognize requested receipt. The low APU BUSY bit concerns voices and commands instead.

After DMA DONE, STOP strictly before the surplus deadline cancels those four pending words and discards the 12 FIFO leftovers. STOP exactly at 1174, or 645 for the short request, first completes the due prefill, then clears the FIFO. The moving offset consequently remains O + 128 or O + 64 before the deadline, versus O + 144 or O + 80 at it. STOP preserves sample RAM, copied destination words and the DATA latch; DONE remains set. STOP does not rewind the moving offset or change the programmed O. `TRANSFER_ADDRESS` still reads the programmed O, while CPU DATA reads return the latch rather than draining the FIFO. The STOP cycles refer to register-write edges. A polling loop adds its own instruction time before that write.

### 8.18.1 Download words in BLua

Own DMA0, the interrupt mask, the sample-transfer ports and sample-RAM bytes 0–159. Start with no voices using those bytes and no transfer in flight. This sequence writes recognizable raw words manually, downloads twenty words, then downloads four overlapping words from offset `40H`. It does not require an audio source format or start a voice.

```
bss received: word[24]
rodata counts: word[2] = { 20, 4 }
rodata offsets: word[2] = { 0, 0x40 }
local irq_ack<const>: *word = 0x08000004
local irq_mask<const>: *word = 0x08000008
local dma0<const>: *word = 0x0800000c
local address<const>: *word = 0x080001e8
local data<const>: *word = 0x080001ec
local control<const>: *word = 0x080001f0

*irq_mask = 0
*control = 0
*address = 0
*control = 1                -- MANUAL_WRITE: each CPU DATA write commits RAM.
for index = 0, 39 do
    *data = 0x5a000000 | index
end
for phase = 0, 1 do
    *control = 0
    *address = offsets[phase]
    *irq_ack = 1
    dma0[0] = 0x080001ec
    dma0[1] = &received[phase * 20]
    dma0[2] = counts[phase]
    dma0[3] = 0x00003c12    -- APU_READ/FORCE, fixed source, incrementing RAM.
    dma0[5] = 1
    *control = 3            -- DMA_READ: prefill before the first DMA block.
    while (dma0[4] & 1) ~= 0 do
    end
    *control = 0            -- Requested receipt is complete; discard surplus.
end
*irq_ack = 1
```

After successful bus accesses, the results are:

| Destination | Words |
|---|---|
| `received[0..19]` | `5a000000H OR 0..19` |
| `received[20..23]`, from byte offset `40H` | `5a000010H–5a000013H` |

The second receipt shows that STOP preserves the overlapping sample-RAM bytes. Both channel completions have count zero and STATUS=DONE, even with their interrupts masked. STOP clears the transfer FIFO and any not-yet-due prefetch; it does not clear the DMA status or copied destination. The timing tables above show how the moving offset differs when STOP occurs before or exactly at the prefetch deadline.

# 9 Geometry batch unit

The geometry batch unit transforms two-dimensional vertex arrays and tests convex shapes for intersection. A game can use it to move collision hulls into world coordinates, test candidate pairs and obtain overlap information while the CPU performs other work. GEO reads records and geometry from addressed memory and writes its results to main RAM.

Choose the operation according to the required coordinates and result:

| Command word | Operation | Coordinate representation | One outer record |
|---|---|---|---|
| 00000020H | XFORM2_BATCH | Signed Q16.16 | Transform a vertex span; optionally write its bounds |
| 00000021H | SAT2_BATCH | Signed Q16.16 | Test one supplied convex-polygon pair |
| 00000022H | OVERLAP2D_PASS | Binary32 | Test one candidate pair, or one full-pass outer instance |

The unit executes one job at a time. To transform hulls and then test them, complete XFORM2 before submitting SAT2 with the transformed vertex buffer. CMD captures the job configuration, while operand bytes are read during processing. Keep those buffers unchanged until completion. A CMD write while BUSY discards the running job and rejects the new submission.

XFORM2 and SAT2 share signed Q16.16 coordinates. OVERLAP2D takes binary32 geometry and adds instance translations; prepare that representation separately. The GTE provides the console's register-based transformation and projection arithmetic. To draw GEO vertices or diagnostics, convert the results into GPU packets as illustrated below.

## 9.1 GEO specifications

| Resource | Specification |
|---|---|
| Programmed bank | Sixteen 32-bit words, 64 bytes, at 08000024H…08000060H |
| Local fault acknowledgement | Separate word at 08010224H; not the next bank word |
| Active job capacity | One captured job; no command FIFO |
| COUNT / PROCESSED | Full unsigned 32-bit outer-record counts |
| XFORM2 span / SAT2 polygon | At most 64 vertices |
| OVERLAP2D polygon / clipping workspace | At most 64 input vertices / 128 clipping vertices |
| Installed main RAM | 4 MiB; addressed operand and result storage, not a private GEO payload buffer |
| Assigned GEO workspace | 512 KiB of main RAM at 08040000H…080bffffH; not an automatic operand destination |
| System-cycle timebase | 33,868,800 cycles/s |
| GEO work-credit rate | 16,384,000 outer-record units/s |

Program source and destination addresses explicitly. The assigned workspace is available in main RAM; commands use the addresses supplied by the program. Record sizes and operation-specific limits appear in the command reference.

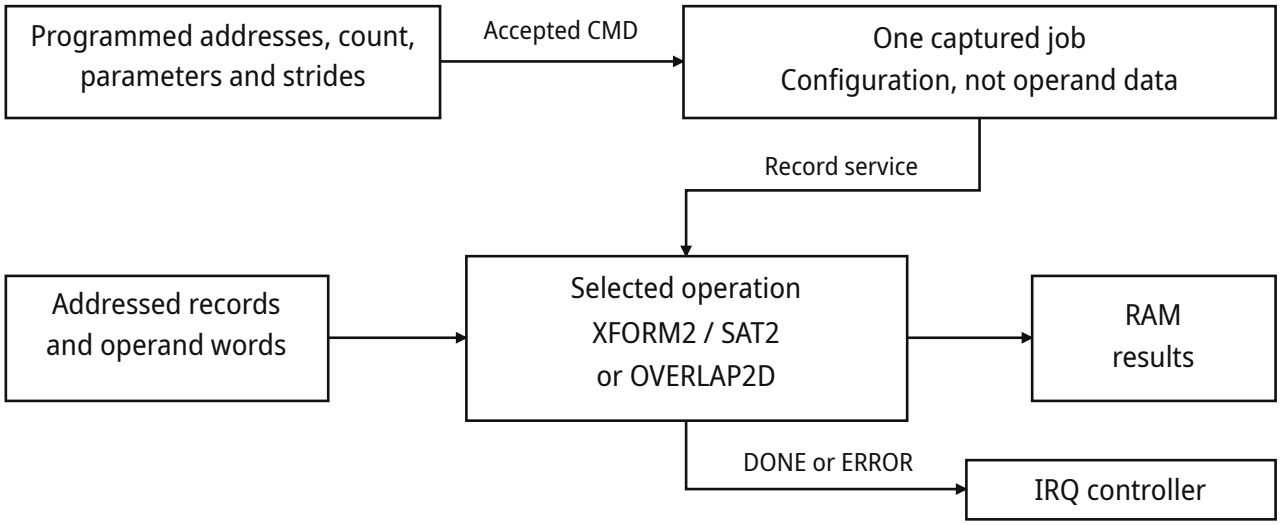



Figure 9.1: GEO captured configuration and live operand paths

A command latches addresses, counts, parameters and strides. Processing then reads the addressed records and writes their results. OVERLAP2D clears its summary when the command is accepted. A zero-count job completes at launch without processing a record.

## 9.2 An authored sprite and its collision hull

The following scene uses two copies of a 16 × 16 indexed drone sprite and a separately defined convex collision hull. The six direct16 RGB555 palette words are `0000H`, `8000H`, `294AH`, `4631H`, `6318H` and `7FFFH`. Index 0 is transparent; index 1 is opaque black. The GPU draws the sprite while GEO operates on this explicit octagon, in the stated vertex order:

```
(4,0), (12,0), (16,4), (16,12),
(12,16), (4,16), (0,12), (0,4)
```

The sprite and hull share local origin `(0,0)`; the white cross marks their center `(8,8)`. These poses use translation only. A remains at `(0,0)` while B moves to `(20,0)`, `(16,0)` and `(12,0)`. Add the instance translation to each local vertex to obtain its world coordinates.

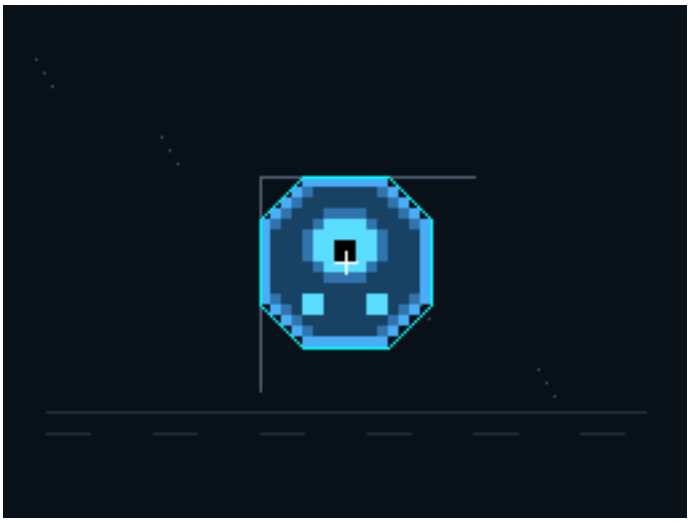


Figure 9.2: Local drone with cyan octagon, gray coordinate axes and white pivot cross.

**Local coordinates.** Gray axes start at the image origin. Cyan marks the collision hull and the white cross marks `(8,8)`. The view maps local coordinates to display pixels as `(96+4x,64+4y)`, placing the cross at `(128,96)`.

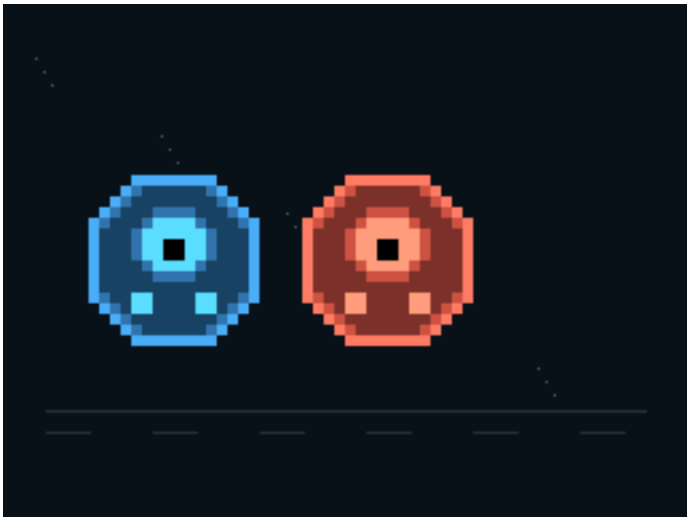


Figure 9.3: Two separated blue and red drones against a star and lane backdrop, without geometry overlays.

**Artwork alone.** The drones share one texture against a star and lane background. The world views use display coordinates `(32+4x,64+4y)`: positive X runs right and positive Y down, at four pixels per world unit.

### 9.2.1 Transform first, then test the resulting vertices

XFORM2 and SAT2 use signed Q16.16 coordinates: the signed value of each 32-bit word divided by 65536. The integer coordinates here are encoded exactly by multiplying by 65536. An affine matrix contains `m00, m01, tx, m10, m11, ty`; each matrix here is `[00010000H,0,tx,0,00010000H,0]`.

Store one local eight-vertex hull, two matrices, and these XFORM2 records. Fields are `flags, source_index, destination_index, matrix_index, vertex_count, bounds_index`:

```
[0, 0, 0, 0, 8, 0]
[0, 0, 8, 1, 8, 1]
```

They share the local hull and write A's eight world vertices followed by B's: 128 bytes altogether, plus two 16-byte Q16 bounds records. `source_index` is multiplied by STRIDE1; vertices within a hull are consecutive eight-byte records. `destination_index` is a vertex index, not a byte offset.

For XFORM2, set SRC0=records, SRC1=local vertices, SRC2=matrices, DST0=world vertices, DST1=bounds, COUNT=2 and strides `24,8,24`; submit CMD=`20H` after setup.

SAT2 must consume **that completed world buffer**. Its descriptors contain `flags`, `vertex_count`, `vertex_offset_bytes`, `reserved`:

```
A: [1, 8, 0, 0]
B: [1, 8, 64, 0]
```

The pair is `[0,0,0,1,0]`, in field order `flags, shape_a, result_index, shape_b, flags2`. Set SRC0=pair, SRC1=descriptors, SRC2=XFORM2's world buffer, DST0=20-byte result, COUNT=1, STRIDE0=20 and STRIDE1=16; submit CMD=`21H`.

Retain XFORM2's records, local hull and matrices until completion. For this job, successful completion gives STATUS=2, FAULT=0 and PROCESSED=2. Then submit SAT2, retaining its pair, descriptors and world vertices until it completes with STATUS=2, FAULT=0 and PROCESSED=1. Read the results before reusing the buffers.

### 9.2.2 Three exact poses

A's world bounds remain `(0,0,16,16)`. SAT2 returns `hit, nx, ny, depth, meta`; normal and depth words are Q16.16.

| Pose | B translation word | B world bounds | SAT2 result words |
|---|---|---|---|
| Separated | `00140000H` | `(20,0,36,16)` | `(0,0,0,0,0)` |
| Touching | `00100000H` | `(16,0,32,16)` | `(1,00010000H,0,0,2)` |
| Overlapping | `000C0000H` | `(12,0,28,16)` | `(1, 00010000H, 0, 00040000H, 2)` |

Touching occurs along `x=16`, from `y=4` to `y=12`. SAT2 counts it as a hit with zero depth. The overlapping pose has depth 4. Both normals are `(1,0)`, from A towards B.

Metadata 2 selects A's edge 2, the directed segment `(16,4)` → `(16,12)`. SAT2 packs the shape selector into the high 16 bits and edge index into the low 16 bits; flipping the normal does not change that identity. SAT2 supplies no contact point.

### 9.2.3 A separate floating-point contact query

For OVERLAP2D, prepare binary32 vertices and bounds separately from the Q16 buffer. Convert the local coordinates to numeric values and store them as F32. Both instances share the polygon and bounds `(0,0,16,16)`, with F32 translations matching the poses. Instances apply translation only; rotation and scale require transformed F32 geometry and matching bounds.

The 96-byte polygon object has kind=3, count=8, data offset=32 and bounds offset=16, with offsets relative to its descriptor. Both instances use layer=mask=1. The candidate is `[0,1,44524F4EH]`; its final word is cartridge-supplied pair information, not a computed feature. Set SRC0=instances, SRC1=candidate, DST0=36-byte contact storage, DST1=16-byte summary, COUNT=1, PARAM0=1, PARAM1=1 and strides `20,12,2`; submit CMD=`22H`.

The completed summary is `result_count, exact_pairs, broadphase_pairs, flags`:

| Pose | Summary | Valid contact |
|---|---|---|
| Separated | `(0,0,1,0)` | None |
| Touching | `(0,1,1,0)` | None |
| Overlapping | `(1,1,1,0)` | Normal `(-1,0)`, depth 4, world point `(14,8)` |

OVERLAP2D requires positive depth for a contact and points its normal from B towards A, opposite to SAT2's convention. The overlapping contact has piece indices `0,0` and feature 2. Here feature 2 is the winning edge index only, without SAT2's shape selector.

For this vertex order, the clipped intersection has six vertices. Their arithmetic mean is `(14,8)`. The reported point is the world-space mean of clipped vertices. This differs from an area-weighted centroid or a boundary contact. Supply the directed winding required by the clipping rule.

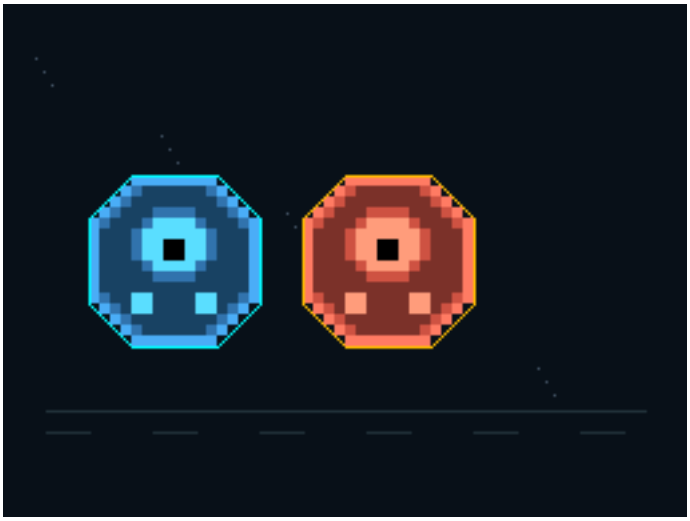


Figure 9.4: Separated drones with cyan and amber hull outlines and no result diagnostics.

**Separated.** Cyan identifies A's transformed hull; amber identifies B's. SAT2 reports no hit and OVERLAP2D reports no contact, so no arrow, contact cross or depth bracket is drawn.

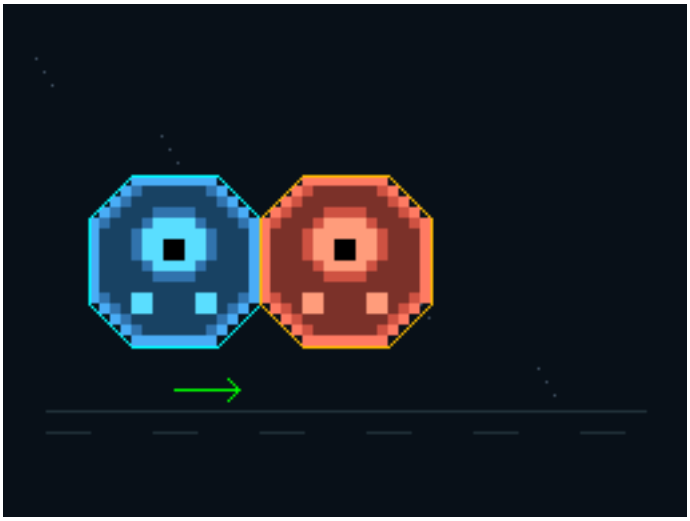


Figure 9.5: Touching hulls with an amber shared edge and a green rightward direction arrow.

**Touching.** B's amber outline is drawn after A's cyan outline, making the shared segment amber. The green arrow shows SAT2's A-to-B normal. Its deliberately chosen 24-pixel length is 6 world units, not penetration depth. SAT2's depth is zero: no yellow bracket is drawn, and OVERLAP2D supplies no contact arrow or point cross.

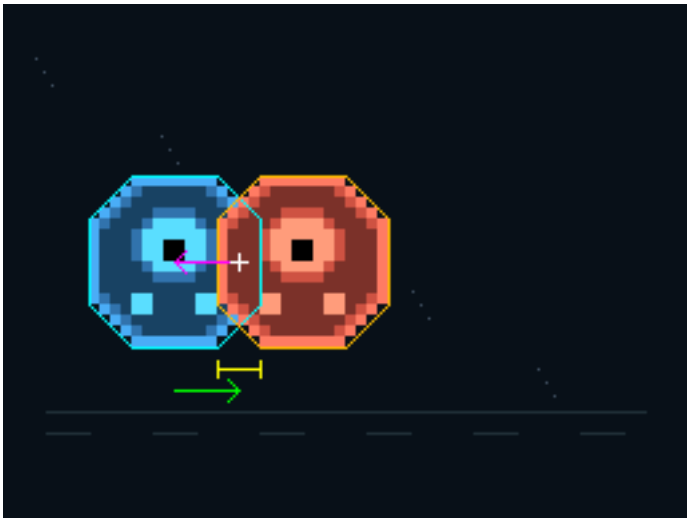


Figure 9.6: Overlapping hulls with green and magenta opposing arrows, yellow depth bracket and white contact cross.

**Overlapping.** The green SAT2 arrow points A-to-B from an illustrative anchor, not a reported point. The magenta OVERLAP2D arrow points B-to-A from the actual world contact point `(14,8)`, marked by the white cross at display pixel `(88,96)`. Both arrows are 24 pixels long for visibility, not depth measures. The yellow bracket below the hulls is the depth measure: its endpoints `(80,136)` and `(96,136)` are 16 pixels apart, representing 4 world units.

After reuse, trust the **current completed summary**, not retained contact bytes. The cartridge returns to separation using the same live contact buffer without clearing it. Its previous normal, depth, point and feature remain, but the new result count is zero, so no contact slot is valid and no old marker is submitted. The resulting picture is identical to the separated plate. SAT2 differs: separation overwrites all five result words with zero.

### 9.2.4 From geometry results to video packets

The image is enlarged before upload: each authored pixel becomes a 4 × 4 block in a 64 × 64 RGB555 texture. The 8192-byte texture is uploaded to VRAM origin `(640,0)` using image-upload command `A0H`, origin word `00000280H` and size `00400040H`. Its 2048 payload words each contain two adjacent 16-bit texels. These are repeated scalar writes to the fixed GP0

port. The 64 × 64 textured rectangles do not themselves scale a 16 × 16 image; they use the prepared texture, modulated blue for A and red for B.

Sprites are submitted before hulls. Each hull becomes eight finite flat-line packets: command/color with opcode `40H`, then two endpoints. Command colors use RGB888 with red in the low byte, then green and blue. Endpoints pack signed 11-bit integer pixel X in the low halfword and Y in the high halfword. For these positive integral poses, shifting a Q16 coordinate right by 14 combines division by 65536 with the four-pixel scale; add the display origin afterwards.

The yellow bracket uses the winning horizontal projection interval. The green arrow has a separate illustrative anchor. The magenta arrow and white cross use the valid F32 world point. The cross is drawn last. These arrowheads and bracket placements are authored for the illustrated horizontal normals, not an arbitrary rotated-normal renderer.

The scene uses two 256 × 192 GX16 drawing surfaces: VRAM word-X `[0,256)`, rows `[0,192)` and `[256,448)`. The texture occupies word-X `[640,704)`, rows `[0,64)`. Each VRAM row has 1024 halfwords. These rectangles leave the BIOS reservation untouched: in this example its queried origin is `(704,720)` and size `320 × 252`. Query the installed BIOS reservation before allocating VRAM; its extent can differ between firmware images.

PCRTC is reset and display output disabled during setup. Drawing selects the surface's draw area and offset. Fill command `02H` ignores both, so clearing the second surface explicitly uses its raw VRAM origin `(0,256)` rather than relying on the draw offset.

After drawing, wait for GPUSTAT bit 26, then select the completed framebuffer and enable display. PCRTC applies the display configuration at its VBlank latch; wait for a fresh VBlank to observe the new picture. With exclusive use of the interrupt mask, mask delivery during setup, acknowledge old VBlank bit `4`, record the handler's VBlank counter and enable the owned sources with mask `1CH`. A change in that counter identifies the required edge. The illustrations hold each pose for eight further VBlank intervals for viewing.

The scene therefore has three explicit hand-offs: XFORM2 output becomes SAT2 input, completed collision results select drawing packets, and a completed GPU picture becomes a PCRTC source. None of these hand-offs launches the next unit automatically.

## 9.3 Launching and observing a job

A write to `CMD` is the launch doorbell. Writing the source, destination, count, parameter, or stride registers only changes the programmed register words; it does not begin processing. Submit a job in this order:

1. Read `STATUS`. Do not write `CMD` while `BUSY` is set: that write terminates the running job rather than waiting for it.
2. If `ERROR` or `REJECTED` is set, inspect `FAULT` and `PROCESSED`, then acknowledge the local fault before submitting another job.
3. Prepare the command's records and payloads in memory. Program every register required by that command, including `COUNT`.
4. Write the complete command word to `CMD` once.
5. Observe `STATUS`, or use the central GEO interrupt sources. Read `PROCESSED` and `FAULT` to distinguish successful completion, execution failure, and rejection.

There is one active job and no command queue. `DONE` does not require a local acknowledgement before the next launch. An accepted launch clears the previous `STATUS`, `PROCESSED`, and `FAULT` before proceeding.

| State before the `CMD` write | Effect of the write |
|---|---|
| Idle or `DONE` | A supported complete command word starts a new job. Any other command word produces `REJECT_BAD_CMD`. |
| `BUSY` | The active job is discarded and `REJECT_BUSY` is reported. The new command is not launched. |
| `ERROR` or `REJECTED` | No new job is launched. The first fault, status, and progress remain unchanged until local fault acknowledgement. |

Every `CMD` write changes its raw readback, including a write ignored because a fault is still latched. Reading `CMD` therefore identifies the last word written, not necessarily the command that produced the current result. The supported complete words are `XFORM2_BATCH = 0x20`, `SAT2_BATCH = 0x21`, and `OVERLAP2D_PASS = 0x22`. These are whole-word encodings, not independent command bits. Extra set bits do not retain the meaning of a supported command.

### 9.3.1 Captured operands and live memory

At launch, GEO captures `CMD`, all three source words, both destination words, `COUNT`, both parameter words, and all three stride words. Subsequent writes to these registers change their readback and the configuration available to a later launch. They do not redirect or resize the active job. In particular, reading `COUNT` during processing returns the programmed word, which may differ from the count captured by that job; use `PROCESSED` for published progress.

Launch does not capture the payload bytes. Records and their referenced data are read from memory as processing reaches them. Keep an active job's input memory stable when a consistent batch is required. Changing a register and changing the memory addressed by a captured register are different actions: only the latter can change data still to be read by that job.

Command acceptance checks the complete opcode, not the record pointers, strides, parameters, alignment, flags, or payload shape. Those checks belong to the selected operation when it processes a record. A supported command with `COUNT = 0` completes immediately without processing a record; it is not a way to validate a record layout or payload. An unsupported command is still rejected when `COUNT = 0`.

`OVERLAP2D_PASS` has a launch side effect: after command acceptance, GEO writes four zero summary words at `DST1 + 0`, `+4`, `+8`, and `+12`. This happens before testing `COUNT`, including for a zero-count job. No launch-time destination validation precedes those summary writes. Program `DST1` for this operation even when no outer records will run.

### 9.3.2 Transform two squares and pass them to SAT2

This entry sequence starts with GEO idle and no sticky fault. It owns GEO and masks interrupt delivery while polling STATUS. The input square has side 4; the second matrix translates it by (+1,+1). The two jobs share the **same RAM vertex buffer**: no coordinate conversion or copying separates XFORM2 and SAT2. All numbers in the arrays are the complete words in the command layouts below.

```
module<entry>

local geo<const>: *word = 0x08000024
local irq_ack<const>: *word = 0x08000004
local irq_mask<const>: *word = 0x08000008
rodata square: word[8] = {
    0, 0, 0x00040000, 0, 0x00040000, 0x00040000, 0, 0x00040000,
}
rodata matrices: word[12] = {
    0x00010000, 0, 0, 0, 0x00010000, 0,
    0x00010000, 0, 0x00010000, 0, 0x00010000, 0x00010000,
}
rodata transforms: word[12] = {
    0, 0, 0, 0, 4, 0,
    0, 0, 4, 1, 4, 1,
}
rodata pair: word[5] = { 0, 0, 0, 1, 0 }
rodata shapes: word[8] = { 1, 4, 0, 0, 1, 4, 32, 0 }
bss vertices: word[16]
bss bounds: word[8]
bss contact: word[5]

*irq_mask = 0
geo[0] = transforms                 -- SRC0: two 24-byte records
geo[1] = square                     -- SRC1: shared local vertices
geo[2] = matrices                   -- SRC2: two 24-byte matrices
geo[3] = vertices                   -- DST0: eight world vertices
geo[4] = bounds                     -- DST1: two 16-byte bounds
geo[5] = 2                          -- COUNT
geo[11] = 24                        -- STRIDE0
geo[12] = 8                         -- STRIDE1: run-selection step
geo[13] = 24                        -- STRIDE2: matrix step
geo[6] = 0x20                       -- CMD: XFORM2_BATCH
while (geo[8] & 1) ~= 0 do          -- STATUS.BUSY
end
```

Successful completion gives STATUS=2, PROCESSED=2 and FAULT=0. The world vertices describe squares `[0,4] × [0,4]` and `[1,5] × [1,5]`; their bounds are (`0`, `0`, `00040000H`, `00040000H`) and (`00010000H`, `00010000H`, `00050000H`, `00050000H`). If ERROR or REJECTED is set, use the fault/recovery reference rather than launching the dependent job. Continue after successful completion:

```
geo[0] = pair                          -- SRC0: one pair record
geo[1] = shapes                        -- SRC1: two descriptors
geo[2] = vertices                      -- SRC2: completed XFORM2 output
geo[3] = contact                       -- DST0: one five-word SAT2 result
geo[5] = 1
geo[11] = 20                           -- Pair-record bytes
geo[12] = 16                           -- Descriptor bytes
geo[6] = 0x21                          -- CMD: SAT2_BATCH
while (geo[8] & 1) ~= 0 do
end
*irq_ack = 0x00000018                  -- Acknowledge owned central sources
```

The descriptor offsets 0 and 32 are **bytes**, not vertex indices. SAT2 ignores DST1, PARAM0, PARAM1 and STRIDE2, so the previous values need not be rewritten. Successful completion gives STATUS=2, PROCESSED=1, FAULT=0 and result words `1,0,00010000H,00030000H,0`: hit, +Y unit normal, depth 3, A's edge 0. The interrupt acknowledgement does not acknowledge a sticky local fault.

### 9.3.3 Extend the sequence to a contact query or fault recovery

OVERLAP2D requires separately prepared binary32 shapes and instances; the Q16 vertex array above is not its input format. For two axis-aligned squares of side 4 at translations `(0,0)` and `(1,1)`, use kind 1 descriptors with bounds `(0,0,4,4)`, layer=mask=1, and candidate words `[0,1,12345678H]`. Program the following words, then issue CMD=22H and wait until STATUS.BUSY clears:

| Register | Candidate mode | Full-pass mode over the same two instances |
|---|---|---|
| SRC0 | Address of the two 20-byte instances | Same |
| SRC1 | Address of the 12-byte candidate | 0; unused |
| DST0 | Address of at least one 36-byte contact | Same |
| DST1 | Address of a writable 16-byte summary | Same |
| COUNT | 1 | 2 |
| PARAM0 / PARAM1 | 1 / 1 | 2 / 1 |
| STRIDE0 / STRIDE1 / STRIDE2 | 20 / 12 / 2 | 20 / 0 / 0 |

Both modes produce normal `(0,−1)`, depth 3 and world contact `(2.5,2.5)`; piece indices and feature are zero. The normal points **B to A**, unlike SAT2. Both summaries are `(1,1,1,0)`. Candidate mode retains pair information 12345678H; full-pass mode generates 1 for pair `(0,1)`. PROCESSED is respectively 1 and 2: even the last full-pass instance consumes an outer record.

To see the fault boundary, substitute an XFORM2 record whose flags word is 1, set COUNT=1 and CMD=20H. It produces STATUS=6, PROCESSED=0 and FAULT=00080000H before fetching vertices or a matrix. Acknowledge the local fault, then submit an empty job:

```
local fault_ack<const>: *word = 0x08010224
*fault_ack = 1                         -- Clear sticky FAULT; STATUS becomes 2
*irq_ack = 0x00000010                  -- Separate GEO_ERROR acknowledgement
geo[5] = 0
geo[6] = 0x20
```

The zero-count job immediately completes with STATUS=2, PROCESSED=0 and FAULT=0 without reading the still-programmed bad record. It raises GEO_DONE; acknowledge central bit 3 separately when that request is no longer needed. No recovery step erases the earlier RAM results.

## 9.4 Register reference

| Offset | Address | Register | Access | Reset | Word interpretation |
|---|---|---|---|---|---|
| `+0x00` | `0x08000024` | `SRC0` | RW | `0` | Full 32-bit captured source word; command-specific meaning. |
| `+0x04` | `0x08000028` | `SRC1` | RW | `0` | Full 32-bit captured source word; command-specific meaning. |
| `+0x08` | `0x0800002c` | `SRC2` | RW | `0` | Full 32-bit captured source word; command-specific meaning. |
| `+0x0c` | `0x08000030` | `DST0` | RW | `0` | Full 32-bit captured destination word; command-specific meaning. |
| `+0x10` | `0x08000034` | `DST1` | RW | `0` | Full 32-bit captured destination word; command-specific meaning. |
| `+0x14` | `0x08000038` | `COUNT` | RW | `0` | Unsigned 32-bit outer-record count, captured at launch. |
| `+0x18` | `0x0800003c` | `CMD` | RW | `0` | Whole-word launch doorbell: `0x20`, `0x21`, or `0x22`. Other complete words reject when launch is permitted. |
| `+0x1c` | `0x08000040` | `CTRL` | RW | `0` | Bit 0: self-clearing `ABORT` pulse. Bits 31:1 retain the written bits and define no further controller action. |
| `+0x20` | `0x08000044` | `STATUS` | RO | `0` | Bit 0 `BUSY`; bit 1 `DONE`; bit 2 `ERROR`; bit 3 `REJECTED`. No further status bits are defined. |
| `+0x24` | `0x08000048` | `PARAM0` | RW | `0` | Full 32-bit captured parameter word; command-specific meaning. |

| Offset | Address | Register | Access | Reset | Word interpretation |
|---|---|---|---|---|---|
| +0x28 | 0x0800004c | PARAM1 | RW | 0 | Full 32-bit captured parameter word; command-specific meaning. |
| +0x2c | 0x08000050 | STRIDE0 | RW | 0 | Full 32-bit captured stride word; command-specific meaning. |
| +0x30 | 0x08000054 | STRIDE1 | RW | 0 | Full 32-bit captured stride word; command-specific meaning. |
| +0x34 | 0x08000058 | STRIDE2 | RW | 0 | Full 32-bit captured stride word; command-specific meaning. |
| +0x38 | 0x0800005c | PROCESSED | RO | 0 | Full 32-bit number of completed outer records. |
| +0x3c | 0x08000060 | FAULT | RO | 0 | Bits 31:16: fault code. Bits 15:0: failing outer-record index, or rejection sentinel 0xffff. |
| Separate | 0x08010224 | FAULT_ACK | RW | 0 | Zero write does nothing. Any nonzero write acknowledges the local fault and immediately returns readback to zero; no individual acknowledgement bit is assigned. |

The bank begins at 0x08000024 and occupies 16 consecutive 32-bit words. The offsets below are relative to that address. All bank words and the separate FAULT_ACK word reset to zero. Use aligned, mapped 32-bit accesses: unaligned mapped word accesses and mapped byte or halfword writes fault. A mapped write to a read-only GEO word raises the memory READ_ONLY bus fault and does not change that word.

Writable configuration words retain all written bits without controller-side masking or decoding. Do not assume that bits unused by a particular command read as zero. CMD also retains its complete written word. CTRL retains the written word except that bit 0 is cleared whenever written as one; it has no start, interrupt-enable, or reset bit.

GEO's RAM data path accesses bytes directly, including RAM at addresses reserved for CPU I/O. Writing a GEO result there changes the underlying RAM rather than operating an I/O register. Apply each command's range rules when allocating buffers, and keep results out of memory reserved for other uses.

## 9.5 Work budget and timing

GEO accrues work credit at 16,384,000 units/s against the 33,868,800-Hz system timebase. One unit permits one outer record to run, including a record that faults. XFORM2 consumes one record per vertex span, SAT2 one per supplied pair, and OVERLAP2D one per candidate or full-pass outer instance.

The cost within a record does not add credit units. In particular, an OVERLAP2D full-pass record can test several pairs and produce several contacts. Use the outer-record count, rather than vertex or contact count, for this timing budget.

A fresh nonzero launch starts with no accumulated whole work units and no fractional carry. Idle time does not bank work. While busy, elapsed CPU cycles add credit at the ratio

`16,384,000 / 33,868,800 = 640 / 1323` work units per CPU cycle.

Whole credit can be spent on outer records; the fractional remainder carries forward. Credit is capped by the number of records still remaining. GEO requests service at the first cycle that can fund the next outer record, or immediately when sufficient whole credit is already available. Service can process several records when several units have accrued.

For a fresh, continuously accrued budget, the credit threshold for `N > 0` outer records is `ceiling(N × 1323 / 640)` CPU cycles:

| Outer-record invocations funded | CPU-cycle credit threshold |
|---|---|
| 1 | 3 |
| 2 | 5 |
| 4 | 9 |
| 16 | 34 |
| 64 | 133 |
| 640 | 1323 |

The thresholds cover GEO work credit. CPU setup, delayed service and interrupt handling add to an operation's elapsed time; a fault, abort or rejection can terminate it early. The credit rate is fixed.

COUNT accepts the full unsigned 32-bit range. Per-record limits still apply: 64 vertices for an XFORM2 span or SAT2/OVERLAP polygon, and 128 clipping vertices in OVERLAP2D.

## 9.6 Completion, faults, and recovery

Each successful outer record increments `PROCESSED` by one. Successful completion publishes `STATUS = 0x2` (`DONE` only), the completed count, and `FAULT = 0`, then raises `GEO_DONE`.

An execution fault publishes `STATUS = 0x6` (`DONE | ERROR`) and raises `GEO_ERROR`. The failing record is not added to `PROCESSED`. `FAULT` contains its zero-based outer-record index in bits 15:0 and the fault code in bits 31:16. The index is truncated to 16 bits independently of the full-width count and progress words. Preserve `PROCESSED` when diagnosing a large batch; the fault word alone does not retain the index's high bits.

A rejected submission publishes `STATUS = 0x8` (`REJECTED` without `DONE`), resets `PROCESSED` to zero, and uses fault index `0xffff`. It raises `GEO_ERROR`. Rejection during `BUSY` discards the active job even if some records had already completed. Neither rejection nor execution failure rolls back output memory already written.

| Fault code | Name |
|---|---|
| 0x0001 | ABORTED |
| 0x0002 | BAD_RECORD_ALIGNMENT |
| 0x0003 | BAD_VERTEX_COUNT |
| 0x0004 | SRC_RANGE |
| 0x0005 | DST_RANGE |
| 0x0006 | DESCRIPTOR_KIND |
| 0x0007 | NUMERIC_OVERFLOW_INTERNAL |
| 0x0008 | BAD_RECORD_FLAGS |
| 0x0009 | RESULT_CAPACITY |
| 0x8001 | REJECT_BUSY |
| 0x8002 | REJECT_BAD_CMD |

The command reference below identifies which conditions produce each fault.

To stop a busy job, write CTRL with bit 0 set. ABORT terminates it with DONE | ERROR, code ABORTED, and the current completed-record index. Earlier outputs remain. The pulse clears itself. In idle or DONE, it changes no job result. With ERROR or REJECTED already latched, it cannot replace the first fault.

After inspecting a fault, write any nonzero word to FAULT_ACK. This clears ERROR, REJECTED, and FAULT; it preserves DONE, BUSY, and PROCESSED. An execution-error result consequently becomes DONE; a rejection becomes idle. A zero write has no effect. Acknowledgement never resumes the discarded job.

Local fault acknowledgement and central interrupt acknowledgement are separate. GEO_DONE uses central bit 3 (0x8); GEO_ERROR uses bit 4 (0x10). Success raises only the done source; execution fault, abort, and rejection raise only the error source. Pending sources are recorded regardless of masking. The central IRQ registers are FLAGS at 0x08000000, ACK at 0x08000004, and MASK at 0x08000008. Clear a pending source by writing its one bit to central ACK; that write leaves other pending sources untouched. Central ACK then reads zero. Masking a source does not clear it. FAULT_ACK does not clear central pending bits, and central acknowledgement does not release a sticky GEO fault. Use the interrupt-controller chapter for CPU interrupt entry.

GEO reset cancels service, discards the active job and its credit, returns to idle, and zeroes all 16 bank words and FAULT_ACK. It does not itself clear central IRQ pending state; full device reset resets the interrupt controller separately. No GEO register write provides a reset operation.

## 9.7 XFORM2_BATCH: transform vertices and build bounds

Command word: **0x20**.

XFORM2_BATCH applies one affine matrix to each selected run of two-dimensional vertices. Each record selects its source run, matrix, destination run and, optionally, an axis-aligned bounding box (AABB).

### 9.7.1 Registers and address selection

| Register | XFORM2_BATCH role |
|---|---|
| SRC0 | Transformation-record table base |
| STRIDE0 | Byte step between transformation records |
| SRC1 | Local-vertex data base |
| STRIDE1 | Byte step used to select a source run; not the step between its vertices |
| SRC2 | Matrix-table base |
| STRIDE2 | Byte step between selectable matrices |
| DST0 | Transformed-vertex output base; vertices occupy 8 bytes each |
| DST1 | AABB output base; boxes occupy 16 bytes each |
| COUNT | Number of transformation records |
| PARAM0, PARAM1 | Ignored |

For record ordinal n, beginning at zero, the selected addresses are:

| Object | Address | Size accessed |
|---|---|---|
| Record | `SRC0 + n × STRIDE0` | 24 bytes |
| Source run | `SRC1 + src_index × STRIDE1` | `vertex_count × 8` bytes |
| Matrix | `SRC2 + aux_index × STRIDE2` | 24 bytes |
| Output run | `DST0 + dst_index × 8` | `vertex_count × 8` bytes |
| AABB | `DST1 + dst1_index × 16` | 16 bytes, unless suppressed |

Thus dst_index counts vertices, while dst1_index counts boxes. The source vertices within a run are consecutive 8-byte records. Neither a field in the record nor a destination index changes the batch record ordinal.

### 9.7.2 Record and data layouts

All offsets and strides below are in **bytes**. Each word occupies four little-endian bytes. Index, count and flags words are unsigned. Signed Q16 words represent the signed 32-bit word divided by 65536; the same representation is used for coordinates, matrix coefficients, translations and bounds.

**Transformation record — 24 bytes**

| Offset | Word | Meaning |
|---|---|---|
| +0 | flags | Must be zero |
| +4 | src_index | Source-run index, scaled by STRIDE1 |
| +8 | dst_index | Output-vertex index, scaled by 8 |
| +12 | aux_index | Matrix index, scaled by STRIDE2 |
| +16 | vertex_count | Number of consecutive vertices; 0–64 |
| +20 | dst1_index | AABB index, scaled by 16; `0xffffffff` suppresses the box |

The `0xffffffff` value is special only in `dst1_index`: it suppresses both AABB address checking and writing. No equivalent sentinel applies to the other indices. A zero vertex count succeeds after the record and flags checks, without checking or accessing the source run, matrix, vertex output or AABB.

| Data object | Size | Word offsets and contents |
|---|---|---|
| Vertex | 8 bytes | +0: x; +4: y, both signed Q16 |
| Matrix | 24 bytes | +0: m00; +4: m01; +8: tx; +12: m10; +16: m11; +20: ty, all signed Q16 |
| AABB | 16 bytes | +0: min_x; +4: min_y; +8: max_x; +12: max_y, all signed Q16 |

### 9.7.3 Transformation arithmetic

For each input vertex, the two matrix rows produce the output x and y independently. Using the signed **raw words**, a row `(a, b, t)` is evaluated as follows:

1. Form the exact signed 64-bit products `a × x` and `b × y`.
2. Add those products, saturating the sum to the signed 64-bit range.
3. Add `t × 65536`, again saturating to signed 64-bit range.
4. Shift arithmetically right by 16 bits, then saturate to signed 32-bit range.

The first row is `(m00, m01, tx)` and the second is `(m10, m11, ty)`. Saturation does not wrap, and arithmetic overflow does not raise a numeric-overflow fault. The shift is not round-to-nearest: a negative fractional raw unit rounds downward. For example, raw `a = -1`, `x = 1`, with the other terms zero, produces raw `-1` (`0xffffffff`), not zero. An identity x coefficient with a +1 translation applied to raw `2147483647` remains `2147483647` through saturation.

The matrix is read before processing the vertices. Vertices are then read and written in ascending order, with each source vertex read immediately before its output x and y are written. Bounds are formed from the saturated output words, initialized from the first vertex, and written only after all vertices: min_x, min_y, max_x, max_y. Overlapping source and output storage is not prohibited or copied aside; an earlier output can change a later source read. The already-read matrix is unaffected.

### 9.7.4 Address and memory rules

These rules also apply to SAT2_BATCH below. For a nonempty indexed span, the byte displacement, base-plus-displacement and span end must not overflow the 32-bit address space. A span end exactly `0x100000000` passes this arithmetic check. A zero stride is permitted and repeatedly selects the same address.

Every input span must lie wholly within one readable region: the system-ROM window `0x00000000–0x00ffffff`, installed main RAM beginning at `0x08000000`, or the cartridge-bus window `0x10000000–0x30ffffff`. The cartridge window is not a promise of ROM at every address; it includes cartridge RAM and device mappings. System-ROM reads beyond the populated contents, but within its window, return zero. Output spans must lie wholly in installed main RAM. The RAM checks do not exclude its I/O-address window, and these word transfers do not invoke mapped-I/O handling.

XFORM2_BATCH imposes no separate word-alignment check on these addresses or strides. In particular, the fault name BAD_RECORD_ALIGNMENT must not be read as an additional alignment requirement.

### 9.7.5 Fault priority and output preservation

The first applicable entry below determines the current record's fault. All checks precede output writes; a failed record writes neither vertices nor its AABB.

| Priority | Condition | Fault word |
|---|---|---|
| 1 | Record address/span overflows | 2 — BAD_RECORD_ALIGNMENT |
| 2 | Record span is not readable | 4 — SRC_RANGE |
| 3 | flags is nonzero | 8 — BAD_RECORD_FLAGS |
| 4 | vertex_count is zero | Success; no further checks or writes |
| 5 | vertex_count exceeds 64 | 3 — BAD_VERTEX_COUNT |
| 6 | Source-run span overflows or is not readable | 4 — SRC_RANGE |
| 7 | Matrix span overflows or is not readable | 4 — SRC_RANGE |
| 8 | Vertex-output span overflows or is outside installed main RAM | 5 — DST_RANGE |
| 9 | Unsuppressed AABB span overflows or is outside installed main RAM | 5 — DST_RANGE |

### 9.7.6 Worked transformation

Take four local vertices `(0,0), (4,0), (4,4), (0,4)`. Their raw words use `4 = 262144`. Matrix 0 is identity with zero translation; matrix 1 is identity with `tx = ty = 65536`, translating by `(1,1)`.

| Record | flags | src_index | dst_index | aux_index | vertex_count | dst1_index |
|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 4 | 0 |
| 1 | 0 | 0 | 4 | 1 | 4 | 1 |

The first output run is unchanged. The second starts 32 bytes after DST0 and contains `(1,1), (5,1), (5,5), (1,5)`. For example, its x coordinate for input `(4,0)` is raw `327680`, representing 5. The AABB words are `(0,0,262144,262144)` and `(65536,65536,327680,327680)` respectively. These eight consecutive output vertices can also supply the two SAT2 shapes below.

## 9.8 SAT2_BATCH: test convex-polygon pairs

Command word: **0x21**.

SAT2_BATCH tests pairs of polygon descriptors using their edge normals as separating axes. It returns a hit flag, selected normal, interval-overlap depth and the edge that supplied that axis. Polygon coordinates are signed Q16 words in one common coordinate space; SAT2 does not transform them.

### 9.8.1 Registers and layouts

| Register | SAT2_BATCH role |
|---|---|
| SRC0, STRIDE0 | Pair-table base and byte step between pair records |
| SRC1, STRIDE1 | Shared descriptor-table base and byte step for both shape indices |
| SRC2 | Shared vertex-pool base |
| DST0 | Result-table base; each result occupies 20 bytes |

| Register | SAT2_BATCH role |
|---|---|
| COUNT | Number of pair records |
| DST1, PARAM0, PARAM1, STRIDE2 | Ignored |

| Object | Address | Size |
|---|---|---|
| Pair n | `SRC0 + n × STRIDE0` | 20 bytes |
| Descriptor A/B | `SRC1 + shape_A/B_index × STRIDE1` | 16 bytes each |
| Vertex run A/B | `SRC2 + vertex_offset_bytes` | `vertex_count × 8` bytes each |
| Result | `DST0 + result_index × 20` | 20 bytes |

A descriptor's vertex offset counts **bytes**, not vertices. Vertices are consecutive x/y records in the same 8-byte format as XFORM2. All pair and descriptor fields are unsigned words, with no index sentinel.

**Pair record — 20 bytes**

| Offset | Word | Requirement or unit |
|---|---|---|
| +0 | flags | Must be zero |
| +4 | shape_A_index | Descriptor index, scaled by STRIDE1 |
| +8 | result_index | Result index, scaled by 20 |
| +12 | shape_B_index | Descriptor index, scaled by STRIDE1 |
| +16 | flags2 | Must be zero |

**Polygon descriptor — 16 bytes**

| Offset | Word | Requirement or unit |
|---|---|---|
| +0 | flags | Exactly 1: convex polygon |
| +4 | vertex_count | 3–64 |
| +8 | vertex_offset_bytes | Byte offset from SRC2; divisible by 4 |
| +12 | reserved | Must be zero, not ignored |

There is no separate alignment test for pair, descriptor or result addresses. The vertex-offset test checks the offset alone, not the final address or an 8-byte boundary. No winding, convexity, area, distinct-vertex or topology check is performed.

**Result — 20 bytes**

| Offset | Word | Interpretation |
|---|---|---|
| +0 | hit | Unsigned 0 or 1 |
| +4 | nx | Signed Q16 normal x component |
| +8 | ny | Signed Q16 normal y component |
| +12 | depth | Signed Q16 overlap depth |
| +16 | meta | Shape selector in high 16 bits; zero-based edge index in low 16 bits |

The selector is 0 for A and 1 for B; edge indices are 0–63. There are no hit or normal-sign bits in meta.

### 9.8.2 Axis selection and arithmetic

Axes are visited in A edge order, then B edge order, including each closing edge back to vertex 0. An edge from `(x0,y0)` to `(x1,y1)` supplies the perpendicular `(-(y1-y0), x1-x0)`, normalized by its Euclidean length. Zero-length edges are skipped. Axes are not deduplicated; up to 128 candidate edges may be visited.

Edge differences, normalization and projections use double-precision floating-point arithmetic on signed coordinate raw words, without a Q16 product shift. Each projection is `x × axis_x + y × axis_y`; every vertex contributes to the interval

minimum and maximum. An axis separates only when `A.min > B.max` or `B.min > A.max`. The first separating axis writes five zero words and finishes the pair successfully; no separating-edge metadata is returned.

Otherwise its overlap is `min(A.max,B.max) − max(A.min,B.min)`. This is interval intersection length, including containment, not a special containment escape distance. The smallest overlap wins; equal overlaps retain the earlier axis. Touching intervals therefore can return hit 1 with depth 0.

For a hit, the normal is flipped only if its dot product with `meanB − meanA` is negative. Each mean is the arithmetic mean of vertices, not an area centroid. A zero dot product preserves the edge-derived direction. Flipping does not change meta. Normal components are multiplied by 65536 before rounding to output words. Depth is already in Q16 raw coordinate units and is **not** multiplied by 65536 again. Both round to nearest, with exact halves toward positive infinity, then saturate to signed 32-bit range.

Duplicate vertices are allowed through the stated checks. If all edges of both shapes have zero length, the pair faults without a result write. One nonzero edge is sufficient for evaluation, even for collinear, zero-area or nonconvex lists. Identical collinear lists `(0,0), (1,0), (2,0)` consequently return a hit with normal `(0,1)` and zero depth.

### 9.8.3 Fault priority and writes

| Priority | Condition | Fault word |
|---|---|---|
| 1 | Pair address/span overflows | 2 — BAD_RECORD_ALIGNMENT |
| 2 | Pair is not readable | 4 — SRC_RANGE |
| 3 | Either pair flags word is nonzero | 8 — BAD_RECORD_FLAGS |
| 4 | Result span overflows or is outside installed main RAM | 5 — DST_RANGE |
| 5 | Either descriptor span overflows or is not readable | 4 — SRC_RANGE |
| 6 | Either descriptor flags differs from 1, or reserved is nonzero | 6 — DESCRIPTOR_KIND |
| 7 | Either vertex count is below 3 | 3 — BAD_VERTEX_COUNT |
| 8 | Either vertex offset is not divisible by 4 | 2 — BAD_RECORD_ALIGNMENT |
| 9 | Either vertex count exceeds 64 | 3 — BAD_VERTEX_COUNT |
| 10 | Either vertex span overflows or is not readable | 4 — SRC_RANGE |
| 11 | No nonzero edge axis exists | 6 — DESCRIPTOR_KIND |

Thus a misaligned vertex offset takes priority over a count above 64, but not over a count below 3. Neither operation returns fault 7, NUMERIC_OVERFLOW_INTERNAL.

Failures leave the current result untouched. Successful hits and misses overwrite all five words in layout order. Source data for the pair is read before its result is written. No alias restriction or automatic result clearing is imposed: a later pair can observe an earlier result write if their buffers overlap.

### 9.8.4 Worked pair and output interpretation

Use the two square runs from the worked transformation above. Descriptor 0 contains `(flags=1, count=4, offset=0, reserved=0)`; descriptor 1 uses offset 32. The pair words are `(0,0,0,1,0)`, selecting A=0, B=1 and result 0.

A edge 0 supplies axis `(0,+1)`. The projected intervals are raw `[0,262144]` and `[65536,327680]`, giving overlap `196608`, or 3 units. A edge 1 has the same overlap and loses the strict tie. The mean displacement is `(1,1)`, so the normal is not flipped. The five result words are **`(1,0,65536,196608,0)`**: hit, normal `(0,1)`, depth 3, A edge 0.

With B translated by `(4,0)` instead, the squares touch and return `(1,65536,0,0,1)`: depth zero, normal `(1,0)`, A edge 1. Translation by `(5,0)` returns five zero words. Identical squares retain A edge 0 and return `(1,0,65536,262144,0)`. For extreme coordinates, depth saturates: identical squares spanning raw `-2147483648` to `2147483647` on both axes return depth `2147483647`, not a wrapped value.

## 9.9 OVERLAP2D_PASS: translated-shape contacts

`OVERLAP2D_PASS` (`0x22`) finds strict two-dimensional overlaps between translated shapes. It accepts AABBs, convex polygons, and compounds of those primitives. For each overlapping instance pair it emits at most one contact: a normal, penetration depth, contact point, and identifiers for the selected pieces and feature. Non-overlapping pairs occupy no output slots.

Use **candidate mode** when the cartridge already has a pair list, needs a particular pair order, or carries its own pair metadata. Use **full pass** to examine every distinct pair in an instance array. The two modes share the same mask, bounds, exact-overlap, and contact operations.

All addresses and offsets below are byte quantities unless explicitly identified as indices or counts. `U32` means an unsigned 32-bit register or memory word. `F32` means the IEEE 754 binary32 bit pattern stored in one word; geometry is not fixed-point. Source F32 values are decoded for double-precision calculation, and contact components are rounded back to F32 on output.

### 9.9.1 Pass configuration

The pass uses the configuration captured at launch:

| Launch word | Candidate mode | Full pass |
|---|---|---|
| `SRC0` | Instance-array address | Instance-array address |
| `STRIDE0` | Instance stride in bytes | Instance stride in bytes |
| `SRC1` | Candidate-pair-array address | Unused |
| `STRIDE1` | Candidate-pair stride in bytes | Unused |
| `COUNT` | Number of candidate records | Number of instances |
| `STRIDE2` | Number of addressable instances | Unused |
| `SRC2` | Unused | Unused |
| `DST0` | Packed contact-result array | Packed contact-result array |
| `DST1` | 16-byte summary address | 16-byte summary address |
| `PARAM1` | Result capacity, in records | Result capacity, in records |

Only `PARAM0 & 3` selects the mode. Selector `1` is candidate mode; selectors `0`, `2`, and `3` are full pass. Thus `2` is the named full-pass encoding, but is not its only accepted encoding.

The remaining `PARAM0` bits have no effect on this pass. In particular, the named broadphase field (bits 2–3, mask `0x0000000C`, with NONE=`0` and LOCAL_BOUNDS=`4`), contact field (bits 4–5, mask `0x00000030`, with CLIPPED_FEATURE=`0`), output field (bits 6–7, mask `0x000000C0`, with STOP_ON_OVERFLOW=`0`), and reserved mask `0xFFFF0000` are not selectable policies or rejected encodings. Local-bounds filtering and stop-on-overflow operate regardless of these bits. For example, `PARAM0=6` selects full pass; it does not enable an additional broadphase operation.

A launch clears the processed, result, exact-pair, and broadphase-pair counts and writes a zero summary, including when `COUNT=0`. There is no separate OVERLAP2D configuration-validation stage.

### 9.9.2 Input records

#### 9.9.2.1 Instances and candidate pairs

An instance occupies five words, 20 bytes:

| Byte offset | Encoding | Meaning |
|---|---|---|
| `+0` | U32 | Absolute shape-descriptor address |
| `+4` | F32 | Translation `tx` |
| `+8` | F32 | Translation `ty` |
| `+12` | U32 | Layer bitmask |
| `+16` | U32 | Collision mask |

Instance index `i` addresses `SRC0 + i × STRIDE0`. Several instances may reference the same descriptor with different translations. Translation is the only instance transform: there is no rotation or scale field.

A candidate record occupies three words, 12 bytes:

| Byte offset | Encoding | Meaning |
|---|---|---|
| +0 | U32 | Instance A index |
| +4 | U32 | Instance B index |
| +8 | U32 | Opaque pair metadata |

These are instance indices, not byte offsets. Records are visited in array order at `SRC1 + recordIndex × STRIDE1`. A self-pair faults before the instance-index bounds check. Otherwise both indices must be less than `STRIDE2`. A is read before B. Repeated and reversed records remain separate operations; neither ordering nor metadata is normalized.

Full pass visits `(0,1), (0,2), …, (1,2), …`, with A index increasing in the outer loop and B running from A+1 to `COUNT−1`. It generates no self-pairs or reversed pairs. Its pair metadata is `((A & 0xFFFF) << 16) | (B & 0xFFFF)`. Only metadata is truncated to 16-bit halves; instance addressing uses the complete indices, with no separate 65,535- or 65,536-instance limit.

#### 9.9.2.2 Shape descriptors and bounds

Every shape descriptor is 16 bytes:

| Byte offset | Encoding | Meaning |
|---|---|---|
| +0 | U32 | Kind: AABB=1, convex polygon=3, compound=4 |
| +4 | U32 | Data count; interpretation depends on kind |
| +8 | U32 | Data offset in bytes, relative to this descriptor |
| +12 | U32 | Bounds offset in bytes, relative to this descriptor |

A bounds block is four consecutive F32 words: left at `+0`, top at `+4`, right at `+8`, bottom at `+12`. Its world bounds add `tx` to left/right and `ty` to top/bottom. Bounds are neither reordered nor recomputed from exact geometry, and their consistency with the shape is not checked. Supply bounds even if the named broadphase NONE encoding is present in `PARAM0`.

| Kind | Exact data at descriptor + data offset | Data count |
|---|---|---|
| AABB (`1`) | Four F32 words: left, top, right, bottom | Exactly `4` |
| Convex polygon (`3`) | Vertices, each 8 bytes: x F32 at `+0`, y F32 at `+4` | Vertex count, `3` through `64` |
| Compound (`4`) | Inline 16-byte child descriptors | Nonzero piece count |

An AABB uses four world vertices in this order: `(left,top)`, `(right,top)`, `(right,bottom)`, `(left,bottom)`, with the instance translation added. The exact-data block may also serve as its bounds block. Setting both offsets to `16` therefore gives a 32-byte AABB shape.

Polygon data spans `dataCount × 8` bytes. Translation is added to every vertex read. Convexity, winding, finite coordinates, duplicate vertices, and nonzero area are not validated or repaired.

Compound piece `i` is the descriptor at `parent + dataOffset + i × 16`: the array contains descriptors, not pointers or indices. Each child's offsets are relative to that child descriptor. All children inherit the instance translation; there is no per-piece transform. There is no explicit upper piece-count limit. The zero-count check occurs only after the outer instance bounds overlap. Child descriptors, bounds, and exact data are checked as they are reached. Narrowphase accepts only AABB or polygon children, so a nested compound faults if exact loading reaches it; a child whose bounds miss need not reach that check.

#### 9.9.2.3 Address spans

Descriptor offsets are unsigned byte offsets, not signed displacements. Indexed address formation retains the low 32 bits, but an accepted input span must also pass the no-overflow checks: the index-times-stride product and base-plus-offset must fit in 32 bits, and the span end must not exceed `0x100000000`. There is no alignment predicate in this pass.

Guarded inputs must lie in the readable system-ROM, main-RAM, or cartridge-bus windows. System-ROM and RAM words are little-endian; cartridge reads use the cartridge bus. Results require installed main RAM. Summary writes use ordinary word writes without a preceding summary-span or RAM check; an invalid `DST1` can produce a bus fault rather than an OVERLAP2D destination-range fault. The four summary words are written separately, in order.

### 9.9.3 Filtering, order, and counts

For each instance pair, the pass checks readability of both shape descriptors before applying masks. The pair proceeds only when both `(maskA & layerB) != 0` and `(maskB & layerA) != 0`. A rejected pair reads no bounds and changes none of the three summary counts.

A mask-passing pair increments `broadphasePairCount`, then reads A bounds followed by B bounds. Bounds reject only when A lies strictly beyond B on an axis: `leftA > rightB`, `rightA < leftB`, `topA > bottomB`, or `bottomA < topB`. Touching bounds therefore pass. Bounds are used exactly as supplied.

After overlapping outer bounds, the pass checks top-level kinds and compound piece counts, then increments `exactPairCount` once for the instance pair—not for each piece test or hit. Piece iteration visits A pieces in ascending order and, within each, B pieces in ascending order. Before the B-piece loop, the pass checks A's piece-descriptor span and reads A's bounds. It checks B's descriptor span and reads B's bounds for each piece pair. Disjoint piece bounds skip exact loading; overlapping piece bounds cause both descriptors to be read again for exact loading.

All overlapping piece pairs are tested. The selected contact has the smallest penetration depth; ties choose the smaller A piece index, then smaller B piece index, then smaller feature metadata. A later fault aborts the record even if an earlier piece pair had produced a provisional contact.

### 9.9.4 Exact overlap and contact

AABB and polygon pieces use the same polygon operation; there is no separate AABB/AABB shortcut. Exact loading reads A before B. An AABB count other than four, or a polygon count outside 3–64, faults before its exact-data range check.

The separating-axis test visits A's edges first, then B's, in ascending edge order, with the closing edge last. For an edge from `(x0,y0)` to `(x1,y1)`, its axis is the left normal `(-(y1−y0), x1−x0)` divided by `sqrt((x1−x0)² + (y1−y0)²)`. An edge is skipped unless its length is positive. Every world vertex is projected onto each usable axis.

The overlap on an axis is `min(maxA,maxB) − max(minA,minB)`. Unless it is strictly positive, the pair has no contact. Touching is not overlap, and a pair with no usable axes has no contact. Depth is the smallest of these interval-intersection lengths, without a special containment-depth correction. Equal depths favor an A-owned edge, then the lower edge index. Feature metadata contains only that edge index; it does not identify which shape supplied the edge.

The normal is directed using the arithmetic means of the two shapes' vertices. If `(meanA−meanB) · axis` is negative, the chosen axis is flipped; if zero, its original left-normal direction is retained. The ordinary direction is therefore **from B toward A**, not A toward B. Changing the supplied A/B order changes this orientation rule.

The contact point is computed separately by clipping world polygon A successively against B's directed edges. A point is inside when `(x1−x0) × (py−y0) − (y1−y0) × (px−x0) >= 0`: the left half-plane, including its boundary. Winding is not normalized.

For each clipping plane, processing starts with the last input vertex as S and visits each E in ascending order:

| S → E classification | Output |
|---|---|
| Inside → inside | E |
| Inside → outside | Intersection |
| Outside → inside | Intersection, then E |
| Outside → outside | Nothing |

For signed distances sd and ed, the intersection uses t = sd / (sd−ed) and S + (E−S) × t. The two clipping buffers exchange roles after each plane; processing stops if the input becomes empty. Vertices are not deduplicated.

A nonempty clipped polygon produces the arithmetic mean of its output vertices—not an area-weighted centroid. Duplicate vertices retain their weight. An empty clipped polygon does not cancel a successful separating-axis test: its point is (meanA + meanB) / 2. Clipping does not change normal or depth.

The clipping workspace holds 128 vertices in each of two buffers; there is no explicit clipping-capacity fault. Zero-length edges are skipped, and zero projected overlap rejects contact. Non-finite values receive the stated arithmetic and comparisons, not a separate value-classification check.

### 9.9.5 Output records and summary

Results are packed at DST0 + resultCount × 36, independent of input strides:

| Byte offset | Encoding | Meaning |
|---|---|---|
| +0, +4 | F32 | Normal x, y |
| +8 | F32 | Penetration depth |
| +12, +16 | F32 | Contact-point x, y |
| +20, +24 | U32 | Winning A, B piece indices |
| +28 | U32 | Winning edge index, without edge-owner bit |
| +32 | U32 | Candidate metadata or generated full-pass metadata |

Primitive shapes report piece index zero. There is no hit flag: misses emit nothing.

The summary at DST1 is:

| Byte offset | Encoding | Meaning |
|---|---|---|
| +0 | U32 | resultCount |
| +4 | U32 | exactPairCount |
| +8 | U32 | broadphasePairCount |
| +12 | U32 | Flags: bit 0 overflow; ordinary value zero |

The exact-pair and broadphase-pair counters are unsigned 32-bit counters. Each increment wraps modulo 2^32: FFFFFFFFH + 1 becomes 00000000H. The counters do not impose a smaller limit on COUNT or terminate a full pass at rollover.

It contains neither capacity nor processed count. The processed count advances only after an entire input record succeeds: one candidate record in candidate mode, one outer instance in full pass. Full pass reads its final instance and publishes a summary even though that record has no later partner.

### 9.9.6 Capacity, partial progress, and faults

Capacity is checked only after the complete piece search selects a hit. If resultCount >= PARAM1, the pass publishes current counters with overflow bit 0 set, then faults with RESULT_CAPACITY; it writes no new result and does not increment resultCount. Capacity zero can succeed when there are no hits. This check precedes destination validation.

Each actual result separately requires a non-wrapping 36-byte span in installed main RAM. The entire declared capacity is not preflighted. resultCount advances after all nine result words are written. Earlier results remain after a later fault.

The summary is written initially, after every successful input record, and on capacity overflow. Other faults do not refresh it. In particular, a failing full-pass outer record may already have advanced counters or written contacts that its last published summary does not include. The failing record does not advance the processed count.

| Code | OVERLAP2D fault |
|---|---|
| 3 | BAD_VERTEX_COUNT: invalid AABB or polygon count |
| 4 | SRC_RANGE: invalid input span or candidate instance index |

| Code | OVERLAP2D fault |
|---|---|
| 5 | `DST_RANGE`: invalid result span or non-RAM result destination |
| 6 | `DESCRIPTOR_KIND`: unsupported shape kind or zero-piece compound |
| 8 | `BAD_RECORD_FLAGS`: candidate self-pair |
| 9 | `RESULT_CAPACITY`: selected hit exceeds result capacity |

The declared alignment fault (`2`) and numeric-overflow fault (`7`) are not emitted by this pass. Reserved parameter bits are not rejected.

In candidate mode, fault checks begin with candidate-record span/readability, then self-pair rejection, instance-index bounds, and the A and B instance-record spans/readability in that order. Full pass first reads/checks its outer A instance, then each B instance.

For each reached pair, the order is descriptor readability, mask filtering, A bounds, B bounds, outer-bounds rejection, top-level kind/count checks, exact-count increment, A-piece descriptor/bounds, B-piece descriptor/bounds, piece-bounds rejection, exact A kind/count/data, exact B kind/count/data, contact selection, capacity, and result destination. The first encountered fault stops the record.

### 9.9.7 Small AABB example

Build one 32-byte shape from descriptor words `[1, 4, 16, 16]`, followed by F32 bounds/data `[0, 0, 4, 4]`. Two 20-byte instances share it: A has translation `(0,0)`, B `(1,1)`, and both have layer and mask `1`. Supply candidate `[0, 1, 0x12345678]`.

Set `SRC0` to the instances, `STRIDE0=20`, `SRC1` to the candidate, `STRIDE1=12`, `COUNT=1`, `STRIDE2=2`, `PARAM0=1`, and `PARAM1>=1`, with result and summary storage provided as described above. The result is normal `(0,−1)`, depth `3`, point `(2.5,2.5)`, pieces `0/0`, feature `0`, and unchanged metadata `0x12345678`. The normal's x component may carry signed zero; positive-zero bits are not promised. Summary is `[1,1,1,0]`, processed count `1`.

With full-pass `PARAM0=2` and `COUNT=2`, the contact is the same, metadata is `1`, and processed count is `2`. Moving B to `(4,0)` yields touching bounds but no exact overlap: summary `[0,1,1,0]`, even with capacity zero. At `(5,0)`, summary is `[0,0,1,0]`. Either mask set to zero gives `[0,0,0,0]`, provided both descriptor spans remain readable.

Repeating the original overlapping candidate twice with capacity one leaves the first result intact. The second hit publishes `[1,2,2,1]` and faults with code `9` at record index `1`; processed count remains `1`.

# 10 Image decompression unit

## 10.1 Function and resources

The image decompression unit expands compressed texture and palette data into GPU upload packets. A program supplies the compressed stream and VRAM destination fields; IMGDEC produces a texture-upload header and decoded words, followed by a CLUT upload when palette data is present. This permits compressed image data in ROM to be uploaded through DMA with little CPU involvement.

The DATA port has separate input and output FIFOs. CPU writes or an input DMA channel feed compressed words into it. CPU reads or an output DMA channel remove the generated packets. Route the output to GPU GP0 for an upload, or to RAM for later use. The GPU performs the resulting VRAM writes.

Input and output DMA can remain armed while decoding proceeds. The shared DMA controller serves one block at a time. Request lines regulate FIFO flow: missing input holds decoding, and a full output FIFO holds the next output batch. If the output remains blocked, input can fill in turn. A direct DATA-to-GP0 DMA block starts when both IMGDEC output and GPU write requests permit it.

The programming interface, compressed input, decoded payload, and output packets use 32-bit words. Input and output counts are word counts, not byte counts. Decoded payload words and generated packet-header words are distinct: only the former contribute to DECODED_WORD_COUNT.

For DMA input from ROM or main RAM, serialize each compressed word in little-endian byte order: `31444D49H` occupies bytes `49 4D 44 31` at increasing addresses. This is data-word order, not Book II, “CPU and BLua32 instruction reference”, under “Instruction words and execution resources”; see Memory — Addresses and data sizes.

| Resource | Capacity or basis |
|---|---|
| Register word | 32 bits |
| Input FIFO | 32 words |
| Output FIFO | 64 words |
| Decompression history | 4096 decoded payload words |
| System-cycle timebase | 33,868,800 cycles per second |
| Output-word charge | 2 system cycles per output word, including generated headers |

Each generated word, including packet headers, costs two system cycles. Budget source transport, output DMA and GPU processing separately. IMGDEC DONE indicates that the complete output stream has been read; wait for GPU completion before using the uploaded image.

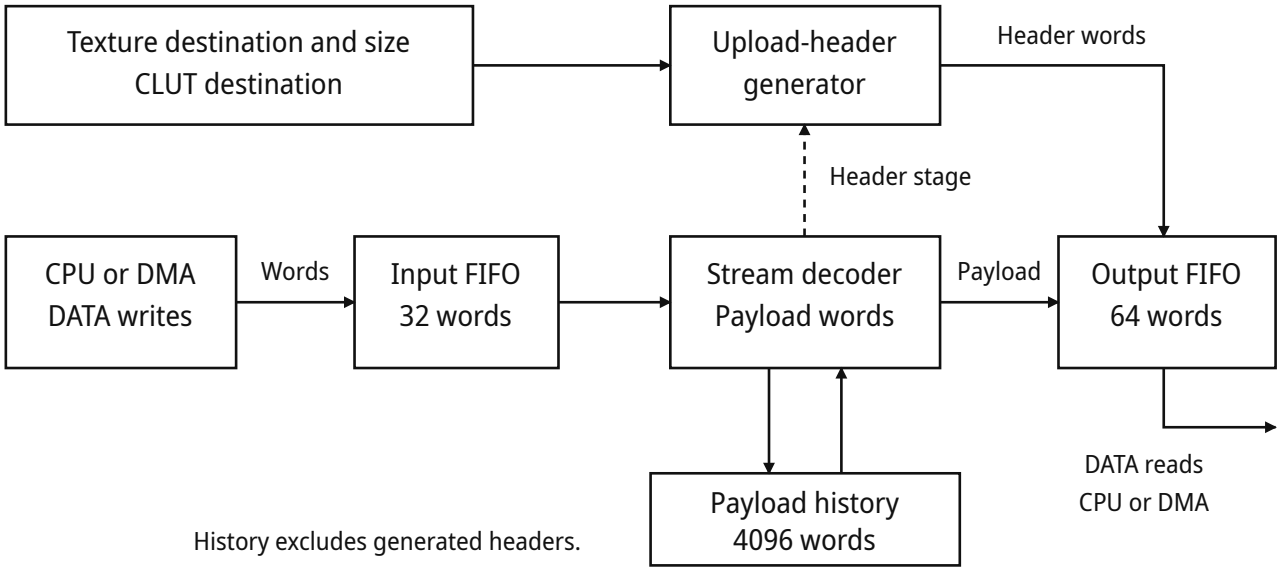



Figure 10.1: IMGDEC stream, header and payload-history paths

Compressed words select literals, repeated values, zeros or copies from payload history. Generated GPU headers bypass that history. DATA reads drain the resulting packet stream for RAM storage or GPU processing.

## 10.2 Starting a transfer

1. Prepare the complete compressed word stream and determine both its input count and the generated output count. Output includes packet headers as well as decoded payload; the stream and output layouts below give those counts.
2. While IMGDEC is inactive, program INPUT_WORD_COUNT, TEXTURE_DESTINATION, TEXTURE_SIZE and CLUT_DESTINATION. The destination words are GP0 packet fields, not input-memory pointers.
3. Provide both an input producer and an output consumer. For the two-channel DMA route, arm the output transfer, arm the input transfer, then write CONTROL with START set. The BLua programming sequence uses request-gated DMA for both sides; it does not wait for all output to be produced before draining it.
4. Distinguish input DMA completion, output transfer and IMGDEC's terminal result. Inspect STATUS to separate DONE from FORMAT_FAULT and read exactly the expected output count. Completion is not a reason to make speculative extra DATA reads.

## 10.3 Programming IMGDEC in BLua

### 10.3.1 Decode the worked stream into RAM

The following nineteen-word stream produces twenty decoded payload words and six header words, for twenty-six output words. With IMGDEC and both DMA channels idle, arm output before starting the input and decoder. The fragments assume exclusive use of those units and the interrupt mask. Array indices address consecutive hardware registers.

```
rodata stream: word[19] = {
    0x31444d49, 12, 8, 1, 0x11112222, 0x33334444,
    0x40000001, 0x55556666, 0x80001003, 0xc0000003,
    7, 0, 0x03e0001f, 0x7fff7c00, 0x42104210,
    0x001f001f, 0x03e003e0, 0x7c007c00, 0x7fff7fff,
}
bss ram_output: word[26]
local dma0<const>: *word = 0x0800000c
local dma1<const>: *word = 0x080103e4
local decoder<const>: *word = 0x080103fc
local irq_ack<const>: *word = 0x08000004
```

```
local irq_mask<const>: *word = 0x08000008

*irq_mask = 0
*irq_ack = 0x00000181
-- IMGDEC input count and raw GP0 texture/CLUT fields.
decoder[0] = 19
decoder[1] = 0x00400040
decoder[2] = 0x00060004
decoder[3] = 0x00800040
-- Output: fixed DATA -> incrementing RAM, IMGDEC_READ/FORCE.
dma1[0] = 0x08010414
dma1[1] = ram_output
dma1[2] = 26
dma1[3] = 0x00003c1a
dma1[5] = 1
-- Input: incrementing ROM -> fixed DATA, FORCE/IMGDEC_WRITE.
dma0[0] = stream
dma0[1] = 0x08010414
dma0[2] = 19
dma0[3] = 0x00003d41
dma0[5] = 1
decoder[4] = 1
while (dma0[4] & 1) ~= 0 or (dma1[4] & 1) ~= 0 do
end
while (decoder[5] & 1) ~= 0 do
end
*irq_ack = 0x00000181
```

After successful bus accesses, `ram_output` contains the 26 packet words in the worked expansion. IMGDEC STATUS is `02H`, INPUT_WORDS_RECEIVED is 19 and DECODED_WORD_COUNT is 20. Both DMA counts are zero and both DMA statuses are DONE. The three separate pending interrupt bits are raised even though delivery is masked.

For this valid stream, both DMA channels complete normally. For a general loader, also monitor IMGDEC STATUS while waiting: FORMAT_FAULT withdraws its requests and can leave a channel BUSY. Stop or recover that transfer before reusing its resources. Inspect bus faults as well as decoder status. DMA, Completion and interrupts gives the interrupt-driven completion sequence.

## 10.4 Register reference

IMGDEC occupies nine consecutive 32-bit registers, beginning at `0x080103FC`. Adjacent registers are four bytes apart. All reset values in the table are complete 32-bit words. R denotes read-only; R/W denotes read/write. The five configuration registers share the inactive-only write condition described below.

| Address | Name | Access | Bits | Reset | Function |
|---|---|---|---|---|---|
| `0x080103FC` | INPUT_WORD_COUNT | R/W, configuration | 31:0 | `0x00000000` | Total number of compressed input words to submit for the stream. |
| `0x08010400` | TEXTURE_DESTINATION | R/W, configuration | 31:0 | `0x00000000` | Raw destination word placed in the texture upload header. |
| `0x08010404` | TEXTURE_SIZE | R/W, configuration | 31:0 | `0x00000000` | Raw size word placed in the texture upload header. |
| `0x08010408` | CLUT_DESTINATION | R/W, configuration | 31:0 | `0x00000000` | Raw destination word placed in the CLUT upload header, when present. |

| Address | Name | Access | Bits | Reset | Function |
|---|---|---|---|---|---|
| `0x0801040C` | CONTROL | R/W, configuration | START at 0; retained bits 31:1 | `0x00000000` | Starts a new stream when written with START set. |
| `0x08010410` | STATUS | R | Defined fields 4:0; zero 31:5 | `0x00000000` | Activity, terminal result, and input/output request state. |
| `0x08010414` | DATA | R/W, data port | 31:0 | `0x00000000` | Writes submit compressed input; reads obtain output through the shared data latch. |
| `0x08010418` | INPUT_WORDS_RECEIVED | R | 31:0 | `0x00000000` | Live count of submitted DATA writes, not words consumed by decompression. |
| `0x0801041C` | DECODED_WORD_COUNT | R | 31:0 | `0x00000000` | Live count of decoded payload words produced; excludes generated headers. |

Writes to STATUS, INPUT_WORDS_RECEIVED, or DECODED_WORD_COUNT raise `BUS_FAULT_READ_ONLY`; they do not acknowledge or clear those registers.

### 10.4.1 Configuration writes

INPUT_WORD_COUNT, TEXTURE_DESTINATION, TEXTURE_SIZE, and CLUT_DESTINATION retain the complete words written to them. Reads return those retained words. IMGDEC forwards the destination and texture-size words unchanged into the corresponding output headers; it does not normalize them on write or readback. The destination and size words are packet fields, not addresses from which IMGDEC fetches input.

Program all four words before START. Configuration and CONTROL accept writes while IMGDEC is inactive and no system pause is requested. A CPU write made while this condition is false waits until it can proceed. To restart after a fault, wait for the stopped state and issue a new START; writing START during active decoding waits rather than aborting the stream.

The programming words remain in their registers during the stream. The input count governs input submission and completion checks; destination and size words supply the headers as those headers are produced.

### 10.4.2 CONTROL — `0x0801040C`

| Bits | Mask | Field | Write behavior and readback |
|---|---|---|---|
| 0 | `0x00000001` | START | A written 1 starts a new stream and is cleared as part of that start. A written 0 does not start a stream. |
| 31:1 | `0xFFFFFFFE` | Retained, no defined decoding function | Retained exactly as written, including when START is written as 1. |

An accepted CONTROL write retains its whole word before START is acted upon. Starting clears only bit 0 of CONTROL; bits 31:1 are not cleared or interpreted as additional controls. CONTROL readback therefore shows the retained word with START cleared after a start, not necessarily an all-zero word.

START initializes a fresh stream and replaces STATUS with BUSY before input processing and request-state evaluation. It does not clear the four programming words. Do not require BUSY to be observable after an invalid start: with INPUT_WORD_COUNT equal to zero, the new stream immediately reports FORMAT_FAULT because its required input header cannot be supplied.

### 10.4.3 STATUS — `0x08010410`

| Bit(s) | Mask | Field | Meaning when set |
|---|---|---|---|
| 0 | `0x00000001` | BUSY | A stream is active. |
| 1 | `0x00000002` | DONE | The stream has completed successfully, including the required output reads. |
| 2 | `0x00000004` | INPUT_REQUEST | The input DMA-request condition is asserted. |
| 3 | `0x00000008` | OUTPUT_REQUEST | The output DMA-request condition is asserted. |
| 4 | `0x00000010` | FORMAT_FAULT | The stream stopped with a format fault. |
| 31:5 | `0xFFFFFFE0` | Unused | Zero. |

START replaces the previous status word with BUSY. Request-state evaluation then updates INPUT_REQUEST and OUTPUT_REQUEST without changing the other fields. These request fields describe the DMA request conditions, not merely whether one DATA access could proceed.

A successful terminal stop replaces the status word with DONE; a format-fault stop replaces it with FORMAT_FAULT. These are replacement results, not bits accumulated alongside the previous BUSY or request fields. STATUS is not a write-one-to-clear register. See Data transfer and completion for the precise request predicates, output-drain requirement, and interrupt handling.

### 10.4.4 Progress registers

INPUT_WORDS_RECEIVED reports how many input DATA writes have been submitted. It is not an input-FIFO occupancy count and does not measure how much of the compressed stream has already been decoded.

DECODED_WORD_COUNT reports produced payload words across the texture and CLUT payloads. Generated upload headers do not increment it. Reading output through DATA does not turn this value into a count of words remaining in the output FIFO. Both progress counts are 32-bit counts and begin again at zero for a new stream.

## 10.5 Data transfer and completion

DATA is a bidirectional word port with one retained data latch. Writes supply compressed input; reads remove generated output. The input FIFO holds 32 words and the output FIFO holds 64 words. DATA reads return generated output, not the submitted input stream. DATA writes do not add words directly to the output FIFO.

For a CPU DATA write to proceed, all of the following must hold:

- A job is active and no system pause is requested.
- INPUT_WORDS_RECEIVED is less than INPUT_WORD_COUNT.
- At least one input FIFO word is free.
- DMA does not own the DATA write port.

A blocked CPU write waits and retries; it does not submit a word or advance input progress. A new CPU write needs one free FIFO word. DMA input uses the separate block-request threshold and programmed transfer count; pace it through that request line.

Every DATA write that executes replaces the retained DATA word. It enters the input FIFO if space remains and increments INPUT_WORDS_RECEIVED modulo 2^32. The count advances even if the FIFO is full and the word cannot be enqueued. INPUT_WORDS_RECEIVED counts submitted writes, not compressed words already consumed by decoding. Do not use unrequested DMA writes to probe readiness.

A CPU DATA read removes one output FIFO word when available, retains that word in DATA, and advances the internal output-read count modulo 2^32. If DMA owns the DATA read port, a CPU read returns the retained word immediately: it neither removes output nor performs the completion or scheduling checks below. Without that ownership, an empty output FIFO also returns the retained word, but the read still performs those checks. Consequently, a repeated DATA value is not evidence that another output word was generated.

DECODED_WORD_COUNT counts decoded payload words, modulo 2^32. It advances when a payload word enters the output FIFO and history, not when software reads it. Generated output-header words do not contribute to this register. It is neither the number of DATA reads performed nor the total generated packet length.

### 10.5.1 DMA requests and block boundaries

IMGDEC input uses DMA request selector 5; IMGDEC output uses selector 6. INPUT_REQUEST and OUTPUT_REQUEST report the corresponding request predicates. Both predicates are false while the unit is inactive or a system pause is requested. Their thresholds use a 16-word block quantum:

| Request | Remaining count | Assertion condition while active and no system pause is requested |
|---|---|---|
| Input, selector 5 | INPUT_WORD_COUNT minus INPUT_WORDS_RECEIVED | Remaining is nonzero and input FIFO free words are at least `min(remaining, 16)` |
| Output, selector 6 | Expected total output words minus words already read | Remaining is nonzero and queued output words are at least `min(remaining, 16)` |

Both remaining-count subtractions are modulo 2^32. A tail shorter than 16 words uses its exact remaining length as the threshold. The output threshold is not merely “one word available.” Similarly, INPUT_REQUEST can be clear even when a CPU DATA write has enough space to proceed. Request bits describe block readiness, not a separate one-word CPU readiness flag. IMGDEC releases a waiting CPU DATA write when the input request is asserted and DMA does not own the DATA write port. This release condition differs from the one-free-word test for a new CPU write: an already waiting write is not released merely because one FIFO slot becomes free.

Each DMA word still performs its DATA latch, FIFO and counter operation. An input DMA write without BLOCK_END then returns without scheduling more decoding or recomputing the IMGDEC requests. The block-ending input write performs those operations after submitting its word. An output DMA read without BLOCK_END likewise returns after its DATA operation, without testing completion, scheduling more decoding or recomputing the requests. The block-ending output read performs those operations after removing its word. CPU DATA accesses perform the corresponding scheduling and request-update work immediately, except for the read-port ownership case described above.

#### 10.5.1.1 Successful completion and stopped output

Finishing payload generation alone does not set DONE. After each CPU DATA read, and after each block-ending DMA DATA read, completion is tested against the output-read count, including any word just removed. Success requires that count to equal the entire expected output, including headers. At that point the output stage must be complete, the decoder must be at its token state, INPUT_WORDS_RECEIVED must equal INPUT_WORD_COUNT, and the input FIFO must be empty. For DMA output, this completion check occurs only at BLOCK_END.

A successful stop replaces STATUS with DONE; a format-error stop replaces it with FORMAT_FAULT. Both stops end active processing, cancel the pending decode batch, withdraw both DMA requests and raise the IMGDEC interrupt. Waiting configuration writes can then proceed when the system-pause condition also permits them. Neither stop empties the FIFOs nor clears DATA or the programming registers. FORMAT_FAULT can therefore leave a generated output prefix in the output FIFO. With output request 6 withdrawn, an armed DMA channel can remain BUSY waiting for more output. Inspect and recover that channel separately. CPU reads may remove retained output when DMA does not own the read port.

**Caution — read exactly the known output count.** After a drained successful stream, an extra CPU DATA read without a DMA read-port owner returns the last word and repeats the completion check. It can raise IRQ_IMGDEC again without producing or consuming a new word. Poll STATUS rather than making extra DATA reads for completion. After FORMAT_FAULT, an empty read does not by itself satisfy successful-completion conditions.

### 10.5.2 Interrupts and fault recovery

IMGDEC raises central interrupt bit 7, `00000080H`, for both DONE and FORMAT_FAULT. The interrupt bit does not distinguish success from failure; inspect STATUS. The central interrupt registers are FLAGS at `08000000H`, ACK at `08000004H`, and MASK at `08000008H`. Raising an interrupt ORs its bit into FLAGS independently of MASK. The interrupt line is asserted when `FLAGS & MASK` is nonzero. MASK does not clear pending flags.

Write `00000080H` to ACK to clear the pending IMGDEC flag; ACK reads as zero. ACK clears only the bits written as ones. A shared handler must acknowledge only the interrupt sources it services. DMA channel 0 uses bit 0 and DMA channel 1 uses bit 8; these are distinct from IMGDEC bit 7. Neither a new IMGDEC START nor an IMGDEC unit reset acknowledges the central pending flag.

If completion is awaited with interrupts, test the owned completion predicate in a loop. An accepted interrupt records a CPU event when no HALT wait is latched. A subsequent `halt_until_irq` consumes that event instead of sleeping; otherwise it establishes the HALT wait. This permits the predicate to be rechecked even when an interrupt is accepted between its test and HALT. Recheck IMGDEC STATUS after waking, since another interrupt source can also wake the CPU.

All five IMGDEC configuration registers accept writes only while inactive and no system pause is requested. There is no active-job ABORT or restart command through CONTROL. A CPU CONTROL write during an active job waits; setting START does not bypass this gate. A DMA configuration write presented while the gate is closed is not stored and does not start a job. After a stopped job, program the next job and issue a new START when this gate is open. START discards the previous FIFOs, DATA word, progress, run state and available history before beginning the new stream; it clears only CONTROL bit 0 and replaces STATUS with BUSY. Handle any pending central IRQ separately.

### 10.5.3 Reset and system pause

An IMGDEC unit reset cancels scheduled decoding and deasserts both IMGDEC DMA requests. It makes the unit inactive, empties both FIFOs, clears available history and resets all stream progress. It clears DATA, the four programming words, CONTROL and STATUS, and removes the system-pause request. The decoder returns to the initial magic-word phase and texture-header output stage; run, repeat, back-reference distance and scheduled-word state are cleared. History availability is reset; no history word remains valid for a new stream. The central interrupt flag is not acknowledged by this unit reset.

System pause preserves the active stream and FIFO contents. On a pause request, IMGDEC withdraws both DMA requests and holds new CPU DATA writes, configuration writes and output batches. A batch already in progress completes at its normal edge, after which the unit is quiescent. Either FIFO can still contain words, and STATUS can still indicate an active stream.

On resume, decoding and request generation continue. A waiting configuration write still requires the stream to become inactive before it proceeds.

## 10.6 Work timing and transfer budget

All word counts in this section refer to 32-bit words. IMGDEC charges **two system cycles per output word**, including packet headers. A three-word texture or CLUT header therefore costs six cycles. Stream-header and token parsing do not carry a separate per-input-word charge. Compressed input length alone does not determine the output-work budget.

### 10.6.1 Output batches

IMGDEC processes one output batch at a time. A batch of `N` words starting at cycle `A` completes at:

`A + 2 × N`

The entire batch enters the output FIFO at that edge. No word from it is available during the intervening cycles. An output header is always a three-word batch and requires three free FIFO slots. A payload batch contains the smallest of:

- the words remaining in the current run;
- the payload words remaining in the current texture or CLUT stage;
- the free output FIFO slots;
- 16 words;
- for a literal run, the input words currently available.

A new batch waits for any required header, token or repeat word. Input starvation therefore adds waiting time rather than a fixed parsing charge. A full output FIFO prevents a new batch; a header also waits if fewer than three slots are free. Draining the output can therefore be necessary for further work. Any next batch is timed from the preceding batch's completion edge. Input or output waiting does not create a backlog that later emits at a faster rate.

### 10.6.2 Installed output-work rate

The installed system timebase is **33,868,800 cycles/s**. At two cycles per word, uninterrupted IMGDEC output work corresponds to:

- **16,934,400 32-bit output words/s**;
- **67,737,600 output bytes/s**.

These are output rates with sufficient input, free FIFO space and uninterrupted operation. Headers consume the same budget as payload. For a complete upload, add DMA transport and any waiting for input or GPU readiness, as in the following example.

### 10.6.3 Controlled ROM-to-RAM transfer

The following timeline starts with both DMA channels already armed for 16-word blocks before START at cycle 0. Input comes from cartridge ROM; output goes to ordinary RAM. Only these two channels and IMGDEC perform work. No CPU instruction, configuration, startup or interrupt-handler time is included.

The stream contains **19 input words** and declares texture count `T = 12` and CLUT count `C = 8`. The output contains 20 payload words plus two three-word headers: **26 output words**. Thus IMGDEC charges `26 × 2 = 52` output-work cycles. The input count includes stream headers and tokens; it is not the decoded count.

For these transfers, each ROM input block costs `4 + 8 × N` cycles and each RAM output block costs `1 + N` cycles. Each new block incurs its setup charge. The 19-word input splits into 16 and 3 words, costing 132 and 28 cycles. The 26-word output splits into 16 and 10 words, costing 17 and 11 cycles. The input and output costs overlap decoding where their request conditions permit.

| Cycle | Event | Input received | Payload decoded | Output queued | Output read |
|---:|---|---:|---:|---:|---:|
| 132 | First 16-word input block ends | 16 | 0 | 0 | 0 |
| 138 | Three-word texture header published | 16 | 0 | 3 | 0 |
| 142 | Two literal payload words published | 16 | 2 | 5 | 0 |
| 146 | Two repeat payload words published | 16 | 4 | 7 | 0 |
| 154 | Four back-reference payload words published | 16 | 8 | 11 | 0 |
| 160 | Three-word input tail ends; input DMA DONE | 19 | 8 | 11 | 0 |
| 162 | Four zero payload words published | 19 | 12 | 15 | 0 |

| Cycle | Event | Input received | Payload decoded | Output queued | Output read |
|---:|---|---:|---:|---:|---:|
| 168 | Three-word CLUT header published; 16-word output block admitted | 19 | 12 | 18 | 0 |
| 184 | Eight literal CLUT payload words published | 19 | 20 | 26 | 0 |
| 185 | First 16-word output block ends | 19 | 20 | 10 | 16 |
| 196 | Ten-word output tail ends; IMGDEC and output DMA DONE | 19 | 20 | 0 | 26 |

The first header is due at `132 + 6 = 138`; subsequent batch charges are 4, 4, 8, 8, 6 and 16 cycles, placing the last output batch at cycle 184. The input tail ends independently at `132 + 28 = 160`. At cycle 162 only 15 output words are queued, below the first 16-word block size. The CLUT header raises that total to 18 at cycle 168, admitting the output block. That block ends at `168 + 17 = 185`; its ten-word tail ends at `185 + 11 = 196`.

The **52 charged output-work cycles** and **196-cycle controlled timeline** describe separate quantities. Input transfer, output transfer and decoding overlap; adding their individual budgets would count overlapping time more than once. The timeline ends with output in RAM. Add CPU setup and result handling; a direct GPU upload also requires GPU processing and PCRTC display timing.

## 10.7 Compressed word stream

IMGDEC consumes a stream of 32-bit words. The stream contains a three-word header followed by tokens and their input data. All lengths in this section count words, not bytes. A decoded payload word is one word produced by a token; it is not necessarily one pixel or one color entry.

### 10.7.1 Stream header

| Input word | Meaning |
|---|---|
| 0 | Magic: exactly `0x31444d49`. |
| 1 | `T`: number of decoded texture payload words. |
| 2 | `C`: number of decoded CLUT payload words. |

The configured input-word count includes these three words, every token, every literal data word, and every repeat-value word. It is separate from both the decoded payload count and the generated output count. There is no end token.

Header words are consumed in order. The decoder can wait between header words for more input. A wrong magic word causes `FORMAT_FAULT` immediately. `T` and `C` are accepted as raw count words; they are not checked against the programmed texture size.

### 10.7.2 Token layouts

Bits 31:30 select the token kind. The remaining bits have the following layouts; every bit shown is part of a field.

| Kind | Bits 31:30 | Bits 29:0 | Following input words |
|---|---|---|---|
| Literal | `00` | Length minus 1 | Exactly `L` literal words. |
| Repeat | `01` | Length minus 1 | One word: the value to repeat. |
| Back-reference | `10` | Bits 29:12: length minus 3; bits 11:0: distance minus 1 | None. |
| Zero | `11` | Length minus 1 | None. |

| Field | Decoding | Encoded field range | Decoded range |
|---|---|---|---|
| Literal, repeat, or zero length `L` | `(token & 0x3fffffff) + 1` | `0x00000000`–`0x3fffffff` | 1–1,073,741,824 words. |
| Back-reference length `L` | `((token >> 12) & 0x3ffff) + 3` | `0x00000`–`0x3ffff` | 3–262,146 words. |
| Back-reference distance `D` | `(token & 0x00000fff) + 1` | `0x000`–`0xfff` | 1–4096 words. |

The right shift in the table extracts bits from the unsigned token word.

A literal token copies its next `L` input words into the decoded payload. A repeat token first consumes its one value word, then emits that value `L` times. A zero token emits `L` words of `0x00000000`. Repeat and zero emission consume no further input words. A missing repeat value is not an implicit zero value: the decoder waits for that word, or faults when the declared input is exhausted.

A token may span the texture/CLUT boundary. Do not insert a new token merely because texture payload ends: any remaining words of the current run continue as CLUT payload after the generated CLUT header.

### 10.7.3 History and overlapping back-references

The history holds the most recent 4096 decoded payload words. A fresh stream starts with zero valid history words; output from a preceding stream is unavailable. Every payload word, including a repeated, zero, or back-referenced word, is added to history. Generated GP0 header words are not added.

A back-reference distance must not exceed the number of currently valid history words. This is checked when its token is consumed, before any word of that back-reference is emitted. Distance 1 selects the most recently decoded payload word; distance 4096 is usable only after history contains 4096 valid words.

For each back-reference word, IMGDEC reads the word `D` positions behind the current history write position, emits it, and advances history. The next word uses the updated position. Consequently, a back-reference can read words emitted earlier by the same run. A distance-1 run repeats the last payload word; a run longer than its distance repeats the corresponding preceding sequence. The history position wraps modulo 4096, and the valid history count remains at 4096 once full.

Texture and CLUT payload share this history. Neither the history nor a partially completed run is reset when IMGDEC inserts the CLUT header.

## 10.8 Generated GP0 output

IMGDEC queues GP0 command and payload words in its output FIFO. Producing these words is not itself a direct VRAM write. The destination and texture-size settings are raw downstream GP0 words: IMGDEC forwards them unchanged. In particular, a destination setting is not an IMGDEC RAM address, and no byte or pixel coordinates are implied by the example destination words below.

### 10.8.1 Packet words

| Packet | Word 0 | Word 1 | Word 2 | Following words |
|---|---|---|---|---|
| Texture; always present | `0xa0000000` | Programmed texture destination word | Programmed texture size word | `T` decoded payload words. |
| CLUT; present only if `C != 0` | `0xa0000000` | Programmed CLUT destination word | `0x00010010` | `C` decoded payload words. |

The command word is the GP0 CPU-to-VRAM command. The CLUT size word is fixed: `0x00010010`, the GP0 4-bit CLUT size word. IMGDEC does not select a different size word from `C`.

Each three-word header is queued together and requires at least three free output FIFO words. Header production does not consume a token and does not increment the decoded payload count. Texture payload ends when that count equals `T`. If `C` is nonzero, the CLUT header follows and decoding continues until the count equals `T + C`, with 32-bit wraparound. If `C` is zero, no CLUT header or payload is generated.

Even when `T` is zero, the texture header is generated. Thus `T = 0, C = 0` produces a three-word texture packet with no IMGDEC payload. If `T = 0, C != 0`, the texture header is followed by the CLUT header and CLUT payload.

### 10.8.2 Three different word counts

| Quantity | What it counts |
|---|---|
| Input-word count | Compressed header, tokens, literal words, and repeat-value words supplied to IMGDEC. |
| Decoded payload count | Only words emitted by literal, repeat, back-reference, and zero runs. |
| Expected output-word count | Decoded payload plus the generated GP0 headers. |

The expected output-word count is calculated as:

`T + 3 + (C == 0 ? 0 : C + 3)`, modulo `2^32`.

The combined CLUT endpoint `T + C`, decoded count increments, and subtraction of decoded count from a stage endpoint also use modulo-`2^32` arithmetic. Count overflow is not saturated or rejected. Do not replace this arithmetic with an assumed nonwrapping image-size calculation.

### 10.8.3 Size words and count mismatches

The downstream GP0 size interpretation is distinct from IMGDEC's `T` and `C` counters:

| GP0 dimension | Size-word interpretation | Range |
|---|---|---|
| Width | `(((size & 0x0000ffff) - 1) & 0x3ff) + 1` | 1–1024. |
| Height | `((((size >> 16) & 0x0000ffff) - 1) & 0x1ff) + 1` | 1–512. |

For example, `0x00060004` describes GP0 dimensions 4 × 6, `0x00010010` describes 16 × 1, and raw size word zero describes 1024 × 512, not an empty transfer. IMGDEC does not use these dimension calculations when accepting its payload counts.

`T` controls IMGDEC's texture payload endpoint, independently of the texture-size word. Likewise, `C` controls the CLUT payload endpoint while the emitted CLUT size word remains fixed. The value 16 in `0x00010010` is not a declaration of 16 compressed-stream payload words. An accepted stream can therefore have payload counts that do not agree with its downstream transfer description. IMGDEC acceptance is not a packet-size consistency check; cartridge software must supply a stream appropriate for the downstream GP0 transfer. No pixel packing, color-channel interpretation, or destination-coordinate interpretation is defined by this compressed-stream format.

## 10.9 Worked stream: 19 input words, 26 output words

Use these programmed words:

| Setting | Value |
|---|---|
| Input-word count | `19` (`0x00000013`) |
| Texture destination | `0x00400040` |
| Texture size | `0x00060004` |
| CLUT destination | `0x00800040` |

The following complete stream uses all four token kinds. `A`, `B`, and `R` below are names for raw words, not pixel values.

| Input index | Word | Meaning |
|---|---|---|
| 0 | `0x31444d49` | Magic. |
| 1 | `0x0000000c` | `T = 12`. |
| 2 | `0x00000008` | `C = 8`. |
| 3 | `0x00000001` | Literal length 2. |
| 4 | `0x11112222` | `A`. |
| 5 | `0x33334444` | `B`. |
| 6 | `0x40000001` | Repeat length 2. |
| 7 | `0x55556666` | Repeat value `R`. |
| 8 | `0x80001003` | Back-reference length 4, distance 4. |
| 9 | `0xc0000003` | Zero length 4. |
| 10 | `0x00000007` | Literal length 8. |
| 11 | `0x00000000` | CLUT payload word 0. |
| 12 | `0x03e0001f` | CLUT payload word 1. |
| 13 | `0x7fff7c00` | CLUT payload word 2. |
| 14 | `0x42104210` | CLUT payload word 3. |
| 15 | `0x001f001f` | CLUT payload word 4. |
| 16 | `0x03e003e0` | CLUT payload word 5. |
| 17 | `0x7c007c00` | CLUT payload word 6. |
| 18 | `0x7fff7fff` | CLUT payload word 7. |

After the first two tokens, history is `A, B, R, R`. Token `0x80001003` has length `1 + 3 = 4` and distance `3 + 1 = 4`, reproducing those four words. The zero run completes the texture payload:

`A, B, R, R, A, B, R, R, 0, 0, 0, 0`.

The final literal supplies the eight CLUT payload words. The input has 3 header words, 5 token words, 10 literal words, and 1 repeat-value word: 19 words. It produces 20 decoded payload words and 6 generated header words: 26 output words.

| Output index | Word | Role |
|---|---|---|
| 0 | `0xa0000000` | Texture command. |
| 1 | `0x00400040` | Texture destination. |
| 2 | `0x00060004` | Texture size. |
| 3 | `0x11112222` | Texture payload 0. |
| 4 | `0x33334444` | Texture payload 1. |
| 5 | `0x55556666` | Texture payload 2. |
| 6 | `0x55556666` | Texture payload 3. |
| 7 | `0x11112222` | Texture payload 4. |
| 8 | `0x33334444` | Texture payload 5. |
| 9 | `0x55556666` | Texture payload 6. |
| 10 | `0x55556666` | Texture payload 7. |
| 11 | `0x00000000` | Texture payload 8. |
| 12 | `0x00000000` | Texture payload 9. |
| 13 | `0x00000000` | Texture payload 10. |
| 14 | `0x00000000` | Texture payload 11. |
| 15 | `0xa0000000` | CLUT command. |

| Output index | Word | Role |
|---|---|---|
| 16 | `0x00800040` | CLUT destination. |
| 17 | `0x00010010` | Fixed CLUT size. |
| 18 | `0x00000000` | CLUT payload 0. |
| 19 | `0x03e0001f` | CLUT payload 1. |
| 20 | `0x7fff7c00` | CLUT payload 2. |
| 21 | `0x42104210` | CLUT payload 3. |
| 22 | `0x001f001f` | CLUT payload 4. |
| 23 | `0x03e003e0` | CLUT payload 5. |
| 24 | `0x7c007c00` | CLUT payload 6. |
| 25 | `0x7fff7fff` | CLUT payload 7. |

## 10.10 Worked stream: overlapping history across the CLUT header

Program input count 10, texture destination `0x00200040`, texture size `0x00010010`, and CLUT destination `0x00300060`.

| Input index | Word | Meaning |
|---|---|---|
| 0 | `0x31444d49` | Magic. |
| 1 | `0x00000008` | `T = 8`. |
| 2 | `0x00000008` | `C = 8`. |
| 3 | `0x00000001` | Literal length 2. |
| 4 | `0x11223344` | `A`. |
| 5 | `0xaabbccdd` | `B`. |
| 6 | `0x40000001` | Repeat length 2. |
| 7 | `0x55667788` | Repeat value `R`. |
| 8 | `0xc0000000` | Zero length 1. |
| 9 | `0x80008004` | Back-reference length 11, distance 5. |

Before the back-reference, history is `A, B, R, R, 0`. The token's length is `8 + 3 = 11` and its distance is `4 + 1 = 5`. Its first three words complete texture payload. Its remaining eight words follow the CLUT header, without another token.

| Decoded index | Payload word | Source | Stage |
|---|---|---|---|
| 0 | `A` | Literal. | Texture. |
| 1 | `B` | Literal. | Texture. |
| 2 | `R` | Repeat. | Texture. |
| 3 | `R` | Repeat. | Texture. |
| 4 | `0` | Zero. | Texture. |
| 5 | `A` | Back-reference to decoded index 0. | Texture. |
| 6 | `B` | Back-reference to decoded index 1. | Texture. |
| 7 | `R` | Back-reference to decoded index 2. | Texture. |
| 8 | `R` | Back-reference to decoded index 3. | CLUT. |
| 9 | `0` | Back-reference to decoded index 4. | CLUT. |
| 10 | `A` | Back-reference to decoded index 5, emitted by this run. | CLUT. |
| 11 | `B` | Back-reference to decoded index 6, emitted by this run. | CLUT. |
| 12 | `R` | Back-reference to decoded index 7, emitted by this run. | CLUT. |
| 13 | `R` | Back-reference to decoded index 8, emitted by this run. | CLUT. |

| Decoded index | Payload word | Source | Stage |
|---|---|---|---|
| 14 | `0` | Back-reference to decoded index 9, emitted by this run. | CLUT. |
| 15 | `A` | Back-reference to decoded index 10, emitted by this run. | CLUT. |

The generated output is 22 words:

| Output indices | Words, in order |
|---|---|
| 0–2 | `0xa0000000`, `0x00200040`, `0x00010010` |
| 3–10 | `0x11223344`, `0xaabbccdd`, `0x55667788`, `0x55667788`, `0x00000000`, `0x11223344`, `0xaabbccdd`, `0x55667788` |
| 11–13 | `0xa0000000`, `0x00300060`, `0x00010010` |
| 14–21 | `0x55667788`, `0x00000000`, `0x11223344`, `0xaabbccdd`, `0x55667788`, `0x55667788`, `0x00000000`, `0x11223344` |

The inserted three-word header does not increase the back-reference distance: distance counts decoded payload history, not output FIFO positions.

## 10.11 Completion, malformed input, and partial output

### 10.11.1 Exact end of stream

A stream is complete only when all four conditions hold:

1. Both required payload stages are complete.
2. The decoder is between tokens, with no unfinished run or pending repeat value.
3. The input words received equal the configured input-word count.
4. The input FIFO is empty.

Finishing the requested payload is therefore insufficient if input remains or the last run has not ended exactly. Conversely, an empty-payload stream requires no token.

Stream completion is also distinct from `DONE`: queued output must be read. The DATA-read completion check requires the output-words-read count to equal the expected output-word count as well as the complete-stream conditions above. A DMA read before the end of its block does not perform that check. Merely producing the last output word does not invoke `DONE`.

### 10.11.2 Waiting versus exhaustion

An empty input FIFO alone is not an exhaustion fault. When the decoder requires more input, exhaustion is recognized by equality between received input words and the configured input-word count. If they are not equal, this check does not declare exhaustion. It is an equality test, not a greater-than-or-equal test; counters use modulo-`2^32` arithmetic.

The decoder may wait for a missing header word, token, repeat value, or literal data. Once a repeat, zero, or back-reference run is prepared, its emission needs no further compressed input. A literal can emit an available prefix before it needs to wait or fault for more literal words.

### 10.11.3 Ordering and output backpressure

After header parsing, IMGDEC advances any finished payload stage before selecting further work. It then observes output space in this order:

1. If the output FIFO is full, it returns before inspecting a malformed tail.
2. If a generated header is pending, it waits for three free words, then emits that header without preparing a token.
3. If all output stages are complete, it checks the exact completion conditions. Queued input or an unfinished token/run causes `FORMAT_FAULT`. If the input FIFO is empty and the decoder is between tokens but the received count has not reached the configured count, it waits for the remaining input rather than declaring exhaustion.

4. Otherwise, it prepares or continues payload work.

A payload group emits at most 16 words. Its length is also limited by the current run's remaining length, the current payload stage's remaining count, and free output FIFO space. A literal group is additionally limited by the available input words. When no payload word can be selected, the input-exhaustion check is invoked. Only consuming the entire current run returns the decoder to the between-tokens state.

These rules have two important consequences. An invalid first payload token can fault after the texture header has already been queued. An overlong final run is not rejected before output: it emits up to the payload endpoint, then faults because the run is unfinished. Output backpressure can postpone the malformed-tail check.

### 10.11.4 Fault causes

| Condition | Point of detection | Partial effect |
|---|---|---|
| Magic is not `0x31444d49`. | On consuming the first header word. | No generated header or payload from this stream has been emitted. |
| Declared input ends within the three-word header. | After available header words have been consumed and another is required. | No generated header or payload has been emitted. |
| Declared input ends while a token is required. | On finding no token word available. | Earlier headers and payload remain. |
| Declared input ends before a repeat value. | After consuming its repeat token, before emitting that run. | Earlier output remains; that repeat run emits no value word. |
| Declared input ends within a literal. | After any available literal prefix has been emitted. | That prefix and earlier output remain. |
| Back-reference distance exceeds valid history. | On consuming the back-reference token. | No word of that back-reference is emitted; earlier output remains. |
| A run extends beyond the final payload endpoint. | After emitting up to that endpoint, when the completed-output stage finds an unfinished run. | Payload up to the endpoint remains, not the excess requested words. |
| Input FIFO is nonempty after all required payload is complete. | At the completed-output stage, after the output-space check. | Earlier headers and payload remain; a trailing token need not be consumed. |

A fault terminates decoding without rolling back the output. Queued output remains readable through DATA after stop, and decoded/input counters and the current run/history state are retained. Actual output pops still update the DATA latch and output-words-read count; an empty output FIFO leaves the latch unchanged. CPU reads while DMA owns the read port return the latch without popping output.

### 10.11.5 Compact fault and empty-stream examples

For each row, start a fresh stream, configure the input count to the exact listed length, supply all listed input, and allow normal decoding with adequate FIFO space. In the three malformed cases, wait for the fault before reading the retained output. In the valid empty-payload case, read the three known output words to obtain DONE; do not wait for a fault. `D` and `S` denote the programmed raw texture destination and size words.

| Input words, in order | Result |
|---|---|
| `0x31444d49`, `0x00000003`, `0x00000000`, `0x80000000` | The first back-reference has length 3 and distance 1, but history is empty. `FORMAT_FAULT`; decoded count 0. Three output words remain: `0xa0000000`, `D`, `S`. |
| `0x31444d49`, `0x00000002`, `0x00000000`, `0xc0000002` | Zero run length 3 exceeds final target 2. Two zeros are emitted, leaving one run word outstanding. `FORMAT_FAULT`; decoded count 2. Five output words remain: `0xa0000000`, `D`, `S`, `0x00000000`, `0x00000000`. |
| `0x31444d49`, `0x00000002`, `0x00000000`, `0x00000001`, `0x12345678` | Literal length 2 has only one data word. That word is emitted before exhaustion faults. Decoded count 1; four output words remain: `0xa0000000`, `D`, `S`, `0x12345678`. |
| `0x31444d49`, `0x00000000`, `0x00000000` | Valid empty-payload stream. Its only output is `0xa0000000`, `D`, `S`. No token is required; `DONE` still requires output-read completion. |

## 10.12 Use the output packets

### 10.12.1 Forward the same packets to GP0

Own GP0 as well as both DMA channels. There must be no partial packet or unread readback. Before starting, wait for GPUSTAT bit 26 (`04000000H`) at `08010240H`, write `e6000000H` to GP0 to disable both mask-protection modes, and write `04000002H` to GP1 to select CPU_TO_GP0 DMA direction. Wait for GPU idle again before arming the transfer.

Repeat the configuration and start sequence above, changing only these output operands:

| Register | RAM receipt | Direct upload |
|---|---|---|
| DMA1 WRITE_ADDR, `080103e8H` | `ram_output` | `0801023cH` (GP0) |
| DMA1 CONTROL, `080103f0H` | `00003c1aH` | `00003c58H` |

The direct control word selects fixed DATA and GP0 addresses, IMGDEC_READ and GX_WRITE requests, and sixteen-word blocks. Keep count 26. It waits for both request lines; decoder output readiness alone cannot make GP0 ready. After both DMA completions and IMGDEC DONE, wait separately for GPU idle. GPU processing continues after the transfer; PCRTC displays the completed source according to its own latch timing.

### 10.12.2 Inspect a fault, then start a fresh stream

Use CPU input with neither DMA channel armed. Leave the three destination/size words from the worked stream in place, and configure only the new input count. This first back-reference requests history that does not yet exist.

```
bss fault_header: word[3]
rodata fault_stream: word[4] = { 0x31444d49, 3, 0, 0x80000000 }
decoder[0] = 4
decoder[4] = 1
for index = 0, 3 do
    decoder[6] = fault_stream[index]
end
while (decoder[5] & 1) ~= 0 do
end
for index = 0, 2 do
    fault_header[index] = decoder[6]
end
*irq_ack = 0x00000080
```

STATUS is `10H` (FORMAT_FAULT), received count is 4 and decoded count is zero. The three retained output words are `a0000000H`, `00400040H`, `00060004H`. They are a generated header, not a complete GPU packet: do not forward this prefix to GP0. A decoder fault neither cancels an admitted DMA block nor removes GP0 words already accepted downstream.

A fresh START discards the faulted stream state. A minimum valid input is `31444d49H, 0, 0`: three header words with no token. Set INPUT_WORD_COUNT to 3, TEXTURE_DESTINATION to `00200020H` and TEXTURE_SIZE to zero, then START and supply those three words. Wait for OUTPUT_REQUEST and read exactly three DATA words into RAM. They are `a0000000H, 00200020H, 0`; their final read completes the stream with STATUS=`02H`, received count 3 and decoded count zero. Acknowledge bit `80H` separately. Keep this output in RAM: raw GP0 size zero means 1024 × 512, not an empty rectangle, so the header alone is not a complete GPU upload.

### 10.12.3 Display the uploaded indexed patch

For the pictured result, use the successful direct-to-GP0 upload, not either fault/recovery prefix. Before that upload, prepare this disjoint framebuffer window with ordered word writes. CSR RESET supplies zero texture window and drawing offset; the following GP0 words select drawing state and fill the background. Wait for that fill to reach GPU idle before arming the upload.

| Order | Port address | Words written in order |
|---|---|---|
| 1 | PCRTC CSR, `080103d4H` | `00000200H` |
| 2 | PCRTC PMODE, `08010354H` | `00000000H` (disable output) |
| 3 | PCRTC DISPFB1, `0801035cH` | `000fa000H` |
| 4 | PCRTC source origin, `08010360H` | `00000100H` |
| 5 | PCRTC display extent, `08010368H` | `000bf3ffH` |
| 6 | GP0, `0801023cH` | `e1000400H`, `e3000100H`, `e402fdffH`, `02181008H`, `00000100H`, `00c00100H` |

The 256 × 192 source window begins at VRAM X=256. It is disjoint from the texture at (64,64) and CLUT at (64,128). After the upload and GPU-idle wait, send this purple underlay and palette4 textured quad:

```
rodata picture_commands: word[12] = {
    0x60602040, 320 | (72 << 16), 120 | (40 << 16),
    0x2d000000,
    320 | (72 << 16), 0x20044000,
    440 | (72 << 16), 0x0001400f,
    320 | (112 << 16), 0x00004500,
    440 | (112 << 16), 0x0000450f,
}
local gp0<const>: *word = 0x0801023c
local gp1<const>: *word = 0x08010240
local pmode<const>: *word = 0x08010354
for index = 0, #picture_commands - 1 do
    *gp0 = picture_commands[index]
end
*gp1 = 0x02000000
*irq_ack = 0x00000040
*gp0 = 0x1f000000
while (*gp1 & 0x01000000) == 0 do
end
while (*gp1 & 0x04000000) == 0 do
end
*gp1 = 0x02000000
*irq_ack = 0x00000040
*pmode = 0x0000ff21
```

The marker, GPU idle and PCRTC display latch are separate boundaries. The PMODE write enables the prepared composition for display latching; it is not itself a displayed-frame acknowledgement.

### 10.12.4 Read back the uploaded words

With exclusive GP0 ownership and no DMA read owner, send each following header as three ordered GP0 writes. Wait for GPUSTAT bit 27 (`08000000H`) before reading the transaction's data at `0801023cH`. Drain its full count before sending the next header.

| Storage | GP0 header words, in order | Expected DATA reads |
|---|---|---|
| Texture | `c0000000H`, `00400040H`, `00060004H` | 12 words, equal to `ram_output[3..14]` |
| CLUT | `c0000000H`, `00800040H`, `00010010H` | 8 words, equal to `ram_output[18..25]` |

Each returned word contains two raw storage halfwords, first in bits 15:0. The last read clears READY. Readback returns the stored halfwords independently of display selection. GPU idle alone reports no pixel values. See GPU for complete upload, drawing and readback rules.

### 10.12.5 From compressed words to a displayed patch



Figure 10.2: The decoded indexed patch over a purple underlay, surrounded by a dark-blue background.

The six source rows make the token kinds visible:

| Source Y (V) | Token kind | 16 nibble indices | Output colour meaning, left to right |
|---|---|---|---|
| 64 | Literal | `2222111144443333` | Red, purple, blue, green |
| 65 | Repeat | `6666555566665555` | Gray, white, gray, white |
| 66 | Back-reference | `2222111144443333` | Red, purple, blue, green |
| 67 | Back-reference | `6666555566665555` | Gray, white, gray, white |
| 68 | Zero | `0000000000000000` | Purple throughout |
| 69 | Zero | `0000000000000000` | Purple throughout |

Indices `2`–`6` select red, green, blue, white and gray. CLUT entries `0` and `1` both contain `0000H`; these zero texels leave the purple `3088H` underlay unchanged, rather than writing black.

IMGDEC supplies **4 × 6 halfwords** of texture plus the CLUT upload. Palette4 reads four low-nibble-first indices per halfword: **16 × 6 texels**. The GPU draws the **120 × 40** patch; PCRTC maps it to screen **(64,72)**.

For UV endpoints `(0,64)` through `(15,69)`, the outermost columns and rows occupy four displayed pixels each; interior columns and rows occupy eight. The scaling is not uniformly eightfold.

# 11 Input controller

## 11.1 Function and resources

The input controller samples a keyboard, a pointing device and four game pads. A program requests one sample with ARM. At the next VBlank-begin edge, the controller copies the control states into its registers and advances the sample sequence. The program can then read a stable set of key, button and axis values while it updates the game. Each subsequent sample requires another ARM.

Sampling follows the display timing, so a game can acquire one set of controls per display interval without tying input acquisition to the duration of its update routine. Register reads leave the captured values unchanged.

The controller provides eight keyboard words, a pointer button word with three pointer data words, and four pad records. Each pad record contains a button word, two pairs of stick-axis words and two trigger words. A separate output port accepts vibration requests for a selected pad; its capability word is sampled with the input data.

The register block occupies 47 consecutive 32-bit words, from byte address `08000064H` through `0800011CH`. Use aligned, whole-word accesses.

| Resource | Capacity or condition |
|---|---|
| Register word | 32 bits |
| Mapped register block | 47 words, 188 bytes |
| Sampled data | 41 words, 164 bytes: 8 keyboard + 4 pointer + 28 pad + 1 capability |
| Keyboard bitmap | 256 HID usage bits |
| Pad records | 4 pads; 7 words, 28 bytes per pad |
| Sample sequence | 32 bits, incrementing modulo $2^{32}$ |
| Sample publication | A pending ARM request accepted at VBlank begin |

The sampling interval follows the programmed PCRTC timing. Interrupt controller describes the VBlank interrupt used to wait for this edge.

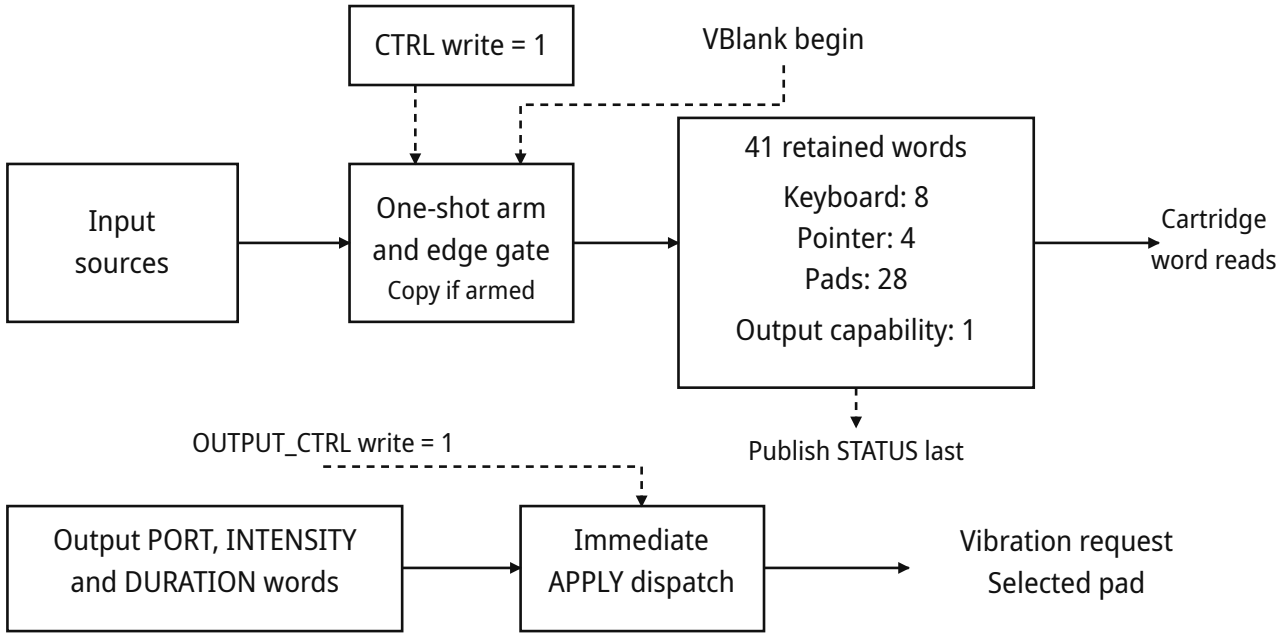



Figure 11.1: Armed sample latch and independent output request

The capture path transfers control states into the snapshot registers. The separate output path sends vibration parameters to the selected pad when APPLY is written. Solid arrows carry data; dashed arrows show the control sequence. The capability register belongs to the captured input data.

## 11.2 Sampling and snapshot reads

The sampling path is one-shot. Write the complete word `00000001H` (ARM) to `IO_INP_CTRL` to leave one sample request pending. At an accepted VBlank-begin edge, the controller latches the keyboard, pointer, all four pad records and output-capability word as one snapshot. It then increments `IO_INP_STATUS` modulo $2^{32}$. The accepted sample clears the pending request; it does not clear the retained ARM word in `IO_INP_CTRL`.

ARM chooses the next display edge at which to take a sample. STATUS tells the program when that choice has taken effect. Until the sequence changes, reading a key or axis can still return the preceding sample. Once it changes, the program can read several control words from the same captured state without another display edge changing them: the request has already been consumed.

Use this order to obtain a new snapshot:

1. Read and retain `IO_INP_STATUS`.
2. Write ARM to `IO_INP_CTRL`.
3. Read `IO_INP_STATUS` until it differs from the retained value.
4. Read the required snapshot words before requesting another sample.

Use the sequence change as the completion indication; CTRL retains its last written command rather than reporting a pending request. The captured words remain stable until the next accepted sample or reset. To detect button edges, compare successive snapshots as described under Retained levels and software edges. A press and release between samples is lost.

Repeated ARM writes before an accepted sample leave one pending request, not several. An unarmed VBlank does not latch a snapshot or advance `IO_INP_STATUS`. All sampled register words retain their raw bits; the latch does not clear unnamed bits or clamp sampled data.

## 11.3 Sample input in BLua

The following fragment obtains KeyA and pad 0's left-stick X from one capture. It assumes a running beam, ordinary BIOS cartridge entry and exclusive use of INPUT. It polls the sequence without changing the interrupt configuration.

```
local control<const>: *word = 0x08000064
local status<const>: *word = 0x08000068
local keys<const>: *word = 0x0800006c
local pad0<const>: *word = 0x0800009c

local before<const> = *status
*control = 1
while *status == before do
end
local key_a_down<const> = (keys[0] & 0x00000010) ~= 0
local left_x_word<const> = pad0[1]
```

KeyA is bit 4 of the first keyboard word. The next word after pad 0's button word is its left X axis. For a captured keyboard word of `00000010H` and an axis word of `00008000H`, `key_a_down` is true and `left_x_word` represents +0.5, halfway from the stick's center to its right endpoint. The bit assignments and signed 16.16 interpretation are specified in the next section.

Read any other required snapshot words before the next ARM. With exclusive use of INPUT, all those reads refer to the same capture. Retaining the current STATUS as the baseline accommodates samples taken during BIOS boot; it need not begin at zero.

Polling is not required. A program with an enabled VBlank handler can wait with HALT, then recheck the same sequence predicate after each wake-up. VBlank provides the opportunity to capture; only a sequence change establishes that a new sample was accepted. See VBlank ordering, interrupts and access faults for interrupt setup requirements and publication order.

## 11.4 Raw input representations

Keyboard and button words are bitmaps; stick, trigger and pointer data words are signed 16.16 numbers. The following fields describe the meaning and units of the captured state. Their register addresses follow in the register map.

### 11.4.1 Keyboard usage bitmap

The keyboard occupies eight words: 256 bits indexed by USB HID keyboard/keypad usage on page `07H`. A set bit means that usage is pressed. The bitmap identifies key usages; this table does not define character or text-composition input.

For usage `u`, select word `u >> 5` and bit `u & 31`. Its word is at the keyboard bitmap base plus `4 × (u >> 5)` bytes. The mask within that word is `1 << (u & 31)`.

For example, `KeyA` is usage `04H`: word 0, bit 4, mask `00000010H`. This is not pad A. Pad A is bit 0 of a pad's buttons word, with mask `00000001H`.

The following table gives the keyboard usage assignments. Usage IDs are decimal.

| Key names | Usage IDs |
|---|---|
| `KeyA` … `KeyZ` | 4 … 29, consecutive |
| `Digit1` … `Digit9`, `Digit0` | 30 … 38, 39 |
| `Enter`, `Escape`, `Backspace`, `Tab`, `Space` | 40, 41, 42, 43, 44 |
| `Minus`, `Equal`, `BracketLeft`, `BracketRight`, `Backslash` | 45, 46, 47, 48, 49 |
| `Semicolon`, `Quote`, `Backquote`, `Comma`, `Period`, `Slash` | 51, 52, 53, 54, 55, 56 |
| `CapsLock` | 57 |
| `F1` … `F12` | 58 … 69, consecutive |
| `PrintScreen`, `ScrollLock`, `Pause` | 70, 71, 72 |
| `Insert`, `Home`, `PageUp`, `Delete`, `End`, `PageDown` | 73, 74, 75, 76, 77, 78 |
| `ArrowRight`, `ArrowLeft`, `ArrowDown`, `ArrowUp` | 79, 80, 81, 82 |
| `NumLock`, `NumpadDivide`, `NumpadMultiply`, `NumpadSubtract`, `NumpadAdd`, `NumpadEnter` | 83, 84, 85, 86, 87, 88 |
| `Numpad1` … `Numpad9`, `Numpad0` | 89 … 97, 98 |
| `NumpadDecimal`, `IntlBackslash`, `ContextMenu`, `Power`, `NumpadEqual` | 99, 100, 101, 102, 103 |
| `F13` … `F24` | 104 … 115, consecutive |
| `IntlRo`, `IntlYen` | 135, 137 |
| `Lang1`, `Lang2` | 144, 145 |
| `ControlLeft`, `ShiftLeft`, `AltLeft`, `MetaLeft` | 224, 225, 226, 227 |
| `ControlRight`, `ShiftRight`, `AltRight`, `MetaRight` | 228, 229, 230, 231 |

### 11.4.2 Pad records

There are four raw pad records, indexed 0 through 3. Each record occupies 28 bytes: one buttons word followed by six signed 16.16 axis words. Offsets below are bytes from that pad's base.

| Offset | Word | Axis index |
|---|---|---|
| `+00H` | Buttons | — |
| `+04H` | Left X | 0 |
| `+08H` | Left Y | 1 |
| `+0CH` | Right X | 2 |
| `+10H` | Right Y | 3 |
| `+14H` | Left trigger | 4 |
| `+18H` | Right trigger | 5 |

The pad button assignments are:

| Bit | Button | Bit | Button |
|---|---|---|---|
| 0 | A | 9 | Start |
| 1 | B | 10 | Left stick |
| 2 | X | 11 | Right stick |
| 3 | Y | 12 | Up |
| 4 | Left bumper | 13 | Down |
| 5 | Right bumper | 14 | Left |
| 6 | Left trigger | 15 | Right |
| 7 | Right trigger | 16 | Home |
| 8 | Select | 17 | Touchpad |

Stick axes use −1 at the left or upper endpoint, zero at center, and +1 at the right or lower endpoint. Trigger travel uses zero at release and one at full travel. Digital trigger attachments supply only zero or one; analog attachments also supply intermediate values. Travel updates independently of the trigger's button bit, including movement while the button remains pressed. The button threshold belongs to the attachment, not the controller latch. Neutral words alone do not identify whether a pad is connected.

### 11.4.3 Pointer words

The pointer has a buttons word, X, Y and wheel, in that order. Its button positions are:

| Bit | Button |
|---|---|
| 0 | Primary |
| 1 | Aux |
| 2 | Secondary |
| 3 | Back |
| 4 | Forward |

X and Y are retained signed 16.16 positions in the displayed image's raster coordinate system, not movement deltas or physical screen coordinates. The upper-left pixel is (0,0); X increases rightward and Y downward. Standard pointing attachments map the image to its displayed extent and clip position to `0..width−1` and `0..height−1`. Changing the display mode does not itself clear the retained controller snapshot.

The wheel word is a signed 16.16 delta in logical scroll steps, positive downward and negative upward. One step is `00010000H`; a half step is `00008000H`. It describes the most recent attachment update, not an absolute wheel position or an accumulation since ARM. A subsequent stationary update supplies zero. Reading the retained controller word does not consume or clear it, so skipped controller samples can miss movement.

### 11.4.4 Signed 16.16 values

Interpret a signed 16.16 word as a signed 32-bit integer divided by 65536.

| Raw word | Numeric value |
|---|---|
| `00000000H` | 0 |
| `00010000H` | +1 |
| `FFFF0000H` | −1 |

The representation has a quantum of `1/65536` and a range of −32768 through `32767 + 65535/65536`. Stick and trigger operating ranges are given above; pointer positions use raster coordinates. The controller retains the attachment's words unchanged. Apply any additional calibration or dead zone in the program.

### 11.4.5 Retained levels and software edges

Keyboard and button words describe sampled levels. They are not an event queue. A cartridge can compare a current mask with its own previous mask:

- pressed: `current & ~previous`;
- released: `previous & ~current`.

These expressions detect transitions between the samples the cartridge actually observes. They cannot recover a press and release that both occurred between those samples. A pointer movement delta likewise requires subtraction of retained positions; X and Y themselves are not movement-delta registers.

### 11.4.6 Worked KeyA transition sequence

The KeyA mask is `00000010H`, not pad A's `00000001H`. Apply the pressed and released expressions above to this mask, keeping the current mask as `previous` for the next sample. With `previous` initially zero, four completed samples give:

| Sample | KeyA level | Pressed mask | Released mask | Cumulative presses | Cumulative releases |
|---|---|---|---|---|---|
| 1 | `0` | `0` | `0` | 0 | 0 |
| 2 | `10H` | `10H` | `0` | 1 | 0 |
| 3 | `10H` | `0` | `0` | 1 | 0 |
| 4 | `0` | `0` | `10H` | 1 | 1 |

The cumulative columns show the counts a program obtains from the edge masks. An unchanged released sample has both masks zero.

## 11.5 Register map

All addresses below are byte addresses. `R` denotes a read-only word; `R/W` denotes a readable word with write behavior. Command words must be written in full, rather than treated as independently writable control bits. Every listed word has reset value `00000000H`.

### 11.5.1 Sampling control

| Address | Register | Access | Bits and use | Reset |
|---|---|---|---|---|
| `08000064H` | `IO_INP_CTRL` | R/W | 31:0: exact command word; ARM = `00000001H`, RESET = `00000002H` | `00000000H` |
| `08000068H` | `IO_INP_STATUS` | R | 31:0: accepted-sample sequence, modulo $2^{32}$ | `00000000H` |

### 11.5.2 Keyboard and pointer snapshot

| Address | Register | Access | Bits and use | Reset |
|---|---|---|---|---|
| `0800006CH` to `08000088H`, stride 4 | `IO_INP_KEYS[0..7]` | R | 31:0 of each word: keyboard bitmap | All `00000000H` |
| `0800008CH` | `IO_INP_POINTER_BUTTONS` | R | 31:0: pointer button word | `00000000H` |
| `08000090H` | `IO_INP_POINTER_X` | R | 31:0: pointer X word | `00000000H` |
| `08000094H` | `IO_INP_POINTER_Y` | R | 31:0: pointer Y word | `00000000H` |
| `08000098H` | `IO_INP_POINTER_WHEEL` | R | 31:0: pointer wheel word | `00000000H` |

### 11.5.3 Pad snapshot records

IO_INP_PADS contains four records. Each record occupies seven words, or 1CH bytes.

| Pad index | Record base |
|---|---|
| 0 | 0800009CH |
| 1 | 080000B8H |
| 2 | 080000D4H |
| 3 | 080000F0H |

The following offsets apply to each record base.

| Offset | Register field | Access | Bits and use | Reset |
|---|---|---|---|---|
| +00H | Buttons | R | 31:0: pad button word | 00000000H |
| +04H | Left X | R | 31:0: left-stick X word | 00000000H |
| +08H | Left Y | R | 31:0: left-stick Y word | 00000000H |
| +0CH | Right X | R | 31:0: right-stick X word | 00000000H |
| +10H | Right Y | R | 31:0: right-stick Y word | 00000000H |
| +14H | Left trigger | R | 31:0: left-trigger word | 00000000H |
| +18H | Right trigger | R | 31:0: right-trigger word | 00000000H |

### 11.5.4 Pad output port

| Address | Register | Access | Bits and use | Reset |
|---|---|---|---|---|
| 0800010CH | IO_INP_OUTPUT_PORT | R/W | 1:0: selected pad index; 31:2 retained | 00000000H |
| 08000110H | IO_INP_OUTPUT_INTENSITY_Q16 | R/W | 31:0: raw intensity word | 00000000H |
| 08000114H | IO_INP_OUTPUT_DURATION_MS | R/W | 31:0: unsigned duration in milliseconds | 00000000H |
| 08000118H | IO_INP_OUTPUT_STATUS | R | 3:0: sampled vibration-capability bits for pads 3 through 0; 31:4 retain supplied bits | 00000000H |
| 0800011CH | IO_INP_OUTPUT_CTRL | R/W | 31:0: exact command word; APPLY = 00000001H; reads zero after a command write | 00000000H |

## 11.6 Control words and reset

IO_INP_CTRL recognizes complete words, not combinable flags.

| Written word | Effect |
|---|---|
| 00000000H | Retains zero in CTRL; does not arm, reset or cancel a pending request |
| 00000001H | Arms one sample; leaves CTRL equal to 00000001H |
| 00000002H | Resets the sampling register file and sequence; CTRL becomes zero |
| 00000003H | Retains 00000003H; does not combine ARM and RESET and does not cancel a pending request |
| Any other word | Retains the complete word; leaves the pending request, sequence and other register state unchanged |

RESET takes effect on the control write. It clears any pending sample, sets `IO_INP_STATUS` to zero and zeroes the keyboard, pointer and pad words. It also zeroes the output port, intensity, duration and capability words. A console input-controller reset additionally sets `IO_INP_OUTPUT_CTRL` to zero.

The RESET command does not clear the previously sampled supervisor-request level. Holding that wire high across this command therefore does not create a new rising edge. A whole-unit reset initializes its edge history to low.

An accepted sample does not replace the output parameter words. ARM itself does not sample immediately, increment the sequence or clear the existing snapshot. Unrecognized control words cause no input-specific fault.

## 11.7 VBlank ordering, interrupts and access faults

At VBlank begin, the display controller presents the frame, INPUT captures any armed sample, and the central controller raises `IRQ_VBLANK` (`00000004H`), in that order. The interrupt handler can therefore read the new snapshot and STATUS sequence. Handler entry depends on the CPU interrupt-enable state and other pending work.

Independently of ARM, the controller also observes the dedicated supervisor-request wire at each VBlank. A low-to-high transition is forwarded to SYSTEM; holding the wire high does not generate another transition. This wire is not a keyboard usage or a pad bit, and does not require a cartridge sample. See SYSTEM, Privilege and supervisor entry.

There is no separate input-complete interrupt. VBlank pending state is coalesced, not a count of completed samples, so test the sample sequence even when using VBlank interrupts. A cartridge that waits with HALT must first enable an owned VBlank interrupt route, and must recheck `IO_INP_STATUS` after waking. An interrupt source that is merely pending but masked does not guarantee a wakeup.

The snapshot words, `IO_INP_STATUS` and `IO_INP_OUTPUT_STATUS` are read-only. A mapped whole-word write to one of these words raises the shared bus fault `BUS_FAULT_READ_ONLY` (code 3), without changing the word. The controller defines no separate input fault register. General alignment and access-size rules are those of the memory interface.

### 11.7.1 BIOS handoff and supervisor sampling

The BIOS samples this controller during boot and does not reset INPUT when handing control to the cartridge. The register table's reset words are not a cartridge-entry snapshot. Retain the **current** STATUS as the baseline for the sampling sequence above; do not require an initial sequence of zero.

INPUT is not context-banked. Cartridge and supervisor share its pending ARM request, accepted-sample sequence and register file. The BIOS monitor arms captures while it runs: each accepted capture replaces the shared snapshot and advances the same sequence. LEAVE does not restore the cartridge's earlier snapshot or sequence. After resuming, request and observe a new capture before treating those words as a fresh cartridge sample. See SYSTEM, Device boundaries during supervisor service, for the other units' preservation rules.

## 11.8 Pad output commands

The output port uses three retained parameter words and one command word. Write the selected pad to `IO_INP_OUTPUT_PORT`, the requested intensity to `IO_INP_OUTPUT_INTENSITY_Q16`, and the requested duration to `IO_INP_OUTPUT_DURATION_MS`. These parameter writes alone do not request output. Write the complete word `00000001H` (APPLY) to `IO_INP_OUTPUT_CTRL` after preparing the parameters.

APPLY uses the parameters present at that write. The selected pad is `IO_INP_OUTPUT_PORT & 3`, so only the low two bits select pads 0 through 3. The upper bits remain stored and readable; they do not select additional pads.

The command is dispatched on the APPLY write, without waiting for ARM or a VBlank edge. There is no output command queue. Each exact APPLY write issues a request with the current parameters. APPLY does not alter those parameters or the sampled capability word.

`IO_INP_OUTPUT_CTRL` tests the entire command word. A word such as `00000003H`, `00000101H` or `80000001H` is not APPLY, even though bit 0 is set. Other command words issue no request. After a command write completes, the control word reads zero; that zero is not an output-completion indication.

### 11.8.1 Intensity and duration

The normal intensity interval is `00000000H` through `00010000H`, inclusive. Intensity is the raw word divided by 65536: zero at `00000000H`, one-half at `00008000H`, and maximum at `00010000H`, with a step of 1/65536. The actuator's physical response depends on the attached pad.

The intensity register retains all 32 bits, and APPLY decodes the full unsigned U16.16 word exactly: `FFFFFFFFH` requests `65535 + 65535/65536`. The output attachment saturates requests above one to its maximum intensity. This saturation does not rewrite the programmed word. Unsupported output attachments produce no effect.

`IO_INP_OUTPUT_DURATION_MS` holds the requested duration as an unsigned 32-bit millisecond count. The complete word is retained unchanged throughout the effect; readback gives the programmed duration, not remaining time.

For example, the following ordered writes request pad 0 at half intensity for 100 milliseconds. The last write is the command; the preceding writes only prepare its operands.

| Order | Register | Written word |
|---|---|---|
| 1 | `IO_INP_OUTPUT_PORT` | `00000000H` |
| 2 | `IO_INP_OUTPUT_INTENSITY_Q16` | `00008000H` |
| 3 | `IO_INP_OUTPUT_DURATION_MS` | `00000064H` |
| 4 | `IO_INP_OUTPUT_CTRL` | `00000001H` |

### 11.8.2 Capability and completion

`IO_INP_OUTPUT_STATUS` contains sampled vibration-capability bits, one per pad in bits 0 through 3. It is refreshed with the other input words when an armed sample is accepted. Until then it retains the previous snapshot's value; after reset it is zero until a new sample is accepted. The complete supplied word is retained, including bits above bit 3.

Use the capability bits to identify pads supporting vibration. They retain their sampled values when APPLY is written. The output port supplies no busy, completion or error indication and raises no output-complete interrupt.

Input RESET zeroes the output parameters and capability word without issuing a stop or cancellation command for an earlier request. Do not use a register reset as an output-stop operation.

# 12 System control and timing

The system controller maintains console-time readouts, accepts reset requests and coordinates BIOS supervisor service. Its fault-report words retain CPU diagnostics for the monitor. A separate pair of ports provides line-oriented diagnostic text output.

Cartridge programs normally use TIME_MS to measure elapsed time and the two frame-timing words to inspect the programmed video period. VBlank interrupts provide frame synchronization; these readouts provide measurements. Diagnostic output appends codepoints through PRINT_CHAR and submits a complete line through PRINT_FLUSH.

Supervisor service preserves selected cartridge device state while BIOS uses its own IRQ, DMA and GPU context. It is coordinated with CPU exception entry, but device-context restoration and CPU exception return remain separate steps: LEAVE restores the devices, while RFE returns the CPU. Firmware follows both protocols before cartridge execution resumes.

USER, ENTRY_VECTOR and ACTIVE name stages in this description, not additional register fields. STATUS exposes only the flags listed below.

Use aligned 32-bit accesses to the registers below. `CONTROL` is a command port, not a retained command mask. Each accepted write is processed and its readable word returns to zero, including writes containing ignored bits. `STATUS` reports the current supervisor-context flags.

## 12.1 SYSTEM specifications

| Resource | Specification |
|---|---|
| Register interface | Thirteen 32-bit word ports at the noncontiguous addresses listed in this chapter; four-byte words |
| Addressed register bytes | 52 bytes in total, not one contiguous reservation or payload RAM |
| Supervisor fault report | Six raw words, 24 bytes including its publication sequence |
| Diagnostic output queue | 8192 bytes, shared by completed unread text and the current partial line |

## 12.2 Measure elapsed console time

This polling example needs no interrupt source: TIME_MS advances with system cycles, not with timer interrupts. It leaves the inherited display and interrupt configuration unchanged. The interval must be shorter than one complete 2^32-millisecond wrap.

```
local time_ms<const>: *word = 0x08010228
local start<const> = *time_ms
while ((*time_ms - start) % 0x100000000) < 100 do
end
local finish<const> = *time_ms
local elapsed<const> = (finish - start) % 0x100000000
```

`elapsed` is at least 100 milliseconds, not necessarily exactly 100. For `start = fffffff0H` and `finish = 00000054H`, the wrapping distance is 100 milliseconds. BIOS `os.clock()` instead expresses a fresh TIME_MS read in seconds; `os.time()` without arguments returns whole seconds. Neither supplies a civil clock. A HALT-based wait additionally needs an owned interrupt route and must recheck its time predicate after each wake-up; advancing TIME_MS alone cannot wake HALT.

## 12.3 Control register reference

| Register | Address | Access | Reset value |
|---|---|---|---|
| CONTROL | 0x0801034c | Command write; reads return 0 | 0x00000000 |
| STATUS | 0x08010350 | Read-only | 0x00000000 |

### 12.3.1 CONTROL

Each set command bit is considered independently. One write is processed in the order **RESET, ENTER, FAULT, FAULT_PUBLISH, LEAVE**; this is not a choice of one winning command. Later operations see state changes made by earlier operations in the same write. RESET is considered in either CPU mode. The other four operations are considered only when the CPU's **current** user-mode bit is clear; in user mode those bits are ignored, not converted into a supervisor request. Bits 5–31 have no effect.

| Bits | Mask | Operation | Effect |
|---|---|---|---|
| 0 | 0x00000001 | RESET | Latch a system-reset request and request a CPU yield. The write itself does not perform the reboot. |
| 1 | 0x00000002 | SUPERVISOR_ENTER (ENTER) | Activate the supervisor device context **only at ENTRY_VECTOR**. It has no effect in any other stage and does not initiate entry from USER. |
| 3 | 0x00000008 | SUPERVISOR_FAULT (FAULT) | Capture the CPU fault words. In ACTIVE, make no further transition. Otherwise select the fault transition, clear resumability, cancel any pending NMI request and request a yield; only USER starts a new entry-quiescence sequence. |
| 4 | 0x00000010 | SUPERVISOR_FAULT_PUBLISH (FAULT_PUBLISH) | Increment the retained fault sequence modulo 2^32. Do not capture or clear the payload, change supervisor flags or enter an exception. |
| 2 | 0x00000004 | SUPERVISOR_LEAVE (LEAVE) | Begin return to the user device context only in ACTIVE with RESUMABLE set. Clear EXIT_REQUESTED and request a yield. Otherwise do nothing. |
| 31:5 | 0xffffffe0 | Unused | Ignored. |

For example, FAULT | FAULT_PUBLISH captures the payload before advancing its publication sequence. ENTER | FAULT first attempts activation; if that succeeds, FAULT takes its ACTIVE behavior. Reading zero from CONTROL says neither that a requested reboot has completed nor that a supervisor transition has finished.

### 12.3.2 STATUS

| Bits | Mask | Flag | Meaning when set |
|---|---|---|---|
| 0 | 0x00000001 | SUPERVISOR_ACTIVE | SYSTEM is not in USER. This includes entry quiescence, ENTRY_VECTOR, ACTIVE and exit quiescence. |
| 1 | 0x00000002 | EXIT_REQUESTED | A supervisor-request edge has requested exit from an ACTIVE, resumable context. The flag does not itself execute LEAVE. |
| 2 | 0x00000004 | RESUMABLE | The supervisor-context resumability latch is set. LEAVE also requires the ACTIVE stage. |

| Bits | Mask | Flag | Meaning when set |
|---|---|---|---|
| 31:3 | 0xfffffff8 | Unused | Always zero. |

These flags are independent. Entry can set SUPERVISOR_ACTIVE before RESUMABLE; an admitted LEAVE keeps both set while restoration is in progress, then clears them when USER is restored. SUPERVISOR_ACTIVE does not identify CPU privilege, and RESUMABLE does not certify that an exception has been repaired or that CPU exception return has already occurred.

## 12.4 Privilege and supervisor entry

SYSTEM supervisor commands require current CPU supervisor mode. Normal cartridge entry runs in user mode; exception entry clears the current user-mode bit before its handler executes. Thus a cartridge IRQ handler can perform privileged operations even though the interrupted cartridge code could not. CP0 access in user mode raises coprocessor-unusable. Ordinary function calls retain the caller's CPU mode.

Normal monitor entry begins with a rising edge on the supervisor-request wire. INPUT samples that wire at every VBlank, independently of its ARM capture request. In USER, an edge starts the entry sequence. In ACTIVE with RESUMABLE set, an edge sets EXIT_REQUESTED so firmware can leave the monitor. Other stages ignore the edge. The wire must return low before a later rising edge can be recognized.

SUPERVISOR_ACTIVE becomes visible during entry quiescence. Once the device boundary is reached, SYSTEM enters ENTRY_VECTOR and requests NMI. The CPU accepts NMI through its normal interrupt boundary and enters the system exception function. Firmware then writes ENTER to activate the supervisor device context. ENTER is accepted only at ENTRY_VECTOR; initiating entry uses the request wire. The public cartridge BIOS imports provide no callable request for this wire.

A synchronous fault takes a different route. The CPU enters the system exception function, and the firmware writes FAULT to capture its diagnostics. If this starts from USER, SYSTEM quiesces the devices and then activates the supervisor context directly, without issuing another NMI. A FAULT received during another non-ACTIVE stage redirects the target without restarting that stage; in ACTIVE it only captures the report. Activation sets RESUMABLE even on the fault route. The detailed device banks and quiescence rules are separate from these CPU-entry and command edges.

## 12.5 Device boundaries during supervisor service

Entry waits for accepted producer work, then the admitted DMA block, then the GPU boundary. This ordering lets data already in flight reach its proper stopping point before the register contexts are exchanged. Transition time depends on that work. Main RAM, VRAM and APU sample RAM remain shared storage.

| Entry boundary | Condition before proceeding | Cartridge consequence |
|---|---|---|
| Producer boundary | GEO is no longer BUSY; IMGDEC has no scheduled decode batch | GEO may complete normally or report its normal error. IMGDEC need not finish the stream or empty its FIFOs. |
| DMA boundary | No admitted block and no DMA service in progress | An already admitted block completes; a longer BUSY transfer can retain a nonzero remaining count. No next block is admitted during this stage. |
| GPU boundary | Ingress has stopped, no command completion is pending, readback is idle or its unread result is complete, and no commands remain queued for rendering | The completed unread result and words waiting behind it can be retained. An unfinished readback must complete before this boundary; the fence does not consume pixels. |

At entry's first stage, new GEO CMD, DMA TRIGGER and GPU GP1 writes are held. IMGDEC lowers both DMA requests and stops after its scheduled decode batch. Shared RAM and other device words remain accessible under their normal rules. The ordinary request path allows CPU execution during producer quiescence; software observing SUPERVISOR_ACTIVE can therefore still be running before NMI.

After these boundaries, the ordinary request path can take the NMI and complete ENTER. IRQ state is banked before the supervisor GPU context is initialized. The following preservation rules apply to that activated context:

| Unit | Retained or held state | Limit |
|---|---|---|
| IRQ | Cartridge FLAGS and MASK are retained; supervisor FLAGS and MASK start at zero. Leave restores the retained cartridge words. | Supervisor pending bits are not merged into the restored cartridge word. A request explicitly directed to the retained cartridge bank can accumulate there. |
| DMA | Channel register words and the next-channel arbitration position are retained. Supervisor channels start clear. Leave restores the cartridge channels and resumes arbitration. | Payload RAM is not copied. Quiescing one admitted block is not completion of the entire transfer. |
| GPU | Cartridge drawing/display registers, both ingress FIFOs, partial packet/image-load/polyline assembly, and unread readback bytes, descriptor, cursor and GPUREAD latch are retained and restored. Supervisor starts with empty ingress and readback state. | Parked cartridge readback does not drive the supervisor DMA request. VRAM remains shared; restoring the cartridge bank does not undo supervisor writes to framebuffer or texture bytes. |
| PCRTC | Programmed and presented composition words are retained separately. | Beam timing words and beam phase are not part of this composition bank. |
| GEO | The current job reaches a terminal phase; new CMD admission remains held until leave. | There is no separate supervisor GEO job/register bank. Its ordinary result and fault words remain meaningful. |
| IMGDEC | Stream state remains in place at a scheduled-batch boundary and can resume on leave. | Quiescence is not stream DONE, and there is no separate supervisor decoder bank. |
| APU | The voice clock is held after synchronization to the entry cycle; queued output samples are discarded. | Voice state is not reset or separately banked. Already scheduled sample-transfer work can still complete. |
| INPUT | One shared register file, pending ARM request and sample sequence remain in use, including the BIOS monitor's captures. | There is no cartridge INPUT bank to restore on leave. Obtain a new sample on resumption; see BIOS handoff and supervisor sampling. |

While the APU voice clock is held, voice playback position stays at the synchronized entry point. Leave reanchors voice timing and resumes playback from that position. The separate sample-transfer engine can finish accepted FIFO/RAM batches during the hold.

Supervisor PCRTC composition initially routes the cartridge's **presented** circuit-1 source and display words into circuit 2, copies its presented background color, and maps its circuit-1 enable into circuit-2 enable. This reuses source addresses; it does not copy framebuffer pixels. The firmware can then prepare its own display. Respect the firmware surface and system-texture reservation explained under **Memory and program storage**; a retained cartridge GPU context does not grant ownership of those VRAM bytes.

LEAVE crosses the DMA and GPU boundaries for the supervisor's work before restoring the cartridge contexts. Supervisor ingress and its own completed unread readback are transient scratch discarded by this explicit context teardown; cartridge readback resumes at its retained cursor, with its DMA request derived from the restored direction and readiness. This is not an implicit read or cancellation of the cartridge transfer. LEAVE releases GEO/IMGDEC admission and the APU voice hold, and restores the cartridge IRQ bank. That device restoration is separate from the CPU's exception return. Neither LEAVE nor a restored register bank repairs the faulting cartridge instruction or grants safe reuse of shared memory by another owner.

## 12.6 Supervisor fault registers

The supervisor fault bank contains six raw 32-bit words. All are read-only to cartridge MMIO writes and reset to zero. The register names in the next table have the prefix `SUPERVISOR_FAULT_`; all 32 bits of each word are retained.

| Register | Address | Access | Reset value | Recorded value |
|---|---|---|---|---|
| SEQUENCE | 0x08010428 | Read-only | 0x00000000 | Publication sequence, incremented by FAULT_PUBLISH modulo 2^32. |
| CAUSE | 0x0801042c | Read-only | 0x00000000 | CPU Cause word at FAULT capture. |
| EPC | 0x08010430 | Read-only | 0x00000000 | CPU EPC word at FAULT capture. |
| BAD_ADDRESS | 0x08010434 | Read-only | 0x00000000 | Retained CPU BadAddress word at FAULT capture. |
| LUA_REASON | 0x08010438 | Read-only | 0x00000000 | Retained CPU LuaReason word at FAULT capture; not the Lua error value. |
| DOMAIN | 0x0801043c | Read-only | 0x00000000 | Interrupted execution-image domain: cartridge 0 or 1, or system 0xffffffff. This is not CPU privilege or SYSTEM stage. |

FAULT copies the five CPU payload words. FAULT_PUBLISH advances SEQUENCE modulo 2^32 and leaves that payload unchanged. Software can capture first and publish later; a new sequence identifies publication rather than the instant of CPU exception entry. Zero is a valid sequence after reset and after wrap.

The firmware's synchronous-fault path publishes after preparing the monitor display and waiting another VBlank. Publication is therefore later than CPU fault admission. An NMI entry alone does not refresh this bank: the words change through FAULT capture, not by following the CPU diagnostic registers continuously.

Interpret the payload using Cause. Synchronous entry records the faulting instruction's EPC; asynchronous entry records the interrupted PC. A subsequent firmware EPC write is included in a later FAULT capture. Address exceptions update BadAddress, while data-bus errors leave its previous value. Lua traps update LuaReason; unrelated exceptions retain that word. These retained fields must be read with the recorded cause.

The raw Cause word uses the following values. The exception-code mask is 0x0000007c; IRQ and NMI indications are listed separately.

| Value | Meaning |
|---|---|
| 0x00000400 | Maskable IRQ indication |
| 0x00010000 | NMI indication |
| 0x00000010 | Address-error load |
| 0x00000014 | Address-error store |
| 0x00000018 | Instruction-bus error |
| 0x0000001c | Data-bus error |
| 0x00000028 | Reserved instruction |
| 0x0000002c | Coprocessor unusable |
| 0x00000034 | Trap |

| LuaReason | Meaning |
|---|---|
| 0 | UNKNOWN |
| 1 | CALL_NON_FUNCTION |
| 2 | INDEX_NON_TABLE |
| 3 | ASSIGN_NON_TABLE |
| 4 | INDEX_NIL |
| 5 | METATABLE_LOOP |
| 6 | ITERATE_NON_TABLE |

| LuaReason | Meaning |
|---|---|
| 7 | XPCALL_HANDLER_NOT_FUNCTION |
| 8 | EXPLICIT_ERROR |
| 9 | OUT_OF_MEMORY |
| 10 | INVALID_ARGUMENT |

Interpret LuaReason with Cause rather than treating every retained reason as a new Lua failure.

## 12.7 Faults, interrupts and continuation

The SYSTEM NMI is distinct from a maskable IRQ. A pending NMI takes precedence and is not gated by CPU interrupt-enable or the IRQ mask; it enters the system exception function. Maskable interrupt acceptance and source acknowledgement follow the **IRQ** chapter. Neither the request edge nor these priority rules establishes a fixed request-to-vector latency.

The shared access and fault contract is described in **Memory and program storage**. An ordinary typed load or store checks its scalar alignment first: a misaligned word or halfword takes the CPU address-error path before MMIO is accessed. For the aligned SYSTEM ports here, a word write to STATUS or any supervisor-fault word raises `READ_ONLY` (`3`) without changing the word. Halfword reads and byte or halfword writes that reach these ports raise `UNALIGNED_IO` (`2`). A byte read at a word-aligned port address instead performs the normal read and returns its low eight bits; a byte read at another byte of that port raises `UNALIGNED_IO`. A newly raised mapped bus fault takes the CPU data-bus-error path, not a successful load of a zero result. Consult the memory fault record rather than assuming supervisor BadAddress holds that MMIO address.

Lua failures that are handled by protected-call or coroutine error handling do not necessarily reach the system exception path. An unhandled Lua failure enters a trap exception and supplies its Lua reason. This does not imply that protected Lua calls catch hardware address, MMIO or privilege exceptions.

Continuing the monitor is a retry, not a repair. For an invocation that owns the supervisor context, the firmware restores its saved EPC and CPU status and issues LEAVE before returning. CPU exception return is separate from device-context restoration and does not automatically advance a synchronous-fault EPC. The `CONT` command can therefore retry the same failing instruction and fault again; `RESUMABLE` does not promise successful program recovery.

## 12.8 Reset requests

RESET is available in either CPU mode. It latches one pending request and asks the CPU to yield; repeated RESET writes coalesce until the request is consumed. CONTROL still reads zero, and STATUS has no reset-pending flag. Reading either word neither consumes the request nor proves that reboot has occurred. Other admitted command bits in the same write are still processed in the defined order.

A pending request is serviced at a CPU execution boundary. Servicing it performs a system reboot: it clears I/O and bus faults, resets hardware and CPU execution state, and starts firmware again. There is no fixed request-to-reboot cycle latency. This is not a local device reset or a bus-fault acknowledgement, and it is not a promise that all RAM or every device buffer is zeroed.

At SYSTEM reset, CONTROL, STATUS and all six fault words are zero; no supervisor transition, exit request or resumable context remains. These are the reset values of this unit, not a substitute for the reset and acknowledgement rules of other devices. Automatic zero readback after a CONTROL write is command-port behavior, not an acknowledgement of a bus fault or completion of a machine reboot.

## 12.9 System time and frame timing

The installed system-cycle frequency is 33,868,800 Hz. `TIME_MS` reports elapsed system-cycle time in whole milliseconds: fractional milliseconds are discarded, and the result wraps modulo 2^32. Consecutive reads can therefore return the same word. This is console time, not a civil date or a wall clock.

| Address | Register | Access | Word interpretation | Reset/state value |
|---|---|---|---|---|
| 0x08010228 | TIME_MS | R | Bits 31–0: unsigned whole milliseconds, modulo 2^32 | 0x00000000 after hardware time reset |
| 0x0801022c | FRAME_MS_Q16 | R | Bits 31–0: unsigned Q16 milliseconds; bits 15–0 are fractional | 0x00141893 at installed hardware reset; zero while stopped |
| 0x08010238 | CYCLES_PER_FRAME | R | Bits 31–0: unsigned system-cycle budget for a VBlank period | 0x000a62b4 at installed hardware reset; zero while stopped |

These reset-state readouts apply before firmware advances execution. They are not fixed future budgets or values promised at cartridge entry. The two frame words are derived from the programmed beam and its fractional phase, not separate constant registers.

Use aligned word reads for these registers. Writes are read-only-access faults. The reads are independent samples, not an atomic snapshot of all three quantities.

FRAME_MS_Q16 describes the currently programmed beam period. Divide its unsigned word by 0x00010000 to express the reported value in milliseconds. The refresh value used to form this report is first truncated to whole millionths of a hertz; the Q16 period is then truncated and reduced to its low 32 bits. The full period is maintained separately, rather than recovered from this register. A sufficiently long period can therefore wrap the reported word without shortening the complete period.

CYCLES_PER_FRAME is a whole-period budget, not the number of cycles remaining until the next VBlank. Fractional beam timing can make successive budgets differ by one cycle. Reprogramming a beam-timing word with different bits immediately changes operational timing and restarts the beam; writing identical bits does not. These SYSTEM words describe programmed timing, not confirmation that a new image has been presented. When beam timing is stopped, both frame-timing words are zero; TIME_MS is not stopped with them.

### 12.9.1 Software timing

For two raw TIME_MS samples, compute elapsed milliseconds modulo 2^32. In BLua, use `(current - start) % 0x100000000` for the nonnegative wrapping distance. A bitwise `& 0xffffffff` instead returns a signed 32-bit result and cannot represent distances from 80000000H through ffffffffH as nonnegative Numbers. The modulo result measures the interval only when the actual elapsed time is shorter than one complete 2^32-millisecond period. A zero result is valid when both samples fall in the same millisecond. os.difftime is ordinary subtraction, not wrapping hardware-time arithmetic.

The BIOS os.clock() returns TIME_MS / 1000, in seconds with millisecond resolution. The no-argument os.time() returns TIME_MS // 1000, in whole seconds. Both inherit the hardware word's wrap and neither measures time spent exclusively executing cartridge instructions. os.time(date_table) instead calculates a civil timestamp from the supplied fields; it does not set the console clock. These functions do not establish a wall-clock or real-time-clock service.

## 12.10 Diagnostic text output

| Address | Register | Access | Word interpretation | Reset |
|---|---|---|---|---|
| 0x08010230 | PRINT_CHAR | R/W | Bits 31–0: Unicode codepoint to append; reads return the full last written word | 0x00000000 |
| 0x08010234 | PRINT_FLUSH | R/W | All written values request line publication; reads return the full last written word | 0x00000000 |

Use aligned word writes. `PRINT_CHAR` accepts a Unicode codepoint, not a memory address, string reference, packed group of characters, or UTF-8 byte. Values `0x00000000`–`0x0000007f` encode as one UTF-8 byte, `0x00000080`–`0x000007ff` as two, `0x00000800`–`0x0000ffff` as three except for surrogates, and `0x00010000`–`0x0010ffff` as four. Surrogate words `0x0000d800`–`0x0000dfff` and words above `0x0010ffff` append the single byte `0x3f` (`?`). Readback still returns the original word, including an invalid codepoint word.

PRINT_CHAR extends the current partial line, including when its codepoint is line feed. Every PRINT_FLUSH write appends an ASCII line-feed byte and submits the partial line when space permits; zero and nonzero write values have the same action. Submitted text is available on the diagnostic channel. Screen text uses the separate BIOS console drawing path described below.

Completed unread lines and the current partial line share one 8192-byte queue. Each character needs space for its complete one- to four-byte UTF-8 encoding. If a character cannot fit, the current partial line is discarded, completed unread lines are preserved, and further character writes are ignored until the next flush. That flush publishes nothing and clears the overflow condition. If the flush itself first encounters insufficient space for its added line-feed byte, it likewise discards the partial line, publishes nothing, and leaves overflow cleared. In particular, 8192 unflushed ASCII characters leave no room for the flush's line feed, so that entire partial line is lost. No readiness, free-space, or accepted-line counter is available through these ports. Reset empties the queue and clears both register words.

### 12.10.1 BIOS console integration

BIOS `print(...)` converts each argument with `tostring`, separates arguments with tabs, writes to the BIOS console, and ends the line. Console writes send the full codepoints to `PRINT_CHAR` and also append to a separate 8192-byte BIOS history; codepoints above `0xff` become `?` in that history. Ending the line appends a history line feed and writes `PRINT_FLUSH = 1`. Thus `print` sends text to both diagnostic output and BIOS history, but does not itself flush the console display.

Direct writes to `PRINT_CHAR` and `PRINT_FLUSH` do not add BIOS-history entries or draw glyphs. The BIOS console display is updated separately from its retained history. Diagnostic publication, retained console text, and visible console drawing are distinct events.

## 12.11 Send a diagnostic line

Assume no other diagnostic writer and enough queue space for the line. These writes send four codepoints, then publish the line. A zero FLUSH word is still a flush command.

```
local print_char<const>: *word = 0x08010230
local print_flush<const>: *word = 0x08010234

print('SYSTEM timer experiment')
*print_char = 0x00000053
*print_char = 0x000000e9
*print_char = 0x0001f600
*print_char = 0xffffffff
local char_readback<const> = *print_char
*print_flush = 0
local flush_readback<const> = *print_flush
print('SYSTEM samples complete')
```

CHAR readback is `ffffffffH`, even though the invalid codepoint emits `?`. FLUSH readback is zero. Both are last-written words, not receiver-ready or completion flags. The raw middle line is encoded as:

```
53 C3 A9 F0 9F 98 80 3F 0A
```

The diagnostic text is:

```
SYSTEM timer experiment
Sé😀?
SYSTEM samples complete
```

Only the two BIOS `print` calls also add text to the BIOS console history. The direct codepoint line neither enters that history nor draws glyphs.




Figure 12.1: BIOS console after the timer experiment

The monitor displays the two BIOS print lines from retained console history. The raw diagnostic line does not appear in that history.